

Oracle® Database

Advanced Security Guide



19c
E96301-28
July 2026

ORACLE®

Contributors: Sudha Duraiswamy, Michael Hwa, Sudha Iyer, Supriya Kalyanasundaram, Lakshmi Kethana, Peter Knaggs, Andrew Koyfman, Dah-Yoh Lim, Adam Lee, Adam Lindsey, Rahil Mir, Gopal Mulagund, Andy Philips, Preetam Ramakrishna, Saikat Saha, Philip Thornton, Peter Wahl, Lixia Yuan, Paul Youn

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Oracle customer access to and use of Oracle support services will be pursuant to the terms and conditions specified in their Oracle order for the applicable services.

Contents

Preface

Audience	i
Documentation Accessibility	i
Related Documents	i
Conventions	ii

Changes in This Release for Oracle Database Advanced Security Guide

Changes in Oracle Database Advanced Security 19c	i
Updates to Oracle Advanced Database Security 19c	iii

1 Introduction to Oracle Advanced Security

1.1 Transparent Data Encryption	1
1.2 Oracle Data Redaction	1

Part I Using Transparent Data Encryption

2 Quick-start Setup Guide for Wallet Based Transparent Data Encryption

3 Introduction to Transparent Data Encryption

3.1 What Is Transparent Data Encryption?	1
3.2 Benefits of Using Transparent Data Encryption	2
3.3 Who Can Configure Transparent Data Encryption?	2
3.4 Types and Components of Transparent Data Encryption	3
3.4.1 About Transparent Data Encryption Types and Components	3
3.4.2 How Transparent Data Encryption Tablespace Encryption Works	3
3.4.3 How Transparent Data Encryption Column Encryption Works	5
3.4.4 How the Keystore for the Storage of TDE Master Encryption Keys Works	6
3.4.4.1 About the Keystore Storage of TDE Master Encryption Keys	6
3.4.4.2 Benefits of the Keystore Storage Framework	6

3.4.4.3	Types of Keystores	7
3.4.5	Supported Encryption and Integrity Algorithms	9
3.5	How the Multitenant Option Affects Transparent Data Encryption	11

4 Configuring Transparent Data Encryption

4.1	About Configuring Transparent Data Encryption	1
4.2	Transparent Data Encryption Keystore Search Order	2
4.3	Configuring a TDE Wallet	2
4.3.1	About Configuring a TDE Wallet	3
4.3.2	Cryptographic Provider Prerequisite for TDE Configuration	3
4.3.3	Step 1: Configure the TDE Wallet Location and Type	4
4.3.4	Step 2: Create the TDE Wallet	5
4.3.4.1	About Creating TDE Wallets	5
4.3.4.2	Creating a Password-Protected Software Keystore	6
4.3.4.3	Creating an Auto-Login or a Local Auto-Login Software Keystore	6
4.3.5	Step 3: Open the TDE Wallet	7
4.3.5.1	About Opening TDE Wallets	8
4.3.5.2	Opening a TDE Wallet	8
4.3.6	Step 4: Set the TDE Master Encryption Key in the TDE Wallet	9
4.3.6.1	About Setting the TDE Wallet TDE Master Encryption Key	9
4.3.6.2	Setting the TDE Master Encryption Key in the TDE Wallet	10
4.3.7	Step 5: Encrypt Your Data	10
4.4	Configuring an External Keystore	11
4.4.1	About Configuring an External Keystore	11
4.4.2	Step 1: Configure the External Keystore	11
4.4.3	Step 2: Open the Connection to Oracle Key Vault	12
4.4.3.1	About Opening the Connection to Oracle Key Vault	12
4.4.3.2	Opening an External Keystore	13
4.4.4	Step 3: Set the TDE Master Encryption Key in Oracle Key Vault	13
4.4.4.1	About Setting the External Keystore TDE Master Encryption Key	14
4.4.4.2	Setting a New TDE Master Encryption Key	14
4.4.4.3	Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS	15
4.4.5	Step 4: Encrypt Your Data in United Mode	15
4.5	Encrypting Columns in Tables	16
4.5.1	About Encrypting Columns in Tables	16
4.5.2	Data Types That Can Be Encrypted with TDE Column Encryption	17
4.5.3	Restrictions on Using TDE Column Encryption	18
4.5.4	Creating Tables with Encrypted Columns	18
4.5.4.1	About Creating Tables with Encrypted Columns	19
4.5.4.2	Creating a Table with an Encrypted Column Using the Default Algorithm	19

4.5.4.3	Creating a Table with an Encrypted Column Using No Algorithm or a Non-Default Algorithm	20
4.5.4.4	Using the NOMAC Parameter to Save Disk Space and Improve Performance	21
4.5.4.5	Example: Using the NOMAC Parameter in a CREATE TABLE Statement	21
4.5.4.6	Example: Changing the Integrity Algorithm for a Table	21
4.5.4.7	Creating an Encrypted Column in an External Table	22
4.5.5	Encrypting Columns in Existing Tables	23
4.5.5.1	About Encrypting Columns in Existing Tables	24
4.5.5.2	Adding an Encrypted Column to an Existing Table	24
4.5.5.3	Encrypting an Unencrypted Column	24
4.5.5.4	Disabling Encryption on a Column	24
4.5.6	Creating an Index on an Encrypted Column	25
4.5.7	Adding Salt to an Encrypted Column	25
4.5.8	Removing Salt from an Encrypted Column	25
4.5.9	Changing the Encryption Key or Algorithm for Tables with Encrypted Columns	26
4.6	Encryption Conversions for Tablespaces and Databases	26
4.6.1	About Encryption Conversion for Tablespaces and Databases	27
4.6.2	Cryptographic Provider Considerations for Tablespace Encryption	28
4.6.3	Impact of a Closed TDE Keystore on Encrypted Tablespaces	29
4.6.4	Restrictions on Using Transparent Data Encryption Tablespace Encryption	30
4.6.5	Creating an Encrypted New Tablespace	31
4.6.5.1	Step 1: Set the COMPATIBLE Initialization Parameter for Tablespace Encryption	31
4.6.5.2	Step 2: Set the Tablespace TDE Master Encryption Key	33
4.6.5.3	Step 3: Create the Encrypted Tablespace	33
4.6.6	Setting the Tablespace Encryption Default Algorithm	35
4.6.7	Encrypting Future Tablespaces	35
4.6.7.1	About Encrypting Future Tablespaces	35
4.6.7.2	Setting Future Tablespaces to be Encrypted	36
4.6.8	Encrypted Sensitive Credential Data in the Data Dictionary	37
4.6.9	Encryption Conversions for Existing Offline Tablespaces	37
4.6.9.1	About Encryption Conversions for Existing Offline Tablespaces	37
4.6.9.2	Encrypting an Existing User-Defined Tablespace with Offline Conversion	38
4.6.9.3	Decrypting an Existing Tablespace with Offline Conversion	40
4.6.10	Encryption Conversions for Existing Online Tablespaces	40
4.6.10.1	About Encryption Conversions for Existing Online Tablespaces	41
4.6.10.2	Encrypting an Existing Tablespace with Online Conversion	42
4.6.10.3	Rekeying an Existing Tablespace with Online Conversion	44
4.6.10.4	Decrypting an Existing Tablespace with Online Conversion	45
4.6.10.5	Finishing an Interrupted Online Encryption Conversion	46
4.6.11	Encryption Conversions for Existing Databases	47
4.6.11.1	About Encryption Conversions for Existing Databases	47

4.6.11.2	Encrypting an Existing Database with Offline Conversion	48
4.6.11.3	Encrypting an Existing Database with Online Conversion	49
4.7	Transparent Data Encryption Data Dynamic and Data Dictionary Views	50

5 Switching from MES to OpenSSL Cryptographic Provider

6 Managing the Keystore and the Master Encryption Key

6.1	Managing the Keystore	1
6.1.1	Performing Operations That Require a Keystore Password	2
6.1.2	Changing the Password of a Software Keystore	3
6.1.2.1	About Changing the Password of a Password-Protected Software Keystore	3
6.1.2.2	Changing the Password-Protected Software Keystore Password	3
6.1.3	Changing the Oracle Key Vault Password	4
6.1.4	Configuring an External Store for a Keystore Password	4
6.1.4.1	About Configuring an External Store for a Keystore Password	5
6.1.4.2	Configuring the External Keystore Password Store with WALLET_ROOT	5
6.1.4.3	When to Use the EXTERNAL STORE Clause After Configuration	5
6.1.5	Backing Up Password-Protected Software Keystores	6
6.1.5.1	About Backing Up Password-Protected TDE Wallets	6
6.1.5.2	Creating a Backup Identifier String for the Backup TDE Wallet	6
6.1.5.3	Backing Up a Password-Protected TDE Wallet	7
6.1.6	How the V\$ENCRYPTION_WALLET View Interprets Backup Operations	8
6.1.7	Backups of the External Keystore	8
6.1.8	Merging TDE Wallets	9
6.1.8.1	About Merging TDE Wallets	9
6.1.8.2	Merging One TDE Wallet into an Existing TDE Wallet	10
6.1.8.3	Merging Two TDE Wallets into a Third New TDE Wallet	10
6.1.8.4	Merging an Auto-Login TDE Wallet into an Existing Password-Protected TDE Wallet	11
6.1.8.5	Reversing a TDE Wallet Merge Operation	11
6.1.9	Moving a TDE Wallet to a New Location	12
6.1.10	Moving a TDE Wallet Out of Automatic Storage Management	13
6.1.11	Migrating from a TDE Wallet to Oracle Key Vault	13
6.1.11.1	Migrating from a Password-Protected TDE Wallet to an External Keystore	14
6.1.11.2	Migrating from an External Keystore to a Password-Based TDE Wallet	16
6.1.11.3	Keystore Order After a Migration	18
6.1.12	Migration of Keystores to and from Oracle Key Vault	19
6.1.13	Configuring Keystores for Automatic Storage Management	19
6.1.13.1	About Configuring Keystores for Automatic Storage Management	20

6.1.13.2	Configuring a Keystore on a Standalone Database to Point to an ASM Location	20
6.1.13.3	Configuring a Keystore to Point to an ASM Location	21
6.1.13.4	Configuring a Keystore to Point to an ASM Location When the WALLET_ROOT Location Does Not Follow OMF Guidelines	22
6.1.14	Closing a Keystore	22
6.1.14.1	About Closing Keystores	23
6.1.14.2	Closing a TDE Wallet	23
6.1.14.3	Closing an External Keystore	24
6.1.15	Backup and Recovery of Encrypted Data	24
6.1.16	Dangers of Deleting Keystores	25
6.1.17	Features That Are Affected by Deleted Keystores	26
6.2	Managing the TDE Master Encryption Key	27
6.2.1	Creating User-Defined TDE Master Encryption Keys	28
6.2.1.1	About User-Defined TDE Master Encryption Keys	28
6.2.1.2	Creating a User-Defined TDE Master Encryption Key	28
6.2.2	Creating TDE Master Encryption Keys for Later Use	31
6.2.2.1	About Creating a TDE Master Encryption Key for Later Use	31
6.2.2.2	Creating a TDE Master Encryption Key for Later Use	31
6.2.2.3	Example: Creating a TDE Master Encryption Key in a Single Database	32
6.2.3	Activating TDE Master Encryption Keys	33
6.2.3.1	About Activating TDE Master Encryption Keys	33
6.2.3.2	Activating a TDE Master Encryption Key	33
6.2.3.3	Example: Activating a TDE Master Encryption Key	34
6.2.4	TDE Master Encryption Key Attribute Management	35
6.2.4.1	TDE Master Encryption Key Attributes	35
6.2.4.2	Finding the TDE Master Encryption Key That Is in Use	36
6.2.5	Creating Custom TDE Master Encryption Key Attributes for Reports	36
6.2.5.1	About Creating Custom Attribute Tags	36
6.2.5.2	Creating a Custom Attribute Tag	37
6.2.6	Setting or Rekeying the TDE Master Encryption Key in the Keystore	37
6.2.6.1	About Setting or Rekeying the TDE Master Encryption Key in the Keystore	38
6.2.6.2	Creating, Tagging, and Backing Up a TDE Master Encryption Key	38
6.2.6.3	About Rekeying the TDE Master Encryption Key	39
6.2.6.4	Rekeying the TDE Master Encryption Key	40
6.2.6.5	Changing the TDE Master Encryption Key for a Tablespace	41
6.2.7	Exporting and Importing the TDE Master Encryption Key	42
6.2.7.1	About Exporting and Importing the TDE Master Encryption Key	42
6.2.7.2	About Exporting TDE Master Encryption Keys	43
6.2.7.3	Exporting a TDE Master Encryption Key	43
6.2.7.4	Example: Exporting a TDE Master Encryption Key by Using a Subquery	44
6.2.7.5	Example: Exporting a List of TDE Master Encryption Key Identifiers to a File	44

6.2.7.6	Example: Exporting All TDE Master Encryption Keys of the Database	45
6.2.7.7	About Importing TDE Master Encryption Keys	45
6.2.7.8	Importing a TDE Master Encryption Key	45
6.2.7.9	Example: Importing a TDE Master Encryption Key	46
6.2.7.10	How Keystore Merge Differs from TDE Master Encryption Key Export or Import	46
6.2.8	Moving TDE Master Encryption Keys into a New Keystore	47
6.2.8.1	About Moving TDE Master Encryption Keys into a New Keystore	47
6.2.8.2	Moving a TDE Master Encryption Key into a New Keystore	48
6.2.9	Converting from ENCRYPTION_WALLET_LOCATION to WALLET_ROOT and TDE_CONFIGURATION	49
6.2.10	Management of TDE Master Encryption Keys Using Oracle Key Vault	50
6.3	Storing Oracle Database Secrets	50
6.3.1	About Storing Oracle Database Secrets in a Keystore	51
6.3.2	Storage of Oracle Database Secrets in a Software Keystore	51
6.3.3	Example: Adding an Oracle Key Vault Password to a Software Keystore	53
6.3.4	Example: Changing an Oracle Key Vault Password Stored as a Secret in a Software Keystore	53
6.3.5	Example: Deleting an Oracle Key Vault Password Stored as a Secret in a Software Keystore	54
6.3.6	Storage of Oracle Database Secrets in an External Keystore	54
6.3.7	Example: Adding an Oracle Database Secret to an External Keystore	55
6.3.8	Example: Changing an Oracle Database Secret in an External Keystore	56
6.3.9	Example: Deleting an Oracle Database Secret in an External Keystore	56
6.3.10	Configuring Auto-Open Connections into External Key Managers	56
6.3.10.1	About Auto-Open Connections into External Key Managers	56
6.3.10.2	Configuring an Auto-Open Connection into an External Key Manager	57
6.4	Storing Oracle GoldenGate Secrets in a Keystore	58
6.4.1	About Storing Oracle GoldenGate Secrets in Keystores	58
6.4.2	Oracle GoldenGate Extract Classic Capture Mode TDE Requirements	59
6.4.3	Configuring Keystore Support for Oracle GoldenGate	59
6.4.3.1	Step 1: Decide on a Shared Secret for the Keystore	60
6.4.3.2	Step 2: Configure Oracle Database for TDE Support for Oracle GoldenGate	60
6.4.3.3	Step 3: Store the TDE GoldenGate Shared Secret in the Keystore	60
6.4.3.4	Step 4: Set the TDE Oracle GoldenGate Shared Secret in the Extract Process	62

7 Managing Keystores and TDE Master Encryption Keys in United Mode

7.1	About Managing Keystores and TDE Master Encryption Keys in United Mode	1
7.2	Operations That Are Allowed in United Mode	2
7.3	Operations That Are Not Allowed in a United Mode PDB	6
7.4	Configuring United Mode with the Initialization Parameter File and ALTER SYSTEM	7

7.5	Configuring a Software Keystore for Use in United Mode	9
7.5.1	About Configuring a Software Keystore in United Mode	9
7.5.2	Creating a Password-Protected TDE Wallet	10
7.5.3	Step 2: Open the TDE Wallet in a United Mode PDB	10
7.5.4	Step 3: Set the TDE Master Encryption Key in the TDE Wallet in United Mode	12
7.6	Configuring a Container Database with United Mode PDBs for Oracle Key Vault	13
7.6.1	About Configuring a Container Database with United Mode PDBs for Oracle Key Vault	13
7.6.2	Step 1: Configure Oracle Key Vault for United Mode	14
7.6.3	Step 2: Open the Connection to Oracle Key Vault	14
7.6.3.1	About Opening the Connection to Oracle Key Vault	14
7.6.3.2	Opening the Oracle Key Vault Connection in a United Mode PDB	15
7.6.4	Step 3: Set the TDE Master Encryption Key in Oracle Key Vault	16
7.6.4.1	About Setting the External Keystore TDE Master Encryption Key	16
7.6.4.2	Heartbeat Batch Size for External Keystores	16
7.6.4.3	Setting the TDE Master Encryption Key in the United Mode External Keystore	18
7.6.4.4	Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS	19
7.6.5	Step 4: Encrypt Your Data in United Mode	20
7.7	Administering Keystores and TDE Master Encryption Keys in United Mode	20
7.7.1	Changing the Keystore Password in United Mode	21
7.7.1.1	Changing the Password-Protected TDE Wallet Password in United Mode	21
7.7.1.2	Changing the Password of an External Keystore in United Mode	22
7.7.2	Backing Up a Password-Protected TDE Wallet in United Mode	23
7.7.3	Closing Keystores in United Mode	24
7.7.3.1	Closing a Software Keystore in United Mode	24
7.7.3.2	Closing an External Keystore in United Mode	25
7.7.4	Creating a User-Defined TDE Master Encryption Key in United Mode	26
7.7.5	Example: Creating a Master Encryption Key in All PDBs	27
7.7.6	Creating a TDE Master Encryption Key for Later Use in United Mode	28
7.7.7	Activating a TDE Master Encryption Key in United Mode	29
7.7.8	Rekeying the TDE Master Encryption Key in United Mode	30
7.7.9	Finding the TDE Master Encryption Key That Is in Use in United Mode	31
7.7.10	Creating a Custom Attribute Tag in United Mode	31
7.7.11	Moving a TDE Master Encryption Key into a New Keystore in United Mode	32
7.7.12	Automatically Removing Inactive TDE Master Encryption Keys in United Mode	33
7.7.13	Isolating a Pluggable Database Keystore	33
7.8	Administering Transparent Data Encryption in United Mode	34
7.8.1	Moving PDBs from One CDB to Another in United Mode	35
7.8.2	Unplugging and Plugging a PDB with Encrypted Data in a CDB in United Mode	35
7.8.2.1	Unplugging a PDB That Has Encrypted Data in United Mode	36
7.8.2.2	Plugging a PDB That Has Encrypted Data into a CDB in United Mode	36

7.8.2.3	Unplugging a PDB That Has Master Encryption Keys Stored in an External Keystore in United Mode	38
7.8.2.4	Plugging a PDB That Has Master Encryption Keys Stored in an External Keystore in United Mode	38
7.8.3	Managing Cloned PDBs with Encrypted Data in United Mode	39
7.8.3.1	About Managing Cloned PDBs That Have Encrypted Data in United Mode	39
7.8.3.2	Cloning a PDB with Encrypted Data in a CDB in United Mode	39
7.8.3.3	Performing a Remote Clone of PDB with Encrypted Data Between Two CDBs in United Mode	40
7.8.3.4	TDE Academy Videos: Remotely Cloning and Upgrading Encrypted PDBs	42
7.8.3.5	Relocating a PDB with Encrypted Data Across CDBs in United Mode	42
7.8.4	How Keystore Open and Close Operations Work in United Mode	43
7.8.5	Finding the Keystore Status for All of the PDBs in United Mode	44
7.8.6	Unplugging a PDB That Has Encrypted Data in United Mode Without ENCRPYT Clause	45
7.8.7	Plugging a PDB That Has Encrypted Data into a CDB in United Mode Without DECRYPT Clause	46

8 Managing Keystores and TDE Master Encryption Keys in Isolated Mode

8.1	About Configuring Isolated Mode	1
8.2	Operations That Are Allowed in Isolated Mode	2
8.3	Operations That Are Not Allowed in an Isolated Mode PDB	6
8.4	Configuring the Keystore Location and Type for Isolated Mode	6
8.4.1	Configuring the Keystore Location and Keystore Type for an Isolated Mode CDB	7
8.4.2	Example: Restoring an Older Version of a Control File	8
8.4.3	Example: Addressing the Problem of a Lost Control File	9
8.4.4	Example: Configuring Isolated Mode in an Oracle Real Application Clusters Environment	10
8.4.5	Resolving the Transition From an Isolated Mode PDB to a United Mode PDB in a Data Guard Environment	10
8.5	Configuring a TDE Wallet and TDE Master Encryption Key in Isolated Mode	10
8.5.1	About Configuring a TDE Wallet in Isolated Mode	11
8.5.2	Step 1: Create a TDE Wallet in a PDB Configured in Isolated Mode	11
8.5.3	Step 2: Open the TDE Wallet in an Isolated Mode PDB	12
8.5.4	Step 3: Set the TDE Master Encryption Key in the TDE Wallet of the Isolated Mode PDB	13
8.5.5	Step 4: Encrypt Your Data in Isolated Mode	14
8.6	Configuring a Container Database with Isolated Mode PDBs for Oracle Key Vault	14
8.6.1	About Configuring an External Keystore in Isolated Mode	14
8.6.2	Step 1: Configure Isolated PDBs for Oracle Key Vault	15
8.6.3	Step 2: Open the External Keystore in an Isolated Mode PDB	15
8.6.4	Step 3: Set the First TDE Master Encryption Key in the External Keystore	15

8.6.4.1	Setting the TDE Master Encryption Key in the Isolated Mode External Keystore	16
8.6.4.2	Migration of a Previously Configured Encryption Key in Isolated Mode	17
8.6.5	Step 4: Encrypt Your Data in Isolated Mode	17
8.7	Administering Keystores and TDE Master Encryption Keys in Isolated Mode	17
8.7.1	Changing the Keystore Password in Isolated Mode	18
8.7.1.1	Changing the Password-Protected TDE Wallet Password in Isolated Mode	19
8.7.1.2	Changing the Password of an External Keystore in Isolated Mode	20
8.7.2	Backing Up a Password-Protected TDE Wallet in Isolated Mode	21
8.7.3	Merging TDE Wallets in Isolated Mode	22
8.7.3.1	Merging One TDE Wallet into an Existing TDE Wallet in Isolated Mode	22
8.7.3.2	Merging Two TDE Wallets into a Third New TDE Wallet in Isolated Mode	23
8.7.4	Closing Keystores in Isolated Mode	23
8.7.4.1	Closing a TDE Wallet in Isolated Mode	24
8.7.4.2	Closing an External Keystore in Isolated Mode	24
8.7.5	Creating a User-Defined TDE Master Encryption Key in Isolated Mode	25
8.7.6	Creating a TDE Master Encryption Key for Later Use in Isolated Mode	26
8.7.7	Activating a TDE Master Encryption Key in Isolated Mode	27
8.7.8	Rekeying the TDE Master Encryption Key in Isolated Mode	28
8.7.9	Moving a TDE Master Encryption Key into a New Keystore in Isolated Mode	29
8.7.10	Creating a Custom Attribute Tag in Isolated Mode	30
8.7.11	Exporting and Importing the TDE Master Encryption Key in Isolated Mode	31
8.7.11.1	Exporting a TDE Master Encryption Key in Isolated Mode	31
8.7.11.2	Importing a TDE Master Encryption Key in Isolated Mode	32
8.7.12	Storing Oracle Database Secrets in Isolated Mode	32
8.7.12.1	Storing Oracle Database Secrets in a TDE Wallet in Isolated Mode	33
8.7.12.2	Storing Oracle Database Secrets in an External Keystore in Isolated Mode	34
8.7.13	Migrating Keystores in Isolated Mode	35
8.7.13.1	Reverse Migrating an Isolated PDB from Oracle Key Vault to a TDE Wallet	35
8.7.13.2	Migrating from an External Keystore to a Password-Protected TDE Wallet in Isolated Mode	36
8.7.14	Uniting a Pluggable Database Keystore	37
8.7.15	Creating a Keystore When the PDB Is Closed	38
8.7.15.1	About Creating a Keystore When the PDB Is Closed	38
8.7.15.2	Reverting a Keystore Creation Operation When a PDB Is Closed	39
8.8	Administering Transparent Data Encryption in Isolated Mode	40
8.8.1	Cloning or Relocating Encrypted PDBs in Isolated Mode	41
8.8.2	Unplugging and Plugging a PDB with Encrypted Data in a CDB in Isolated Mode	41
8.8.2.1	Unplugging a PDB That Has Encrypted Data in Isolated Mode	41
8.8.2.2	Plugging a PDB That Has Encrypted Data into a CDB in Isolated Mode	42

8.8.2.3	Unplugging a PDB That Has Master Encryption Keys Stored in an External Keystore in Isolated Mode	43
8.8.2.4	Plugging a PDB That Has Master Encryption Keys Stored in an External Keystore in Isolated Mode	43
8.8.3	Cloning a PDB with Encrypted Data in a CDB in Isolated Mode	44
8.8.4	Performing a Remote Clone of PDB with Encrypted Data Between Two CDBs in Isolated Mode	45
8.8.5	Relocating an Encrypted PDB in Isolated Mode	46
8.8.6	How Keystore Open and Close Operations Work in Isolated Mode	47
8.8.7	Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode	48
8.8.7.1	About Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode	48
8.8.7.2	Exporting or Importing a Master Encryption Key for a PDB in Isolated Mode	49
8.8.7.3	Example: Exporting a Master Encryption Key from a PDB in Isolated Mode	50
8.8.7.4	Example: Importing a Master Encryption Key into a PDB in Isolated Mode	50

9 General Considerations of Using Transparent Data Encryption

9.1	Migrating Encrypted TDE Columns or Tablespaces after a Database Upgrade from Release 11g	1
9.2	Compression and Data Deduplication of Encrypted Data	3
9.3	Security Considerations for Transparent Data Encryption	4
9.3.1	Transparent Data Encryption General Security Advice	4
9.3.2	Transparent Data Encryption Column Encryption-Specific Advice	4
9.3.3	Managing Security for Plaintext Fragments	5
9.4	Performance and Storage Overhead of Transparent Data Encryption	5
9.4.1	Performance Overhead of Transparent Data Encryption	5
9.4.2	Storage Overhead of Transparent Data Encryption	6
9.5	Modifying Your Applications for Use with Transparent Data Encryption	7
9.6	How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT	7
9.7	Data Loads from External Files to Tables with Encrypted Columns	11
9.8	Transparent Data Encryption and Database Close Operations	11

10 Using Transparent Data Encryption with Other Oracle Features

10.1	How Transparent Data Encryption Works with Export and Import Operations	1
10.1.1	About Exporting and Importing Encrypted Data	2
10.1.2	Exporting and Importing Tables with Encrypted Columns	2
10.1.3	Using Oracle Data Pump to Encrypt Entire Dump Sets	3
10.1.4	Using Oracle Data Pump with Encrypted Data Dictionary Data	4
10.1.5	Data Pump Encryption Algorithm Configuration	5
10.2	How Transparent Data Encryption Works with Oracle Data Guard	5
10.2.1	About Using Transparent Data Encryption with Oracle Data Guard	6

10.2.2	Encryption of Tablespaces in an Oracle Data Guard Environment	6
10.2.2.1	About the Encryption of Tablespace in an Oracle Data Guard Environment	6
10.2.2.2	Configuring the Encryption of Tablespaces in an Oracle Data Guard Environment	8
10.2.3	Configuring TDE and Oracle Key Vault in an Oracle Data Guard Environment	10
10.2.4	Configuring Wallet-Based Transparent Data Encryption in Oracle Data Guard	17
10.2.5	Migrating a TDE Wallet in an Oracle Data Guard Environment to Oracle Key Vault	22
10.2.6	Isolating an Encrypted PDB in an Oracle Data Guard Environment	28
10.3	How Transparent Data Encryption Works with Oracle Real Application Clusters	28
10.3.1	About Using Transparent Data Encryption with Oracle Real Application Clusters	29
10.3.2	Configuring TDE in Oracle Real Application Clusters for Oracle Key Vault	30
10.4	How Transparent Data Encryption Works with SecureFiles	36
10.4.1	About Transparent Data Encryption and SecureFiles	36
10.4.2	Example: Creating a SecureFiles LOB with a Specific Encryption Algorithm	37
10.4.3	Example: Creating a SecureFiles LOB with a Column Password Specified	37
10.5	How Transparent Data Encryption Works with Oracle Call Interface	37
10.6	How Transparent Data Encryption Works with Editions	37
10.6.1	How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT	38
10.7	Configuring Transparent Data Encryption to Work in a Multidatabase Environment	42
10.8	RMAN Encryption Algorithm Configuration	43

11 Frequently Asked Questions About Transparent Data Encryption

11.1	Transparency Questions About Transparent Data Encryption	1
11.2	Performance Questions About Transparent Data Encryption	4

12 Using sqlnet.ora to Configure Transparent Data Encryption Keystores

12.1	About the Keystore Location in the sqlnet.ora File	1
12.2	Configuring the sqlnet.ora File for a Software Keystore Location	2
12.3	Example: Configuring a Software Keystore for a Regular File System	3
12.4	Example: Configuring a TDE Wallet When Multiple Databases Share the Same Host	4
12.5	Example: Configuring a Software Keystore for an Oracle Automatic Storage Management Disk Group	4

Part II Using Oracle Data Redaction

13 Introduction to Oracle Data Redaction

13.1	What Is Oracle Data Redaction?	1
13.2	When to Use Oracle Data Redaction	2

13.3	Benefits of Using Oracle Data Redaction	2
13.4	Target Use Cases for Oracle Data Redaction	2
13.4.1	Oracle Data Redaction for Sensitive Data in Read-Only Static Pages	3
13.4.2	Oracle Data Redaction for Preventing Data Exposure by Management Tools	3
13.4.3	Oracle Data Redaction to Prevent Disclosure of Data from Offline Analytics	3
13.4.4	Oracle Data Redaction Use with Database Applications	3
13.4.5	Oracle Data Redaction with Ad Hoc Database Queries Considerations	4

14 Oracle Data Redaction Features and Capabilities

14.1	Full Data Redaction to Redact All Data	1
14.2	Partial Data Redaction to Redact Sections of Data	2
14.3	Regular Expressions to Redact Patterns of Data	3
14.4	Redaction Using Null Values	4
14.5	Random Data Redaction to Generate Random Values	4
14.6	Comparison of Full, Partial, and Random Redaction Based on Data Types	5
14.6.1	Oracle Built-in Data Types Redaction Capabilities	5
14.6.2	ANSI Data Types Redaction Capabilities	6
14.6.3	Built-in and ANSI Data Types Full Redaction Capabilities	7
14.6.4	User-Defined Data Types or Oracle Supplied Types Redaction Capabilities	8
14.7	No Redaction for Testing Purposes	9
14.8	Central Management of Named Data Redaction Policy Expressions	9

15 Configuring Oracle Data Redaction Policies

15.1	About Oracle Data Redaction Policies	2
15.2	Who Can Create Oracle Data Redaction Policies?	3
15.3	Planning an Oracle Data Redaction Policy	4
15.4	General Syntax of the DBMS_REDACT.ADD_POLICY Procedure	4
15.5	Using Expressions to Define Conditions for Data Redaction Policies	7
15.5.1	About Using Expressions in Data Redaction Policies	7
15.5.2	Supported Functions for Data Redaction Expressions	8
15.5.2.1	Expressions Using Namespace Functions	8
15.5.2.2	Expressions Using the SUBSTR Function	9
15.5.2.3	Expressions Using Length of Character String Functions	10
15.5.2.4	Expressions Using Oracle Application Express Functions	10
15.5.2.5	Expressions Using Oracle Label Security Functions	10
15.5.3	Applying the Redaction Policy Based on User Environment	11
15.5.4	Applying the Redaction Policy Based on Database Roles	12
15.5.5	Applying the Redaction Policy Based on Oracle Label Security Label Dominance	12
15.5.6	Applying the Redaction Policy Based on Application Express Session States	13

15.5.7	Applying the Redaction Policy to All Users	13
15.6	Creating and Managing Multiple Named Policy Expressions	14
15.6.1	About Data Redaction Policy Expressions to Define Conditions	14
15.6.2	Creating and Applying a Named Data Redaction Policy Expression	15
15.6.3	Updating a Named Data Redaction Policy Expression	16
15.6.4	Dropping a Named Data Redaction Expression Policy	17
15.6.5	Tutorial: Creating and Sharing a Named Data Redaction Policy Expression	17
15.6.5.1	Step 1: Create Users for This Tutorial	18
15.6.5.2	Step 2: Create an Oracle Data Redaction Policy	18
15.6.5.3	Step 3: Test the Oracle Data Redaction Policy	19
15.6.5.4	Step 4: Create and Apply a Policy Expression to the Redacted Table Columns	20
15.6.5.5	Step 5: Test the Data Redaction Policy Expression	20
15.6.5.6	Step 6: Modify the Data Redaction Policy Expression	21
15.6.5.7	Step 7: Test the Modified Policy Expression	21
15.6.5.8	Step 8: Remove the Components of This Tutorial	22
15.7	Creating a Full Redaction Policy and Altering the Full Redaction Value	23
15.7.1	Creating a Full Redaction Policy	23
15.7.1.1	About Creating Full Data Redaction Policies	24
15.7.1.2	Syntax for Creating a Full Redaction Policy	24
15.7.1.3	Example: Full Redaction Policy	24
15.7.1.4	Example: Fully Redacted Character Values	25
15.7.2	Altering the Default Full Data Redaction Value	26
15.7.2.1	About Altering the Default Full Data Redaction Value	26
15.7.2.2	Syntax for the DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES Procedure	26
15.7.2.3	Modifying the Default Full Data Redaction Value	27
15.8	Creating a DBMS_REDACT.NULLIFY Redaction Policy	27
15.8.1	About Creating a Policy That Returns Null Values	28
15.8.2	Syntax for Creating a Policy That Returns Null Values	28
15.8.3	Example: Redaction Policy That Returns Null Values	28
15.9	Creating a Partial Redaction Policy	29
15.9.1	About Creating Partial Redaction Policies	29
15.9.2	Syntax for Creating a Partial Redaction Policy	30
15.9.3	Creating Partial Redaction Policies Using Fixed Character Formats	30
15.9.3.1	Settings for Fixed Character Formats	31
15.9.3.2	Example: Partial Redaction Policy Using a Fixed Character Format	33
15.9.4	Creating Partial Redaction Policies Using Character Data Types	33
15.9.4.1	Settings for Character Data Types	34
15.9.4.2	Example: Partial Redaction Policy Using a Character Data Type	35
15.9.5	Creating Partial Redaction Policies Using Number Data Types	35
15.9.5.1	Settings for Number Data Types	35

15.9.5.2	Example: Partial Redaction Policy Using a Number Data Type	36
15.9.6	Creating Partial Redaction Policies Using Date-Time Data Types	37
15.9.6.1	Settings for Date-Time Data Types	37
15.9.6.2	Example: Partial Redaction Policy Using Date-Time Data Type	37
15.10	Creating a Regular Expression-Based Redaction Policy	38
15.10.1	About Creating Regular Expression-Based Redaction Policies	38
15.10.2	Syntax for Creating a Regular Expression-Based Redaction Policy	39
15.10.3	Regular Expression-Based Redaction Policies Using Formats	40
15.10.3.1	Regular Expression Formats	41
15.10.3.2	Example: Regular Expression Redaction Policy Using Formats	44
15.10.4	Custom Regular Expression Redaction Policies	44
15.10.4.1	Settings for Custom Regular Expressions	44
15.10.4.2	Example: Custom Regular Expression Redaction Policy	45
15.11	Creating a Random Redaction Policy	46
15.11.1	Syntax for Creating a Random Redaction Policy	46
15.11.2	Example: Random Redaction Policy	46
15.12	Creating a Policy That Uses No Redaction	47
15.12.1	Syntax for Creating a Policy with No Redaction	47
15.12.2	Example: Performing No Redaction	48
15.13	Exemption of Users from Oracle Data Redaction Policies	48
15.14	Altering an Oracle Data Redaction Policy	49
15.14.1	About Altering Oracle Data Redaction Policies	49
15.14.2	Syntax for the DBMS_REDACT.ALTER_POLICY Procedure	50
15.14.3	Parameters Required for DBMS_REDACT.ALTER_POLICY Actions	51
15.14.4	Tutorial: Altering an Oracle Data Redaction Policy	51
15.15	Redacting Multiple Columns	54
15.15.1	Adding Columns to a Data Redaction Policy for a Single Table or View	54
15.15.2	Example: Redacting Multiple Columns	54
15.16	Disabling and Enabling an Oracle Data Redaction Policy	55
15.16.1	Disabling an Oracle Data Redaction Policy	55
15.16.2	Enabling an Oracle Data Redaction Policy	56
15.17	Dropping an Oracle Data Redaction Policy	56
15.18	Tutorial: SQL Expressions to Build Reports with Redacted Values	57
15.19	Using Trace Files to Troubleshoot Oracle Data Redaction Policies	58
15.20	Oracle Data Redaction Policy Data Dictionary Views	59

16 Managing Oracle Data Redaction Policies in Oracle Enterprise Manager

16.1	About Using Oracle Data Redaction in Oracle Enterprise Manager	1
16.2	Oracle Data Redaction Workflow	2
16.3	Management of Sensitive Column Types in Enterprise Manager	2
16.4	Managing Oracle Data Redaction Formats Using Enterprise Manager	4

16.4.1	About Managing Oracle Data Redaction Formats Using Enterprise Manager	5
16.4.2	Creating a Custom Oracle Data Redaction Format Using Enterprise Manager	5
16.4.3	Editing a Custom Oracle Data Redaction Format Using Enterprise Manager	8
16.4.4	Viewing Oracle Data Redaction Formats Using Enterprise Manager	9
16.4.5	Deleting a Custom Oracle Data Redaction Format Using Enterprise Manager	10
16.5	Managing Oracle Data Redaction Policies Using Enterprise Manager	10
16.5.1	About Managing Oracle Data Redaction Policies Using Enterprise Manager	11
16.5.2	Creating an Oracle Data Redaction Policy Using Enterprise Manager	11
16.5.3	Editing an Oracle Data Redaction Policy Using Enterprise Manager	14
16.5.4	Viewing Oracle Data Redaction Policy Details Using Enterprise Manager	15
16.5.5	Enabling or Disabling an Oracle Data Redaction Policy in Enterprise Manager	16
16.5.6	Deleting an Oracle Data Redaction Policy Using Enterprise Manager	16
16.6	Managing Named Data Redaction Policy Expressions Using Enterprise Manager	17
16.6.1	About Named Data Redaction Policy Expressions in Enterprise Manager	17
16.6.2	Creating a Named Data Redaction Policy Expression in Enterprise Manager	18
16.6.3	Editing a Named Data Redaction Policy Expression in Enterprise Manager	19
16.6.4	Viewing Named Data Redaction Policy Expressions in Enterprise Manager	20
16.6.5	Deleting a Named Data Redaction Policy Expression in Enterprise Manager	20

17 Using Oracle Data Redaction with Oracle Database Features

17.1	Oracle Data Redaction General Usage Guidelines	2
17.2	Oracle Data Redaction and DML and DDL Operations	3
17.3	Oracle Data Redaction and Nested Functions, Inline Views, and the WHERE Clause	3
17.4	Oracle Data Redaction and Queries on Columns Protected by Data Redaction Policies	4
17.5	Oracle Data Redaction and Database Links	4
17.6	Oracle Data Redaction and Aggregate Functions	4
17.7	Oracle Data Redaction and Object Types	4
17.8	Oracle Data Redaction and XML Generation	5
17.9	Oracle Data Redaction and Editions	5
17.10	Oracle Data Redaction and Oracle Data Warehouse Query Rewrite Operation	5
17.11	Oracle Data Redaction in a Multitenant Environment	5
17.12	Oracle Data Redaction and Oracle Virtual Private Database	5
17.13	Oracle Data Redaction and Oracle Database Real Application Security	6
17.14	Oracle Data Redaction and Oracle Database Vault	6
17.15	Oracle Data Redaction and Oracle Data Pump	6
17.15.1	Oracle Data Pump Security Model for Oracle Data Redaction	7
17.15.2	Export of Objects That Have Oracle Data Redaction Policies Defined	7
17.15.2.1	Finding Type Names Used by Oracle Data Pump	8
17.15.2.2	Exporting Only the Data Dictionary Metadata Related to Data Redaction Policies	8
17.15.2.3	Importing Objects Using the INCLUDE Parameter in IMPDP	9

17.15.3	Export of Data Using the EXPDP Utility access_method Parameter	9
17.15.4	Import of Data into Objects Protected by Oracle Data Redaction	9
17.16	Oracle Data Redaction and Data Masking and Subsetting Pack	10
17.17	Oracle Data Redaction and JSON	10

18 Security Considerations for Oracle Data Redaction

18.1	Oracle Data Redaction General Security Guidelines	1
18.2	Restriction of Administrative Access to Oracle Data Redaction Policies	2
18.3	How Oracle Data Redaction Affects the SYS, SYSTEM, and Default Schemas	2
18.4	Policy Expressions That Use SYS_CONTEXT Attributes	3
18.5	Oracle Data Redaction Policies on Materialized Views	3
18.6	REDACTION_COLUMNS Data Dictionary View Behavior When a View Is Invalid	3
18.7	Dropped Oracle Data Redaction Policies When the Recycle Bin Is Enabled	4

Glossary

Index

Preface

Welcome to *Oracle Database Advanced Security Guide*. This guide describes how to implement, configure, and administer Transparent Data Encryption (TDE) and Oracle Data Redaction.

- [Audience](#)
- [Documentation Accessibility](#)
- [Related Documents](#)
- [Conventions](#)

Audience

Oracle Database Advanced Security Guide is intended for users and systems professionals involved with the implementation, configuration, and administration of Oracle Advanced Security including:

- Implementation consultants
- System administrators
- Security administrators
- Database administrators (DBAs)

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Access to Oracle Support

Oracle customer access to and use of Oracle support services will be pursuant to the terms and conditions specified in their Oracle order for the applicable services.

Related Documents

Before you configure Oracle Advanced Security features, you should be familiar with the following guides:

- *Oracle Database Administrator's Guide*
- *Oracle Database Security Guide*
- *Oracle Database SQL Language Reference*
- *Oracle Database PL/SQL Packages and Types Reference*
- *Oracle Multitenant Administrator's Guide*

Many books in the documentation set use the sample schemas of the default database. Refer to *Oracle Database Sample Schemas* for information about how these schemas were created and how you can use them.

Oracle Technical Services

To download the product data sheet, frequently asked questions, links to the latest product documentation, product download, and other collateral, visit Oracle Technical Resources (formerly Oracle Technology Network). You must register online before using Oracle Technical Services. Registration is free and can be done at

<https://www.oracle.com/technical-resources/>

My Oracle Support

You can find information about security patches, certifications, and the support knowledge base by visiting My Oracle Support (formerly Oracle*MetaLink*) at

<https://support.oracle.com>

Conventions

The following text conventions are used in this document:

Convention	Meaning
boldface	Boldface type indicates graphical user interface elements associated with an action, or terms defined in text or the glossary.
<i>italic</i>	Italic type indicates book titles, emphasis, or placeholder variables for which you supply particular values.
monospace	Monospace type indicates commands within a paragraph, URLs, code in examples, text that appears on the screen, or text that you enter.

Changes in This Release for Oracle Database Advanced Security Guide

This preface contains:

- [Changes in Oracle Database Advanced Security 19c](#)
- [Updates to Oracle Advanced Database Security 19c](#)

Changes in Oracle Database Advanced Security 19c

The following are changes in *Oracle Database Advanced Security Guide* for Oracle Database 19c.

- [OpenSSL Cryptographic Provider Support](#)
Starting with this release update, Oracle Database 19c supports the next-generation cryptographic provider based on OpenSSL, as an alternative to the default Dell BSAFE Crypto-C Micro Edition Suite, or MES (legacy provider). The next-generation provider enables FIPS 140-3 compliance.
- [Improved Key Management Support for Encrypting Oracle-Managed Tablespaces](#)
In this release, closing a TDE keystore is now allowed even when the Oracle-managed tablespaces (SYSTEM, SYSAUX, TEMP, and UNDO tablespaces) are encrypted.
- [Transparent Online Conversion Support for Auto-Renaming in Non-Oracle-Managed Files Mode](#)
Starting with this release, online tablespace encryption no longer requires you to specify the FILE_NAME_CONVERT clause in the ALTER TABLESPACE ENCRYPT SQL statement. The file names will retain their original names.
- [Support for More Algorithms for Offline Tablespace Encryption](#)
In previous releases, only the AES128 encryption algorithm was supported for offline tablespace encryption. In addition to AES128, this release introduces support for the AES192 and AES256 encryption algorithms, as well as ARIA, GOST, and 3DES encryption algorithms for offline tablespace encryption.

OpenSSL Cryptographic Provider Support

Starting with this release update, Oracle Database 19c supports the next-generation cryptographic provider based on OpenSSL, as an alternative to the default Dell BSAFE Crypto-C Micro Edition Suite, or MES (legacy provider). The next-generation provider enables FIPS 140-3 compliance.

The following changes affect Transparent Data Encryption (TDE) when using the next-generation provider:

- 3DES168 (deprecated) is desupported for data encryption when the next-generation provider operates in FIPS 140-3 mode. 3DES encryption is outdated, slow, and no longer considered secure. Oracle recommends transitioning to AES. Review your encryption

configurations to ensure you do not use 3DES. Existing data encrypted with 3DES168 must be re-encrypted with an AES algorithm before enabling FIPS 140-3.

- The GOST and SEED master encryption key algorithms are not supported with the next-generation provider. If your TDE master key uses one of these algorithms, rekey to AES256 before switching to the next-generation provider.
- Wallet compatibility is maintained between providers. However, keystore files created before Oracle Database 19.22 may use 3DES for PKCS#12 container encryption and must be converted to AES-256 using the `orapki wallet convert` command before enabling FIPS 140-3 mode.
- AES-CFB remains the only supported mode for tablespace encryption on Oracle Database 19c. AES-XTS mode is not available on this release.
- The default tablespace encryption algorithm is AES128 (CFB mode). The default column encryption algorithm is AES192 (CBC mode). Oracle recommends AES256 for both, as all modes of symmetric encryption using AES256 are considered quantum resistant.

FIPS Parameter Configuration

To enable FIPS mode with the next-generation provider, set the following parameters in the `fips.ora` file. The `fips.ora` file is located by default in `$ORACLE_HOME/ldap/admin`, or in a location specified by the `FIPS_HOME` environment variable:

- `FIPS_140=TRUE` enables FIPS mode.
- `FIPS_VERSION=FIPS140-3` (recommended) enables FIPS 140-3 mode.
- `FIPS_VERSION=FIPS140-2` (deprecated) explicitly selects FIPS 140-2 mode. In a future release, this value will be desupported and FIPS140-3 will become the default FIPS version.

In FIPS mode, only AES128, AES192, and AES256 are available for TDE operations.

Related Topics

- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

Improved Key Management Support for Encrypting Oracle-Managed Tablespaces

In this release, closing a TDE keystore is now allowed even when the Oracle-managed tablespaces (SYSTEM, SYSAUX, TEMP, and UNDO tablespaces) are encrypted.

Internal operations on these tablespaces when they are encrypted continue to be unaffected even when the TDE keystore is in the CLOSED state.

Closing the TDE keystore has no effect on queries of an encrypted SYSTEM, SYSAUX, TEMP, or UNDO tablespace, unlike queries of a user-created tablespace, which continue to return an ORA-28365 wallet is not open error when the TDE keystore is closed.

User-initiated operations such as decrypt on any encrypted Oracle-managed tablespace still require the TDE keystore to be in the OPEN state.

Related Topics

- [Impact of a Closed TDE Keystore on Encrypted Tablespaces](#)
A TDE keystore can be closed or migrated when an Oracle-managed tablespace is encrypted, and the database system itself must be shut down to disallow operations on an Oracle-managed tablespace.

Transparent Online Conversion Support for Auto-Renaming in Non-Oracle-Managed Files Mode

Starting with this release, online tablespace encryption no longer requires you to specify the `FILE_NAME_CONVERT` clause in the `ALTER TABLESPACE ENCRYPT` SQL statement. The file names will retain their original names.

This enhancement helps to prevent you from having to rename files back to the original name afterward, sometimes missing files.

Related Topics

- [Encrypting an Existing Tablespace with Online Conversion](#)
To encrypt an existing tablespace with online conversion, use `ALTER TABLESPACE` with the `ONLINE` and `ENCRYPT` clauses.

Support for More Algorithms for Offline Tablespace Encryption

In previous releases, only the AES128 encryption algorithm was supported for offline tablespace encryption. In addition to AES128, this release introduces support for the AES192 and AES256 encryption algorithms, as well as ARIA, GOST, and 3DES encryption algorithms for offline tablespace encryption.

This enhancement benefits scenarios in which you have concerns about auxiliary space usage required by online tablespace encryption.

Related Topics

- [About Encryption Conversion for Tablespaces and Databases](#)
The `CREATE TABLESPACE` SQL statement can be used to encrypt new tablespaces. `ALTER TABLESPACE` can encrypt existing tablespaces.
- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

Updates to Oracle Advanced Database Security 19c

Oracle Advanced Database Security release 19c has one update from the last update of release 19c.

- [Oracle Data Guard Redo Decryption for Hybrid Disaster Recovery Configurations](#)
Available for Oracle Database release 19.16, Oracle Data Guard enables you to decrypt redo operations in hybrid cloud disaster recovery configurations where the Cloud database is encrypted with TDE and the on-premises database is not.

Oracle Data Guard Redo Decryption for Hybrid Disaster Recovery Configurations

Available for Oracle Database release 19.16, Oracle Data Guard enables you to decrypt redo operations in hybrid cloud disaster recovery configurations where the Cloud database is encrypted with TDE and the on-premises database is not.

Hybrid disaster recovery is often considered a quick-stepping stone to cloud adoption. By enabling the ability to quickly configure disaster recovery even in cases where on-premises databases might not already be encrypted with TDE, the steps required to configure hybrid disaster recovery environments are reduced while still ensuring that redo data is still encrypted during the transportation process.

To enable this feature, Oracle Database introduces the `TABLESPACE_ENCRYPTION` initialization parameter, which enables you to control the automatic encryption of tablespaces in both the primary and standby databases, for on-premises and Oracle Cloud Infrastructure (OCI) environments. For example, an on-premises database can be unencrypted and an OCI database can be encrypted.

Related Topics

- [Encryption of Tablespaces in an Oracle Data Guard Environment](#)
You can control tablespace encryption in the primary and standby databases in an Oracle Data Guard environment.

1

Introduction to Oracle Advanced Security

Two features comprise Oracle Advanced Security: Transparent Data Encryption and Oracle Data Redaction.

- [Transparent Data Encryption](#)
Transparent Data Encryption (TDE) transparently encrypts data at rest and is transparent to database users and applications.
- [Oracle Data Redaction](#)
Data Redaction enables you to dynamically mask data as it is selected from the Oracle Database.

1.1 Transparent Data Encryption

Transparent Data Encryption (TDE) transparently encrypts data at rest and is transparent to database users and applications.

Use TDE to protect sensitive data from attacks that bypass the database and attempt to directly read the underlying database files. Examples of this type of attack might include ransomware (most ransomware also scrapes data from disk and infiltrates it for use in extortion attempts), lost, or stolen backup files, or direct access to the stored data using operating system tools like strings or cat. You can encrypt individual columns in a database table, or you can encrypt an entire tablespace. In almost all cases, tablespace encryption should be your first choice.

To use Transparent Data Encryption, you do not need to modify your applications. TDE enables your applications to continue working seamlessly as before. It automatically encrypts data when it is written to disk, and then automatically decrypts the data when your applications access it. Key management is built-in, eliminating the complex task of managing and securing encryption keys.

1.2 Oracle Data Redaction

Data Redaction enables you to dynamically mask data as it is selected from the Oracle Database.

Unlike the static data masking (see Oracle Data Safe [Oracle Data Safe](#) or Oracle Data Masking and Subsetting [Oracle Data Masking and Subsetting](#) for more information on this type of masking), Oracle Data Redaction does not actually change the stored data. It simply alters the presentation of that data at query time to prevent disclosure of sensitive information for those sessions that should not be able to view the actual data values, while still making the data available to authorized users under acceptable session conditions.

Data Redaction performs the redaction before the data is returned by the SQL query. Redaction is suited for production systems when you want to selectively obfuscate selected sensitive data for certain database users. While the data is being redacted, Oracle Database is able to process all of the data normally and to preserve the back-end referential integrity constraints. Data redaction can help you to comply with industry regulations such as Payment Card Industry Data Security Standard (PCI DSS) and the Sarbanes-Oxley Act by minimizing exposure of sensitive data such as PII to unauthorized persons.

Part I

Using Transparent Data Encryption

Part I describes how to use Transparent Data Encryption.

- [Quick-start Setup Guide for Wallet Based Transparent Data Encryption](#)
- [Introduction to Transparent Data Encryption](#)
Transparent data encryption enables you to encrypt database data files or selected columns of data. This helps you protect sensitive data contained in your database, such as credit card numbers or Social Security numbers.
- [Configuring Transparent Data Encryption](#)
You can configure software or external keystores, for use on both individual table columns or entire tablespaces.
- [Switching from MES to OpenSSL Cryptographic Provider](#)
Before switching from Dell BSAFE Crypto-C Micro Edition Suite, or MES (legacy provider) to OpenSSL (next-generation provider), complete the following pre-switch verification steps:
- [Managing the Keystore and the Master Encryption Key](#)
You can modify settings for the keystore and TDE master encryption key, and store Oracle Database and store Oracle GoldenGate secrets in a keystore.
- [Managing Keystores and TDE Master Encryption Keys in United Mode](#)
United mode enables you to create a common keystore for the CDB and the PDBs for which the keystore is in united mode.
- [Managing Keystores and TDE Master Encryption Keys in Isolated Mode](#)
Isolated mode enables you to create a keystore for each pluggable database (PDB).
- [General Considerations of Using Transparent Data Encryption](#)
When you use Transparent Data Encryption, you should consider factors such as security, performance, and storage overheads.
- [Using Transparent Data Encryption with Other Oracle Features](#)
You can use Oracle Data Encryption with other Oracle features, such as Oracle Data Guard or Oracle Real Application Clusters.
- [Frequently Asked Questions About Transparent Data Encryption](#)
Users frequently have questions about transparency and performance issues with Transparent Data Encryption.
- [Using sqlnet.ora to Configure Transparent Data Encryption Keystores](#)
Configuring TDE with `sqlnet.ora` parameters has been deprecated, but is still possible depending on your database configuration.

2

Quick-start Setup Guide for Wallet Based Transparent Data Encryption

1. Create the directories that will hold the TDE wallet (a PKCS#12 container that is encrypted with a key that is derived from the TDE wallet password).
The last two commands change the ownership of the directories to `oracle:oinstall` and reduce the file privileges to the minimum:

The `ORACLE_SID` in this example is `finance`:

```
mkdir -pv /etc/ORACLE/KEYSTORES/finance
chown -Rv oracle:oinstall /etc/ORACLE
chmod -Rv 700 /etc/ORACLE
```

2. Set static system parameter `WALLET_ROOT` to the directory that you just created:

```
SYS> alter system set WALLET_ROOT = '/etc/ORACLE/KEYSTORES/$ORACLE_SID'
scope = spfile;
```

3. Restart the database to activate this parameter.
4. The next parameter defines `AES256` as the default encryption encryption algorithm (it needs to be executed before the `create keystore` command, otherwise the default encryption algorithm remains `AES128`).

```
SYS> alter system set "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM" =
'AES256' scope = both;
```

5. The last parameter configures the database to use a TDE wallet for file-based TDE setup:

```
SYS> alter system set TDE_CONFIGURATION = "KEYSTORE_CONFIGURATION=FILE"
scope = both;
```

6. Create a new password-protected and local auto-open TDE wallet; the local auto-open wallet enables automatic database restarts without DBA intervention to open the password-protected TDE wallet:
(N.B.: This command also creates the `<WALLET_ROOT>/tde` directory)

```
SYSKM> administer key management CREATE KEYSTORE identified by <wallet-
pwd>;
```

```
SYSKM> administer key management CREATE LOCAL AUTO_LOGIN KEYSTORE from
keystore identified by <wallet-pwd>;
```

7. In the root container database, set the first TDE master key:

```
SYSKM> administer key management SET KEY force keystore identified by
EXTERNAL STORE with backup container = current;
```

8. Create either a united or isolated PDB:

- **United PDB** In the PDB, set the first TDE master key:

```
SYSKM:FINPDB19C> administer key management SET KEY force keystore
identified by EXTERNAL STORE with backup;
```

- **Isolated PDB**

- a. : Create an isolated PDB with its own individual keystore and keystore password:

```
SYSKM:FINPDB19C> administer key management CREATE KEYSTORE
identified by <PDB-wallet-pwd>;
```

This next command does three things:

- i. It sets TDE_CONFIGURATION to FILE for the isolated PDB
 - ii. It creates the <PDB_GUID>/tde directories under <WALLET_ROOT>
 - iii. It creates an individual wallet for the PDB, with its own TDE wallet password (that is potentially unknown to the DBA of the root container)
- b. Create a (local) auto-open wallet for the isolated PDB:

```
SYSKM:FINPDB19C> administer key management CREATE LOCAL AUTO_LOGIN
KEYSTORE from keystore identified by <PDB-wallet-pwd>;
```

9. Encrypt the tablespaces in the PDB:

```
SYS:FINPDB19C> alter tablespace USERS encryption ONLINE encrypt;
SYS:FINPDB19C> alter tablespace SYSTEM encryption ONLINE encrypt;
SYS:FINPDB19C> alter tablespace SYSAUX encryption ONLINE encrypt;
```

10. Confirm:

```
SYS> select c.name as PDB_NAME, t.name as TBS_NAME, e.ENCPTIONALG as
ALG, e.STATUS from v$tablespace t, v$encrypted_tablespaces e, v$containers
c where e.ts# = t.ts# and e.con_id = t.con_id and e.con_id = c.con_id
order by e.con_id, t.name;
```

PDB_NAME	TBS_NAME	ALG	STATUS
-----	-----	-----	-----
FINPDB19C	SYSAUX	AES256	NORMAL
FINPDB19C	SYSTEM	AES256	NORMAL
FINPDB19C	USERS	AES256	NORMAL

3

Introduction to Transparent Data Encryption

Transparent data encryption enables you to encrypt database data files or selected columns of data. This helps you protect sensitive data contained in your database, such as credit card numbers or Social Security numbers.

- [What Is Transparent Data Encryption?](#)
Transparent Data Encryption (TDE) enables you to encrypt sensitive data that you store in tables and tablespaces. It also enables you to encrypt database backups.
- [Benefits of Using Transparent Data Encryption](#)
Transparent Data Encryption (TDE) ensures that sensitive data is encrypted, helps address compliance requirements, and provides functionality that streamlines encryption operations.
- [Who Can Configure Transparent Data Encryption?](#)
You must be granted the ADMINISTER KEY MANAGEMENT system privilege to configure Transparent Data Encryption (TDE).
- [Types and Components of Transparent Data Encryption](#)
Transparent Data Encryption can be applied to individual columns or entire tablespaces.
- [How the Multitenant Option Affects Transparent Data Encryption](#)
In a multitenant environment, you can configure keystores for either the entire container database (CDB) or for individual pluggable databases (PDBs).

3.1 What Is Transparent Data Encryption?

Transparent Data Encryption (TDE) enables you to encrypt sensitive data that you store in tables and tablespaces. It also enables you to encrypt database backups.

After the data is encrypted, this data is transparently decrypted for authorized users or applications when they access this data. TDE helps protect data stored on media (also called data at rest) in the event that the storage media or data file is stolen.

Oracle Database uses authentication, authorization, and auditing mechanisms to secure data in the database, but not in the operating system data files where data is stored. To protect these data files, Oracle Database provides Transparent Data Encryption (TDE). TDE encrypts sensitive data stored in data files. To prevent unauthorized decryption, TDE stores the encryption keys in a security module that is external to the database. This security module can be referred to as follows:

- **TDE wallets** are wallets used for TDE. They cannot contain other security artifacts such as certificates. In previous releases, they were called software keystores or just wallets.
- **External keystores** refer to Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS).
- **Keystores** is a generic term for both TDE wallets and external keystores.

You can configure Oracle Key Vault as part of the TDE implementation. This enables you to centrally manage keystores in your enterprise. For example, you can upload a TDE wallet to Oracle Key Vault, migrate the database to use Oracle Key Vault as the default keystore, and then share the contents of this keystore with other primary and standby Oracle Real

Application Clusters (Oracle RAC) nodes of that database to streamline daily database administrative operations with encrypted databases.

Related Topics

- *Oracle Key Vault Administrator's Guide*

3.2 Benefits of Using Transparent Data Encryption

Transparent Data Encryption (TDE) ensures that sensitive data is encrypted, helps address compliance requirements, and provides functionality that streamlines encryption operations.

Benefits are as follows:

- As a security administrator, you can be sure that sensitive data is encrypted and therefore safe in the event that the storage media or data file is stolen.
- Using TDE helps you address security-related regulatory compliance issues.
- You do not need to create auxiliary tables, triggers, or views to decrypt data for the authorized user or application. Data from tables is transparently decrypted for the database user and application. An application that processes sensitive data can use TDE to provide strong data encryption with little or no change to the application.
- Data is transparently decrypted for database users and applications that access this data. Database users and applications do not need to be aware that the data they are accessing is stored in encrypted form.
- You can encrypt data with zero downtime on production systems by using online table redefinition or you can encrypt it offline during maintenance periods.
- You do not need to modify your applications to handle the encrypted data. The database manages the data encryption and decryption.
- Oracle Database automates TDE master encryption key and keystore management operations. The user or application does not need to manage TDE master encryption keys.

3.3 Who Can Configure Transparent Data Encryption?

You must be granted the `ADMINISTER KEY MANAGEMENT` system privilege to configure Transparent Data Encryption (TDE).

If you must open the keystore at the mount stage, then you must be granted the `SYSKM` administrative privilege, which includes the `ADMINISTER KEY MANAGEMENT` system privilege and other necessary privileges.

When you grant the `SYSKM` administrative privilege to a user, ensure that you create a password file for it so that the user can connect to the database as `SYSKM` using a password. This enables the user to perform actions such as querying the `V$DATABASE` view.

To use TDE, you do not need the `SYSKM` or `ADMINISTER KEY MANAGEMENT` privileges. You must have the following additional privileges to encrypt table columns and tablespaces:

- `CREATE TABLE`
- `ALTER TABLE`
- `CREATE TABLESPACE`
- `ALTER TABLESPACE` (for online and offline tablespace encryption)
- `ALTER DATABASE` (for fast offline tablespace encryption)

3.4 Types and Components of Transparent Data Encryption

Transparent Data Encryption can be applied to individual columns or entire tablespaces.

- [About Transparent Data Encryption Types and Components](#)
You can encrypt sensitive data at the column level or the tablespace level.
- [How Transparent Data Encryption Tablespace Encryption Works](#)
Transparent Data Encryption (TDE) tablespace encryption enables you to encrypt an entire tablespace.
- [How Transparent Data Encryption Column Encryption Works](#)
Transparent Data Encryption (TDE) column encryption protects confidential data, such as credit card and Social Security numbers, that is stored in table columns.
- [How the Keystore for the Storage of TDE Master Encryption Keys Works](#)
To control the encryption, you use a keystore and a TDE master encryption key.
- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

3.4.1 About Transparent Data Encryption Types and Components

You can encrypt sensitive data at the column level or the tablespace level.

At the column level, you can encrypt sensitive data in application table columns. TDE tablespace encryption enables you to encrypt all of the data that is stored in a tablespace.

Both TDE column encryption and TDE tablespace encryption use a two-tiered key-based architecture. Unauthorized users, such as intruders who are attempting security attacks, cannot read the data from storage and back up media unless they have the TDE master encryption key to decrypt it.

3.4.2 How Transparent Data Encryption Tablespace Encryption Works

Transparent Data Encryption (TDE) tablespace encryption enables you to encrypt an entire tablespace.

All of the objects that are created in the encrypted tablespace are automatically encrypted. TDE tablespace encryption is useful if your tables contain sensitive data in multiple columns, or if you want to protect the entire table and not just individual columns. You do not need to perform a granular analysis of each table column to determine the columns that need encryption.

In addition, TDE tablespace encryption takes advantage of bulk encryption and caching to provide enhanced performance. The actual performance impact on applications can vary.

TDE tablespace encryption encrypts all of the data stored in an encrypted tablespace including its redo data. TDE tablespace encryption does not encrypt data that is stored outside of the tablespace. For example, BFILE data is not encrypted because it is stored outside the database. If you create a table with a BFILE column in an encrypted tablespace, then this particular column will not be encrypted.

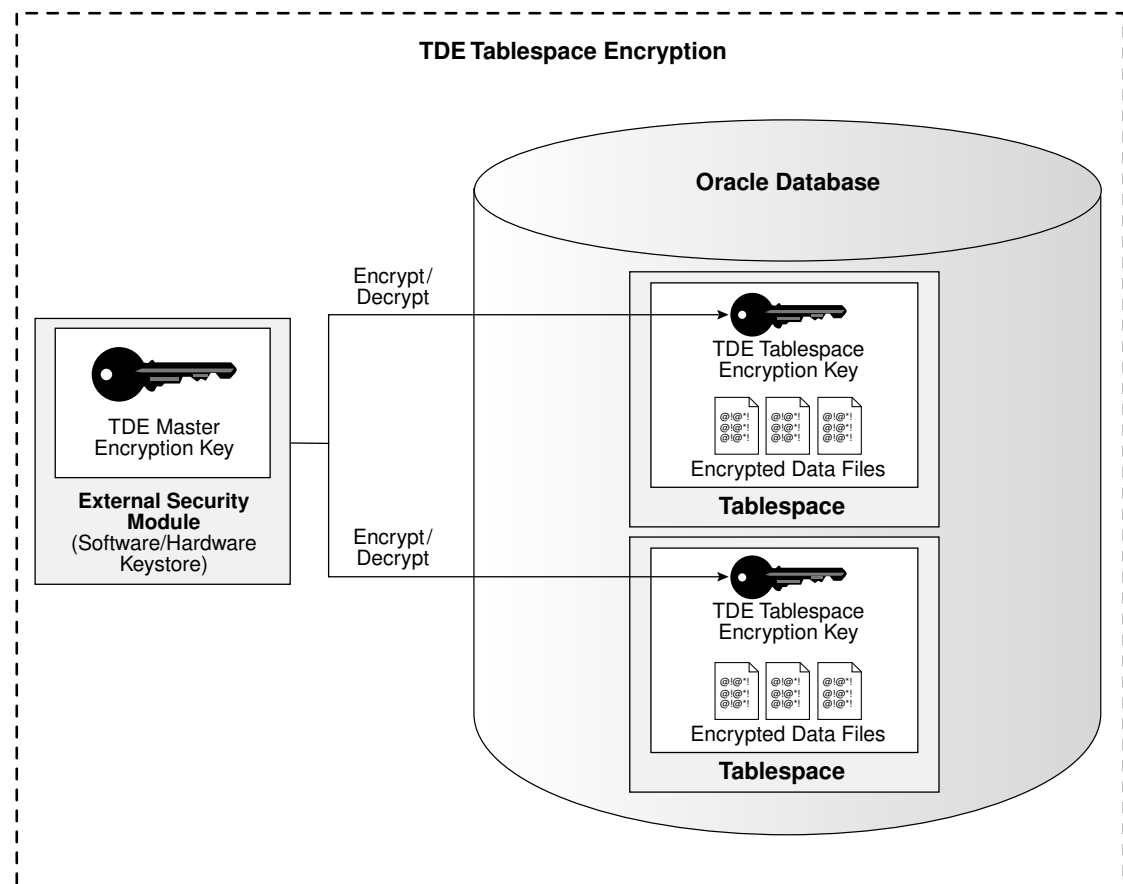
All of the data in an encrypted tablespace is stored in encrypted format on the disk. Data is transparently decrypted for an authorized user having the necessary privileges to view or

modify the data. A database user or application does not need to know if the data in a particular table is encrypted on the disk. In the event that the data files on a disk or backup media is stolen, the data remains protected.

TDE tablespace encryption uses the two-tiered, key-based architecture to transparently encrypt (and decrypt) tablespaces. The TDE master encryption key is stored in a security module (Oracle wallet, Oracle Key Vault, or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS)). This TDE master encryption key is used to encrypt the TDE [tablespace encryption key](#), which in turn is used to encrypt and decrypt data in the tablespace.

[Figure 3-1](#) shows an overview of the TDE tablespace encryption process.

Figure 3-1 TDE Tablespace Encryption



Note

The encrypted data is protected during operations such as JOIN and SORT. This means that the data is safe when it is moved to temporary tablespaces. Data in undo and redo logs is also protected.

TDE tablespace encryption also allows index range scans on data in encrypted tablespaces. It does not interfere with Exadata Hybrid Columnar Compression (EHCC), Oracle Advanced Compression, or Oracle Recovery Manager (Oracle RMAN) compression. This is not possible with TDE column encryption.

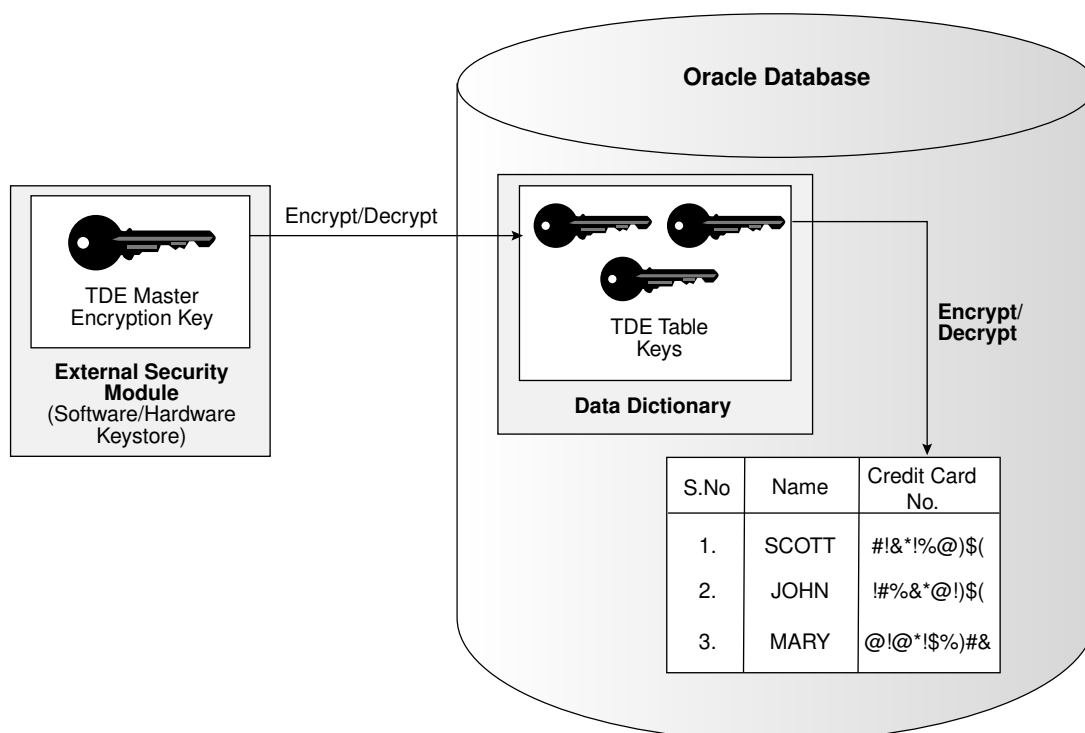
3.4.3 How Transparent Data Encryption Column Encryption Works

Transparent Data Encryption (TDE) column encryption protects confidential data, such as credit card and Social Security numbers, that is stored in table columns.

TDE column encryption uses the two-tiered key-based architecture to transparently encrypt and decrypt sensitive table columns. The TDE master encryption key is stored in an external keystore, which can be an Oracle wallet or in Oracle Key Vault. The Oracle wallet is a PKCS#12 container for certificates and encryption keys. It is encrypted with an AES256 key that is derived from the TDE wallet password. This TDE master encryption key encrypts and decrypts the TDE table key, which in turn encrypts and decrypts data in the table column.

[Figure 3-2](#) shows an overview of the TDE column encryption process.

Figure 3-2 TDE Column Encryption Overview



As shown in [Figure 3-2](#), the TDE master encryption key is stored in an external security module that is outside of the database and accessible only to a user who was granted the appropriate privileges. For this external security module, Oracle Database uses an Oracle TDE wallet (TDE wallet, in previous releases) or Oracle Key Vault. Storing the TDE master encryption key in this way prevents its unauthorized use.

Using an external security module separates ordinary program functions from encryption operations, making it possible to assign separate, distinct duties to database administrators and security administrators. Security is enhanced because the keystore password can be unknown to the database administrator, requiring the security administrator to provide the password.

When a table contains encrypted columns, TDE uses a single TDE table key regardless of the number of encrypted columns. Each TDE table key is individually encrypted with the TDE master encryption key.

3.4.4 How the Keystore for the Storage of TDE Master Encryption Keys Works

To control the encryption, you use a keystore and a TDE master encryption key.

- [About the Keystore Storage of TDE Master Encryption Keys](#)
Oracle Database provides a key management framework for Transparent Data Encryption (TDE) that stores and manages keys and credentials.
- [Benefits of the Keystore Storage Framework](#)
The key management framework provides several benefits for Transparent Data Encryption.
- [Types of Keystores](#)
Oracle Database supports TDE wallets, Oracle Key Vault, and Oracle Cloud Infrastructure (OCI) key management systems (KMS).

3.4.4.1 About the Keystore Storage of TDE Master Encryption Keys

Oracle Database provides a key management framework for Transparent Data Encryption (TDE) that stores and manages keys and credentials.

The key management framework includes the keystore to securely store the TDE master encryption keys and the management framework to securely and efficiently manage keystore and key operations for various database components.

The Oracle keystore stores a history of retired TDE master encryption keys, which enables you to rotate the TDE master encryption key, and still be able to decrypt data (for example, for incoming Oracle Recovery Manager (Oracle RMAN) backups) that was encrypted under an earlier TDE master encryption key.

3.4.4.2 Benefits of the Keystore Storage Framework

The key management framework provides several benefits for Transparent Data Encryption.

- Enables separation of duty between the database administrator and the security administrator who manages the keys. You can grant the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege to users who are responsible for managing the keystore and key operations.
- Facilitates compliance, because it helps you to track encryption keys and implement requirements such as keystore password rotation and TDE master encryption key re-key operations. Both wallet password rotations and TDE master key re-key operation do not require database or application downtime.
- Facilitates and helps enforce keystore backup requirements. A backup is a copy of the password-protected TDE wallet that is created for all of the critical keystore operations.

The mandatory `WITH BACKUP` clause of the `ADMINISTER KEY MANAGEMENT` statement creates a backup of the password-protected wallet before the changes are applied to the original password-protected wallet.

- Enables the keystore to be stored on an Oracle Automatic Storage Management (Oracle ASM) file system. This is particularly useful for Oracle Real Application Clusters (Oracle RAC) environments where database instances share a unified file system view. In Oracle

RAC, you must store the Oracle wallet in a shared location (Oracle ASM or Oracle Advanced Cluster File System (ACFS)), to which all Oracle RAC instances that belong to one database, have access to. Individual TDE wallets for each Oracle RAC instances are not supported.

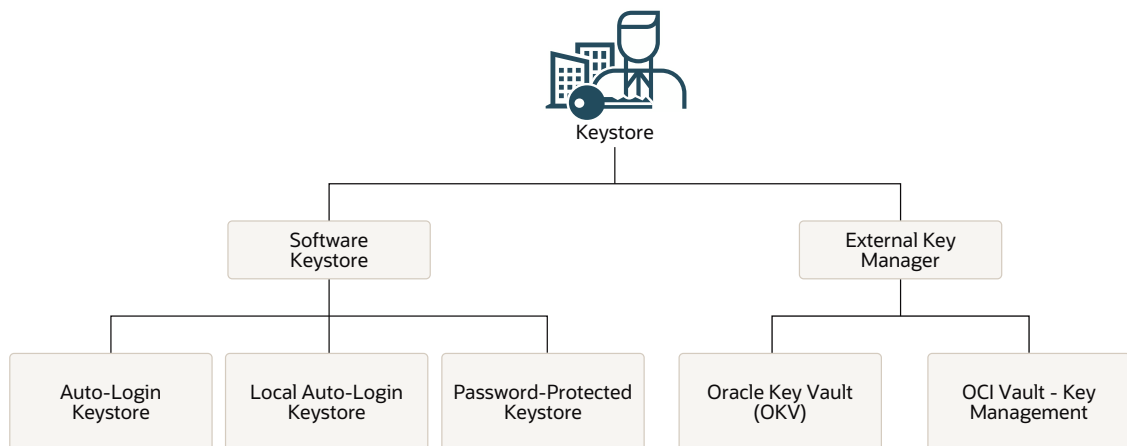
- Enables reverse migration from an external keystore to a file system-based TDE wallet. This option is useful if you must migrate back to a TDE wallet.

3.4.4.3 Types of Keystores

Oracle Database supports TDE wallets, Oracle Key Vault, and Oracle Cloud Infrastructure (OCI) key management systems (KMS).

[Figure 3-3](#) illustrates the types of keystores that Oracle Database supports.

Figure 3-3 Oracle Database Supported Keystores



These keystores are as follows:

- **Auto-login TDE wallets:** Auto-login TDE wallets are protected by a system-generated password, and do not need to be explicitly opened by a security administrator. Auto-login TDE wallets are automatically opened when accessed at database startup. Auto-login TDE wallets can be used across different systems. If your environment does not require the extra security provided by a keystore that must be explicitly opened for use, then you can use an auto-login TDE wallet. Auto-login TDE wallets are ideal for unattended scenarios (for example, Oracle Data Guard standby databases).
- **Local auto-login TDE wallets:** Local auto-login TDE wallets are auto-login TDE wallets that are local to the computer on which they are created. Local auto-login keystores cannot be opened on any computer other than the one on which they are created. This type of keystore is typically used for scenarios where additional security is required (that is, to limit the use of the auto-login for that computer) while supporting an unattended operation. You cannot use local auto-open wallets in Oracle RAC-enabled databases, because only shared wallets (in ACFS or ASM) are supported.
- **Password-protected TDE wallets:** Password-protected TDE wallets are protected by using a password that you create. You must open this type of keystore before the keys can be retrieved or used and use a password to open this type of keystore.

TDE wallets can be stored in Oracle Automatic Storage Management (Oracle ASM), Oracle Automatic Storage Management Cluster File System (Oracle ACFS), or regular file systems.

Under External Keystore Manager are the following categories:

- **Oracle Key Vault (OKV):** Oracle Key Vault is a software appliance that provides continuous key availability and scalable key management through clustering with up to 16 Oracle Key Vault nodes, potentially deployed across geographically distributed data centers. It is purpose-built for Oracle Database and its many deployment models (Oracle RAC, Oracle Data Guard, Exadata, multitenant environments). In addition, Oracle Key Vault provides online key management for Oracle GoldenGate encrypted trail files and encrypted ACFS. It is also certified for ExaDB-C@C and Autonomous Database (dedicated) (ADB-C@C). Oracle Key Vault is distributed as a full-stack software appliance for installation on dedicated hardware. It is also available in the OCI Marketplace and can be deployed in your OCI tenancy quickly and easily. See the video [Deploying Oracle Key Vault in OCI](#).
- **OCI Vault - Key Management:** The Oracle Cloud Infrastructure (OCI) Key Management Service (KMS) is a cloud-based service that provides centralized management and control of encryption keys for data stored in OCI. It enables integration of encryption with other OCI services such as storage, database, and Fusion Applications for protecting data stored in these services.

Cryptographic Provider Considerations for Keystores

The behavior of TDE software keystores is consistent across both the legacy provider and the next-generation provider. Keystore files created with one provider can be used with the other provider without conversion, provided the keystore encryption algorithm is supported by the target provider.

Wallet Encryption Algorithm

Wallets created with `orapki` from Oracle Database 19.22 onward are encrypted by default with the AES-256 algorithm. In earlier releases, the wallet was encrypted with the now obsolete 3DES algorithm. If you have upgraded from earlier database releases using the existing wallet, you may still be using 3DES.

To determine the encryption algorithm your wallet is using, run:

```
openssl pkcs12 -info -in ./ewallet.p12 -nodes
```

If the wallet uses 3DES encryption, convert it to AES-256 before switching to the next-generation provider in FIPS 140-3 mode:

```
orapki wallet convert -wallet wallet_location -compat_v12
```

The wallets are encrypted with a user-supplied passphrase up to 128 bytes. For TDE wallets, use the `ADMINISTER KEY MANAGEMENT` command to change its passphrase, else use `orapki`.

Note

If the keystore file (`ewallet.p12`) uses 3DES encryption and you enable the next-generation provider in FIPS 140-3 mode without first converting the wallet, the TDE master encryption key becomes inaccessible and encrypted data cannot be read.

Related Topics

- *Oracle Key Vault Administrator's Guide*

- [How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT](#)
Many of the clauses from the ALTER SYSTEM statement correspond to the ADMINISTER KEY MANAGEMENT statement.

3.4.5 Supported Encryption and Integrity Algorithms

The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

for TDE column encryption, [salt](#) is added by default to plaintext before encryption unless specified otherwise. You cannot add salt to indexed columns that you want to encrypt. For indexed columns, choose the NO SALT parameter for the SQL ENCRYPT clause.

For TDE tablespace encryption and database encryption, the default is to use the Advanced Encryption Standard with a 128-bit length cipher key (AES128). By default, Transparent Data Encryption (TDE) column encryption uses the Advanced Encryption Standard (AES) with a 192-bit length cipher key (AES192).

You can change encryption algorithms and encryption keys on existing encrypted columns by setting a different algorithm with the SQL ENCRYPT clause.

[Table 3-1](#) lists the supported encryption algorithms.

Table 3-1 Supported Encryption Algorithms for Transparent Data Encryption

Algorithm	Key Size	Parameter Name
Advanced Encryption Standard (AES)	• 128 bits (default for tablespace encryption) • 192 bits (default for column encryption) • 256 bits	• AES128 • AES192 • AES256
ARIA	• 128 bits • 192 bits • 256 bits	• ARIA128 • ARIA192 • ARIA256
GOST	256 bits	GOST256
SEED	128 bits	SEED128
Triple Encryption Standard (DES)	168 bits	3DES168

For integrity protection of TDE column encryption, the SHA-1 hashing algorithm is used. If you have storage restrictions, then use the NOMAC option.

Comparing OpenSSL vs. MES Encryption Algorithms

Note

3DES168 encryption is outdated, slow, and no longer considered secure. FIPS 140-3 desupports 3DES. Oracle recommends transitioning to AES. Review your encryption configurations to ensure you do not use 3DES.

When the database is configured to use the next-generation provider in FIPS 140-3 mode, 3DES168 is desupported and is not available for column encryption or tablespace encryption. If you have existing columns or tablespaces encrypted with 3DES168 and plan to enable FIPS 140-3 mode with the next-generation provider, you must re-encrypt the data using an AES algorithm (AES128, AES192, or AES256) before switching to FIPS 140-3 mode.

All modes of symmetric encryption using AES256 are considered quantum resistant. Oracle recommends using AES256 for new encrypted columns and tablespaces.

Table 3-2 TDE Data Encryption Algorithms for Oracle Database 19c

Algorithm	Default (Tablespace)	Default (Column)	Legacy provider (Non-FIPS)	Legacy provider (FIPS 140-2)	Next-Gen provider (Non-FIPS)	Next-Gen provider (FIPS 140-3)
3DES168 (deprecated)	No	No	Supported	Supported	Supported	Desupported
AES128	Yes (CFB)	No	Supported	Supported	Supported	Supported
AES192	No	Yes (CBC)	Supported	Supported	Supported	Supported
AES256	No	No	Supported	Supported	Supported	Supported
ARIA128	No	No	Supported	Not Supported	Supported	Not Supported
ARIA192	No	No	Supported	Not Supported	Supported	Not Supported
ARIA256	No	No	Supported	Not Supported	Supported	Not Supported
GOST256	No	No	Supported	Not Supported	Availability varies	Not Supported
SEED128	No	No	Supported	Not Supported	Supported	Not Supported

Though the default tablespace encryption algorithm is AES128 (CFB mode) and the default column encryption algorithm is AES192 (CBC mode), Oracle recommends using AES256 for both.

CFB is the only supported encryption mode for tablespace encryption on Oracle Database 19c. AES-XTS mode is not available on this release.

In FIPS mode (both FIPS 140-2 and FIPS 140-3), only AES128, AES192, and AES256 are available for TDE operations.

To change the default tablespace encryption algorithm from AES128, set the hidden initialization parameter:

```
ALTER SYSTEM SET "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM"='AES256'
SCOPE=BOTH;
```

If the database uses the next-generation provider in FIPS 140-3 mode, any attempt to encrypt data with 3DES168 returns an error. Existing data encrypted with 3DES168 remains readable only while the database operates in a mode that supports 3DES168.

Note

The SHA-1 based message authentication code (MAC) used for TDE column encryption integrity checking continues to be supported with both the legacy provider and the next-generation provider. Although SHA-1 is deprecated for digital signatures under FIPS 140-3, its use as an HMAC for data integrity in TDE column encryption remains permitted under the FIPS 140-3 standard. No action is required for existing columns that use SHA-1 MAC integrity checking when switching to the next-generation provider.

Related Topics

- [Setting the Tablespace Encryption Default Algorithm](#)
The `TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM` applies to specific encryption scenarios.

3.5 How the Multitenant Option Affects Transparent Data Encryption

In a multitenant environment, you can configure keystores for either the entire container database (CDB) or for individual pluggable databases (PDBs).

Oracle Database supports the following multitenant modes for the management of keystores:

- United mode enables you to configure one keystore for the CDB root and any associated united mode PDBs. United mode operates much the same as how TDE was managed in an multitenant environment in previous releases.
- Isolated mode enables you to create and manage both keystores and TDE master encryption keys in an individual PDB. Different isolated mode PDBs can have different keystore types.

Oracle Database supports isolated PDBs with TDE wallets (wallets) and Oracle Key Vault. The cloud tooling in Oracle Cloud Infrastructure (OCI) and the OCI Key Management Service (KMS), do not support isolated PDBs. This includes Oracle Exadata Cloud@Customer (ExaDB-C@C), Autonomous Database Cloud@Customer (ADB-C@C), and Oracle Exadata Database Service (ExaDB-D and ExaDB-D@Azure).

Depending on your site's needs, you can use a mixture of both united mode and isolated mode. For example, if you want most of the PDBs to use one type of a keystore, then you can configure the keystore type in the CDB root (united mode). For the PDBs in this CDB that must use a different type of keystore, then you can configure the PDB itself to use the keystore it needs (isolated mode). The isolated mode setting for the PDB will override the united mode setting for the CDB.

Before you can configure keystores for use in united or isolated mode, you must perform a one-time configuration by using initialization parameters. To configure keystores for united mode and isolated mode, you use the `ADMINISTER KEY MANAGEMENT` statement.

Related Topics

- [Managing Keystores and TDE Master Encryption Keys in United Mode](#)
United mode enables you to create a common keystore for the CDB and the PDBs for which the keystore is in united mode.
- [Managing Keystores and TDE Master Encryption Keys in Isolated Mode](#)
Isolated mode enables you to create a keystore for each pluggable database (PDB).

- [Using sqlnet.ora to Configure Transparent Data Encryption Keystores](#)
Configuring TDE with sqlnet.ora parameters has been deprecated, but is still possible depending on your database configuration.

4

Configuring Transparent Data Encryption

You can configure software or external keystores, for use on both individual table columns or entire tablespaces.

- [About Configuring Transparent Data Encryption](#)
To configure Transparent Data Encryption, you must perform a one-time setup before you create keystores and encrypt data.
- [Transparent Data Encryption Keystore Search Order](#)
The search order for the TDE keystore depends on how you have set either the instance initialization parameters, the `sqlnet.ora` parameters, or the environment variables.
- [Configuring a TDE Wallet](#)
A TDE wallet is a container for the master encryption key, and it resides in the software file system.
- [Configuring an External Keystore](#)
There are two types of external keystores that Oracle Database supports: Oracle Key Vault keystores and OCI Vault - Key Management keystores.
- [Encrypting Columns in Tables](#)
You can use Transparent Data Encryption to encrypt individual columns in database tables.
- [Encryption Conversions for Tablespaces and Databases](#)
You can perform encryption operations on both offline and online tablespaces and databases.
- [Transparent Data Encryption Data Dynamic and Data Dictionary Views](#)
You can query a set of dynamic and data dictionary views to find more information about Transparent Data Encryption (TDE) data.

4.1 About Configuring Transparent Data Encryption

To configure Transparent Data Encryption, you must perform a one-time setup before you create keystores and encrypt data.

Before you can begin to encrypt data, you must perform a one-time configuration using the static `WALLET_ROOT` parameter and the dynamic `TDE_CONFIGURATION` parameter to designate the location and type of keystores that you plan to use.

The `WALLET_ROOT` parameter specifies the top directory for many different software keystores (such as TDE, Oracle Enterprise User Security (EUS), TLS). For TDE, the directory for automated discovery is `WALLET_ROOT/tde`.

The `TDE_CONFIGURATION` parameter specifies the type of keystore (software keystore or Oracle Key Vault). After you set the type of keystore using `TDE_CONFIGURATION`, when you create the keystore, Oracle Database creates a directory within the `WALLET_ROOT` location for the keystore type. For example, if you set `TDE_CONFIGURATION` to `FILE`, then Oracle Database creates a TDE wallet in `WALLET_ROOT/tde`. To use Oracle Key Vault, install the Oracle Key Vault client software into `WALLET_ROOT/okv` and set `TDE_CONFIGURATION` to `OKV`. To establish an auto-open Oracle Key Vault configuration, add the Oracle Key Vault password into an auto-open wallet in `WALLET_ROOT/tde`, and set `TDE_CONFIGURATION` to `OKV|FILE`. If you want to migrate from one

keystore type to another, then you must first set `TDE_CONFIGURATION` parameter to the keystore type that you want to use, and then use the `ADMINISTER KEY MANAGEMENT` statement to perform the migration. For example, you can migrate from a TDE keystore to OKV by setting it to OKV|FILE.

The `KESTORE_MODE` column of the `V$ENCRYPTION_WALLET` dynamic view shows whether united mode or isolated mode has been configured. If you want to change the database default encryption algorithm from AES128, then after you apply patch 30398099, you can set a dynamic initialization parameter by executing the following statement:

```
ALTER SYSTEM SET "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM" = 'AES256' SCOPE
= BOTH SID = '*';
```

① Note

In previous releases, the `SQLNET.ENCRYPTION_WALLET_LOCATION` parameter was used to define the keystore directory location. This parameter has been deprecated. Oracle recommends that you use the `WALLET_ROOT` static initialization parameter and `TDE_CONFIGURATION` dynamic initialization parameter instead.

Related Topics

- [Transparent Data Encryption Keystore Search Order](#)
The search order for the TDE keystore depends on how you have set either the instance initialization parameters, the `sqlnet.ora` parameters, or the environment variables.

4.2 Transparent Data Encryption Keystore Search Order

The search order for the TDE keystore depends on how you have set either the instance initialization parameters, the `sqlnet.ora` parameters, or the environment variables.

Oracle Database retrieves the keystore by searching in these locations, in the following order:

1. The location set by the `WALLET_ROOT` instance initialization parameter, when the `KESTORE_CONFIGURATION` attribute of the `TDE_CONFIGURATION` initialization parameter is set to FILE. Oracle recommends that you use this parameter to configure the keystore location.
2. If the `KESTORE_CONFIGURATION` attribute of the `TDE_CONFIGURATION` initialization parameter is not set to FILE or `WALLET_ROOT` is not set, then the location specified by the `WALLET_LOCATION` setting in the `sqlnet.ora` file. (Note that you cannot have multiple `WALLET_LOCATION` settings in this file.)
3. If `WALLET_ROOT` and `WALLET_LOCATION` are not set, then the location specified by the `ENCRYPTION_WALLET_LOCATION` parameter (now deprecated in favor of `WALLET_ROOT`) in the `sqlnet.ora` file.
4. If none of these parameters are set, and if the `ORACLE_BASE` environment variable is set, then the `$ORACLE_BASE/admin/db_unique_name/wallet` directory. If `ORACLE_BASE` is not set, then `$ORACLE_HOME/admin/db_unique_name/wallet`.

4.3 Configuring a TDE Wallet

A TDE wallet is a container for the master encryption key, and it resides in the software file system.

- [About Configuring a TDE Wallet](#)
A software keystore is a container that stores the TDE master encryption key.
- [Cryptographic Provider Prerequisite for TDE Configuration](#)
Before configuring TDE, verify which cryptographic provider the database is using. The cryptographic provider determines which encryption algorithms are available for TDE operations and which FIPS modes can be enabled.
- [Step 1: Configure the TDE Wallet Location and Type](#)
You must configure the TDE wallet location and type by setting `WALLET_ROOT` in `init.ora` and `TDE_CONFIGURATION` in the database instance.
- [Step 2: Create the TDE Wallet](#)
After you have specified a directory location for the software keystore, you can create the TDE wallet.
- [Step 3: Open the TDE Wallet](#)
Depending on the type of keystore you create, you must manually open the TDE wallet before you can use it.
- [Step 4: Set the TDE Master Encryption Key in the TDE Wallet](#)
Once the TDE wallet is open, you can set a TDE master encryption key for it.
- [Step 5: Encrypt Your Data](#)
Now that you have completed the configuration, you can begin to encrypt data in the PDB.

4.3.1 About Configuring a TDE Wallet

A software keystore is a container that stores the TDE master encryption key.

Before you can configure the TDE wallet, you first must define a location for it by setting the static initialization parameter `WALLET_ROOT`. Then, after a database restart, you must set the dynamic initialization parameter `TDE_CONFIGURATION` to instruct the database to retrieve the master encryption key from a TDE wallet, Oracle Key Vault, or Oracle Cloud Interface (OCI) Key Management Service (KMS), according to their documentation. The database locates this TDE wallet by first checking the `WALLET_ROOT` setting. If this setting has not been created, then the database checks the `sqlnet.ora` file. You can create other TDE wallets, such as copies of the wallet and export files that contain keys, depending on your needs. If you must remove or delete the TDE wallet that you configured in the `WALLET_ROOT` location, then you must do so **only** after you copied **all** TDE wallets (including backups and auto-login wallets) in the old to the new location. Then you must reset `WALLET_ROOT` to point to the new location of the TDE wallet.

After you configure the TDE wallet location by using the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters, you can log in to the CDB to create and open the keystore, and then set the TDE master encryption key. After you complete these steps, you can begin to encrypt data.

4.3.2 Cryptographic Provider Prerequisite for TDE Configuration

Before configuring TDE, verify which cryptographic provider the database is using. The cryptographic provider determines which encryption algorithms are available for TDE operations and which FIPS modes can be enabled.

- Dell BSAFE Crypto-C Micro Edition Suite, or MES (legacy provider): The default cryptographic provider on Oracle Database 19c. Supports all TDE algorithms listed in this guide, including 3DES168 (deprecated), ARIA (128, 192, 256), SEED128, and GOST256.
- OpenSSL (next-generation provider): Available when the database is patched and configured to use the OpenSSL cryptographic library. Supports AES-based algorithms for

data encryption. Does not support 3DES168 in FIPS 140-3 mode. Does not support GOST or SEED for master encryption keys.

To determine the active cryptographic provider, query the `V$ENCRYPTION_WALLET` view or review the database alert log for provider initialization messages.

The choice of cryptographic provider does not affect the keystore configuration steps described in this chapter. The `WALLET_ROOT` and `TDE_CONFIGURATION` parameters function identically with both providers.

FIPS Mode Configuration

To enable FIPS mode with the next-generation provider, set the following parameters in the `fips.ora` file. The `fips.ora` file is located by default in `$ORACLE_HOME/ldap/admin`, or in a location specified by the `FIPS_HOME` environment variable:

- `FIPS_140=TRUE` enables FIPS mode.
- `FIPS_VERSION=FIPS140-3` (recommended) enables FIPS 140-3 mode. In FIPS 140-3 mode, only AES128, AES192, and AES256 are available for TDE operations.
- `FIPS_VERSION=FIPS140-2` (deprecated) explicitly selects FIPS 140-2 mode. In a future release, this value will be desupported and FIPS140-3 will become the default FIPS version.

Note

When FIPS mode is enabled, append the `-fips140_mode` flag to `orapki` commands.

For Oracle Database 19c without the next-generation provider, FIPS usage requires setting parameters in three separate locations:

- For TLS: Set `SSLFIPS_140=true` in the `fips.ora` file, located by default in `$ORACLE_HOME/ldap/admin`
- For NNE: Set `SQLNET.FIPS_140=TRUE` in the `sqlnet.ora` file, located by default in `$ORACLE_HOME/network/admin`
- For TDE and DBMS_CRYPT0: Set initialization parameter `DBFIPS_140=true`

4.3.3 Step 1: Configure the TDE Wallet Location and Type

You must configure the TDE wallet location and type by setting `WALLET_ROOT` in `init.ora` and `TDE_CONFIGURATION` in the database instance.

1. Log in to the server where the Oracle database resides.
2. If necessary, create a wallet directory.

Typically, the wallet directory is located in the `$ORACLE_BASE/admin/db_unique_name` directory, and it is named `wallet`. Preferably, this directory should be empty.

3. Log in to the database instance as a user who has been granted the SYSDBA administrative privilege.

For example:

```
sqlplus sec_admin as sysdba
Enter password: password
```

4. Set the `WALLET_ROOT` static initialization parameter for the wallet directory.

The TDE wallets will be created in `WALLET_ROOT/tde`. For example, for a database instance named `orcl`:

```
ALTER SYSTEM SET WALLET_ROOT = '$ORACLE_BASE/ADMIN/ORCL/WALLET' SCOPE = SPFILE SID = '*' ;
```

5. Set the `TDE_CONFIGURATION` parameter to specify the TDE wallet type.

For example:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KEYSTORE_CONFIGURATION=keystore_type" SCOPE=BOTH SID = '*' ;
```

In this specification, *keystore_type* must be set to `FILE` to configure a TDE wallet.

To configure a TDE keystore if the server parameter file (spfile) is in use, set scope to both:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KEYSTORE_CONFIGURATION=FILE" SCOPE = BOTH SID = '*' ;
```

4.3.4 Step 2: Create the TDE Wallet

After you have specified a directory location for the software keystore, you can create the TDE wallet.

- [About Creating TDE Wallets](#)
There are three different types of TDE wallets.
- [Creating a Password-Protected Software Keystore](#)
A password-protected software keystore requires a password, which is used to protect the TDE master keys.
- [Creating an Auto-Login or a Local Auto-Login Software Keystore](#)
As an alternative to password-protected keystores, you can create either an auto-login or local auto-login software keystore.

4.3.4.1 About Creating TDE Wallets

There are three different types of TDE wallets.

You can create password-protected TDE wallets, auto-login TDE wallets, and local auto-login TDE wallets.

Be aware that running the query `SELECT * FROM V$ENCRYPTION_WALLET` will automatically open an auto-login TDE wallet. For example, suppose you have a password-protected TDE wallet and an auto-login TDE wallet. If the password-protected TDE wallet is open and you close the password-protected TDE wallet and then query the `V$ENCRYPTION_WALLET` view, then the output will indicate that a TDE wallet is open. However, this is because `V$ENCRYPTION_WALLET` opened up the auto-login TDE wallet and then displayed the status of the auto-login wallet.

Related Topics

- [Types of Keystores](#)
Oracle Database supports TDE wallets, Oracle Key Vault, and Oracle Cloud Infrastructure (OCI) key management systems (KMS).

4.3.4.2 Creating a Password-Protected Software Keystore

A password-protected software keystore requires a password, which is used to protect the TDE master keys.

A TDE wallet can only contain TDE-related security objects, and no security objects used by other database or application components. In addition, do not use `orapki` or `mkstore` to create password-protected or (local) auto-open TDE wallets. Instead, use the `ADMINISTER KEY MANAGEMENT` statement.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Use the `SHOW PARAMETER` command to confirm that `WALLET_ROOT` is set, and `TDE_CONFIGURATION` is set to `KEYSTORE_CONFIGURATION=FILE`.
3. Run the `ADMINISTER KEY MANAGEMENT` SQL statement to create the keystore using the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY
software_keystore_password;
```

The `/tde` directory is automatically created under `WALLET_ROOT`, if it does not already exist. In Oracle Automatic Storage Management (Oracle ASM), for the `/tde` directory to be automatically created if it does not already exist, `WALLET_ROOT` must point to `+DATA/ORACLE_UNQNAME`.

In this specification, *software_keystore_password* is the password of the keystore that you, the security administrator, creates.

For example, to create the keystore in the `WALLET_ROOT/tde` directory:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY password;

keystore altered.
```

After you run this statement, the `ewallet.p12` file, which is the keystore, appears in the keystore location.

4.3.4.3 Creating an Auto-Login or a Local Auto-Login Software Keystore

As an alternative to password-protected keystores, you can create either an auto-login or local auto-login software keystore.

Both of these keystores have system-generated passwords. They are also PKCS#12-based files. The auto-login software keystore can be opened from different computers from the computer where this keystore resides, but the local auto-login software keystore can only be opened from the computer on which it was created. Both the auto-login and local auto-login keystores are created from the password-protected software keystores. Creating any of them does not require database downtime.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.

Confirm that `WALLET_ROOT` is set, and `TDE_CONFIGURATION` is set to `KEYSTORE_CONFIGURATION=FILE`.

2. Create the auto-login or local auto-login keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE [LOCAL] AUTO_LOGIN KEYSTORE  
FROM KEYSTORE IDENTIFIED BY software_keystore_password;
```

In this specification:

- LOCAL enables you to create a local auto-login software keystore. Otherwise, omit this clause if you want the keystore to be accessible by other computers. LOCAL creates a local auto-login wallet file, `cwallet.sso`, and this wallet will be tied to the host on which it was created. For an Oracle Real Application Clusters (Oracle RAC) environment, omit the LOCAL keyword, because each Oracle RAC node has a different host name. If you configure a local auto-login wallet for the Oracle RAC instance, then only the first Oracle RAC node, where the `cwallet.sso` file was created, would be able to access the software keystore. If you try to open the keystore from another node instead of from that first node, there would be a problem auto-opening `cwallet.sso`, and so it would result in a failure to auto-open the software keystore. This restriction applies if you are using a shared location to hold the `cwallet.sso` file for the Oracle RAC cluster, because using LOCAL only works if you have a separate `cwallet.sso` file (containing the same credentials) on each node of the Oracle RAC environment.
- *software_keystore_password* is the password of the keystore from which you want to create.

For example, to create an auto-login software keystore of the password-protected keystore that is located in the `/etc/ORACLE/WALLETS/tde` directory:

```
ADMINISTER KEY MANAGEMENT CREATE AUTO_LOGIN KEYSTORE  
FROM KEYSTORE IDENTIFIED BY password;
```

keystore altered.

After you run this statement, the `cwallet.sso` file appears in the keystore location. The `ewallet.p12` file is the password-protected wallet.

Follow these guidelines:

- Do not remove the PKCS#12 wallet (`ewallet.p12` file) after you create the auto-login keystore (`.sso` file). You must have the PKCS#12 wallet to regenerate or rekey the TDE master encryption key in the future.
- Remember that Transparent Data Encryption uses an auto login keystore only if it is available at the correct location (`WALLET_ROOT/tde`), `ENCRYPTION_WALLET_LOCATION`, or the default keystore location), and the SQL statement to open an encrypted keystore has not already been executed. If you have the `ENCRYPTION_WALLET_LOCATION` parameter set, then be aware this parameter is deprecated. Oracle recommends that you use the `WALLET_ROOT` static initialization parameter and `TDE_CONFIGURATION` dynamic initialization parameter instead.

Related Topics

- [Creating a Password-Protected Software Keystore](#)
A password-protected software keystore requires a password, which is used to protect the TDE master keys.

4.3.5 Step 3: Open the TDE Wallet

Depending on the type of keystore you create, you must manually open the TDE wallet before you can use it.

- [About Opening TDE Wallets](#)
A password-protected TDE wallet must be open before any TDE master encryption keys can be created or accessed in the TDE wallet.
- [Opening a TDE Wallet](#)
To open a TDE wallet, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.

4.3.5.1 About Opening TDE Wallets

A password-protected TDE wallet must be open before any TDE master encryption keys can be created or accessed in the TDE wallet.

Many Transparent Data Encryption operations require the TDE wallet to be open. There are two ways that you can open the TDE wallet:

- Manually open the TDE wallet by issuing the `ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN` statement. Afterward, you can perform the operation.
- Include the `FORCE KEYSTORE` clause in the `ADMINISTER KEY MANAGEMENT` statement that is used to perform the operation. `FORCE KEYSTORE` temporarily opens the TDE wallet for the duration of the operation, and when the operation completes, the TDE wallet is closed again. `FORCE KEYSTORE` is useful for situations when the database is heavily loaded. In this scenario, because of concurrent access to encrypted objects in the database, the auto-login TDE wallet continues to open immediately after it has been closed but before a user has had chance to open the password-based TDE wallet.

TDE wallets can be in the following states: open, closed, open but with no master encryption key, open but with an unknown master encryption key, undefined, or not available (that is, not present in the `WALLET_ROOT/tde` location).

After you manually open a TDE wallet, it remains open until you manually close it. Each time you restart a database instance, you must manually open the password TDE wallet to reenable encryption and decryption operations.

You can check the status of whether a TDE wallet is open or not by querying the `STATUS` column of the `V$ENCRYPTION_WALLET` view.

Related Topics

- [Performing Operations That Require a Keystore Password](#)
Many `ADMINISTER KEY MANAGEMENT` operations require access to a keystore password, for both TDE wallets and external keystores.
- [How Keystore Open and Close Operations Work in United Mode](#)
You should be aware of how keystore open and close operations work in united mode.

4.3.5.2 Opening a TDE Wallet

To open a TDE wallet, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Run the `ADMINISTER KEY MANAGEMENT` statement to open the TDE wallet.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
IDENTIFIED BY TDE_wallet_password;
keystore altered.
```

To switch over to opening the password-protected TDE wallet when an auto-login keystore is configured and is currently open, specify the `FORCE KEYSTORE` clause as follows.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
FORCE KEYSTORE  
IDENTIFIED BY EXTERNAL STORE;  
keystore altered.
```

Here, the `IDENTIFIED BY EXTERNAL STORE` clause is included in the statement because the TDE wallet credentials exist in an external store. This enables the password-protected TDE wallet to be opened without specifying the TDE wallet password within the statement itself.

If the `WALLET_ROOT` parameter has been set, then Oracle Database finds the external store by searching in this path: `WALLET_ROOT/PDB_GUID/tde_seps`.

3. Confirm that the TDE wallet is open.

```
SELECT STATUS FROM V$ENCRYPTION_WALLET;
```

Note that if the TDE wallet is open but you have not created a TDE master encryption key yet, the `STATUS` column of the `V$ENCRYPTION_WALLET` view reminds you with an `OPEN_NO_MASTER_KEY` status.

4.3.6 Step 4: Set the TDE Master Encryption Key in the TDE Wallet

Once the TDE wallet is open, you can set a TDE master encryption key for it.

- [About Setting the TDE Wallet TDE Master Encryption Key](#)
The TDE master encryption key is stored in the TDE wallet.
- [Setting the TDE Master Encryption Key in the TDE Wallet](#)
To set the TDE master encryption key in a TDE wallet, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause.

4.3.6.1 About Setting the TDE Wallet TDE Master Encryption Key

The TDE master encryption key is stored in the TDE wallet.

The TDE master encryption key protects the TDE table keys and tablespace encryption keys. By default, the TDE master encryption key is a key that TDE generates. You can find if a TDE wallet has no TDE master encryption key set or an unknown TDE master encryption key by querying the `STATUS` column of the `V$ENCRYPTION_WALLET` view.

You can import a master encryption key (bring your own key (BYOK)), that was created outside of the database, into the TDE wallet. You can set a key for immediate use, using `ADMINISTER KEY MANAGEMENT SET`, or create a key for later use, by activating it with the `ADMINISTER KEY MANAGEMENT USE KEY` statement.

Related Topics

- [Creating User-Defined TDE Master Encryption Keys](#)
You can create a user-defined TDE master encryption key outside the database by generating a TDE master encryption key ID.
- [Creating TDE Master Encryption Keys for Later Use](#)
You can create a TDE master encryption key that can be activated at a later date.

4.3.6.2 Setting the TDE Master Encryption Key in the TDE Wallet

To set the TDE master encryption key in a TDE wallet, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Ensure that the database is open in `READ WRITE` mode.

You can set the TDE master encryption key if `OPEN_MODE` is set to `READ WRITE`. To find the status, query the `OPEN_MODE` column of the `V$DATABASE` dynamic view. (If you cannot access this view, then connect as `SYSDBA` and try the query again. In order to connect as `SYSKM` for this type of query, you must create a password file for it.)

3. Set the TDE master encryption key in the software by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEY
[USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY EXTERNAL STORE | software_keystore_password
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- `tag` is the associated attributes and information that you define. Enclose this setting in single quotation marks (' ').
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `FORCE KEYSTORE` should be included if the keystore is closed. This automatically opens the keystore before setting the TDE master encryption key. The `FORCE KEYSTORE` clause also switches over to opening the password-protected TDE wallet when an auto-login TDE wallet is configured and is currently open.
- `IDENTIFIED BY` specifies the TDE wallet password. Alternatively, if the TDE wallet password is in an external store, you can use the `IDENTIFIED BY EXTERNAL STORE` clause.
- `WITH BACKUP` creates a backup of the TDE wallet. You must use this option for password-protected TDE wallets. Optionally, you can use the `USING` clause to add a brief description of the backup. Enclose this description in single quotation marks (' '). This identifier is appended to the named TDE wallet file (for example, `ewallet_time_stamp_emp_key_backup.p12`, with `emp_key_backup` being the backup identifier). Follow the file naming conventions that your operating system uses.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE | TDE_wallet_password
WITH BACKUP USING 'emp_key_backup';
```

4.3.7 Step 5: Encrypt Your Data

Now that you have completed the configuration, you can begin to encrypt data in the PDB.

You can encrypt data in individual table columns or in entire tablespaces or databases.

Related Topics

- [Encrypting Columns in Tables](#)
You can use Transparent Data Encryption to encrypt individual columns in database tables.
- [Encryption Conversions for Tablespaces and Databases](#)
You can perform encryption operations on both offline and online tablespaces and databases.

4.4 Configuring an External Keystore

There are two types of external keystores that Oracle Database supports: Oracle Key Vault keystores and OCI Vault - Key Management keystores.

- [About Configuring an External Keystore](#)
An external keystore is a separate server or device that provides security storage for encryption keys.
- [Step 1: Configure the External Keystore](#)
You can configure the external keystore by setting the TDE_CONFIGURATION parameter.
- [Step 2: Open the Connection to Oracle Key Vault](#)
After you have configured the database to use Oracle Key Vault for TDE key management, you must open the connection to Oracle Key Vault before you can use it.
- [Step 3: Set the TDE Master Encryption Key in Oracle Key Vault](#)
After you have opened the connection to Oracle Key Vault, you are ready to set the TDE master encryption key.
- [Step 4: Encrypt Your Data in United Mode](#)
Now that you have completed the configuration for an external keystore or for an Oracle Key Vault keystore, you can begin to encrypt data.

4.4.1 About Configuring an External Keystore

An external keystore is a separate server or device that provides security storage for encryption keys.

External keystores are external to an Oracle database. Oracle Database can interface with external keystores but cannot manipulate them outside of the Oracle interface. The Oracle database can request the external keystore to create a key but it cannot define how this key is stored in an external database. (Conversely, for software keystores that are created using TDE, Oracle Database has full control: that is, you can use SQL statements to manipulate this type of keystore.) Examples of external keystores are Oracle Key Vault keystores. Supported external keystores are Oracle Key Vault and the Oracle Cloud Infrastructure Vault.

To configure an external keystore, you must first include the keystore type in the `sqlnet.ora` file, open the external keystore, and then set the external keystore TDE master encryption key.

For Oracle Key Vault:

```
IDENTIFIED BY "Oracle_Key_Vault_password"
```

After you configure the external keystore, you are ready to begin encrypting your data.

4.4.2 Step 1: Configure the External Keystore

You can configure the external keystore by setting the TDE_CONFIGURATION parameter.

1. If the `WALLET_ROOT` parameter is set, then install the Oracle Key Vault client software into the `WALLET_ROOT/okv` directory.
2. Log in to the database instance as a user who has been granted the `ALTER SYSTEM` administrative privilege.
3. Set the `TDE_CONFIGURATION` dynamic initialization parameter to specify the keystore type by using the following syntax:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=keystore_type" SCOPE=BOTH  
SID = '*';
```

In this specification:

- *keystore_type* can be one of the following types:
 - `OKV` to configure an Oracle Key Vault keystore
 - `HSM` to configure a hardware security module (HSM) keystore

For example, to configure your database to use Oracle Key Vault:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=OKV"  
SCOPE=BOTH SID = '*';
```

4.4.3 Step 2: Open the Connection to Oracle Key Vault

After you have configured the database to use Oracle Key Vault for TDE key management, you must open the connection to Oracle Key Vault before you can use it.

- [About Opening the Connection to Oracle Key Vault](#)
You must open the connection to Oracle Key Vault so that it is accessible to the database before you can perform any encryption or decryption.
- [Opening an External Keystore](#)
To open an external keystore, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.

4.4.3.1 About Opening the Connection to Oracle Key Vault

You must open the connection to Oracle Key Vault so that it is accessible to the database before you can perform any encryption or decryption.

If a recovery operation is needed on your database (for example, if the database was not cleanly shut down, and has an encrypted tablespace that needs recovery), then you must open the connection to Oracle Key Vault before you can open the database itself.

There are two ways that you can open the Oracle Key Vault connection:

- Manually open the keystore by issuing the `ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN` statement. Afterward, you can perform the operation.
- Include the `FORCE KEYSTORE` clause in the `ADMINISTER KEY MANAGEMENT` statement. `FORCE KEYSTORE` temporarily opens the keystore for the duration of the operation, and when the operation completes, the keystore is closed again. `FORCE KEYSTORE` is useful for situations when the database is heavily loaded. In this scenario, because of concurrent access to encrypted objects in the database, the auto-login keystore continues to open immediately after it has been closed but before a user has had a chance to open the password-based keystore.

To check the status of the keystore, query the STATUS column of the V\$ENCRYPTION_WALLET view. Keystores can be in the following states: CLOSED, NOT_AVAILABLE (that is, not present in the WALLET_ROOT location), OPEN, OPEN_NO_MASTER_KEY, OPEN_UNKNOWN_MASTER_KEY_STATUS.

Be aware that for external keystores, if the database is in the mounted state, then it cannot check if the master key is set because the data dictionary is not available. In this situation, the status will be OPEN_UNKNOWN_MASTER_KEY_STATUS.

Related Topics

- [How Keystore Open and Close Operations Work in United Mode](#)
You should be aware of how keystore open and close operations work in united mode.

4.4.3.2 Opening an External Keystore

To open an external keystore, use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE OPEN clause.

1. Log in to the database instance as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Open the external keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
[FORCE KEYSTORE]  
IDENTIFIED BY EXTERNAL STORE | "external_key_manager_password"  
[CONTAINER = ALL | CURRENT];
```

In this specification:

- FORCE KEYSTORE enables the keystore operation if the keystore is closed.
- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.
 - *external_key_manager_password* is for an external keystore manager, which can be Oracle Key Vault or OCI Vault - Key Management. Enclose this password in double quotation marks. For Oracle Key Vault, enter the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

For Oracle Key Vault in a multitenant environment, to open the connection into Oracle KeyVault for the CDB\$ROOT and all open PDBs:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN [FORCE KEYSTORE]  
IDENTIFIED BY Oracle_Key_Vault_password  
CONTAINER = ALL;
```

3. Repeat this procedure each time you restart the database instance.

4.4.4 Step 3: Set the TDE Master Encryption Key in Oracle Key Vault

After you have opened the connection to Oracle Key Vault, you are ready to set the TDE master encryption key.

- [About Setting the External Keystore TDE Master Encryption Key](#)
You must create a TDE master encryption key that is stored inside the external keystore.

- [Setting a New TDE Master Encryption Key](#)
You should complete this procedure if you have not previously configured an external keystore for Transparent Data Encryption.
- [Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS](#)
To switch from a TDE wallet to centralized key management with Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS), after you upload all current and retired TDE master keys you must migrate the database from the TDE wallet to Oracle Key Vault or OCI KMS.

4.4.4.1 About Setting the External Keystore TDE Master Encryption Key

You must create a TDE master encryption key that is stored inside the external keystore.

Oracle Database uses the master encryption key from Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS) to encrypt or decrypt TDE table keys or tablespace encryption keys (data encryption keys) inside the database.

If you have not previously configured TDE with a wallet, then you must set the master encryption key in Oracle Key Vault or OCI KMS. If you have already configured TDE with a wallet, then you must migrate the database to Oracle Key Vault or OCI KMS.

Along with the current master encryption key, all TDE keystores (TDE wallet, Oracle Key Vault, and OCI KMS) maintain historical master encryption keys that are generated after every re-key operation that rekeys the master encryption key. These historical master keys help to restore Oracle Database backups that were taken previously using one of the master encryption keys. Only Oracle Key Vault allows you to upload all historical key from a TDE wallet before migrating the database to use Oracle Key Vault. After migration to Oracle Key Vault, the TDE wallet can be deleted, which satisfies security regulation that mandate that encryption keys cannot reside on the encrypting server.

4.4.4.2 Setting a New TDE Master Encryption Key

You should complete this procedure if you have not previously configured an external keystore for Transparent Data Encryption.

1. Log in to the database instance as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Ensure that the database is open in READ WRITE mode.

You can set the master encryption key if OPEN_MODE is set to READ WRITE. To find the status, for a non-multitenant environment, query the OPEN_MODE column of the V\$DATABASE dynamic view. If you are in a multitenant environment, then run the show_pdb command.

3. Set the new master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEY  
[USING TAG 'tag'] [FORCE KEYSTORE]  
IDENTIFIED BY [EXTERNAL STORE | "external_key_manager_password"];
```

In this specification:

- *tag* is the associated attributes and information that you define. Enclose this setting in single quotation marks (' ').
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- IDENTIFIED BY can be one of the following settings:

- EXTERNAL_STORE uses the keystore password stored in the external store to perform the keystore operation.
- `external_key_manager_password` is for an external keystore manager, which can be Oracle Key Vault or OCI Vault - Key Management. Enclose this password in double quotation marks. For Oracle Key Vault, enter the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY  
FORCE KEYSTORE  
IDENTIFIED BY "external_key_manager_password";
```

keystore altered.

Related Topics

- [Creating a TDE Master Encryption Key for Later Use](#)
A keystore must be opened before you can create a TDE master encryption key for use later on.
- *Oracle Database Administrator's Guide*

4.4.4.3 Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS

To switch from a TDE wallet to centralized key management with Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS), after you upload all current and retired TDE master keys you must migrate the database from the TDE wallet to Oracle Key Vault or OCI KMS.

Tools such as Oracle Data Pump and Oracle Recovery Manager require access to the old TDE wallet to perform decryption and encryption operations on data exported or backed up using the TDE wallet. Along with the current master encryption key, Oracle keystores maintain historical master encryption keys that are generated after every re-key operation that rotates the master encryption key. These historical master encryption keys help to restore Oracle database backups that were taken previously using one of the historical master encryption keys.

Related Topics

- [Migrating from a TDE Wallet to Oracle Key Vault](#)
You can migrate between password-protected TDE wallets and external keystores in Oracle Key Vault.

4.4.5 Step 4: Encrypt Your Data in United Mode

Now that you have completed the configuration for an external keystore or for an Oracle Key Vault keystore, you can begin to encrypt data.

Related Topics

- [Encrypting Columns in Tables](#)
You can use Transparent Data Encryption to encrypt individual columns in database tables.

- [Encryption Conversions for Tablespaces and Databases](#)
You can perform encryption operations on both offline and online tablespaces and databases.
- *Oracle Key Vault Administrator's Guide*

4.5 Encrypting Columns in Tables

You can use Transparent Data Encryption to encrypt individual columns in database tables.

- [About Encrypting Columns in Tables](#)
You can encrypt individual columns in tables.
- [Data Types That Can Be Encrypted with TDE Column Encryption](#)
Oracle Database supports a specific set of data types that can be used with TDE column encryption.
- [Restrictions on Using TDE Column Encryption](#)
TDE column encryption is performed at the SQL layer. Oracle Database utilities that bypass the SQL layer cannot use TDE column encryption services.
- [Creating Tables with Encrypted Columns](#)
Oracle Database provides a selection of different algorithms that you can use to define the encryption used in encrypted columns.
- [Encrypting Columns in Existing Tables](#)
You can encrypt columns in existing tables. As with new tables, you have a choice of different algorithms to use to define the encryption.
- [Creating an Index on an Encrypted Column](#)
You can create an index on an encrypted column.
- [Adding Salt to an Encrypted Column](#)
Salt, which is a random string added to data before encryption, is a way to strengthen the security of encrypted data. .
- [Removing Salt from an Encrypted Column](#)
You can use the ALTER TABLE SQL statement to remove salt from an encrypted column.
- [Changing the Encryption Key or Algorithm for Tables with Encrypted Columns](#)
You can use the ALTER TABLE SQL statement to change the encryption key or algorithm used in encrypted columns.

4.5.1 About Encrypting Columns in Tables

You can encrypt individual columns in tables.

Whether you choose to encrypt individual columns or entire tablespaces depends on the data types that the table has. There are also several features that do not support TDE column encryption.

Related Topics

- [Data Types That Can Be Encrypted with TDE Column Encryption](#)
Oracle Database supports a specific set of data types that can be used with TDE column encryption.
- [Restrictions on Using TDE Column Encryption](#)
TDE column encryption is performed at the SQL layer. Oracle Database utilities that bypass the SQL layer cannot use TDE column encryption services.

4.5.2 Data Types That Can Be Encrypted with TDE Column Encryption

Oracle Database supports a specific set of data types that can be used with TDE column encryption.

You can encrypt data columns that use a variety of different data types.

Supported data types are as follows:

- BINARY_DOUBLE
- BINARY_FLOAT
- CHAR
- DATE
- INTERVAL DAY TO SECOND
- INTERVAL YEAR TO MONTH
- NCHAR
- NUMBER
- NVARCHAR2
- RAW (legacy or extended)
- TIMESTAMP (includes TIMESTAMP WITH TIME ZONE and TIMESTAMP WITH LOCAL TIME ZONE)
- VARCHAR2 (legacy or extended)

If you want to encrypt large binary objects (LOBs), then you can use Oracle SecureFiles. Oracle SecureFiles enables you to store LOB data securely. To encrypt a LOB using SecureFiles, you use the CREATE TABLE or ALTER TABLE statements.

You cannot encrypt a column if the encrypted column size is greater than the size allowed by the data type of the column.

[Table 4-1](#) shows the maximum allowable sizes for various data types.

Table 4-1 Maximum Allowable Size for Data Types

Data Type	Maximum Size
CHAR	1932 bytes
VARCHAR2 (legacy)	3932 bytes
VARCHAR2 (extended)	32,699 bytes
NVARCHAR2 (legacy)	1966 bytes
NVARCHAR2 (extended)	16,315 bytes
NCHAR	966 bytes
RAW (extended)	32,699 bytes

Note

TDE tablespace encryption does not have these data type restrictions.

Related Topics

- [Restrictions on Using Transparent Data Encryption Tablespace Encryption](#)
You should be aware of restrictions on using Transparent Data Encryption when you encrypt a tablespace.
- *Oracle Database SecureFiles and Large Objects Developer's Guide*

4.5.3 Restrictions on Using TDE Column Encryption

TDE column encryption is performed at the SQL layer. Oracle Database utilities that bypass the SQL layer cannot use TDE column encryption services.

Do not use TDE column encryption with the following database features:

- Index types other than B-tree
- Range scan search through an index
- Synchronous change data capture
- Transportable tablespaces
- Columns that have been created as identity columns

In addition, you cannot use TDE column encryption to encrypt columns used in foreign key constraints.

Applications that must use these unsupported features can use the DBMS_CRYPTO PL/SQL package for their encryption needs.

Transparent Data Encryption protects data stored on a disk or other media. It does not protect data in transit. Use the network encryption solutions discussed in *Oracle Database Security Guide* to encrypt data over the network.

Related Topics

- [How Transparent Data Encryption Works with Export and Import Operations](#)
Oracle Data Pump can export and import tables that contain encrypted columns, as well as encrypt entire dump sets.
- [Data Types That Can Be Encrypted with TDE Column Encryption](#)
Oracle Database supports a specific set of data types that can be used with TDE column encryption.

4.5.4 Creating Tables with Encrypted Columns

Oracle Database provides a selection of different algorithms that you can use to define the encryption used in encrypted columns.

- [About Creating Tables with Encrypted Columns](#)
You can use the CREATE TABLE SQL statement to create a table with an encrypted column.
- [Creating a Table with an Encrypted Column Using the Default Algorithm](#)
By default, TDE uses the AES encryption algorithm with a 192-bit key length (AES192).
- [Creating a Table with an Encrypted Column Using No Algorithm or a Non-Default Algorithm](#)
You can use the CREATE TABLE SQL statement to create a table with an encrypted column.
- [Using the NOMAC Parameter to Save Disk Space and Improve Performance](#)
You can bypass checks that TDE performs. This can save up to 20 bytes of disk space per encrypted value.

- [Example: Using the NOMAC Parameter in a CREATE TABLE Statement](#)
You can use the CREATE TABLE SQL statement to encrypt a table column using the NOMAC parameter.
- [Example: Changing the Integrity Algorithm for a Table](#)
You can use the ALTER TABLE SQL statement in different foregrounds to convert different offline tablespaces in parallel.
- [Creating an Encrypted Column in an External Table](#)
The external table feature enables you to access data in external sources as if the data were in a database table.

4.5.4.1 About Creating Tables with Encrypted Columns

You can use the CREATE TABLE SQL statement to create a table with an encrypted column.

To create relational tables with encrypted columns, you can specify the SQL ENCRYPT clause when you define database columns with the CREATE TABLE SQL statement.

4.5.4.2 Creating a Table with an Encrypted Column Using the Default Algorithm

By default, TDE uses the AES encryption algorithm with a 192-bit key length (AES192).

If you encrypt a table column without specifying an algorithm, then the column is encrypted using the AES192 algorithm.

TDE adds [salt](#) to plaintext before encrypting it. Adding salt makes it harder for attackers to steal data through a brute force attack. TDE also adds a Message Authentication Code (MAC) to the data for integrity checking. The SHA-1 integrity algorithm is used by default.

- To create a table that encrypts a column, use the CREATE TABLE SQL statement with the ENCRYPT clause.

For example, to encrypt a table column using the default algorithm:

```
CREATE TABLE employee (  
    first_name VARCHAR2(128),  
    last_name VARCHAR2(128),  
    empID NUMBER,  
    salary NUMBER(6) ENCRYPT);
```

This example creates a new table with an encrypted column (salary). The column is encrypted using the default encryption algorithm (AES192). Salt and MAC are added by default. This example assumes that the keystore is open and a master encryption key is set.

Note

If there are multiple encrypted columns in a table, then all of these columns must use the same pair of encryption and integrity algorithms.

Salt is specified at the column level. This means that an encrypted column in a table can choose not to use salt irrespective of whether or not other encrypted columns in the table use salt.

4.5.4.3 Creating a Table with an Encrypted Column Using No Algorithm or a Non-Default Algorithm

You can use the CREATE TABLE SQL statement to create a table with an encrypted column.

By default, TDE adds [salt](#) to plaintext before encrypting it. Adding salt makes it harder for attackers to steal data through a brute force attack. However, if you plan to index the encrypted column, then you must use the NO SALT parameter.

- To create a table that uses an encrypted column that is a non-default algorithm or no algorithm, run the CREATE TABLE SQL statement as follows:
 - If you do not want to use any algorithm, then include the ENCRYPT NO SALT clause.
 - If you want to use a non-default algorithm, then use the ENCRYPT USING clause, followed by one of the following algorithms enclosed in single quotation marks:
 - * 3DES168
 - * AES128
 - * AES192 (default)
 - * AES256

The following example shows how to specify encryption settings for the empID and salary columns.

```
CREATE TABLE employee (  
    first_name VARCHAR2(128),  
    last_name VARCHAR2(128),  
    empID NUMBER ENCRYPT NO SALT,  
    salary NUMBER(6) ENCRYPT USING '3DES168');
```

In this example:

- The empID column is encrypted and does not use salt. Both the empID and salary columns will use the 3DES168 encryption algorithm, because all of the encrypted columns in a table must use the same encryption algorithm.
- The salary column is encrypted using the 3DES168 encryption algorithm. Note that the string that specifies the algorithm must be enclosed in single quotation marks (' '). The salary column uses salt by default.

Note

The 3DES168 algorithm (deprecated) is not available for column encryption when the database uses the next-generation provider in FIPS 140-3 mode. 3DES encryption is outdated, slow, and no longer considered secure. If you specify ENCRYPT 3DES168 in this configuration, the operation fails with an error.

Oracle recommends using AES256 for new encrypted columns. AES256 provides the strongest encryption among the supported AES key sizes, is compatible with all cryptographic provider and FIPS mode combinations, and all modes of symmetric encryption using AES256 are considered quantum resistant.

The default column encryption algorithm is AES192 in CBC mode. To encrypt a column with AES256:

```
ALTER TABLE schema.table MODIFY (column_name ENCRYPT USING 'AES256');
```

If existing columns are encrypted with 3DES168 and you plan to switch to the next-generation provider in FIPS 140-3 mode, you must re-encrypt those columns with an AES algorithm before the switch:

```
ALTER TABLE schema.table MODIFY (column_name ENCRYPT USING 'AES256');
```

4.5.4.4 Using the NOMAC Parameter to Save Disk Space and Improve Performance

You can bypass checks that TDE performs. This can save up to 20 bytes of disk space per encrypted value.

If the number of rows and encrypted columns in the table is large, then bypassing TDE checks can add up to a significant amount of disk space. In addition, this saves processing cycles and reduces the performance overhead associated with TDE.

TDE uses the SHA-1 integrity algorithm by default. All of the encrypted columns in a table must use the same integrity algorithm. If you already have a table column using the SHA-1 algorithm, then you cannot use the NOMAC parameter to encrypt another column in the same table.

- To bypass the integrity check during encryption and decryption operations, use the NOMAC parameter in the CREATE TABLE and ALTER TABLE statements.

Related Topics

- [Performance and Storage Overhead of Transparent Data Encryption](#)
The performance of Transparent Data Encryption can vary.

4.5.4.5 Example: Using the NOMAC Parameter in a CREATE TABLE Statement

You can use the CREATE TABLE SQL statement to encrypt a table column using the NOMAC parameter.

[Example 4-1](#) creates a table with an encrypted column. The empID column is encrypted using the NOMAC parameter.

Example 4-1 Using the NOMAC parameter in a CREATE TABLE statement

```
CREATE TABLE employee (  
    first_name VARCHAR2(128),  
    last_name VARCHAR2(128),  
    empID NUMBER ENCRYPT 'NOMAC' ,  
    salary NUMBER(6));
```

4.5.4.6 Example: Changing the Integrity Algorithm for a Table

You can use the ALTER TABLE SQL statement in different foregrounds to convert different offline tablespaces in parallel.

[Example 4-2](#) shows how to change the integrity algorithm for encrypted columns in a table. The encryption algorithm is set to 3DES168 and the integrity algorithm is set to SHA-1. The second ALTER TABLE statement sets the integrity algorithm to NOMAC.

Example 4-2 Changing the Integrity Algorithm for a Table

```
ALTER TABLE EMPLOYEE REKEY USING '3DES168' 'SHA-1';  
  
ALTER TABLE EMPLOYEE REKEY USING '3DES168' 'NOMAC';
```

Note

The 3DES168 algorithm (deprecated) is not available for column encryption when the database uses the next-generation provider in FIPS 140-3 mode. 3DES encryption is outdated, slow, and no longer considered secure. If you specify ENCRYPT 3DES168 in this configuration, the operation fails with an error.

Oracle recommends using AES256 for new encrypted columns. AES256 provides the strongest encryption among the supported AES key sizes, is compatible with all cryptographic provider and FIPS mode combinations, and all modes of symmetric encryption using AES256 are considered quantum resistant.

The default column encryption algorithm is AES192 in CBC mode. To encrypt a column with AES256:

```
ALTER TABLE schema.table MODIFY (column_name ENCRYPT USING 'AES256');
```

If existing columns are encrypted with 3DES168 and you plan to switch to the next-generation provider in FIPS 140-3 mode, you must re-encrypt those columns with an AES algorithm before the switch:

```
ALTER TABLE schema.table MODIFY (column_name ENCRYPT USING 'AES256');
```

4.5.4.7 Creating an Encrypted Column in an External Table

The external table feature enables you to access data in external sources as if the data were in a database table.

External tables can be updated using the ORACLE_DATAPUMP access driver.

To encrypt specific columns in an external table, use the ENCRYPT clause when you define those columns:

A system-generated key encrypts the columns. For example, the following CREATE TABLE SQL statement encrypts the ssn column using the 3DES168 algorithm:

```
CREATE TABLE emp_ext (  
    first_name,  
    ....  
    ssn ENCRYPT USING '3DES168',  
    ....
```

If you plan to move an external table to a new location, then you cannot use a randomly generated key to encrypt the columns. This is because the randomly generated key will not be available at the new location.

For such scenarios, you should specify a password while you encrypt the columns. After you move the data, you can use the same password to regenerate the key required to access the encrypted column data at the new location.

Table partition exchange also requires a password-protected [TDE table key](#).

The below example creates an external table using a password to create the [TDE table key](#).

Note

The 3DES168 algorithm (deprecated) is not available for column encryption when the database uses the next-generation provider in FIPS 140-3 mode. 3DES encryption is outdated, slow, and no longer considered secure. If you specify ENCRYPT 3DES168 in this configuration, the operation fails with an error.

Oracle recommends using AES256 for new encrypted columns. AES256 provides the strongest encryption among the supported AES key sizes, is compatible with all cryptographic provider and FIPS mode combinations, and all modes of symmetric encryption using AES256 are considered quantum resistant.

The default column encryption algorithm is AES192 in CBC mode. To encrypt a column with AES256:

```
ALTER TABLE schema.table MODIFY (column_name ENCRYPT USING 'AES256');
```

If existing columns are encrypted with 3DES168 and you plan to switch to the next-generation provider in FIPS 140-3 mode, you must re-encrypt those columns with an AES algorithm before the switch:

```
ALTER TABLE schema.table MODIFY (column_name ENCRYPT USING 'AES256');
```

Example 4-3 Creating a New External Table with a Password-Generated TDE Table Key

```
CREATE TABLE emp_ext (  
    first_name,  
    last_name,  
    empID,  
    salary,  
    ssn ENCRYPT IDENTIFIED BY password  
) ORGANIZATION EXTERNAL  
(  
    TYPE ORACLE_DATAPUMP  
    DEFAULT DIRECTORY "D_DIR"  
    LOCATION('emp_ext.dat')  
)  
REJECT LIMIT UNLIMITED  
AS SELECT * FROM EMPLOYEE;
```

4.5.5 Encrypting Columns in Existing Tables

You can encrypt columns in existing tables. As with new tables, you have a choice of different algorithms to use to definite the encryption.

- [About Encrypting Columns in Existing Tables](#)
The ALTER TABLE SQL statement enables you to encrypt columns in an existing table.
- [Adding an Encrypted Column to an Existing Table](#)
You can encrypt columns in existing tables, use a different algorithm, and use NO SALT to index the column.

- [Encrypting an Unencrypted Column](#)
You can use the ALTER TABLE MODIFY statement to encrypt an existing unencrypted column.
- [Disabling Encryption on a Column](#)
You may want to disable encryption for reasons of compatibility or performance.

4.5.5.1 About Encrypting Columns in Existing Tables

The ALTER TABLE SQL statement enables you to encrypt columns in an existing table.

To add an encrypted column to an existing table, or to encrypt or decrypt an existing column, you use the ALTER TABLE SQL statement with the ADD or MODIFY clause.

4.5.5.2 Adding an Encrypted Column to an Existing Table

You can encrypt columns in existing tables, use a different algorithm, and use NO SALT to index the column.

- To add an encrypted column to an existing table, use the ALTER TABLE ADD statement, specifying the new column with the ENCRYPT clause.

[Example 4-4](#) adds an encrypted column, ssn, to an existing table, called employee. The ssn column is encrypted with the default AES192 algorithm. Salt and MAC are added by default.

Example 4-4 Adding an Encrypted Column to an Existing Table

```
ALTER TABLE employee ADD (ssn VARCHAR2(11) ENCRYPT);
```

4.5.5.3 Encrypting an Unencrypted Column

You can use the ALTER TABLE MODIFY statement to encrypt an existing unencrypted column.

- To encrypt an existing unencrypted column, use the ALTER TABLE MODIFY statement, specifying the unencrypted column with the ENCRYPT clause.

The following example encrypts the first_name column in the employee table. The first_name column is encrypted with the default AES192 algorithm. Salt is added to the data, by default. You can encrypt the column using a different algorithm. If you want to index a column, then you must specify NO SALT. You can also bypass integrity checks by using the NOMAC parameter.

```
ALTER TABLE employee MODIFY (first_name ENCRYPT);
```

The following example encrypts the first_name column in the employee table using the NOMAC parameter.

```
ALTER TABLE employee MODIFY (first_name ENCRYPT 'NOMAC');
```

4.5.5.4 Disabling Encryption on a Column

You may want to disable encryption for reasons of compatibility or performance.

- To disable column encryption, use the ALTER TABLE MODIFY command with the DECRYPT clause.

[Example 4-5](#) decrypts the first_name column in the employee table.

Example 4-5 Turning Off Column Encryption

```
ALTER TABLE employee MODIFY (first_name DECRYPT);
```

4.5.6 Creating an Index on an Encrypted Column

You can create an index on an encrypted column.

The column being indexed must be encrypted without [salt](#). If the column is encrypted with salt, then the ORA-28338: cannot encrypt indexed column(s) with salt error is raised.

- To create an index on an encrypted column, use the CREATE INDEX statement with the ENCRYPT NO SALT clause.

The example below shows how to create an index on a column that has been encrypted without salt.

Example 4-6 Creating Index on a Column Encrypted Without Salt

```
CREATE TABLE employee (  
    first_name VARCHAR2(128),  
    last_name VARCHAR2(128),  
    empID NUMBER ENCRYPT NO SALT,  
    salary NUMBER(6) ENCRYPT USING 'AES256');  
  
CREATE INDEX employee_idx on employee (empID);
```

4.5.7 Adding Salt to an Encrypted Column

Salt, which is a random string added to data before encryption, is a way to strengthen the security of encrypted data. .

Salt ensures that the same plaintext data does not always translate to the same encrypted text. Salt removes the one common method that intruders use to steal data, namely, matching patterns of encrypted text. Adding [salt](#) requires an additional 16 bytes of storage per encrypted data value.

- To add or remove salt from encrypted columns, use the ALTER TABLE MODIFY SQL statement.

For example, suppose you want to encrypt the first_name column using salt. If the first_name column was encrypted without salt earlier, then the ALTER TABLE MODIFY statement reencrypts it using salt.

```
ALTER TABLE employee MODIFY (first_name ENCRYPT SALT);
```

4.5.8 Removing Salt from an Encrypted Column

You can use the ALTER TABLE SQL statement to remove salt from an encrypted column.

- To remove salt from an encrypted column, use the ENCRYPT NO SALT clause in the ALTER TABLE SQL statement.

For example, suppose you wanted to remove salt from the first_name column. If you must index a column that was encrypted using salt, then you can use this statement to remove the salt before indexing

```
ALTER TABLE employee MODIFY (first_name ENCRYPT NO SALT);
```

4.5.9 Changing the Encryption Key or Algorithm for Tables with Encrypted Columns

You can use the ALTER TABLE SQL statement to change the encryption key or algorithm used in encrypted columns.

Each table can have only one [TDE table key](#) for its columns. You can regenerate the TDE table key with the ALTER TABLE statement. This process generates a new key, decrypts the data in the table using the previous key, reencrypts the data using the new key, and then updates the table metadata with the new key information. You can also use a different encryption algorithm for the new TDE table key.

- To change the encryption key or algorithm for tables that contain encrypted columns, use the ALTER TABLE SQL statement with the REKEY or REKEY USING clause.

For example:

```
ALTER TABLE employee REKEY;
```

The example below regenerates the TDE table key for the employee table by using the AES256 algorithm.

Example 4-7 Changing an Encrypted Table Column Encryption Key and Algorithm

```
ALTER TABLE employee REKEY USING 'AES256';
```

4.6 Encryption Conversions for Tablespaces and Databases

You can perform encryption operations on both offline and online tablespaces and databases.

- [About Encryption Conversion for Tablespaces and Databases](#)
The CREATE TABLESPACE SQL statement can be used to encrypt new tablespaces. ALTER TABLESPACE can encrypt existing tablespaces.
- [Cryptographic Provider Considerations for Tablespace Encryption](#)
The default tablespace encryption algorithm on Oracle Database 19c is AES128 using CFB mode. This default applies regardless of the cryptographic provider in use. CFB is the only supported encryption mode for tablespace encryption on Oracle Database 19c.
- [Impact of a Closed TDE Keystore on Encrypted Tablespaces](#)
A TDE keystore can be closed or migrated when an Oracle-managed tablespace is encrypted, and the database system itself must be shut down to disallow operations on an Oracle-managed tablespace.
- [Restrictions on Using Transparent Data Encryption Tablespace Encryption](#)
You should be aware of restrictions on using Transparent Data Encryption when you encrypt a tablespace.
- [Creating an Encrypted New Tablespace](#)
When you create a new tablespace, you can configure its encryption settings during the creation process.
- [Setting the Tablespace Encryption Default Algorithm](#)
The TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM applies to specific encryption scenarios.
- [Encrypting Future Tablespaces](#)
You can configure Oracle Database to automatically encrypt future tablespaces that you will create.

- [Encrypted Sensitive Credential Data in the Data Dictionary](#)
You can encrypt sensitive credential data in the SYS.LINK\$ and SYS.SCHEDULER\$_CREDENTIAL system tables.
- [Encryption Conversions for Existing Offline Tablespaces](#)
You can perform offline encryption conversions by using the ALTER TABLESPACE SQL statement OFFLINE, ENCRYPT, and DECRYPT clauses.
- [Encryption Conversions for Existing Online Tablespaces](#)
You can encrypt and decrypt an online existing tablespace by using the ALTER TABLESPACE SQL statement with the ONLINE and ENCRYPT or DECRYPT clauses.
- [Encryption Conversions for Existing Databases](#)
You can encrypt both offline and online databases.

4.6.1 About Encryption Conversion for Tablespaces and Databases

The CREATE TABLESPACE SQL statement can be used to encrypt new tablespaces. ALTER TABLESPACE can encrypt existing tablespaces.

In addition to encrypting new and existing tablespaces, you can encrypt full databases, which entails the encryption of the [Oracle-managed tablespaces](#) (in this release, the SYSTEM, SYSAUX, TEMP, and UNDO tablespaces). To encrypt a full database, you use the ALTER TABLESPACE statement, not ALTER DATABASE, to encrypt the Oracle-managed tablespaces.

The following table compares the differences between an offline and an online encryption conversion of tablespaces and databases.

Table 4-2 Offline and Online Tablespace and Database Encryption Conversions

Functionality	Offline Conversion	Online Conversion
Release with minimum conversion capability	Oracle Database 11g release 2 (11.2)	Oracle Database 12c release 2 (12.2) and later
What can be backported?	The ability to encrypt or decrypt a data file with the AES128 algorithm (using ALTER DATABASE DATAFILE <i>data_file</i> ENCRYPT or DECRYPT) can be used in Oracle Database releases 12.1.0.2 and 11.2.0.4.	No
Algorithms supported	All symmetric encryption algorithms that TDE supports. See Supported Encryption and Integrity Algorithms . See also About Encryption Conversions for Existing Offline Tablespaces .	All symmetric encryption algorithm that TDE supports. See Supported Encryption and Integrity Algorithms . See also About Encryption Conversions for Existing Online Tablespaces .
When can the conversion be run?	When the tablespace is offline or the database is in the mount stage.	When the tablespace is online and database is open in read/write mode.
Is auxiliary space required for the conversion?	No	Yes. See Encrypting an Existing Tablespace with Online Conversion for guidelines.

Table 4-2 (Cont.) Offline and Online Tablespace and Database Encryption Conversions

Functionality	Offline Conversion	Online Conversion
Oracle Data Guard conversion guidelines	Convert both the primary and standby manually. Convert the standby first and then switch over to minimum downtime	After you convert the primary, the standby conversion takes place automatically. You cannot perform an online conversion directly on the standby.
Encrypt the SYSTEM, SYSAUX, and UNDO tablespaces (database conversion)	Oracle Database 12c release 2 (12.2) and later only. You must set COMPATIBILITY to 12.2.0.0. Only auto-login keystores are supported.	Oracle Database 12c release 2 (12.2) and later only. You must set COMPATIBILITY to 12.2.0.0.
Can an existing TEMP tablespace be converted?	No, but you can create an encrypted TEMP tablespace in Oracle Database 12c release 2 (12.2) and later, make it the default temporary tablespace, and then drop the original TEMP tablespace.	No, but you can create an encrypted TEMP tablespace in Oracle Database 12c release 2 (12.2) and later, make it the default temporary tablespace, and then drop the original TEMP tablespace.
Can an existing tablespace be decrypted?	You only can decrypt a tablespace or data file that was previously encrypted by an offline encrypt operation. Oracle does not recommend that you decrypt the UNDO tablespace once it is encrypted.	Yes, but Oracle does not recommend that you decrypt the UNDO tablespace once it is encrypted.
Can encryption keys be rekeyed?	No, but after the tablespace is encrypted, you can then use online conversion to rekey in Oracle Database 12c release 2 (12.2) compatibility.	Yes
Can encryption operations be run in parallel?	You can run parallel encryption conversions at the data file level with multiple user sessions running.	You can run parallel encryption conversions at the tablespace level with multiple user sessions running.
What to do if an encryption conversion SQL statement fails to complete?	Re-issue the encryption or decryption SQL statement to ensure that all the data files within the tablespace are consistently either encrypted or decrypted.	Rerun the SQL statement but use the FINISH clause.

4.6.2 Cryptographic Provider Considerations for Tablespace Encryption

The default tablespace encryption algorithm on Oracle Database 19c is AES128 using CFB mode. This default applies regardless of the cryptographic provider in use. CFB is the only supported encryption mode for tablespace encryption on Oracle Database 19c.

Note

AES-XTS mode is not available for tablespace encryption on Oracle Database 19c. XTS mode is available starting with Oracle AI Database 26ai.

Oracle recommends AES256 for tablespace encryption. To change the default tablespace encryption algorithm, set the hidden initialization parameter:

```
ALTER SYSTEM SET "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM"='AES256'  
SCOPE=BOTH;
```

The following considerations apply when using the next-generation provider:

- 3DES168 (deprecated) is desupported for tablespace encryption when the next-generation provider operates in FIPS 140-3 mode. 3DES encryption is outdated, slow, and no longer considered secure. If you specify `ENCRYPTION USING 3DES168` in a `CREATE TABLESPACE` or `ALTER TABLESPACE` statement while operating in FIPS 140-3 mode, the operation fails with an error.
- If existing tablespaces are encrypted with 3DES168, you must perform an online or offline conversion to an AES algorithm before enabling FIPS 140-3 mode with the next-generation provider.

To convert a tablespace from 3DES168 to AES256:

```
ALTER TABLESPACE tablespace_name ENCRYPTION ONLINE USING 'AES256' ENCRYPT;
```

All modes of symmetric encryption using AES256 are considered quantum resistant.

4.6.3 Impact of a Closed TDE Keystore on Encrypted Tablespaces

A TDE keystore can be closed or migrated when an Oracle-managed tablespace is encrypted, and the database system itself must be shut down to disallow operations on an Oracle-managed tablespace.

A closed TDE keystore has no impact on operations that involve an encrypted [Oracle-managed tablespace](#) (in this release, the `SYSTEM`, `SYSAUX`, `TEMP`, and `UNDO` tablespaces). This enables operations that are performed by background processes (for example, the log writer) to continue to work on these tablespaces while the TDE keystore is closed. If you want to disallow operations on an encrypted Oracle-managed tablespace, then you must shut down the database.

With regard to user-created tablespaces, a closed TDE keystore causes operations such as rotating a key or decrypting the tablespace to fail with an `ORA-28365 wallet is not open` error, just as it did in earlier releases. If you want to disallow operations on the user-created tablespace, then close the TDE keystore (or shut down the database).

User-created data can be copied into an encrypted Oracle-managed tablespace (for example, by an internal process such as `DBMS_STATS` statistics gathering) from a user-created tablespace while the TDE keystore is open. Closing the keystore does not prevent a user from viewing this data afterward, when the TDE keystore is in the `CLOSED` state at the time that you query the `V$ENCRYPTION_WALLET` view. Access to the original data by attempting to query an encrypted user-created tablespace will fail, resulting in an `ORA-28365 wallet is not open` error.

[Table 4-3](#) describes the operations that are necessary to disallow or allow operations on encrypted data in user-created tablespaces and Oracle-managed tablespaces. For example, in the first scenario, both the user-created tablespaces and the Oracle-managed tablespaces are encrypted. In this case, for the encrypted data in the encrypted user-created tablespace, an administrator can close or open keystores, and shut down and open a database with an encrypted user-created tablespace. When an encrypted Oracle-managed tablespace is configured, the administrator can disallow operations by shutting down the database, and can allow operations by starting up in mount mode, opening the TDE keystore, and then opening

the database. (It is necessary to open the TDE keystore *before* opening the database because the system may need the TDE master encryption key to decrypt the bootstrap dictionary tables, which are located in the encrypted Oracle-managed tablespace.) The N/A flags in this table refer to non-encrypted data, which you can always operate on, unless the instance is shut down.

Table 4-3 Necessary Commands to Disallow or Allow Operations on Encrypted Data

Tablespace Encryption Scenarios	Commands to Disallow Operations on Encrypted User-Created Tablespace Data	Commands to Disallow Operations on Encrypted Oracle-Managed Tablespace Data	Commands to Allow Operations on Encrypted User-Created Tablespace Data	Commands to Allow Operations on Encrypted Oracle-Managed Tablespace Data
Both user-created and Oracle-managed tablespaces encrypted	- ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE IDENTIFIED BY <i>password</i>; - SHUTDOWN	SHUTDOWN	- STARTUP MOUNT; - ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY <i>password</i>; - ALTER DATABASE OPEN	- STARTUP MOUNT; - ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY <i>password</i>; - ALTER DATABASE OPEN
User tablespace encrypted; Oracle-managed tablespace not encrypted	- ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE IDENTIFIED BY <i>password</i>; - SHUTDOWN	N/A	- STARTUP MOUNT; - ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY <i>password</i>; - ALTER DATABASE OPEN	N/A
User tablespace not encrypted; Oracle-managed tablespace encrypted	N/A	SHUTDOWN	N/A	- STARTUP MOUNT; - ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY <i>password</i>; - ALTER DATABASE OPEN
Neither user nor Oracle-managed tablespaces encrypted	N/A	N/A	N/A	N/A

4.6.4 Restrictions on Using Transparent Data Encryption Tablespace Encryption

You should be aware of restrictions on using Transparent Data Encryption when you encrypt a tablespace.

Note the following restrictions:

- Transparent Data Encryption (TDE) tablespace encryption encrypts or decrypts data during read and write operations, as opposed to TDE column encryption, which encrypts and decrypts data at the SQL layer. This means that most restrictions that apply to TDE column encryption, such as data type restrictions and index type restrictions, do not apply to TDE tablespace encryption.

- To perform import and export operations, use Oracle Data Pump.

Related Topics

- [Encrypting an Unencrypted Column](#)
You can use the ALTER TABLE MODIFY statement to encrypt an existing unencrypted column.
- [Data Types That Can Be Encrypted with TDE Column Encryption](#)
Oracle Database supports a specific set of data types that can be used with TDE column encryption.
- *Oracle Database Utilities*

4.6.5 Creating an Encrypted New Tablespace

When you create a new tablespace, you can configure its encryption settings during the creation process.

- [Step 1: Set the COMPATIBLE Initialization Parameter for Tablespace Encryption](#)
You must set the COMPATIBLE initialization parameter before creating an encrypted tablespace.
- [Step 2: Set the Tablespace TDE Master Encryption Key](#)
You should ensure that you have configured the TDE master encryption key.
- [Step 3: Create the Encrypted Tablespace](#)
After you have set the COMPATIBLE initialization parameter, you are ready to create the encrypted tablespace.

4.6.5.1 Step 1: Set the COMPATIBLE Initialization Parameter for Tablespace Encryption

You must set the COMPATIBLE initialization parameter before creating an encrypted tablespace.

- [About Setting the COMPATIBLE Initialization Parameter for Tablespace Encryption](#)
A minimum COMPATIBLE initialization parameter setting of 11.2.0.0 enables the full set of tablespace encryption features.
- [Setting the COMPATIBLE Initialization Parameter for Tablespace Encryption](#)
To set the COMPATIBLE initialization parameter, you must edit the initialization parameter file for the database instance.

4.6.5.1.1 About Setting the COMPATIBLE Initialization Parameter for Tablespace Encryption

A minimum COMPATIBLE initialization parameter setting of 11.2.0.0 enables the full set of tablespace encryption features.

Setting the compatibility to 11.2.0.0 enables the following functionality:

- The 11.2.0.0 setting enables the database to use any of the four supported algorithms for data encryption (3DES168, AES128, AES192, and AES256).
- The 11.2.0.0 setting enables the migration of a key from a TDE wallet to an external keystore (ensure that the TDE master encryption key was configured for the external keystore)
- The 11.2.0.0 setting enables rekeying the TDE master encryption key

Be aware that once you set the COMPATIBLE parameter to 11.2.0.0, the change is irreversible. To use tablespace encryption, ensure that the compatibility setting is at the minimum, which is 11.2.0.0.

Related Topics

- *Oracle Database SQL Language Reference*
- *Oracle Database Administrator's Guide*

4.6.5.1.2 Setting the COMPATIBLE Initialization Parameter for Tablespace Encryption

To set the COMPATIBLE initialization parameter, you must edit the initialization parameter file for the database instance.

1. Log in to the database instance.
2. Check the current setting of the COMPATIBLE parameter.

For example:

```
SHOW PARAMETER COMPATIBLE
```

NAME	TYPE	VALUE

compatible	string	11.2.0.0
noncdbcompatible	BOOLEAN	FALSE

3. If you must change the COMPATIBLE parameter, then complete the remaining steps in this procedure.

The value should be 12.2.0.0 or later.

4. From the command line, locate the initialization parameter file for the database instance.
 - **UNIX systems:** This file is in the `ORACLE_HOME/dbs` directory and is named `initORACLE_SID.ora` (for example, `initmydb.ora`).
 - **Windows systems:** This file is in the `ORACLE_HOME\database` directory and is named `initORACLE_SID.ora` (for example, `initmydb.ora`).

5. Edit the initialization parameter file to use the new COMPATIBLE setting.

For example:

```
COMPATIBLE=12.2.0.0
```

6. In SQL*Plus, connect as a user who has the SYSDBA administrative privilege, and then restart the database.

For example:

```
CONNECT /AS SYSDBA
SHUTDOWN
STARTUP
```

If tablespace encryption is in use, then open the keystore at the database mount. The keystore must be open before you can access data in an encrypted tablespace.

For example:

```
STARTUP MOUNT;
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY keystore_password;
ALTER DATABASE OPEN;
```

4.6.5.2 Step 2: Set the Tablespace TDE Master Encryption Key

You should ensure that you have configured the TDE master encryption key.

- Set the TDE master encryption key as follows:
 - For software TDE master encryption keys, see [Step 4: Set the TDE Master Encryption Key in the TDE Wallet](#).
 - For hardware TDE master encryption keys, see [Step 3: Set the TDE Master Encryption Key in Oracle Key Vault](#).

4.6.5.3 Step 3: Create the Encrypted Tablespace

After you have set the COMPATIBLE initialization parameter, you are ready to create the encrypted tablespace.

- [About Creating Encrypted Tablespaces](#)
To create an encrypted tablespace, you can use the CREATE TABLESPACE SQL statement.
- [Creating an Encrypted Tablespace](#)
To create an encrypted tablespace, you must use the CREATE TABLESPACE statement with the ENCRYPTION USING clause.
- [Example: Creating an Encrypted Tablespace That Uses AES192](#)
You can use the CREATE TABLESPACE SQL statement to create an encrypted tablespace.
- [Example: Creating an Encrypted Tablespace That Uses the Default Algorithm](#)
You can use the CREATE TABLESPACE SQL statement to create an encrypted tablespace that uses the default algorithm.

4.6.5.3.1 About Creating Encrypted Tablespaces

To create an encrypted tablespace, you can use the CREATE TABLESPACE SQL statement.

You must have the CREATE TABLESPACE system privilege to create an encrypted tablespace.

You can import data into an encrypted tablespace by using Oracle Data Pump. You can also use a SQL statement such as CREATE TABLE...AS SELECT... or ALTER TABLE...MOVE... to move data into an encrypted tablespace. The CREATE TABLE...AS SELECT... statement creates a table from an existing table. The ALTER TABLE...MOVE... statement moves a table into the encrypted tablespace.

For security reasons, you cannot encrypt a tablespace with the NO SALT option.

You can query the ENCRYPTED column of the DBA_TABLESPACES and USER_TABLESPACES data dictionary views to verify if a tablespace was encrypted.

See Also

Oracle Database Reference for more information about the DBA_TABLESPACES and USER_TABLESPACES data dictionary views

4.6.5.3.2 Creating an Encrypted Tablespace

To create an encrypted tablespace, you must use the `CREATE TABLESPACE` statement with the `ENCRYPTION USING` clause.

- Run the `CREATE TABLESPACE` statement, using its encryption clauses.

For example:

```
CREATE TABLESPACE encrypt_ts
  DATAFILE '$ORACLE_HOME/dbs/encrypt_df.dbf' SIZE 1M
  ENCRYPTION USING 'AES256' ENCRYPT;
```

In this specification:

- `ENCRYPTION USING 'AES256' ENCRYPT` specifies the encryption algorithm and the key length for the encryption. The `ENCRYPT` clause encrypts the tablespace. Enclose this setting in single quotation marks (' '). The key lengths are included in the names of the algorithms. If you do not specify an encryption algorithm, then the default encryption algorithm, AES128, is used.

Related Topics

- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

4.6.5.3.3 Example: Creating an Encrypted Tablespace That Uses AES192

You can use the `CREATE TABLESPACE` SQL statement to create an encrypted tablespace.

[Example 4-8](#) creates a tablespace called `seurespace_1` that is encrypted using the AES192 algorithm.

Example 4-8 Creating an Encrypted Tablespace That Uses AES192

```
CREATE TABLESPACE seurespace_1
  DATAFILE '/home/user/oradata/secure01.dbf'
  SIZE 150M
  ENCRYPTION USING 'AES192' ENCRYPT;
```

4.6.5.3.4 Example: Creating an Encrypted Tablespace That Uses the Default Algorithm

You can use the `CREATE TABLESPACE` SQL statement to create an encrypted tablespace that uses the default algorithm.

[Example 4-9](#) creates a tablespace called `seurespace_2`. Because no encryption algorithm is specified, the default encryption algorithm (AES128) is used. The key length is 128 bits.

Example 4-9 Creating an Encrypted Tablespace That Uses the Default Algorithm

```
CREATE TABLESPACE seurespace_2
  DATAFILE '/home/user/oradata/secure01.dbf'
  SIZE 150M
  ENCRYPTION ENCRYPT;
```

4.6.6 Setting the Tablespace Encryption Default Algorithm

The `TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM` applies to specific encryption scenarios.

These scenarios are as follows:

- Encryption commands that do not allow to specify the encryption algorithm
- New tablespaces that are created without the encryption syntax
- The encryption algorithm for the `SYSTEM` tablespace

`_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM` only becomes effective if it is set before the first `SET KEY` operation with Oracle Key Vault, or the `CREATE KEYSTORE` command for software keystore-based TDE configuration.

`_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM` applies to both offline and online tablespace encryption operations. It also applies to future encrypted tablespaces, if `TABLESPACE_ENCRYPTION` has been set appropriately.

- Enter the following `ALTER SYSTEM` statement:

```
ALTER SYSTEM SET "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM" = value SCOPE=BOTH;
```

In this specification, *value* can be one of the following encryption algorithms: `AES128`, `AES192`, `AES256`, `3DES168`, `ARIA128`, `ARIA192`, or `ARIA256`. The default encryption algorithm is `AES128`.

- You can also add the following statement to the `dbca create database` or `dbaascli create database` command:

```
-initparams "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM" = AES256
```

4.6.7 Encrypting Future Tablespaces

You can configure Oracle Database to automatically encrypt future tablespaces that you will create.

- [About Encrypting Future Tablespaces](#)
The `ENCRYPT_NEW_TABLESPACES` dynamic database initialization parameter controls if future tablespaces are encrypted.
- [Setting Future Tablespaces to be Encrypted](#)
You can set the `ENCRYPT_NEW_TABLESPACES` database initialization parameter to automatically encrypt future tablespaces that you create.

4.6.7.1 About Encrypting Future Tablespaces

The `ENCRYPT_NEW_TABLESPACES` dynamic database initialization parameter controls if future tablespaces are encrypted.

By default, all Oracle Cloud databases are encrypted. If you install an off-the-shelf application into such a database, its installation scripts most likely do not have the encryption syntax. In this case, because `ENCRYPT_NEW_TABLESPACES` is set to `CLOUD_ONLY`, those tablespaces would be created encrypted regardless.

In an Oracle Cloud environment, the following scenarios may occur when you create encrypted tablespaces in Oracle Cloud and on-premises environments:

- You create a database in Oracle Cloud; the tablespaces are encrypted because the `TABLESPACE_ENCRYPTION` parameter is set to `AUTO_ENABLE`. However, you may not have the intention or even an Advanced Security Option license to bring the encrypted database back on premises. For this use case, Oracle Recovery Manager (Oracle RMAN) provides the option to duplicate or restore AS `DECRYPTED`.
- You create a hybrid environment where the primary database is on premises and the standby database is on Oracle Cloud. If a switchover operation takes place, then the new primary is on Oracle Cloud. For example, suppose you do not have an Advanced Security Option (ASO) license, and you have an automatically encrypted tablespace in the Oracle Cloud. The standby database on premises is also automatically encrypted. In this case, you either need an Advanced Security license (which includes Transparent Data Encryption and Data Redaction) for the on-premises standby database, or you cannot use the standby database.

4.6.7.2 Setting Future Tablespaces to be Encrypted

You can set the `ENCRYPT_NEW_TABLESPACES` database initialization parameter to automatically encrypt future tablespaces that you create.

Note

Starting with Oracle Database 19.16, the `ENCRYPT_NEW_TABLESPACES` is deprecated.

- In SQL*Plus, enter the following `ALTER SYSTEM` statement:

```
ALTER SYSTEM SET ENCRYPT_NEW_TABLESPACES = value;
```

In this specification, *value* can be:

- `CLOUD_ONLY` transparently encrypts the tablespace in the Cloud using the tablespace encryption default algorithm if you do not specify the `ENCRYPTION` clause of the `CREATE TABLESPACE` SQL statement. In Oracle Cloud Infrastructure (OCI) databases, this value cannot be changed, to comply with Oracle's encryption mandate. If you create the tablespace on premises, then it will follow the `CREATE TABLESPACE` statement specification that you enter. For example, if you omit the `ENCRYPTION` clause, then the tablespace is created unencrypted. If you include this clause and use a different algorithm, then the tablespace will use that algorithm. `CLOUD_ONLY` is the default.
- `AUTO_ENABLE` automatically encrypts the tablespace using the tablespace encryption default algorithm and cipher mode if you do not specify the `ENCRYPTION` clause of the `CREATE TABLESPACE` SQL statement, or to the algorithm specified by the `TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM` dynamic parameter.
- `ALWAYS` automatically encrypts the tablespace using the tablespace encryption default algorithm if you omit the `ENCRYPTION` clause of `CREATE TABLESPACE`, for both the Cloud and premises scenarios.

If you do provide the `ENCRYPTION` clause, however, the algorithm that you specify takes precedence over the tablespace encryption default algorithm.
- DDL encrypts the tablespace using the specified setting of the `ENCRYPTION` clause of `CREATE TABLESPACE`, for both Oracle Cloud and on-premises environments.

Related Topics

- `ENCRYPT_NEW_TABLESPACES`

4.6.8 Encrypted Sensitive Credential Data in the Data Dictionary

You can encrypt sensitive credential data in the SYS.LINK\$ and SYS.SCHEDULER\$_CREDENTIAL system tables.

By default, the credential data in the SYS.LINK\$ and SYS.SCHEDULER\$_CREDENTIAL system tables is obfuscated. However, because of the availability of many types of de-obfuscation algorithms, Oracle recommends that you encrypt this sensitive credential data. To check the status the data dictionary credentials, you can query the DICTIONARY_CREDENTIALS_ENCRYPT data dictionary view.

The encryption of sensitive credential data in these two system tables uses Transparent Data Encryption. Encryption of credential data uses the AES256 algorithm. To encrypt credential data, you do not need an Oracle Advanced Security Option license, but you must be granted the SYSKM administrative privilege and the database must have an open keystore.

Related Topics

- [Oracle Database Security Guide](#)

4.6.9 Encryption Conversions for Existing Offline Tablespaces

You can perform offline encryption conversions by using the ALTER TABLESPACE SQL statement OFFLINE, ENCRYPT, and DECRYPT clauses.

- [About Encryption Conversions for Existing Offline Tablespaces](#)
You can encrypt or decrypt an existing data file of a user tablespace when the tablespace is offline or when the database is not open.
- [Encrypting an Existing User-Defined Tablespace with Offline Conversion](#)
To encrypt an existing tablespace with offline conversion, you can use the ALTER TABLESPACE SQL statement with the OFFLINE and ENCRYPT clauses.
- [Decrypting an Existing Tablespace with Offline Conversion](#)
To decrypt an existing tablespace with offline conversion, you can use the ALTER TABLESPACE SQL statement with the OFFLINE and DECRYPT clauses.

4.6.9.1 About Encryption Conversions for Existing Offline Tablespaces

You can encrypt or decrypt an existing data file of a user tablespace when the tablespace is offline or when the database is not open.

Use the offline encryption method if you do not plan to change the compatibility of your databases from Oracle Database 11c release 2 (11.2) or Oracle Database 12c release 1 (12.1) to release 19c, which is irreversible. The offline encryption method is also useful if you want to quickly make use of Transparent Data Encryption before you upgrade this database to release 18c. You can both encrypt and decrypt offline tablespaces.

Note the following:

- If you want to encrypt the Oracle Database-supplied tablespaces (SYSTEM, SYSAUX, and UNDO) using the offline conversion method, then you must use the method that is recommended when you encrypt an existing database with offline conversion.
- You can use the online method to rekey, change the encryption algorithm, or decrypt a tablespace that was previously encrypted with the offline method.

- If you have configured Oracle Data Guard, then you can minimize downtime by encrypting the tablespaces on the standby first, switching over to the primary, and then performing an offline encryption the tablespaces on the new standby database. Offline encryption (both on the data file and tablespace level) are performed on the standby first. Online encryption is an Oracle Data Guard transaction, and as such, it is replayed on the standby database.
- You can use the USING ... ENCRYPT clause to specify an encryption algorithm. Supported algorithms include AES128, AES192, AES256, and others, such as ARIA and GOST. To check the encryption key, query the ENCRYPTIONALG column in the V\$DATABASE_KEY_INFO view.
- To set the default encryption algorithm for future offline tablespace encryption operations, set the TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM dynamic parameter.
- You can use the ALTER TABLESPACE statement to convert offline tablespaces in parallel by using multiple foreground sessions to encrypt different data files.
- If you are using Oracle Data Guard, you can minimize the downtime by encrypting the tablespaces on the standby first, switching over, and then encrypting the tablespaces on the original primary next.
- For Oracle Database 11g release 2 (11.2.0.4) and Oracle Database 12c release 1 (12.1.0.2), you cannot perform an offline encryption of the SYSTEM and SYSAUX tablespaces. Also, Oracle does not recommend encrypting offline the UNDO tablespace in these releases. Doing so prevents the keystore from being closed, and this prevents the database from functioning. In addition, encrypting the UNDO tablespace while the database is offline is not necessary because all undo records that are associated with any encrypted tablespaces are already automatically encrypted in the UNDO tablespace. If you want to encrypt the TEMP tablespace, you must drop and then recreate it as encrypted.

Related Topics

- [Encrypting an Existing Database with Offline Conversion](#)
When you encrypt an existing database with offline conversion, for the Oracle-managed tablespaces, you do not specify an encryption algorithm.
- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.
- [Setting the Tablespace Encryption Default Algorithm](#)
The TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM applies to specific encryption scenarios.

4.6.9.2 Encrypting an Existing User-Defined Tablespace with Offline Conversion

To encrypt an existing tablespace with offline conversion, you can use the ALTER TABLESPACE SQL statement with the OFFLINE and ENCRYPT clauses.

The procedure that is described in this section applies to the case where you want to encrypt individual user-created tablespaces within a database. These tablespaces can be encrypted offline. However, the Oracle Database-supplied SYSTEM and UNDO tablespaces cannot be brought offline. If you want to encrypt the tablespaces offline, then you must use the method that is described in [Encrypting an Existing Database with Offline Conversion](#).

1. Connect to the united mode CDB root or isolated mode PDB as a user who has been granted administrative privileges.

You must have the SYSDBA administrative privilege to work with the SYSTEM and SYSAUX tablespaces. Otherwise, connect with the SYSKM administrative privilege.
2. Bring the tablespace offline.


```
ALTER TABLESPACE users OFFLINE NORMAL;
```

3. Back up the tablespace.

The offline conversion method does not use auxiliary disk space or files, and it operates directly in-place to the data files. Therefore, you should perform a full backup of the user tablespace before converting it offline.

4. As a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege, open the TDE wallet.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY TDEk_wallet_password;
```

5. Encrypt the tablespace.

For example, to encrypt an entire tablespace, include its data files:

```
ALTER TABLESPACE users1 ENCRYPTION OFFLINE ENCRYPT;
```

This example encrypts the tablespace with the default encryption algorithm, AES128. (AES256 is the default.) To use a different encryption algorithm, enter a statement similar to the following:

```
ALTER TABLESPACE users2 ENCRYPTION OFFLINE USING 'AES128' ENCRYPT;
```

To encrypt individual data files within a tablespace, use the ALTER DATABASE DATAFILE SQL statement. For example, to encrypt the data files user_01.dbf and user_02.dbf:

```
ALTER DATABASE DATAFILE 'user_01.dbf' ENCRYPT;  
ALTER DATABASE DATAFILE 'user_02.dbf' ENCRYPT;
```

In the same database session, these statements encrypt each of the data files in sequence, one after another. If you run each statement in its own database session, then they will be run in parallel.

If the encryption process is interrupted, then rerun the ALTER TABLESPACE statement. The kinds of errors that you can expect in an interruption are general errors, such as file system or storage file system errors. The data files within the tablespace should be consistently encrypted. For example, suppose you offline a tablespace that has 10 files but for some reason, the encryption only completes for nine of the files, leaving one decrypted. Although it is possible to bring the tablespace back online with such inconsistent encryption if the COMPATIBLE parameter is set to 12.2.0.0 or later, then it is not recommended to leave the tablespace in this state. If COMPATIBLE is less than 12.2.0.0, then it is not possible to bring the tablespace online if the encryption property is inconsistent across the data files.

6. Bring the tablespace back online or open the database.

- To bring the tablespace back online:

```
ALTER TABLESPACE users ONLINE;
```

- To open a database in a non-multitenant environment:

```
ALTER DATABASE OPEN
```

- In a multitenant environment, you can encrypt a data file or tablespace with the offline method if the root is open and the PDB is not open. For example, for a PDB named hr_pdb:

```
ALTER PLUGGABLE DATABASE hr_pdb OPEN
```

7. Perform a full backup of the converted tablespace.

Related Topics

- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

4.6.9.3 Decrypting an Existing Tablespace with Offline Conversion

To decrypt an existing tablespace with offline conversion, you can use the ALTER TABLESPACE SQL statement with the OFFLINE and DECRYPT clauses.

1. Connect as a user who has the SYSDBA administrative privilege.
You must have the SYSDBA administrative privilege if you plan to decrypt the SYSTEM and SYSAUX tablespaces. Otherwise, connect with the SYSKM administrative privilege.
2. Bring the tablespace offline.

```
ALTER TABLESPACE users OFFLINE NORMAL;
```
3. As a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege, open the keystore.
For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY TDE_wallet_password;
```
4. Run the ALTER TABLESPACE SQL statement to perform the decryption.
For example, for a tablespace called users:

```
ALTER TABLESPACE users ENCRYPTION OFFLINE DECRYPT;
```

If the decryption process is interrupted, then rerun the ALTER TABLESPACE statement. The kinds of errors that you can expect in an interruption are general errors, such as file system or storage file system errors. The data files within the tablespace should be consistently decrypted. For example, suppose you offline a tablespace that has 10 files but for some reason, the decryption only completes for nine of the files, leaving one encrypted. Although it is possible to bring the tablespace back online with such inconsistent decryption if the COMPATIBLE parameter is set to 12.2.0.0 or later, then it is not recommended to leave the tablespace in this state. If COMPATIBLE is less than 12.2.0.0, then it is not possible to bring the tablespace online if the encryption property is inconsistent across the data files.

5. Bring the tablespace online.

```
ALTER TABLESPACE users ONLINE;
```

Related Topics

- [Opening a TDE Wallet](#)
To open a TDE wallet, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE OPEN clause.

4.6.10 Encryption Conversions for Existing Online Tablespaces

You can encrypt and decrypt an online existing tablespace by using the ALTER TABLESPACE SQL statement with the ONLINE and ENCRYPT or DECRYPT clauses.

- [About Encryption Conversions for Existing Online Tablespaces](#)
You can encrypt, decrypt, or rekey existing user tablespaces, and the SYSTEM, SYSAUX, and UNDO tablespaces when they are online.

- [Encrypting an Existing Tablespace with Online Conversion](#)
To encrypt an existing tablespace with online conversion, use ALTER TABLESPACE with the ONLINE and ENCRYPT clauses.
- [Rekeying an Existing Tablespace with Online Conversion](#)
To rekey an existing tablespace that is online, you can use the REKEY clause of the ALTER TABLESPACE SQL statement.
- [Decrypting an Existing Tablespace with Online Conversion](#)
To decrypt an existing tablespace with online conversion, you can use the ALTER TABLESPACE SQL statement with DECRYPT clause.
- [Finishing an Interrupted Online Encryption Conversion](#)
If an online encryption process is interrupted, then you can complete the conversion by rerunning the ALTER TABLESPACE statement using the FINISH clause.

4.6.10.1 About Encryption Conversions for Existing Online Tablespaces

You can encrypt, decrypt, or rekey existing user tablespaces, and the SYSTEM, SYSAUX, and UNDO tablespace when they are online.

However, you cannot encrypt, decrypt, or rekey a temporary tablespace online.

An online tablespace can be created by using the ONLINE clause of the CREATE TABLESPACE SQL statement. When you encrypt or rekey a tablespace online, the tablespace will have its own independent encryption keys and algorithms.

Note the following:

- If an offline tablespace has been encrypted, then you can rekey it online to use a different algorithm.
- You can encrypt multiple tablespaces online in parallel by using multiple foreground sessions to encrypt different tablespaces. Within each tablespace, the data files are encrypted sequentially.
- If the conversion is interrupted, then you can resume the process by issuing the FINISH clause of the ALTER TABLESPACE SQL statement.
- A redo log is generated for each online tablespace conversion.
- Do not encrypt the SYSTEM and UNDO tablespaces concurrently with other tablespaces.
- You cannot use the transportable tablespace feature with Oracle Data Pump while you are encrypting a tablespace.
- You cannot run the ALTER TABLESPACE statement concurrently with the following features:
 - ADMINISTER KEY MANAGEMENT SET KEY SQL statement
 - FLASHBACK DATABASE SQL statement
- If you are using Oracle-managed files for the data files, then the encryption process rekeys the data files that are associated with the tablespace and then copies or moves them to the default Oracle-managed files location.
- You can add new files to the tablespace after you have encrypted it. Oracle Database reformats the new file with the new encryption key. Blocks will be encrypted using the new key.
- Previous operations that took place in the root or the PDB may require the control files to be cross-checked against the data dictionary before you can begin the online conversion process. An ORA-241 operation disallowed: control file is not yet checked

against data dictionary error may occur. To resolve this problem, restart the root or PDB, and then try issuing the online conversion commands again.

- For security reasons, once online conversion processes a data file, Oracle will zero out the original data file before deletion. This prevents the database from leaving ghost data on disk sectors. However, there is a known limitation that can occur if you are performing an online tablespace conversion at the same time that Oracle Recovery Manager (Oracle RMAN) is validating files. The online tablespace conversion processes each file one at a time. If Oracle RMAN is validating a file at the same time that it is being processed by the online tablespace conversion, then Oracle RMAN could report a corruption problem (ORA-01578: ORACLE data block corrupted (file # , block #)). It does this because it sees the blocks that comprise the file as zero. This is a false alarm and you can ignore the error. If this occurs, then try running the Oracle RMAN validation process again.

Related Topics

- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

4.6.10.2 Encrypting an Existing Tablespace with Online Conversion

To encrypt an existing tablespace with online conversion, use ALTER TABLESPACE with the ONLINE and ENCRYPT clauses.

1. Connect as a user who has the SYSDBA administrative privilege.

You must have the SYSDBA administrative privilege if you plan to encrypt the SYSTEM and SYSAUX tablespaces. Otherwise, connect with the SYSKM administrative privilege.

2. Ensure that the COMPATIBLE initialization parameter is set to 12.2.0.0 or later.

You can use the SHOW PARAMETER command to check the current setting of a parameter.

3. Ensure that the database is open in read-write mode.

You can query the STATUS column of the V\$INSTANCE dynamic view to find if a database is open and the OPEN_MODE column of the V\$DATABASE view to find if it in read-write mode.

4. If necessary, open the database in read-write mode.

```
ALTER DATABASE OPEN READ WRITE;
```

5. Ensure that the auxiliary space is at least the same size as the largest data file of this tablespace.

This size requirement is because Oracle Database performs the conversion one file at a time. For example, if the largest data file of the tablespace is 32 GB, then ensure that you have 32 GB of auxiliary space. To find the space used by a data file, query the BYTES or BLOCKS column of the V\$DATAFILE dynamic performance view.

6. Create and open a master encryption key.

For example:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE 'TDE_wallet_location' IDENTIFIED BY  
TDE_wallet_password;  
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY TDE_wallet_password;  
ADMINISTER KEY MANAGEMENT SET KEY IDENTIFIED BY TDE_wallet_password WITH BACKUP;
```

7. Run the ALTER TABLESPACE statement using the ENCRYPTION and ENCRYPT clauses to perform the encryption.

Note

For the SYSTEM tablespace, you can use the ENCRYPT clause to encrypt the tablespace, but you cannot specify an encryption algorithm because it is encrypted with the existing database key the first time. After encrypting the SYSTEM tablespace, use the REKEY clause to specify the algorithm. See [Rekeying an Existing Tablespace with Online Conversion](#) for more information.

For example, for a non-Oracle managed files tablespace named users:

```
ALTER TABLESPACE users ENCRYPTION ONLINE USING 'AES192' ENCRYPT FILE_NAME_CONVERT = ('users.dbf', 'users_enc.dbf');
```

In this example:

- ENCRYPTION ONLINE USING 'AES192' ENCRYPT sets the statement to encrypt the tablespace users while it is online and assigns it the AES192 encryption algorithm. If you omit the USING *algorithm* clause, then the default algorithm, AES128, is used.
- FILE_NAME_CONVERT specifies one or more pairs of data files that are associated with the tablespace. The first name in the pair is an existing data file, and the second name is for the encrypted version of this data file, which will be created after the ALTER TABLESPACE statement successfully executes. If the tablespace has more than one data file, then you must process them all in this statement. Note the following:

- Separate each file name with a comma, including multiple pairs of files. For example:

```
FILE_NAME_CONVERT = ('users1.dbf', 'users1_enc.dbf', 'users2.dbf', 'users2_enc.dbf')
```

- You can specify directory paths in the FILE_NAME_CONVERT clause. For example, the following clause converts and moves the matching files of the tablespace from the dbs directory to the dbs/enc directory:

```
FILE_NAME_CONVERT = ('dbs', 'dbs/enc')
```

- The FILE_NAME_CONVERT clause recognizes patterns. The following example converts the data files users_1.dbf and users_2.dbf to users_enc1.dbf and users_enc2.dbf:

```
FILE_NAME_CONVERT = ('users', 'users_enc')
```

- In an Oracle Data Guard environment, include the name of the standby database data file in the FILE_NAME_CONVERT settings.
- If you are using Oracle-managed file mode, then the new file name is internally assigned, so this file name should not affect your site's file-naming standards. If you are using non-Oracle-managed file mode and if you omit the FILE_NAME_CONVERT clause, then Oracle Database internally assigns an auxiliary file name, and then later renames it back to the original name. This enables the encryption process to use the name that you had originally given the file to be encrypted. The renaming operation is effectively creating another copy of the file, hence it is slower than explicitly including the FILE_NAME_CONVERT clause. For better performance, include the FILE_NAME_CONVERT clause.
- You can find the data files for a tablespace by querying the V\$DATAFILE or V\$DATAFILE_HEADER dynamic views.

By default, data files are in the \$ORACLE_HOME/dbs directory. If the data files are located there, then you do not have to specify a path.

After you complete the conversion, you can check the encryption status by querying the STATUS column of the V\$ENCRYPTED_TABLESPACES dynamic view. The ENCRYPTIONALG column of this view shows the encryption algorithm that is used. If the conversion process was interrupted, then you can resume it by running ALTER TABLESPACE with the FINISH clause. For example, if the primary data file converts but the standby data file does not, then you can run ALTER TABLESPACE ... FINISH on the standby database for the standby data files.

Related Topics

- [Setting the COMPATIBLE Initialization Parameter for Tablespace Encryption](#)
To set the COMPATIBLE initialization parameter, you must edit the initialization parameter file for the database instance.
- [Finishing an Interrupted Online Encryption Conversion](#)
If an online encryption process is interrupted, then you can complete the conversion by rerunning the ALTER TABLESPACE statement using the FINISH clause.

4.6.10.3 Rekeying an Existing Tablespace with Online Conversion

To rekey an existing tablespace that is online, you can use the REKEY clause of the ALTER TABLESPACE SQL statement.

Before you perform a rekey operation, be aware of the following:

- You cannot rekey the TEMP tablespace. If you want to assign a different encryption algorithm to a TEMP tablespace, then drop TEMP and recreate it with the correct encryption algorithm.
- Do not perform an online tablespace rekey operation with a master key operation concurrently. To find if any tablespaces are currently being rekeyed, issue the following query to find the rekey status of encrypted tablespaces:

```
SELECT TS#, ENCRYPTIONALG, STATUS FROM V$ENCRYPTED_TABLESPACES;
```

A status of REKEYING means that the corresponding tablespace is still being rekeyed. Do not rekey the master key while this status is in effect.

To rekey an existing tablespace with online conversion:

1. Connect as a user who has the SYSDBA administrative privilege.

For example:

```
sqlplus sec_admin as sysdba  
Enter password: password
```

You must have the SYSDBA administrative privilege if you plan to rekey the SYSTEM and SYSAUX tablespaces. Otherwise, connect with the SYSKM administrative privilege.

2. Ensure that the following requirements are met:
 - The COMPATIBLE initialization parameter is set to 12.2.0.0 or later.
 - The database is open and in read-write mode.
 - A master encryption key has been created and is open.
3. Query the KEY_VERSION and STATUS columns of the V\$ENCRYPTED_TABLESPACES dynamic view to find the current status of the encryption algorithm used by the master encryption key.
4. Perform the rekey operation, based on the status returned by the V\$ENCRYPTED_TABLESPACES dynamic view:

- If the key version status of the tablespace is NORMAL, then specify the new algorithm of the online tablespace rekey.

For example:

```
ALTER TABLESPACE users ENCRYPTION USING 'AES192' REKEY FILE_NAME_CONVERT =  
( 'users.dbf', 'users_enc.dbf' );
```

- If the key version status is ENCRYPTING, DECRYPTING, or REKEYING, then use the FINISH clause.

For example:

```
ALTER TABLESPACE users ENCRYPTION ONLINE FINISH REKEY FILE_NAME_CONVERT =  
( 'users.dbf', 'users_enc.dbf' );
```

5. If the ORA-00241 operation disallowed: control file inconsistent with data dictionary error appears, then restart the database.

In a multitenant environment, restart the CDB root database and then retry Step 4.

If the conversion process was interrupted, then you can resume it by running ALTER TABLESPACE with the FINISH clause.

Related Topics

- [Encrypting an Existing Tablespace with Online Conversion](#)
To encrypt an existing tablespace with online conversion, use ALTER TABLESPACE with the ONLINE and ENCRYPT clauses.
- [About Encryption Conversions for Existing Online Tablespaces](#)
You can encrypt, decrypt, or rekey existing user tablespaces, and the SYSTEM, SYSAUX, and UNDO tablespace when they are online.
- [Finishing an Interrupted Online Encryption Conversion](#)
If an online encryption process is interrupted, then you can complete the conversion by rerunning the ALTER TABLESPACE statement using the FINISH clause.

4.6.10.4 Decrypting an Existing Tablespace with Online Conversion

To decrypt an existing tablespace with online conversion, you can use the ALTER TABLESPACE SQL statement with DECRYPT clause.

1. Connect as a user who has the SYSDBA administrative privilege.

You must have the SYSDBA administrative privilege if you plan to decrypt the SYSTEM and SYSAUX tablespaces. Otherwise, connect with the SYSKM administrative privilege.

2. Ensure that the following requirements are met:
 - The COMPATIBLE initialization parameter is set to 12.2.0.0 or later.
 - The database is open and in read-write mode.
 - A master encryption key has been created and is open.
 - There is enough auxiliary space to complete the decryption.
3. Run the ALTER TABLESPACE SQL statement with the DECRYPT clause.

For example:

```
ALTER TABLESPACE users ENCRYPTION ONLINE DECRYPT FILE_NAME_CONVERT =  
( 'users_enc.dbf', 'users.dbf' );
```

In this specification:

- When you specify the files to decrypt, enter them in the reverse order in which they were originally encrypted. That is, first enter the name of the encrypted file (users_enc.dbf), followed by the data file (users.dbf).
- Do not provide an algorithm key for the decryption.

If the conversion process was interrupted, then you can resume it by running ALTER TABLESPACE with the FINISH clause.

Related Topics

- [Encrypting an Existing Tablespace with Online Conversion](#)
To encrypt an existing tablespace with online conversion, use ALTER TABLESPACE with the ONLINE and ENCRYPT clauses.
- [Finishing an Interrupted Online Encryption Conversion](#)
If an online encryption process is interrupted, then you can complete the conversion by rerunning the ALTER TABLESPACE statement using the FINISH clause.

4.6.10.5 Finishing an Interrupted Online Encryption Conversion

If an online encryption process is interrupted, then you can complete the conversion by rerunning the ALTER TABLESPACE statement using the FINISH clause.

An interrupted encryption process (encryption, rekey, or decryption) can be, for example, an ORA-28425: missing a valid FILE_NAME_CONVERT clause error in the FILE_NAME_CONVERT clause of the ALTER TABLESPACE SQL statement. Other examples of interrupted processes are if the conversion skips a data file, which can happen if there is an error when an Oracle DataBase WRiter (DBWR) process offlines a data file, or if there is not enough space for the auxiliary file. The tablespace should be operational even if you do not rerun the ALTER TABLESPACE statement with the FINISH clause.

In addition to interrupted encryption processes, the tablespace encryption process can fail during the period when the status is ENCRYPTING. In this case, you can either decrypt the tablespace back to its original state, or you can resume the encryption by using the ENCRYPTION ONLINE FINISH ENCRYPT clause of ALTER TABLESPACE.

1. Query the V\$ENCRYPTED_TABLESPACES to check the STATUS column for the tablespace.
If the STATUS column reports ENCRYPTING, DECRYPTING, or REKEYING, then re-run the ALTER TABLESPACE statement with the FINISH clause, as described in this procedure. If the STATUS reports NORMAL, then you can rerun ALTER TABLESPACE without the FINISH clause.
You can find the tablespace name that matches the TS# and TABLESPACE_NAME columns by querying the V\$DATAFILE_HEADER view.
2. If necessary query the following additional views to find information about the tablespace whose online conversion was interrupted:
 - DBA_TABLESPACES to find if the STATUS of the tablespace indicates if it is online or offline.
 - V\$ENCRYPTED_TABLESPACES to find if the STATUS of the tablespace indicates if it is encrypted, and what the KEY_VERSION of the encryption key is.
 - V\$DATAFILE and V\$DATAFILE_HEADER to find the data files that are associated with a tablespace.
3. Run the ALTER TABLESPACE statement using the FINISH clause.

Examples are as follows:

- For an encryption operation:


```
ALTER TABLESPACE users ENCRYPTION ONLINE FINISH ENCRYPT FILE_NAME_CONVERT =  
( 'users.dbf', 'users_enc.dbf' );
```

- For a decryption operation:

```
ALTER TABLESPACE users ENCRYPTION ONLINE FINISH DECRYPT FILE_NAME_CONVERT =  
( 'users_enc.dbf', 'users.dbf' );
```

Note the order in which the files are specified: first, the name of the encrypted file, and then the name of the data file. (In the encryption operation, the name of the data file is specified first, followed by the name of the encrypted file.)

- For a rekey operation:

```
ALTER TABLESPACE users ENCRYPTION ONLINE FINISH REKEY FILE_NAME_CONVERT =  
( 'users.dbf', 'users_enc.dbf' );
```

You cannot specify an algorithm when you use the FINISH clause in an ALTER TABLESPACE statement.

4. To check the conversion, query the STATUS column of the V\$ENCRYPTED_TABLESPACES view.

The status should be NORMAL. In an Oracle Data Guard environment, if the database does not have NORMAL as the STATUS, then run the ALTER TABLESPACE ... FINISH statement on the primary or the standby data file that did not successfully convert.

4.6.11 Encryption Conversions for Existing Databases

You can encrypt both offline and online databases.

- [About Encryption Conversions for Existing Databases](#)
The encryption conversion of an entire database encrypts all tablespaces, including the Oracle-supplied SYSTEM, SYSAUX, UNDO, and TEMP tablespaces.
- [Encrypting an Existing Database with Offline Conversion](#)
When you encrypt an existing database with offline conversion, for the Oracle-managed tablespaces, you do not specify an encryption algorithm.
- [Encrypting an Existing Database with Online Conversion](#)
When you encrypt an existing database with online conversion, you do not specify an encryption algorithm.

4.6.11.1 About Encryption Conversions for Existing Databases

The encryption conversion of an entire database encrypts all tablespaces, including the Oracle-supplied SYSTEM, SYSAUX, UNDO, and TEMP tablespaces.

Note the following:

- If you are using Database Configuration Assistant (DBCA) to create or configure a database, then you can create a TDE wallet in the database as part of the creation or configuration process. When you drop a database by using DBCA, any TDE wallets that are in this database are also removed. **Important: Before you drop a database by using DBCA, and if it has any local TDE wallets, then back these wallets up to a secure location.** If the database has been migrated to use Oracle Key Vault, then be aware that its TDE encryption keys reside in the Oracle Key Vault server. It is the responsibility of the Oracle Key Vault administrator to back up Oracle Key Vault.
- To perform the encryption, you can use the offline and online functionality of the tablespace encryption conversions.

- You can encrypt any or all of the Oracle-supplied tablespaces, and in any order. The encryption of the Oracle-supplied tablespaces has no impact on the encryption of user-created tablespaces.
- When you encrypt the Oracle-supplied tablespaces, Oracle Database prevents the keystore from being closed.
- You cannot encrypt an existing temporary tablespace, but you can create an encrypted temporary tablespace, configure the database to use that new encrypted temporary tablespace as the default temporary tablespace, and then drop the old clear-text temporary tablespace.
- The UNDO and TEMP metadata that is generated from sensitive data in an encrypted tablespace is already automatically encrypted. Therefore, encrypting UNDO and TEMP is optional.
- The performance effect of encrypting all the tablespaces in a database depends on the workload and platform. Many modern CPUs provide built-in hardware acceleration, which results in a minimal performance impact.
- In a multitenant environment, you can encrypt any tablespaces in any pluggable databases (PDBs), including the Oracle-supplied tablespaces. However, the keystore in the CDB root must be open at all times so that a PDB can open its keystore. You can check the status of whether a keystore is open by querying the STATUS column of the V\$ENCRYPTION_WALLET view

4.6.11.2 Encrypting an Existing Database with Offline Conversion

When you encrypt an existing database with offline conversion, for the Oracle-managed tablespaces, you do not specify an encryption algorithm.

1. Connect as a user who has the SYSDBA administrative privilege.

You must have the SYSDBA administrative privilege to encrypt the SYSTEM and SYSAUX tablespaces.

2. Mount the database.

```
STARTUP MOUNT
```

3. Open the keystore.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY keystore_password;
```

4. Run the ALTER TABLESPACE SQL statement to encrypt the SYSTEM, SYSAUX, and UNDO tablespaces. Do not encrypt the SYSTEM tablespace concurrently with the encryption of other tablespaces.

For example, to encrypt the SYSTEM tablespace:

```
ALTER TABLESPACE SYSTEM ENCRYPTION OFFLINE ENCRYPT;
```

5. Open the database.

For example, to open the database in read/write mode:

```
ALTER DATABASE OPEN READ WRITE;
```

6. Run the ALTER TABLESPACE SQL statement to encrypt other user tablespaces.

Alternatively, you can proceed to the next step and open the database first, and then perform the steps described in [Encrypting an Existing User-Defined Tablespace with Offline Conversion](#).

7. Open the database.

```
ALTER DATABASE OPEN;
```

After you have encrypted the tablespace, if you want to use a different encryption algorithm (change the TDE master encryption key) for the SYSTEM, SYSAUX, and UNDO tablespaces, then you must use online conversion. In addition to AES128, supported encryption algorithms are AES192 and AES256, in addition to other algorithms such as ARIA and GOST.

Related Topics

- [Changing the TDE Master Encryption Key for a Tablespace](#)
You can use the ENCRYPT and REKEY clauses of the ALTER TABLESPACE statement to encrypt a tablespace.
- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

4.6.11.3 Encrypting an Existing Database with Online Conversion

When you encrypt an existing database with online conversion, you do not specify an encryption algorithm.

The reason that you do not need to specify an encryption algorithm the first time you perform the encryption is that the tablespaces that you must use to encrypt the database are automatically encrypted with the database key. If you want to change the algorithm, then you can issue the ALTER TABLESPACE ENCRYPTION REKEY SQL statement after the initial encryption.

1. Perform the following tasks, which are described in [Encrypting an Existing Tablespace with Online Conversion](#):
 - a. Connect as a user who has been granted the SYSDBA administrative privilege.
 - b. Ensure that the COMPATIBLE parameter is set to 12.2.0.0 or later.
 - c. Ensure that the database is open in read-write mode.
 - d. Ensure that you have enough auxiliary space to complete the encryption.
 - e. Back up the tablespaces that you must encrypt.
 - f. Open the keystore.
2. Run the ALTER TABLESPACE SQL statement to encrypt the SYSTEM, SYSAUX, and UNDO tablespaces. Do not specify an algorithm, and do not encrypt the SYSTEM tablespace concurrently with the encryption of other tablespaces.

For example, to encrypt the SYSTEM tablespace:

```
ALTER TABLESPACE SYSTEM ENCRYPTION ONLINE ENCRYPT
FILE_NAME_CONVERT=('system01.dbf','system01_enc.dbf');
```

3. Create a temporary tablespace.
 - a. Create a new (encrypted) TEMP tablespace with the identical characteristics as the original TEMP tablespace, as follows:

```
SELECT ' CREATE TEMPORARY TABLESPACE '
||tablespace_name||'_ENC tempfile '''
||substr(file_name,1,length(file_name)-4)
||'_enc.dbf' size '
||bytes||' ENCRYPTION USING 'AES256' ENCRYPT;'
AS " Create new encrypted TEMP tablespace"
FROM DBA_TEMP_FILES;
```

Alternatively, you could extract the original parameters by running `DBMS_METADATA.GET_DDL`. If you omit the `USING algorithm` clause, then Oracle Database applies the default algorithm.

- b. Run the command (that is, the output of the `SELECT` statement).
- c. Change the database default temporary tablespace to the new encrypted TEMP tablespace.

```
ALTER DATABASE DEFAULT TEMPORARY TABLESPACE temp_enc;
```

- d. Drop the original TEMP tablespace.

```
DROP TABLESPACE TEMP INCLUDING CONTENTS AND DATAFILES;
```

Optionally, you can rename `temp_enc` to `TEMP`.

Related Topics

- [Changing the TDE Master Encryption Key for a Tablespace](#)
You can use the `ENCRYPT` and `REKEY` clauses of the `ALTER TABLESPACE` statement to encrypt a tablespace.

4.7 Transparent Data Encryption Data Dynamic and Data Dictionary Views

You can query a set of dynamic and data dictionary views to find more information about Transparent Data Encryption (TDE) data.

[Table 4-4](#) describes these dynamic and data dictionary views.

Table 4-4 Transparent Data Encryption Related Views

View	Description
<code>ALL_ENCRYPTED_COLUMNS</code>	Displays encryption information about encrypted columns in the tables accessible to the current user
<code>DICTIONARY_CREDENTIALS_ENCRYPT</code>	Indicates if credential data in the <code>SYS.LINK\$</code> and <code>SYS.SCHEDULER\$_CREDENTIAL</code> system tables is encrypted
<code>DBA_ENCRYPTED_COLUMNS</code>	Displays encryption information for all of the encrypted columns in the database
<code>USER_ENCRYPTED_COLUMNS</code>	Displays encryption information for encrypted table columns in the current user's schema
<code>DBA_TABLESPACE_USAGE_METRICS</code>	Describes tablespace usage metrics for all types of tablespaces, including permanent, temporary, and undo tablespaces
<code>V\$CLIENT_SECRETS</code>	Lists the properties of the strings (secrets) that were stored in the keystore for various features (clients). In a multitenant environment, when you query this view in a PDB, then it displays information about keys that were created or activated for the current PDB. If you query this view in the root, then it displays this information about keys for all of the PDBs.
<code>V\$DATABASE_KEY_INFO</code>	Displays information about the default encryption key that is used for the current database. The default is AES128.

Table 4-4 (Cont.) Transparent Data Encryption Related Views

View	Description
V\$ENCRYPTED_TABLESPACES	Displays information about the tablespaces that are encrypted
V\$ENCRYPTION_KEYS	When used with keys that have been rekeyed with the ADMINISTER KEY MANAGEMENT statement, displays information about the TDE master encryption keys. In a multitenant environment, when you query this view in a PDB, it displays information about keys that were created or activated for the current PDB. If you query this view in the root, it displays this information about keys for all of the PDBs.
V\$ENCRYPTION_WALLET	Displays information on the status of the keystore and the keystore location for TDE
V\$RMAN_ENCRYPTION_ALGORITHMS	Displays supported encryption algorithms in the current PDB and is used by Oracle Recovery Manager (Oracle RMAN) to validate user-requested algorithms

See Also

Oracle Database Reference for detailed information about these views

5

Switching from MES to OpenSSL Cryptographic Provider

Before switching from Dell BSAFE Crypto-C Micro Edition Suite, or MES (legacy provider) to OpenSSL (next-generation provider), complete the following pre-switch verification steps:

1. Inventory TDE Data Encryption Algorithms
Query your encrypted tablespaces and columns to identify any using 3DES168 (deprecated). 3DES encryption is outdated, slow, and no longer considered secure. Re-encrypt to AES256 before switching to FIPS 140-3 mode.
2. Inventory TDE Master Encryption Key Algorithm
Verify that the TDE master encryption key uses AES256. If the MEK uses ARIA256, SEED128, or GOST256, rekey to AES256:

```
ADMINISTER KEY MANAGEMENT SET KEY USING ALGORITHM 'AES256' IDENTIFIED BY  
keystore_password WITH BACKUP;
```

3. Check and Convert Wallet Encryption
Wallets created before Oracle Database 19.22 may use 3DES. Check and convert:

```
openssl pkcs12 -info -in ./ewallet.p12 -nodes
```

```
orapki wallet convert -wallet wallet_location -compat_v12
```

4. Verify RMAN Encryption Algorithm
If using RMAN backup encryption, verify the algorithm is AES-based:

```
CONFIGURE ENCRYPTION ALGORITHM 'AES256';
```

5. Verify Data Pump Encryption Algorithm
Use ENCRYPTION_ALGORITHM=AES256 for Data Pump exports.

6. Review FIPS Mode Configuration

If enabling FIPS 140-3 mode:

- Set FIPS_140=TRUE and FIPS_VERSION=FIPS140-3
- In FIPS mode, only AES128, AES192, and AES256 are available
- Append -fips140_mode to orapki commands

7. Benchmark Performance

Run representative workloads after switching providers to verify performance.

All modes of symmetric encryption using AES256 are considered quantum resistant. Standardizing on AES256 across all TDE operations provides the broadest compatibility with both current and future cryptographic requirements.

6

Managing the Keystore and the Master Encryption Key

You can modify settings for the keystore and TDE master encryption key, and store Oracle Database and store Oracle GoldenGate secrets in a keystore.

- [Managing the Keystore](#)
You can perform maintenance activities on keystores such as changing passwords, and backing up, merging, and moving keystores.
- [Managing the TDE Master Encryption Key](#)
You can manage the TDE master encryption key in several ways.
- [Storing Oracle Database Secrets](#)
Secrets are data that support internal Oracle Database features that integrate external clients such as Oracle GoldenGate into the database.
- [Storing Oracle GoldenGate Secrets in a Keystore](#)
You can store Oracle GoldenGate secrets in Transparent Data Encryption keystores.

6.1 Managing the Keystore

You can perform maintenance activities on keystores such as changing passwords, and backing up, merging, and moving keystores.

- [Performing Operations That Require a Keystore Password](#)
Many ADMINISTER KEY MANAGEMENT operations require access to a keystore password, for both TDE wallets and external keystores.
- [Changing the Password of a Software Keystore](#)
Oracle Database enables you to easily change password-protected software keystore passwords.
- [Changing the Oracle Key Vault Password](#)
To change the password of Oracle Key Vault, you use `okvutil`, which is part of the Oracle Key Vault endpoint software on the database host.
- [Configuring an External Store for a Keystore Password](#)
An external store for a keystore password stores the keystore password in a centrally accessed and managed location.
- [Backing Up Password-Protected Software Keystores](#)
When you back up a password-protected software keystore, you can create a backup identifier string to describe the backup type.
- [How the V\\$ENCRYPTION_WALLET View Interprets Backup Operations](#)
The BACKUP column of the V\$ENCRYPTION_WALLET view indicates how a copy of the keystore was created.
- [Backups of the External Keystore](#)
You cannot use Oracle Database to back up external keystores.
- [Merging TDE Wallets](#)
You can merge TDE wallets in a variety of ways.

- [Moving a TDE Wallet to a New Location](#)
You move a TDE wallet to a new location after you have updated the `WALLET_ROOT` parameter.
- [Moving a TDE Wallet Out of Automatic Storage Management](#)
You can use the `ADMINISTER KEY MANAGEMENT` statement to move a TDE wallet out of Automatic Storage Management.
- [Migrating from a TDE Wallet to Oracle Key Vault](#)
You can migrate between password-protected TDE wallets and external keystores in Oracle Key Vault.
- [Migration of Keystores to and from Oracle Key Vault](#)
You can use Oracle Key Vault to migrate both TDE wallets and external keystores to and from Oracle Key Vault.
- [Configuring Keystores for Automatic Storage Management](#)
You can store a TDE wallet on an Automatic Storage Management (ASM) disk group.
- [Closing a Keystore](#)
You can manually close software and external keystores.
- [Backup and Recovery of Encrypted Data](#)
For TDE wallets, you cannot access encrypted data without the TDE master encryption key.
- [Dangers of Deleting Keystores](#)
Oracle strongly recommends that you do not delete keystores.
- [Features That Are Affected by Deleted Keystores](#)
Some features can be adversely affected if a keystore is deleted and a TDE master encryption key residing in that keystore is later needed.

6.1.1 Performing Operations That Require a Keystore Password

Many `ADMINISTER KEY MANAGEMENT` operations require access to a keystore password, for both TDE wallets and external keystores.

In some cases, a keystore depends on an auto-login TDE wallet before the operation can succeed. Auto-login TDE wallets open automatically when they are configured and a key is requested. They are generally used for operations where the TDE wallet could be closed but a database operation needs a key (for example, after the database is restarted). Because the auto-login TDE wallet opens automatically, it can be retrieved to perform a database operation without manual intervention. However, some keystore operations that require the keystore password cannot be performed when the auto-login keystore is open. The auto-login TDE wallet must be closed and the password-protected keystore must be opened for the keystore operations that require a password.

In a multitenant environment, the re-opening of keystores affects other PDBs. For example, an auto-login TDE wallet in the root must be accessible by the PDBs in the CDB for this root.

You can temporarily open the TDE wallet by including the `FORCE KEYSTORE` clause in the `ADMINISTER KEY MANAGEMENT` statement when you perform the following operations: rotating a TDE wallet password; creating, using, rekeying, tagging, importing, exporting, migrating, or reverse migrating encryption keys; opening or backing up TDE wallet; adding, updating, or deleting secret TDE wallets. In a multitenant environment, if no TDE wallet is open in the root, then `FORCE KEYSTORE` opens the password-protected TDE wallet in the root.

6.1.2 Changing the Password of a Software Keystore

Oracle Database enables you to easily change password-protected software keystore passwords.

- [About Changing the Password of a Password-Protected Software Keystore](#)
You can only change the password for protected-protected software keystores.
- [Changing the Password-Protected Software Keystore Password](#)
To change the password of a password-protected software keystore, you must use the `ADMINISTER KEY MANAGEMENT` statement.

6.1.2.1 About Changing the Password of a Password-Protected Software Keystore

You can only change the password for protected-protected software keystores.

You can change this password at any time, as per the security policies, compliance guidelines, and other security requirements of your site. As part of the command to change the password, you will be forced to specify the `WITH BACKUP` clause, and thus forced to make a backup of the current keystore. During the password change operation, Transparent Data Encryption operations such as encryption and decryption will continue to work normally.

You can change this password at any time. You may want to change this password if you think it was compromised.

6.1.2.2 Changing the Password-Protected Software Keystore Password

To change the password of a password-protected software keystore, you must use the `ADMINISTER KEY MANAGEMENT` statement.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Change the password of the password-protected software keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD  
[FORCE KEYSTORE]  
IDENTIFIED BY  
old_password SET new_password  
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- *old_password* is the current keystore password that you want to change.
- *new_password* is the new password that you will set for the keystore.
- `WITH BACKUP` creates a backup of the current keystore before the password is changed. You must include this clause.
- *backup_identifier* specifies an optional identifier string for the backup that is created. The *backup_identifier* is added to the name of the backup file. Enclose *backup_identifier* in single quotation marks (' '). This identifier is appended to the named keystore file (for example, `ewallet_time_stamp_emp_key_pwd_change.p12`).

The following example backs up the current keystore and then changes the password for the keystore:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
FORCE KEYSTORE
IDENTIFIED BY
old_password SET new_password
WITH BACKUP USING 'pwd_change';
```

keystore altered.

6.1.3 Changing the Oracle Key Vault Password

To change the password of Oracle Key Vault, you use `okvutil`, which is part of the Oracle Key Vault endpoint software on the database host.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Close the external keystore.

- Close the connection to the external key manager:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE
IDENTIFIED BY Oracle_Key_Vault_password | EXTERNAL STORE CONTAINER = ALL;
```

If the keystore was auto-opened by the database, then close the connection to Oracle Key Vault as follows:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE
CONTAINER = ALL;
```

3. Change the Oracle Key Vault password.

```
WALLET_ROOT/okv/bin/okvutil changepwd -t wallet -l WALLET_ROOT/okv/ssl
```

4. Open the external keystore.

- For example, for Oracle Key Vault:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN FORCE KEYSTORE
IDENTIFIED BY new_Oracle_Key_Vault_pwd CONTAINER =ALL;
```

- If the old Oracle Key Vault password was stored in a [local] auto-open wallet in the `WALLET_ROOT/tde_seps` directory, then update the password using the following syntax:

```
ADMINISTER KEY MANAGEMENT UPDATE SECRET 'new-Oracle_Key_Vault_password'
FOR CLIENT 'Oracle_Key_Vault_password' TO [LOCAL] AUTO_LOGIN KEYSTORE
'WALLET_ROOT/tde_seps';
```

Then you can open the connection to Oracle Key Vault as follows:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
[FORCE KEYSTORE] IDENTIFIED BY EXTERNAL STORE CONTAINER = ALL;
```

6.1.4 Configuring an External Store for a Keystore Password

An external store for a keystore password stores the keystore password in a centrally accessed and managed location.

- [About Configuring an External Store for a Keystore Password](#)
An external store for a keystore password allows you to easily remove that keystore password from the ADMINISTER KEY MANAGEMENT command line.
- [Configuring the External Keystore Password Store with WALLET_ROOT](#)
When you configure TDE by using the WALLET_ROOT parameter, the external keystore password store is auto-discovered in the WALLET_ROOT/tde_seps directory.
- [When to Use the EXTERNAL STORE Clause After Configuration](#)
After you configure the external store for a keystore password, you can use the EXTERNAL_STORE clause in the ADMINISTER KEY MANAGEMENT statement.

6.1.4.1 About Configuring an External Store for a Keystore Password

An external store for a keystore password allows you to easily remove that keystore password from the ADMINISTER KEY MANAGEMENT command line.

This feature implements separation of duties between database administrators and key administrators. It is also useful for situations in which you use automated tools to perform Transparent Data Encryption operations that require a password, when the scripts that run the automated tools include hard-coded password. To avoid hard-coding the password in a script, you can store this password in an external store on the database server. In a multitenant environment, different PDBs can make use of the external store.

In a multitenant environment, all PDBs in united mode use the hidden password of the root container. In isolated mode, each PDB can have its own keystore password in its own external store.

Related Topics

- [Storing Oracle Database Secrets](#)
Secrets are data that support internal Oracle Database features that integrate external clients such as Oracle GoldenGate into the database.

6.1.4.2 Configuring the External Keystore Password Store with WALLET_ROOT

When you configure TDE by using the WALLET_ROOT parameter, the external keystore password store is auto-discovered in the WALLET_ROOT/tde_seps directory.

- Run the ADMINISTER KEY MANAGEMENT statement by using the following syntax:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'keystore_password'  
FOR CLIENT 'TDE_WALLET|OKV_PASSWORD'  
TO [LOCAL] AUTO_LOGIN KEYSTORE 'WALLET_ROOT/tde_seps';
```

Related Topics

- [When to Use the EXTERNAL STORE Clause After Configuration](#)
After you configure the external store for a keystore password, you can use the EXTERNAL_STORE clause in the ADMINISTER KEY MANAGEMENT statement.

6.1.4.3 When to Use the EXTERNAL STORE Clause After Configuration

After you configure the external store for a keystore password, you can use the EXTERNAL_STORE clause in the ADMINISTER KEY MANAGEMENT statement.

You must use the `EXTERNAL STORE` clause in the `ADMINISTER KEY MANAGEMENT` statement for the following operations: opening, closing, backing up the keystore; adding, updating, or deleting a secret keystore; creating, using, rekeying, tagging, importing, exporting encryption keys.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY EXTERNAL STORE;
```

You can change or delete external keystore passwords by using the `ADMINISTER KEY MANAGEMENT UPDATE CLIENT SECRET` statement or the `ADMINISTER KEY MANAGEMENT DELETE CLIENT SECRET` statement.

6.1.5 Backing Up Password-Protected Software Keystores

When you back up a password-protected software keystore, you can create a backup identifier string to describe the backup type.

- [About Backing Up Password-Protected TDE Wallets](#)
You must back up password-protected TDE wallets, as per the security policy and requirements of your site.
- [Creating a Backup Identifier String for the Backup TDE Wallet](#)
The backup file name of a software password wallet is derived from the name of the password-protected TDE wallet.
- [Backing Up a Password-Protected TDE Wallet](#)
The `BACKUP KEYSTORE` clause of the `ADMINISTER KEY MANAGEMENT` statement backs up a password-protected TDE wallet.

6.1.5.1 About Backing Up Password-Protected TDE Wallets

You must back up password-protected TDE wallets, as per the security policy and requirements of your site.

A backup of the TDE wallet contains all of the keys contained in the original TDE wallet. Oracle Database prefixes the backup TDE wallet with the creation time stamp (UTC). If you provide an identifier string, then this string is inserted between the time stamp and TDE wallet name.

After you complete the backup operation, the keys in the original TDE wallet are marked as "backed up". You can check the status of keys querying the `V$ENCRYPTION_WALLET` data dictionary view.

You cannot back up auto-login or local auto-login TDE wallets. No new keys can be added to them directly through the `ADMINISTER KEY MANAGEMENT` statement operations. The information in these TDE wallets is only read and hence there is no need for a backup.

You must include the `WITH BACKUP` clause in any `ADMINISTER KEY MANAGEMENT` statement that changes the wallet (for example, changing the wallet password, or setting the master encryption key).

6.1.5.2 Creating a Backup Identifier String for the Backup TDE Wallet

The backup file name of a software password wallet is derived from the name of the password-protected TDE wallet.

- To create a backup identifier string for a backup TDE wallet, use the `ADMINISTER KEY MANAGEMENT SQL` statement with the `BACKUP KEYSTORE` clause, with the following syntax:

```
ewallet_creation-time-stamp-in-UTC_user-defined-string.p12
```

When you create the backup identifier (*user_defined_string*), use the operating system file naming convention. For example, in UNIX systems, you may want to ensure that this setting does not have spaces.

The following example shows the creation of a backup TDE wallet that uses a user-identified string, and how the resultant TDE wallet appears in the file system. This example includes the `FORCE KEYSTORE` clause in the event the auto-login TDE wallet is in use or the TDE wallet is closed.

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE USING 'Monthly-backup-2013-04'
FORCE KEYSTORE
IDENTIFIED BY TDE_wallet_password;
```

This version is for a scenario in which the password is stored in an external store:

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE USING 'Monthly-backup-2013-04'
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE;
```

Resultant TDE wallet file:

```
ewallet_2013041513244657_Monthly-backup-2013-04.p12
```

6.1.5.3 Backing Up a Password-Protected TDE Wallet

The `BACKUP KEYSTORE` clause of the `ADMINISTER KEY MANAGEMENT` statement backs up a password-protected TDE wallet.

- Back up the keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
[USING 'backup_identifier']
FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password]
[TO 'keystore_location'];
```

In this specification:

- `USING backup_identifier` is an optional string that you can provide to identify the backup. Enclose this identifier in single quotation marks (' '). This identifier is appended to the named keystore file (for example, `ewallet_time-stamp_emp_key_backup.p12`).
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` can be one of the following settings:
 - * `EXTERNAL STORE` uses the keystore password stored in the external store to perform the keystore operation.
 - * `TDE_wallet_password` is the password for the keystore.
- `keystore_location` is the path at which the backup keystore is stored. If you do not specify the `keystore_location`, then the backup is created in the same directory as the original keystore. Enclose this location in single quotation marks (' ').

The following example backs up a TDE wallet into another location.

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
USING 'hr.emp_wallet'
FORCE KEYSTORE
```

```
IDENTIFIED BY TDE_wallet_password  
TO '/etc/ORACLE/KEYSTORE/DB1/';
```

keystore altered.

In the following version, the password for the TDE wallet is external, so the EXTERNAL STORE clause is used. The TDE wallet is backed up into the same directory as the current TDE wallet.

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE  
USING 'hr.emp_wallet'  
FORCE KEYSTORE  
IDENTIFIED BY EXTERNAL STORE;
```

After you run this statement, an `ewallet_identifier.p12` file (for example, `ewallet_timestamp_hr.emp_wallet.p12`) appears in the keystore location.

6.1.6 How the V\$ENCRYPTION_WALLET View Interprets Backup Operations

The BACKUP column of the V\$ENCRYPTION_WALLET view indicates how a copy of the keystore was created.

The column indicates if a copy of the keystore had been created with the WITH BACKUP clause of the ADMINISTER KEY MANAGEMENT statement or the ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE statement.

When you modify a key or a secret, the modifications that you make do not exist in the previously backed-up copy, because you make a copy and then modify the key itself. Because there is no copy of the modification in the previous keystores, the BACKUP column is set to NO, even if the BACKUP had been set to YES previously. Hence, if the BACKUP column is YES, then after you perform an operation that requires a backup, such as adding a custom attribute tag, the BACKUP column value changes to NO.

6.1.7 Backups of the External Keystore

You cannot use Oracle Database to back up external keystores.

Uploading TDE wallets into Oracle Key Vault is another way of backing up the wallet and having it available immediately if the need arises (for example after accidental deletion of the wallet, or file corruption). If the database is not migrated to online key management with Oracle Key Vault, then it keeps relying on the TDE wallet, even if the wallet has been uploaded into Oracle Key Vault.

You can use the Oracle Key Vault `okvutil upload` and `okvutil download` commands to upload and download TDE wallets to and from Oracle Key Vault.

For example, to upload a TDE wallet to Oracle Key Vault:

```
$ okvutil upload -l "/etc/oracle/wallets" -t wallet -g "HRWallet"  
Enter wallet password (<enter> for auto-login): password  
Enter Oracle Key Vault endpoint password: Key_Vault_endpoint_password
```

This example shows how to download a TDE wallet from Oracle Key Vault:

```
$ okvutil download -l "/etc/oracle/wallets/orcl/" -t WALLET -g HRWallet
Enter new wallet password(<enter> for auto-login): Oracle_wallet_password
Confirm new wallet password: Oracle_wallet_password
Enter Oracle Key Vault endpoint password: Key_Vault_endpoint_password
```

Related Topics

- [Oracle Key Vault Administrator's Guide](#)

6.1.8 Merging TDE Wallets

You can merge TDE wallets in a variety of ways.

- [About Merging TDE Wallets](#)
You can merge any combination of TDE wallets, but the merged keystore must be password-protected. It can have a password that is different from the constituent wallets.
- [Merging One TDE Wallet into an Existing TDE Wallet](#)
You can use the ADMINISTER KEY MANAGEMENT statement with the MERGE KEYSTORE clause to merge one TDE wallet into another existing TDE wallet.
- [Merging Two TDE Wallets into a Third New TDE Wallet](#)
You can merge two TDE wallets into a third new TDE wallet. The two existing source TDE wallets are not changed.
- [Merging an Auto-Login TDE Wallet into an Existing Password-Protected TDE Wallet](#)
You can merge an auto-login TDE wallet into an existing password-protected TDE wallet.
- [Reversing a TDE Wallet Merge Operation](#)
You cannot directly reverse a TDE wallet merge operation.

6.1.8.1 About Merging TDE Wallets

You can merge any combination of TDE wallets, but the merged keystore must be password-protected. It can have a password that is different from the constituent wallets.

To use the merged TDE wallet, you must explicitly open the merged TDE wallet after you create it, even if one of the constituent TDE wallets was already open before the merge.

Whether a common key from two source TDE wallets is added or overwritten to a merged TDE wallet depends on how you write the ADMINISTER KEY MANAGEMENT merge statement. For example, if you merge TDE wallet 1 and TDE wallet 2 to create TDE wallet 3, then the key in TDE wallet 1 is added to TDE wallet 3. If you merge TDE wallet 1 into TDE wallet 2, then the common key in TDE wallet 2 is not overwritten.

The ADMINISTER KEY MANAGEMENT merge statement has no bearing on the configured TDE wallet that is in use. However, the merged TDE wallet can be used as the new configured database TDE wallet if you want. Remember that you must reopen the TDE wallet if you are using the newly created TDE wallet as the TDE wallet for the database at the location configured by the WALLET_ROOT parameter.

Related Topics

- [Migrating from a TDE Wallet to Oracle Key Vault](#)
You can migrate between password-protected TDE wallets and external keystores in Oracle Key Vault.

6.1.8.2 Merging One TDE Wallet into an Existing TDE Wallet

You can use the `ADMINISTER KEY MANAGEMENT` statement with the `MERGE KEYSTORE` clause to merge one TDE wallet into another existing TDE wallet.

- To perform this type of merge, use the following SQL statement:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE 'TDE_wallet1_location'  
[IDENTIFIED BY TDE_wallet1_password]  
INTO EXISTING KEYSTORE 'TDE_wallet2_location'  
IDENTIFIED BY TDE_wallet2_password  
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *TDE_wallet1_location* is the directory location of the first TDE wallet, which will be left unchanged after the merge. Enclose this path in single quotation marks (' ').
- The `IDENTIFIED BY` clause is required for the first TDE wallet if it is a password-protected TDE wallet. *TDE_wallet1_password* is the password for the first TDE wallet.
- *TDE_wallet2_location* is the directory location of the second TDE wallet into which the first TDE wallet is to be merged. Enclose this path in single quotation marks (' ').
- *TDE_wallet2_password* is the password for the second keystore.
- `WITH BACKUP` creates a backup of the TDE wallet. Optionally, you can use the `USING` clause to add a brief description of the backup. Enclose this description in single quotation marks (' '). This identifier is appended to the named TDE wallet file (for example, *ewallet_time_stamp_emp_key_backup.p12*, with *emp_key_backup* being the backup identifier). Follow the file naming conventions that your operating system uses.

The resultant TDE wallet after the merge operation is always a password-protected TDE wallet.

6.1.8.3 Merging Two TDE Wallets into a Third New TDE Wallet

You can merge two TDE wallets into a third new TDE wallet. The two existing source TDE wallets are not changed.

- Merge the TDE wallets by using the following syntax:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE 'TDE_wallet1_location'  
[IDENTIFIED BY TDE_wallet1_password]  
AND KEYSTORE 'TDE_wallet2_location'  
[IDENTIFIED BY TDE_wallet2_password]  
INTO NEW KEYSTORE 'TDE_wallet3_location'  
IDENTIFIED BY TDE_wallet3_password;
```

In this specification:

- *TDE_wallet1_location* is the directory location of the first TDE wallet, which will be left unchanged after the merge. Enclose this path in single quotation marks (' ').
- The `IDENTIFIED BY` clause is required for the first TDE wallet if it is a password-protected TDE wallet. *TDE_wallet1_password* is the current password for the first TDE wallet.
- *TDE_wallet2_location* is the directory location of the second TDE wallet. Enclose this path in single quotation marks (' ').

- The IDENTIFIED BY clause is required for the second TDE wallet if it is a password-protected TDE wallet. *TDE_wallet2_password* is the current password for the second TDE wallet.
- *TDE_wallet3_location* specifies the directory location of the new, merged TDE wallet. Enclose this path in single quotation marks (' '). If there is already an existing TDE wallet at this location, the command exits with an error.
- *TDE_wallet3_password* is the new password for the merged TDE wallet.

The following example merges an auto-login TDE wallet with a password-protected TDE wallet to create a merged password-protected TDE wallet at a new location:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE '/etc/ORACLE/KEYSTORE/DB1'
AND KEYSTORE '/etc/ORACLE/KEYSTORE/DB2'
IDENTIFIED BY existing_password_for_keystore_2
INTO NEW KEYSTORE '/etc/ORACLE/KEYSTORE/DB3'
IDENTIFIED BY new_password_for_keystore_3;
```

keystore altered.

6.1.8.4 Merging an Auto-Login TDE Wallet into an Existing Password-Protected TDE Wallet

You can merge an auto-login TDE wallet into an existing password-protected TDE wallet.

- Use the ADMINISTER KEY MANAGEMENT MERGE KEYSTORE SQL statement to merge an auto-login TDE wallet into an existing password-protected TDE wallet.

The following example shows how to merge an auto-login TDE wallet into a password-protected TDE wallet. It also creates a backup of the second TDE wallet before creating the merged TDE wallet.

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE '/etc/ORACLE/KEYSTORE/DB1'
INTO EXISTING KEYSTORE '/etc/ORACLE/KEYSTORE/DB2'
IDENTIFIED BY keystore_password WITH BACKUP;
```

In this specification:

- MERGE KEYSTORE must specify the auto-login TDE wallet.
- EXISTING KEYSTORE refers to the password TDE wallet.

6.1.8.5 Reversing a TDE Wallet Merge Operation

You cannot directly reverse a TDE wallet merge operation.

When you merge a TDE wallet into an existing TDE wallet (rather than creating a new one), you must include the WITH BACKUP clause in the ADMINISTER KEY MANAGEMENT statement to create a backup of this existing TDE wallet. Later on, if you decide that you must reverse the merge, you can replace the merged TDE wallet with the one that you backed up. In other words, suppose you want merge TDE wallet A into TDE wallet B. By using the WITH BACKUP clause, you create a backup for TDE wallet B before the merge operation begins. (The original TDE wallet A is still intact.) To reverse the merge operation, revert to the backup that you made of TDE wallet B.

- Use the ADMINISTER KEY MANAGEMENT MERGE KEYSTORE SQL statement to perform merge operations.
 - For example, to perform a merge operation into an existing TDE wallet:


```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE '/etc/ORACLE/KEYSTORE/DB1'  
INTO EXISTING KEYSTORE '/etc/ORACLE/KEYSTORE/DB2'  
IDENTIFIED BY password WITH BACKUP USING "merge1";
```

Replace the new TDE wallet with the backup TDE wallet, which in this case would be named `ewallet_time-stamp_merge1.p12`.

- To merge an auto-login TDE wallet into a password-based TDE wallet, use the `ADMINISTER KEY MANAGEMENT MERGE KEYSTORE` SQL statement.

6.1.9 Moving a TDE Wallet to a New Location

You move a TDE wallet to a new location after you have updated the `WALLET_ROOT` parameter.

If you are using Oracle Key Vault, then you can configure a TDE direct connection where Key Vault directly manages the master encryption keys. In this case, you will never need to manually move the TDE wallet to a new location.

1. Connect to the united mode CDB root or isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.

2. Back up the TDE wallet.

For example:

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE  
USING 'hr.emp_keystore'  
FORCE KEYSTORE  
IDENTIFIED BY  
TDE_wallet_password TO '/etc/ORACLE/KEYSTORE/DB1/';
```

3. Close the TDE wallet.

Examples of ways that you can close the TDE wallet are as follows.

For an auto-login TDE wallet:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE;
```

For a password-protected TDE wallet:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY TDE_wallet_password;
```

For a TDE wallet for which the password is stored externally:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY EXTERNAL STORE;
```

4. Exit the database session.

For example, if you are logged in to SQL*Plus:

```
EXIT
```

5. In the `init.ora` file for the database instance, update the `WALLET_ROOT` parameter to point to the new location where you want to move the TDE wallet.
6. Use the operating system move command (such as `mv`) to move the TDE wallet with all of its keys to the new directory location.

Related Topics

- [Oracle Key Vault Administrator's Guide](#)

6.1.10 Moving a TDE Wallet Out of Automatic Storage Management

You can use the `ADMINISTER KEY MANAGEMENT` statement to move a TDE wallet out Automatic Storage Management.

1. Connect to the united mode CDB root or isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Initialize a target TDE wallet on the file system by using the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE target_TDE_wallet_path
IDENTIFIED BY target_TDE_wallet_password;
```

In this specification:

- *target_TDE_wallet_path* is the directory path to the target TDE wallet on the file system.
- *target_TDE_wallet_password* is a password that you create for the TDE wallet.

For example:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE '/etc/ORACLE/KEYSTORE/DB1/' IDENTIFIED BY
"target_TDE_wallet_password";
```

3. Copy the TDE wallet from ASM to the target TDE wallet that you just created.

This step requires that you merge the TDE wallet from ASM to the file system, as follows:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE source_TDE_wallet_path
IDENTIFIED BY source_TDE_wallet_password
INTO EXISTING KEYSTORE target_TDE_wallet_path
IDENTIFIED BY target_TDE_wallet_password
WITH BACKUP USING backupIdentifier;
```

In this specification:

- *source_TDE_wallet_path* is the directory path to the source TDE wallet.
- *source_TDE_wallet_password* is the source TDE wallet password.
- *target_TDE_wallet_path* is the path to the target TDE wallet.
- *target_TDE_wallet_password* is the target TDE wallet password.
- *backupIdentifier* is the backup identifier to be added to the backup file name.

For example:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE '+DATAFILE'
IDENTIFIED BY "source_TDE_wallet_password"
INTO EXISTING KEYSTORE '/etc/ORACLE/KEYSTORE/DB1/'
IDENTIFIED BY "target_TDE_wallet_password"
WITH BACKUP USING "bkup";
```

6.1.11 Migrating from a TDE Wallet to Oracle Key Vault

You can migrate between password-protected TDE wallets and external keystores in Oracle Key Vault.

- [Migrating from a Password-Protected TDE Wallet to an External Keystore](#)
You can migrate from a password-protected TDE wallet to an external keystore.

- [Migrating from an External Keystore to a Password-Based TDE Wallet](#)
You can migrate an external keystore to a TDE wallet.
- [Keystore Order After a Migration](#)
After you perform a migration, keystores can be either primary or secondary in their order.

6.1.11.1 Migrating from a Password-Protected TDE Wallet to an External Keystore

You can migrate from a password-protected TDE wallet to an external keystore.

- [Step 1: Convert the TDE Wallet to Open with the External Keystore](#)
Some Oracle tools require access to the old TDE wallet to encrypt or decrypt data that was exported or backed up using the TDE wallet.
- [Step 2: Configure the External Keystore Type](#)
You can use the ALTER SYSTEM statement to configure the external keystore type.
- [Step 3: Perform the External Keystore Migration](#)
You can use the ADMINISTER KEY MANAGEMENT SQL statement to perform an external keystore migration.

6.1.11.1.1 Step 1: Convert the TDE Wallet to Open with the External Keystore

Some Oracle tools require access to the old TDE wallet to encrypt or decrypt data that was exported or backed up using the TDE wallet.

Examples of these tools are Oracle Data Pump and Oracle Recovery Manager.

- Use the ADMINISTER KEY MANAGEMENT SQL statement to convert a TDE wallet to open with an external keystore.
 - To set the TDE wallet password as that of the external keystore, use the following syntax:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
FORCE KEYSTORE
IDENTIFIED BY TDE_wallet_password
SET "external_key_manager_password" WITH BACKUP
[USING 'backup_identifier'];
```

In this specification:

- * *TDE_wallet_password* is the same password that you used when creating the TDE wallet.
- * *external_key_manager_password* is the new TDE wallet password which is the same as the password of the external keystore.
- * WITH BACKUP creates a backup of the TDE wallet. Optionally, you can use the USING clause to add a brief description of the backup. Enclose this description in single quotation marks (' '). This identifier is appended to the named TDE wallet file (for example, ewallet_time-stamp_emp_key_backup.p12, with emp_key_backup being the backup identifier). Follow the file naming conventions that your operating system uses.

- To create an auto-login TDE wallet for a TDE wallet, use the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE [LOCAL] AUTO_LOGIN KEYSTORE
FROM KEYSTORE 'keystore_location'
IDENTIFIED BY TDE_wallet_password;
```

In this specification:

- * LOCAL enables you to create a local auto-login TDE wallet. Otherwise, omit this clause if you want the TDE wallet to be accessible by other computers.
- * *TDE_wallet_location* is the path to the TDE wallet directory location of the wallet that is configured in the `sqlnet.ora` file.
- * *TDE_wallet_password* is the existing password of the configured TDE wallet.

6.1.11.1.2 Step 2: Configure the External Keystore Type

You can use the `ALTER SYSTEM` statement to configure the external keystore type.

For the TDE wallet to open with the external keystore, either the TDE wallet must have the same password as the external keystore, or alternatively, you can create an auto-login TDE wallet for the TDE wallet.

1. Connect to the united mode CDB root or isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Set the `TDE_CONFIGURATION` dynamic initialization parameter.

This example migrates the database from a TDE wallet to Oracle Key Vault.

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=OKV|FILE" SCOPE = "BOTH"
SID = "*";
```

6.1.11.1.3 Step 3: Perform the External Keystore Migration

You can use the `ADMINISTER KEY MANAGEMENT SQL` statement to perform an external keystore migration.

To migrate from the TDE wallet to external keystore, you must use the `MIGRATE USING external_key_manager_password` clause in the `ADMINISTER KEY MANAGEMENT SET KEY SQL` statement to decrypt the existing TDE table keys and tablespace encryption keys with the TDE master encryption key in the TDE wallet and then reencrypt them with the newly created TDE master encryption key in the external keystore. After you complete the migration, you do not need to restart the database, nor do you need to manually re-open the external keystore. The migration process automatically reloads the keystore keys in memory.

- Migrate the external keystores by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
IDENTIFIED BY "external_key_manager_password"
MIGRATE USING TDE_wallet_password
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *external_key_manager_password* is the password that was created when the external keystore was created. Enclose this setting in double quotation marks (" ").
- *TDE_wallet_password* is the same password that you used when you created the TDE wallet or that you have changed to (when converting a TDE wallet to open with an external keystore).
- `USING` enables you to add a brief description of the backup. Enclose this description in single quotation marks (' '). This identifier is appended to the named keystore file (for example, `ewallet_time-stamp_emp_key_backup.p12`, with *emp_key_backup* being the backup identifier). Follow the file naming conventions that your operating system uses.

6.1.11.2 Migrating from an External Keystore to a Password-Based TDE Wallet

You can migrate an external keystore to a TDE wallet.

- [About Migrating Back from an External Keystore](#)
To switch from using an external keystore solution to a TDE wallet, you can use reverse migration of the TDE wallet.
- [Step 1: Configure the External Keystore Type](#)
You can use the ALTER SYSTEM statement to configure the external keystore type.
- [Step 2: Configure the Keystore for the Reverse Migration](#)
The ADMINISTER KEY MANAGEMENT statement with the SET ENCRYPTION KEY and REVERSE MIGRATE clauses can be used to reverse the migration of a keystore.
- [Step 3: Configure the External Keystore to Open with the TDE Wallet](#)
After you complete the migration, the migration process automatically reloads the keystore keys in memory.

6.1.11.2.1 About Migrating Back from an External Keystore

To switch from using an external keystore solution to a TDE wallet, you can use reverse migration of the TDE wallet.

After you complete the switch, keep the external keystore, in case earlier backup files rely on the TDE master encryption keys in the external key manager.

If you had originally migrated from the TDE wallet to the external keystore and reconfigured the TDE wallet, then you already have an existing TDE wallet with the same password as the external keystore password. Reverse migration configures this keystore to act as the new TDE wallet with a new password. If your existing TDE wallet is an auto-login TDE wallet and you have the password-based TDE wallet for this auto-login TDE wallet, then use the password-based TDE wallet. If the password-based TDE wallet is not available, then merge the auto-login TDE wallet into a newly created empty password-based TDE wallet, and use the newly created password-based TDE wallet.

If you do not have an existing TDE wallet, then you must specify a TDE wallet location using the WALLET_ROOT parameter in the init.ora file. When you perform the reverse migration, migrate to the previous TDE wallet so that you do not lose the keys.

Related Topics

- [Merging TDE Wallets](#)
You can merge TDE wallets in a variety of ways.
- [Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS](#)
To switch from a TDE wallet to centralized key management with Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS), after you upload all current and retired TDE master keys you must migrate the database from the TDE wallet to Oracle Key Vault or OCI KMS.

6.1.11.2.2 Step 1: Configure the External Keystore Type

You can use the ALTER SYSTEM statement to configure the external keystore type.

1. Connect to the united mode CDB root or isolated mode PDB as a user who has been granted the ALTER SYSTEM privilege.
2. Set the TDE_CONFIGURATION dynamic initialization parameter to specify the TDE keystore type.

For example:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=OKV" SCOPE = BOTH SID =
'*';
```

Setting `KESTORE_CONFIGURATION` to "OKV" indicates a configuration where the connection into Oracle Key Vault must be opened by providing the Oracle Key Vault password. To use an auto-open Oracle Key Vault configuration, you set `KESTORE_CONFIGURATION` to "OKV|FILE", where the Oracle Key Vault password is stored in an auto-open keystore in `WALLET_ROOT/tde`.

3. Restart the database (for the CDB root) or close and re-open the PDB.

To restart from the CDB root:

```
SHUTDOWN IMMEDIATE
STARTUP
```

To close and re-open a PDB:

```
ALTER PLUGGABLE DATABASE pdb_name CLOSE IMMEDIATE;
ALTER PLUGGABLE DATABASE pdb_name OPEN;
```

6.1.11.2.3 Step 2: Configure the Keystore for the Reverse Migration

The `ADMINISTER KEY MANAGEMENT` statement with the `SET ENCRYPTION KEY` and `REVERSE MIGRATE` clauses can be used to reverse the migration of a keystore.

1. Connect to the united mode CDB root or isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Reverse migrate the keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
IDENTIFIED BY TDE_wallet_password
REVERSE MIGRATE USING "external_key_manager_password"
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *TDE_wallet_password* is the password for the existing keystore or the new keystore.
- *external_key_manager_password* is the password that was created when you first created the external keystore. If the pre-external TDE wallet is the new keystore, then you must ensure that it has the same password as the *external_key_manager_password* before issuing the reverse migration command. Enclose this setting in double quotation marks (" ").
- `WITH BACKUP` creates a backup of the TDE wallet. Optionally, you can include the `USING` clause to add a brief description of the backup. Enclose this description in single quotation marks (' '). This identifier is appended to the named keystore file (for example, *ewallet_time-stamp_emp_key_backup.p12*, with *emp_key_backup* being the backup identifier). Follow the file naming conventions that your operating system uses.

For example:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
IDENTIFIED BY TDE_wallet_password
REVERSE MIGRATE USING "external_key_manager_password" WITH BACKUP;
```

keystore altered.

3. Optionally, change the keystore password.

For example:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
IDENTIFIED BY old_TDE_wallet_password
SET new_TDE_wallet_password
WITH BACKUP USING 'before_password_was_changed';
```

6.1.11.2.4 Step 3: Configure the External Keystore to Open with the TDE Wallet

After you complete the migration, the migration process automatically reloads the keystore keys in memory.

You do not need to restart the database, nor do you need to manually re-open the TDE wallet.

The external keystore may still be required after reverse migration because the old keys are likely to have been used for Oracle Data Pump Export and Oracle Recovery Manager (Oracle RMAN)-encrypted backups. You should add the external keystore credentials to the keystore so that the HSM can be opened with the TDE wallet.

Related Topics

- [Configuring Auto-Open Connections into External Key Managers](#)
An external key manager can be configured to use the auto-login capability.

6.1.11.3 Keystore Order After a Migration

After you perform a migration, keystores can be either primary or secondary in their order.

The `WALLET_ORDER` column of the `V$ENCRYPTION_WALLET` dynamic view describes whether a TDE wallet is primary (that is, it holds the current TDE master encryption key) or if it is secondary (it holds the previous TDE master encryption key). The `WRL_TYPE` column describes the type of locator for the TDE wallet (for example, `FILE` for the `sqlnet.ora` file). The `WALLET_ORDER` column shows `SINGLE` if two TDE wallets are not configured together and no migration was ever performed previously.

[Table 6-1](#) describes how the keystore order works after you perform a migration.

Table 6-1 Keystore Order After a Migration

Type of Migration Done	WRL_TYPE	WALLET_ORDER	Description
Migration of TDE wallet to external keystore	OKV	PRIMARY	Both the external and TDE wallet are configured. The TDE master encryption key can be either in Oracle Key Vault or the TDE wallet. The TDE master encryption key is first searched in Oracle Key Vault. If the TDE master encryption key is not in the primary keystore (Oracle Key Vault), then it will be searched for in the TDE wallet. All of the new TDE master encryption keys will be created in the primary keystore (in this case, Oracle Key Vault).
	FILE	SECONDARY	

Table 6-1 (Cont.) Keystore Order After a Migration

Type of Migration Done	WRL_TYPE	WALLET_ORDER	Description
Reverse migration from Oracle Key Vault (HSM) to TDE wallet	FILE HSM	PRIMARY SECONDARY	Both the external and TDE wallet are configured. The TDE master encryption key can be either in the external keystore or the TDE wallet. The TDE master encryption key is first searched for in the TDE wallet. If the TDE master encryption key is not present in the primary (that is, software) TDE wallet, then it will be searched for in the HSM's external keystore. All of the new TDE master encryption keys will be created in the primary keystore (in this case, the TDE wallet).

6.1.12 Migration of Keystores to and from Oracle Key Vault

You can use Oracle Key Vault to migrate both TDE wallets and external keystores to and from Oracle Key Vault.

This enables you to manage the keystores centrally, and then share the keystores as necessary with other TDE-enabled databases in your enterprise.

Oracle Key Vault enables you to upload a keystore to a container called a virtual wallet, and then create a new virtual wallet from the contents of previously uploaded keystore. For example, suppose you previously uploaded a keystore that contains 5 keys. You can create a new virtual wallet that consists of only 3 of these keys. You then can download this keystore to another TDE-enabled database. This process does not modify the original keystore.

In addition to Oracle keystores, Oracle Key Vault enables you to securely share other security objects, such as credential files and Java keystores, across the enterprise. It prevents the loss of keys and keystores due to forgotten passwords or accidentally deleted keystores. You can use Oracle Key Vault with products other than TDE: Oracle Real Application Security, Oracle Active Data Guard, and Oracle GoldenGate. Oracle Key Vault facilitates the movement of encrypted data using Oracle Data Pump and Oracle Transportable Tablespaces.

Related Topics

- [Oracle Key Vault Administrator's Guide](#)

6.1.13 Configuring Keystores for Automatic Storage Management

You can store a TDE wallet on an Automatic Storage Management (ASM) disk group.

- [About Configuring Keystores for Automatic Storage Management](#)
You can configure a TDE wallet for Automatic Storage Management (ASM) for a standalone database or a multitenant environment. The `WALLET_ROOT` location can be compliant or non-compliant with Oracle Managed File (OMF) systems.
- [Configuring a Keystore on a Standalone Database to Point to an ASM Location](#)
On a standalone database system, you can set the `WALLET_ROOT` parameter to point to an ASM location.

- [Configuring a Keystore to Point to an ASM Location](#)
You can set `WALLET_ROOT` to point to an ASM directory within which the TDE wallet of the CDB root (which all united mode PDBs share) and the TDE wallets of all isolated mode PDBs are located.
- [Configuring a Keystore to Point to an ASM Location When the `WALLET\_ROOT` Location Does Not Follow OMF Guidelines](#)
If the chosen `WALLET_ROOT` location does not comply with the Oracle Managed File (OMF) guidelines, then the Oracle database cannot perform automation of the directory creation.

6.1.13.1 About Configuring Keystores for Automatic Storage Management

You can configure a TDE wallet for Automatic Storage Management (ASM) for a standalone database or a multitenant environment. The `WALLET_ROOT` location can be compliant or non-compliant with Oracle Managed File (OMF) systems.

You should use the `WALLET_ROOT` and `TDE_CONFIGURATION` initialization parameters to configure the TDE wallet location in an ASM system. The `TDE_CONFIGURATION` parameter must be set with the attribute `KEystore_CONFIGURATION=FILE` in order for the `WALLET_ROOT` parameter to work. Note that starting with Oracle Database release 19c, the `ENCRYPTION_WALLET_LOCATION`, set in the `sqlnet.ora` file, is deprecated in favor of `WALLET_ROOT` and `TDE_CONFIGURATION`.

To perform the configuration, you must specify a + sign, followed by the ASM disk group and path where the TDE wallet will be located. For example:

```
WALLET_ROOT=+disk_group/path
```

Note the following:

- When you open a local TDE wallet, it opens only on the ASM node on which it was created.
- When you designate the path for the `WALLET_ROOT` for databases in standalone or multitenant environments, or environments where the `WALLET_ROOT` location either complies or does not comply with the Oracle Managed File (OMF) directory naming convention, be aware that this path must follow certain conventions so that the database can automate the creation of the directory components of the TDE wallet locations for you. Otherwise, you must manually create the directories under the `WALLET_ROOT` location.
- If you must move or merge TDE wallets between a regular file system and an ASM file system, then you can use the same TDE wallet merge statements that are used to merge TDE wallets.
- To run commands to manage TDE wallets in an ASM environment, you can use the `ASMCMD` utility.

Related Topics

- [Merging TDE Wallets](#)
You can merge TDE wallets in a variety of ways.
- *Oracle Automatic Storage Management Administrator's Guide*

6.1.13.2 Configuring a Keystore on a Standalone Database to Point to an ASM Location

On a standalone database system, you can set the `WALLET_ROOT` parameter to point to an ASM location.

1. Ensure that the `KESTORE_CONFIGURATION` attribute of the `TDE_CONFIGURATION` dynamic initialization parameter is set to `FILE`.

For example, in SQL*Plus:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE";
```

2. In the `init.ora` file, set the `WALLET_ROOT` static initialization parameter to the ASM disk group location.

Optionally, include `OMF` (in upper case) to provide Oracle Managed File (OMF) compliance and to facilitate automation of the directory creation.

For example:

```
WALLET_ROOT=+disk_group_name/OMF
```

This setting places the TDE keystore of the standalone database system in the `WALLET_ROOT/tde` directory (that is, `+disk_group_name/OMF/tde` with the `tde` component of the path being automatically created by the database server).

6.1.13.3 Configuring a Keystore to Point to an ASM Location

You can set `WALLET_ROOT` to point to an ASM directory within which the TDE wallet of the CDB root (which all united mode PDBs share) and the TDE wallets of all isolated mode PDBs are located.

1. Ensure that the `KESTORE_CONFIGURATION` attribute of the `TDE_CONFIGURATION` dynamic initialization parameter is set to `FILE`.

For a CDB, set `TDE_CONFIGURATION` in the CDB root; for an isolated PDB, set it in the PDB.

For example, in SQL*Plus:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE";
```

2. Set the `WALLET_ROOT` static initialization parameter to the ASM disk group location followed by the `DB_UNIQUE_NAME` initialization parameter value.

The inclusion of the value of `DB_UNIQUE_NAME` is necessary to allow the database server to automate the creation of the necessary directories under this location.

You must not use `OMF` as a directory component of the `WALLET_ROOT` location (unlike in the standalone database configuration section).

For example:

```
WALLET_ROOT=+disk_group_name/db_unique_name
```

This setting locates the TDE wallet that is used by the root and by all of the united mode PDBs in the `WALLET_ROOT/db_unique_name/tde` directory (that is, in `+disk_group_name/db_unique_name/tde`).

This setting locates the TDE wallet which is used by each isolated mode PDB in the `WALLET_ROOT/db_unique_name/pdb_guid/tde` directory (that is, in `+disk_group_name/db_unique_name/pdb_guid/tde`).

6.1.13.4 Configuring a Keystore to Point to an ASM Location When the WALLET_ROOT Location Does Not Follow OMF Guidelines

If the chosen WALLET_ROOT location does not comply with the Oracle Managed File (OMF) guidelines, then the Oracle database cannot perform automation of the directory creation.

In this case, you must use the ALTER DISKGROUP command to manually create the necessary directories under the WALLET_ROOT location. You must use the ALTER DISKGROUP ... ADD DIRECTORY statement to manually create the necessary directories, because no automation of the directory creation is possible when the WALLET_ROOT parameter is not using an OMF-compliant value.

1. Ensure that the KEYSTORE_CONFIGURATION attribute of the TDE_CONFIGURATION dynamic initialization parameter is set to FILE.

For example, in SQL*Plus:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KEYSTORE_CONFIGURATION=FILE";
```

2. In the init.ora file, set the WALLET_ROOT static initialization parameter to the ASM disk group location.

For example, the following path after *disk_group_name* contains no uppercase OMF directory elements:

```
WALLET_ROOT="+disk_group_name/mydir/wallets"
```

3. Connect to the database instance using the SYSASM administrative privilege.

```
connect / as sysasm
```

4. Execute the ALTER DISKGROUP statements to create the necessary directories.

You must perform this step because the database server cannot automate the creation of these directories, since the location chosen for WALLET_ROOT is not compliant with the Oracle Managed Files guideline (that is, it does not have the OMF component included in it in uppercase letters).

For example:

```
ALTER DISKGROUP "disk_group_name" ADD DIRECTORY
'+disk_group_name/mydir/wallets/tde';
```

For a multitenant environment, find the PDB GUID of the PDB that will store the keystore, as follows:

```
SELECT GUID FROM DBA_PDBS WHERE PDB_NAME = 'pdb name';
```

Next, include the PDB GUID in the following ALTER DISKGROUP statements to create the necessary directories for the isolated mode PDB within the WALLET_ROOT location. For example, assuming the GUID is 4756C705E52A8768E053F82DC40A5329:

```
ALTER DISKGROUP "disk_group_name" ADD DIRECTORY
'+disk_group_name/mydir/wallets/4756C705E52A8768E053F82DC40A5329'
```

```
ALTER DISKGROUP "disk_group_name" ADD DIRECTORY
'+disk_group_name/mydir/wallets/4756C705E52A8768E053F82DC40A5329/tde';
```

6.1.14 Closing a Keystore

You can manually close software and external keystores.

- [About Closing Keystores](#)
After you open a keystore, it remains open until you shut down the database instance.
- [Closing a TDE Wallet](#)
You can manually close password-based TDE wallets, auto-login TDE wallets, and local auto-login TDE wallets.
- [Closing an External Keystore](#)
To close an external keystore, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE CLOSE` clause.

6.1.14.1 About Closing Keystores

After you open a keystore, it remains open until you shut down the database instance.

When you restart the database instance, then auto-login and local auto-login TDE wallets automatically open when required (that is, when the TDE master encryption key must be accessed). However, TDE wallet password-based and external keystores do not automatically open. You must manually open them again before you can use them.

When you close a TDE wallet or an external keystore, you disable all of the encryption and decryption operations on the database. Hence, a database user or application cannot perform any operation involving encrypted data until the TDE wallet or keystore is reopened.

When you re-open a TDE wallet or keystore after closing it, its contents are reloaded back into the database. Thus, if the contents had been modified (such as during a migration), the database will have the latest contents.

You can check if a TDE wallet or keystore is closed by querying the `STATUS` column of the `V$ENCRYPTION_WALLET` view.

The following data operations will fail if the TDE wallet or keystore is not accessible:

- `SELECT` data from an encrypted column
- `INSERT` data into on an encrypted column
- `CREATE` a table with encrypted columns
- `CREATE` an encrypted tablespace

6.1.14.2 Closing a TDE Wallet

You can manually close password-based TDE wallets, auto-login TDE wallets, and local auto-login TDE wallets.

In the case of an auto-login TDE wallet, which opens automatically when it is accessed, manually close it if you moved it to a new location. You do this if you are changing your configuration from an auto-login TDE wallet to a password-based TDE wallet: you move out the auto-login TDE wallet, and then close the auto-login TDE wallet.

- Run the `ADMINISTER KEY MANAGEMENT SQL` statement.
 - For a password-based TDE wallet, use the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
[IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password]];
```

In this specification:

- * `IDENTIFIED BY` can be one of the following:

- * `EXTERNAL_STORE` uses the TDE wallet password stored in the external store to perform the TDE wallet operation.
 - * `TDE_wallet_password` is the password of the user who created the TDE wallet.
- For an auto-login or local auto-login TDE wallet, use the following SQL statement:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE;
```

You do not need to specify a password for this statement.

Closing a TDE wallet disables all of the encryption and decryption operations. Any attempt to encrypt or decrypt data or access encrypted data results in an error.

6.1.14.3 Closing an External Keystore

To close an external keystore, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE CLOSE` clause.

1. Log into the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Close the external keystore by using this syntax:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY [EXTERNAL_STORE | "external_key_manager_password"];
```

In this specification:

- `IDENTIFIED BY` can be one of the following settings:
 - `EXTERNAL_STORE` uses the keystore password stored in the external store to perform the keystore operation.
 - `external_key_manager_password` is for an external keystore manager, which can be Oracle Key Vault or OCI Vault - Key Management. Enclose this password in double quotation marks. For Oracle Key Vault, enter the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the `ADMINISTER KEY MANAGEMENT` statement becomes `NULL`.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY "external_key_manager_password";
```

6.1.15 Backup and Recovery of Encrypted Data

For TDE wallets, you cannot access encrypted data without the TDE master encryption key.

Because the TDE master encryption key is stored in the TDE wallet, you should periodically back up the TDE wallet in a secure location. You must back up a copy of the TDE wallet whenever you set a new TDE master encryption key or perform any operation that writes to the TDE wallet.

Do not back up the TDE wallet in the same location as the encrypted data. Back up the TDE wallet separately. This is especially true when you use the auto-login TDE wallet, which does not require a password to open. In case the backup tape is lost, a malicious user should not be able to get both the encrypted data and the TDE wallet.

Oracle Recovery Manager (Oracle RMAN) does not back up the TDE wallet as part of the database backup. When using a media manager such as Oracle Secure Backup with Oracle

RMAN, Oracle Secure Backup automatically excludes auto-open TDE wallets (the `cwallet.sso` files). However, it does not automatically exclude encryption TDE wallets (the `ewallet.p12` files). It is a good practice to add the following `exclude data set` statement to your Oracle Secure Backup configuration:

```
exclude name *.p12
```

This setting instructs Oracle Secure Backup to exclude the encryption TDE wallet from the backup set.

If you lose the TDE wallet that stores the TDE master encryption key, then you can restore access to encrypted data by copying the backed-up version of the TDE wallet to the appropriate location. If you archived the restored TDE wallet after the last time that you reset the TDE master encryption key, then you do not need to take any additional action.

If the restored TDE wallet does not contain the most recent TDE master encryption key, then you can recover old data up to the point when the TDE master encryption key was reset by rolling back the state of the database to that point in time. All of the modifications to encrypted columns after the TDE master encryption key was reset are lost.

Related Topics

- *Oracle Database Backup and Recovery User's Guide*

6.1.16 Dangers of Deleting Keystores

Oracle strongly recommends that you do not delete keystores.

If a keystore becomes overly full, any TDE master encryption key other than the currently active TDE master encryption key can be moved to a new keystore to reduce the overall size of the keystore, but it is important to keep a backup of the old and new keystores because even though the keys have been moved out of the currently active keystore, they may still be needed by other Oracle features, such as Oracle Recovery Manager backup operations. (See Related Topics at the end of this topic for a listing of features that are affected by deleted keystores.)

Deleting a keystore that still contains keys is particularly dangerous if you have configured Transparent Data Encryption and the keystore is in use. You can find if a keystore is in use by querying the `STATUS` column of the `V$ENCRYPTION_WALLET` view after you open the keystore. How you should proceed depends on whether you are using united mode or isolated mode.

- In isolated mode, if the `STATUS` column of the `V$ENCRYPTION_WALLET` is `OPEN_NO_MASTER_KEY`, then it is safe to archive and later delete the this keystore, because there are no keys in it.
- In united mode, you must execute the query of `V$ENCRYPTION_WALLET` from the `CDB$ROOT`, not the `PDB`. If you execute the query in a `PDB` that does not yet have a key set, then the `STATUS` is `OPEN_NO_MASTER_KEY`. However, this can be misleading, because a key could have been set in the `CDB$ROOT`. After you execute the query in the root and if the `STATUS` is `OPEN_NO_MASTER_KEY`, then you can safely archive and later delete the keystore.

The reason that you should be cautious when moving keys out of the currently-active TDE keystore is that this keystore may contain keys that are still needed by the database (even though the TDE master encryption key has been rekeyed). Deleting the keystore deletes these keys, and could result in the loss of encrypted data. Even if you decrypted all of the data in your database, you still should not delete the keystore, because doing so could still hamper the normal functioning of the Oracle database. This is because a TDE master encryption key in the keystore can also be required for other Oracle Database features. (See Related Topics at the end of this topic for a listing of features that are affected by deleted keystores.)

Even after you performed TDE keystore migration (which rekeys in such a way that the location of your currently-active TDE master encryption key changes place between your software keystore and your external keystore), you still should not delete your original keystore. The keys in the original keystore may be needed at a later time (for example, when you must recover an offline encrypted tablespace). Even if all online tablespaces are not encrypted, the key may still be in use.

The exception is in the case of software auto-login (or local auto-login) keystores. If you do not want to use this type of keystore, then ideally you should move it to a secure directory. Only delete an auto-login keystore if you are sure that it was created from a specific password-based keystore because an auto-login keystore is always based on an ordinary software keystore. The keystore should be available and known.

If you must delete a keystore, then do so with great caution. You must first move the keys within the keystore to a new keystore by using the `ADMINISTER KEY MANAGEMENT MOVE KEYS TO NEW KEYSTORE` statement.

Related Topics

- [Features That Are Affected by Deleted Keystores](#)
Some features can be adversely affected if a keystore is deleted and a TDE master encryption key residing in that keystore is later needed.
- [Moving a TDE Master Encryption Key into a New Keystore](#)
The `ADMINISTER KEY MANAGEMENT MOVE KEYS` moves an existing TDE master encryption key into a new keystore from an existing keystore.
- [Moving a TDE Master Encryption Key into a New Keystore in United Mode](#)
In united mode, you can move an existing TDE master encryption key into a new keystore from an existing software password keystore.
- [Moving a TDE Master Encryption Key into a New Keystore in Isolated Mode](#)
In isolated mode, you can move an existing TDE master encryption key into a new TDE wallet from an existing password TDE wallet.

6.1.17 Features That Are Affected by Deleted Keystores

Some features can be adversely affected if a keystore is deleted and a TDE master encryption key residing in that keystore is later needed.

Before you delete a keystore, consider the impact that the deletion will have in the event that you need the any TDE master encryption key in the TDE keystore at a later time. The following features and activities are affected:

- Offlined tablespace operations
- Oracle Secure Backup operations
- Media recovery and block media recovery operations
- Point-in-time recovery operations
- Physical and logical Oracle Data Guard standby operations
- Golden Gate operations
- Oracle Streams operations
- Oracle Recovery Manager operations, including restoring Oracle Recovery Manager backups
- Applying archived redo logs to a database during database crash recovery operations

- Database online block recovery. (Online block recovery implies that the database is still open. Deleting a wallet in an open database with encrypted tablespaces will cause additional problems other than those associated with online block recovery.) These problems can include the following:
 - Encrypted online data in encrypted tablespaces would no longer be decrypted. Encrypted metadata in the SYSTEM, UNDO, and TEMP tablespaces would no longer be decrypted. You will no longer have control over what metadata is encrypted or where that metadata can reside.
 - Buffered data or metadata needs to be encrypted before it can be written back to the disk, but if the wallet is deleted, then the buffered data or metadata would no longer be encrypted. This could cause redo generation to fail, and the DBWR background process would not be able to write the data, which would possibly lead to a database instance hang or crash.
 - After a database instance crash, the database instance recovery and database crash recovery would fail, leading to the database not being able to be restarted.

Related Topics

- [Dangers of Deleting Keystores](#)
Oracle strongly recommends that you do not delete keystores.

6.2 Managing the TDE Master Encryption Key

You can manage the TDE master encryption key in several ways.

- [Creating User-Defined TDE Master Encryption Keys](#)
You can create a user-defined TDE master encryption key outside the database by generating a TDE master encryption key ID.
- [Creating TDE Master Encryption Keys for Later Use](#)
You can create a TDE master encryption key that can be activated at a later date.
- [Activating TDE Master Encryption Keys](#)
After you activate a TDE master encryption key, it can be used.
- [TDE Master Encryption Key Attribute Management](#)
TDE master encryption key attributes store information about the TDE master encryption key.
- [Creating Custom TDE Master Encryption Key Attributes for Reports](#)
Custom TDE master encryption key attributes enable you to define attributes that are specific to your needs.
- [Setting or Rekeying the TDE Master Encryption Key in the Keystore](#)
You can set or rekey the TDE master encryption key for both TDE wallets and external keystores.
- [Exporting and Importing the TDE Master Encryption Key](#)
You can export and import the TDE master encryption key in different ways.
- [Moving TDE Master Encryption Keys into a New Keystore](#)
You can move an existing TDE master encryption key into a new keystore from an existing password-protected keystore.
- [Converting from ENCRYPTION_WALLET_LOCATION to WALLET_ROOT and TDE_CONFIGURATION](#)
You can convert the wallet location from using the `sqlnet.ora` `ENCRYPTION_WALLET_LOCATION` parameter to using the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters instead.

- [Management of TDE Master Encryption Keys Using Oracle Key Vault](#)
You can use Oracle Key Vault to manage and share TDE master encryption keys across an enterprise.

6.2.1 Creating User-Defined TDE Master Encryption Keys

You can create a user-defined TDE master encryption key outside the database by generating a TDE master encryption key ID.

- [About User-Defined TDE Master Encryption Keys](#)
A TDE master encryption key that is outside the database has its own user-generated ID, which tracks the use of the TDE master encryption key.
- [Creating a User-Defined TDE Master Encryption Key](#)
To create a user-defined TDE master encryption key, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET | CREATE [ENCRYPTION] KEY` clause.

6.2.1.1 About User-Defined TDE Master Encryption Keys

A TDE master encryption key that is outside the database has its own user-generated ID, which tracks the use of the TDE master encryption key.

You can use the `ADMINISTER KEY MANAGEMENT` to create and set user-defined TDE master encryption key IDs. After you generate the TDE master encryption key, you can bring this key into the database. Optionally, you can specify the TDE master encryption key ID in various `ADMINISTER KEY MANAGEMENT` statements.

This type of configuration benefits Oracle Fusion SaaS Cloud environments in that it enables you to generate a TDE master encryption key this complies with your site's requirements. This key that you generate supports the current encryption algorithms and can be used for TDE wallets.

After you generate the TDE master encryption key ID, you can encrypt your data as you normally would.

The TDE master encryption key and its corresponding ID will not be captured by any auditing logs.

6.2.1.2 Creating a User-Defined TDE Master Encryption Key

To create a user-defined TDE master encryption key, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET | CREATE [ENCRYPTION] KEY` clause.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Create the user-defined TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET | CREATE [ENCRYPTION] KEY  
'[mkid:mk | mk]'  
[USING ALGORITHM 'algorithm']  
[FORCE KEYSTORE]  
IDENTIFIED BY [EXTERNAL STORE | keystore_password]  
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- **SET | CREATE** : Enter SET if you want to create the master and activate the TDE master encryption key now, or select CREATE if you want to create the key for later use, without activating it yet.
- ***mkid* and *mk***:
 - *mkid*, the TDE master encryption key ID, is a 16-byte hex-encoded value that you can specify or have Oracle Database generate.
 - *mk*, the TDE master encryption key, is a hex-encoded value that you can specify or have Oracle Database generate, either 32 bytes (for the for AES256, ARIA256, and GOST256 algorithms) or 16 bytes (for the SEED128 algorithm).

If you omit the *mkid:mk|mkid* clause but include the *mk* value, then Oracle Database generates the *mkid* for the *mk*.

If you omit the entire *mkid:mk|mkid* clause, then Oracle Database generates these values for you.

- **USING ALGORITHM**: Specify one of the following supported algorithms:
 - AES256 (default): 256-bit key, CBC mode
 - ARIA256: 256-bit key, CBC mode
 - SEED128: 128-bit key, CBC mode
 - GOST256: 256-bit key, CBC mode

If you omit the algorithm, then the default, AES256, is used.

Table 6-2 Cryptographic Provider Support for Master Key Algorithms

Algorithm	Legacy Provider	Next-Generation Provider
AES256	Supported	Supported
ARIA256	Supported	Supported only in non-FIPS mode
GOST256	Supported	Availability varies by platform
SEED128	Supported	Not supported

When using the next-generation provider, only AES256 is supported for master encryption keys. GOST and SEED are not supported for master keys with the next-generation provider. If you specify GOST or SEED with the next-generation provider, the SET KEY operation fails with an error.

If you plan to switch from the legacy provider to the next-generation provider, and your existing master encryption keys use GOST or SEED algorithms, you must rekey the TDE master encryption key to AES256 before switching providers:

```
ADMINISTER KEY MANAGEMENT SET KEY USING ALGORITHM 'AES256' IDENTIFIED
BY keystore_password WITH BACKUP;
```

 Tip

Oracle recommends using AES256 for the TDE master encryption key. AES256 is the default algorithm, is supported by both cryptographic providers in all modes, and all modes of symmetric encryption using AES256 are considered quantum resistant.

- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

The following example creates a TDE master encryption key without the key-ID is when a database administrator performs that operation:

First, generate the TDE master encryption key. For example, using OpenSSL:

```
$ openssl rand -hex 32
f24ce8cdae39742b5da7293da206fbd05a28b1e44c4781f801768cfdc3dbb6e2
```

Set (create and use) this ARIA master key with the following ADMINISTER KEY MANAGEMENT statement.

For example:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
'f24ce8cdae39742b5da7293da206fbd05a28b1e44c4781f801768cfdc3dbb6e2'
USING ALGORITHM 'ARIA256'
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE | keystore_password
WITH BACKUP [USING 'backup_identifier'];
```

The next example, where TDE master encryption key and key-ID are generated outside of the database, shows how to support separation of duties. A key administrator inserts the key with a key_ID into the wallet by using the ADMINISTER KEY MANAGEMENT CREATE [ENCRYPTION] KEY statement. Then, the key administrator sends the key_ID to the database administrator, who then activates the key by using the ADMINISTER KEY MANAGEMENT USE KEY clause.

- a. Create the key-ID, converting characters to upper case. For example, using OpenSSL:

```
$ openssl rand 16 | xxd -u -p
D20765EB721AF44D054B30FE87F8E49A
```

- b. Create the TDE master encryption key. For example, using OpenSSL:

```
$ openssl rand -hex 32
5a089c8ea6ee21cba774f61e58d102665df4a979a34757836b3d066a3eb3db11
```

- c. Create (but do not activate) a default AES256 master encryption key by inserting both the random key-ID and the key itself, separated by a :, into the wallet.

```
ADMINISTER KEY MANAGEMENT CREATE ENCRYPTION KEY
'D20765EB721AF44D054B30FE87F8E49A:5a089c8ea6ee21cba774f61e58d102665df4a979a347578
36b3d066a3eb3db11'
IDENTIFIED BY EXTERNAL STORE | software_keystore_password WITH BACKUP;
```

- d. The database administrator can retrieve the BASE64 key-ID from the database with the following SELECT statement that creates the proper SQL command to activate the key and add a tag to it:

```
SELECT ' ADMINISTER KEY MANAGEMENT USE KEY '''||KEY_ID|''''
USING TAG '''||SYS_CONTEXT('USERENV', 'CON_NAME')||' '||TO_CHAR (SYS_EXTRACT_UTC
(SYSTIMESTAMP),
'YYYY-MM-DD HH24:MI:SS"Z"')||''
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE|software_keystore_password WITH BACKUP;
```

Related Topics

- [Supported Encryption and Integrity Algorithms](#)

The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

6.2.2 Creating TDE Master Encryption Keys for Later Use

You can create a TDE master encryption key that can be activated at a later date.

- [About Creating a TDE Master Encryption Key for Later Use](#)
The `CREATE KEY` clause of the `ADMINISTER KEY MANAGEMENT` statement can create a TDE master encryption key to be activated at a later date.
- [Creating a TDE Master Encryption Key for Later Use](#)
A keystore must be opened before you can create a TDE master encryption key for use later on.
- [Example: Creating a TDE Master Encryption Key in a Single Database](#)
You can use the `ADMINISTER KEY MANAGEMENT CREATE KEY USING TAG` statement to create a TDE master encryption key in a single database.

6.2.2.1 About Creating a TDE Master Encryption Key for Later Use

The `CREATE KEY` clause of the `ADMINISTER KEY MANAGEMENT` statement can create a TDE master encryption key to be activated at a later date.

You then can activate this key on the same database or export it to another database and activate it there.

This method of TDE master encryption key creation is useful in a multitenant environment when you must re-create the TDE master encryption keys. The `CREATE KEY` clause enables you to use a single SQL statement to generate a new TDE master encryption key for all of the PDBs within a multitenant environment. The creation time of the new TDE master encryption key is later than the activation of the TDE master encryption key that is currently in use. Hence, the creation time can serve as a reminder to all of the PDBs to activate the most recently created TDE master encryption key as soon as possible.

6.2.2.2 Creating a TDE Master Encryption Key for Later Use

A keystore must be opened before you can create a TDE master encryption key for use later on.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Create the TDE master encryption key by using this syntax:

```
ADMINISTER KEY MANAGEMENT CREATE KEY [USING TAG 'tag']  
[FORCE KEYSTORE]  
IDENTIFIED BY EXTERNAL STORE | keystore_password  
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- `tag` is the associated attribute and information that you define. Enclose this setting in single quotation marks (' ').
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` can be one of the following settings:
 - `EXTERNAL STORE` uses the keystore password stored in the external store to perform the keystore operation.

- *keystore_password* is the mandatory keystore password that you used when you created the original keystore. It is case sensitive.
 - **WITH BACKUP** backs up the TDE master encryption key in the same location as the key, as identified by the **WRL_PARAMETER** column of the **V\$ENCRYPTION_WALLET** view. To find the key locations for all of the database instances, query the **GV\$ENCRYPTION_WALLET** view.
- You must back up password-based software keystores. You do not need to back up auto-login or local auto-login software keystores. Optionally, include the **USING backup_identifier** clause to add a description of the backup. Enclose *backup_identifier* in single quotation marks (' ').
3. If necessary, activate the TDE master encryption key.

- a. Find the key ID.

```
SELECT KEY_ID FROM V$ENCRYPTION_KEYS;

KEY_ID
-----
AWShwVYC2U+Nv3RVphn/yAIAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

- b. Use this key ID to activate the key.

```
ADMINISTER KEY MANAGEMENT USE KEY
'AWShwVYC2U+Nv3RVphn/yAIAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
USING TAG 'quarter:second;description:Activate Key on standby'
IDENTIFIED BY password
WITH BACKUP;
```

Related Topics

- [Step 3: Open the TDE Wallet](#)
Depending on the type of keystore you create, you must manually open the TDE wallet before you can use it.
- [Opening an External Keystore](#)
To open an external keystore, use the **ADMINISTER KEY MANAGEMENT** statement with the **SET KEYSTORE OPEN** clause.
- [Activating TDE Master Encryption Keys](#)
After you activate a TDE master encryption key, it can be used.

6.2.2.3 Example: Creating a TDE Master Encryption Key in a Single Database

You can use the **ADMINISTER KEY MANAGEMENT CREATE KEY USING TAG** statement to create a TDE master encryption key in a single database.

[Example 6-1](#) shows how to create a TDE master encryption key in a single database. After you run this statement, a TDE master encryption key with the tag definition is created in the keystore for that database. You can query the **TAG** column of the **V\$ENCRYPTION_KEYS** view for the identifier of the newly created key. You can query the **CREATION_TIME** column to find the most recently created key, which would be the key that you created from this statement. You can export this key to another database if you want or activate it locally later on, as described in [Activating TDE Master Encryption Keys](#).

Example 6-1 Creating a TDE Master Encryption Key in a Single Database

```
ADMINISTER KEY MANAGEMENT CREATE KEY USING TAG
'source:admin@source;target:db1@target'
IDENTIFIED BY password WITH BACKUP;
```

keystore altered.

6.2.3 Activating TDE Master Encryption Keys

After you activate a TDE master encryption key, it can be used.

- [About Activating TDE Master Encryption Keys](#)
You can activate a previously created or imported TDE master encryption key by using the `USE KEY` clause of `ADMINISTER KEY MANAGEMENT`.
- [Activating a TDE Master Encryption Key](#)
To activate a TDE master encryption key, you must open the keystore and use `ADMINISTER KEY MANAGEMENT` with the `USE KEY` clause.
- [Example: Activating a TDE Master Encryption Key](#)
You can use the `ADMINISTER KEY MANAGEMENT` SQL statement to activate a TDE master encryption key.

6.2.3.1 About Activating TDE Master Encryption Keys

You can activate a previously created or imported TDE master encryption key by using the `USE KEY` clause of `ADMINISTER KEY MANAGEMENT`.

After you activate the master encryption key, it is used to encrypt all data encryption keys in your database. The key will be used to protect all of the column keys and all of the [tablespace encryption keys](#). If you have deployed a logical standby database, then you must export the TDE master encryption keys after recreating them, and then import them into the standby database. You can have the TDE master encryption key in use on both the primary and the standby databases. To do so, you must activate the TDE master encryption key after you import it to the logical standby database.

6.2.3.2 Activating a TDE Master Encryption Key

To activate a TDE master encryption key, you must open the keystore and use `ADMINISTER KEY MANAGEMENT` with the `USE KEY` clause.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Query the `KEY_ID` column of the `V$ENCRYPTION_KEYS` view to find the key identifier.

For example:

```
SELECT KEY_ID FROM V$ENCRYPTION_KEYS;

KEY_ID
-----
ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

3. Use this key identifier to activate the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT USE KEY 'key_identifier_from_V$ENCRYPTION_KEYS'
[USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *key_identifier_from_V\$ENCRYPTION_KEYS* is the key identifier. Enclose this setting in single quotation marks (' ').
- *tag* is the associated attributes and information that you define. Enclose this setting in single quotation marks (' ').
- **FORCE KEYSTORE** temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- **IDENTIFIED BY** can be one of the following settings:
 - **EXTERNAL STORE** uses the keystore password stored in the external store to perform the keystore operation.
 - *keystore_password* is the mandatory keystore password that you used when you created the original keystore.
- **WITH BACKUP** backs up the TDE master encryption key in the same location as the key, as identified by the *WRL_PARAMETER* column of the *V\$ENCRYPTION_WALLET* view. To find the key locations for all of the database instances, query the *GV\$ENCRYPTION_WALLET* view.

You must back up password-based software keystores. You do not need to back up auto-login or local auto-login software keystores. Optionally, include the *USING backup_identifier* clause to add a description of the backup. Enclose *backup_identifier* in single quotation marks (' ').

Related Topics

- [Opening a TDE Wallet](#)
To open a TDE wallet, you must use the **ADMINISTER KEY MANAGEMENT** statement with the **SET KEYSTORE OPEN** clause.
- [Opening an External Keystore](#)
To open an external keystore, use the **ADMINISTER KEY MANAGEMENT** statement with the **SET KEYSTORE OPEN** clause.

6.2.3.3 Example: Activating a TDE Master Encryption Key

You can use the **ADMINISTER KEY MANAGEMENT** SQL statement to activate a TDE master encryption key.

[Example 6-2](#) shows how to activate a previously imported TDE master encryption key and then update its tag. This key is activated with the current database time stamp and time zone.

Example 6-2 Activating a TDE Master Encryption Key

```
ADMINISTER KEY MANAGEMENT USE KEY
'ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
USING TAG 'quarter:second;description:Activate Key on standby'
IDENTIFIED BY password WITH BACKUP;
```

keystore altered.

In this version of the same operation, the **FORCE KEYSTORE** clause is added in the event that the auto-login keystore is in use, or if the keystore is closed. The password of the keystore is stored externally, so the **EXTERNAL STORE** setting is used for the **IDENTIFIED BY** clause.

```
ADMINISTER KEY MANAGEMENT USE KEY
'ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
USING TAG 'quarter:second;description:Activate Key on standby'
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE WITH BACKUP;
```


keystore altered.

6.2.4 TDE Master Encryption Key Attribute Management

TDE master encryption key attributes store information about the TDE master encryption key.

- [TDE Master Encryption Key Attributes](#)
TDE master encryption key attributes include detailed information about the TDE master encryption key.
- [Finding the TDE Master Encryption Key That Is in Use](#)
A TDE master encryption key that is in use is the encryption key that was activated most recently for the database.

6.2.4.1 TDE Master Encryption Key Attributes

TDE master encryption key attributes include detailed information about the TDE master encryption key.

The information contains the following types:

- **Key time stamp information:** Internal security policies and compliance policies usually determine the key rekeying frequency. You should expire keys when they reach the end of their lifetimes and then generate new keys. Time stamp attributes such as key creation time and activation time help you to determine the key age accurately, and automate key generation.

The V\$ENCRYPTION_KEYS view includes columns such as CREATION_TIME and ACTIVATION_TIME. See *Oracle Database Reference* for a complete description of the V\$ENCRYPTION_KEYS view.

- **Key owner information:** Key owner attributes help you to determine the user who created or activated the key. These attributes can be important for security, auditing, and tracking purposes. Key owner attributes also include key use information, such as whether the key is used for standalone TDE operations or used in a multitenant environment.

The V\$ENCRYPTION_KEYS view includes columns such as CREATOR, CREATOR_ID, USER, USER_ID, and KEY_USE.

- **Key source information:** Keys often must be moved between databases for operations such as import-export operations and Data Guard-related operations. Key source attributes enable you to track the origin of each key. You can track whether a key was created locally or imported, and the database name and instance number of the database that created the key. In a multitenant environment, you can track the PDB where the key was created.

The V\$ENCRYPTION_KEYS view includes columns such as CREATOR_DBNAME, CREATOR_DBID, CREATOR_INSTANCE_NAME, CREATOR_INSTANCE_NUMBER, CREATOR_PDBNAME, and so on.

- **Key usage information:** Key usage information determines the database or PDB where the key is being used. It also helps determine whether a key is in active use or not.

The V\$ENCRYPTION_KEYS view includes columns such as ACTIVATING_DBNAME, ACTIVATING_DBID, ACTIVATING_INSTANCE_NAME, ACTIVATING_PDBNAME, and so on.

- **User-defined information and other information:** When creating a key, you can tag it with information using the TAG option. Each key contains important information such as whether or not it has been backed up.

The V\$ENCRYPTION_KEYS view includes columns such as KEY_ID, TAG, and other miscellaneous columns, for example BACKED_UP.

Note

TDE Master Key Attributes and Tag are only supported with Oracle Key Vault and Oracle Cloud Infrastructure (OCI) Key Management Service (KMS).

6.2.4.2 Finding the TDE Master Encryption Key That Is in Use

A TDE master encryption key that is in use is the encryption key that was activated most recently for the database.

- To find the TDE master encryption key, query the V\$ENCRYPTION_KEYS dynamic view.

For example:

```
SELECT KEY_ID
FROM V$ENCRYPTION_KEYS
WHERE ACTIVATION_TIME = (SELECT MAX(ACTIVATION_TIME)
                        FROM V$ENCRYPTION_KEYS
                        WHERE ACTIVATING_DBID = (SELECT DBID FROM V$DATABASE));
```

6.2.5 Creating Custom TDE Master Encryption Key Attributes for Reports

Custom TDE master encryption key attributes enable you to define attributes that are specific to your needs.

- [About Creating Custom Attribute Tags](#)
Attribute tags enable you to monitor specific activities users perform, such as accessing a particular terminal ID.
- [Creating a Custom Attribute Tag](#)
To create a custom attribute tag, you must use the SET TAG clause of the ADMINISTER KEY MANAGEMENT statement.

6.2.5.1 About Creating Custom Attribute Tags

Attribute tags enable you to monitor specific activities users perform, such as accessing a particular terminal ID.

By default, Oracle Database defines a set of attributes that describe various characteristics of the TDE master encryption keys that you create, such as the creation time, database in which the TDE master encryption key is used, and so on. These attributes are captured by the V\$ENCRYPTION_KEY dynamic view.

You can create custom attributes that can be captured by the TAG column of the V\$ENCRYPTION_KEYS dynamic view. This enables you to define behaviors that you may want to monitor, such as users who perform activities on encryption keys. The tag can encompass multiple attributes, such as session IDs from a specific terminal.

After you create the tag for a TDE master encryption key, its name should appear in the TAG column of the V\$ENCRYPTION_KEYS view for that TDE master encryption key. If you create a tag for the secret, then the tag appears in the SECRET_TAG column of the V\$CLIENT_SECRETS view. If you create a secret with a tag, then the tag appears in the SECRET_TAG column of the V\$CLIENT_SECRETS view.

6.2.5.2 Creating a Custom Attribute Tag

To create a custom attribute tag, you must use the `SET TAG` clause of the `ADMINISTER KEY MANAGEMENT` statement.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. If necessary, query the `TAG` column of the `V$ENCRYPTION_KEY` dynamic view to find a listing of existing tags for the TDE master encryption keys.

When you create a new tag for a TDE master encryption key, it overwrites the existing tag for that TDE master encryption key.

3. Create the custom attribute tag by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET TAG 'tag'
FOR 'master_key_identifier'
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification

- *tag* is the associated attributes or information that you define. Enclose this information in single quotation marks (' ').
- *master_key_identifier* identifies the TDE master encryption key for which the *tag* is set. To find a list of TDE master encryption key identifiers, query the `KEY_ID` column of the `V$ENCRYPTION_KEYS` dynamic view.
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` can be one of the following settings:
 - `EXTERNAL STORE` uses the keystore password stored in the external store to perform the keystore operation.
 - *keystore_password* is the password that was used to create the keystore.
- *backup_identifier* defines the tag values. Enclose this setting in single quotation marks (' ') and separate each value with a colon.

For example, to create a tag that uses two values, one to capture a specific session ID and the second to capture a specific terminal ID:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
USING TAG 'sessionid=3205062574:terminal=xcvt'
IDENTIFIED BY keystore_password
WITH BACKUP;
```

keystore altered.

Both the session ID (3205062574) and terminal ID (xcvt) can derive their values by using either the `SYS_CONTEXT` function with the `USERENV` namespace, or by using the `USERENV` function.

6.2.6 Setting or Rekeying the TDE Master Encryption Key in the Keystore

You can set or rekey the TDE master encryption key for both TDE wallets and external keystores.

- [About Setting or Rekeying the TDE Master Encryption Key in the Keystore](#)
You can set or rekey the TDE master encryption key for both software password-based and external keystores.
- [Creating, Tagging, and Backing Up a TDE Master Encryption Key](#)
The `ADMINISTER KEY MANAGEMENT` statement enables you to create, tag, and back up a TDE master encryption key.
- [About Rekeying the TDE Master Encryption Key](#)
Oracle Database uses a unified TDE Master Encryption Key for both TDE column encryption and TDE tablespace encryption.
- [Rekeying the TDE Master Encryption Key](#)
You can use the `ADMINISTER KEY MANAGEMENT` statement to rekey a TDE master encryption key.
- [Changing the TDE Master Encryption Key for a Tablespace](#)
You can use the `ENCRYPT` and `REKEY` clauses of the `ALTER TABLESPACE` statement to encrypt a tablespace.

6.2.6.1 About Setting or Rekeying the TDE Master Encryption Key in the Keystore

You can set or rekey the TDE master encryption key for both software password-based and external keystores.

The TDE master encryption key is stored in an external security module (keystore), and it is used to protect the [TDE table keys](#) and [tablespace encryption keys](#). By default, the TDE master encryption key is a system-generated random value created by Transparent Data Encryption (TDE).

Use the `ADMINISTER KEY MANAGEMENT` statement to set or reset (`REKEY`) the TDE master encryption key. When the master encryption key is set, then TDE is considered enabled and cannot be disabled.

Before you can encrypt or decrypt database columns or tablespaces, you must generate a TDE master encryption key. Oracle Database uses the same TDE master encryption key for both TDE column encryption and TDE tablespace encryption. The instructions for setting a software or hardware TDE master encryption key explain how to generate a TDE master encryption key.

Related Topics

- [Step 4: Set the TDE Master Encryption Key in the TDE Wallet](#)
Once the TDE wallet is open, you can set a TDE master encryption key for it.
- [Step 3: Set the TDE Master Encryption Key in Oracle Key Vault](#)
After you have opened the connection to Oracle Key Vault, you are ready to set the TDE master encryption key.

6.2.6.2 Creating, Tagging, and Backing Up a TDE Master Encryption Key

The `ADMINISTER KEY MANAGEMENT` statement enables you to create, tag, and back up a TDE master encryption key.

Oracle Database uses the keystore in the keystore location specified by the `WALLET_R00T` parameter in the initialization parameter file to store the TDE master encryption key.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.

2. Create and back up the TDE master encryption key, and apply a tag, by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
[USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- *tag* is the tag that you want to create. Enclose this tag in single quotation marks (' ').
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.
 - *keystore_password* is either *software_keystore_password* or *external_key_manager_password*. As with software passwords, it is case sensitive. You must enclose the password string in double quotation marks (" "). Separate *user_name* and *password* with a colon (:).
- WITH BACKUP backs the TDE master encryption key up in the same location as the key, as identified by the WRL_PARAMETER column of the V\$ENCRYPTION_WALLET view. To find the WRL_PARAMETER values for all of the database instances, query the GV\$ENCRYPTION_WALLET view.

You must back up password-based TDE wallets. You do not need to use it for auto-login or local auto-login TDE wallets. Optionally, include the USING *backup_identifier* clause to add a description of the backup. Enclose this identifier in single quotation marks (' ').

For example:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
USING TAG 'backups'
IDENTIFIED BY password
WITH BACKUP USING 'hr.emp_key_backup';
```

keystore altered.

Related Topics

- [Creating Custom TDE Master Encryption Key Attributes for Reports](#)
Custom TDE master encryption key attributes enable you to define attributes that are specific to your needs.

6.2.6.3 About Rekeying the TDE Master Encryption Key

Oracle Database uses a unified TDE Master Encryption Key for both TDE column encryption and TDE tablespace encryption.

When you rekey the TDE master encryption key for TDE column encryption, the TDE Master Encryption Key for TDE tablespace encryption also is rekeyed. Rekey the TDE Master Encryption Key only if it was compromised or as per the security policies of the organization. This process deactivates the previous TDE master encryption key.

For better security and to meet compliance regulations, periodically rekey the TDE master encryption key. This process deactivates the previous TDE master encryption key, creates a

new TDE master encryption key, and then activates it. You can check the keys that were created recently by querying the `CREATION_TIME` column in the `V$ENCRYPTION_KEYS` view. To find the keys that were activated recently, query the `ACTIVATION_TIME` column in the `V$ENCRYPTION_KEYS` view.

You cannot change the TDE master encryption key or rekey a TDE master encryption key for an auto-login keystore. Because auto-login keystores do not have a password, an administrator or a privileged user can change the keys without the knowledge of the security officer. However, if both the auto-login and the password-based keystores are present in the configured location (as set in the `sqlnet.ora` file), then when you rekey the TDE master encryption key, a TDE master encryption key is added to both the auto-login and password-based keystores. If the auto-login keystore is in use in a location that is different from that of the password-based keystore, then you must re-create the auto-login keystore.

Do not perform a rekey operation of the master key concurrently with an online tablespace rekey operation. You can find if an online tablespace is in the process of being TDE Master Encryption Keyed by issuing the following query:

```
SELECT TS#, ENCRYPTIONALG, STATUS FROM V$ENCRYPTED_TABLESPACES;
```

A status of `REKEYING` means that the corresponding tablespace is still being rekeyed.

Note

You cannot add new information to auto-login keystores separately.

6.2.6.4 Rekeying the TDE Master Encryption Key

You can use the `ADMINISTER KEY MANAGEMENT` statement to rekey a TDE master encryption key.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. If you are rekeying the TDE master encryption key for a keystore that has auto login enabled, then ensure that both the auto login keystore, identified by the `.sso` file, and the encryption keystore, identified by the `.p12` file, are present.

You can find the location of these files by querying the `WRL_PARAMETER` column of the `V$ENCRYPTION_WALLET` view. To find the `WRL_PARAMETER` values for all of the database instances, query the `GV$ENCRYPTION_WALLET` view.

3. Rekey the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY  
[FORCE KEYSTORE]  
[USING TAG 'tag_name']  
IDENTIFIED BY [EXTERNAL STORE | keystore_password]  
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- `tag` is the associated attributes and information that you define. Enclose this setting in single quotation marks (`' '`).
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.
 - *keystore_password* is the mandatory keystore password that you created when you created the keystore in [Step 2: Create the TDE Wallet](#).
- WITH BACKUP creates a backup of the keystore. You must use this option for password-based and external keystores. Optionally, you can use the USING clause to add a brief description of the backup. Enclose this description in single quotation marks (' '). This identifier is appended to the named keystore file (for example, ewallet_time-stamp_emp_key_backup.p12). Follow the file naming conventions that your operating system uses.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY  
FORCE KEYSTORE  
IDENTIFIED BY keystore_password  
WITH BACKUP USING 'emp_key_backup';
```

keystore altered.

Related Topics

- [Step 3: Open the TDE Wallet](#)
Depending on the type of keystore you create, you must manually open the TDE wallet before you can use it.
- [Step 2: Open the Connection to Oracle Key Vault](#)
After you have configured the database to use Oracle Key Vault for TDE key management, you must open the connection to Oracle Key Vault before you can use it.

6.2.6.5 Changing the TDE Master Encryption Key for a Tablespace

You can use the ENCRYPT and REKEY clauses of the ALTER TABLESPACE statement to encrypt a tablespace.

1. Ensure that the tablespace open in read-write mode.

You can query the STATUS column of the V\$INSTANCE dynamic view to find if a database is open and the OPEN_MODE column of the V\$DATABASE view to find if it in read-write mode.

2. If necessary, open the database in read-write mode.

```
ALTER DATABASE OPEN READ WRITE;
```

3. Run the ALTER TABLESPACE SQL statement to encrypt the tablespace.

If the tablespace has not yet been encrypted, then use the ENCRYPT clause:

```
ALTER TABLESPACE encrypt_ts ENCRYPTION USING 'AES256' ENCRYPT;
```

To change the encryption of the SYSTEM, SYSAUX, or UNDO tablespace, you must rekey the tablespace online. Use the ONLINE and REKEY clauses. For example, to change the encryption algorithm of the SYSTEM tablespace:

```
ALTER TABLESPACE SYSTEM ENCRYPTION ONLINE USING 'AES256' REKEY;
```

Related Topics

- [About Encryption Conversions for Existing Online Tablespaces](#)
You can encrypt, decrypt, or rekey existing user tablespaces, and the SYSTEM, SYSAUX, and UNDO tablespace when they are online.

6.2.7 Exporting and Importing the TDE Master Encryption Key

You can export and import the TDE master encryption key in different ways.

- [About Exporting and Importing the TDE Master Encryption Key](#)
Oracle Database features such as transportable tablespaces and Oracle Data Pump move data that is possibly encrypted between databases.
- [About Exporting TDE Master Encryption Keys](#)
You can use `ADMINISTER KEY MANAGEMENT EXPORT` to export TDE master encryption keys from a keystore, and then import them into another keystore.
- [Exporting a TDE Master Encryption Key](#)
The `ADMINISTER KEY MANAGEMENT` statement with the `EXPORT [ENCRYPTION] KEYS WITH SECRET` clause exports a TDE master encryption key.
- [Example: Exporting a TDE Master Encryption Key by Using a Subquery](#)
The `ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS` statement can export a TDE master encryption key by using a subquery.
- [Example: Exporting a List of TDE Master Encryption Key Identifiers to a File](#)
The `ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS WITH SECRET` statement can export a list of TDE master encryption key identifiers to a file.
- [Example: Exporting All TDE Master Encryption Keys of the Database](#)
The `ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS SQL` statement can export all TDE master encryption keys of a database.
- [About Importing TDE Master Encryption Keys](#)
The `ADMINISTER KEY MANAGEMENT IMPORT` statement can import exported TDE master encryption keys from a key export file into a target keystore.
- [Importing a TDE Master Encryption Key](#)
The `ADMINISTER KEY MANAGEMENT` statement with the `IMPORT [ENCRYPTION] KEYS WITH SECRET` clause can import a TDE master encryption key.
- [Example: Importing a TDE Master Encryption Key](#)
You can use the `ADMINISTER KEY MANAGEMENT IMPORT KEYS SQL` statement to import a TDE master encryption key.
- [How Keystore Merge Differs from TDE Master Encryption Key Export or Import](#)
The keystore merge operation differs from the TDE master encryption key export and import operations.

6.2.7.1 About Exporting and Importing the TDE Master Encryption Key

Oracle Database features such as transportable tablespaces and Oracle Data Pump move data that is possibly encrypted between databases.

These are some common scenarios in which you can choose to export and import TDE master encryption keys to move them between source and target keystores. For Data Guard (Logical Standby), you must copy the keystore that is in the primary database to the standby database. Instead of merging the primary database keystore with the standby database, you can export the TDE master encryption key that is in use and then import it to the standby database. Moving transportable tablespaces that are encrypted between databases requires that you

export the TDE master encryption key at the source database and then import it into the target database.

6.2.7.2 About Exporting TDE Master Encryption Keys

You can use `ADMINISTER KEY MANAGEMENT EXPORT` to export TDE master encryption keys from a keystore, and then import them into another keystore.

A TDE master encryption key is exported together with its key identifier and key attributes. The exported keys are protected with a password (secret) in the export file.

You can specify the TDE master encryption keys to be exported by using the `WITH IDENTIFIER` clause of the `ADMINISTER KEY MANAGEMENT EXPORT` statement. To export the TDE master encryption keys, you can either specify their key identifiers as a comma-separated list, or you can specify a query that enumerates their key identifiers. Be aware that Oracle Database executes the query determining the key identifiers within the current user's rights and not with definer's rights.

If you omit the `WITH IDENTIFIER` clause, then all of the TDE master encryption keys of the database are exported.

Related Topics

- [Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode](#)
In isolated mode, the `EXPORT` and `IMPORT` clauses of `ADMINISTER KEY MANAGEMENT EXPORT` can export or import master encryption keys for a PDB.
- [Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode](#)
In isolated mode, the `EXPORT` and `IMPORT` clauses of `ADMINISTER KEY MANAGEMENT EXPORT` can export or import master encryption keys for a PDB.

6.2.7.3 Exporting a TDE Master Encryption Key

The `ADMINISTER KEY MANAGEMENT` statement with the `EXPORT [ENCRYPTION] KEYS WITH SECRET` clause exports a TDE master encryption key.

1. Log in to the database instance as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSM` privilege.
2. Export the TDE master encryption keys by using the following syntax:

```
ADMINISTER KEY MANAGEMENT EXPORT [ENCRYPTION] KEYS
WITH SECRET "export_secret"
TO 'file_path'
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH IDENTIFIER IN 'key_id1', 'key_id2', 'key_idn' | (SQL_query)];
```

In this specification:

- `export_secret` is a password that you can specify to encrypt the export file that contains the exported keys. Enclose this secret in double quotation marks (" "), or you can omit the quotation marks if the secret has no spaces.
- `file_path` is the complete path and name of the file to which the keys must be exported. Enclose this path in single quotation marks (' '). You can export to regular file systems only.
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL_STORE uses the keystore password stored in the external store to perform the keystore operation.
 - *TDE_wallet_password* is the password of the keystore containing the keys.
- *key_id1, key_id2, key_idn* is a string of one or more TDE master encryption key identifiers for the TDE master encryption key being exported. Separate each key identifier with a comma and enclose each of these key identifiers in single quotation marks (' '). To find a list of TDE master encryption key identifiers, query the KEY_ID column of the V\$ENCRYPTION_KEYS dynamic view.
- *SQL_query* is a query that fetches a list of the TDE master encryption key identifiers. It should return only one column which contains the TDE master encryption key identifiers. This query is executed with current user rights.

Related Topics

- [Step 3: Open the TDE Wallet](#)
Depending on the type of keystore you create, you must manually open the TDE wallet before you can use it.

6.2.7.4 Example: Exporting a TDE Master Encryption Key by Using a Subquery

The ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS statement can export a TDE master encryption key by using a subquery.

[Example 6-4](#) shows how to export TDE master encryption keys whose identifiers are fetched by a query to a file called `export.exp`. The TDE master encryption keys in the file are encrypted using the secret `my_secret`. The SELECT statement finds the identifiers for the TDE master encryption keys to be exported.

Example 6-3 Exporting a List of TDE Master Encryption Key Identifiers to a File

```
ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS
WITH SECRET "my_secret"
TO '/TDE/export.exp'
FORCE KEYSTORE
IDENTIFIED BY password
WITH IDENTIFIER IN 'AdoxnJ0uH08cv7xkz83ovwsAAAAAAAAAAAAAAAAAAAAAAAAAAAA',
'AW5z3CoyKE/yv3cNT5CWCXUAAAAAAAAAAAAAAAAAAAAAAAAAAAA';
```

keystore altered.

6.2.7.5 Example: Exporting a List of TDE Master Encryption Key Identifiers to a File

The ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS WITH SECRET statement can export a list of TDE master encryption key identifiers to a file.

[Example 6-3](#) shows how to export TDE master encryption keys by specifying their identifiers as a list, to a file called `export.exp`. TDE master encryption keys in the file are encrypted using the secret `my_secret`. The identifiers of the TDE master encryption key to be exported are provided as a comma-separated list.

Example 6-4 Exporting TDE Master Encryption Key Identifiers by Using a Subquery

```
ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS
WITH SECRET "my_secret" TO '/etc/TDE/export.exp'
FORCE KEYSTORE
IDENTIFIED BY password
```

```
WITH IDENTIFIER IN (SELECT KEY_ID FROM V$ENCRYPTION_KEYS WHERE ROWNUM <3);
```

keystore altered.

6.2.7.6 Example: Exporting All TDE Master Encryption Keys of the Database

The `ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS` SQL statement can export all TDE master encryption keys of a database.

[Example 6-5](#) shows how to export all of the TDE master encryption keys of the database to a file called `export.exp`. The TDE master encryption keys in the file are encrypted using the secret `my_secret`.

Example 6-5 Exporting All of the TDE Master Encryption Keys of the Database

```
ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS
WITH SECRET "my_secret" TO '/etc/TDE/export.exp'
FORCE KEYSTORE
IDENTIFIED BY password;
```

keystore altered.

6.2.7.7 About Importing TDE Master Encryption Keys

The `ADMINISTER KEY MANAGEMENT IMPORT` statement can import exported TDE master encryption keys from a key export file into a target keystore.

You cannot re-import TDE master encryption keys that have already been imported.

Related Topics

- [Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode](#)
In isolated mode, the `EXPORT` and `IMPORT` clauses of `ADMINISTER KEY MANAGEMENT EXPORT` can export or import master encryption keys for a PDB.

6.2.7.8 Importing a TDE Master Encryption Key

The `ADMINISTER KEY MANAGEMENT` statement with the `IMPORT [ENCRYPTION] KEYS WITH SECRET` clause can import a TDE master encryption key.

- Use the following syntax to import a TDE master encryption key:

```
ADMINISTER KEY MANAGEMENT IMPORT [ENCRYPTION] KEYS
WITH SECRET "import_secret"
FROM 'file_name'
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *import_secret* is the same password that was used to encrypt the keys during the export operation. Enclose this secret in double quotation marks (" "), or you can omit the quotation marks if the secret has no spaces.
- *file_name* is the complete path and name of the file from which the keys need to be imported. Enclose this setting in single quotation marks (' ').
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

- IDENTIFIED BY can be one of the following settings:
 - * EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.
 - * *TDE_wallet_password* is the password of the TDE wallet where the keys are being imported.
- WITH BACKUP must be used in case the target keystore was not backed up before the import operation. *backup_identifier* is an optional string that you can provide to identify the keystore backup. Enclose this setting in single quotation marks (' ').

6.2.7.9 Example: Importing a TDE Master Encryption Key

You can use the `ADMINISTER KEY MANAGEMENT IMPORT KEYS` SQL statement to import a TDE master encryption key.

[Example 6-6](#) shows how to import the TDE master encryption key identifiers that are stored in the file `export.exp` and encrypted with the secret `my_secret`.

Example 6-6 Importing TDE Master Encryption Key Identifiers from an Export File

```
ADMINISTER KEY MANAGEMENT IMPORT KEYS
WITH SECRET "my_secret"
FROM '/etc/TDE/export.exp'
FORCE KEYSTORE
IDENTIFIED BY password WITH BACKUP;
```

keystore altered.

6.2.7.10 How Keystore Merge Differs from TDE Master Encryption Key Export or Import

The keystore merge operation differs from the TDE master encryption key export and import operations.

Even though both the `ADMINISTER KEY MANAGEMENT MERGE` statement and the `ADMINISTER KEY MANAGEMENT EXPORT` and `IMPORT` statements eventually move the TDE master encryption keys from one keystore to the next, there are differences in how these two statements function.

- The `MERGE` statement merges two keystores whereas the `EXPORT` and `IMPORT` statements export the keys to a file or import the keys from a file. The keystore is different from the export file, and the two cannot be used interchangeably. The export file is not a keystore and cannot be configured to be used with a database as a keystore. Similarly, the `IMPORT` statement cannot extract the TDE master encryption keys from the keystore.
- The `MERGE` statement merges all of the TDE master encryption keys of the specified keystores whereas the `EXPORT` and `IMPORT` statements can be selective.
- The `EXPORT` and `IMPORT` statements require the user to provide both a location (filepath) and the file name of the export file, whereas the `MERGE` statement only takes in the location of the keystores.
- The file name of the keystores is fixed and is determined by the `MERGE` operation and can be either `ewallet.p12` or `cwallet.sso`. The file names for the export files used in the `EXPORT` or `IMPORT` statements are specified by the user.
- The keystores on Automatic Storage Management (ASM) disk groups or regular file systems can be merged with `MERGE` statements. The export files used in the `EXPORT` and the

IMPORT statements can only be a regular operating system file and cannot be located on an ASM disk group.

- The keystores merged using the MERGE statement do not need to be configured or in use with the database. The EXPORT statement can only export the keys from a keystore that is configured and in use with the database and is also open when the export is done. The IMPORT statement can only import the keys into a keystore that is open, configured, and in use with the database.
- The MERGE statement never modifies the metadata associated with the TDE master encryption keys. The EXPORT and IMPORT operations can modify the metadata of the TDE master encryption keys when required, such as during a PDB plug operation.

6.2.8 Moving TDE Master Encryption Keys into a New Keystore

You can move an existing TDE master encryption key into a new keystore from an existing password-protected keystore.

- [About Moving TDE Master Encryption Keys into a New Keystore](#)
You can move an unused (and safely archived) TDE master encryption key into a new keystore.
- [Moving a TDE Master Encryption Key into a New Keystore](#)
The ADMINISTER KEY MANAGEMENT MOVE KEYS moves an existing TDE master encryption key into a new keystore from an existing keystore.

6.2.8.1 About Moving TDE Master Encryption Keys into a New Keystore

You can move an unused (and safely archived) TDE master encryption key into a new keystore.

Use great caution when you decide to run ADMINISTER KEY MANAGEMENT MOVE KEYS. Even though this statement will not move an active master encryption key, it can still affect keys that are necessary for a range of database features. If you have deleted a key, then the data that was encrypted by that key is rendered permanently inaccessible (equivalent to deleting the data) for these features to be used. See Related Topics at the end of this topic for more information about features that are affected by deleted keystores.

Therefore, before you move the keystore, it is very important that you safely archive it. Delete the keystore only after a period of time has passed, to ensure that the keystore is no longer really useful.

To move a TDE master encryption key into a new keystore, you use the ADMINISTER KEY MANAGEMENT MOVE KEYS statement. This statement does not move the active TDE master encryption key (that is, the key that is currently in use at the time that ADMINISTER KEY MANAGEMENT MOVE KEYS is issued) because the database is currently using it.

If you mistakenly use the ADMINISTER KEY MANAGEMENT MOVE KEYS statement instead of ADMINISTER KEY MANAGEMENT MERGE KEYSTORE when you are configuring a new TDE keystore (for example, when you are changing the TDE keystore configuration from one where the TDE wallet is located in the operating system's file system to one where the TDE wallet is located in Oracle Automatic Storage Management (Oracle ASM)), then the following symptoms may help you to identify the TDE misconfiguration that was introduced by the use of the wrong key management command:

- When you open the TDE keystore that was the target of the earlier ADMINISTER KEY MANAGEMENT MOVE KEYS operation, an ORA-28374: typed master key not found in

wallet error is seen, because the active TDE master encryption key was not moved to that keystore.

- The value shown in the STATUS column of the V\$ENCRYPTION_WALLET view is OPEN_NO_MASTER_KEY after you open the new keystore. The OPEN_NO_MASTER_KEY status is expected, because the new TDE keystore that was mistakenly populated by means of the ADMINISTER KEY MANAGEMENT MOVE KEYS statement does not contain the active TDE master encryption key.

Related Topics

- [Dangers of Deleting Keystores](#)
Oracle strongly recommends that you do not delete keystores.
- [Features That Are Affected by Deleted Keystores](#)
Some features can be adversely affected if a keystore is deleted and a TDE master encryption key residing in that keystore is later needed.

6.2.8.2 Moving a TDE Master Encryption Key into a New Keystore

The ADMINISTER KEY MANAGEMENT MOVE KEYS moves an existing TDE master encryption key into a new keystore from an existing keystore.

This feature enables you to delete unused keystores. Before you move the keystore, ensure that it is safely archived. After you move the encryption key to a new keystore, and when you are sure that the old keystore is no longer needed, you then can delete the old keystore.

1. Log in to the database instance as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.

For example:

```
sqlplus sec_admin as syskm
Enter password: password
Connected.
```

2. Query the KEY_ID column of the V\$ENCRYPTION_KEYS view to find the key identifier of the keys you want to move to the "key archival" keystore.

For example:

```
SELECT KEY_ID FROM V$ENCRYPTION_KEYS;

KEY_ID
-----
ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

3. Move the key into a new keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT
MOVE [ENCRYPTION] KEYS
TO NEW KEYSTORE 'keystore_location1'
IDENTIFIED BY keystore1_password
FROM [FORCE] KEYSTORE
IDENTIFIED BY keystore_password
[WITH IDENTIFIER IN
{ 'key_identifier' [, 'key_identifier' ]... | ( subquery ) } ]
[WITH BACKUP [USING 'backup_identifier' ]];
```

In this specification:

- *keystore_location1* is the path to the wallet directory that will store the new keystore .p12 file. By default, this directory is in \$ORACLE_BASE/admin/*db_unique_name*/wallet.

- *keystore1_password* is the password for the keystore from which the new keystore is moved.
- *keystore_password* is the password for the keystore from which the key is moving.
- *key_identifier* is the key identifier that you find from querying the KEY_ID column of the V\$ENCRYPTION_KEYS view. Enclose this setting in single quotation marks (' ').
- *subquery* can be used to find the exact key identifier that you want.
- *backup_identifier* is an optional description of the backup. Enclose *backup_identifier* in single quotation marks (' ').

For example:

```
ADMINISTER KEY MANAGEMENT MOVE KEYS
TO NEW KEYSTORE $ORACLE_BASE/admin/orcl/wallet
IDENTIFIED BY keystore_password
FROM FORCE KEYSTORE
IDENTIFIED BY keystore_password
WITH IDENTIFIER IN
(SELECT KEY_ID FROM V$ENCRYPTION_KEYS WHERE ROWNUM < 2);
```

4. To delete the old keystore, go to the wallet directory and then manually delete the .p12 file containing the keystore.

To find the location of the keystore, open the keystore, and then query the WRL_PARAMETER column of the V\$ENCRYPTION_WALLET view.

Related Topics

- [Dangers of Deleting Keystores](#)
Oracle strongly recommends that you do not delete keystores.
- [Features That Are Affected by Deleted Keystores](#)
Some features can be adversely affected if a keystore is deleted and a TDE master encryption key residing in that keystore is later needed.

6.2.9 Converting from ENCRYPTION_WALLET_LOCATION to WALLET_ROOT and TDE_CONFIGURATION

You can convert the wallet location from using the sqlnet.ora ENCRYPTION_WALLET_LOCATION parameter to using the WALLET_ROOT and TDE_CONFIGURATION parameters instead.

The conversion applies to either file systems or Oracle Automatic Storage Management (Oracle ASM) configurations. This procedure assumes that you have already set the WALLET_ROOT static initialization parameter and created a *WALLET_ROOT/tde* directory.

1. Copy all the wallets to the *WALLET_ROOT/tde* directory.
2. Restart the database.
3. Back up the database.
4. Connect to the CDB root as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
5. Set the TDE_CONFIGURATION dynamic initialization parameter to FILE.

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE" SCOPE =
"BOTH" SID = "*";
```

6. Rename the wallets from where you copied them.

7. Remove the old ENCRYPTION_WALLET_LOCATION configuration setting from the sqlnet.ora file.

6.2.10 Management of TDE Master Encryption Keys Using Oracle Key Vault

You can use Oracle Key Vault to manage and share TDE master encryption keys across an enterprise.

Oracle Key Vault securely stores the keys in a central repository, along with other security objects such as credential files and Java keystores, and enables you to share these objects with other TDE-enabled databases.

Related Topics

- [Migration of Keystores to and from Oracle Key Vault](#)
You can use Oracle Key Vault to migrate both TDE wallets and external keystores to and from Oracle Key Vault.
- *Oracle Key Vault Administrator's Guide*

6.3 Storing Oracle Database Secrets

Secrets are data that support internal Oracle Database features that integrate external clients such as Oracle GoldenGate into the database.

- [About Storing Oracle Database Secrets in a Keystore](#)
Keystores can store secrets that support internal Oracle Database features and integrate external clients such as Oracle GoldenGate.
- [Storage of Oracle Database Secrets in a Software Keystore](#)
The ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET statements can add secrets, update secrets, and delete secrets from a keystore.
- [Example: Adding an Oracle Key Vault Password to a Software Keystore](#)
The ADMINISTER KEY MANAGEMENT ADD SECRET statement can add an Oracle Key Vault password to a software keystore.
- [Example: Changing an Oracle Key Vault Password Stored as a Secret in a Software Keystore](#)
The ADMINISTER KEY MANAGEMENT UPDATE SECRET statement can change an Oracle Key Vault password that is stored as a secret in a software keystore.
- [Example: Deleting an Oracle Key Vault Password Stored as a Secret in a Software Keystore](#)
The ADMINISTER KEY MANAGEMENT DELETE SECRET statement can delete Oracle Key Vault passwords that are stored as secrets in a software keystore.
- [Storage of Oracle Database Secrets in an External Keystore](#)
The ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET statements can add, update, and delete secrets.
- [Example: Adding an Oracle Database Secret to an External Keystore](#)
The ADMINISTER KEY MANAGEMENT ADD SECRET statement can add an Oracle Database secret to an external keystore.
- [Example: Changing an Oracle Database Secret in an External Keystore](#)
The ADMINISTER KEY MANAGEMENT MANAGEMENT UPDATE SECRET statement can change an Oracle Database secret in an external keystore.

- [Example: Deleting an Oracle Database Secret in an External Keystore](#)
The `ADMINISTER KEY MANAGEMENT DELETE SECRET FOR CLIENT` statement can delete an Oracle Database secret that is in an external keystore.
- [Configuring Auto-Open Connections into External Key Managers](#)
An external key manager can be configured to use the auto-login capability.

6.3.1 About Storing Oracle Database Secrets in a Keystore

Keystores can store secrets that support internal Oracle Database features and integrate external clients such as Oracle GoldenGate.

The secret key must be a string adhering to Oracle identifier rules. You can add, update, or delete a client secret in an existing keystore. The Oracle GoldenGate Extract process must have data encryption keys to decrypt the data that is in data files and in REDO or UNDO logs. Keys are encrypted with shared secrets when you share the keys between an Oracle database and an Oracle GoldenGate client. The software keystore stores the shared secrets.

Depending on your site's requirements, you may require automated open keystore operations even when an external keystore is configured. For this reason, the external key manager password can be stored in a software auto-login keystore, which enables the auto-login capability for the external keystore. The Oracle Database side can also store the credentials for the database to log in to an external storage server in the software keystore.

You can store Oracle Database secrets in both software keystores and external keystores:

- **Software keystores:** You can store secrets in software password-based, auto-login, and local auto-login software keystores. If you want to store secrets in an auto-login (or auto-login local) keystore, then note the following:
 - If the software auto-login keystore is in the same location as its corresponding password-based software keystore, then the secrets are added automatically.
 - If the software auto-login keystore is in a different location from its corresponding password-based software keystore, then you must create the auto-login keystore again from the password-based keystore, and keep the two keystores in synchronization.
- **External keystores:** You can store secrets in standard external key managers.

Related Topics

- [Storage of Oracle Database Secrets in an External Keystore](#)
The `ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET` statements can add, update, and delete secrets.
- [Configuring Auto-Open Connections into External Key Managers](#)
An external key manager can be configured to use the auto-login capability.

6.3.2 Storage of Oracle Database Secrets in a Software Keystore

The `ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET` statements can add secrets, update secrets, and delete secrets from a keystore.

As with all of the `ADMINISTER KEY MANAGEMENT` statements, you must have the `ADMINISTER KEY MANAGEMENT` or the `SYSKM` administrative privilege. To find information about existing secrets, you can query the `V$CLIENT_SECRETS` dynamic view.

- **Adding a secret:** Use the following syntax:

```
ADMINISTER KEY MANAGEMENT
ADD SECRET 'secret' FOR CLIENT 'client_identifier'
[USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING backup_id];
```

- **Updating a secret:** Use the following syntax:

```
ADMINISTER KEY MANAGEMENT
UPDATE SECRET 'secret' FOR CLIENT 'client_identifier'
[USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING backup_id];
```

- **Deleting a secret:** Use the following syntax:

```
ADMINISTER KEY MANAGEMENT
DELETE SECRET FOR CLIENT 'client_identifier'
[FROM [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING backup_id];
```

In all of these statements, the specification is as follows:

- *secret* is the client secret key to be stored, updated, or deleted. Enclose this setting in single quotation marks (' ') or omit the quotation marks if the secret has no spaces. To find information about existing secrets and their client identifiers, query the V\$CLIENT_SECRETS dynamic view.
- *client_identifier* is an alphanumeric string used to identify the secret key. *client_identifier* does not have a default value. Enclose this setting in single quotation marks (' ').
- TO [[LOCAL] AUTO_LOGIN] KEYSTORE refers to the location of an auto-login keystore, which is specified in the sqlnet.ora file.
- *tag* is an optional, user-defined description for the secret key to be stored. You can use *tag* with the ADD and UPDATE operations. Enclose this setting in single quotation marks (' '). This tag appears in the SECRET_TAG column of the V\$CLIENT_SECRETS view.

WITH BACKUP is required in case the keystore was not backed up before the ADD, UPDATE, or DELETE operation. *backup_identifier* is an optional user-defined description for the backup. Enclose *backup_identifier* in single quotation marks (' ').

- [TO | FROM [[LOCAL] AUTOLOGIN] KEYSTORE specifies the location of the auto-login or password keystore, which is set in the sqlnet.ora file.
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation if an auto-login TDE wallet is open (and in use) or if the TDE wallet is closed.
- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.
 - *keystore_password* is the password for the keystore.

Related Topics

- [Creating Custom TDE Master Encryption Key Attributes for Reports](#)
Custom TDE master encryption key attributes enable you to define attributes that are specific to your needs.

6.3.3 Example: Adding an Oracle Key Vault Password to a Software Keystore

The `ADMINISTER KEY MANAGEMENT ADD SECRET` statement can add an Oracle Key Vault password to a software keystore.

[Example 6-7](#) shows how to add an Oracle Key Vault password as a secret into an existing software keystore (for example, after migrating to Oracle Key Vault, the Oracle Key Vault password can be added to the old TDE software keystore to set up an auto-open connection into Oracle Key Vault).

Example 6-7 Adding an Oracle Database Secret to a Software Keystore

```
ADMINISTER KEY MANAGEMENT
ADD SECRET 'external_key_manager_password' FOR CLIENT 'OKV_PASSWORD'
IDENTIFIED BY software_keystore_password WITH BACKUP;
```

Before migrating from a software keystore to Oracle Key Vault, you can upload the software keystore into the virtual wallet of that endpoint in Oracle Key Vault. After migrating, you now can delete the old TDE wallet, because its key is in Oracle Key Vault. In order to configure auto-open Oracle Key Vault without a wallet being already present, execute the following statement:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'external_keystore_password'
FOR CLIENT 'OKV_PASSWORD' INTO [LOCAL] AUTO_LOGIN KEYSTORE 'WALLET_ROOT/tde';
```

Note that the setting `TDE_CONFIGURATION='KEYSTORE_CONFIGURATION=OKV'` is for a password-protected connection into Oracle Key Vault. After the Oracle Key Vault password has been inserted into an existing or newly created wallet, change the `TDE_CONFIGURATION` setting to `'KEYSTORE_CONFIGURATION=OKV|FILE'`.

6.3.4 Example: Changing an Oracle Key Vault Password Stored as a Secret in a Software Keystore

The `ADMINISTER KEY MANAGEMENT UPDATE SECRET` statement can change an Oracle Key Vault password that is stored as a secret in a software keystore.

[Example 6-8](#) shows how to change an Oracle Key Vault password that is stored as a secret in a software keystore.

Example 6-8 Changing an Oracle Key Vault Password Secret to a Software Keystore

```
ADMINISTER KEY MANAGEMENT
UPDATE SECRET admin_password FOR CLIENT 'admin@myhost'
USING TAG 'new_host_credentials' FORCE KEYSTORE
IDENTIFIED BY software_keystore_password;
```

In this version, the password for the keystore is in an external store:

```
ADMINISTER KEY MANAGEMENT
UPDATE SECRET admin_password FOR CLIENT 'admin@myhost'
```

```
USING TAG 'new_host_credentials' FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE;
```

6.3.5 Example: Deleting an Oracle Key Vault Password Stored as a Secret in a Software Keystore

The `ADMINISTER KEY MANAGEMENT DELETE SECRET` statement can delete Oracle Key Vault passwords that are stored as secrets in a software keystore.

[Example 6-9](#) shows how to delete an Oracle Key Vault password that is stored as a secret in the software keystore.

Example 6-9 Deleting an Oracle Key Vault Password Secret in a Software Keystore

```
ADMINISTER KEY MANAGEMENT
DELETE SECRET FOR CLIENT 'OKV_PASSWORD'
FORCE KEYSTORE
IDENTIFIED BY password WITH BACKUP;
```

In this version, the password for the keystore is in an external store:

```
ADMINISTER KEY MANAGEMENT
DELETE SECRET FOR CLIENT 'OKV_PASSWORD'
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE WITH BACKUP;
```

6.3.6 Storage of Oracle Database Secrets in an External Keystore

The `ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET` statements can add, update, and delete secrets.

As with all `ADMINISTER KEY MANAGEMENT` statements, you must have the `ADMINISTER KEY MANAGEMENT` or the `SYSKM` administrative privilege. When you add, update, or delete secrets from a keystore that is in use or presently open, then you must run `ADMINISTER KEY MANAGEMENT` in the root.

You can store Oracle Database secrets in both HSM and Oracle Key Vault external keystores, but be aware that automatic logins do not work if you store the `HSM_PASSWORD` in a Key Vault keystore.

- **Adding a secret:** Use the following syntax:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'secret'
FOR CLIENT 'client_identifier' [USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY "external_key_manager_password"
[WITH BACKUP [USING backup_id]];
```

- **Updating a secret:** Use the following syntax:

```
ADMINISTER KEY MANAGEMENT UPDATE SECRET 'secret'
FOR CLIENT 'client_identifier' [USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY "external_key_manager_password"
[WITH BACKUP [USING backup_id]];
```

- **Deleting a secret:** Use the following syntax:

```
ADMINISTER KEY MANAGEMENT DELETE SECRET
FOR CLIENT 'client_identifier'
[FROM [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY "external_key_manager_password"
[WITH BACKUP [USING backup_id]];
```

In all of these statements, the specification as follows:

- *secret* is the client secret key to be stored, updated, or deleted. Enclose this setting in double quotation marks (' ') or omit the quotation marks if the secret has no spaces. To find information about existing secrets and their client identifiers, query the V\$CLIENT_SECRETS dynamic view.
- *client_identifier* is an alphanumeric string used to identify the secret key. *client_identifier* does not have a default value. Enclose this setting in single quotation marks (' ').
- *tag* is an optional, user-defined description for the secret key to be stored. You can use *tag* with the ADD and UPDATE operations. Enclose this setting in single quotation marks (' '). This tag appears in the SECRET_TAG column of the V\$CLIENT_SECRETS view.
- [TO | FROM [[LOCAL] AUTO_LOGIN] KEYSTORE specifies the location of the keystore used for the external keystore.
- *external_key_manager_password* is for an external keystore manager, which can be Oracle Key Vault or OCI Vault - Key Management. Enclose this password in double quotation marks. For Oracle Key Vault, enter the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

Related Topics

- [Creating Custom TDE Master Encryption Key Attributes for Reports](#)
Custom TDE master encryption key attributes enable you to defined attributes that are specific to your needs.

6.3.7 Example: Adding an Oracle Database Secret to an External Keystore

The ADMINISTER KEY MANAGEMENT ADD SECRET statement can add an Oracle Database secret to an external keystore.

[Example 6-10](#) shows how to add a password for a user to an external keystore.

Example 6-10 Adding an Oracle Database Secret to an External Keystore

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'password'
FOR CLIENT 'admin@myhost' USING TAG 'myhost admin credentials'
IDENTIFIED BY "external_key_manager_password";
```

In this version, the keystore password is in an external store, so the EXTERNAL STORE setting is used for IDENTIFIED BY:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'password'
FOR CLIENT 'admin@myhost' USING TAG 'myhost admin credentials'
IDENTIFIED BY EXTERNAL STORE;
```

6.3.8 Example: Changing an Oracle Database Secret in an External Keystore

The `ADMINISTER KEY MANAGEMENT MANAGEMENT UPDATE SECRET` statement can change an Oracle Database secret in an external keystore.

[Example 6-11](#) shows how to change a password that is stored as a secret in an external keystore.

Example 6-11 Changing an Oracle Database Secret in an External Keystore

```
ADMINISTER KEY MANAGEMENT MANAGEMENT UPDATE SECRET 'password2'  
FOR CLIENT 'admin@myhost' USING TAG 'New host credentials'  
IDENTIFIED BY "external_key_manager_password";
```

In this version, the password for the keystore is in an external store:

```
ADMINISTER KEY MANAGEMENT MANAGEMENT UPDATE SECRET 'password2'  
FOR CLIENT 'admin@myhost' USING TAG 'New host credentials'  
IDENTIFIED BY EXTERNAL STORE;
```

6.3.9 Example: Deleting an Oracle Database Secret in an External Keystore

The `ADMINISTER KEY MANAGEMENT DELETE SECRET FOR CLIENT` statement can delete an Oracle Database secret that is in an external keystore.

[Example 6-12](#) shows how to delete an external key manager password that is stored as a secret in the external keystore.

Example 6-12 Deleting an Oracle Database Secret in an External Keystore

```
ADMINISTER KEY MANAGEMENT DELETE SECRET FOR CLIENT 'admin@myhost'  
IDENTIFIED BY "external_key_manager_password";
```

In this version, the password for the keystore is in an external store:

```
ADMINISTER KEY MANAGEMENT DELETE SECRET FOR CLIENT 'admin@myhost'  
IDENTIFIED BY EXTERNAL STORE;
```

6.3.10 Configuring Auto-Open Connections into External Key Managers

An external key manager can be configured to use the auto-login capability.

- [About Auto-Open Connections into External Key Managers](#)
An auto-open connection into an external key manager stores the external keystore credentials in an auto-login keystore.
- [Configuring an Auto-Open Connection into an External Key Manager](#)
To configure the auto-open connection, you must use the `ADMINISTER KEY MANAGEMENT` statement to add or update a client secret to authenticate to the external key manager.

6.3.10.1 About Auto-Open Connections into External Key Managers

An auto-open connection into an external key manager stores the external keystore credentials in an auto-login keystore.

You can configure a connection to an external key manager so that the database can open the keystore without prompting for the keystore password. This configuration is essential in Oracle

Real Application Clusters (Oracle RAC) environments, and is highly recommended for Oracle Data Guard standby databases. Be aware that this configuration reduces the security of the system as a whole. However, this configuration does support unmanned or automated operations, and is useful in deployments where TDE-enabled databases that are enrolled into an external keystore for key management can start automatically.

Be aware that executing the query `SELECT * FROM V$ENCRYPTION_WALLET` will automatically open an auto-login external keystore. For example, suppose you have an auto-login external keystore configured. If you close the keystore and query the `V$ENCRYPTION_WALLET` view, then the output will indicate that a keystore is open. This is because `V$ENCRYPTION_WALLET` opened up the auto-login external keystore and then displayed the status of the auto-login keystore.

To enable the auto-login capability for an external keystore, you must store the external keystore's credentials in an auto-login wallet.

When you use the `ADMINISTER KEY MANAGEMENT` statement, there are conceptually two sets of commands that act on client secrets:

- `ADMINISTER KEY MANAGEMENT` commands that act on the wallet that is currently in use (in other words, a wallet that contains an active TDE master encryption key).
- `ADMINISTER KEY MANAGEMENT` commands that act on a wallet that is not currently being used to hold the active TDE master encryption key. Oracle recommends that you use this approach when you configure an auto-login external keystore.

Related Topics

- [Configuring an Auto-Open Connection into an External Key Manager](#)
To configure the auto-open connection, you must use the `ADMINISTER KEY MANAGEMENT` statement to add or update a client secret to authenticate to the external key manager.

6.3.10.2 Configuring an Auto-Open Connection into an External Key Manager

To configure the auto-open connection, you must use the `ADMINISTER KEY MANAGEMENT` statement to add or update a client secret to authenticate to the external key manager.

Before you begin this procedure, ensure that you have configured the external keystore.

In this procedure, the wallet that is created does not contain any keys. It only holds the client secret. So, when you query the `V$ENCRYPTION_WALLET` dynamic view for this wallet, the `STATUS` column shows `OPEN_NO_MASTER_KEY` rather than `OPEN`, because the wallet only contains the client secret.

1. Reconfigure the `WALLET_ROOT` parameter in the `init.ora` file to include the location of the TDE wallet, if it is not already present.

The TDE wallet location may already be present if you had previously migrated to using the external key manager.

For example:

```
WALLET_ROOT=/etc/ORACLE/WALLETS/orcl
```

2. Add or update the secret in the TDE wallet.

The secret is the external keystore password and the client is the `OKV_PASSWORD`. `OKV_PASSWORD` is an Oracle-defined client name that is used to represent the external key manager password as a secret in the TDE wallet.

For example:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'external_key_manager_password'  
FOR CLIENT 'OKV_PASSWORD'
```



```
TO LOCAL AUTO_LOGIN KEYSTORE TDE_wallet_location
WITH BACKUP;
```

In this example:

- TDE_wallet_location* is the location of the TDE wallet within the *WALLET_ROOT* location that you just defined in Step 1.

For the CDB root and for any PDB that is configured in united mode, the value to use for the *TDE_wallet_location* location is *WALLET_ROOT/tde*.

For any PDB that is configured in isolated mode, the value to use for the *TDE_wallet_location* location is *WALLET_ROOT/pdb_guid/tde*. When you are in the PDB, run the following query to find this GUID: `SELECT GUID FROM V$PDBS;`
- LOCAL creates a local auto-login wallet file, `cwallet.sso`, to hold the credentials for the external key manager. This wallet is tied to the host on which it was created.

For an Oracle Real Application Clusters environment, omit the LOCAL keyword, because each Oracle RAC node has a different host name, yet they all use the same external key manager. If you configure a local auto-login wallet for the Oracle RAC instance, then only the first Oracle RAC node, where the `cwallet.sso` file was created, would be able to access the external key manager credentials. If you try to open the TDE wallet from another node instead of from that first node, there would be a problem auto-opening `cwallet.sso`, and so it would result in a failure to auto-open the auto-login external keystore. This restriction applies if you are using a shared location to hold the `cwallet.sso` file for the Oracle RAC cluster, because using LOCAL only works if you have a separate `cwallet.sso` file (containing the same credentials) on each node of the Oracle RAC environment.

At this stage, the next time a TDE operation runs, the external key manager auto-login TDE wallet opens automatically. An example of a TDE operation is to query the `V$ENCRYPTION_WALLET` view, for example:

```
SELECT * FROM V$ENCRYPTION_WALLET;
```

6.4 Storing Oracle GoldenGate Secrets in a Keystore

You can store Oracle GoldenGate secrets in Transparent Data Encryption keystores.

- [About Storing Oracle GoldenGate Secrets in Keystores](#)

You can use a keystore to store secret keys for tools and external clients such as Oracle GoldenGate.
- [Oracle GoldenGate Extract Classic Capture Mode TDE Requirements](#)

Ensure that you meet the requirements for Oracle GoldenGate Extract to support Transparent Data Encryption capture.
- [Configuring Keystore Support for Oracle GoldenGate](#)

You can configure Transparent Data Encryption keystore support for Oracle GoldenGate by using a shared secret for the keystore.

6.4.1 About Storing Oracle GoldenGate Secrets in Keystores

You can use a keystore to store secret keys for tools and external clients such as Oracle GoldenGate.

The secret key must be a string adhering to Oracle identifier rules. You can add, update, or delete a client secret in an existing keystore. This section describes how to capture Transparent Data Encryption encrypted data in the Oracle GoldenGate Extract (Extract) process using classic capture mode.

TDE support when Extract is in classic capture mode requires the exchange of the following keys:

- TDE support for Oracle GoldenGate in the classic capture mode of the Extract process requires that an Oracle database and the Extract process share the secret to encrypt sensitive information being exchanged. The shared secret is stored securely in the Oracle database and Oracle GoldenGate domains. The shared secret is stored in the software keystore or the external keystore as the database secret.
- The decryption key is a password known as the shared secret that is stored securely in the Oracle database and Oracle GoldenGate domains. Only a party that has possession of the shared secret can decrypt the table and redo log keys.

After you configure the shared secret, Oracle GoldenGate Extract uses the shared secret to decrypt the data. Oracle GoldenGate Extract does not handle the TDE master encryption key itself, nor is it aware of the keystore password. The TDE master encryption key and password remain within the Oracle database configuration.

Oracle GoldenGate Extract only writes the decrypted data to the Oracle GoldenGate trail file, which Oracle GoldenGate persists during transit. You can protect this file using your site's operating system standard security protocols, as well as the Oracle GoldenGate AES encryption options. Oracle GoldenGate does not write the encrypted data to a discard file (specified with the DISCARDFILE parameter). The word ENCRYPTED will be written to any discard file that is in use.

Oracle GoldenGate does require that the keystore be open when processing encrypted data. There is no performance effect of Oracle GoldenGate feature on the TDE operations.

6.4.2 Oracle GoldenGate Extract Classic Capture Mode TDE Requirements

Ensure that you meet the requirements for Oracle GoldenGate Extract to support Transparent Data Encryption capture.

The requirements are as follows:

- To maintain high security standards, ensure that the Oracle GoldenGate Extract process runs as part of the Oracle user (the user that runs the Oracle database). That way, the keys are protected in memory by the same privileges as the Oracle user.
- Run the Oracle GoldenGate Extract process on the same computer as the Oracle database installation.

6.4.3 Configuring Keystore Support for Oracle GoldenGate

You can configure Transparent Data Encryption keystore support for Oracle GoldenGate by using a shared secret for the keystore.

- [Step 1: Decide on a Shared Secret for the Keystore](#)
A shared secret for a keystore is a password.
- [Step 2: Configure Oracle Database for TDE Support for Oracle GoldenGate](#)
The DBMS_INTERNAL_CLKM PL/SQL package enables you to configure TDE support for Oracle GoldenGate.
- [Step 3: Store the TDE GoldenGate Shared Secret in the Keystore](#)
The ADMINISTER KEY MANAGEMENT statement can store a TDE GoldenGate shared secret in a keystore.

- [Step 4: Set the TDE Oracle GoldenGate Shared Secret in the Extract Process](#)
The GoldenGate Software Command Interface (GGSCI) utility sets the TDE Oracle GoldenGate shared secret in the extract process.

6.4.3.1 Step 1: Decide on a Shared Secret for the Keystore

A shared secret for a keystore is a password.

- Decide on a shared secret that meets or exceeds Oracle Database password standards.

Do not share this password with any user other than trusted administrators who are responsible for configuring Transparent Data Encryption to work with Oracle GoldenGate Extract.

Related Topics

- *Oracle Database Security Guide*

6.4.3.2 Step 2: Configure Oracle Database for TDE Support for Oracle GoldenGate

The DBMS_INTERNAL_CLKM PL/SQL package enables you to configure TDE support for Oracle GoldenGate.

1. Log in to the database instance as user SYS with the SYSDBA administrative privilege.

For example

```
sqlplus sys as sysdba
Enter password: password
Connected.
```

2. Load the Oracle Database-supplied DBMS_INTERNAL_CLKM PL/SQL package.

For example:

```
@?/app/oracle/product/12.2/rdbms/admin/prvtclkm.plb
```

The prvtclkm.plb file also enables Oracle GoldenGate to extract encrypted data from an Oracle database.

3. Grant the EXECUTE privilege on the DBMS_INTERNAL_CLKM PL/SQL package to the Oracle GoldenGate Extract database user.

For example:

```
GRANT EXECUTE ON DBMS_INTERNAL_CLKM TO psmith;
```

This procedure enables the Oracle database and Oracle GoldenGate Extract to exchange information.

4. Exit SQL*Plus.

6.4.3.3 Step 3: Store the TDE GoldenGate Shared Secret in the Keystore

The ADMINISTER KEY MANAGEMENT statement can store a TDE GoldenGate shared secret in a keystore.

Before you begin this procedure, ensure that you have configured the TDE software or external keystore, based on [Configuring a TDE Wallet](#) and [Configuring an External Keystore](#).

1. Set the Oracle GoldenGate-TDE key in the keystore by using the following syntax.

```
ADMINISTER KEY MANAGEMENT ADD|UPDATE|DELETE SECRET 'secret'
FOR CLIENT 'secret_identifier' [USING TAG 'tag']
IDENTIFIED BY keystore_password [WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *secret* is the client secret key to be stored, updated, or deleted. Enclose this setting in single quotation marks (' ').
- *secret_identifier* is an alphanumeric string used to identify the secret key. *secret_identifier* does not have a default value. Enclose this setting in single quotation marks (' ').
- *tag* is an optional, user-defined description for the secret key to be stored. *tag* can be used with the ADD and UPDATE operations. Enclose this setting in single quotation marks (' '). This tag appears in the SECRET_TAG column of the V\$CLIENT_SECRETS view. [Creating Custom TDE Master Encryption Key Attributes for Reports](#)
- *keystore_password* is the password for the keystore that is configured.
- WITH BACKUP is required in case the keystore was not backed up before the ADD, UPDATE or DELETE operation. *backup_identifier* is an optional user-defined description for the backup. Enclose *backup_identifier* in single quotation marks (' ').

The following example adds a secret key to the keystore and creates a backup in the same directory as the keystore:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'some_secret'
FOR CLIENT 'ORACLE_GG' USING TAG 'GoldenGate Secret'
IDENTIFIED BY password WITH BACKUP USING 'GG backup';
```

2. Verify the entry that you just created.

For example:

```
SELECT CLIENT, SECRET_TAG FROM V$CLIENT_SECRETS WHERE CLIENT = 'ORACLEGG';
```

```
CLIENT    SECRET_TAG
-----
ORACLEGG  some_secret
```

3. Switch the log files.

```
CONNECT / AS SYSDBA

ALTER SYSTEM SWITCH LOGFILE;
```

See Also

- [Creating Custom TDE Master Encryption Key Attributes for Reports](#) for more information about tags
- *Oracle Database Administrator's Guide* for more information about switching log files
- [How Transparent Data Encryption Works with Oracle Real Application Clusters](#) if you are having problems using this procedure in an Oracle Real Application Clusters environment

6.4.3.4 Step 4: Set the TDE Oracle GoldenGate Shared Secret in the Extract Process

The GoldenGate Software Command Interface (GGSCI) utility sets the TDE Oracle GoldenGate shared secret in the extract process.

1. Start the GGSCI utility.

For example:

```
ggsci
```

2. In the GGSCI utility, run the `ENCRYPT PASSWORD` command to encrypt the shared secret within the Oracle GoldenGate Extract parameter file.

```
ENCRYPT PASSWORD shared_secret algorithm ENCRYPTKEY keyname
```

In this specification:

- *shared_secret* is the clear-text shared secret that you created when you decided on a shared secret for the keystore. This setting is case sensitive.
- *algorithm* is one of the following values to specify AES encryption:
 - AES128
 - AES192
 - AES256
- *keyname* is the logical name of the encryption key in the ENCKEYS lookup file. Oracle GoldenGate uses this name to look up the actual key in the ENCKEYS file.

For example:

```
ENCRYPT PASSWORD password AES256 ENCRYPTKEY mykey1
```

3. In the Oracle GoldenGate Extract parameter file, set the `DBOPTIONS` parameter with the `DECRYPTPASSWORD` option.

As input, supply the encrypted shared secret and the Oracle GoldenGate-generated or user-defined decryption key.

```
DBOPTIONS DECRYPTPASSWORD shared_secret algorithm ENCRYPTKEY keyname
```

In this specification:

- *shared_secret* is the clear-text shared secret that you created when you decided on a shared secret for the keystore. This setting is case sensitive.
- *algorithm* is one of the following values to specify AES encryption:
 - AES128
 - AES192
 - AES256
- *keyname* is the logical name of the encryption key in the ENCKEYS lookup file.

For example:

```
DBOPTIONS DECRYPTPASSWORD AACAAAAAAAAAAIALCKDZIRH0JBH0JUH AES256 ENCRYPTKEY  
mykey1
```

Related Topics

- [Step 1: Decide on a Shared Secret for the Keystore](#)
A shared secret for a keystore is a password.

7

Managing Keystores and TDE Master Encryption Keys in United Mode

United mode enables you to create a common keystore for the CDB and the PDBs for which the keystore is in united mode.

The keys for the CDB and the PDBs reside in the common keystore.

- [About Managing Keystores and TDE Master Encryption Keys in United Mode](#)
In united mode, you create the keystore and TDE master encryption key for CDB and PDBs that reside in the same keystore.
- [Operations That Are Allowed in United Mode](#)
Many ADMINISTER KEY MANAGEMENT operations performed in the CDB root apply to keystores and encryption keys in the united mode PDB.
- [Operations That Are Not Allowed in a United Mode PDB](#)
ADMINISTER KEY MANAGEMENT operations that are not allowed in a united mode PDB can be performed in the CDB root.
- [Configuring United Mode with the Initialization Parameter File and ALTER SYSTEM](#)
If your environment relies on server parameter files (spfile), then you can set WALLET_ROOT and TDE_CONFIGURATION using ALTER SYSTEM SET with SCOPE.
- [Configuring a Software Keystore for Use in United Mode](#)
In united mode, the software keystore resides in the CDB root but the master keys from this keystore are available for the PDBs that have their keystore in united mode.
- [Configuring a Container Database with United Mode PDBs for Oracle Key Vault](#)
The TDE master keys of all united mode PDBs and the CDB reside in the same virtual wallet in Oracle Key Vault.
- [Administering Keystores and TDE Master Encryption Keys in United Mode](#)
After you configure a keystore and master encryption key for use in united mode, you can perform tasks such as rekeying TDE master encryption keys.
- [Administering Transparent Data Encryption in United Mode](#)
You can perform general administrative tasks with Transparent Data Encryption in united mode.

7.1 About Managing Keystores and TDE Master Encryption Keys in United Mode

In united mode, you create the keystore and TDE master encryption key for CDB and PDBs that reside in the same keystore.

The keys for PDBs having keystore in united mode, can be created from CDB root or from the PDB.

This design enables you to have one keystore to manage the entire CDB environment, enabling the PDBs to share this keystore, but you can customize the behavior of this keystore in the individual united mode PDBs. For example, in a united mode PDB, you can configure a

TDE master encryption key for the PDB in the united keystore that you created in the CDB root, open the keystore locally, and close the keystore locally. In order to perform these actions, the keystore in the CDB root must be open.

Before you configure your environment to use united mode or isolated mode, all the PDBs in the CDB environment are considered to be in united mode.

To use united mode, you must follow these general steps:

1. In the CDB root, configure the database to use united mode by setting the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters.

The `WALLET_ROOT` parameter sets the location for the wallet directory and the `TDE_CONFIGURATION` parameter sets the type of keystore to use.
2. Restart the database so that these settings take effect.
3. In the CDB root, create the keystore, open the keystore, and then create the TDE master encryption key.
4. In each united mode PDB, perform TDE master encryption key tasks as needed, such as opening the keystore locally in the united mode PDB and creating the TDE master encryption key for the PDB. Remember that the keystore is managed by the CDB root, but must contain a TDE master encryption key that is specific to the PDB for the PDB to be able to use TDE.

When you run `ADMINISTER KEY MANAGEMENT` statements in united mode from the CDB root, if the statement accepts the `CONTAINER` clause, and if you set it to `ALL`, then the statement applies only to the CDB root and its associated united mode PDBs. Any PDB that is in isolated mode is not affected.

7.2 Operations That Are Allowed in United Mode

Many `ADMINISTER KEY MANAGEMENT` operations performed in the CDB root apply to keystores and encryption keys in the united mode PDB.

Available United Mode-Related Operations in a CDB Root

[Table 7-1](#) describes the `ADMINISTER KEY MANAGEMENT` operations that you can perform in the CDB root.

Table 7-1 ADMINISTER KEY MANAGEMENT United Mode Operations in a CDB Root

Operation	Syntax	United Mode Notes
Creating a keystore	<code>ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY <i>keystore_password</i>;</code>	After you create the keystore in the CDB root, by default it is available in the united mode PDBs. Do not include the <code>CONTAINER</code> clause.
Opening a keystore	<code>ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>] [CONTAINER = ALL CURRENT];</code>	In this operation, the <code>EXTERNAL STORE</code> clause fetches the keystore password from the SSO wallet located in the <code>WALLET_ROOT/tde_seps</code> directory.

Table 7-1 (Cont.) ADMINISTER KEY MANAGEMENT United Mode Operations in a CDB Root

Operation	Syntax	United Mode Notes
Changing a keystore password	ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD IDENTIFIED BY <i>old_keystore_password</i> SET <i>new_keystore_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '];	Do not include the CONTAINER clause.
Backing up a keystore	ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE [USING ' <i>backup_identifier</i> '] [FORCE KEYSTORE] IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>] [TO ' <i>keystore_location</i> '];	Do not include the CONTAINER clause.
Closing a keystore without force	ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE [IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>]] [CONTAINER = ALL CURRENT];	-
Closing a keystore with force	ADMINISTER KEY MANAGEMENT FORCE KEYSTORE CLOSE [IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>]] [CONTAINER = ALL CURRENT];	-
Creating and activating a new TDE master encryption key (rekeying)	ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY [FORCE KEYSTORE] [USING TAG ' <i>tag_name</i> '] IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>] WITH BACKUP [USING ' <i>backup_identifier</i> '] [CONTAINER = ALL CURRENT]	-
Creating a user-defined TDE master encryption key for either now (SET) or later on (CREATE)	ADMINISTER KEY MANAGEMENT [SET CREATE] [ENCRYPTION] KEY ' <i>mkid:mk</i> <i>mk</i> ' [USING ALGORITHM ' <i>algorithm</i> '] [FORCE KEYSTORE] [USING TAG ' <i>tag_name</i> '] IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>] [WITH BACKUP [USING ' <i>backup_identifier</i> ']] [CONTAINER = CURRENT];	-

Table 7-1 (Cont.) ADMINISTER KEY MANAGEMENT United Mode Operations in a CDB Root

Operation	Syntax	United Mode Notes
Activating an existing TDE master encryption key	ADMINISTER KEY MANAGEMENT USE [ENCRYPTION] KEY 'key_id' IDENTIFIED BY [EXTERNAL STORE keystore_password] WITH BACKUP [USING 'backup_identifier'];	Do not include the CONTAINER clause.
Tagging a TDE master encryption key	ADMINISTER KEY MANAGEMENT SET TAG 'tag' FOR 'key_id' IDENTIFIED BY [EXTERNAL STORE keystore_password] WITH BACKUP [USING 'backup_identifier'];	Do not include the CONTAINER clause.
Moving a TDE master encryption key to a new keystore	ADMINISTER KEY MANAGEMENT MOVE [ENCRYPTION] KEYS TO NEW KEYSTORE 'keystore_location1' IDENTIFIED BY keystore1_password FROM [FORCE] KEYSTORE IDENTIFIED BY keystore_password [WITH IDENTIFIER IN { 'key_id' [, 'key_id']... (subquery) }] WITH BACKUP [USING 'backup_identifier'];	You can only move the master encryption key to a keystore that is within the same container (for example, between keystores in the CDB root or between keystores in the same PDB). You cannot move the master encryption key from a keystore in the CDB root to a keystore in a PDB, and vice versa. Do not include the CONTAINER clause.

Available Operations in a United Mode PDB

[Table 7-2](#) describes the ADMINISTER KEY MANAGEMENT operations that you can perform in a united mode PDB.

Table 7-2 ADMINISTER KEY MANAGEMENT United Mode PDB Operations

Operation	Syntax	United Mode Notes
Opening a keystore	ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE [IDENTIFIED BY EXTERNAL STORE keystore_password] [CONTAINER = CURRENT];	In this operation, the EXTERNAL_STORE clause uses the password in the Secure Sockets Layer (SSL) wallet. This wallet is located in the tde_seps directory in the WALLET_ROOT location.
Closing a keystore without force	ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE [IDENTIFIED BY keystore_password] [CONTAINER = CURRENT];	-

Table 7-2 (Cont.) ADMINISTER KEY MANAGEMENT United Mode PDB Operations

Operation	Syntax	United Mode Notes
Closing a keystore with force	ADMINISTER KEY MANAGEMENT FORCE KEYSTORE CLOSE IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>] [CONTAINER = CURRENT];	-
Creating and activating a new TDE master encryption key (rekeying or rotating)	ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY [FORCE KEYSTORE] [USING TAG ' <i>tag_name</i> '] IDENTIFIED BY EXTERNAL STORE <i>keystore_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '] [CONTAINER = CURRENT];	-
Creating a user-defined TDE master encryption key for use either now (SET) or later on (CREATE)	ADMINISTER KEY MANAGEMENT SET CREATE [ENCRYPTION] KEY ' <i>mkid</i> : <i>mk</i> <i>mk</i> ' [USING ALGORITHM ' <i>algorithm</i> '] [FORCE KEYSTORE] [USING TAG ' <i>tag</i> '] IDENTIFIED BY EXTERNAL STORE <i>keystore_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '] [CONTAINER = CURRENT];	-
Activating an existing TDE master encryption key	ADMINISTER KEY MANAGEMENT USE [ENCRYPTION] KEY ' <i>key_id</i> ' IDENTIFIED BY EXTERNAL STORE <i>keystore_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '];	Do not include the CONTAINER clause.
Tagging a TDE master encryption key	ADMINISTER KEY MANAGEMENT SET TAG ' <i>tag</i> ' FOR ' <i>key_id</i> ' [FORCE KEYSTORE] IDENTIFIED BY EXTERNAL STORE <i>keystore_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '];	Do not include the CONTAINER clause.

Table 7-2 (Cont.) ADMINISTER KEY MANAGEMENT United Mode PDB Operations

Operation	Syntax	United Mode Notes
Moving an encryption key to a new keystore	<pre>ADMINISTER KEY MANAGEMENT MOVE [ENCRYPTION] KEYS TO NEW KEYSTORE 'new_keystore_location' IDENTIFIED BY new_keystore_password FROM [FORCE] KEYSTORE IDENTIFIED BY keystore_password [WITH IDENTIFIER IN { 'key_id' [, 'key_id']... (subquery) }] WITH BACKUP [USING 'backup_identifier'];</pre>	Do not include the CONTAINER clause.
Moving a key from a united mode keystore in the CDB root to an isolated mode keystore in a PDB	<pre>ADMINISTER KEY MANAGEMENT ISOLATE KEYSTORE IDENTIFIED BY isolated_keystore_password FROM ROOT KEYSTORE [FORCE KEYSTORE] IDENTIFIED BY EXTERNAL STORE united_keystore_password WITH BACKUP [USING backup_id];</pre>	Do not include the CONTAINER clause.
Using the FORCE clause when a clone of a PDB is using the TDE master encryption key that is being isolated; then copying (rather than moving) the TDE master encryption keys from the keystore that is in the CDB root into the isolated mode keystore of the PDB	<pre>ADMINISTER KEY MANAGEMENT FORCE ISOLATE KEYSTORE IDENTIFIED BY isolated_keystore_password FROM ROOT KEYSTORE [FORCE KEYSTORE] IDENTIFIED BY [EXTERNAL STORE united_keystore_password] [WITH BACKUP [USING backup_id]];</pre>	-

7.3 Operations That Are Not Allowed in a United Mode PDB

ADMINISTER KEY MANAGEMENT operations that are not allowed in a united mode PDB can be performed in the CDB root.

These operations are as follows:

- Keystore operations:
 - Performing merge operations on keystores
 - Exporting a keystore
 - Importing a keystore

- Migrating a keystore
- Reverse-migrating a keystore
- Moving the keys of a keystore that is in the CDB root into the keystores of a PDB
- Moving the keys from a PDB into a united mode keystore that is in the CDB root
- Encryption key operations:
 - Using the CONTAINER = ALL clause to create a new TDE master encryption key for later user in each pluggable database (PDB)
- Client secret operations:
 - Adding client secrets
 - Updating client secrets
 - Deleting client secrets

7.4 Configuring United Mode with the Initialization Parameter File and ALTER SYSTEM

If your environment relies on server parameter files (spfile), then you can set WALLET_ROOT and TDE_CONFIGURATION using ALTER SYSTEM SET with SCOPE.

1. Log in to the server where the CDB root of the Oracle database resides.
2. If necessary, create a wallet directory.

This directory cannot contain any wallets. Oracle recommends that you create a directory outside of \$ORACLE_HOME or \$ORACLE_BASE, to avoid backing up the keystore (wallet or Oracle Key Vault installation tree) with the encrypted database during an operating system-level backup.

3. Edit the initialization parameter file to include the WALLET_ROOT static initialization parameter for the wallet directory.

By default, the initialization parameter file is located in the \$ORACLE_HOME/dbs directory.

For example, for a database instance named orcl:

```
wallet_root=$ORACLE_BASE/admin/orcl/wallet
```

4. Log in to the CDB root as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
5. Restart the database.

```
SHUTDOWN IMMEDIATE  
STARTUP
```

To start the database by pointing to the location of the initialization file where you added the WALLET_ROOT setting, issue a STARTUP command similar to the following:

```
STARTUP PFILE = /u01/oracle/dbs/init.ora
```

6. Check the WALLET_ROOT setting.

```
SHOW PARAMETER WALLET_ROOT
```

The VALUE column of the output should show the absolute path location of the wallet directory.

For example:

```
wallet_root string /etc/ORACLE/KEYSTORES/finance
```

7. Set the TDE_CONFIGURATION dynamic initialization parameter to specify the keystore type, using the following syntax:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=keystore_type"
SCOPE=scope_type;
```

In this specification:

- *keystore_type* can be one of the following settings:
 - FILE configures a TDE keystore.
 - OKV configures an Oracle Key Vault keystore.
- *scope_type* sets the type of scope (for example, both, memory, spfile, pfile).

For example, to configure a TDE keystore if the parameter file (pfile) is in use, set scope to memory:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE"
SCOPE=memory;
```

To configure a TDE keystore if the server parameter file (spfile) is in use, set scope to both:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE"
SCOPE=both;
```

8. Check the TDE_CONFIGURATION parameter setting.

```
SHOW PARAMETER TDE_CONFIGURATION
```

The VALUE column should show the keystore type, prepended with KESTORE_CONFIGURATION=.

9. Confirm that the TDE_CONFIGURATION parameter was set correctly.

```
SELECT CON_ID, KEYSTORE_MODE FROM V$ENCRYPTION_WALLET;
```

The output should be similar to the following:

```
CON_ID KEYSTORE
-----
1 NONE
2 UNITED
3 UNITED
4 UNITED
5 UNITED
```

The CDB root (CON_ID 1) will always be in the NONE state, and at this stage, the remaining CON_IDs should be set to UNITED. PDBs can be either UNITED or ISOLATED, depending on how you configure them. When you query the V\$ENCRYPTION_WALLET view, if the ORA-46691: The value of the KESTORE_CONFIGURATION attribute is invalid error appears, then check the initialization parameter file where you added this setting.

After you configure united mode, you can create keystores and master encryption keys, and when these are configured, you can encrypt data.

7.5 Configuring a Software Keystore for Use in United Mode

In united mode, the software keystore resides in the CDB root but the master keys from this keystore are available for the PDBs that have their keystore in united mode.

- [About Configuring a Software Keystore in United Mode](#)
In united mode, the keystore that you create in the CDB root will be accessible by the united mode PDBs.
- [Creating a Password-Protected TDE Wallet](#)
In united mode, you must create the keystore in the CDB root.
- [Step 2: Open the TDE Wallet in a United Mode PDB](#)
To open a TDE wallet in united mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.
- [Step 3: Set the TDE Master Encryption Key in the TDE Wallet in United Mode](#)
To set the TDE master encryption key in the TDE wallet when the PDB is configured in united mode, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause.

7.5.1 About Configuring a Software Keystore in United Mode

In united mode, the keystore that you create in the CDB root will be accessible by the united mode PDBs.

In general, to configure a united mode software keystore after you have enabled united mode, you create and open the keystore in the CDB root, and then create a master encryption key for this keystore. Afterward, you can begin to encrypt data for tables and tablespaces that will be accessible throughout the CDB environment.

The `V$ENCRYPTION_WALLET` dynamic view describes the status and location of the keystore. For example, the following query shows the open-closed status and the keystore location of the CDB root keystore (`CON_ID 1`) and its associated united mode PDBs. The `WRL_PARAMETER` column shows the CDB root keystore location being in the `$ORACLE_BASE/wallet/tde` directory.

```
SELECT CON_ID, STATUS, WRL_PARAMETER FROM V$ENCRYPTION_WALLET;
```

```
CON_ID STATUS WRL_PARAMETER
-----
1 OPEN      /app/oracle/wallet/tde/
2 CLOSED
3 OPEN
4 OPEN
5 OPEN
```

In this output, there is no keystore path listed for the other PDBs in this CDB because these PDBs use the keystore in the CDB root. If any of these PDBs are isolated and you create a keystore in the isolated mode PDB, then when you perform this query, the `WRL_PARAMETER` column will show the keystore path for the isolated mode PDB.

You can create a secure external store for the software keystore. This feature enables you to hide the password from the operating system: it removes the need for storing clear-text keystore passwords in scripts or other tools that can access the database without user intervention, such as overnight batch scripts. The location for this keystore is set by the `EXTERNAL_KEYSTORE_CREDENTIAL_LOCATION` initialization parameter. In a multitenant environment, different PDBs can access this external store location when you run the `ADMINISTER KEY MANAGEMENT` statement using the `IDENTIFIED BY EXTERNAL STORE` clause.

This way, you can centrally locate the password and then update it only once in the external store.

7.5.2 Creating a Password-Protected TDE Wallet

In united mode, you must create the keystore in the CDB root.

After you create this TDE wallets in the CDB root, it becomes available in any united mode PDB, but not in any isolated mode PDBs.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.

For example:

```
CONNECT c##sec_admin AS SYSKM
Enter password: password
```

2. Use the SHOW PARAMETER command to confirm that WALLET_ROOT is set, and TDE_CONFIGURATION is set to KEYSTORE_CONFIGURATION=FILE.
3. Run the ADMINISTER KEY MANAGEMENT SQL statement to create the keystore using the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE
IDENTIFIED BY TDE_wallet_password;
```

The /tde directory is automatically created under WALLET_ROOT, if it does not already exist. In Oracle Automatic Storage Management (Oracle ASM), for the /tde directory to be automatically created if it does not already exist, WALLET_ROOT must point to +DATA/ORACLE_UNQNAME.

In this specification, *TDE_wallet_password* is the password of the TDe wallet that you, the security administrator, creates.

For example, to create the TDE wallet in the *WALLET_ROOT*/tde directory:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY TDE_wallet_password;
```

keystore altered.

After you run this statement, the ewallet.p12 file, which is the TDE wallet, appears in the wallet location.

After you complete these steps, the ewallet.p12 file, which contains the keystore, appears in the designated keystore location. For example, if you had set the WALLET_ROOT parameter to \$ORACLE_BASE/wallet and the TDE_CONFIGURATION parameter to FILE (for TDE, which creates a tde directory in the wallet root location), then the keystore will be created in the /etc/ORACLE/KEYSTORES/\${ORACLE_SID}/tde directory. The name of the TDE wallet is ewallet.p12.

Related Topics

- [Configuring an External Store for a Keystore Password](#)
An external store for a keystore password stores the keystore password in a centrally accessed and managed location.

7.5.3 Step 2: Open the TDE Wallet in a United Mode PDB

To open a TDE wallet in united mode, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE OPEN clause.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Open the TDE wallet in the CDB root.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
IDENTIFIED BY password;
```

keystore altered.

If the CDB is configured with the TDE wallet password stored in `WALLET_ROOT/tde_seps` and has a (local) auto-login TDE wallet in `WALLET_ROOT/tde`, you must include the `FORCE KEYSTORE` clause and the `IDENTIFIED BY EXTERNAL STORE` clause in the `ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN` statement, as follows:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE;
```

keystore altered.

If the `WALLET_ROOT` parameter has been set, then Oracle Database finds the external store in the `WALLET_ROOT/tde_seps` directory.

3. Ensure that the PDB in which you want to open the TDE wallet is in READ WRITE mode.

For example:

```
SHOW PDBS
```

CON_ID	CON_NAME	OPEN MODE	RESTRICTED
2	PDB\$SEED	READ ONLY	NO
3	CDB1_PDB1	READ WRITE	NO

If any PDB has an `OPEN MODE` value that is different from `READ WRITE`, then run the following statement to open the PDB, which will set it to `READ WRITE` mode:

```
ALTER PLUGGABLE DATABASE CDB1_PDB1 OPEN;
```

Now the TDE wallet can be opened in both the CDB root and the PDB.

4. Connect to the PDB.
5. Run the `ADMINISTER KEY MANAGEMENT` statement to open the TDE wallet.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
IDENTIFIED BY password | EXTERNAL STORE;
keystore altered.
```

To open a password-protected TDE wallet when a (local) auto-login TDE wallet is open, specify the `FORCE KEYSTORE` clause as follows.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
FORCE KEYSTORE
IDENTIFIED BY TDE_wallet_password | EXTERNAL STORE;
keystore altered.
```

`FORCE KEYSTORE` is also useful for databases that are heavily loaded. The `IDENTIFIED BY EXTERNAL STORE` clause is included in the statement because the TDE wallet credentials exist in an external store. This enables the password-protected TDE wallet to be opened without specifying the TDE wallet password within the statement itself.

If the `WALLET_ROOT` parameter has been set in the CDB, then united PDBs find the external store in the `WALLET_ROOT/tde_seps` directory.

6. Confirm that the TDE wallet is open.

```
SELECT STATUS FROM V$ENCRYPTION_WALLET;
```

Note that if the TDE wallet is open but you have not created a TDE master encryption key yet, the `STATUS` column of the `V$ENCRYPTION_WALLET` view reminds you with an `OPEN_NO_MASTER_KEY` status.

7.5.4 Step 3: Set the TDE Master Encryption Key in the TDE Wallet in United Mode

To set the TDE master encryption key in the TDE wallet when the PDB is configured in united mode, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause.

1. Log in to the CDB root or to the PDB that is configured for united mode as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Ensure that the database is open in `READ WRITE` mode.

To find the status, run the `show pdbs` command.

3. Run the `ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY` statement to create or rekey the TDE master encryption key in the TDE wallet.

For example, if the TDE wallet is password-protected and open, and you want to create or rekey the TDE master encryption key in the current container:

```
ADMINISTER KEY MANAGEMENT SET KEY  
IDENTIFIED BY keystore_password  
WITH BACKUP USING 'emp_key_backup';
```

keystore altered.

If the keystore is closed:

```
ADMINISTER KEY MANAGEMENT SET KEY  
FORCE KEYSTORE  
IDENTIFIED BY keystore_password  
WITH BACKUP USING 'emp_key_backup';
```

keystore altered.

In this specification:

- `FORCE KEYSTORE` should be included if the TDE wallet is closed. This automatically opens the keystore before setting the TDE master encryption key. The `FORCE KEYSTORE` clause also switches over to opening the password-protected TDE wallet when an auto-login keystore is configured and is currently open.
 - `IDENTIFIED BY` specifies the TDE wallet password. Alternatively, if the TDE wallet password is in an external store, you can use the `IDENTIFIED BY EXTERNAL STORE` clause.
4. Confirm that the TDE master encryption key is set.

```
SELECT MASTERKEY_ACTIVATED FROM V$DATABASE_KEY_INFO;
```

The output should be `YES`.

Related Topics

- [About Setting the TDE Wallet TDE Master Encryption Key](#)
The TDE master encryption key is stored in the TDE wallet.

7.6 Configuring a Container Database with United Mode PDBs for Oracle Key Vault

The TDE master keys of all united mode PDBs and the CDB reside in the same virtual wallet in Oracle Key Vault.

- [About Configuring a Container Database with United Mode PDBs for Oracle Key Vault](#)
In united mode, you can configure Oracle Key Vault by setting the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters in the container database (CDB).
- [Step 1: Configure Oracle Key Vault for United Mode](#)
You can configure Oracle Key Vault for united mode PDBs by setting the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters in the container database (CDB).
- [Step 2: Open the Connection to Oracle Key Vault](#)
After you have configured the database to use Oracle Key Vault for TDE key management, you must open the connection to Oracle Key Vault before you can use it.
- [Step 3: Set the TDE Master Encryption Key in Oracle Key Vault](#)
After you have opened the connection to Oracle Key Vault, you are ready to set the TDE master encryption key.
- [Step 4: Encrypt Your Data in United Mode](#)
Now that you have completed the configuration for an external keystore or for an Oracle Key Vault keystore, you can begin to encrypt data.

7.6.1 About Configuring a Container Database with United Mode PDBs for Oracle Key Vault

In united mode, you can configure Oracle Key Vault by setting the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters in the container database (CDB).

Oracle recommends that you set the parameters `WALLET_ROOT` and `TDE_CONFIGURATION` for new deployments. Alternatively, you can migrate from the old configuration in the `sqlnet.ora` file to the new configuration with `WALLET_ROOT` and `TDE_CONFIGURATION` at your earliest convenience (for example, the next time you apply a quarterly bundle patch).

United Mode is the default TDE setup that is used in Oracle Database release 12.1.0.2 and later with the TDE configuration in `sqlnet.ora`. In Oracle Database release 18c and later, TDE configuration in `sqlnet.ora` is deprecated. You must first set the static initialization parameter `WALLET_ROOT` to an existing directory; for this change to be picked up, a database restart is necessary. After the restart, set the `KEYSTORE_CONFIGURATION` attribute of the dynamic `TDE_CONFIGURATION` parameter to `OKV` (for a password-protected connection into Oracle Key Vault), or `OKV|FILE` for an auto-open connection into Oracle Key Vault, and then open the configured external keystore, and then set the TDE master encryption keys. After you complete these tasks, you can begin to encrypt data in your database.

7.6.2 Step 1: Configure Oracle Key Vault for United Mode

You can configure Oracle Key Vault for united mode PDBs by setting the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters in the container database (CDB).

1. If the `WALLET_ROOT` parameter is set, then install the Oracle Key Vault client software into the `WALLET_ROOT/okv` directory.
2. Log in to the database instance as a user who has been granted the `ALTER SYSTEM` administrative privilege.
3. Set the `TDE_CONFIGURATION` dynamic initialization parameter to specify the keystore type by using the following syntax:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=keystore_type" SCOPE=BOTH  
SID = '*';
```

In this specification:

- `keystore_type` is `OKV`, to configure an Oracle Key Vault keystore.

For example, to configure your database to use Oracle Key Vault:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=OKV"  
SCOPE=BOTH SID = '*';
```

7.6.3 Step 2: Open the Connection to Oracle Key Vault

After you have configured the database to use Oracle Key Vault for TDE key management, you must open the connection to Oracle Key Vault before you can use it.

- [About Opening the Connection to Oracle Key Vault](#)
You must open the connection to Oracle Key Vault so that it is accessible to the database before you can perform any encryption or decryption.
- [Opening the Oracle Key Vault Connection in a United Mode PDB](#)
To open the Oracle Key Vault connection in united mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.

7.6.3.1 About Opening the Connection to Oracle Key Vault

You must open the connection to Oracle Key Vault so that it is accessible to the database before you can perform any encryption or decryption.

If a recovery operation is needed on your database (for example, if the database was not cleanly shut down, and has an encrypted tablespace that needs recovery), then you must open the connection to Oracle Key Vault before you can open the database itself.

There are two ways that you can open the Oracle Key Vault connection:

- Manually open the keystore by issuing the `ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN` statement. Afterward, you can perform the operation.
- Include the `FORCE KEYSTORE` clause in the `ADMINISTER KEY MANAGEMENT` statement. `FORCE KEYSTORE` temporarily opens the keystore for the duration of the operation, and when the operation completes, the keystore is closed again. `FORCE KEYSTORE` is useful for situations when the database is heavily loaded. In this scenario, because of concurrent access to encrypted objects in the database, the auto-login keystore continues to open immediately

after it has been closed but before a user has had a chance to open the password-based keystore.

To check the status of the keystore, query the STATUS column of the V\$ENCRYPTION_WALLET view. Keystores can be in the following states: CLOSED, NOT_AVAILABLE (that is, not present in the WALLET_ROOT location), OPEN, OPEN_NO_MASTER_KEY, OPEN_UNKNOWN_MASTER_KEY_STATUS.

Be aware that for external keystores, if the database is in the mounted state, then it cannot check if the master key is set because the data dictionary is not available. In this situation, the status will be OPEN_UNKNOWN_MASTER_KEY_STATUS.

Related Topics

- [How Keystore Open and Close Operations Work in United Mode](#)
You should be aware of how keystore open and close operations work in united mode.

7.6.3.2 Opening the Oracle Key Vault Connection in a United Mode PDB

To open the Oracle Key Vault connection in united mode, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE OPEN clause.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Open the keystore in the CDB root by using the following syntax.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY EXTERNAL STORE | "Oracle_Key_Vault_password"  
[CONTAINER = ALL | CURRENT];
```

In this specification:

- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.
 - *Oracle_Key_Vault_password* is the Oracle Key Vault password that was given during the Oracle Key Vault client installation. If no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL. Enclose this password in double quotation marks.
- CONTAINER specifies the scope in which to open the Oracle Key Vault connection.
 - ALL opens the connection to Oracle Key Vault for the root container and all open united PDBs.
 - CURRENT opens the connection to Oracle Key Vault for the root container.

If the password contains a semi-colon (;), then enclose the password in double quotation marks.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY "Oracle_Key_Vault_password";
```

3. Ensure that the PDB in which you want to open the keystore is in READ WRITE mode.
4. Connect to the PDB and run the ADMINISTER KEY MANAGEMENT statement to open the keystore.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY "external_key_manager_password";
```


5. Confirm that the keystore is open.

```
SELECT STATUS FROM V$ENCRYPTION_WALLET;
```

6. Repeat this procedure each time you restart the PDB.

7.6.4 Step 3: Set the TDE Master Encryption Key in Oracle Key Vault

After you have opened the connection to Oracle Key Vault, you are ready to set the TDE master encryption key.

- [About Setting the External Keystore TDE Master Encryption Key](#)
You must create a TDE master encryption key that is stored inside the external keystore.
- [Heartbeat Batch Size for External Kestores](#)
You can control the size of the batch of heartbeats issued during each heartbeat period.
- [Setting the TDE Master Encryption Key in the United Mode External Keystore](#)
To set the TDE master encryption key in the keystore when the PDB is configured in united mode, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause.
- [Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS](#)
To switch from a TDE wallet to centralized key management with Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS), after you upload all current and retired TDE master keys you must migrate the database from the TDE wallet to Oracle Key Vault or OCI KMS.

7.6.4.1 About Setting the External Keystore TDE Master Encryption Key

You must create a TDE master encryption key that is stored inside the external keystore.

Oracle Database uses the master encryption key from Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS) to encrypt or decrypt TDE table keys or tablespace encryption keys (data encryption keys) inside the database.

If you have not previously configured TDE with a wallet, then you must set the master encryption key in Oracle Key Vault or OCI KMS. If you have already configured TDE with a wallet, then you must migrate the database to Oracle Key Vault or OCI KMS.

Along with the current master encryption key, all TDE keystores (TDE wallet, Oracle Key Vault, and OCI KMS) maintain historical master encryption keys that are generated after every re-key operation that rekeys the master encryption key. These historical master keys help to restore Oracle Database backups that were taken previously using one of the master encryption keys. Only Oracle Key Vault allows you to upload all historical key from a TDE wallet before migrating the database to use Oracle Key Vault. After migration to Oracle Key Vault, the TDE wallet can be deleted, which satisfies security regulation that mandate that encryption keys cannot reside on the encrypting server.

7.6.4.2 Heartbeat Batch Size for External Kestores

You can control the size of the batch of heartbeats issued during each heartbeat period.

When a PDB is configured to use an external key manager, the `GEN0` background process must perform a heartbeat request on behalf of the PDB to the external key manager. This background process ensures that the external key manager is available and that the TDE master encryption key of the PDB is available from the external key manager and can be used for both encryption and decryption. The `GEN0` background process must complete this request within the heartbeat period (which defaults to three seconds).

When a very large number of PDBs (for example, 1000) are configured to use an external key manager, you can configure the `HEARTBEAT_BATCH_SIZE` database instance initialization parameter to batch heartbeats and thereby mitigate the possibility of the hang analyzer mistakenly flagging the `GEN0` process as being stalled when there was not enough time for it to perform a heartbeat for each PDB within the allotted heartbeat period.

By setting the heartbeat batch size, you can stagger the heartbeats across batches of PDBs to ensure that for each batch a heartbeat can be completed for each PDB within the batch during the heartbeat period, and also ensure that PDB master encryption keys can be reliably fetched from an Oracle Key Vault server and cached in the Oracle Key Vault persistent cache. The `HEARTBEAT_BATCH_SIZE` parameter configures the size of the batch of heartbeats sent per heartbeat period to the external key manager. The value must be between 2 and 100 and it defaults to 5. The default duration of the heartbeat period is three seconds.

For example, if 500 PDBs are configured and are using Oracle Key Vault, the usual time taken by `GEN0` to perform a heartbeat on behalf of a single PDB is less than half a second. In addition, assume that the `CDB$ROOT` has been configured to use an external key manager such as Oracle Key Vault (OKV). Therefore, it should generally be possible to send five heartbeats (one for the `CDB$ROOT` and four for a four-PDB batch) in a single batch within every three-second heartbeat period.

Even though the `HEARTBEAT_BATCH_SIZE` parameter configures the number of heartbeats sent in a batch, if the `CDB$ROOT` is configured to use an external key manager, then each heartbeat batch must include a heartbeat for the `CDB$ROOT`. The minimum value of the `HEARTBEAT_BATCH_SIZE` parameter is 2 and its maximum value is 100. When the `CDB$ROOT` is configured to use an external key manager, then each batch of heartbeats includes one heartbeat for the `CDB$ROOT`. This is why the minimum batch size is two: one must be reserved for the `CDB$ROOT`, because it might be configured to use an external key manager.

For example, suppose you set the `HEARTBEAT_BATCH_SIZE` parameter as follows:

```
ALTER SYSTEM SET HEARTBEAT_BATCH_SIZE=3 SCOPE=BOTH SID='*';
```

Each iteration corresponds to one `GEN0` three-second heartbeat period.

Example 1: Setting the Heartbeat for Containers That Are Configured to Use Oracle Key Vault

Suppose the container list is 1 2 3 4 5 6 7 8 9 10, with all containers configured to use Oracle Key Vault (OKV). The iterations are as follows:

- Iteration 1: batch consists of containers: 1 2 3
- Iteration 2: batch consists of containers: 1 4 5
- Iteration 3: batch consists of containers: 1 6 7
- Iteration 4: batch consists of containers: 1 8 9
- Iteration 5: batch consists of containers: 1 10
- Repeat this cycle.

Example 2: Setting the Heartbeat for Isolated PDBs with Different Keystores (Root Container in Oracle Key Vault)

In this example, the container list is 1 2 3 4 5 6 7 8 9 10, with only odd-numbered PDBs configured to use OKV, and the even-numbered PDBs configured to use a TDE wallet (FILE).

- Iteration 1: batch consists of containers: 1 3 5

- Iteration 2: batch consists of containers: 1 7 9
- Iteration 3: batch consists of containers: 1
- Repeat this cycle.

Example 3: Setting the Heartbeat for Isolated PDBs with Different Keystores (Root Container in TDE Wallet)

Assume that the container list is 1 2 3 4 5 6 7 8 9 10, with only even-numbered container numbers configured to use Oracle Key Vault, and the odd-numbered containers configured to use FILE. In the following example, there is no heartbeat for the CDB\$ROOT, because it is configured to use FILE.

- Iteration 1: batch consists of containers: 2 4 6
- Iteration 2: batch consists of containers: 8 10
- Repeat this cycle.

Related Topics

- [Setting the TDE Master Encryption Key in the United Mode External Keystore](#)
To set the TDE master encryption key in the keystore when the PDB is configured in united mode, use the ADMINISTER KEY MANAGEMENT statement with the SET KEY clause.
- [Setting the TDE Master Encryption Key in the Isolated Mode External Keystore](#)
You should complete this procedure if you have not previously configured an external keystore for Transparent Data Encryption.

7.6.4.3 Setting the TDE Master Encryption Key in the United Mode External Keystore

To set the TDE master encryption key in the keystore when the PDB is configured in united mode, use the ADMINISTER KEY MANAGEMENT statement with the SET KEY clause.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.

For example:

```
CONNECT c##sec_admin AS SYSKM
Enter password: password
```

2. Ensure that the database is open in READ WRITE mode.

You can set the master encryption key if OPEN_MODE is set to READ WRITE. To find the status, for a non-multitenant environment, query the OPEN_MODE column of the V\$DATABASE dynamic view. If you are in a multitenant environment, then run the show_pdb command.

3. Run the ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY statement to set the TDE master encryption key in the keystore.

```
ADMINISTER KEY MANAGEMENT SET KEY
[USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | "external_key_manager_password"];
```

In this specification:

- *tag* is the associated attributes and information that you define. Enclose this setting in single quotation marks (' ').
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL_STORE uses the keystore password stored in the external store to perform the keystore operation.
 - *external_key_manager_password* is for an external keystore manager, which can be Oracle Key Vault or OCI Vault - Key Management. Enclose this password in double quotation marks. For Oracle Key Vault, enter the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEYSTORE
IDENTIFIED BY "psmith:password";
```

keystore altered.

4. To configure the heartbeat batch size, set the HEARTBEAT_BATCH_SIZE initialization parameter.

The HEARTBEAT_BATCH_SIZE parameter configures the size of the "batch of heartbeats" sent per heartbeat period to the external key manager. Enter a value between 2 and 100. The default value is 5. The default duration of the heartbeat period is three seconds. By setting the heartbeat batch size, you can stagger the heartbeats across batches of PDBs to ensure that for each batch a heartbeat can be completed for each PDB within the batch during the heartbeat period, and also ensure that the TDE master encryption key of the PDB can be reliably fetched from an Oracle Key Vault server and cached in the Oracle Key Vault persistent cache. (See [Heartbeat Batch Size for External Keystores](#) for details about how HEARTBEAT_BATCH_SIZE works.)

```
ALTER SYSTEM SET HEARTBEAT_BATCH_SIZE=3 SCOPE=BOTH SID='*';
```

5. Confirm that the TDE master encryption key is set.

```
SELECT MASTERKEY_ACTIVATED FROM V$DATABASE_KEY_INFO;
```

The output should be YES.

7.6.4.4 Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS

To switch from a TDE wallet to centralized key management with Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS), after you upload all current and retired TDE master keys you must migrate the database from the TDE wallet to Oracle Key Vault or OCI KMS.

Tools such as Oracle Data Pump and Oracle Recovery Manager require access to the old TDE wallet to perform decryption and encryption operations on data exported or backed up using the TDE wallet. Along with the current master encryption key, Oracle keystores maintain historical master encryption keys that are generated after every re-key operation that rotates the master encryption key. These historical master encryption keys help to restore Oracle database backups that were taken previously using one of the historical master encryption keys.

Related Topics

- [Migrating from a TDE Wallet to Oracle Key Vault](#)
You can migrate between password-protected TDE wallets and external keystores in Oracle Key Vault.

7.6.5 Step 4: Encrypt Your Data in United Mode

Now that you have completed the configuration for an external keystore or for an Oracle Key Vault keystore, you can begin to encrypt data.

Related Topics

- [Encrypting Columns in Tables](#)
You can use Transparent Data Encryption to encrypt individual columns in database tables.
- [Encryption Conversions for Tablespaces and Databases](#)
You can perform encryption operations on both offline and online tablespaces and databases.

7.7 Administering Keystores and TDE Master Encryption Keys in United Mode

After you configure a keystore and master encryption key for use in united mode, you can perform tasks such as rekeying TDE master encryption keys.

- [Changing the Keystore Password in United Mode](#)
You can change the password of either a TDE wallet or an external keystore only in the CDB root.
- [Backing Up a Password-Protected TDE Wallet in United Mode](#)
The `BACKUP KEYSTORE` clause of the `ADMINISTER KEY MANAGEMENT` statement backs up a password-protected TDE wallet.
- [Closing Keystores in United Mode](#)
You can close both TDE wallet and external keystores in united mode, unless the system tablespace is encrypted.
- [Creating a User-Defined TDE Master Encryption Key in United Mode](#)
To create a user-defined TDE master encryption key, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET | CREATE [ENCRYPTION] KEY` clause.
- [Example: Creating a Master Encryption Key in All PDBs](#)
You can use the `ADMINISTER KEY MANAGEMENT CREATE KEY USING TAG` statement to create a TDE master encryption key in all PDBs.
- [Creating a TDE Master Encryption Key for Later Use in United Mode](#)
A keystore must be opened before you can create a TDE master encryption key for use later on in united mode.
- [Activating a TDE Master Encryption Key in United Mode](#)
To activate a TDE master encryption key in united mode, you must open the keystore and use `ADMINISTER KEY MANAGEMENT` with the `USE KEY` clause.
- [Rekeying the TDE Master Encryption Key in United Mode](#)
You can use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause to rekey a TDE master encryption key.

- [Finding the TDE Master Encryption Key That Is in Use in United Mode](#)
A TDE master encryption key that is in use is the key that was activated most recently for the database.
- [Creating a Custom Attribute Tag in United Mode](#)
To create a custom attribute tag in united mode, you must use the SET TAG clause of the ADMINISTER KEY MANAGEMENT statement.
- [Moving a TDE Master Encryption Key into a New Keystore in United Mode](#)
In united mode, you can move an existing TDE master encryption key into a new keystore from an existing software password keystore.
- [Automatically Removing Inactive TDE Master Encryption Keys in United Mode](#)
In united mode, the REMOVE_INACTIVE_STANDBY_TDE_MASTER_KEY initialization parameter can configure the automatic removal of inactive TDE master encryption keys.
- [Isolating a Pluggable Database Keystore](#)
Isolating a PDB keystore moves the master encryption key from the CDB root keystore into an isolated mode keystore in the a PDB.

7.7.1 Changing the Keystore Password in United Mode

You can change the password of either a TDE wallet or an external keystore only in the CDB root.

- [Changing the Password-Protected TDE Wallet Password in United Mode](#)
To change the password of a password-protected TDE wallet in united mode, you must use the ADMINISTER KEY MANAGEMENT statement in the CDB root.
- [Changing the Password of an External Keystore in United Mode](#)
To change the password of an external keystore, you must close the external keystore and then change the password from the external keystore management interface.

7.7.1.1 Changing the Password-Protected TDE Wallet Password in United Mode

To change the password of a password-protected TDE wallet in united mode, you must use the ADMINISTER KEY MANAGEMENT statement in the CDB root.

You can change this password at any time, as per the security policies, compliance guidelines, and other security requirements of your site. As part of the command to change the password, you will be forced to specify the WITH BACKUP clause, and thus forced to make a backup of the current TDE wallet. During the password change operation, Transparent Data Encryption operations such as encryption and decryption will continue to work normally. You can change this password at any time. You should change this password if you think it was compromised.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Use the following syntax to change the password for the keystore:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
[FORCE KEYSTORE]
IDENTIFIED BY
old_TDE_wallet_password SET new_TDE_wallet_password
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation if the TDE wallet is closed, if an auto-login TDE wallet is configured and is

currently open, or if a password-protected TDE wallet is configured and is currently closed.

- *old_TDE_wallet_password* is the current password that you want to change.
- *new_TDE_wallet_password* is the new password.
- You do not need to include the CONTAINER clause because the password can only be changed locally, in the CDB root.

The following example creates a backup of the keystore and then changes the password:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
IDENTIFIED BY old_password SET new_password
WITH BACKUP USING 'before_password_change';
```

keystore altered.

This example performs the same operation but uses the FORCE KEYSTORE clause in case the auto-login TDE wallet is in use or the password-protected TDE wallet is closed.

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
FORCE KEYSTORE
IDENTIFIED BY old_password SET new_password
WITH BACKUP USING 'before_password_change';
```

keystore altered.

Related Topics

- [Performing Operations That Require a Keystore Password](#)

Many ADMINISTER KEY MANAGEMENT operations require access to a keystore password, for both TDE wallets and external keystores.

7.7.1.2 Changing the Password of an External Keystore in United Mode

To change the password of an external keystore, you must close the external keystore and then change the password from the external keystore management interface.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Close the external keystore.

- Close the connection to the external key manager:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE
IDENTIFIED BY "external_key_manager_password"|EXTERNAL STORE
CONTAINER = ALL;
```

- If the keystore was auto-opened by the database, then close the connection to the external key manager as follows:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE
CONTAINER = ALL;
```

3. Change the Oracle Key Vault password.

```
WALLET_ROOT/okv/bin/okvutil changepwd -t wallet -l WALLET_ROOT/okv/ssl
```


4. Update the Oracle Key Vault password that enables an auto-open Oracle Key Vault keystore to use the new password that you set in step 3.

```
ADMINISTER KEY MANAGEMENT UPDATE SECRET 'new_Oracle_Key_Vault_password'
FOR CLIENT 'OKV-PASSWORD' TO [LOCAL] AUTO-LOGIN KEYSTORE 'WALLET_ROOT/tde';
```

5. Update the Oracle Key Vault password that enables IDENTIFIED BY EXTERNAL STORE (IBES) to the new password that you set in step 3.

```
ADMINISTER KEY MANAGEMENT UPDATE SECRET 'new_Oracle_Key_Vault_password'
FOR CLIENT 'OKV-PASSWORD' TO [LOCAL] AUTO-LOGIN KEYSTORE 'WALLET_ROOT/
tde_seps';
```

6. Open the connection to Oracle Key Vault.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
IDENTIFIED BY EXTERNAL STORE;
```

Related Topics

- [Performing Operations That Require a Keystore Password](#)

Many ADMINISTER KEY MANAGEMENT operations require access to a keystore password, for both TDE wallets and external keystores.

7.7.2 Backing Up a Password-Protected TDE Wallet in United Mode

The BACKUP KEYSTORE clause of the ADMINISTER KEY MANAGEMENT statement backs up a password-protected TDE wallet.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Back up the keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
[USING 'backup_identifier']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password]
[TO 'keystore_location'];
```

In this specification:

- USING *backup_identifier* is an optional string that you can provide to identify the backup. Enclose this identifier in single quotation marks (' '). This identifier is appended to the named keystore file (for example, ewallet_time-stamp_emp_key_backup.p12).
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- IDENTIFIED BY is required for the BACKUP KEYSTORE operation on a password-protected keystore because although the backup is simply a copy of the existing keystore, the status of the TDE master encryption key in the password-protected keystore must be set to BACKED UP and for this change the keystore password is required.
- *keystore_location* is the path at which the backup keystore is stored. This setting is restricted to the PDB when the PDB lockdown profile EXTERNAL_FILE_ACCESS setting is blocked in the PDB or when the PATH_PREFIX variable was not set when the PDB was

created. If you do not specify the *keystore_location*, then the backup is created in the same directory as the original keystore. Enclose this location in single quotation marks (' ').

- You do not need to include the CONTAINER clause because the keystore can only be backup up locally, in the CDB root.

The following example backs up a TDE wallet in the same location as the source keystore.

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
USING 'hr.emp_keystore'
FORCE KEYSTORE
IDENTIFIED BY
TDE_wallet_password ;
```

keystore altered.

In the following version, the password for the keystore is external, so the EXTERNAL STORE clause is used.

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
USING 'hr.emp_keystore'
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE;
```

After you run this statement, an *ewallet_identifier.p12* file (for example, *ewallet_timestamp_hr.emp_keystore.p12*) appears in the TDE wallet backup location.

Related Topics

- [Backing Up Password-Protected Software Keystores](#)
When you back up a password-protected software keystore, you can create a backup identifier string to describe the backup type.

7.7.3 Closing Keystores in United Mode

You can close both TDE wallet and external keystores in united mode, unless the system tablespace is encrypted.

- [Closing a Software Keystore in United Mode](#)
You can close password-protected keystores, auto-login keystores, and local auto-login software keystores in united mode.
- [Closing an External Keystore in United Mode](#)
To close an external keystore, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE CLOSE clause.

7.7.3.1 Closing a Software Keystore in United Mode

You can close password-protected keystores, auto-login keystores, and local auto-login software keystores in united mode.

In the case of an auto-login keystore, which opens automatically when it is accessed, you must first move it to a new location where it cannot be automatically opened, then you must manually close it. You must do this if you are changing your configuration from an auto-login keystore to a password-protected keystore: you change the configuration to stop using the auto-login keystore (by moving the auto-login keystore to another location where it cannot be automatically opened), and then closing the auto-login keystore.

1. Log in to the CDB root or the united mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.

2. Close the software keystore.

- For a password-protected software keystore, use the following syntax if you are in the CDB root:

```
ADMINISTER KEY MANAGEMENT SET | FORCE KEYSTORE CLOSE  
[IDENTIFIED BY [EXTERNAL STORE | software_keystore_password]]  
[CONTAINER = ALL | CURRENT];
```

Use the SET clause to close the keystore without force. If there is a dependent keystore that is open (for example, an isolated mode PDB keystore and you are trying to close the CDB root keystore), then an ORA-46692 cannot close wallet error appears. If this happens, then use the FORCE clause instead of SET to temporarily close the dependent keystore during the close operation. The STATUS column of the V\$ENCRYPTION_WALLET view shows if a keystore is open.

If you are in the united mode PDB, then either omit the CONTAINER clause or set it to CURRENT.

- For an auto-login or local auto-login software keystore, use this syntax if you are in the CDB root:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
[CONTAINER = ALL | CURRENT];
```

Closing a keystore disables all of the encryption and decryption operations. Any attempt to encrypt or decrypt data or access encrypted data results in an error.

Related Topics

- [About Closing Keystores](#)

After you open a keystore, it remains open until you shut down the database instance.

7.7.3.2 Closing an External Keystore in United Mode

To close an external keystore, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE CLOSE clause.

- Log in to the CDB root or the united mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
- Close the external keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY [EXTERNAL STORE | "external_key_manager_password"]  
[CONTAINER = ALL | CURRENT];
```

For example, for an HSM:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY "external_key_manager_password"  
CONTAINER = ALL;
```

If an ORA-46692 cannot close wallet error appears, then check if any isolated mode keystores are open. To find the status of a keystore, query the STATUS column of the V\$ENCRYPTION_WALLET view.

Closing a keystore disables all of the encryption and decryption operations. Any attempt to encrypt or decrypt data or access encrypted data results in an error.

Related Topics

- [About Closing Keystores](#)

After you open a keystore, it remains open until you shut down the database instance.

7.7.4 Creating a User-Defined TDE Master Encryption Key in United Mode

To create a user-defined TDE master encryption key, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET | CREATE [ENCRYPTION] KEY` clause.

1. Connect to the CDB root as a common user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Create the user-defined TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET | CREATE [ENCRYPTION] KEY
'[mkid:mk | mk]'
[USING ALGORITHM 'algorithm']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- `SET | CREATE` : Enter `SET` if you want to create the master and activate the TDE master encryption key now, or select `CREATE` if you want to create the key for later use, without activating it yet. For `SET KEY` and `USE KEY`, you must be connected to the database that is supposed to use that key (CDB root container, united or isolated PDB). For `CREATE KEY`, you can be connected any united PDB, or the CDB root container, or the specific isolated PDB that is supposed to use that key later.
- `mkid` and `mk`:
 - `mkid`, the TDE master encryption key ID, is a 16-byte hex-encoded value that you can specify or have Oracle Database generate.
 - `mk`, the TDE master encryption key, is a hex-encoded value that you can specify or have Oracle Database generate, either 32 bytes (for the for AES256, ARIA256, and GOST256 algorithms) or 16 bytes (for the SEED128 algorithm).

If you omit the `mkid:mk|mkid` clause but include the `mk` value, then Oracle Database generates the `mkid` for the `mk`.

If you omit the entire `mkid:mk|mkid` clause, then Oracle Database generates these values for you.

- `USING ALGORITHM`: Specify one of the following supported algorithms:
 - AES256
 - ARIA256
 - SEED128
 - GOST256

If you omit the algorithm, then the default, AES256, is used.

- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

The following example creates a TDE master encryption key without the key-ID is when a database administrator performs that operation:

First, generate the TDE master encryption key. For example, using OpenSSL:

```
$ openssl rand -hex 32
f24ce8cdae39742b5da7293da206fbd05a28b1e44c4781f801768cfdc3dbb6e2
```

Set (create and activate) this ARIA master key with the following ADMINISTER KEY MANAGEMENT statement:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
'f24ce8cdae39742b5da7293da206fbd05a28b1e44c4781f801768cfdc3dbb6e2'
USING ALGORITHM 'ARIA256'
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE
WITH BACKUP USING 'backup_identifler';
```

The next example, where TDE master encryption key and key-ID are generated outside of the database, shows how to support separation of duties. A key administrator inserts the key with a key_ID into the wallet by using the ADMINISTER KEY MANAGEMENT CREATE [ENCRYPTION] KEY statement. Then, the key administrator sends the key_ID to the database administrator, who then activates the key by using the ADMINISTER KEY MANAGEMENT USE KEY clause.

- a. Create the key-ID, converting characters to upper case. For example, using OpenSSL:

```
$ openssl rand 16 | xxd -u -p
D20765EB721AF44D054B30FE87F8E49A
```

- b. Create the TDE master encryption key. For example, using OpenSSL:

```
$ openssl rand -hex 32
5a089c8ea6ee21cba774f61e58d102665df4a979a34757836b3d066a3eb3db11
```

- c. Create (but do not activate) a default AES256 master encryption key by inserting both the random key-ID and the key itself, separated by a :, into the wallet.

```
ADMINISTER KEY MANAGEMENT CREATE ENCRYPTION KEY
'D20765EB721AF44D054B30FE87F8E49A:5a089c8ea6ee21cba774f61e58d102665df4a979a347578
36b3d066a3eb3db11'
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
WITH BACKUP USING 'backup_identifler' WITH BACKUP;
```

- d. The database administrator can retrieve the BASE64 key-ID from the database with the following SELECT statement:

```
SELECT ' ADMINISTER KEY MANAGEMENT USE KEY ''||KEY_ID||''
USING TAG ''||SYS_CONTEXT('USERENV', 'CON_NAME')||' '||TO_CHAR (SYS_EXTRACT_UTC
(SYSTIMESTAMP),
'YYYY-MM-DD HH24:MI:SS"Z"')||''
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE WITH BACKUP;
' AS "USE KEY COMMAND" FROM DUAL, V$ENCRYPTION_KEYS WHERE TAG IS NULL;
```

Related Topics

- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

7.7.5 Example: Creating a Master Encryption Key in All PDBs

You can use the ADMINISTER KEY MANAGEMENT CREATE KEY USING TAG statement to create a TDE master encryption key in all PDBs.

[Example 7-1](#) shows how to create a master encryption key in all of the PDBs in a multitenant environment. It uses the `FORCE KEYSTORE` clause in the event that the auto-login keystore in the CDB root is open. The password is stored externally, so the `EXTERNAL STORE` setting is used for the `IDENTIFIED BY` clause. After you execute this statement, a master encryption key is created in each PDB. You can find the identifiers for these keys as follows:

- Log in to the PDB and then query the `TAG` column of the `V$ENCRYPTION_KEYS` view.
- Log in to the CDB root and then query the `INST_ID` and `TAG` columns of the `GV$ENCRYPTION_KEYS` view.

You also can check the `CREATION_TIME` column of these views to find the most recently created key, which would be the key that you created from this statement. After you create the keys, you can individually activate the keys in each of the PDBs.

Example 7-1 Creating a Master Encryption Key in All of the PDBs

```
ADMINISTER KEY MANAGEMENT CREATE KEY USING TAG
'scope:all pdbs;description:Create Key for ALL PDBS'
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE
WITH BACKUP
CONTAINER = ALL;

keystore altered.
```

7.7.6 Creating a TDE Master Encryption Key for Later Use in United Mode

A keystore must be opened before you can create a TDE master encryption key for use later on in united mode.

1. Log in to the CDB root or the united mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Create the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE [ENCRYPTION] KEY
[FORCE KEYSTORE]
[USING TAG 'tag_name']
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']]
[CONTAINER = ALL | CURRENT];
```

In this specification:

- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `CONTAINER`: In the CDB root, set `CONTAINER` to either `ALL` or `CURRENT`. In a PDB, set it to `CURRENT`. In both cases, omitting `CONTAINER` defaults to `CURRENT`.

For example:

```
ADMINISTER KEY MANAGEMENT CREATE KEY
FORCE KEYSTORE
IDENTIFIED BY keystore_password
WITH BACKUP
CONTAINER = CURRENT;
```

3. If necessary, activate the TDE master encryption key.
 - a. Find the key ID.

```
SELECT KEY_ID FROM V$ENCRYPTION_KEYS;
```

```
KEY_ID
-----
AWShwVYC2U+Nv3RVphn/yAIAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

- b. Use this key ID to activate the key.

```
ADMINISTER KEY MANAGEMENT USE KEY
'AWShwVYC2U+Nv3RVphn/yAIAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
USING TAG 'quarter:second;description:Activate Key on standby'
IDENTIFIED BY password
WITH BACKUP;
```

Related Topics

- [Creating TDE Master Encryption Keys for Later Use](#)
You can create a TDE master encryption key that can be activated at a later date.

7.7.7 Activating a TDE Master Encryption Key in United Mode

To activate a TDE master encryption key in united mode, you must open the keystore and use `ADMINISTER KEY MANAGEMENT` with the `USE KEY` clause.

1. Connect to the CDB root as a common user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Query the `ORIGIN` and `KEY_ID` columns of the `V$ENCRYPTION_KEYS` view to find the key identifier.

For example:

```
SELECT ORIGIN, KEY_ID FROM V$ENCRYPTION_KEYS;

ORIGIN  KEY_ID
-----  -----
LOCAL   ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

3. Use this key identifier to activate the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT USE KEY 'key_identifier_from_V$ENCRYPTION_KEYS'
[USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `WITH BACKUP` backs up the wallet in the same location as original wallet, as identified by `WALLET_ROOT/tde`. To find the key locations for all of the database instances, query the `V$ENCRYPTION_WALLET` or `GV$ENCRYPTION_WALLET` view.

The `WITH BACKUP` clause is mandatory for all `ADMINISTER KEY MANAGEMENT` statements that modify the wallet. Optionally, include the `USING backup_identifier` clause to add a description of the backup. Enclose `backup_identifier` in single quotation marks (' ').

For example:

```
ADMINISTER KEY MANAGEMENT USE KEY
'ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
FORCE KEYSTORE
```



```
IDENTIFIED BY EXTERNAL STORE  
WITH BACKUP;
```

7.7.8 Rekeying the TDE Master Encryption Key in United Mode

You can use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause to rekey a TDE master encryption key.

1. Log in to the CDB root or the united mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. If you are rekeying the TDE master encryption key for a keystore that has auto login enabled, then ensure that both the auto login keystore, identified by the `.sso` file, and the encryption keystore, identified by the `.p12` file, are present.

You can find the location of these files by querying the `WRL_PARAMETER` column of the `V$ENCRYPTION_WALLET` view. To find the `WRL_PARAMETER` values for all of the database instances, query the `GV$ENCRYPTION_WALLET` view.

3. Rekey the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY  
[FORCE KEYSTORE]  
[USING TAG 'tag_name']  
IDENTIFIED BY [EXTERNAL STORE | keystore_password]  
[WITH BACKUP [USING 'backup_identifier']]  
[CONTAINER = ALL | CURRENT];
```

In this specification:

- `tag` is the associated attributes and information that you define. Enclose this setting in single quotation marks (' ').
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` can be one of the following settings:
 - `EXTERNAL STORE` uses the keystore password stored in the external store to perform the keystore operation.
 - `keystore_password` is the password that was created for this keystore.
- `CONTAINER`: In the CDB root, set `CONTAINER` to either `ALL` or `CURRENT`. In a PDB, set it to `CURRENT`. In both cases, omitting `CONTAINER` defaults to `CURRENT`.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY  
FORCE KEYSTORE  
IDENTIFIED BY keystore_password  
WITH BACKUP USING 'emp_key_backup'  
CONTAINER = CURRENT;
```

keystore altered.

Related Topics

- [About Rekeying the TDE Master Encryption Key](#)
Oracle Database uses a unified TDE Master Encryption Key for both TDE column encryption and TDE tablespace encryption.

7.7.9 Finding the TDE Master Encryption Key That Is in Use in United Mode

A TDE master encryption key that is in use is the key that was activated most recently for the database.

In united mode, the TDE master encryption key in use of the PDB is the one that was activated most recently for that PDB.

- To find the TDE master encryption key that is in use, query the `V$ENCRYPTION_KEYS` dynamic view.

For example:

```
SELECT KEY_ID
FROM V$ENCRYPTION_KEYS
WHERE ACTIVATION_TIME = (SELECT MAX(ACTIVATION_TIME)
                        FROM V$ENCRYPTION_KEYS
                        WHERE ACTIVATING_PDBID = SYS_CONTEXT('USERENV', 'CON_ID'));
```

7.7.10 Creating a Custom Attribute Tag in United Mode

To create a custom attribute tag in united mode, you must use the `SET TAG` clause of the `ADMINISTER KEY MANAGEMENT` statement.

1. Log in to the CDB root or the united mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. If necessary, query the `TAG` column of the `V$ENCRYPTION_KEY` dynamic view to find a listing of existing tags for the TDE master encryption keys.

When you create a new tag for a TDE master encryption key, it overwrites the existing tag for that TDE master encryption key.

3. Create the custom attribute tag by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET TAG 'tag'
FOR 'master_key_identifier'
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
WITH BACKUP [USING 'backup_identifier'];
```

In this specification

- *tag* is the associated attributes or information that you define. Enclose this information in single quotation marks (' ').
- *master_key_identifier* identifies the TDE master encryption key for which the *tag* is set. To find a list of TDE master encryption key identifiers, query the `KEY_ID` column of the `V$ENCRYPTION_KEYS` dynamic view.
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` can be one of the following settings:
 - `EXTERNAL STORE` uses the keystore password stored in the external store to perform the keystore operation.
 - *keystore_password* is the password that was created for this keystore.
- *backup_identifier* defines the tag values. Enclose this setting in single quotation marks (' ') and separate each value with a colon.

For example, to create a tag that uses two values, one to capture a specific session ID and the second to capture a specific terminal ID:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
USING TAG 'sessionid=3205062574:terminal=xcvt'
IDENTIFIED BY keystore_password
WITH BACKUP;
```

keystore altered.

Both the session ID (3205062574) and terminal ID (xcvt) can derive their values by using either the SYS_CONTEXT function with the USERENV namespace, or by using the USERENV function.

Related Topics

- [About Creating Custom Attribute Tags](#)
Attribute tags enable you to monitor specific activities users perform, such as accessing a particular terminal ID.

7.7.11 Moving a TDE Master Encryption Key into a New Keystore in United Mode

In united mode, you can move an existing TDE master encryption key into a new keystore from an existing software password keystore.

This feature enables you to delete unused keys. After you move the key to a new keystore, you then can delete the old keystore.

1. Log in to the CDB root or the united mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Query the KEY_ID column of the V\$ENCRYPTION_KEYS view to find the key identifier of the keystore to which you want to move the keys.

For example:

```
SELECT CREATION_TIME, KEY_ID FROM V$ENCRYPTION_KEYS;
```

```
CREATION TIME
```

```
-----
22-SEP-17 08.55.12.956170 PM +00:00
```

```
KEY_ID
```

```
-----
ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

3. Move the key into a new keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT
MOVE [ENCRYPTION] KEYS
TO NEW KEYSTORE 'keystore_location1'
IDENTIFIED BY keystore1_password
FROM [FORCE] KEYSTORE
IDENTIFIED BY keystore_password
[WITH IDENTIFIER IN
{ 'key_identifier' [, 'key_identifier' ]... | ( subquery ) } ]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *keystore_location1* is the path to the wallet directory that will store the new keystore .p12 file. By default, this directory is in `$ORACLE_BASE/admin/db_unique_name/wallet`.
- `FORCE` temporarily opens the keystore for this operation.
- *keystore_password* is the password for the keystore from which the key is moving.

For example:

```
ADMINISTER KEY MANAGEMENT MOVE KEYS
TO NEW KEYSTORE '$ORACLE_BASE/admin/orcl/wallet'
IDENTIFIED BY keystore_password
FROM FORCE KEYSTORE
IDENTIFIED BY keystore_password
WITH IDENTIFIER IN
(SELECT KEY_ID FROM V$ENCRYPTION_KEYS WHERE ROWNUM < 2)
WITH BACKUP;
```

4. To delete the old keystore, go to the wallet directory and then manually delete the .p12 file containing the keystore.

To find the location of the keystore, open the keystores, and then query the `WRL_PARAMETER` column of the `V$ENCRYPTION_WALLET` view.

Related Topics

- [Dangers of Deleting Keystores](#)
Oracle strongly recommends that you do not delete keystores.

7.7.12 Automatically Removing Inactive TDE Master Encryption Keys in United Mode

In united mode, the `REMOVE_INACTIVE_STANDBY_TDE_MASTER_KEY` initialization parameter can configure the automatic removal of inactive TDE master encryption keys.

1. Log in to the server where the CDB root or the united mode PDB of the Oracle standby database resides.
2. Locate the initialization parameter file for the database.
By default, the initialization parameter file is located in the `$ORACLE_HOME/dbs` directory.
3. Edit the initialization parameter file to include the `REMOVE_INACTIVE_STANDBY_TDE_MASTER_KEY` initialization parameter.

For example:

```
remove_inactive_standby_tde_master_key = true
```

Setting this parameter to `TRUE` enables the automatic removal of inactive TDE master encryption keys; setting it to `FALSE` disables the automatic removal.

7.7.13 Isolating a Pluggable Database Keystore

Isolating a PDB keystore moves the master encryption key from the CDB root keystore into an isolated mode keystore in the a PDB.

This process enables the keystore to be managed as a separate keystore in isolated mode. This way, an administrator who has been locally granted the `ADMINISTER KEY MANAGEMENT` privilege for the PDB can manage the keystore.

1. Connect to the CDB root as a common user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Move the keys from the keystore of the CDB root into the isolated mode keystore of the PDB by using the following syntax:

```
ADMINISTER KEY MANAGEMENT [FORCE] ISOLATE KEYSTORE  
IDENTIFIED BY isolated_keystore_password  
FROM ROOT KEYSTORE  
[FORCE KEYSTORE]  
IDENTIFIED BY  
[EXTERNAL STORE | united_keystore_password]  
[WITH BACKUP [USING backup_id]];
```

In this specification:

- FORCE is used when a clone of the PDB is using the master encryption key that is being isolated. The ADMINISTER KEY MANAGEMENT statement then copies (rather than moves) the keys from the wallet of the CDB root into the isolated mode PDB.
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation if an auto-login TDE wallet is open (and in use) or if the TDE wallet is closed.
- *united_keystore_password*: Knowledge of this password does not enable the user who performs the ISOLATE KEYSTORE operation privileges to perform ADMINISTER KEY MANAGEMENT UNITE KEYSTORE operations on the CDB root. This password is the same as the keystore password in the CDB root.

After the keystore of a CDB root has been united with that of a PDB, all of the previously active (historical) master encryption keys that were associated with the CDB are moved to the keystore of the PDB.

3. Confirm that the united mode PDB is now an isolated mode PDB.

```
SELECT KEYSTORE_MODE FROM V$ENCRYPTION_WALLET;
```

The output should be ISOLATED.

After the united mode PDB has been converted to an isolated mode PDB, you can change the password of the keystore.

7.8 Administering Transparent Data Encryption in United Mode

You can perform general administrative tasks with Transparent Data Encryption in united mode.

- [Moving PDBs from One CDB to Another in United Mode](#)
You can clone or relocate encrypted PDBs within the same container database, or across container databases.
- [Unplugging and Plugging a PDB with Encrypted Data in a CDB in United Mode](#)
In united mode, for a PDB that has encrypted data, you can plug it into a CDB. Conversely, you can unplug this PDB from the CDB.
- [Managing Cloned PDBs with Encrypted Data in United Mode](#)
In united mode, you can clone a PDB that has encrypted data in a CDB.
- [How Keystore Open and Close Operations Work in United Mode](#)
You should be aware of how keystore open and close operations work in united mode.

- [Finding the Keystore Status for All of the PDBs in United Mode](#)
You can create a convenience function that uses the V\$ENCRYPTION_WALLET view to find the status for keystores in all PDBs in a CDB.
- [Unplugging a PDB That Has Encrypted Data in United Mode Without ENCRPYT Clause](#)
In united mode, you can unplug a PDB with encrypted data and export data into an XML file or an archive file and then export TDE master encryption keys separately into a file.
- [Plugging a PDB That Has Encrypted Data into a CDB in United Mode Without DECRYPT Clause](#)
To plug a PDB that has encrypted data into a CDB, you first plug in the PDB, open the PDB, import the TDE master encryption keys, and then you create a master encryption key for the PDB.

7.8.1 Moving PDBs from One CDB to Another in United Mode

You can clone or relocate encrypted PDBs within the same container database, or across container databases.

If you are trying to move a non-CDB or a PDB in which the SYSTEM, SYSAUX, UNDO, or TEMP tablespace is encrypted, and using the manual export or import of keys, then you must first import the keys for the non-CDB or PDB in the target database's CDB\$ROOT before you create the PDB. Import of the keys are again required inside the PDB to associate the keys to the PDB.

- Clone or relocate the non-CDB or PDB using the following syntax:

```
CREATE|RELOCATE PLUGGABLE DATABASE database_name KEYSTORE  
IDENTIFIED BY EXTERNAL STORE|target_keystore_password [NO REKEY];
```

Related Topics

- *Oracle Multitenant Administrator's Guide*

7.8.2 Unplugging and Plugging a PDB with Encrypted Data in a CDB in United Mode

In united mode, for a PDB that has encrypted data, you can plug it into a CDB. Conversely, you can unplug this PDB from the CDB.

- [Unplugging a PDB That Has Encrypted Data in United Mode](#)
In united mode, you can unplug a PDB with encrypted data and export it into an XML file or an archive file.
- [Plugging a PDB That Has Encrypted Data into a CDB in United Mode](#)
To plug a PDB that has encrypted data into a CDB, you first plug in the PDB and then you create a master encryption key for the PDB.
- [Unplugging a PDB That Has Master Encryption Keys Stored in an External Keystore in United Mode](#)
You can unplug a PDB from one CDB that has been configured with an external keystore and then plug it into another CDB also configured with an external keystore.
- [Plugging a PDB That Has Master Encryption Keys Stored in an External Keystore in United Mode](#)
The ADMINISTER KEY MANAGEMENT statement can import a TDE master encryption key from an external keystore to a PDB that has been moved to another CDB.

7.8.2.1 Unplugging a PDB That Has Encrypted Data in United Mode

In united mode, you can unplug a PDB with encrypted data and export it into an XML file or an archive file.

You can check if a PDB has been unplugged by querying the STATUS column of the DBA_PDBS data dictionary view.

1. In the CDB root, query the V\$ENCRYPTION_WALLET dynamic view to ensure that the keystore is open.
2. Use the ENCRYPT USING *transport_secret* clause in the ALTER PLUGGABLE DATABASE statement when you unplug the PDB.

This process extracts the master encryption keys that belong to that PDB from the open wallet, and encrypts those keys with the *transport_secret* clause.

You must use this clause if the PDB has encrypted data. Otherwise, an ORA-46680: master keys of the container database must be exported error is returned.

- For example, to export the PDB data into an XML file:

```
ALTER PLUGGABLE DATABASE CDB1_PDB2
UNPLUG INTO '/tmp/cdb1_pdb2.xml'
ENCRYPT USING transport_secret;
```

- To export the PDB data into an archive file:

```
ALTER PLUGGABLE DATABASE CDB1_PDB2
UNPLUG INTO '/tmp/cdb1_pdb2.pdb'
ENCRYPT USING transport_secret;
```

Related Topics

- [Step 2: Open the TDE Wallet in a United Mode PDB](#)
To open a TDE wallet in united mode, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE OPEN clause.
- *Oracle Multitenant Administrator's Guide*
- *Oracle Database SQL Language Reference*

7.8.2.2 Plugging a PDB That Has Encrypted Data into a CDB in United Mode

To plug a PDB that has encrypted data into a CDB, you first plug in the PDB and then you create a master encryption key for the PDB.

After you plug the PDB into the target CDB, and you must create a master encryption key that is unique to this plugged-in PDB. This is because the plugged-in PDB initially uses the key that was extracted from the wallet of the source PDB. When you plug an unplugged PDB into another CDB, the key version is set to 0 because this operation invalidates the history of the previous keys. You can check the key version by querying the KEY_VERSION column of the V\$ENCRYPTED_TABLESPACES dynamic view. Similarly, if a control file is lost and recreated, then the previous history of the keys is reset to 0. You can check if a PDB has already been plugged in by querying the STATUS column of the DBA_PDBS data dictionary view.

1. From the CDB root, create the PDB by plugging the unplugged PDB into the CDB.

To perform this operation for united mode, include the DECRYPT USING *transport_secret* clause.

You must use this clause if the XML or archive file for the PDB has encrypted data. Otherwise, an ORA-46680: master keys of the container database must be exported error is returned.

- For example, if you had exported the PDB data into an XML file:

```
CREATE PLUGGABLE DATABASE CDB1_PDB2
USING '/tmp/cdb1_pdb2.xml'
KEystore IDENTIFIED BY EXTERNAL STORE|TDE_wallet_password
DECRYPT USING transport_secret;
```

- If you had exported the PDB into an archive file:

```
CREATE PLUGGABLE DATABASE CDB1_PDB2
USING '/tmp/cdb1_pdb2.pdb'
KEystore IDENTIFIED BY EXTERNAL STORE|TDE_wallet_password
DECRYPT USING transport_secret;
```

During the open operation of the PDB after the plug operation, Oracle Database determines if the PDB has encrypted data. If so, it opens the PDB in the RESTRICTED mode.

If you want to create the PDB by cloning another PDB or from a non-CDB, and if the source database has encrypted data or a TDE master encryption key that has been set, then you must provide the keystore password of the target keystore by including the `KEystore IDENTIFIED BY keystore_password` clause in the `CREATE PLUGGABLE DATABASE ... FROM SQL` statement. You must provide this password even if the target database is using an auto-login TDE wallet. You can find if the source database has encrypted data or a TDE master encryption key set in the TDE wallet by querying the `V$ENCRYPTION_KEYS` dynamic view

2. Open the PDB.

For example:

```
ALTER PLUGGABLE DATABASE CDB1_PDB2 OPEN;
```

3. Open the keystore in the CDB root by using one of the following methods:

- If the TDE wallet of the CDB is not open, open it for the container and all open PDBs by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEystore OPEN [FORCE KEystore]
IDENTIFIED BY EXTERNAL STORE|KEystore_PASSWORD CONTAINER = ALL;
```

- If the TDE wallet of the CDB is open, connect to the plugged-in PDB and then open the keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEystore OPEN [FORCE KEystore]
IDENTIFIED BY EXTERNAL STORE|KEystore_PASSWORD [CONTAINER = CURRENT];
```

4. In the plugged-in PDB, set the TDE master encryption key for the PDB by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEY
[FORCE KEystore]
IDENTIFIED BY EXTERNAL STORE|TDE_wallet_password
WITH BACKUP [USING 'backup_identifier'];
```

Related Topics

- [Step 2: Open the TDE Wallet in a United Mode PDB](#)
To open a TDE wallet in united mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEystore OPEN` clause.

- [Step 3: Set the TDE Master Encryption Key in the TDE Wallet in United Mode](#)
To set the TDE master encryption key in the TDE wallet when the PDB is configured in united mode, use the ADMINISTER KEY MANAGEMENT statement with the SET KEY clause.

7.8.2.3 Unplugging a PDB That Has Master Encryption Keys Stored in an External Keystore in United Mode

You can unplug a PDB from one CDB that has been configured with an external keystore and then plug it into another CDB also configured with an external keystore.

1. Unplug the PDB.

You can check if a PDB has already been unplugged by querying the STATUS column of the DBA_PDBS data dictionary view.

2. Move the master encryption keys of the unplugged PDB in the external keystore that was used at the source CDB to the external keystore that is in use at the destination CDB.

Refer to the documentation for the external keystore for information about moving master encryption keys between external keystores.

Related Topics

- *Oracle Multitenant Administrator's Guide*

7.8.2.4 Plugging a PDB That Has Master Encryption Keys Stored in an External Keystore in United Mode

The ADMINISTER KEY MANAGEMENT statement can import a TDE master encryption key from an external keystore to a PDB that has been moved to another CDB.

1. Plug the unplugged PDB into the destination CDB that has been configured with the external keystore.

You can check if a PDB has already been plugged in by querying the STATUS column of the DBA_PDBS data dictionary view.

After the plug-in operation, the PDB that has been plugged in will be in restricted mode.

2. Ensure that the master encryption keys from the external keystore that has been configured with the source CDB are available in the external keystore of the destination CDB.
3. Log in to the plugged PDB as a user who was granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
4. Open the master encryption key of the plugged PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY keystore_password;
```

5. Import the external keystore master encryption key into the PDB.

```
ADMINISTER KEY MANAGEMENT IMPORT ENCRYPTION KEYS  
WITH SECRET "OKV" FROM 'OKV'  
IDENTIFIED BY keystore_password;
```

6. Restart the PDB.

```
ALTER PLUGGABLE DATABASE PDB1 CLOSE;  
ALTER PLUGGABLE DATABASE PDB1 OPEN;
```

Related Topics

- *Oracle Multitenant Administrator's Guide*

7.8.3 Managing Cloned PDBs with Encrypted Data in United Mode

In united mode, you can clone a PDB that has encrypted data in a CDB.

- [About Managing Cloned PDBs That Have Encrypted Data in United Mode](#)
When you clone a PDB, you must make the master encryption key of the source PDB available to cloned PDB.
- [Cloning a PDB with Encrypted Data in a CDB in United Mode](#)
The CREATE PLUGGABLE DATABASE statement with the KEYSTORE IDENTIFIED BY clause can clone a PDB that has encrypted data.
- [Performing a Remote Clone of PDB with Encrypted Data Between Two CDBs in United Mode](#)
The CREATE PLUGGABLE DATABASE statement with the KEYSTORE IDENTIFIED BY clause can remotely clone a PDB that has encrypted data.
- [TDE Academy Videos: Remotely Cloning and Upgrading Encrypted PDBs](#)
The Oracle TDE Academy provides videos on how to remotely clone and upgrade encrypted pluggable databases (PDBs).
- [Relocating a PDB with Encrypted Data Across CDBs in United Mode](#)
The CREATE PLUGGABLE DATABASE statement with the KEYSTORE IDENTIFIED BY clause can relocate a PDB with encrypted data across CDBs.

7.8.3.1 About Managing Cloned PDBs That Have Encrypted Data in United Mode

When you clone a PDB, you must make the master encryption key of the source PDB available to cloned PDB.

This allows a cloned PDB to operate on the encrypted data. To perform the clone, you do not need to export and import the keys because Oracle Database transports the keys for you even if the cloned PDB is in a remote CDB. However, you will need to provide the keystore password of the CDB where you are creating the clone.

If the PDBs have encrypted data, then you can perform remote clone operations on PDBs between CDBs, and relocate PDBs across CDBs.

Cloning a PDB with encrypted data between two CDBs in united mode that use different keystore types may result in runtime errors such as ORA-28315: encryption key mismatched for file # %s. This issue can occur even if the encryption keys are properly exported from the source and imported into the target database. To avoid such problems, it is strongly recommended that both the source and target CDBs use the same keystore type. For example, either use a software keystore on both sides or the same hardware keystore with shared keys.

7.8.3.2 Cloning a PDB with Encrypted Data in a CDB in United Mode

The CREATE PLUGGABLE DATABASE statement with the KEYSTORE IDENTIFIED BY clause can clone a PDB that has encrypted data.

1. In the CDB root, query the STATUS column of the V\$ENCRYPTION_WALLET dynamic view to ensure that the keystore is open in the CDB root.
2. Log in to the PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT and the CREATE PLUGGABLE DATABASE privileges.

3. Use the `CREATE PLUGGABLE DATABASE` statement with the `KEystore IDENTIFIED BY` clause to clone the PDB.

For example:

```
CREATE PLUGGABLE DATABASE cdb1_pdb3
FROM cdb1_pdb1
FILE_NAME_CONVERT=('cdb1_pdb1', 'pdb3/cdb1_pdb3') KEystore
IDENTIFIED BY EXTERNAL STORE|keystore_password [NO REKEY];
```

Replace *keystore_password* with the password of the keystore of the CDB where the *cdb1_pdb3* clone is created.

By default, during a PDB clone or relocate operation, the data encryption keys are rekeyed, which implies a re-encryption of all encrypted tablespaces. This rekey operation can increase the time it takes to clone or relocate a large PDB. With the optional `NO REKEY` clause, the data encryption keys are not renewed, and encrypted tablespaces are not re-encrypted.

After you create the cloned PDB, encrypted data is still accessible by the clone using the master encryption key of the original PDB. After a PDB is cloned, there may be user data in the encrypted tablespaces. This encrypted data is still accessible because the master encryption key of the source PDB is copied over to the destination PDB. Because the clone is a copy of the source PDB but will eventually follow its own course and have its own data and security policies, you should rekey the master encryption key of the cloned PDB.

4. Rekey the master encryption key of the cloned PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEystore
IDENTIFIED BY keystore_password
WITH BACKUP USING 'emp_key_backup';
```

In this example, `FORCE KEystore` is included because the keystore must be open during the rekey operation.

Before you rekey the master encryption key of the cloned PDB, the clone can still use master encryption keys that belong to the original PDB. However, these master encryption keys do not appear in the cloned PDB `V$` dynamic views. Rekeying the master encryption key ensures that the cloned PDB uses its own unique keys, which will be viewable in the `V$` views.

Related Topics

- [Step 2: Open the TDE Wallet in a United Mode PDB](#)

To open a TDE wallet in united mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEystore OPEN` clause.

7.8.3.3 Performing a Remote Clone of PDB with Encrypted Data Between Two CDBs in United Mode

The `CREATE PLUGGABLE DATABASE` statement with the `KEystore IDENTIFIED BY` clause can remotely clone a PDB that has encrypted data.

1. In the CDB root, query the `STATUS` column of the `V$ENCRYPTION_WALLET` dynamic view to ensure that the keystore is open in the CDB root.

2. In this root container of the target database, create a database link that connects to the root container of the source CDB.

```
CREATE DATABASE LINK clone_link
CONNECT TO C##REMOTE_CLONE_USER
IDENTIFIED BY C##REMOTE_CLONE_USER_PASSWORD
USING '//source_ip_address:port/DB_NAME';
```

3. Use the CREATE PLUGGABLE DATABASE statement with the KEYSTORE IDENTIFIED BY clause to perform the clone of the PDB.

For example:

```
CREATE PLUGGABLE DATABASE cdb1_pdb3
FROM cdb_1_pdb1@clone_link
FILE_NAME_CONVERT=('cdb1_pdb1', 'pdb3/cdb1_pdb3') KEYSTORE
KEYSTORE IDENTIFIED BY EXTERNAL STORE|keystore_password;
```

Replace *keystore_password* with the password of the keystore of the CDB where the cdb1_pdb3 clone is created.

After you create the cloned PDB, encrypted data is still accessible by the clone using the master encryption key of the original PDB. After a PDB is cloned, there may be user data in the encrypted tablespaces. This encrypted data is still accessible because the master encryption key of the source PDB is copied over to the destination PDB. Because the clone is a copy of the source PDB but will eventually follow its own course and have its own data and security policies, you should rekey the master encryption key of the cloned PDB.

4. Connect as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
5. Rekey the master encryption key of the remotely cloned PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEYSTORE
IDENTIFIED BY keystore_password
WITH BACKUP USING 'emp_key_backup';
```

In this example, FORCE KEYSTORE is included because the keystore must be open during the rekey operation.

Before you rekey the master encryption key of the cloned PDB, the clone can still use master encryption keys that belong to the original PDB. However, these master encryption keys do not appear in the cloned PDB V\$ dynamic views. Rekeying the master encryption key ensures that the cloned PDB uses its own unique keys, which will be viewable in the V\$ views.

Related Topics

- [Step 2: Open the TDE Wallet in a United Mode PDB](#)
To open a TDE wallet in united mode, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE OPEN clause.
- *Oracle Multitenant Administrator's Guide*
- *Oracle Multitenant Administrator's Guide*
- *Oracle Database SQL Language Reference*

7.8.3.4 TDE Academy Videos: Remotely Cloning and Upgrading Encrypted PDBs

The Oracle TDE Academy provides videos on how to remotely clone and upgrade encrypted pluggable databases (PDBs).

- [TDE Academy #01: Remote clone and upgrade encrypted 18c PDBs to 19c](#)
- [TDE Academy #02: Remote clone and upgrade encrypted 12.2.0.1 PDBs to 19c](#)
- [TDE Academy #03: Remote clone and upgrade encrypted 12.1.0.2 PDBs to 19c](#)

7.8.3.5 Relocating a PDB with Encrypted Data Across CDBs in United Mode

The `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause can relocate a PDB with encrypted data across CDBs.

1. In the CDB root, query the `STATUS` column of the `V$ENCRYPTION_WALLET` dynamic view to ensure that the keystore is open in the CDB root.
2. Create a database link for the PDB that you want to clone.

Use the `CREATE DATABASE LINK SQL` statement to create the database link. You must create the database link by following the database link prerequisites that are required for cloning a remote PDB or a non-CDB.
3. Use the `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause to relocate the PDB.

For example:

```
CREATE PLUGGABLE DATABASE cdb1_pdb3
FROM cdb1_pdb1@clone_link RELOCATE [AVAILABILITY MAX]
[FILE_NAME_CONVERT=('cdb1_pdb1', 'pdb3/cdb1_pdb3')]
KEYSTORE IDENTIFIED BY EXTERNAL STORE|keystore_password;
```

Replace `keystore_password` with the password of the keystore of the CDB where the `cdb1_pdb3` clone is created.

After you have relocated the PDB, the encrypted data is still accessible because the master encryption key of the source PDB is copied over to the destination PDB; however, these master encryption keys do not appear in the cloned PDB `V$` dynamic views.

4. Connect to the PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
5. Rekey the master encryption key of the relocated PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
[FORCE KEYSTORE]
IDENTIFIED BY EXTERNAL STORE|keystore_password
WITH BACKUP [USING 'emp_key_backup'];
```

Related Topics

- [Step 2: Open the TDE Wallet in a United Mode PDB](#)
To open a TDE wallet in united mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.
- *Oracle Multitenant Administrator's Guide*

- *Oracle Multitenant Administrator's Guide*
- *Oracle Database SQL Language Reference*

7.8.4 How Keystore Open and Close Operations Work in United Mode

You should be aware of how keystore open and close operations work in united mode.

For each PDB in united mode, you must explicitly open the password-protected TDE wallet or external keystore in the PDB to enable the Transparent Data Encryption operations to proceed. (Auto-login and local auto-login TDE wallets open automatically.) Closing a keystore on a PDB blocks all of the Transparent Data Encryption operations on that PDB.

The open and close keystore operations in a PDB depend on the open and close status of the keystore in the CDB root.

Note the following:

- You can create a separate keystore password for each PDB in united mode.
- Before you can manually open a password-protected TDE wallet or an external keystore in an individual PDB, you must open the wallet or keystore in the CDB root.
- If an auto-login TDE wallet is in use, or if the keystore is closed, then include the `FORCE KEYSTORE` clause in the `ADMINISTER KEY MANAGEMENT` statement when you open the wallet.
- If the keystore is a password-protected TDE wallet that uses an external store for passwords, then replace the password in the `IDENTIFIED BY` clause with `EXTERNAL STORE`.
- Before you can set a TDE master encryption key in an individual PDB, you must set the key in the CDB root. Oracle highly recommends that you include the `USING TAG` clause when you set keys in PDBs. For example:

```
SELECT ' ADMINISTER KEY MANAGEMENT SET KEY
USING TAG '''||SYS_CONTEXT('USERENV', 'CON_NAME')||' '||TO_CHAR (SYSDATE,
'YYYY-MM-DD HH24:MI:SS')||'''
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE
WITH BACKUP CONTAINER = CURRENT;' AS "SET KEY COMMAND" FROM DUAL;
```

Including the `USING TAG` clause enables you to quickly and easily identify the keys that belong to a certain PDB, and when they were created.

- Auto-login and local auto-login TDE wallets open automatically. You do not need to manually open these from the CDB root first, or from the PDB.
- If you close the keystore in the CDB root, then the keystores in the dependent PDBs also close. A keystore close operation in the root is the equivalent of performing a keystore close operation with the `CONTAINER` clause set to `ALL`.
- If you perform an `ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN` statement in the CDB root and set the `CONTAINER` clause to `ALL`, then the keystore will only be opened in each open PDB that is configured in united mode. Keystores for any PDBs that are configured in isolated mode are not opened.

7.8.5 Finding the Keystore Status for All of the PDBs in United Mode

You can create a convenience function that uses the V\$ENCRYPTION_WALLET view to find the status for keystores in all PDBs in a CDB.

The V\$ENCRYPTION_WALLET view displays the status of the keystore in a PDB, whether it is open, closed, uses a software or an external keystore, and so on.

- To create a function that uses the V\$ENCRYPTION_WALLET view to find the keystore status, use the CREATE PROCEDURE PL/SQL statement.

[Example 7-2](#) shows how to create this function.

Example 7-2 Function to Find the Keystore Status of All of the PDBs in a CDB

```
CREATE OR REPLACE PROCEDURE all_pdb_v$encryption_wallet
IS
    err_occ          BOOLEAN;
    curr_pdb         VARCHAR2(30);
    pdb_name         VARCHAR2(30);
    wrl_type         VARCHAR2(20);
    status           VARCHAR2(30);
    wallet_type      VARCHAR2(20);
    wallet_order     VARCHAR2(12);
    fully_backed_up  VARCHAR2(15);
    wrl_parameter    VARCHAR2(4000);
    cursor sel_pdb IS SELECT NAME FROM V$CONTAINERS
                      WHERE NAME <> 'PDB$SEED' order by con_id desc;
BEGIN
    -- Store the original PDB name
    SELECT sys_context('userenv', 'con_name') INTO curr_pdb FROM DUAL;
    IF curr_pdb <> 'CDB$ROOT' THEN
        dbms_output.put_line('Operation valid in ROOT only');
    END IF;

    err_occ := FALSE;
    dbms_output.put_line('---');
    dbms_output.put_line('PDB_NAME                                WRL_TYPE STATUS');
    dbms_output.put_line('-----');
    dbms_output.put_line('WALLET_TYPE          WALLET_ORDER FULLY_BACKED_UP');
    dbms_output.put_line('-----');
    dbms_output.put_line('WRL_PARAMETER');
    dbms_output.put_line('-----');
    FOR pdbinfo IN sel_pdb LOOP

        pdb_name := DBMS_ASSERT.ENQUOTE_NAME(pdbinfo.name, FALSE);
        EXECUTE IMMEDIATE 'ALTER SESSION SET CONTAINER = ' || pdb_name;

        BEGIN
            pdb_name := rpad(substr(pdb_name,1,30), 30, ' ');
            EXECUTE IMMEDIATE 'SELECT wrl_type from V$ENCRYPTION_WALLET' into wrl_type;
            wrl_type := rpad(substr(wrl_type,1,8), 8, ' ');
            EXECUTE IMMEDIATE 'SELECT status from V$ENCRYPTION_WALLET' into status;
            status := rpad(substr(status,1,30), 30, ' ');
            EXECUTE IMMEDIATE 'SELECT wallet_type from V$ENCRYPTION_WALLET' into wallet_type;
            wallet_type := rpad(substr(wallet_type,1,20), 20, ' ');
            EXECUTE IMMEDIATE 'SELECT wallet_order from V$ENCRYPTION_WALLET' into wallet_order;
            wallet_order := rpad(substr(wallet_order,1,9), 12, ' ');
            EXECUTE IMMEDIATE 'SELECT fully_backed_up from V$ENCRYPTION_WALLET' into fully_backed_up;
            fully_backed_up := rpad(substr(fully_backed_up,1,9), 15, ' ');
```

```

EXECUTE IMMEDIATE 'SELECT wrl_parameter from V$ENCRYPTION_WALLET' into wrl_parameter;
wrl_parameter := rpad(substr(wrl_parameter,1,79), 79, ' ');
dbms_output.put_line(pdb_name || ' ' || wrl_type || ' ' || status);
dbms_output.put_line(wallet_type || ' ' || wallet_order || ' ' || fully_backed_up);
dbms_output.put_line(wrl_parameter);

EXCEPTION
  WHEN OTHERS THEN
    err_occ := TRUE;
END;
END LOOP;

IF err_occ = TRUE THEN
  dbms_output.put_line('One or more PDB resulted in an error');
END IF;
END;
/
set serveroutput on
exec all_pdb_v$encryption_wallet;

```

7.8.6 Unplugging a PDB That Has Encrypted Data in United Mode Without ENCRPYT Clause

In united mode, you can unplug a PDB with encrypted data and export data into an XML file or an archive file and then export TDE master encryption keys separately into a file.

1. Export the keys.
If the source is multitenant then export the keys from the PDB which needs to be unplugged. For example:

```

alter session set container =<cdb1_pdb1>;
administer key management export encryption keys with secret
"transport_secret" to '/tmp/TDE/pdb_01.p12' identified by
<TDE_wallet_password>;

```

If the source is NON-CDB then export the keys from NON-CDB. For example,

```

administer key management export encryption keys with secret
"transport_secret" to '/tmp/TDE/pdb_01.p12' identified by
<TDE_wallet_password>;

```

2. After export unplug the PDB from source.
 - a. Close the PDB which needs to be unplugged:

```

alter session set container=cdb$root;
alter pluggable database <CDB1_PDB1> close;

```

- b. Unplug the PDB:

```

alter pluggable database <CDB1_PDB1> '/tmp/TDE/cdb1_pdb1.xml';

```

- c. If the source is NON-CDB, generate an XML file named `ncdb.xml` in the `/tmp` directory, run the following procedure:

```
BEGIN
DBMS_PDB.DESCRIBE(
    pdb_descr_file => '/tmp/ncdb.xml'
);
END;
/
```

7.8.7 Plugging a PDB That Has Encrypted Data into a CDB in United Mode Without `DECRYPT` Clause

To plug a PDB that has encrypted data into a CDB, you first plug in the PDB, open the PDB, import the TDE master encryption keys, and then you create a master encryption key for the PDB

1. In the destination CDB, import the keys. This step can be skipped if there are no system and sysaux encrypted data in source.

For example,

```
administer key management import encryption keys with secret
"transport_secret" from '/tmp/TDE/pdb_01.p12' identified by
<TDE_wallet_password> with backup;
```

2. Create the PDB using xml created in source.

For example,

```
create pluggable database <CDB1_PDB4_Plug> using '/tmp/TDE/cdb1_pdb1.xml'
file_name_convert=('cdb1_pdb1','cdb1_pdb4_plug') keystore identified by
<TDE_wallet_password>;
```

3. Open the TDE keystore.

For example,

```
alter session set container =<CDB1_PDB4_Plug>;
administer key management set Keystore open identified by
<TDE_wallet_password>;
```

4. Open the new created PDB.

For example,

```
alter session set container=cdb$root;
alter pluggable database <CDB1_PDB4_Plug> open;
```

5. Import the keys.

For example,

```
administer key management import encryption keys with secret
"transport_secret" from '/tmp/TDE/pdb_01.p12' identified by
<TDE_wallet_password> with backup;
```

8

Managing Keystores and TDE Master Encryption Keys in Isolated Mode

Isolated mode enables you to create a keystore for each pluggable database (PDB).

- [About Configuring Isolated Mode](#)
In isolated mode, where a pluggable database (PDB) has its own keystore and keystore password, you manage the keystore and its TDE master encryption keys from the PDB only.
- [Operations That Are Allowed in Isolated Mode](#)
You can perform many ADMINISTER KEY MANAGEMENT operations in isolated mode.
- [Operations That Are Not Allowed in an Isolated Mode PDB](#)
There are several ADMINISTER KEY MANAGEMENT operations that you cannot perform in an isolated mode PDB.
- [Configuring the Keystore Location and Type for Isolated Mode](#)
For isolated mode, you can configure the keystore location and type by using only parameters or a combination of parameters and the ALTER SYSTEM statement.
- [Configuring a TDE Wallet and TDE Master Encryption Key in Isolated Mode](#)
In isolated mode, the TDE wallet is associated with a PDB.
- [Configuring a Container Database with Isolated Mode PDBs for Oracle Key Vault](#)
Isolated PDBs have their own virtual wallet, with their own password, in Oracle Key Vault.
- [Administering Keystores and TDE Master Encryption Keys in Isolated Mode](#)
After you create a keystore and a TDE master encryption key in isolated mode, you can perform administration tasks such as rekeying or tagging encryption keys.
- [Administering Transparent Data Encryption in Isolated Mode](#)
You can perform a number of general administrative tasks with Transparent Data Encryption in isolated mode.

8.1 About Configuring Isolated Mode

In isolated mode, where a pluggable database (PDB) has its own keystore and keystore password, you manage the keystore and its TDE master encryption keys from the PDB only.

Isolated PDBs have been made available for all on-premises deployments starting with Oracle Database release 19.11 with patch [32235513](#). (No patch is necessary for release 19.14 and later.)

Similar to united mode, you must first configure a PDB to use isolated mode by setting the WALLET_ROOT and TDE_CONFIGURATION parameters. After you set these parameters, you can create and manage the keystore from the PDB. In this way, you can have the following scenario:

- United mode PDBs inherit the TDE configuration from the root container. For example, the keystore that you create in the CDB root will be used by the root's associated united mode PDBs.

- The PDBs that are configured in isolated mode are allowed to independently create and manage their own keystore. An isolated mode PDB can have its own keystore, independent of the keystore of the CDB root.

This scenario is useful in cases where you have many PDBs that must use one type of keystore, but you have a few PDBs that must use a different type. By different types of keystores, this refers to either TDE wallet or to one of the external keystores that Oracle supports (for example, Oracle Key Vault or Cloud Key Management Service). You cannot have a mixture of different external keystore types in one CDB environment because the Oracle server can load only one PKCS#11 vendor library. If necessary, you can configure these PDBs in isolated mode so that each PDB can use its own keystore.

① Note

Oracle Cloud Infrastructure (OCI) cloud tooling does not support isolated PDBs. This non-support applies to Oracle Base Database Service (BaseDB), ExaDB-D, ExaDB-D@Azure, ExaDB-C@C, ADB-C@C, even when Oracle Key Vault provides key management for those database deployments.

8.2 Operations That Are Allowed in Isolated Mode

You can perform many ADMINISTER KEY MANAGEMENT operations in isolated mode.

These operations include creating, backing up, opening keystores; changing keystore passwords, merging keystores, closing keystores; creating, activating, tagging, moving, exporting, importing, and migrating encryption keys; and adding, updating, and deleting client secrets.

[Table 8-1](#) describes the ADMINISTER KEY MANAGEMENT operations that you can perform in an isolated mode PDB.

Table 8-1 ADMINISTER KEY MANAGEMENT Isolated Mode Operations

Operation	Syntax	Isolated Mode Notes
Creating a keystore	ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY <i>isolated_PDB_keystore_password</i> ;	You can create password-protected, local auto-login, and auto-login keystores in an isolated mode PDB.
Creating an auto-login keystore	ADMINISTER KEY MANAGEMENT CREATE [LOCAL] AUTO_LOGIN KEYSTORE FROM KEYSTORE IDENTIFIED BY <i>isolated_PDB_keystore_password</i> ;	The isolated PDB knows the keystore location of both password protected and (local) auto-open wallet from <i>WALLET_ROOT/pdb_guid/tde</i> .

Table 8-1 (Cont.) ADMINISTER KEY MANAGEMENT Isolated Mode Operations

Operation	Syntax	Isolated Mode Notes
Opening a keystore	ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN [FORCE KEYSTORE] IDENTIFIED BY EXTERNAL STORE <i>isolated_PDB_keystore_password</i> ; ;	The EXTERNAL_STORE clause retrieves the isolated mode PDB keystore password from a wallet in the <i>tde_seps</i> directory in the <i>WALLET_ROOT/PDB-GUID/</i> location.
Changing a keystore password	ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD IDENTIFIED BY <i>old_isolated_PDB_keystore_password</i> SET <i>new_isolated_PDB_keystore_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '];	-
Backing up a TDE wallet	ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE [USING ' <i>backup_identifier</i> '] IDENTIFIED BY EXTERNAL STORE <i>TDE_wallet_password</i> [TO ' <i>TDE_wallet_backup_location</i> '] ;	-
Merging the contents of one keystore into an existing keystore	ADMINISTER KEY MANAGEMENT MERGE KEYSTORE ' <i>keystore1_location</i> ' [IDENTIFIED BY <i>TDE_wallet1_password</i>] INTO EXISTING KEYSTORE ' <i>keystore2_location</i> ' IDENTIFIED BY <i>TDE_wallet2_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '];	-
Merging the contents of two keystores to create a third keystore	ADMINISTER KEY MANAGEMENT MERGE KEYSTORE ' <i>keystore1_location</i> ' [IDENTIFIED BY <i>TDE_wallet1_password</i>] AND KEYSTORE ' <i>keystore2_password</i> ' [IDENTIFIED BY <i>TDE_wallet2_password</i>] INTO NEW KEYSTORE ' <i>keystore3_location</i> ' IDENTIFIED BY <i>TDE_wallet3_password</i> ;	-

Table 8-1 (Cont.) ADMINISTER KEY MANAGEMENT Isolated Mode Operations

Operation	Syntax	Isolated Mode Notes
Closing a keystore	ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE [IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>]];	-
Closing the keystore of the CDB root when a PDB in isolated mode has its keystore open	ADMINISTER KEY MANAGEMENT FORCE KEYSTORE CLOSE [IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>]];	The FORCE clause allows the keystore to be closed in the CDB root even when a PDB in isolated mode still has its keystore open
Creating and activating a new TDE master encryption key (rekeying)	ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY [USING TAG ' <i>tag_name</i> '] [FORCE KEYSTORE] IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>]WITH BACKUP [USING ' <i>backup_identifier</i> '];	-
Creating a user-defined TDE master encryption key for either now (SET) or later on (CREATE)	ADMINISTER KEY MANAGEMENT SET CREATE [ENCRYPTION] KEY ' <i>mkid:mk</i> <i>mk</i> ' [USING ALGORITHM ' <i>algorithm</i> '] [FORCE KEYSTORE] [USING TAG ' <i>tag_name</i> '] IDENTIFIED BY EXTERNAL STORE <i>TDE_wallet_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '] [CONTAINER = CURRENT];	-
Activating an existing TDE master encryption key	ADMINISTER KEY MANAGEMENT USE [ENCRYPTION] KEY ' <i>key_id</i> ' [USING TAG ' <i>tag</i> '] IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>] WITH BACKUP [USING ' <i>backup_identifier</i> '];	-
Tagging a TDE master encryption key	ADMINISTER KEY MANAGEMENT SET TAG ' <i>tag</i> ' FOR ' <i>key_id</i> ' IDENTIFIED BY [EXTERNAL STORE <i>keystore_password</i>] WITH BACKUP [USING ' <i>backup_identifier</i> '];	-

Table 8-1 (Cont.) ADMINISTER KEY MANAGEMENT Isolated Mode Operations

Operation	Syntax	Isolated Mode Notes
Exporting a TDE master encryption key	<pre>ADMINISTER KEY MANAGEMENT EXPORT [ENCRYPTION] KEYS WITH SECRET <i>secret</i> TO '<i>filename</i>' IDENTIFIED BY <i>TDE_wallet_password</i> [WITH IDENTIFIER IN { '<i>key_id</i>' [, '<i>key_id</i>']... (<i>subquery</i>) }];</pre>	-
Importing a TDE master encryption key	<pre>ADMINISTER KEY MANAGEMENT IMPORT [ENCRYPTION] KEYS WITH SECRET <i>secret</i> FROM '<i>filename</i>' IDENTIFIED BY <i>TDE_wallet_password</i> WITH BACKUP [USING '<i>backup_identifier</i>'];</pre>	-
Migrating an encrypted database from a TDE wallet to Oracle Key Vault	<pre>ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY IDENTIFIED BY <i>Oracle_Key_Vault_password</i> [FORCE KEYSTORE] MIGRATE USING <i>TDE_wallet_password</i>;</pre>	-
Reverse-migrating an encrypted database from an external keystore to a TDE wallet	<pre>ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY IDENTIFIED BY <i>TDE_wallet_password</i> REVERSE MIGRATE USING <i>Oracle_Key_Vault_password</i>;</pre>	-
Adding a client secret	<pre>ADMINISTER KEY MANAGEMENT ADD SECRET '<i>secret</i>' FOR CLIENT '<i>client_identifier</i>' [USING TAG '<i>tag_name</i>'] IDENTIFIED BY [EXTERNAL STORE <i>isolated_PDB_keystore_password</i>] WITH BACKUP [USING '<i>backup_identifier</i>'];</pre>	-

Table 8-1 (Cont.) ADMINISTER KEY MANAGEMENT Isolated Mode Operations

Operation	Syntax	Isolated Mode Notes
Updating a client secret	<pre>ADMINISTER KEY MANAGEMENT UPDATE SECRET 'secret' FOR CLIENT 'client_identifier' [USING TAG 'tag_name'] IDENTIFIED BY [EXTERNAL STORE keystore_password] WITH BACKUP [USING 'backup_identifier'];</pre>	-
Deleting a client secret	<pre>ADMINISTER KEY MANAGEMENT DELETE SECRET FOR CLIENT 'client_identifier' IDENTIFIED BY [EXTERNAL STORE keystore_password] WITH BACKUP [USING 'backup_identifier'];</pre>	-
Isolating a PDB	<pre>ADMINISTER KEY MANAGEMENT ISOLATE KEYSTORE IDENTIFIED BY isolated_keystore_password FROM ROOT KEYSTORE [FORCE KEYSTORE] IDENTIFIED BY [EXTERNAL STORE united_keystore_password] WITH BACKUP [USING backup_id];</pre>	This operation performs two actions. First, it changes the TDE_CONFIGURATION of the PDB so that it is in isolated mode. Second, it moves the PDB's current and previously active TDE master encryption keys from the root keystore to a newly-created isolated keystore of the PDB, having its own isolated keystore password, where the PDB will be able to manage its own keys.

8.3 Operations That Are Not Allowed in an Isolated Mode PDB

There are several ADMINISTER KEY MANAGEMENT operations that you cannot perform in an isolated mode PDB.

These operations include the following:

- Using the CONTAINER = ALL clause to create a new TDE master encryption key for later use in each pluggable database (PDB)
- Moving encryption keys from the keystore of the CDB root into a keystore of a PDB that is configured in isolated mode

8.4 Configuring the Keystore Location and Type for Isolated Mode

For isolated mode, you can configure the keystore location and type by using only parameters or a combination of parameters and the ALTER SYSTEM statement.

 Caution

It is not recommended to configure PDB\$SEED in ISOLATED mode, as it may impact subsequent PDB creation and database upgrade operations.

- [Configuring the Keystore Location and Keystore Type for an Isolated Mode CDB](#)
You can configure isolated mode by setting WALLET_ROOT in the initialization parameter file in the CDB root and TDE_CONFIGURATION in the PDB you want to isolate.
- [Example: Restoring an Older Version of a Control File](#)
You can set TDE_CONFIGURATION if you have an older version of a control file that must be restored and only a few PDBs were configured in isolated mode.
- [Example: Addressing the Problem of a Lost Control File](#)
You can address the problem of a lost control file by using the ALTER SYSTEM statement.
- [Example: Configuring Isolated Mode in an Oracle Real Application Clusters Environment](#)
You can use ALTER SYSTEM to configure isolated mode in an Oracle Real Application Clusters (Oracle RAC) environment.
- [Resolving the Transition From an Isolated Mode PDB to a United Mode PDB in a Data Guard Environment](#)

8.4.1 Configuring the Keystore Location and Keystore Type for an Isolated Mode CDB

You can configure isolated mode by setting WALLET_ROOT in the initialization parameter file in the CDB root and TDE_CONFIGURATION in the PDB you want to isolate.

1. In the root, ensure that the WALLET_ROOT parameter is set correctly.

For example:

```
SHOW PARAMETER WALLET_ROOT
```

2. Connect to the isolated mode PDB as a user who has been granted the ALTER SYSTEM privilege.

For example:

```
CONNECT sec_admin@pdb_name
```

3. As a user who has the ADMINISTER KEY MANAGEMENT or SYSKM privilege, run the following statement in the PDB:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY  
"iso_PDB_TDE_wallet_password";
```

This statement does the following:

- Sets the TDE_CONFIGURATION in the isolated PDB to FILE.
 - Creates the directories *pdb_guid*/tde under WALLET_ROOT.
 - Creates a new TDE wallet with its own password (which might be not known to the CDB administrator).
4. Check the configuration.
 - To check the TDE_CONFIGURATION parameter setting:

```
SHOW PARAMETER TDE_CONFIGURATION
```

The output should reflect the keystore configuration that you set for the current PDB. If it shows a different keystore configuration (for example, FILE if you had set it to OKV), then the setting may be showing the keystore configuration that was set for the CDB root, in united mode.

- To check the keystore mode:

```
SELECT KEYSTORE_MODE FROM V$ENCRYPTION_WALLET;
```

The output should be ISOLATED.

After you configure isolated mode, the CDB root keystore that was available to the PDB when it was in united mode is no longer available to this PDB. At this stage, the PDB is configured to use its own keystore. If the KEYSTORE_CONFIGURATION parameter was FILE (meaning that the PDB is configured to use a TDE wallet), then the keystore location configured for the PDB is WALLET_ROOT/PDB-GUID/tde. If a keystore exists at that location and contains a TDE master encryption key, then that key is only available to this PDB, not to any other PDB. If no keystore exists at that location, you now can now proceed to create a TDE wallet and set a TDE master encryption key. If you later decide that you want the isolated mode PDB to become a united mode PDB again, then you can use the ADMINISTER KEY MANAGEMENT UNITE KEYSTORE statement. When you run ADMINISTER KEY MANAGEMENT UNITE KEYSTORE, it moves the keys from the PDB's keystore to the keystore of the CDB root, but it leaves any client secrets behind. So if there were no client secrets in the first place, then it would leave the PDB's keystore essentially "empty". It can now be backed up, and removed. Always back up keystores before you remove them, even empty keystores.

Related Topics

- [Configuring United Mode with the Initialization Parameter File and ALTER SYSTEM](#)
If your environment relies on server parameter files (spfile), then you can set WALLET_ROOT and TDE_CONFIGURATION using ALTER SYSTEM SET with SCOPE.
- [Uniting a Pluggable Database Keystore](#)
Uniting a PDB keystore moves the TDE master encryption keys from the PDB keystore into the keystore of the CDB root. This enables the administrator of the keystore of the CDB root to manage the keys.

8.4.2 Example: Restoring an Older Version of a Control File

You can set TDE_CONFIGURATION if you have an older version of a control file that must be restored and only a few PDBs were configured in isolated mode.

When the CDB root and the PDB are both in the mount state, then you can only change the PDB's keystore configuration from the CDB root.

1. Log in to the CDB root as a user who was granted the SYSDBA administrative privilege.
2. For each PDB that you want to change, use the following syntax:

```
ALTER SYSTEM SET  
TDE_CONFIGURATION="CONTAINER=pdb_name;KEYSTORE_CONFIGURATION=keystore_type"  
SCOPE=memory;
```

For example, for the hrpdb and salespdb PDBs using FILE (for TDE wallets) as the keystore type:

```
ALTER SYSTEM SET TDE_CONFIGURATION="CONTAINER=hrpdb;KEYSTORE_CONFIGURATION=FILE"  
SCOPE=memory;
```

```
ALTER SYSTEM SET TDE_CONFIGURATION="CONTAINER=salespdb;KESTORE_CONFIGURATION=FILE"
SCOPE=memory;
```

3. After you set the TDE_CONFIGURATION parameter for each PDB, log in to the CDB root and then set TDE_CONFIGURATION for the CDB root itself.

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE";
```

At this stage, CDB root is in the mounted state. The value of the TDE_CONFIGURATION parameter that was set using ALTER SYSTEM with the CONTAINER attribute is only present in the memory of the CDB root. To ensure that the configuration is properly applied to each PDB, you must close and then reopen the PDB. When an isolated mode PDB is opened, the configuration set by the ALTER SYSTEM statement that was issued in the CDB root is read from the control file and then is automatically applied to the PDB.

4. Connect to each PDB and then close and reopen the PDB.

```
ALTER PLUGGABLE DATABASE pdb_name CLOSE IMMEDIATE;
ALTER PLUGGABLE DATABASE pdb_name OPEN;
```

8.4.3 Example: Addressing the Problem of a Lost Control File

You can address the problem of a lost control file by using the ALTER SYSTEM statement.

Running these statements with SCOPE set to memory will store the CONTAINER value in memory. When you open the isolated PDB, this configuration will automatically be updated for the PDB.

If you are using an Oracle Data Guard environment, then to correct the control file, run these statements on both the primary and the standby databases.

1. Log in to the CDB root as a user who was granted the SYSDBA administrative privilege.
2. If you are unsure of the exact state of the system, then you should run ALTER SYSTEM with RESET.

For example:

```
ALTER SYSTEM RESET TDE_CONFIGURATION SCOPE=memory;
```

3. For each PDB that you want to change, use the following syntax:

```
ALTER SYSTEM SET TDE_CONFIGURATION="CONTAINER=pdb_name;KESTORE_CONFIGURATION=FILE"
SCOPE=memory;
```

For example, for the hrpdb and salespdb PDBs with FILE (for TDE wallets) as the keystore type:

```
ALTER SYSTEM SET TDE_CONFIGURATION="CONTAINER=hrpdb;KESTORE_CONFIGURATION=FILE"
SCOPE=memory;
ALTER SYSTEM SET TDE_CONFIGURATION="CONTAINER=salespdb;KESTORE_CONFIGURATION=FILE"
SCOPE=memory;
```

4. After you set the TDE_CONFIGURATION parameter for each PDB, log in to the CDB root and then set TDE_CONFIGURATION for the CDB root itself.

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE";
```

At this stage, CDB root is in the mounted state. The value of the TDE_CONFIGURATION parameter that was set using ALTER SYSTEM with the CONTAINER attribute is only present in the memory of the CDB root. To ensure that the configuration is properly applied to each PDB, you must close and then reopen the PDB. When an isolated mode PDB is opened, the configuration set by the ALTER SYSTEM statement that was issued in the CDB root is read from the control file and then is automatically applied to the PDB.

5. Connect to each PDB and then close and reopen the PDB.

```
ALTER PLUGGABLE DATABASE pdb_name CLOSE IMMEDIATE;  
ALTER PLUGGABLE DATABASE pdb_name OPEN;
```

8.4.4 Example: Configuring Isolated Mode in an Oracle Real Application Clusters Environment

You can use ALTER SYSTEM to configure isolated mode in an Oracle Real Application Clusters (Oracle RAC) environment.

- To ensure that the effect of the ALTER SYSTEM statement is applied on each Oracle RAC node, specify the wildcard (*) in the SID clause of the ALTER SYSTEM statement, as follows. You can run this statement from either the CDB root or a PDB.

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=keystore_type" SID='*';
```

8.4.5 Resolving the Transition From an Isolated Mode PDB to a United Mode PDB in a Data Guard Environment

1. In an Oracle Data Guard environment, convert the isolated PDB to unified mode on the primary database by running the following on the isolated PDB:

```
ALTER SYSTEM RESET tde_configuration
```

2. On the standby (secondary) database, convert the same isolated PDB to united mode by running the following

```
ALTER SYSTEM RESET tde_configuration
```

Running ALTER SYSTEM RESET tde_configuration on the primary alone will not convert the PDB to united mode. To complete the conversion from isolated to united, you must run ALTER SYSTEM RESET tde_configuration for the same PDB on both the primary and the standby. If you only copy the wallets to the standby but do not run the reset command there, running the reset on the primary will not update the standby. The standby will continue to look for the master keys in the isolated PDB wallet directory, which will no longer contain the required keys. After converting the wallets on both the primary and standby to united mode, confirm that the Data Guard sync/redo apply is successful before performing any additional wallet-related operations.

8.5 Configuring a TDE Wallet and TDE Master Encryption Key in Isolated Mode

In isolated mode, the TDE wallet is associated with a PDB.

- [About Configuring a TDE Wallet in Isolated Mode](#)
You can create all types of TDE wallets in isolated mode: password-protected, password protected with the credential provided from an external store, auto-login, local auto-login.
- [Step 1: Create a TDE Wallet in a PDB Configured in Isolated Mode](#)
A password-protected TDE wallet requires a password to protect the keystore keys and credentials.

- [Step 2: Open the TDE Wallet in an Isolated Mode PDB](#)
To open a TDE wallet in isolated mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.
- [Step 3: Set the TDE Master Encryption Key in the TDE Wallet of the Isolated Mode PDB](#)
To set the TDE master encryption key in a TDE wallet in an isolated mode PDB, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause.
- [Step 4: Encrypt Your Data in Isolated Mode](#)
Now that you have completed the configuration, you can begin to encrypt data in the PDB.

8.5.1 About Configuring a TDE Wallet in Isolated Mode

You can create all types of TDE wallets in isolated mode: password-protected, password protected with the credential provided from an external store, auto-login, local auto-login.

To enable encryption in the PDB after it is configured in isolated mode with the `KEYSTORE_CONFIGURATION` attribute set to `FILE` (that is, to use a TDE wallet), you must create a TDE wallet, open the TDE wallet, and then set a TDE master encryption key in the TDE wallet. Afterward, you can begin to encrypt data for tables and tablespaces that will be accessible in the PDB.

In a multitenant environment, you can create a secure external store to hold the credentials of the TDE wallet. This feature enables you to hide the keystore password: it removes the need for storing the wallet password in any script or tool that accesses the database without user intervention, such as an overnight batch script. When the `WALLET_ROOT` parameter is specified, the location of the external store for the CDB root is `WALLET_ROOT/tde_seps` and for the PDB it is `WALLET_ROOT/pdb_guid/tde_seps`. When the `WALLET_ROOT` parameter is set, there is no longer a single central external store, so when a keystore password is updated, the corresponding external store must be updated as well. When the `WALLET_ROOT` parameter is not specified, then the location of the external store is the same for both the CDB root and for every PDB. The external store location must then be set by the `EXTERNAL_KEYSTORE_CREDENTIAL_LOCATION` initialization parameter. When the `WALLET_ROOT` parameter is not specified, then there is a single central external store, so when you update the keystore password, only the central external store at the `EXTERNAL_KEYSTORE_CREDENTIAL_LOCATION` must be updated.

In a multitenant environment, different PDBs can access this external store location when you run the `ADMINISTER KEY MANAGEMENT` statement using the `IDENTIFIED BY EXTERNAL STORE` clause. This way, you can centrally locate the password and then update it only once in the external store.

Related Topics

- [Configuring an External Store for a Keystore Password](#)
An external store for a keystore password stores the keystore password in a centrally accessed and managed location.

8.5.2 Step 1: Create a TDE Wallet in a PDB Configured in Isolated Mode

A password-protected TDE wallet requires a password to protect the keystore keys and credentials.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Use the `SHOW PARAMETER` command to confirm that `WALLET_ROOT` is set, and `TDE_CONFIGURATION` is set to `KEYSTORE_CONFIGURATION=FILE`.

3. Run the `ADMINISTER KEY MANAGEMENT` SQL statement to create the keystore using the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY TDE_wallet_password;
```

This command creates the `/tde` directory under `WALLET_ROOT/pdb_guid` (unless it already exists), and creates a password-protected TDE wallet in this directory. If `WALLET_ROOT` points to a shared directory in ASM (`+DATA/DB_UNIQUE_NAME`), then the `pdb_guid/tde` subdirectory and the TDE wallet is auto-created by the `CREATE KEYSTORE` statement.

In this specification, `TDE_wallet_password` is the password of the keystore that you, the security administrator, creates.

For example, to create the TDE wallet in the `WALLET_ROOT/pdb_guid/tde` directory:

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY password;
```

keystore altered.

After you complete these steps, the `ewallet.p12` file, which is the TDE wallet, appears in the wallet location.

Related Topics

- [Creating an Auto-Login or a Local Auto-Login Software Keystore](#)

As an alternative to password-protected keystores, you can create either an auto-login or local auto-login software keystore.

8.5.3 Step 2: Open the TDE Wallet in an Isolated Mode PDB

To open a TDE wallet in isolated mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Run the `ADMINISTER KEY MANAGEMENT` statement to open the TDE wallet.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY TDE_wallet_password | EXTERNAL STORE;
```

Keystore altered.

To switch over to opening the password-protected TDE wallet when an auto-login wallet is configured and is currently open, specify the `FORCE KEYSTORE` clause as follows.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
FORCE KEYSTORE  
IDENTIFIED BY TDE_wallet_password | EXTERNAL STORE;
```

Keystore altered.

Here, the `IDENTIFIED BY EXTERNAL STORE` clause is included in the statement because the wallet credentials exist in an external store. This enables the password-protected TDE wallet to be opened without specifying the TDE wallet password within the statement itself.

If the `WALLET_ROOT` parameter has been set, then Oracle Database finds the external store by searching in this path: `WALLET_ROOT/pdb_guid/tde_seps`.

3. Confirm that the TDE wallet is open.

```
SELECT STATUS FROM V$ENCRYPTION_WALLET;
```

Related Topics

- [About Opening TDE Wallets](#)

A password-protected TDE wallet must be open before any TDE master encryption keys can be created or accessed in the TDE wallet.

8.5.4 Step 3: Set the TDE Master Encryption Key in the TDE Wallet of the Isolated Mode PDB

To set the TDE master encryption key in a TDE wallet in an isolated mode PDB, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause.

1. Log in to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.

2. Ensure that the database is open in `READ WRITE` mode.

To find the status, run the `show pdbs` command.

3. Run the `ADMINISTER KEY MANAGEMENT SQL` statement to set the key in the TDE wallet.

For example, if the wallet of the PDB is password-protected, the PDB is open, and the wallet of the PDB is open:

```
ADMINISTER KEY MANAGEMENT SET KEY
IDENTIFIED BY keystore_password
WITH BACKUP USING 'emp_key_backup';
```

keystore altered.

If the keystore is closed:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEYSTORE
IDENTIFIED BY keystore_password
WITH BACKUP USING 'emp_key_backup';
```

keystore altered.

In this specification:

- `FORCE KEYSTORE` should be included if the keystore is closed. This automatically opens the keystore before setting the TDE master encryption key. The `FORCE KEYSTORE` clause also switches over to opening the password-protected TDE wallet when an auto-login keystore is configured and is currently open.
 - `IDENTIFIED BY` specifies the keystore password. Alternatively, if the wallet password is in an external store, you can use the `IDENTIFIED BY EXTERNAL STORE` clause.
4. Confirm that the TDE master encryption key is set.

```
SELECT MASTERKEY_ACTIVATED FROM V$DATABASE_KEY_INFO;
```

The output should be YES.

Related Topics

- [About Setting the TDE Wallet TDE Master Encryption Key](#)

The TDE master encryption key is stored in the TDE wallet.

8.5.5 Step 4: Encrypt Your Data in Isolated Mode

Now that you have completed the configuration, you can begin to encrypt data in the PDB.

Related Topics

- [Encrypting Columns in Tables](#)
You can use Transparent Data Encryption to encrypt individual columns in database tables.
- [Encryption Conversions for Tablespaces and Databases](#)
You can perform encryption operations on both offline and online tablespaces and databases.

8.6 Configuring a Container Database with Isolated Mode PDBs for Oracle Key Vault

Isolated PDBs have their own virtual wallet, with their own password, in Oracle Key Vault.

- [About Configuring an External Keystore in Isolated Mode](#)
You can configure an external keystore for a PDB when the PDB is configured in isolated mode.
- [Step 1: Configure Isolated PDBs for Oracle Key Vault](#)
You can configure isolated mode PDBs for Oracle Key Vault by setting the TDE_CONFIGURATION parameter.
- [Step 2: Open the External Keystore in an Isolated Mode PDB](#)
If the isolated PDB does not have an auto-open connection into the external keystore, then you must open it manually before you open the PDB.
- [Step 3: Set the First TDE Master Encryption Key in the External Keystore](#)
After you have opened the external keystore in an isolated mode PDB, you are ready to set the TDE master encryption key for the PDB.
- [Step 4: Encrypt Your Data in Isolated Mode](#)
Now that you have completed the configuration, you can begin to encrypt data in the PDB.

8.6.1 About Configuring an External Keystore in Isolated Mode

You can configure an external keystore for a PDB when the PDB is configured in isolated mode.

To configure an external keystore for a PDB in isolated mode, you first must set the WALLET_ROOT parameter. This is necessary for two reasons: first, to have support for migrating to a TDE wallet in the future, and second, because the configuration file for Oracle Key Vault is retrieved from a location under WALLET_ROOT. Afterwards, you must set the KEYSTORE_CONFIGURATION attribute of the TDE_CONFIGURATION parameter to OKV, open the configured external keystore, and then set the TDE master encryption key for the PDB. After you complete these tasks, you can begin to encrypt data in your database.

How you specify the IDENTIFIED BY clause when you run the ADMINISTER KEY MANAGEMENT statement depends on the type of external keystore. Use the following syntax:

```
IDENTIFIED BY EXTERNAL STORE|Oracle_Key_Vault_password
```

Enter the Oracle Key Vault password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

8.6.2 Step 1: Configure Isolated PDBs for Oracle Key Vault

You can configure isolated mode PDBs for Oracle Key Vault by setting the TDE_CONFIGURATION parameter.

1. If the WALLET_ROOT parameter is set, then install the Oracle Key Vault client software into the WALLET_ROOT/*pdb_guid*/okv directory.
2. Log in to the database instance as a user who has been granted the ALTER SYSTEM administrative privilege.
3. Set the TDE_CONFIGURATION dynamic initialization parameter to specify the keystore type by using the following syntax:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=keystore_type" SCOPE=BOTH  
SID = '*';
```

In this specification:

- *keystore_type* can be OKV, to configure an Oracle Key Vault keystore.

For example, to configure your database to use Oracle Key Vault:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=OKV" SCOPE=BOTH SID = '*';
```

8.6.3 Step 2: Open the External Keystore in an Isolated Mode PDB

If the isolated PDB does not have an auto-open connection into the external keystore, then you must open it manually before you open the PDB.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Open the external keystore as follows, by enclosing the password in double quotation marks:

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY Oracle_Key_Vault_password;
```

3. Repeat this procedure each time you restart the database instance.

You must open the keystore of the CDB root first.

Related Topics

- [About Opening the Connection to Oracle Key Vault](#)
You must open the connection to Oracle Key Vault so that it is accessible to the database before you can perform any encryption or decryption.

8.6.4 Step 3: Set the First TDE Master Encryption Key in the External Keystore

After you have opened the external keystore in an isolated mode PDB, you are ready to set the TDE master encryption key for the PDB.

- [Setting the TDE Master Encryption Key in the Isolated Mode External Keystore](#)
You should complete this procedure if you have not previously configured an external keystore for Transparent Data Encryption.
- [Migration of a Previously Configured Encryption Key in Isolated Mode](#)
You must migrate the previously configured master encryption key if you previously configured a TDE wallet.

8.6.4.1 Setting the TDE Master Encryption Key in the Isolated Mode External Keystore

You should complete this procedure if you have not previously configured an external keystore for Transparent Data Encryption.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.

2. Ensure that the database is open in READ WRITE mode.

You can set the TDE master encryption key if OPEN_MODE is set to READ WRITE. To find the status, run the show pdbs command.

3. To configure the heartbeat batch size, set the HEARTBEAT_BATCH_SIZE initialization parameter.

The HEARTBEAT_BATCH_SIZE parameter configures the size of the "batch of heartbeats" sent per heartbeat period to the external key manager. Enter a value between 2 and 100. The default value is 5. The default duration of the heartbeat period is three seconds. By setting the heartbeat batch size, you can stagger the heartbeats across batches of PDBs to ensure that for each batch a heartbeat can be completed for each PDB within the batch during the heartbeat period, and also ensure that the TDE master encryption key of the PDB can be reliably fetched from an Oracle Key Vault server and cached in the Oracle Key Vault persistent cache. (See [Heartbeat Batch Size for External Keystores](#) for details about how HEARTBEAT_BATCH_SIZE works.)

```
ALTER SYSTEM SET HEARTBEAT_BATCH_SIZE=3 SCOPE=BOTH SID='*';
```

4. Set the new TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEY
[USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY EXTERNAL STORE|Oracle_Key_Vault_password;
```

In this specification:

- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation if the TDE wallet is closed, if an auto-login TDE wallet is configured and is currently open, or if a password-protected TDE wallet is configured and is currently closed.
- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.
 - Oracle_Key_Vault_password is the the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY  
USING TAG 'sessionId=3205062574:terminal=xcvt'  
FORCE KEYSTORE  
IDENTIFIED BY EXTERNAL STORE;
```

keystore altered.

5. Confirm that the TDE master encryption key is set.

```
SELECT MASTERKEY_ACTIVATED FROM V$DATABASE_KEY_INFO;
```

The output should be YES.

Related Topics

- [About Setting the External Keystore TDE Master Encryption Key](#)
You must create a TDE master encryption key that is stored inside the external keystore.

8.6.4.2 Migration of a Previously Configured Encryption Key in Isolated Mode

You must migrate the previously configured master encryption key if you previously configured a TDE wallet.

Related Topics

- [Migration of an Encrypted Database from a TDE Wallet to Oracle Key Vault or OCI KMS](#)
To switch from a TDE wallet to centralized key management with Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS), after you upload all current and retired TDE master keys you must migrate the database from the TDE wallet to Oracle Key Vault or OCI KMS.

8.6.5 Step 4: Encrypt Your Data in Isolated Mode

Now that you have completed the configuration, you can begin to encrypt data in the PDB.

Related Topics

- [Encrypting Columns in Tables](#)
You can use Transparent Data Encryption to encrypt individual columns in database tables.
- [Encryption Conversions for Tablespaces and Databases](#)
You can perform encryption operations on both offline and online tablespaces and databases.

8.7 Administering Keystores and TDE Master Encryption Keys in Isolated Mode

After you create a keystore and a TDE master encryption key in isolated mode, you can perform administration tasks such as rekeying or tagging encryption keys.

To change the password of an external keystore, you must use the administrative interface of the external keystore. You cannot perform this operation by using the ADMINISTER KEY MANAGEMENT statement.

- [Changing the Keystore Password in Isolated Mode](#)
You can change the password of a TDE wallet when the PDB is in isolated mode.

- [Backing Up a Password-Protected TDE Wallet in Isolated Mode](#)
The `BACKUP KEYSTORE` clause of the `ADMINISTER KEY MANAGEMENT` statement backs up a password-protected TDE wallet.
- [Merging TDE Wallets in Isolated Mode](#)
In isolated mode, you can merge TDE wallets.
- [Closing Keystores in Isolated Mode](#)
You can close both software and external keystores in isolated mode, unless the system tablespace is encrypted.
- [Creating a User-Defined TDE Master Encryption Key in Isolated Mode](#)
To create a user-defined TDE master encryption key, use the `ADMINISTER KEY MANAGEMENT` statement with the `SET | CREATE [ENCRYPTION] KEY` clause.
- [Creating a TDE Master Encryption Key for Later Use in Isolated Mode](#)
A keystore must be open before you can create a TDE master encryption key for use later on in isolated mode.
- [Activating a TDE Master Encryption Key in Isolated Mode](#)
To activate a TDE master encryption key in isolated mode, you must open the keystore and use `ADMINISTER KEY MANAGEMENT` with the `USE KEY` clause.
- [Rekeying the TDE Master Encryption Key in Isolated Mode](#)
You can use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause to rekey a TDE master encryption key.
- [Moving a TDE Master Encryption Key into a New Keystore in Isolated Mode](#)
In isolated mode, you can move an existing TDE master encryption key into a new TDE wallet from an existing password TDE wallet.
- [Creating a Custom Attribute Tag in Isolated Mode](#)
To create a custom attribute tag in isolated mode, you must use the `SET TAG` clause of the `ADMINISTER KEY MANAGEMENT` statement.
- [Exporting and Importing the TDE Master Encryption Key in Isolated Mode](#)
You can export and import the TDE master encryption key in different ways in isolated mode.
- [Storing Oracle Database Secrets in Isolated Mode](#)
Secrets are data that support internal Oracle Database features that integrate external clients such as Oracle GoldenGate into the database.
- [Migrating Keystores in Isolated Mode](#)
You can perform migration and reverse migration operations between software keystores and external keystores in isolated mode.
- [Uniting a Pluggable Database Keystore](#)
Uniting a PDB keystore moves the TDE master encryption keys from the PDB keystore into the keystore of the CDB root. This enables the administrator of the keystore of the CDB root to manage the keys.
- [Creating a Keystore When the PDB Is Closed](#)
When you create a keystore in a PDB that is closed, the new keystore is empty and the PDB is converted to isolated mode.

8.7.1 Changing the Keystore Password in Isolated Mode

You can change the password of a TDE wallet when the PDB is in isolated mode.

To change the password of an external keystore, you must use the administrative interface of the external keystore. You cannot perform this operation by using the `ADMINISTER KEY MANAGEMENT` statement.

- [Changing the Password-Protected TDE Wallet Password in Isolated Mode](#)
To change the password of a password-protected TDE wallet in isolated mode, you must use the `ADMINISTER KEY MANAGEMENT` statement.
- [Changing the Password of an External Keystore in Isolated Mode](#)
To change the password of an external keystore, you must close the external keystore and then change the password from the external keystore's management interface.

8.7.1.1 Changing the Password-Protected TDE Wallet Password in Isolated Mode

To change the password of a password-protected TDE wallet in isolated mode, you must use the `ADMINISTER KEY MANAGEMENT` statement.

You can change this password at any time, as per the security policies, compliance guidelines, and other security requirements of your site. As part of the command to change the password, you will be forced to specify the `WITH BACKUP` clause, and thus forced to make a backup of the current TDE wallet. During the password change operation, Transparent Data Encryption operations such as encryption and decryption will continue to work normally. You can change this password at any time. You may want to change this password if you think it was compromised.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Change the password for the TDE wallet by using the following syntax:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
[FORCE KEYSTORE]
IDENTIFIED BY
old_password SET new_password
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation if the TDE wallet is closed, if an auto-login TDE wallet is configured and is currently open, or if a password-protected TDE wallet is configured and is currently closed.
- *old_password* is the current TDE wallet password that you want to change.
- *new_password* is the new password that you set for the TDE wallet.

The following example creates a backup of the TDE wallet and then changes the TDE wallet password:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
IDENTIFIED BY
old_password SET new_password
WITH BACKUP USING 'pwd_change';
```

keystore altered.

This example performs the same operation but uses the `FORCE KEYSTORE` clause in case the auto-login TDE wallet is in use or the password-protected TDE wallet is closed.

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
FORCE KEYSTORE
```

```
IDENTIFIED BY  
old_password SET new_password  
WITH BACKUP USING 'pwd_change';
```

keystore altered.

Related Topics

- [Performing Operations That Require a Keystore Password](#)

Many ADMINISTER KEY MANAGEMENT operations require access to a keystore password, for both TDE wallets and external keystores.

8.7.1.2 Changing the Password of an External Keystore in Isolated Mode

To change the password of an external keystore, you must close the external keystore and then change the password from the external keystore's management interface.

1. Connect to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Close the external keystore.

- For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY "external_key_manager_password";
```

- For an external keystore whose password is stored externally:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY EXTERNAL STORE;
```

3. Update the credentials of the external store to use "new_external_key_manager_password".

Currently, the external store contains the old credentials, which would no longer work.

For example:

```
ADMINISTER KEY MANAGEMENT  
UPDATE SECRET 'new_external_key_manager_password'  
FOR CLIENT 'TDE_WALLET'  
TO LOCAL AUTO_LOGIN KEYSTORE '/etc/ORACLE/WALLETS/orcl/external_store';
```

4. Open the external keystore.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY "new_external_key_manager_password";
```

For an external keystore whose password is stored externally:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY EXTERNAL STORE;
```

Related Topics

- [Performing Operations That Require a Keystore Password](#)

Many ADMINISTER KEY MANAGEMENT operations require access to a keystore password, for both TDE wallets and external keystores.

8.7.2 Backing Up a Password-Protected TDE Wallet in Isolated Mode

The `BACKUP KEYSTORE` clause of the `ADMINISTER KEY MANAGEMENT` statement backs up a password-protected TDE wallet.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Back up the keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
[USING 'backup_identifier']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password]
[TO 'TDE_wallet_location'];
```

In this specification:

- `USING backup_identifier` is an optional string that you can provide to identify the backup. Enclose this identifier in single quotation marks (' '). This identifier is appended to the named keystore file (for example, `ewallet_timestamp_emp_key_backup.p12`).
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` is required for the `BACKUP KEYSTORE` operation on a password-protected keystore because although the backup is simply a copy of the existing keystore, the status of the TDE master encryption key in the password-protected keystore must be set to `BACKED UP` and for this change the keystore password is required.
- `TDE_wallet_location` is the path at which the backup TDE wallet is stored. This setting is restricted to the PDB when the PDB lockdown profile `EXTERNAL_FILE_ACCESS` setting is blocked in the PDB or when the `PATH_PREFIX` variable was set when the PDB was created. If you do not specify the `TDE_wallet_location`, then the backup is created in the same directory as the original TDE wallet. Enclose this location in single quotation marks (' ').

The following example backs up a TDE wallet in the same location as the source TDE wallet.

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
USING 'hr.emp_keystore'
FORCE KEYSTORE
IDENTIFIED BY
'TDE_wallet_password';
```

keystore altered.

In the following version, the password for the TDE wallet is stored externally, so the `EXTERNAL STORE` clause is used.

```
ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE
USING 'hr.emp_keystore'
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE;
```

After you run this statement, an `ewallet_identifier.p12` file (for example, `ewallet_timestamp_hr.emp_keystore.p12`) appears in the TDE wallet backup location.

Related Topics

- [Backing Up Password-Protected Software Keystores](#)
When you back up a password-protected software keystore, you can create a backup identifier string to describe the backup type.

8.7.3 Merging TDE Wallets in Isolated Mode

In isolated mode, you can merge TDE wallets.

- [Merging One TDE Wallet into an Existing TDE Wallet in Isolated Mode](#)
In isolated mode, you can use the `ADMINISTER KEY MANAGEMENT` statement with the `MERGE KEYSTORE` clause to merge one TDE wallet into another existing TDE wallet.
- [Merging Two TDE Wallets into a Third New TDE Wallet in Isolated Mode](#)
In isolated mode, you can merge two TDE wallets into a third new TDE wallet, so that the two existing TDE wallets are not changed and the new TDE wallet contains the keys of both source TDE wallets.

8.7.3.1 Merging One TDE Wallet into an Existing TDE Wallet in Isolated Mode

In isolated mode, you can use the `ADMINISTER KEY MANAGEMENT` statement with the `MERGE KEYSTORE` clause to merge one TDE wallet into another existing TDE wallet.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Merge the TDE wallets by using the following syntax:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE 'TDE_wallet1_location'  
[IDENTIFIED BY TDE_wallet1_password]  
INTO EXISTING KEYSTORE 'TDE_wallet2_location'  
IDENTIFIED BY TDe_wallet2_password  
[WITH BACKUP [USING 'backup_identifier]];
```

In this specification:

- *TDE_wallet1_location* is the directory location of the first TDE wallet, which will be left unchanged after the merge. Enclose this path in single quotation marks (' ').
- The `IDENTIFIED BY` clause is required for the first TDE wallet if it is a password-protected TDE wallet. *TDE_wallet1_password* is the password for the first TDE wallet.
- *TDE_wallet2_location* is the directory location of the second TDE wallet into which the first TDE wallet is to be merged. Enclose this path in single quotation marks (' ').
- *TDE_wallet2_password* is the password for the second TDE wallet.

The target TDE wallet (*TDE_wallet2*) remains a password-protected TDE wallet after the TDE wallet merge operation.

Related Topics

- [About Merging TDE Wallets](#)
You can merge any combination of TDE wallets, but the merged keystore must be password-protected. It can have a password that is different from the constituent wallets.

8.7.3.2 Merging Two TDE Wallets into a Third New TDE Wallet in Isolated Mode

In isolated mode, you can merge two TDE wallets into a third new TDE wallet, so that the two existing TDE wallets are not changed and the new TDE wallet contains the keys of both source TDE wallets.

1. Connect to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Merge the TDE wallets by using the following syntax:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE 'TDE_wallet1_location'  
[IDENTIFIED BY TDE_wallet1_password] AND TDE_wallet 'TDE_wallet2_location'  
[IDENTIFIED BY TDE_wallet2_password]  
INTO NEW KEYSTORE 'TDE_wallet3_location'  
IDENTIFIED BY TDE_wallet3_password;
```

In this specification:

- *TDE_wallet1_location* is the directory location of the first TDE wallet, which will be left unchanged after the merge. Enclose this path in single quotation marks (' ').
- The IDENTIFIED BY clause is required for the first TDE wallet if it is a password-protected TDE wallet. *TDE_wallet1_password* is the current password for the first TDE wallet.
- *TDE_wallet2_location* is the directory location of the second TDE wallet. Enclose this path in single quotation marks (' ').
- The IDENTIFIED BY clause is required for the second TDE wallet if it is a password-protected TDE wallet. *TDE_wallet2_password* is the current password for the second TDE wallet.
- *TDE_wallet3_location* specifies the directory location of the new, merged TDE wallet. Enclose this path in single quotation marks (' '). If there is already an existing TDE wallet at this location, the command exits with an error.
- *TDE_wallet3_password* is the new password for the merged TDE wallet.

The following example merges an auto-login TDE wallet with a password-protected TDE wallet to create a merged password-protected TDE wallet at a new location:

```
ADMINISTER KEY MANAGEMENT MERGE KEYSTORE '/etc/ORACLE/KEYSTORE/DB1'  
AND KEYSTORE '/etc/ORACLE/KEYSTORE/DB2'  
IDENTIFIED BY existing_password_for_TDE_wallet_2  
INTO NEW KEYSTORE '/etc/ORACLE/KEYSTORE/DB3'  
IDENTIFIED BY new_password_for_TDE_wallet_3;
```

keystore altered.

Related Topics

- [About Merging TDE Wallets](#)
You can merge any combination of TDE wallets, but the merged keystore must be password-protected. It can have a password that is different from the constituent wallets.

8.7.4 Closing Keystores in Isolated Mode

You can close both software and external keystores in isolated mode, unless the system tablespace is encrypted.

- [Closing a TDE Wallet in Isolated Mode](#)
You can close password-protected TDE wallets, auto-login TDE wallets, and local auto-login TDE wallets in isolated mode.
- [Closing an External Keystore in Isolated Mode](#)
To close an external keystore, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE CLOSE` clause.

8.7.4.1 Closing a TDE Wallet in Isolated Mode

You can close password-protected TDE wallets, auto-login TDE wallets, and local auto-login TDE wallets in isolated mode.

In the case of an auto-login TDE wallet, which opens automatically when it is accessed, you must first move it to a new location where it cannot be automatically opened, then you must manually close it. You must do this if you are changing your configuration from an auto-login TDE wallet to a password-protected TDE wallet: you change the configuration to stop using the auto-login TDE wallet (by moving the auto-login TDE wallet to another location where it cannot be automatically opened), and then closing the auto-login TDE wallet.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Close the TDE wallet by using the following syntax.

Note that the only difference between the following two `ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE` statements is that a password must be provided for a password-protected TDE wallet.

- For a password-protected TDE wallet:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE  
IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password];
```

Closing a password-protected TDE wallet disables all encryption and decryption operations. Any attempt to encrypt or decrypt data or access encrypted data results in an error.

- For an auto-login or local auto-login TDE wallet:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE;
```

The result of this statement will not necessarily be that the TDE wallet status will change to `CLOSED`, because unless you also moved the `cwallet.sso` file to a location that Oracle Database cannot find, then a background job or background process could automatically re-open the auto-login TDE wallet. This can cause the status to potentially always appear to be `OPEN` even after the `ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE` statement completed successfully.

Related Topics

- [About Closing Keystores](#)
After you open a keystore, it remains open until you shut down the database instance.

8.7.4.2 Closing an External Keystore in Isolated Mode

To close an external keystore, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE CLOSE` clause.

For an Oracle Key Vault keystore, you can only provide the password. No user name is allowed in the `IDENTIFIED BY` clause. Enclose the password in double quotation marks.

1. Connect to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Close the external keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE
IDENTIFIED BY [EXTERNAL STORE | "external_key_manager_password"];
```

Closing a keystore disables all encryption and decryption operations. Any attempt to encrypt or decrypt data or access encrypted data results in an error.

Related Topics

- [About Closing Keystores](#)
After you open a keystore, it remains open until you shut down the database instance.

8.7.5 Creating a User-Defined TDE Master Encryption Key in Isolated Mode

To create a user-defined TDE master encryption key, use the ADMINISTER KEY MANAGEMENT statement with the SET | CREATE [ENCRYPTION] KEY clause.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Create the user-defined TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET | CREATE [ENCRYPTION] KEY
'mkid:mk | mk'
[USING ALGORITHM 'algorithm']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- SET | CREATE : Enter SET if you want to create the master and activate the TDE master encryption key now, or enter CREATE if you want to create the key for later use, without activating it yet.
- *mkid* and *mk*:
 - *mkid*, the TDE master encryption key ID, is a 16-byte hex-encoded value that you can specify or have Oracle Database generate.
 - *mk*, the TDE master encryption key, is a hex-encoded value that you can specify or have Oracle Database generate, either 32 bytes (for the for AES256, ARIA256, and GOST256 algorithms) or 16 bytes (for the SEED128 algorithm).

If you omit the *mkid:mk|mkid* clause but include the *mk* value, then Oracle Database generates the *mkid* for the *mk*.

If you omit the entire *mkid:mk|mkid* clause, then Oracle Database generates these values for you.

- USING ALGORITHM: Specify one of the following supported algorithms:
 - AES256
 - ARIA256
 - SEED128
 - GOST256

If you omit the algorithm, then the default, AES256, is used.

- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

The following example includes a user-created TDE master encryption key but no TDE master encryption key ID, so that the TDE master encryption key ID is generated:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
'3D432109DF88967A541967062A6F4E460E892318E307F017BA048707B402493C'
USING ALGORITHM 'GOST256'
FORCE KEYSTORE
IDENTIFIED BY keystore_password WITH BACKUP;
```

The next example creates user-defined keys for both the master encryption ID and the TDE master encryption key. It omits the algorithm specification, so the default algorithm AES256 is used.

```
ADMINISTER KEY MANAGEMENT CREATE ENCRYPTION KEY
'10203040506070801112131415161718:3D432109DF88967A541967062A6F4E460E892318E307F017BA0
48707B402493C'
IDENTIFIED BY keystore_password WITH BACKUP;
```

Related Topics

- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.

8.7.6 Creating a TDE Master Encryption Key for Later Use in Isolated Mode

A keystore must be open before you can create a TDE master encryption key for use later on in isolated mode.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Create the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE KEY [USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY EXTERNAL STORE | keystore_password
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` is required for the `BACKUP KEYSTORE` operation on a password-protected keystore because although the backup is simply a copy of the existing keystore, the status of the TDE master encryption key in the password-protected keystore must be set to `BACKED UP` and for this change the keystore password is required.
- *keystore_location* is the path at which the backup keystore is stored. This setting is restricted to the PDB when the PDB lockdown profile `EXTERNAL_FILE_ACCESS` setting is blocked in the PDB or when the `PATH_PREFIX` variable was not set when the PDB was created. If you do not specify the *keystore_location*, then the backup is created in the same directory as the original keystore. Enclose this location in single quotation marks (' ').

For example:

```
ADMINISTER KEY MANAGEMENT CREATE KEY
FORCE KEYSTORE
IDENTIFIED BY keystore_password
WITH BACKUP;
```

3. If necessary, activate the TDE master encryption key.

- a. Find the key ID.

```
SELECT KEY_ID FROM V$ENCRYPTION_KEYS;

KEY_ID
-----
AWShwVYC2U+Nv3RVphn/yAIAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

- b. Use this key ID to activate the key.

```
ADMINISTER KEY MANAGEMENT USE KEY
'AWShwVYC2U+Nv3RVphn/yAIAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
USING TAG 'quarter:second;description:Activate Key on standby'
IDENTIFIED BY password
WITH BACKUP;
```

Related Topics

- [About Creating a TDE Master Encryption Key for Later Use](#)
The CREATE KEY clause of the ADMINISTER KEY MANAGEMENT statement can create a TDE master encryption key to be activated at a later date.

8.7.7 Activating a TDE Master Encryption Key in Isolated Mode

To activate a TDE master encryption key in isolated mode, you must open the keystore and use ADMINISTER KEY MANAGEMENT with the USE KEY clause.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Query the KEY_ID column of the V\$ENCRYPTION_KEYS view to find the key identifier.

For example:

```
SELECT KEY_ID FROM V$ENCRYPTION_KEYS;

KEY_ID
-----
ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

3. Use this key identifier to activate the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT USE KEY 'key_identifier_from_V$ENCRYPTION_KEYS'
[USING TAG 'tag']
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

For example:

```
ADMINISTER KEY MANAGEMENT USE KEY
'ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
FORCE KEYSTORE
```

```
IDENTIFIED BY keystore_password
WITH BACKUP;
```

Related Topics

- [About Activating TDE Master Encryption Keys](#)
You can activate a previously created or imported TDE master encryption key by using the `USE KEY` clause of `ADMINISTER KEY MANAGEMENT`.

8.7.8 Rekeying the TDE Master Encryption Key in Isolated Mode

You can use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEY` clause to rekey a TDE master encryption key.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. If you are rekeying the TDE master encryption key for a keystore that has auto login enabled, then ensure that both the auto login keystore, identified by the `.sso` file, and the encryption keystore, identified by the `.p12` file, are present.

You can find the location of these files by querying the `WRL_PARAMETER` column of the `V$ENCRYPTION_WALLET` view. To find the `WRL_PARAMETER` values for all of the database instances, query the `GV$ENCRYPTION_WALLET` view.

3. Rekey the TDE master encryption key by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY
[FORCE KEYSTORE]
[USING TAG 'tag_name']
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *tag* is the associated attributes and information that you define. Enclose this setting in single quotation marks (' ').
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- `IDENTIFIED BY` can be one of the following settings:
 - `EXTERNAL STORE` uses the keystore password stored in the external store to perform the keystore operation.
 - *keystore_password* is the password that was created for this keystore.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEYSTORE
IDENTIFIED BY keystore_password
WITH BACKUP USING 'emp_key_backup';
```

keystore altered.

Related Topics

- [About Rekeying the TDE Master Encryption Key](#)
Oracle Database uses a unified TDE Master Encryption Key for both TDE column encryption and TDE tablespace encryption.

8.7.9 Moving a TDE Master Encryption Key into a New Keystore in Isolated Mode

In isolated mode, you can move an existing TDE master encryption key into a new TDE wallet from an existing password TDE wallet.

This feature enables you to move a subset, or all, of a TDE wallet's keys into a new TDE wallet. After you move the key or keys to the new TDE wallet and then back up the old TDE wallet, optionally you then can delete this old TDE wallet.

1. Connect to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Query the CREATION_TIME and KEY_ID columns of the V\$ENCRYPTION_KEYS view to find the key identifier of the key that you want to move.

For example:

```
SELECT CREATION_TIME, KEY_ID FROM V$ENCRYPTION_KEYS;
```

```
CREATION TIME
```

```
-----  
22-SEP-19 08.55.12.956170 PM +00:00
```

```
KEY_ID
```

```
-----  
ARaHD762tUkkvyLgPzAi6hMAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

3. Move the key into a new TDE wallet by using the following syntax:

```
ADMINISTER KEY MANAGEMENT  
MOVE [ENCRYPTION] KEYS  
TO NEW KEYSTORE 'TDE_wallet_location1'  
IDENTIFIED BY TDE_wallet1_password  
FROM [FORCE] KEYSTORE  
IDENTIFIED BY TDE_wallet2_password  
[WITH IDENTIFIER IN  
{ 'key_identifier' [, 'key_identifier' ]... | ( subquery ) }]  
[WITH BACKUP [USING 'backup_identifier'] ];
```

In this specification:

- *TDE_wallet_location1* is the path to the wallet directory that will store the new TDE wallet .p12 file. By default, this directory is in \$ORACLE_BASE/admin/db_unique_name/wallet.
- *TDE_wallet1_password* is the password for the TDE wallet from which the new TDE wallet is moved.
- FORCE temporarily opens the TDE wallet for this operation.
- *TDE_wallet2_password* is the password for the TDE wallet from which the key is moving.
- *subquery* can be used to find the exact key identifier that you want.
- *backup_identifier* is an optional description of the backup. Enclose *backup_identifier* in single quotation marks (' ').

For example:

```
ADMINISTER KEY MANAGEMENT MOVE KEYS  
TO NEW KEYSTORE '$ORACLE_BASE/admin/orcl/wallet'
```

```

IDENTIFIED BY TDE_wallet1_password
FROM FORCE KEYSTORE
IDENTIFIED BY TDE_wallet2_password
WITH IDENTIFIER IN
(SELECT KEY_ID FROM V$ENCRYPTION_KEYS WHERE ROWNUM < 2)
WITH BACKUP;

```

After the keys are moved to the new TDE wallet, they no longer exist in the old TDE wallet.

4. To delete the old TDE wallet, go to the wallet directory and do the following:
 - a. Back up the .p12 file containing the TDE wallet that you want to delete.
 - b. Manually delete the .p12 file containing the TDE wallet.

To find the location of the TDE wallet, open the TDE wallet, and then query the WRL_PARAMETER column of the V\$ENCRYPTION_WALLET view.

Related Topics

- [About Moving TDE Master Encryption Keys into a New Keystore](#)
You can move an unused (and safely archived) TDE master encryption key into a new keystore.
- [Dangers of Deleting Keystores](#)
Oracle strongly recommends that you do not delete keystores.

8.7.10 Creating a Custom Attribute Tag in Isolated Mode

To create a custom attribute tag in isolated mode, you must use the SET TAG clause of the ADMINISTER KEY MANAGEMENT statement.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. If necessary, query the TAG column of the V\$ENCRYPTION_KEY dynamic view to find a listing of existing tags for the TDE master encryption keys.

When you create a new tag for a TDE master encryption key, it overwrites the existing tag for that TDE master encryption key.

3. Create the custom attribute tag by using the following syntax:

```

ADMINISTER KEY MANAGEMENT SET TAG 'tag'
FOR 'master_key_identifier'
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];

```

In this specification:

- *tag* is the associated attributes or information that you define. Enclose this information in single quotation marks (' ').
- *master_key_identifier* identifies the TDE master encryption key for which the *tag* is set. To find a list of TDE master encryption key identifiers, query the KEY_ID column of the V\$ENCRYPTION_KEYS dynamic view.
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- IDENTIFIED BY can be one of the following settings:
 - EXTERNAL STORE uses the keystore password stored in the external store to perform the keystore operation.

- *keystore_password* is the password that was created for this keystore.
- *backup_identifier* defines the tag values. Enclose this setting in single quotation marks (' ') and separate each value with a colon.

For example, to create a tag that uses two values, one to capture a specific session ID and the second to capture a specific terminal ID:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
USING TAG 'sessionid=3205062574:terminal=xcvt'
IDENTIFIED BY keystore_password
WITH BACKUP;
```

keystore altered.

Both the session ID (3205062574) and terminal ID (xcvt) can derive their values by using either the SYS_CONTEXT function with the USERENV namespace, or by using the USERENV function.

Related Topics

- [About Creating Custom Attribute Tags](#)
Attribute tags enable you to monitor specific activities users perform, such as accessing a particular terminal ID.

8.7.11 Exporting and Importing the TDE Master Encryption Key in Isolated Mode

You can export and import the TDE master encryption key in different ways in isolated mode.

- [Exporting a TDE Master Encryption Key in Isolated Mode](#)
In isolated mode, you can use the ADMINISTER KEY MANAGEMENT statement to export a TDE master encryption key.
- [Importing a TDE Master Encryption Key in Isolated Mode](#)
The ADMINISTER KEY MANAGEMENT statement with the IMPORT [ENCRYPTION] KEYS WITH SECRET clause can import a TDE master encryption key.

8.7.11.1 Exporting a TDE Master Encryption Key in Isolated Mode

In isolated mode, you can use the ADMINISTER KEY MANAGEMENT statement to export a TDE master encryption key.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Export the TDE master encryption keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT EXPORT [ENCRYPTION] KEYS
WITH SECRET "export_secret"
TO 'file_path'
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH IDENTIFIER IN 'key_id1', 'key_id2', 'key_idn' | (SQL_query)];
```

In this specification:

- *export_secret* is a password that you can specify to encrypt the export the file that contains the exported keys. Enclose this secret in double quotation marks (" "), or you can omit the quotation marks if the secret has no spaces.

- *file_path* is the complete path and name of the file to which the keys must be exported. Enclose this path in single quotation marks (' '). You can export to regular file systems only.
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- *key_id1, key_id2, key_idn* is a string of one or more TDE master encryption key identifiers for the TDE master encryption key being exported. Separate each key identifier with a comma and enclose each of these key identifiers in single quotation marks (' '). To find TDE master encryption key identifiers, query the `KEY_ID` column of the `V$ENCRYPTION_KEYS` dynamic view.
- *SQL_query* is a query that fetches a list of the TDE master encryption key identifiers. It should return only one column that contains the TDE master encryption key identifiers. This query is executed with current user rights.

Related Topics

- [Exporting and Importing the TDE Master Encryption Key](#)
You can export and import the TDE master encryption key in different ways.

8.7.11.2 Importing a TDE Master Encryption Key in Isolated Mode

The `ADMINISTER KEY MANAGEMENT` statement with the `IMPORT [ENCRYPTION] KEYS WITH SECRET` clause can import a TDE master encryption key.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Import the TDE master encryption keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT IMPORT [ENCRYPTION] KEYS
WITH SECRET "import_secret"
FROM 'file_name'
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | keystore_password]
[WITH BACKUP [USING 'backup_identifier']];
```

In this specification:

- *import_secret* is the same password that was used to encrypt the keys during the export operation. Enclose this secret in double quotation marks (" "), or you can omit the quotation marks if the secret has no spaces.
- *file_name* is the complete path and name of the file from which the keys need to be imported. Enclose this setting in single quotation marks (' ').
- `FORCE KEYSTORE` temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

Related Topics

- [Exporting and Importing the TDE Master Encryption Key](#)
You can export and import the TDE master encryption key in different ways.

8.7.12 Storing Oracle Database Secrets in Isolated Mode

Secrets are data that support internal Oracle Database features that integrate external clients such as Oracle GoldenGate into the database.

- [Storing Oracle Database Secrets in a TDE Wallet in Isolated Mode](#)
The ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET statements can add secrets, update secrets, and delete secrets in a TDE wallet.
- [Storing Oracle Database Secrets in an External Keystore in Isolated Mode](#)
The ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET statements can add secrets, update secrets, and delete secrets in a keystore.

8.7.12.1 Storing Oracle Database Secrets in a TDE Wallet in Isolated Mode

The ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET statements can add secrets, update secrets, and delete secrets in a TDE wallet.

1. Connect to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Add, update, or delete a database secret in a TDE wallet by using the following syntax:

- To add a secret:

```
ADMINISTER KEY MANAGEMENT
ADD SECRET 'secret' FOR CLIENT 'client_identifier'
[USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE TDE_wallet_location
[WITH BACKUP [USING backup_id]]
| [FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password]
[WITH BACKUP [USING backup_id]]];
```

- To update a secret:

```
ADMINISTER KEY MANAGEMENT
UPDATE SECRET 'secret' FOR CLIENT 'client_identifier'
[USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE TDE_wallet_location
[WITH BACKUP [USING backup_id]]
| [FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password]
[WITH BACKUP [USING backup_id]]];
```

- To delete a secret:

```
ADMINISTER KEY MANAGEMENT
DELETE SECRET FOR CLIENT 'client_identifier'
[FROM [[LOCAL] AUTO_LOGIN] KEYSTORE TDE_wallet_location
[FORCE KEYSTORE]
IDENTIFIED BY [EXTERNAL STORE | TDE_wallet_password]
[WITH BACKUP [USING backup_id]]];
```

The specification is as follows:

- *secret* is the client secret key to be stored, updated, or deleted. To find information about existing secrets and their client identifiers, query the V\$CLIENT_SECRETS dynamic view.
- *client_identifier* is an alphanumeric string used to identify the secret key.
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.

Related Topics

- [Storing Oracle Database Secrets](#)
Secrets are data that support internal Oracle Database features that integrate external clients such as Oracle GoldenGate into the database.

8.7.12.2 Storing Oracle Database Secrets in an External Keystore in Isolated Mode

The ADMINISTER KEY MANAGEMENT ADD SECRET|UPDATE SECRET|DELETE SECRET statements can add secrets, update secrets, and delete secrets in a keystore.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Add, update, or delete a database secret in an external keystore by using the following syntax:

- To add a secret:

```
ADMINISTER KEY MANAGEMENT
ADD SECRET 'secret' FOR CLIENT 'client_identifier'
[USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY "external_key_manager_password"
[WITH BACKUP [USING backup_id]];
```

- To update a secret:

```
ADMINISTER KEY MANAGEMENT
UPDATE SECRET 'secret' FOR CLIENT 'client_identifier'
[USING TAG 'tag']
[TO [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY "external_key_manager_password"
[WITH BACKUP [USING backup_id]];
```

- To delete a secret:

```
ADMINISTER KEY MANAGEMENT
DELETE SECRET FOR CLIENT 'client_identifier'
[FROM [[LOCAL] AUTO_LOGIN] KEYSTORE keystore_location
[FORCE KEYSTORE]
IDENTIFIED BY "external_key_manager_password";
```

The specification is as follows:

- *secret* is the client secret key to be stored, updated, or deleted. To find information about existing secrets and their client identifiers, query the V\$CLIENT_SECRETS dynamic view.
- *client_identifier* is an alphanumeric string used to identify the secret key.
- FORCE KEYSTORE temporarily opens the password-protected TDE wallet for this operation. You must open the TDE wallet for this operation.
- *external_key_manager_password* is for an external keystore manager, which can be Oracle Key Vault or OCI Vault - Key Management. Enclose this password in double quotation marks. For Oracle Key Vault, enter the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

Related Topics

- [Storing Oracle Database Secrets](#)
Secrets are data that support internal Oracle Database features that integrate external clients such as Oracle GoldenGate into the database.

8.7.13 Migrating Keystores in Isolated Mode

You can perform migration and reverse migration operations between software keystores and external keystores in isolated mode.

- [Reverse Migrating an Isolated PDB from Oracle Key Vault to a TDE Wallet](#)
Isolated PDBs have individual keystores (TDE wallets or Oracle Key Vault external keystores), and can individually be migrated from the TDE wallet to Oracle Key Vault, and individually reverse migrated from Oracle Key Vault back to the TDE wallet.
- [Migrating from an External Keystore to a Password-Protected TDE Wallet in Isolated Mode](#)
In isolated mode, you can migrate from an external keystore to a password-protected TDE wallet.

8.7.13.1 Reverse Migrating an Isolated PDB from Oracle Key Vault to a TDE Wallet

Isolated PDBs have individual keystores (TDE wallets or Oracle Key Vault external keystores), and can individually be migrated from the TDE wallet to Oracle Key Vault, and individually reverse migrated from Oracle Key Vault back to the TDE wallet.

For both the TDE wallet and the external keystore to open at the same time, either the TDE wallet must have the same password as the external keystore, or alternatively, after the migration has completed you can create an auto-login keystore for the TDE wallet.

1. Before migrating any database from wallet to Oracle Key Vault, upload the contents of the wallet (the current and retired TDE master encryption keys) into the virtual wallet that you created in Oracle Key Vault for that database.

In the following example, the static initialization parameter `WALLET_ROOT` is set to `/etc/ORACLE/KEYSTORES/${ORACLE_SID}`:

```
/etc/ORACLE/KEYSTORES/${ORACLE_SID}/pdb-guid/okv/bin/okvutil upload -l
/etc/ORACLE/KEYSTORES/${ORACLE_SID}/pdb-guid/tde/ -t wallet -g
virtual_wallet_name_in_Oracle_Key_Vault -v 2
```

2. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.

3. Set the password of the external keystore so that it matches that of the TDE wallet.

Some Oracle tools, such as Oracle Data Pump and Oracle Recovery Manager, require access to the old TDE wallet to decrypt data that was exported or backed up using the TDE master encryption key from the old TDE wallet.

- To set the TDE wallet password so that it is the same as that of the external keystore, use the following syntax:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
[FORCE KEYSTORE]
IDENTIFIED BY TDE_wallet_password
SET "external_key_manager_password"
WITH BACKUP [USING 'backup_identifier'];
```

In this specification:

- `TDE_wallet_password` is the password that was assigned to this wallet when it was created.
- `external_key_manager_password` is for an external keystore manager, which can be Oracle Key Vault or OCI Vault - Key Management. Enclose this password in

double quotation marks. For Oracle Key Vault, enter the password that was given during the Oracle Key Vault client installation. If at that time no password was given, then the password in the ADMINISTER KEY MANAGEMENT statement becomes NULL.

- Alternatively, to create an auto-login keystore for a TDE wallet, use the following syntax:

```
ADMINISTER KEY MANAGEMENT CREATE [LOCAL] AUTO_LOGIN KEYSTORE
FROM KEYSTORE 'keystore_location'
IDENTIFIED BY TDE_wallet_password;
```

4. Provision Oracle Key Vault for the isolated mode PDB by following the instructions in *Oracle Key Vault Administrator's Guide* to install the Oracle Key Vault software onto the endpoint.
5. Set the configuration of the keystore so that the external keystore becomes the new primary, and the password-protected TDE wallet becomes the secondary, as follows:

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=OKV|FILE";
```

6. Migrate the external keystore by using the following syntax:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
IDENTIFIED BY "external_key_manager_password"
[FORCE KEYSTORE]
MIGRATE USING TDE_wallet_password;
```

After you complete the migration, you do not need to restart the database, nor do you need to manually re-open the external keystore.

Related Topics

- [Keystore Order After a Migration](#)
After you perform a migration, keystores can be either primary or secondary in their order.
- [Migration of Keystores to and from Oracle Key Vault](#)
You can use Oracle Key Vault to migrate both TDE wallets and external keystores to and from Oracle Key Vault.

8.7.13.2 Migrating from an External Keystore to a Password-Protected TDE Wallet in Isolated Mode

In isolated mode, you can migrate from an external keystore to a password-protected TDE wallet.

1. Log in to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Ensure that you have enrolled the PDB endpoint and that the Oracle Key Vault configuration is present at the \$WALLET_ROOT/pdb_guid/okv location.

For both the TDE wallet and the external keystore to open at the same time, either the TDE wallet must have the same password as the external keystore, or alternatively, after the reverse migration has completed, you can create an auto-login TDE wallet for the TDE wallet.

3. Set the TDE_CONFIGURATION parameter as follows, so that FILE becomes the new primary keystore, and OKV becomes the secondary keystore.

```
ALTER SYSTEM SET TDE_CONFIGURATION="KESTORE_CONFIGURATION=FILE|OKV";
```

- Now that the keystore configuration has been completed, issue the following statement to reverse migrate from the external keystore to the password-protected TDE wallet:

```
ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY
IDENTIFIED BY TDE_wallet_password
REVERSE MIGRATE USING "external_key_manager_password"
[WITH BACKUP [USING 'backup_identifier']];
```

- Optionally, change the password of the newly migrated TDE wallet.

For example:

```
ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD
IDENTIFIED BY
old_password SET new_password
WITH BACKUP USING 'pwd_change';
```

After you complete these steps, the migration process automatically reloads the keystore keys in memory. You do not need to restart the database, nor do you need to manually re-open the TDE wallet. The external keystore may still be required after reverse migration because the old keys are likely to have been used for encrypted backups or by tools such as Oracle Data Pump and Oracle Recovery Manager. You should create an auto-login keystore and put the HSM_PASSWORD client secret into it. For example:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'external_key_manager_password'
FOR CLIENT 'EXTERNAL_PASSWORD'
TO LOCAL AUTO_LOGIN KEYSTORE TDE_wallet_location
WITH BACKUP;
```

Related Topics

- [About Migrating Back from an External Keystore](#)
To switch from using an external keystore solution to a TDE wallet, you can use reverse migration of the TDE wallet.
- [Keystore Order After a Migration](#)
After you perform a migration, keystores can be either primary or secondary in their order.
- [Migration of Keystores to and from Oracle Key Vault](#)
You can use Oracle Key Vault to migrate both TDE wallets and external keystores to and from Oracle Key Vault.

8.7.14 Uniting a Pluggable Database Keystore

Uniting a PDB keystore moves the TDE master encryption keys from the PDB keystore into the keystore of the CDB root. This enables the administrator of the keystore of the CDB root to manage the keys.

The client secrets are not moved. Instead, they are left behind in the keystore that the PDB used while it was configured in isolated mode. Oracle recommends that you delete client secrets from that keystore before you unite the PDB keystore. Similarly, when a PDB becomes isolated, no client secret contained in the keystore of the CDB root is moved.

- Connect to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
- Unite the PDB keystore, which moves the TDE master encryption keys from the PDB keystore into the keystore of the CDB root, by using the following syntax:

```
ADMINISTER KEY MANAGEMENT UNITE KEYSTORE
IDENTIFIED BY isolated_keystore_password
WITH ROOT KEYSTORE [FORCE KEYSTORE]
IDENTIFIED BY
```

```
[EXTERNAL STORE | keystore_password_of_cdb_root]
[WITH BACKUP [USING backup_id]];
```

In this specification:

- **FORCE KEYSTORE** temporarily opens the password-protected TDE wallet for this operation if an auto-login TDE wallet is open (and in use) or if the TDE wallet is closed.
- *united_keystore_password*: Knowledge of this password does not enable the user who performs the **UNITE KEYSTORE** operation privileges to perform **ADMINISTER KEY MANAGEMENT UNITE KEYSTORE** operations on the PDB.

When the keystore of a PDB is united with a keystore in the CDB root, all of the previously active (historical) TDE master encryption keys that were associated with the PDB are moved to the keystore of the CDB root. (If an ORA-46694: The keys are already in the root keystore error appears, see below.)

3. Confirm that the isolated mode PDB is now a united mode PDB.

```
SELECT KEYSTORE_MODE FROM V$ENCRYPTION_WALLET;
```

The output should be **UNITED**.

The keystore no longer exists but its master encryption key is now in the keystore in the CDB root. If you later decide that you want the united mode PDB to be an isolated mode PDB again, then you can use the **ADMINISTER KEY MANAGEMENT ISOLATE KEYSTORE** statement.

ORA-46694 error: If a wallet is created in the in a PDB context, then it changes the keystore type of the PDB to isolated, and unless a key is set for the PDB, the wallet status will be **OPEN_NO_MASTER_KEY**. Uniting this type of PDB using the **ADMINISTER KEY MANAGEMENT UNITE KEYSTORE** statement will result in an ORA-46694: The keys are already in the root keystore error. To change the **KEYSTORE_MODE** to **UNITED** for this PDB, you must change the TDE configuration for the PDB. In the PDB, run the following statement:

```
ALTER SYSTEM RESET TDE_CONFIGURATION;
```

Related Topics

- [Isolating a Pluggable Database Keystore](#)
Isolating a PDB keystore moves the master encryption key from the CDB root keystore into an isolated mode keystore in the a PDB.

8.7.15 Creating a Keystore When the PDB Is Closed

When you create a keystore in a PDB that is closed, the new keystore is empty and the PDB is converted to isolated mode.

- [About Creating a Keystore When the PDB Is Closed](#)
Creating a keystore in a PDB that is closed could inadvertently cause problems in rekey operations, but the keystore creation can be reverted.
- [Reverting a Keystore Creation Operation When a PDB Is Closed](#)
If you have inadvertently created a keystore in a PDB (and thereby caused it to become configured in isolated mode), then you should reverse the keystore creation operation.

8.7.15.1 About Creating a Keystore When the PDB Is Closed

Creating a keystore in a PDB that is closed could inadvertently cause problems in rekey operations, but the keystore creation can be reverted.

In previous releases, if you tried to create a keystore in a closed PDB, you were prevented and an ORA-65040: operation not allowed from within a pluggable database error would appear. Starting in Oracle Database release 18c, for convenience, when the keystore of the PDB is closed and if you run the `ADMINISTER KEY MANAGEMENT CREATE KEYSTORE` statement in the PDB, Oracle Database allows the operation.

If the closed PDB has not been configured to use encryption (that is, it has never had an `ADMINISTER KEY MANAGEMENT SET KEY` statement performed in it), after you execute `ADMINISTER KEY MANAGEMENT CREATE KEYSTORE`, resulting in an empty keystore and the configuration of the PDB being changed to isolated mode, then you can create a TDE master encryption key in this empty keystore.

If, however, the PDB was already configured to use encryption, then the PDB may be configured in united mode (and thus have its TDE master encryption key being managed in the keystore of the CDB root).

Mistakenly running an `ADMINISTER KEY MANAGEMENT CREATE KEYSTORE` statement on such a closed PDB will create an additional keystore (which will be empty), and will then configure the PDB to be in isolated mode. This effectively misconfigures the PDB, because the PDB is now in isolated mode (whereas it should be in united mode), yet its TDE master encryption key is still in the keystore of the CDB root. This misconfiguration can cause problems later on, if you try to rekey the TDE master encryption key by using the `ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY` statement. An ORA-28362: master key not found error will appear, because when encryption has already been enabled and a key has been set, Oracle Database treats the `ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY` statement as a rekey operation. In order to perform a rekey operation, Oracle Database must locate the currently active TDE master encryption key of the PDB. But in this misconfigured PDB, Oracle Database cannot locate the TDE master encryption key, because the PDB is now in isolated mode and the necessary key is in the keystore of the CDB root. Hence, the PDB is no longer configured to search in the keystore of the PDB, and the rekey operation fails.

To remedy the misconfiguration of the PDB, you must reconfigure the PDB to united mode and you must remove the empty keystore. (Always make a backup before removing any keystore.) When the PDB is configured back to united mode, then the currently active TDE master encryption key is once again available for rekey and other TDE master encryption key operations.

If later on you want to configure the PDB to be in isolated mode, then you can open the PDB and run the `ADMINISTER KEY MANAGEMENT ISOLATE KEYSTORE` statement, which isolates the PDB and moves its TDE master encryption key and previously-active (historical) keys from the keystore of the CDB root to a newly-created keystore for the isolated PDB.

Related Topics

- [Reverting a Keystore Creation Operation When a PDB Is Closed](#)
If you have inadvertently created a keystore in a PDB (and thereby caused it to become configured in isolated mode), then you should reverse the keystore creation operation.

8.7.15.2 Reverting a Keystore Creation Operation When a PDB Is Closed

If you have inadvertently created a keystore in a PDB (and thereby caused it to become configured in isolated mode), then you should reverse the keystore creation operation.

Use this procedure if you created a keystore in a closed PDB that already had encryption enabled (that is, it already had a TDE master encryption key).

1. Log in to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.

2. Confirm the mode of the PDB by querying the `KEYSTORE_MODE` column of the `V$ENCRYPTION_WALLET` dynamic view.
3. If the `V$ENCRYPTION_WALLET` output is `ISOLATED`, then execute the `ALTER SYSTEM` statement to reconfigure the PDB to united mode.
 - When pfile is in use, clear the `TDE_CONFIGURATION` parameter by using the following statement:


```
ALTER SYSTEM RESET TDE_CONFIGURATION SCOPE=MEMORY;
```

In an Oracle Real Application Clusters environment, include the `SID` parameter:

```
ALTER SYSTEM RESET TDE_CONFIGURATION SCOPE=MEMORY SID='*';
```
 - When spfile is in use, clear the `TDE_CONFIGURATION` parameter by using this statement:


```
ALTER SYSTEM RESET TDE_CONFIGURATION SCOPE=BOTH;
```

In an Oracle Real Application Clusters environment, include the `SID` parameter:

```
ALTER SYSTEM RESET TDE_CONFIGURATION SCOPE=BOTH SID='*';
```
4. In the `WALLET_ROOT/PDB_GUID/tde` directory, find and back up the `ewallet.p12` keystore file that was mistakenly created.
5. Delete the mistakenly-created empty keystore file.

At this stage, the PDB will be in united mode and the correct keystore and TDE master encryption key will be available for any future rekey operations.

8.8 Administering Transparent Data Encryption in Isolated Mode

You can perform a number of general administrative tasks with Transparent Data Encryption in isolated mode.

- [Cloning or Relocating Encrypted PDBs in Isolated Mode](#)
You can clone or relocate encrypted PDBs within the same container database, or across container databases.
- [Unplugging and Plugging a PDB with Encrypted Data in a CDB in Isolated Mode](#)
In isolated mode, for a PDB that has encrypted data, you can plug it into a CDB. Conversely, you can unplug this PDB from the CDB.
- [Cloning a PDB with Encrypted Data in a CDB in Isolated Mode](#)
The `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause can clone a PDB that has encrypted data.
- [Performing a Remote Clone of PDB with Encrypted Data Between Two CDBs in Isolated Mode](#)
The `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause can remotely clone a PDB that has encrypted data.
- [Relocating an Encrypted PDB in Isolated Mode](#)
The `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause can relocate across CDBs a cloned PDB that has encrypted data.
- [How Keystore Open and Close Operations Work in Isolated Mode](#)
You should be aware of how keystore (TDE wallets or external keystores) open and close operations work in isolated mode.

- [Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode](#)
In isolated mode, the EXPORT and IMPORT clauses of ADMINISTER KEY MANAGEMENT EXPORT can export or import master encryption keys for a PDB.

8.8.1 Cloning or Relocating Encrypted PDBs in Isolated Mode

You can clone or relocate encrypted PDBs within the same container database, or across container databases.

If you are trying to move a PDB in which the SYSTEM, SYSAUX, UNDO, or TEMP tablespace is encrypted, and using the manual export or import of keys, then you must first import the keys for the PDB in the target database's CDB\$ROOT before you create the PDB. Import of the keys are again required inside the PDB to associate the keys to the PDB.

- Clone or relocate the PDB using the following syntax:

```
CREATE|RELOCATE PLUGGABLE DATABASE database_name KEYSTORE  
IDENTIFIED BY EXTERNAL STORE|target_keystore_password [NO REKEY];
```

Related Topics

- *Oracle Multitenant Administrator's Guide*

8.8.2 Unplugging and Plugging a PDB with Encrypted Data in a CDB in Isolated Mode

In isolated mode, for a PDB that has encrypted data, you can plug it into a CDB. Conversely, you can unplug this PDB from the CDB.

- [Unplugging a PDB That Has Encrypted Data in Isolated Mode](#)
You can unplug a PDB (that has encrypted data) from one CDB and then optionally plug it into another CDB.
- [Plugging a PDB That Has Encrypted Data into a CDB in Isolated Mode](#)
After you plug a PDB that has encrypted data into a CDB, you can set the encryption key in the PDB.
- [Unplugging a PDB That Has Master Encryption Keys Stored in an External Keystore in Isolated Mode](#)
You can unplug a PDB from one CDB that has been configured with an external keystore and then plug it into another CDB also configured with an external keystore.
- [Plugging a PDB That Has Master Encryption Keys Stored in an External Keystore in Isolated Mode](#)
The ADMINISTER KEY MANAGEMENT statement can import an external keystore master encryption key to a PDB that has been moved to another CDB.

8.8.2.1 Unplugging a PDB That Has Encrypted Data in Isolated Mode

You can unplug a PDB (that has encrypted data) from one CDB and then optionally plug it into another CDB.

Unlike united mode, you do not need to specify the ENCRYPT clause in the ALTER PLUGGABLE DATABASE statement. The database that is unplugged contains data files and other associated files. Because each PDB can have its own unique keystore, you do not need to export the TDE master encryption key of the PDB that you want to unplug. You can check if a PDB has already been unplugged by querying the STATUS column of the DBA_PDBS data dictionary view.

- Unplug the isolated mode PDB as you normally unplug PDBs.

For example:

```
ALTER PLUGGABLE DATABASE pdb1
  UNPLUG INTO '/oracle/data/pdb1.xml';
```

Related Topics

- *Oracle Multitenant Administrator's Guide*
- *Oracle Database SQL Language Reference*

8.8.2.2 Plugging a PDB That Has Encrypted Data into a CDB in Isolated Mode

After you plug a PDB that has encrypted data into a CDB, you can set the encryption key in the PDB.

Unlike united mode, you do not need to specify the `DECRYPT` clause in the `CREATE PLUGGABLE DATABASE` statement. When you plug an unplugged PDB into another CDB, the key version is set to 0 because this operation invalidates the history of the previous keys. You can check the key version by querying the `KEY_VERSION` column of the `V$ENCRYPTED_TABLESPACES` dynamic view. Similarly, if a control file is lost and recreated, then the previous history of the keys is reset to 0. You can check if a PDB has already been plugged in by querying the `STATUS` column of the `DBA_PDBS` data dictionary view.

1. Create the PDB by plugging the unplugged PDB into the CDB.

- For example, if you had exported the PDB data into a metadata XML file:

```
CREATE PLUGGABLE DATABASE CDB1_PDB2
  USING '/tmp/cdb1_pdb2.xml'
  NOCOPY KEYSTORE
  IDENTIFIED BY password;
```

- If you had exported the PDB into an archive file:

```
CREATE PLUGGABLE DATABASE CDB1_PDB2
  USING '/tmp/cdb1_pdb2.pdb';
```

During the open operation of the PDB after the plug operation, Oracle Database determines if the PDB has encrypted data. If so, it opens the PDB in the `RESTRICTED` mode.

You can find if the source database has encrypted data or a TDE master encryption key set in the keystore by querying the `V$ENCRYPTION_KEYS` dynamic view.

2. Open the PDB.

For example:

```
ALTER PLUGGABLE DATABASE CDB1_PDB2 OPEN;
```

3. Open the keystore in the CDB root.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
  IDENTIFIED BY password;
```

Optionally, open the keystore in the PDB.

4. In the PDB, open the keystore and set the TDE master encryption key for the PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY  
IDENTIFIED BY keystore_password  
WITH BACKUP USING 'emp_key_backup';
```

8.8.2.3 Unplugging a PDB That Has Master Encryption Keys Stored in an External Keystore in Isolated Mode

You can unplug a PDB from one CDB that has been configured with an external keystore and then plug it into another CDB also configured with an external keystore.

1. Unplug the PDB.

You can check if a PDB has already been unplugged by querying the STATUS column of the DBA_PDBS data dictionary view.

2. Move the master encryption keys of the unplugged PDB from the hardware that was used at the source CDB to the hardware that is in use at the destination CDB.

Refer to the documentation for the external keystore for information about moving master keys between external keystores.

Related Topics

- *Oracle Multitenant Administrator's Guide*

8.8.2.4 Plugging a PDB That Has Master Encryption Keys Stored in an External Keystore in Isolated Mode

The ADMINISTER KEY MANAGEMENT statement can import an external keystore master encryption key to a PDB that has been moved to another CDB.

1. Plug the unplugged isolated mode PDB into the destination CDB that has been configured with the external keystore.

You can check if a PDB has already been plugged in by querying the STATUS column of the DBA_PDBS data dictionary view.

After the plug-in operation, the PDB that has been plugged in will be in restricted mode.

2. Ensure that the master keys from the external keystore that has been configured with the source CDB are available in the external keystore of the destination CDB.
3. Log in to the plugged PDB as a user who was granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
4. Open the keystore of the plugged PDB.

For example, for a PDB called PDB1:

```
ALTER SESSION SET CONTAINER = PDB1;  
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN  
IDENTIFIED BY "external_key_manager_password";
```

5. Import the external keystore master encryption key into the PDB.

```
ADMINISTER KEY MANAGEMENT IMPORT ENCRYPTION KEYS  
WITH SECRET "external_keystore" FROM 'external_keystore'  
IDENTIFIED BY "external_key_manager_password";
```

6. Close and re-open the PDB.

```
ALTER PLUGGABLE DATABASE PDB1 CLOSE;  
ALTER PLUGGABLE DATABASE PDB1 OPEN;
```

Related Topics

- [Oracle Multitenant Administrator's Guide](#)

8.8.3 Cloning a PDB with Encrypted Data in a CDB in Isolated Mode

The `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause can clone a PDB that has encrypted data.

1. Connect to the isolated mode PDB as a user who has been granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Ensure that the TDE wallet of the PDB that you plan to clone is open.

You can query the `STATUS` column of the `V$ENCRYPTION_WALLET` view to find if the TDE wallet is open.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN FORCE KEYSTORE IDENTIFIED BY  
TDE_wallet_password;
```

3. Clone the PDB.

For example:

```
CREATE PLUGGABLE DATABASE cdb1_pdb3 FROM cdb1_pdb1  
FILE_NAME_CONVERT=('cdb1_pdb1', 'pdb3/cdb1_pdb3') KEYSTORE  
IDENTIFIED BY TDE_wallet_password;
```

Replace `TDE_wallet_password` with the password of the TDE wallet of the CDB where the `cdb1_pdb3` clone is created.

After you create the cloned PDB, encrypted data is still accessible by the clone using the master encryption key of the original PDB. After a PDB is cloned, there may be user data in the encrypted tablespaces. This encrypted data is still accessible because the master encryption key of the source PDB is copied over to the destination PDB. Because the clone is a copy of the source PDB but will eventually follow its own course and have its own data and security policies, you should rekey the master encryption key of the cloned PDB.

4. Rekey the master encryption key of the cloned PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY  
IDENTIFIED BY TDE_wallet_password  
WITH BACKUP USING 'emp_key_backup';
```

Before you rekey the master encryption key of the cloned PDB, the clone can still use master encryption keys that belong to the original PDB. However, these master encryption keys do not appear in the cloned PDB `V$` dynamic views. Rekeying the master encryption key ensures that the cloned PDB uses its own unique keys, which will be viewable in the `V$` views.

Related Topics

- [About Managing Cloned PDBs That Have Encrypted Data in United Mode](#)

When you clone a PDB, you must make the master encryption key of the source PDB available to cloned PDB.

8.8.4 Performing a Remote Clone of PDB with Encrypted Data Between Two CDBs in Isolated Mode

The `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause can remotely clone a PDB that has encrypted data.

1. Log in to the isolated mode PDB as a user who was granted the `ADMINISTER KEY MANAGEMENT` or `SYSKM` privilege.
2. Query the `STATUS` column of the `V$ENCRYPTION_WALLET` view to ensure that the TDE wallet of the PDB that you plan to clone is open.
3. Create a database link for the PDB that you want to clone remotely.

Use the `CREATE DATABASE LINK SQL` statement to create the database link. You must create the database link by following the database link prerequisites that are required for cloning a remote PDB or a non-CDB.

4. Use the `CREATE PLUGGABLE DATABASE` statement with the `KEYSTORE IDENTIFIED BY` clause to perform the clone of the PDB.

For example:

```
CREATE PLUGGABLE DATABASE cdb1_pdb3
FROM cdb1_pdb1
FILE_NAME_CONVERT=('cdb1_pdb1', 'pdb3/cdb1_pdb3') KEystore
IDENTIFIED BY TDE_wallet_password;
```

Replace `TDE_wallet_password` with the password of the TDE wallet of the CDB where the `cdb1_pdb3` clone is created.

After you create the cloned PDB, encrypted data is still accessible by the clone using the master encryption key of the original PDB. After a PDB is cloned, there may be user data in the encrypted tablespaces. This encrypted data is still accessible because the master encryption key of the source PDB is copied over to the destination PDB. Because the clone is a copy of the source PDB but will eventually follow its own course and have its own data and security policies, you should rekey the master encryption key of the cloned PDB.

5. Rekey the master encryption key of the remotely cloned PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEystore
IDENTIFIED BY TDE_wallet_password
WITH BACKUP USING 'emp_key_backup';
```

In this example, `FORCE KEystore` is included because the TDE wallet must be open during the rekey operation.

Before you rekey the master encryption key of the cloned PDB, the clone can still use master encryption keys that belong to the original PDB. However, these master encryption keys do not appear in the cloned PDB `V$` dynamic views. Rekeying the master encryption key ensures that the cloned PDB uses its own unique keys, which will be viewable in the `V$` views.

Related Topics

- [About Managing Cloned PDBs That Have Encrypted Data in United Mode](#)
When you clone a PDB, you must make the master encryption key of the source PDB available to cloned PDB.
- [Step 2: Open the TDE Wallet in a United Mode PDB](#)
To open a TDE wallet in united mode, you must use the ADMINISTER KEY MANAGEMENT statement with the SET KEYSTORE OPEN clause.
- *Oracle Multitenant Administrator's Guide*
- *Oracle Multitenant Administrator's Guide*
- *Oracle Database SQL Language Reference*

8.8.5 Relocating an Encrypted PDB in Isolated Mode

The CREATE PLUGGABLE DATABASE statement with the KEYSTORE IDENTIFIED BY clause can relocate across CDBs a cloned PDB that has encrypted data.

1. Connect to the isolated mode PDB as a user who has been granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Query the STATUS column of the V\$ENCRYPTION_WALLET view to ensure that the TDE wallet of the PDB that you want to relocate is open.
3. Create a database link for the PDB that you want to relocate.

Use the CREATE DATABASE LINK SQL statement to create the database link. You must create the database link by following the database link prerequisites that are required for relocating a remote PDB or a non-CDB.

4. Use the CREATE PLUGGABLE DATABASE statement with the KEYSTORE IDENTIFIED BY clause to relocate the PDB.

For example:

```
CREATE PLUGGABLE DATABASE target_pdb_name
FROM src_pdb_name@dblink RELOCATE
KEYSTORE IDENTIFIED BY TDE_wallet_password;
```

Replace *TDE_wallet_password* with the password of the TDE wallet of the CDB where the cdb1_pdb3 clone is created.

After you create the cloned PDB, encrypted data is still accessible by the clone using the master encryption key of the original PDB. After a PDB is cloned, there may be user data in the encrypted tablespaces. This encrypted data is still accessible because the master encryption key of the source PDB is copied over to the destination PDB. Because the clone is a copy of the source PDB but will eventually follow its own course and have its own data and security policies, you should rekey the master encryption key of the cloned PDB.

5. Rekey the master encryption key of the remotely cloned PDB.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
FORCE KEYSTORE
IDENTIFIED BY TDE_wallet_password
WITH BACKUP USING 'emp_key_backup';
```


In this example, `FORCE KEYSTORE` is included because the TDE wallet must be open during the rekey operation.

Before you rekey the master encryption key of the cloned PDB, the clone can still use master encryption keys that belong to the original PDB. However, these master encryption keys do not appear in the cloned PDB `V$` dynamic views. Rekeying the master encryption key ensures that the cloned PDB uses its own unique keys, which will be viewable in the `V$` views.

Related Topics

- [About Managing Cloned PDBs That Have Encrypted Data in United Mode](#)
When you clone a PDB, you must make the master encryption key of the source PDB available to cloned PDB.
- [Step 2: Open the TDE Wallet in a United Mode PDB](#)
To open a TDE wallet in united mode, you must use the `ADMINISTER KEY MANAGEMENT` statement with the `SET KEYSTORE OPEN` clause.
- *Oracle Multitenant Administrator's Guide*
- *Oracle Multitenant Administrator's Guide*
- *Oracle Database SQL Language Reference*

8.8.6 How Keystore Open and Close Operations Work in Isolated Mode

You should be aware of how keystore (TDE wallets or external keystores) open and close operations work in isolated mode.

For each PDB in isolated mode, you must explicitly open the password-protected TDE wallet or external keystore in the PDB to enable the Transparent Data Encryption operations to proceed. (Auto-login and local auto-login TDE wallets open automatically.) Closing a keystore on a PDB blocks all of the Transparent Data Encryption operations on that PDB.

The open and close keystore operations in a PDB depend on the open and close status of the keystore in the PDB.

Note the following:

- You can create a separate keystore password for each PDB in the multitenant environment.
- Before you can manually open a password-protected software or an external keystore in an individual PDB, you must open the keystore in the CDB root.
- If an auto-login TDE wallet is in use, or if the TDE wallet is closed, then include the `FORCE KEYSTORE` clause in the `ADMINISTER KEY MANAGEMENT` statement when you open the TDE wallet.
- If the keystore is a password-protected TDE wallet that uses an external store for passwords, then replace the password in the `IDENTIFIED BY` clause with `EXTERNAL STORE`.
- Before you can set a TDE master encryption key in an individual PDB, you must set the key in the CDB root. Oracle highly recommends that you include the `USING TAG` clause when you set keys in PDBs. For example:

```
SELECT ' ADMINISTER KEY MANAGEMENT SET KEY
USING TAG '''||SYS_CONTEXT('USERENV', 'CON_NAME')||' '||TO_CHAR (SYSDATE,
'YYYY-MM-DD HH24:MI:SS')||'''
```

```
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE
WITH BACKUP CONTAINER = CURRENT;' AS "SET KEY COMMAND" FROM DUAL;
```

Including the USING TAG clause enables you to quickly and easily identify the keys that belong to a certain PDB, and when they were created.

- Auto-login and local auto-login TDE wallets open automatically. You do not need to manually open these from the root first, or from the PDB.
- If there is any PDB configured in isolated mode that has its keystore open, then an attempt to close the keystore in the CDB root would fail with an ORA-46692 cannot close wallet error. Use the FORCE CLOSE clause in the ADMINISTER KEY MANAGEMENT statement to override this behavior.
- If you perform an ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN statement in the CDB root and set the CONTAINER clause to ALL, then the keystore will only be opened in each open PDB that is configured in united mode. Keystores for any PDBs that are configured in isolated mode are not opened.

8.8.7 Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode

In isolated mode, the EXPORT and IMPORT clauses of ADMINISTER KEY MANAGEMENT EXPORT can export or import master encryption keys for a PDB.

- [About Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode](#)
In isolated mode, you can export and import master encryption keys from the CDB root in the same way that you export and import this key for a non-CDB database.
- [Exporting or Importing a Master Encryption Key for a PDB in Isolated Mode](#)
In isolated mode, the ADMINISTER KEY MANAGEMENT statement can export or import a master encryption key for a PDB.
- [Example: Exporting a Master Encryption Key from a PDB in Isolated Mode](#)
The ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS statement can export master encryption keys for a PDB.
- [Example: Importing a Master Encryption Key into a PDB in Isolated Mode](#)
The ADMINISTER KEY MANAGEMENT IMPORT ENCRYPTION KEYS statement can import a master encryption key into a PDB.

8.8.7.1 About Exporting and Importing Master Encryption Keys for a PDB in Isolated Mode

In isolated mode, you can export and import master encryption keys from the CDB root in the same way that you export and import this key for a non-CDB database.

You can export and import all of the master encryption keys that belong to the PDB by exporting and importing the master encryption keys from within a PDB. Export and import operations of master encryption keys in a PDB supports the PDB unplug and plug operations. During a PDB unplug and plug operations, all the master encryption keys that belong to a PDB, as well as the metadata, are involved. Therefore, the WITH IDENTIFIER clause of the ADMINISTER KEY MANAGEMENT EXPORT statement is not allowed when you export keys from within a PDB. The WITH IDENTIFIER clause is only permitted in the CDB root.

You should include the FORCE KEYSTORE clause if the CDB root has an auto-login keystore or if the keystore is closed. If the keystore has been configured to use an external store for the

password, then use the IDENTIFIED BY EXTERNAL STORE clause. For example, to perform an export operation for this scenario:

```
ADMINISTER KEY MANAGEMENT EXPORT KEYS  
WITH SECRET "my_secret"  
TO '/etc/TDE/export.exp'  
FORCE KEYSTORE  
IDENTIFIED BY EXTERNAL STORE;
```

This ADMINISTER KEY MANAGEMENT EXPORT operation exports not only the keys but creates metadata that is necessary for PDB environments (as well as for cloning operations).

Inside a PDB, the export operation of master encryption keys exports the keys that were created or activated by a PDB with the same GUID as the PDB where the keys are being exported. Essentially, all of the keys that belong to a PDB where the export is being performed will be exported.

The importing of master encryption keys from an export file within a PDB takes place only if the master encryption key was exported from another PDB with the same GUID. To support the plug-in of a PDB into a CDB, the import will also import the master encryption keys from an export file that contains the master encryption keys of a non-CDB exported without the WITH IDENTIFIER clause. Because the PDB-specific details, such as the PDB name and database ID, can change from one CDB to the next, the PDB-specific information is modified during the import to reflect the updated PDB information.

Note

Within a PDB, you can only export the keys of a PDB as a whole. The ability to export them selectively based on a query or an identifier is restricted to the root.

8.8.7.2 Exporting or Importing a Master Encryption Key for a PDB in Isolated Mode

In isolated mode, the ADMINISTER KEY MANAGEMENT statement can export or import a master encryption key for a PDB.

1. Log in to the isolated mode PDB as a user who was granted the ADMINISTER KEY MANAGEMENT or SYSKM privilege.
2. Perform the export or import operation.

For example:

```
ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS  
WITH SECRET "hr_secret" TO '/tmp/export.p12'  
FORCE KEYSTORE  
IDENTIFIED BY password;
```

Ensure that you include the FORCE KEYSTORE clause because the keystore must be open for this operation.

Related Topics

- [Example: Exporting a Master Encryption Key from a PDB in Isolated Mode](#)
The ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS statement can export master encryption keys for a PDB.

8.8.7.3 Example: Exporting a Master Encryption Key from a PDB in Isolated Mode

The `ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS` statement can export master encryption keys for a PDB.

[Example 8-1](#) shows how to export a master encryption key from the PDB `hrpdb`. In this example, the `FORCE KEYSTORE` clause is included in case the auto-login keystore is in use, or if the keystore is closed.

Example 8-1 Exporting a Master Encryption Key from a PDB

```
ADMINISTER KEY MANAGEMENT EXPORT ENCRYPTION KEYS  
WITH SECRET "my_secret" TO '/tmp/export.p12'  
FORCE KEYSTORE  
IDENTIFIED BY password;
```

8.8.7.4 Example: Importing a Master Encryption Key into a PDB in Isolated Mode

The `ADMINISTER KEY MANAGEMENT IMPORT ENCRYPTION KEYS` statement can import a master encryption key into a PDB.

[Example 8-2](#) shows how to import a master encryption key into the PDB `hrpdb`.

Example 8-2 Importing a Master Encryption Key into a PDB

```
ADMINISTER KEY MANAGEMENT IMPORT ENCRYPTION KEYS  
WITH SECRET "my_secret"  
FROM '/tmp/export.p12'  
FORCE KEYSTORE  
IDENTIFIED BY password  
WITH BACKUP;
```

9

General Considerations of Using Transparent Data Encryption

When you use Transparent Data Encryption, you should consider factors such as security, performance, and storage overheads.

- [Migrating Encrypted TDE Columns or Tablespaces after a Database Upgrade from Release 11g](#)
If you are upgrading from Oracle Database release 11g, then you must manually convert the typed master encryption keys to united mode master encryption keys.
- [Compression and Data Deduplication of Encrypted Data](#)
When you use Oracle's compression products and options with Transparent Data Encryption (TDE) tablespace encryption, Oracle Database automatically applies compression before performing the encryption.
- [Security Considerations for Transparent Data Encryption](#)
As with all Oracle Database features, you should consider security when you create TDE policies.
- [Performance and Storage Overhead of Transparent Data Encryption](#)
The performance of Transparent Data Encryption can vary.
- [Modifying Your Applications for Use with Transparent Data Encryption](#)
You can modify your applications to use Transparent Data Encryption.
- [How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT](#)
Many of the clauses from the ALTER SYSTEM statement correspond to the ADMINISTER KEY MANAGEMENT statement.
- [Data Loads from External Files to Tables with Encrypted Columns](#)
You can use SQL*Loader to perform data loads from files to tables that have encrypted columns.
- [Transparent Data Encryption and Database Close Operations](#)
You should ensure that the software or external keystore is open before you close the database.

9.1 Migrating Encrypted TDE Columns or Tablespaces after a Database Upgrade from Release 11g

If you are upgrading from Oracle Database release 11g, then you must manually convert the typed master encryption keys to united mode master encryption keys.

You must perform this task for master encryption keys in both column encryption and tablespace encryption configurations.

1. Navigate to directory that has the TDE password-based keystore ewallet.p12.

2. Check the keystore contents and look for typed keys (that is, identifiers starting with ORACLE.SECURITY.TS.ENCRIPTION.B as follows).

```
mkstore -wrl ewallet.p12 -list
Enter wallet password: wallet_password
```

Output similar to the following appears. The entries with **TS** are the typed keys that must be converted.

```
Oracle Secret Store entries:
ORACLE.SECURITY.DB.ENCRIPTION.AYTGHZMNxU9kv77shWptsxkAAAAAAAAAAAAAAAAAAAA
AAAAAA
ORACLE.SECURITY.DB.ENCRIPTION.ARjWMYe05E9Bv4KULq1no8UAAAAAAAAAAAAAAAAAAAA
AAAAAA
ORACLE.SECURITY.DB.ENCRIPTION.ASo02D5jtk/
jv9FPW6rhhGcAAAAAAAAAAAAAAAAAAAAAAAA
ORACLE.SECURITY.DB.ENCRIPTION.MASTERKEY
ORACLE.SECURITY.TS.ENCRIPTION.BW8QuLUTgOpV0PtEt/
1R028CAwAAAAAAAAAAAAAAAAAAAA
ORACLE.SECURITY.TS.ENCRIPTION.BWNFXbAc38AZMdFdi2kuwV0CAwAAAAAAAAAAAAAAAA
AAAAAA
```

3. Back up the keystore.

For example:

```
cp ewallet.p12 ewallet_dddmmmyy.p12
```

4. Create a united key for each typed key that you found in the keystore.

To accomplish this, you must modify the identifier name (Key ID) to have **DB** instead of **TS** and the first letter from **B** to **A**. For this you must add a new entry to the keystore. The following example shows how to convert the first typed key in the list that was shown in the preceding steps.

- a. List the value for identifier to double-check it. The bold text indicates the two changes to be made.

For example:

```
ORACLE.SECURITY.TS.ENCRIPTION.BW8QuLUTgOpV0PtEt/
1R028CAwAAAAAAAAAAAAAAAAAAAA
```

This example must be changed as follows:

```
ORACLE.SECURITY.DB.ENCRIPTION.Aw8QuLUTgOpV0PtEt/
1R028CAwAAAAAAAAAAAAAAAAAAAA
```

- b. Using this updated entry, add a new entry to the keystore.

For example:

```
mkstore -wrl wallet_location -viewEntry
ORACLE.SECURITY.TS.ENCRIPTION.BW8QuLUTgOpV0PtEt/
1R028CAwAAAAAAAAAAAAAAAAAAAA
```

The `-viewEntry` option of this command displays the identifier value for the TS encryption, which you must include in the next command when you create the keystore entry:

```
mkstore -wrl wallet_location -  
createEntry ORACLE.SECURITY.DB.ENCPTION.AW8QuLUTgOpV0PtEt/  
lR028CAwAAAAAAAAAAAAAAAA  
AAAAAAAA value_for_ts_encryption_identifier
```

- c. For an auto-login keystore, rename the old auto-login keystore and then create a new one.
 - i. Back up the master encryption key. For example:

```
mv cwallet.sso cwallet_dddmmmyy.sso
```

- ii. Use the `ADMINISTER KEY MANAGEMENT` command to create the auto-login keystore.

Related Topics

- [Creating an Auto-Login or a Local Auto-Login Software Keystore](#)

As an alternative to password-protected keystores, you can create either an auto-login or local auto-login software keystore.

9.2 Compression and Data Deduplication of Encrypted Data

When you use Oracle's compression products and options with Transparent Data Encryption (TDE) tablespace encryption, Oracle Database automatically applies compression before performing the encryption.

The compression products include the following: Oracle Recovery Manager (Oracle RMAN) compression, Oracle Advanced Compression, Exadata Hybrid Columnar Compression (EHCC), and so on.

This ensures that you receive the maximum space and performance benefits from compression, while also receiving the security of encryption at rest. In the `CREATE TABLESPACE` SQL statement, include both the `COMPRESS` and `ENCRYPT` clauses.

With column encryption, Oracle Database compresses the data after it encrypts the column. This means that compression will have minimal effectiveness on encrypted columns. There is one notable exception: if the column is a SecureFiles LOB, and the encryption is implemented with SecureFiles LOB Encryption, and the compression (and possibly deduplication) are implemented with SecureFiles LOB Compression & Deduplication, then compression is performed before encryption. Similar to the `CREATE TABLESPACE` statement for tablespace encryption, include both the `COMPRESS` and `ENCRYPT` clauses.

Note

The type of compression that you can use depends on the licensing that you have for Advanced Compression. See *Oracle Database Licensing Information User Manual*.

Related Topics

- [Oracle Database Backup and Recovery User's Guide](#)

9.3 Security Considerations for Transparent Data Encryption

As with all Oracle Database features, you should consider security when you create TDE policies.

- [Transparent Data Encryption General Security Advice](#)
Security considerations for Transparent Data Encryption (TDE) operate within the broader area of total system security.
- [Transparent Data Encryption Column Encryption-Specific Advice](#)
Additional security considerations apply to normal database and network operations when using TDE.
- [Managing Security for Plaintext Fragments](#)
You should remove old plaintext fragments that can appear over time.

9.3.1 Transparent Data Encryption General Security Advice

Security considerations for Transparent Data Encryption (TDE) operate within the broader area of total system security.

Follow these general guidelines:

- Identify the degrees of sensitivity of data in your database, the protection that they need, and the levels of risk to be addressed. For example, highly sensitive data requiring stronger protection can be encrypted with the AES256 algorithm. A database that is not as sensitive can be protected with no [salt](#) or the nomac option to enable performance benefits.
- Evaluate the costs and benefits that are acceptable to data and keystore protection. Protection of keys determines the type of keystore to be used: auto-login TDE wallets, password-based TDE wallets, or external keystores.
- Consider having separate security administrators for TDE and for the database.
- Consider having a separate and exclusive keystore for TDE.
- Implement protected back-up procedures for your encrypted data.

9.3.2 Transparent Data Encryption Column Encryption-Specific Advice

Additional security considerations apply to normal database and network operations when using TDE.

Encrypted column data stays encrypted in the data files, undo logs, redo logs, and the buffer cache of the system global area (SGA). However, data is decrypted during expression evaluation, making it possible for decrypted data to appear in the swap file on the disk. Privileged operating system users can potentially view this data.

Column values encrypted using TDE are stored in the data files in encrypted form. However, these data files may still contain some [plaintext](#) fragments, called ghost copies, left over by past data operations on the table. This is similar to finding data on the disk after a file was deleted by the operating system.

9.3.3 Managing Security for Plaintext Fragments

You should remove old plaintext fragments that can appear over time.

Old [plaintext](#) fragments may be present for some time until the database overwrites the blocks containing such values. If privileged operating system users bypass the access controls of the database, then they might be able to directly access these values in the data file holding the tablespace.

To minimize this risk:

1. Create a new tablespace in a new data file.

You can use the `CREATE TABLESPACE` statement to create this tablespace.

2. Move the table containing encrypted columns to the new tablespace. You can use the `ALTER TABLE . . . .MOVE` statement.

Repeat this step for all of the objects in the original tablespace.

3. Drop the original tablespace.

You can use the `DROP TABLESPACE tablespace INCLUDING CONTENTS KEEP DATAFILES` statement. Oracle recommends that you securely delete data files using platform-specific utilities.

4. Use platform-specific and file system-specific utilities to securely delete the old data file. Examples of such utilities include `shred` (on Linux) and `sdelete` (on Windows).

9.4 Performance and Storage Overhead of Transparent Data Encryption

The performance of Transparent Data Encryption can vary.

- [Performance Overhead of Transparent Data Encryption](#)
Transparent Data Encryption tablespace encryption has small associated performance overhead.
- [Storage Overhead of Transparent Data Encryption](#)
TDE tablespace encryption has no storage overhead, but TDE column encryption has some associated storage overhead.

9.4.1 Performance Overhead of Transparent Data Encryption

Transparent Data Encryption tablespace encryption has small associated performance overhead.

The actual performance impact on applications can vary. TDE column encryption affects performance only when data is retrieved from or inserted into an encrypted column. No reduction in performance occurs for operations involving unencrypted columns, even if these columns are in a table containing encrypted columns. Accessing data in encrypted columns involves small performance overhead, and the exact overhead you observe can vary.

The total performance overhead depends on the number of encrypted columns and their frequency of access. The columns most appropriate for encryption are those containing the most sensitive data.

Enabling encryption on an existing table results in a full table update like any other ALTER TABLE operation that modifies table characteristics. Keep in mind the potential performance and redo log impact on the database server before enabling encryption on a large existing table.

A table can temporarily become inaccessible for write operations while encryption is being enabled, [TDE table keys](#) are being rekeyed, or the encryption algorithm is being changed. You can use online table redefinition to ensure that the table is available for write operations during such procedures.

If you enable TDE column encryption on a very large table, then you may need to increase the redo log size to accommodate the operation.

Encrypting an indexed column takes more time than encrypting a column without indexes. If you must encrypt a column that has an index built on it, you can try dropping the index, encrypting the column with NO SALT, and then re-creating the index.

If you index an encrypted column, then the index is created on the encrypted values. When you query for a value in the encrypted column, Oracle Database transparently encrypts the value used in the SQL query. It then performs an index lookup using the encrypted value.

Note

If you must perform range scans over indexed, encrypted columns, then use TDE tablespace encryption in place of TDE column encryption.

Performance Considerations for Cryptographic Providers

Both the legacy provider and the next-generation provider support hardware-accelerated AES encryption using Intel AES-NI and equivalent processor instructions.

Oracle recommends benchmarking workloads after switching cryptographic providers to verify performance meets application requirements.

Related Topics

- [Creating an Encrypted Column in an External Table](#)
The external table feature enables you to access data in external sources as if the data were in a database table.
- *Oracle Database Administrator's Guide*

9.4.2 Storage Overhead of Transparent Data Encryption

TDE tablespace encryption has no storage overhead, but TDE column encryption has some associated storage overhead.

Encrypted column data must have more storage space than [plaintext](#) data. In addition, TDE pads out encrypted values to multiples of 16 bytes. This means that if a credit card number requires nine bytes for storage, then an encrypted credit card value will require an additional seven bytes.

Each encrypted value is also associated with a 20-byte integrity check. This does not apply if you have encrypted columns using the NOMAC parameter. If data was encrypted with [salt](#), then each encrypted value requires an additional 16 bytes of storage.

The maximum storage overhead for each encrypted value is from one to 52 bytes.

Related Topics

- [Creating an Encrypted Column in an External Table](#)
The external table feature enables you to access data in external sources as if the data were in a database table.

9.5 Modifying Your Applications for Use with Transparent Data Encryption

You can modify your applications to use Transparent Data Encryption.

1. Configure the software or external keystore for TDE, and then set the master encryption key.

See the following sections for more information:

- [Configuring a TDE Wallet](#)
- [Configuring an External Keystore](#)

2. Verify that the master encryption key was created by querying the KEY_ID column of the V\$ENCRYPTION_KEYS view.
3. Identify the sensitive columns (such as those containing credit card data) that require Transparent Data Encryption protection.
4. Decide whether to use TDE column encryption or TDE tablespace encryption.

See the following sections for more information:

- [How Transparent Data Encryption Column Encryption Works](#)
- [How Transparent Data Encryption Tablespace Encryption Works](#)

5. Open the keystore.

See the following sections for more information:

- [Step 3: Open the TDE Wallet](#)
- [Step 2: Open the Connection to Oracle Key Vault](#)

6. Encrypt the columns or tablespaces.

See the following sections for more information:

- [Encrypting Columns in Tables](#)
- [Encryption Conversions for Tablespaces and Databases](#)

9.6 How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Many of the clauses from the ALTER SYSTEM statement correspond to the ADMINISTER KEY MANAGEMENT statement.

[Table 10-2](#) compares the Transparent Data Encryption usage of the ALTER SYSTEM statement and the orapki utility from previous releases with the ADMINISTER KEY MANAGEMENT statement.

Table 9-1 How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Behavior	ALTER SYSTEM or orapki	ADMINISTER KEY MANAGEMENT
Creating a keystore	For TDE wallets: <pre>ALTER SYSTEM SET ENCRYPTION KEY ["certificate_ID"] IDENTIFIED BY keystore_password;</pre> For external keystores, the keystore is available after you configure the external keystore.	For TDE wallets: <pre>ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY TDE_wallet_password</pre> For external keystores, the keystore is available after you configure the external keystore.
Creating an auto-login keystore	<pre>orapki wallet create -wallet wallet_location -auto_login [-pwd password]</pre>	For TDE wallets: <pre>ADMINISTER KEY MANAGEMENT CREATE [LOCAL] AUTO_LOGIN KEYSTORE FROM KEYSTORE IDENTIFIED BY TDE_wallet_password;</pre> This type of keystore applies to TDE wallets only.
Opening a keystore	<pre>ALTER SYSTEM SET [ENCRYPTION] WALLET OPEN IDENTIFIED BY keystore_password;</pre>	<pre>ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN [FORCE KEYSTORE] IDENTIFIED BY keystore_password EXTERNAL STORE [CONTAINER = ALL CURRENT];</pre>
Closing a keystore	<pre>ALTER SYSTEM SET [ENCRYPTION] WALLET CLOSE IDENTIFIED BY keystore_password;</pre>	For both software and external keystores: <pre>ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE [IDENTIFIED BY keystore_password] [CONTAINER = ALL CURRENT];</pre>
Migrating from an external keystore to a TDE wallet	<pre>ALTER SYSTEM SET [ENCRYPTION] KEY IDENTIFIED BY wallet_password REVERSE MIGRATE USING "external_key_manager_password" WITH BACKUP;</pre>	<pre>ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY IDENTIFIED BY keystore_password REVERSE MIGRATE USING "external_key_manager_password" WITH BACKUP [USING 'backup_identifier'];</pre>
Migrating from a TDE wallet to Oracle Key Vault or Oracle Cloud Interface (OCI) Key Management Service (KMS)	<pre>ALTER SYSTEM SET [ENCRYPTION] KEY IDENTIFIED BY "Oracle_Key_Vault_password" MIGRATE USING wallet_password;</pre>	<pre>ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY IDENTIFIED BY "Oracle_Key_Vault_password" MIGRATE USING wallet_password;</pre>

Table 9-1 (Cont.) How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Behavior	ALTER SYSTEM or orapki	ADMINISTER KEY MANAGEMENT
Changing a keystore password	orapki wallet change_pwd -wallet <i>wallet_location</i> [-oldpwd <i>password</i>] [-newpwd <i>password</i>]	For password-based TDE wallets: ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD IDENTIFIED BY <i>TDE_wallet_old_password</i> SET <i>TDE_wallet_new_password</i> [WITH BACKUP [USING ' <i>backup_identifier</i> ']]; For external keystores, you close the keystore, change it in the external key manager interface, and then reopen the keystore.
Backing up a password-based TDE wallet	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE [USING ' <i>backup_identifier</i> '] IDENTIFIED BY <i>TDE_wallet_password</i> [TO ' <i>TDE_wallet_location</i> '];
Merging two TDE wallets into a third new keystore	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT MERGE KEYSTORE ' <i>TDE_wallet1_location</i> ' [IDENTIFIED BY <i>TDE_wallet1_password</i>] AND KEYSTORE ' <i>TDE_wallet2_location</i> ' [IDENTIFIED BY <i>TDE_wallet2_password</i>] INTO NEW KEYSTORE ' <i>TDE_wallet3_location</i> ' IDENTIFIED BY <i>TDE_wallet3_password</i> ;
Merging one TDE wallet into another existing keystore	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT MERGE KEYSTORE ' <i>TDE_wallet1_location</i> ' [IDENTIFIED BY <i>TDE_wallet1_password</i>] INTO EXISTNG KEYSTORE ' <i>TDE_wallet2_location</i> ' IDENTIFIED BY <i>TDE_wallet2_password</i> [WITH BACKUP [USING ' <i>backup_identifier</i> ']];
Setting or rekeying the master encryption key	ALTER SYSTEM SET [ENCRYPTION] KEY IDENTIFIED BY <i>keystore_password</i> ; Note: The ALTER SYSTEM SET ENCRYPTION KEY statement does not update the V\$ENCRYPTION_KEYS dynamic view after you rekey the encryption key.	ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY [USING TAG ' <i>tag</i> '] IDENTIFIED BY <i>keystore_password</i> WITH BACKUP [USING ' <i>backup_identifier</i> '] [CONTAINER = ALL CURRENT]; After you rekey the encryption key, the V\$ENCRYPTION_KEYS dynamic view is updated.
Creating a master encryption key for later user	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT CREATE KEY [USING TAG ' <i>tag</i> '] IDENTIFIED BY <i>keystore_password</i> [WITH BACKUP [USING ' <i>backup_identifier</i> ']] [CONTAINER = (ALL CURRENT)];
Activating a master encryption key	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT USE KEY ' <i>key_identifier</i> ' [USING TAG ' <i>tag</i> '] IDENTIFIED BY <i>keystore_password</i> [WITH BACKUP [USING ' <i>backup_identifier</i> ']];

Table 9-1 (Cont.) How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Behavior	ALTER SYSTEM or orapki	ADMINISTER KEY MANAGEMENT
Creating custom tags for master encryption keys	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT SET TAG 'tag' FOR 'master_key_identifier' IDENTIFIED BY keystore_password [WITH BACKUP [USING 'backup_identifier']];
Exporting a master encryption key	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT EXPORT [ENCRYPTION] KEYS WITH SECRET "export_secret" TO 'file_path' IDENTIFIED BY keystore_password [WITH IDENTIFIER IN 'key_id1', 'key_id2', 'key_idn' (SQL_query)]
Importing a master encryption key	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT IMPORT [ENCRYPTION] KEYS WITH SECRET "import_secret" FROM 'file_name' IDENTIFIED BY keystore_password [WITH BACKUP [USING 'backup_identifier']];
Storing Oracle Database secrets in a keystore	No ALTER SYSTEM or orapki equivalent for this functionality	For TDE wallets: ADMINISTER KEY MANAGEMENT ADD SECRET UPDATE SECRET DELETE SECRET "secret" FOR CLIENT 'client_identifier' [USING TAG 'tag'] IDENTIFIED BY keystore_password [WITH BACKUP [USING 'backup_identifier']]; For external keystores: ADMINISTER KEY MANAGEMENT ADD SECRET UPDATE SECRET DELETE SECRET "secret" FOR CLIENT 'client_identifier' [USING TAG 'tag'] IDENTIFIED BY "external_key_manager_password";

Wallet Conversion Before Switching to Next-Generation provider FIPS 140-3 Mode

If you plan to switch the database to the next-generation provider in FIPS 140-3 mode, verify that the keystore file (ewallet.p12) is encrypted with AES-256. Wallets created before Oracle Database 19.22 may use 3DES for the PKCS#12 container encryption, which is not compatible with FIPS 140-3.

Perform this conversion and verify wallet accessibility before enabling FIPS 140-3 mode. If the wallet cannot be opened after enabling FIPS 140-3, the TDE master encryption key is inaccessible and encrypted data cannot be read.

To check the wallet encryption algorithm:

```
openssl pkcs12 -info -in ./ewallet.p12 -nodes
```


Use the following `orapki` command to convert the wallet encryption from 3DES to AES-256:

```
orapki wallet convert -wallet wallet_location -compat_v12
```

When operating in FIPS mode, append the `-fips140_mode` flag:

```
orapki wallet convert -wallet wallet_location -compat_v12 -fips140_mode
```

This command converts the PKCS#12 container encryption to AES-256 while preserving all keys and certificates stored in the wallet. Perform this conversion while the database is still using the legacy provider or the next-generation provider in non-FIPS mode.

The equivalent `ADMINISTER KEY MANAGEMENT` operation is not available for wallet container encryption conversion. You must use the `orapki` utility for this operation.

9.7 Data Loads from External Files to Tables with Encrypted Columns

You can use SQL*Loader to perform data loads from files to tables that have encrypted columns.

Be aware that with SQL*Loader, you cannot include the `ENCRYPT` clause in the column definition of an external table of the type `ORACLE_LOADER`, but you can include it in the column definitions of external tables of type `ORACLE_DATAPUMP`.

- External tables of type `ORACLE_LOADER`

The reason that you cannot include the `ENCRYPT` clause in the column definitions of external tables of the type `ORACLE_LOADER` is because the contents of an external table with the `ORACLE_LOADER` type must come from a user-specified plaintext "backing file," and such plaintext files cannot contain any TDE encrypted data.

If you use the `ENCRYPT` clause in the definition of an external table of type `ORACLE_LOADER`, then when you query the TDE-encrypted column in this external table, the query fails. This is because TDE expects the external data to have been encrypted, and automatically tries to decrypt it on load. This action fails because the "backing file" only contains plaintext.

- External tables of type `ORACLE_DATAPUMP`

You can use TDE column encryption with external tables of type `ORACLE_DATAPUMP`. This is because for external tables of `ORACLE_DATAPUMP` type, the "backing file" is always created by Oracle Database (during an unload operation) and thus does have support for being populated with encrypted data.

9.8 Transparent Data Encryption and Database Close Operations

You should ensure that the software or external keystore is open before you close the database.

The master encryption keys may be required during the database close operation. The database close operation automatically closes the software or external keystore.

Related Topics

- [Step 3: Open the TDE Wallet](#)
Depending on the type of keystore you create, you must manually open the TDE wallet before you can use it.
- [Step 2: Open the Connection to Oracle Key Vault](#)
After you have configured the database to use Oracle Key Vault for TDE key management, you must open the connection to Oracle Key Vault before you can use it.

10

Using Transparent Data Encryption with Other Oracle Features

You can use Oracle Data Encryption with other Oracle features, such as Oracle Data Guard or Oracle Real Application Clusters.

- [How Transparent Data Encryption Works with Export and Import Operations](#)
Oracle Data Pump can export and import tables that contain encrypted columns, as well as encrypt entire dump sets.
- [How Transparent Data Encryption Works with Oracle Data Guard](#)
An Oracle Data Guard primary database and secondary secondary database can share both a TDE wallet and an external keystore.
- [How Transparent Data Encryption Works with Oracle Real Application Clusters](#)
Oracle Real Application Clusters (Oracle RAC) nodes can share both a TDE wallets and an external keystore.
- [How Transparent Data Encryption Works with SecureFiles](#)
SecureFiles, which stores LOBS, has three features: compression, deduplication, and encryption.
- [How Transparent Data Encryption Works with Oracle Call Interface](#)
Transparent Data Encryption does not have any effect on the operation of Oracle Call Interface (OCI).
- [How Transparent Data Encryption Works with Editions](#)
Transparent Data Encryption does not have any effect on the Editions feature of Oracle Database.
- [Configuring Transparent Data Encryption to Work in a Multidatabase Environment](#)
Each Oracle database on the same server (such as databases sharing the same Oracle binary but using different data files) must access its own TDE keystore.
- [RMAN Encryption Algorithm Configuration](#)
RMAN backup encryption on Oracle Database 19c supports AES128, AES192, and AES256 in CFB mode. The default algorithm is AES128. Oracle recommends AES256.

10.1 How Transparent Data Encryption Works with Export and Import Operations

Oracle Data Pump can export and import tables that contain encrypted columns, as well as encrypt entire dump sets.

- [About Exporting and Importing Encrypted Data](#)
You can use Oracle Data Pump to export and import tables that have encrypted columns.
- [Exporting and Importing Tables with Encrypted Columns](#)
You can export and import tables with encrypted columns using the `ENCRYPTION=ENCRYPTED_COLUMNS_ONLY` setting.

- [Using Oracle Data Pump to Encrypt Entire Dump Sets](#)
Oracle Data Pump can encrypt entire dump sets, not just Transparent Data Encryption columns.
- [Using Oracle Data Pump with Encrypted Data Dictionary Data](#)
Oracle Data Pump operations provide protections for encrypted passwords and other encrypted data.
- [Data Pump Encryption Algorithm Configuration](#)
The encryption algorithm for Data Pump exports is controlled by the `ENCRYPTION_ALGORITHM` parameter.

10.1.1 About Exporting and Importing Encrypted Data

You can use Oracle Data Pump to export and import tables that have encrypted columns.

For both software and external keystores, the following points are important when you must export tables containing encrypted columns:

- Sensitive data should remain unintelligible during transport.
- Authorized users should be able to decrypt the data after it is imported at the destination.

When you use Oracle Data Pump to export and import tables containing encrypted columns, it uses the `ENCRYPTION` parameter to enable encryption of data in dump file sets. The `ENCRYPTION` parameter allows the following values:

- `ENCRYPTED_COLUMNS_ONLY`: Writes encrypted columns to the dump file set in encrypted format
- `DATA_ONLY`: Writes all of the data to the dump file set in encrypted format
- `METADATA_ONLY`: Writes all of the metadata to the dump file set in encrypted format
- `ALL`: Writes all of the data and metadata to the dump file set in encrypted format
- `NONE`: Does not use encryption for dump file sets

10.1.2 Exporting and Importing Tables with Encrypted Columns

You can export and import tables with encrypted columns using the `ENCRYPTION=ENCRYPTED_COLUMNS_ONLY` setting.

1. Ensure that the keystore is open before you attempt to export tables containing encrypted columns.

In a multitenant environment, if you are exporting data in a pluggable database (PDB), then ensure that the wallet is open in the PDB. If you are exporting into the root, then ensure that the wallet is open in the root.

To find if the keystore is open, query the `STATUS` column of the `V$ENCRYPTION_WALLET` view. If you must open the keystore, then run the following SQL statement:

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
IDENTIFIED BY software_keystore_password
[CONTAINER = ALL | CURRENT];
```

The `software_keystore_password` setting is the password for the keystore. The keystore must be open because the encrypted columns must be decrypted using the [TDE table keys](#), which requires access to the TDE master encryption key. The columns are reencrypted using a password, before they are exported.

2. Run the EXPDP command, using the ENCRYPTION_PASSWORD parameter to specify a password that is used to encrypt column data in the export dump file set.

The following example exports the employee_data table. The ENCRYPTION_PWD_PROMPT = YES setting enables you to prompt for the password interactively, which is a recommended security practice.

```
expdp hr TABLES=employee_data DIRECTORY=dpump_dir
DUMPFILE=dpdcd2be1.dmp ENCRYPTION=ENCRYPTED_COLUMNS_ONLY
ENCRYPTION_PWD_PROMPT = YES
```

Password: *password_for_hr*

3. To import the exported data into the target database, ensure that you specify the same password that you used for the export operation, as set by the ENCRYPTION_PASSWORD parameter.

The password is used to decrypt the data. Data is reencrypted with the new TDE table keys generated in the target database. The target database must have the keystore open to access the TDE master encryption key. The following example imports the employee_data table:

```
impdp hr TABLES=employee_data DIRECTORY=dpump_dir
DUMPFILE=dpdcd2be1.dmp
ENCRYPTION_PWD_PROMPT = YES
```

Password: *password_for_hr*

10.1.3 Using Oracle Data Pump to Encrypt Entire Dump Sets

Oracle Data Pump can encrypt entire dump sets, not just Transparent Data Encryption columns.

While importing, you can use either the password or the keystore TDE master encryption key to decrypt the data. If the password is not supplied, then the TDE master encryption key in the keystore is used to decrypt the data. The keystore must be present and open at the target database. The open keystore is also required to reencrypt column encryption data at the target database.

You can use the ENCRYPTION_MODE=TRANSPARENT setting to transparently encrypt the dump file set with the TDE master encryption key stored in the keystore. A password is not required in this case. The keystore must be present and open at the target database, and it must *contain* the TDE master encryption key from the *source* database for a successful decryption of column encryption metadata during an import operation.

The open keystore is also required to reencrypt column encryption metadata at the target database. If a keystore already exists on the target database, then you can export the current TDE master encryption key *from* the keystore of the source database and import it *into* the keystore of the target database.

- Use the ENCRYPTION_MODE parameter to specify the encryption mode. ENCRYPTION_MODE=DUAL encrypts the dump set using the TDE master encryption key stored in the keystore and the password provided.

For example, to use dual encryption mode to export encrypted data:

```
expdp hr DIRECTORY=dpump_dir1
DUMPFILE=hr_enc.dmp
ENCRYPTION=all
ENCRYPTION_PASSWORD=encryption_password
ENCRYPTION_PWD_PROMPT=yes
```

```
ENCRYPTION_ALGORITHM=AES256  
ENCRYPTION_MODE=dual
```

```
Password: password_for_hr  
Encryption Password: password_for_encryption
```

Related Topics

- [Exporting and Importing the TDE Master Encryption Key](#)
You can export and import the TDE master encryption key in different ways.
- [Oracle Database Utilities](#)
- [Creating an Encrypted Column in an External Table](#)
The external table feature enables you to access data in external sources as if the data were in a database table.

10.1.4 Using Oracle Data Pump with Encrypted Data Dictionary Data

Oracle Data Pump operations provide protections for encrypted passwords and other encrypted data.

When you enable the encryption of fixed-user database passwords in a source database, then an Oracle Data Pump export operation dump stores a known invalid password for the database link password. This password is in place instead of the encrypted password that the export operation extracts from the database. An ORA-39395: Warning: object <database link name> requires password reset after import warning message is displayed as a result. If you import data into an Oracle Database 18c or later database, then this same warning appears when the database link object with its invalid password is created in the target database. When this happens, you must reset the database link password, as follows:

```
ALTER DATABASE LINK database_link_name CONNECT TO schema_name IDENTIFIED BY password;
```

To find information about the database link, you can query the V\$DBLINK dynamic view.

When the encryption of fixed-user database passwords has been disabled in a source database, then there are no changes to Data Pump. The obfuscated database link passwords are exported and imported as in previous releases.

In this case, Oracle recommends the following:

- Set the ENCRYPTION_PASSWORD parameter on the expdp command so that you can further protect the obfuscated database link passwords.
- Set the ENCRYPTION_PWD_PROMPT parameter to YES so that the password can be entered interactively from a prompt, instead of being echoed on the screen.

Both the ENCRYPTION_PASSWORD and the ENCRYPTION_PWD_PROMPT parameters are available in import operations. ENCRYPTION_PWD_PROMPT is only available with the expdp and impdp command-line clients, whereas ENCRYPTION_PASSWORD is available in both the command-line clients and the DBMS_DATAPUMP PL/SQL package.

During an import operation, whether the keystore is open or closed affects the behavior of whether or not an encryption password must be provided. If the keystore was open during the export operation and you provided an encryption password, then you do not need to provide the password during the import operation. If the keystore is closed during the export operation, then you must provide the password during the import operation.

Related Topics

- [Oracle Database Reference](#)

- *Oracle Database Utilities*

10.1.5 Data Pump Encryption Algorithm Configuration

The encryption algorithm for Data Pump exports is controlled by the `ENCRYPTION_ALGORITHM` parameter.

The following algorithms are available for Oracle Database 19c:

Table 10-1 Data Pump Encryption Algorithms

Algorithm	Default	Recommended
AES128	Yes	No
AES192	No	No
AES256	No	Yes

Oracle recommends specifying `ENCRYPTION_ALGORITHM=AES256` for Data Pump exports:

```
expdp hr DIRECTORY=dpump_dir DUMPFILE=hr_enc.dmp ENCRYPTION=ALL  
ENCRYPTION_ALGORITHM=AES256
```

The Data Pump encryption algorithms are the same regardless of whether the database uses the legacy provider or the next-generation provider. 3DES is not available for Data Pump encryption on Oracle Database 19c.

10.2 How Transparent Data Encryption Works with Oracle Data Guard

An Oracle Data Guard primary database and secondary secondary database can share both a TDE wallet and an external keystore.

- [About Using Transparent Data Encryption with Oracle Data Guard](#)
For both TDE wallets and external keystores, Oracle Data Guard supports Transparent Data Encryption (TDE).
- [Encryption of Tablespaces in an Oracle Data Guard Environment](#)
You can control tablespace encryption in the primary and standby databases in an Oracle Data Guard environment.
- [Configuring TDE and Oracle Key Vault in an Oracle Data Guard Environment](#)
You can configure Oracle Data Guard in a multitenant environment so that it can work with TDE wallets and Oracle Key Vault.
- [Configuring Wallet-Based Transparent Data Encryption in Oracle Data Guard](#)
You can configure wallet-based Transparent Data Encryption (TDE) in an Oracle Data Guard environment.
- [Migrating a TDE Wallet in an Oracle Data Guard Environment to Oracle Key Vault](#)
After you have configured TDE wallet-based Transparent Data Encryption (TDE) in an Oracle Data Guard environment, you can migrate primary and standby databases to Oracle Key Vault, without downtime.

- [Isolating an Encrypted PDB in an Oracle Data Guard Environment](#)
To isolate PDB's in an Oracle Data Guard environment, you must manually perform the configuration.

10.2.1 About Using Transparent Data Encryption with Oracle Data Guard

For both TDE wallets and external keystores, Oracle Data Guard supports Transparent Data Encryption (TDE).

If the primary database uses TDE, then each standby database in a Data Guard configuration must have a copy of the encryption keystore from the primary database. If you reset the TDE master encryption key in the primary database, then you must copy the keystore from the primary database that contains the TDE master encryption key to each standby database.

Note the following:

- Re-key operations with wallet-based TDE will cause the Managed Recovery Process (MRP) on the standby databases to fail because the new TDE master encryption key is not yet available. In order to circumvent this problem, you can split the `set key` command into the `create key` and `use key` commands. After each command, copy the wallets to the standby database and 'force open' the keystore.
- Encrypted data in log files remains encrypted when data is transferred to the standby database. Encrypted data also stays encrypted during transit.

Related Topics

- [Merging TDE Wallets](#)
You can merge TDE wallets in a variety of ways.
- [Configuring Wallet-Based Transparent Data Encryption in Oracle Data Guard](#)
You can configure wallet-based Transparent Data Encryption (TDE) in an Oracle Data Guard environment.

10.2.2 Encryption of Tablespaces in an Oracle Data Guard Environment

You can control tablespace encryption in the primary and standby databases in an Oracle Data Guard environment.

- [About the Encryption of Tablespace in an Oracle Data Guard Environment](#)
In an Oracle Data Guard environment, you can control the automatic encryption of tablespaces in both the primary and standby databases, for on-premises and Oracle Cloud Infrastructure (OCI) environments.
- [Configuring the Encryption of Tablespaces in an Oracle Data Guard Environment](#)
To configure the hybrid encryption of tablespaces, you must set the `TABLESPACE_ENCRYPTION` initialization parameter.

10.2.2.1 About the Encryption of Tablespace in an Oracle Data Guard Environment

In an Oracle Data Guard environment, you can control the automatic encryption of tablespaces in both the primary and standby databases, for on-premises and Oracle Cloud Infrastructure (OCI) environments.

To control the encryption of new tablespaces, you set the `TABLESPACE_ENCRYPTION` initialization parameter.

Oracle recommends that you encrypt primary databases. However, because encryption requirements may vary depending on the site, you can use the `TABLESPACE_ENCRYPTION` parameter to configure a mixed encryption environment for on-premises and in-Cloud environments.

Note the following about using the `TABLESPACE_ENCRYPTION` parameter:

- Redo decryption takes place at the redo transport level.
- For Oracle Database 19.16 and later, if you do not have an Advanced Security Option (ASO) license, which includes Transparent Data Encryption, then you can configure an un-encrypted on-premises primary database to have an encrypted standby database in Oracle Cloud Infrastructure (OCI). Even after a role transition (planned or unplanned), the new on-premises standby database remains un-encrypted, while the primary database in OCI is encrypted. See the Oracle Database Product Management YouTube video [Hybrid Oracle Data Guard without Transparent Data Encryption \(TDE\) License](#). For releases earlier than 19.16, "split-TDE" deployments are complicated to implement, require constant manual intervention on primary and standby databases, and require an ASO license for the un-encrypted on-premises database.
- If the `ENCRYPT_NEW_TABLESPACES` setting that you choose conflicts with the `TABLESPACE_ENCRYPTION` setting, then `TABLESPACE_ENCRYPTION` takes precedence.
- You must set `TABLESPACE_ENCRYPTION` in the CDB root, not in any PDBs.
- The default `TABLESPACE_ENCRYPTION` setting for OCI databases is `AUTO_ENABLE`. The setting is mandatory, and any changes to it are ignored.
- The default `TABLESPACE_ENCRYPTION` setting for on-premises databases is `MANUAL_ENABLE`.

In an Oracle Data Guard environment that uses on-premises databases and Oracle Database in the Cloud (ODBC) or Oracle Exadata Cloud (ExaCS), you can configure tablespace encryption in either of the following scenarios:

- **Encrypt the tablespace in the Cloud standby database but not in the on-premises primary database:** When an unencrypted on-premises primary database creates an unencrypted tablespace, adds a data file, or updates a table, then the redo is not encrypted. However, when the encrypted Cloud standby database applies the redo, it ensures that the tablespace or data file is created, or that the block is updated, and are all encrypted in the Cloud. After the switchover operation, if the encrypted Cloud primary database adds an implicitly encrypted tablespace or data file, or updates a table, then the tablespace is encrypted. The unencrypted on-premises standby database must decrypt the redo, create an unencrypted tablespace or data file, and then ensure that the block is not encrypted on-premises.
- **Encrypt the tablespace in the Cloud primary database but not in the on-premises standby database:** When an encrypted Cloud primary database creates an implicitly encrypted tablespace, adds a data file, or updates a table, then the redo is also encrypted. The unencrypted on-premises standby database must decrypt the redo and ensure that the tablespace and data file are created, or the updated files are all unencrypted. After the switchover operation, if the unencrypted on-premises primary database adds an unencrypted tablespace or data file, or updates a table, then the redo is not encrypted. The encrypted Cloud standby database adds an encrypted tablespace or data file, and then ensures that the block is encrypted in the Cloud.

For example, if you want to use `TABLESPACE_ENCRYPTION` in a configuration that followed the best practice of having both on-premises and OCI databases encrypted, then you would set `TABLESPACE_ENCRYPTION` to `AUTO_ENABLE` for both the on-premises and OCI databases. Alternatively, if the on-premises database is not encrypted in a hybrid disaster recovery configuration with ODBC or Oracle ExaCS, for example, you could set `TABLESPACE_ENCRYPTION` to `DECRYPT_ONLY`. The OCI database is set to `AUTO_ENABLE` by default.

See also the video [Hybrid Oracle Data Guard without Transparent Data Encryption \(TDE\) License](#).

10.2.2.2 Configuring the Encryption of Tablespaces in an Oracle Data Guard Environment

To configure the hybrid encryption of tablespaces, you must set the `TABLESPACE_ENCRYPTION` initialization parameter.

1. Log in to the primary or the standby database instance by using SQL*Plus with the SYSDBA administrative privilege.

In a multitenant environment, this command logs you in to the CDB root. You cannot set this parameter in a pluggable database (PDB).

2. Set the `TABLESPACE_ENCRYPTION` initialization parameter as follows:

```
ALTER SYSTEM SET TABLESPACE_ENCRYPTION = 'value' SCOPE = SPFILE SID = '*';
```

In this specification, replace *value* with one of the following settings:

- `AUTO_ENABLE` encrypts all new tablespaces if the database is licensed for Oracle Advanced Security. This is the default setting for Cloud databases. Note the following:
 - If an existing tablespace is not encrypted, then the database writes a warning to the alert log.
 - Encrypted tablespaces cannot be converted to unencrypted tablespaces. For Oracle Database 19.16, in all Oracle Cloud Infrastructure (OCI) databases (including BaseDB, ExaDB-D, ExaDB-D@Azure, ExaDB-C@C, and ADB-C@C), encrypted tablespaces cannot be converted to unencrypted tablespaces.
 - Because all tablespaces must be encrypted in the Cloud, setting this parameter to `DECRYPT_ONLY` or `MANUAL_ENABLE` on a Cloud database will be ignored and the Cloud database will behave as if the setting is `AUTO_ENABLE`.
 - In the primary database, this setting encrypts the new tablespace with an encryption key.
 - In a standby database, this setting adds a key to the new tablespace and encrypts all blocks.
- `DECRYPT_ONLY` prevents new tablespaces from being encrypted. Use this setting if you do not have a TDE license for your on-premises primary database that you want to protect with an encrypted standby database in OCI. See the Hybrid Oracle Data Guard without Transparent Data Encryption (TDE) License video. This setting is designed for sites that do not have the Advanced Security Option. Note the following:
 - If an existing tablespace is encrypted, then the database writes a warning to the alert log.
 - Unencrypted tablespaces cannot be changed to encrypted tablespaces.
- `MANUAL_ENABLE` enables you to selectively encrypt tablespaces if the database is licensed for Oracle Advanced Security. This is the default for both on-premises primary and standby databases and it uses the same behavior as in previous Oracle Database releases.

In an Oracle Real Application Clusters (Oracle RAC) environment, set `TABSPACE_ENCRYPTION` to the same value for all instances of the primary database, and for all instances of the standby database. Because the default value is `MANUAL_ENABLE`, Oracle recommends that during an upgrade to the current release of Oracle Database, until all database instances are rolled over and upgraded, to not change `TABSPACE_ENCRYPTION` for any of these database instances. When all the database instances are upgraded, then you can modify the `TABSPACE_ENCRYPTION` parameter.

3. Depending on how you set `TABSPACE_ENCRYPTION`, do the following:

- `TABSPACE_ENCRYPTION=AUTO_ENABLE`:
 - Set the master encryption key on the primary database for the root, if you have not done so already.
 - Set master encryption keys on all the PDBs associated with this root, if you have not done so already.
 - Copy the wallet from the primary database to the standby database.
- `TABSPACE_ENCRYPTION=DECRYPT`: Set the master encryption keys on the primary database for the root and all the PDBs, and then copy the wallet to the standby database before creating any new tablespaces.

When both databases are on-premises, you can have primary and standby settings in pairings of the `MANUAL_ENABLE` and `DECRYPT_ONLY` settings. For example, you could set `MANUAL_ENABLE` on the standby database and `DECRYPT_ONLY` on the primary database. In a `DECRYPT_ONLY` (primary) and `MANUAL_ENABLE` (standby) pairing, you must manually encrypt the database on the standby by using TDE offline conversion.

Note the following with regard to rekey operations:

- Modifying the `TABSPACE_ENCRYPTION` parameter does not affect master key rotation operations.
When a tablespace key rotation is performed on the primary database, then the standby database will attempt to rotate the key for the tablespace as well. However, if the standby tablespace is unencrypted and does not have a key, then it will generate an error because there is no key to regenerate. If the standby tablespace is unencrypted but it has inherited a key from primary because of the `DECRYPT_ONLY` setting, then the key will be rotated. In either case, it does not affect the unencrypted tablespace.
- Both the master encryption key and the tablespace key rotation can only be performed on the primary database.

Related Topics

- [Encryption Conversions for Existing Offline Tablespaces](#)
You can perform offline encryption conversions by using the `ALTER TABLESPACE SQL` statement `OFFLINE`, `ENCRYPT`, and `DECRYPT` clauses.
- [Configuring a Software Keystore for Use in United Mode](#)
In united mode, the software keystore resides in the CDB root but the master keys from this keystore are available for the PDBs that have their keystore in united mode.
- [Configuring a TDE Wallet and TDE Master Encryption Key in Isolated Mode](#)
In isolated mode, the TDE wallet is associated with a PDB.

10.2.3 Configuring TDE and Oracle Key Vault in an Oracle Data Guard Environment

You can configure Oracle Data Guard in a multitenant environment so that it can work with TDE wallets and Oracle Key Vault.

The following scenario shows the configuration with Oracle Key Vault in a single-instance, multitenant Oracle Data Guard environment with one physical standby database. The version for the primary and standby databases must be release 19.6 or later. To complete this procedure, you must perform each step in the sequence shown. After you complete the procedure, Oracle Data Guard will use Oracle Key Vault for TDE key management exclusively, and there will be no TDE wallet on your database servers. Oracle recommends that you monitor the alert logs of both primary and standby databases.

1. On both the primary and standby databases, execute the `opatch lspatches` command to check the patch release.

```
$ORACLE_HOME/OPatch/opatch lspatches
```

Output similar to the following appears:

```
31281355;Database Release Update : 19.7.0.0.200414 (30869156)  
29585399;OCW RELEASE UPDATE 19.3.0.0.0 (29585399)
```

2. Download the Oracle Key Vault deployment script that the Oracle Key Vault administrators prepared to enable database administrators to automatically register their Oracle databases with Oracle Key Vault.

Oracle Key Vault RESTful Services Administrator's Guide has an example of how to create a script to automatically enroll Oracle databases as endpoints. The deployment scripts reside on a shared file system from which database administrators can download. There are two different versions of these deployment scripts. The `primary.zip` file is for the primary database, and the `secondary.zip` file is for all standby databases. You can use these scripts for an Oracle Data Guard or an Oracle RAC environment.

Another component that the Oracle Key Vault administrators prepare and add to the deployment script is a configuration file that contains all details for the deployment scripts to connect to Oracle Key Vault.

3. Copy the two deployment scripts (`primary.zip` and `secondary.zip`) that an Oracle Key Vault administrator created for database administrators to download from a shared location.

- a. Copy the `primary.zip` file to the primary database.

```
$ scp user@ip_address:/path/to/file/primary.zip .
```

- b. Copy the `secondary.zip` file to the standby database.

```
$ scp user@ip_address:/path/to/file/secondary.zip .
```

4. On their respective servers, extract the zip files.

```
$ unzip primary.zip
```

```
$ unzip secondary.zip
```

5. Execute the `primary-run-me.sh` and `secondary-run-me.sh` scripts, which contain the commands for the RESTful API to execute in Oracle Key Vault.

The Oracle Key Vault RESTful services will execute these commands in order to register this database in Oracle Key Vault with unique wallet and endpoint names.

- a. **Primary database:** For example:

```
$ more primary-run-me.sh

#!/bin/bash
export EP_NAME=${ORACLE_SID^^}_on_$(hostname -s)
export WALLET_NAME=${ORACLE_SID^^}
curl -Ok --tlsv1.2 https://Oracle_Key_Vault_IP_address:5695/
okvrestclipackage.zip
unzip -oj okvrestclipackage.zip lib/okvrestcli.jar -d ./lib
cat > /home/oracle/deploy-primary.sh << EOF
#!/bin/bash
mkdir -pv ${ORACLE_BASE}/product/okv
okv manage-access wallet create --wallet ${WALLET_NAME} --unique FALSE
okv admin endpoint create --endpoint ${EP_NAME} --description "$(hostname -f), $
(hostname -i)"
    --type ORACLE_DB --platform LINUX64 --subgroup "USE CREATOR SUBGROUP" --
unique FALSE --strict-ip-check TRUE
okv manage-access wallet set-default --wallet ${WALLET_NAME} --endpoint $
{EP_NAME}
expect << _EOF
    set timeout 120
    spawn okv admin endpoint provision --endpoint ${EP_NAME} --location $
{ORACLE_BASE}/product/okv --auto-login FALSE
    expect "Enter Oracle Key Vault endpoint password: "
    send "change-on-install\r"
    expect eof
_EOF
EOF
```

- b. **Standby database:** For example:

```
$ more secondary-run-me.sh

$ more run-me.sh
#!/bin/bash
export EP_NAME=${ORACLE_SID^^}_on_$(hostname -s)
export WALLET_NAME=${ORACLE_SID^^}
curl -Ok --tlsv1.2 https://Oracle_Key_Vault_IP_address:5695/okvrestclipackage.zip
unzip -oj okvrestclipackage.zip lib/okvrestcli.jar -d ./lib
cat > /home/oracle/deploy-standby.sh << EOF
#!/bin/bash
okv admin endpoint create --endpoint ${EP_NAME} --description "$(hostname -f) $
(hostname -i)" --subgroup "USE CREATOR SUBGROUP" --unique FALSE
okv admin endpoint update --endpoint ${EP_NAME} --strict-ip-check TRUE
okv manage-access wallet set-default --wallet ${WALLET_NAME} --endpoint $
{EP_NAME}
expect << _EOF
    set timeout 120
    spawn okv admin endpoint provision --endpoint ${EP_NAME} --location /etc/
ORACLE/KEYSTORES/${ORACLE_UNQNAME^^}/okv --auto-login FALSE
    expect "Enter Oracle Key Vault endpoint password: "
    send "change-on-install\r"
    expect eof
_EOF
EOF
```

6. Create the following directories on the primary database and the standby database.

For example:

```
$ mkdir -pv /u01/opt/oracle/product/okv
$ mkdir -pv /u01/opt/oracle/product/tde
$ mkdir -pv /u01/opt/oracle/product/tde_seps
```

In this specification:

- The /u01/opt/oracle/product directory will be defined as WALLET_ROOT in a later step.
 - /u01/opt/oracle/product/okv is the installation directory for the Oracle Key Vault client software. Depending on how the TDE_CONFIGURATION parameter is set, the Oracle Database will look for the Oracle Key Vault client software in *wallet_root/okv*.
 - /u01/opt/oracle/product/tde will store an auto-login wallet, which only contains the future Oracle Key Vault password, enabling an auto-login Oracle Key Vault configuration. Depending on how TDE_CONFIGURATION is set, the Oracle Database will look for the TDE wallet or an auto-open wallet for Oracle Key Vault, in *wallet_root/tde*.
 - /u01/opt/oracle/product/tde_seps will store an auto-login wallet, which only contains the future Oracle Key Vault password. This will hide the Oracle Key Vault password from the SQL*Plus command line and potentially from the database administrator to enforce separation of duties between Oracle database administrators and Oracle Key Vault administrators.
7. Execute the RESTful API on the primary database first, because the deployment script on the standby databases depends on the presence of the shared virtual wallet in Oracle Key Vault that the script on the primary database creates.
 - Primary database:


```
$ ./deploy-primary.sh
```
 - Standby databases:


```
$ ./deploy-standby.sh
```

8. On the primary and standby databases, execute the root.sh script to deploy the PKCS#11 library.

```
# /u01/opt/oracle/product/okv/bin/root.sh
```

The following output should appear:

```
Creating directory: /opt/oracle/extapi/64/hsm/oracle/1.0.0/
Copying PKCS library to /opt/oracle/extapi/64/hsm/oracle/1.0.0/
Setting PKCS library file permissions
```

9. Execute the `okvutil changepwd` command to change the password for the wallet that you installed, starting from the primary database and then to the standby.

Because all database administrators downloaded the same deployment script, all databases have the same password into Oracle Key Vault. This step enables each database to have a unique password.

```
$ /u01/opt/oracle/product/okv/bin/okvutil changepwd -t wallet -l /u01/opt/oracle/
product/okv/ssl/
```

```
Enter wallet password: default_password
Enter new wallet password: Oracle_Key_Vault_password
Confirm new wallet password: Oracle_Key_Vault_password
Wallet password changed successfully
```

10. On the primary and standby databases, execute the following statements.

- a. Execute the following statement to add the Oracle Key Vault password as a secret into an auto-open wallet to replace the Oracle Key Vault password in the SQL*Plus command line with EXTERNAL_STORE.

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'Oracle_Key_Vault_password'
FOR CLIENT 'OKV_PASSWORD'
TO LOCAL AUTO_LOGIN KEYSTORE '/u01/opt/oracle/product/tde_seps';
```

- b. Execute the following statement to add the Oracle Key Vault password as a secret into an auto-open wallet to enable auto-open Oracle Key Vault.

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'Oracle_Key_Vault_password'
FOR CLIENT 'OKV_PASSWORD'
TO LOCAL AUTO_LOGIN KEYSTORE '/u01/opt/oracle/product/tde';
```

If you are using a release earlier than Oracle Database release 19.13, then use `HSM_PASSWORD`, not `OKV_PASSWORD`.

- c. Optionally, if patch 30398099 is installed, then change the database default algorithm from AES128 to either AES192 or AES256.

Patch 30398099 has been integrated into the April 2021 RU for Oracle Database release 19c (19.11), so that this functionality is available without extra patches. Note that the following parameter is preceded by an underscore.

```
ALTER SYSTEM SET "_tablespace_encryption_default_algorithm" = 'AES256' SCOPE =
BOTH;
```

- d. Configure the primary and standby databases to always encrypt new tablespaces, depending on which release of Oracle Database 19c that you are using:

- For Oracle Database 19.16 and later, run the following ALTER SYSTEM statement:

```
ALTER SYSTEM SET TABLESPACE_ENCRYPTION = 'value' SCOPE = SPFILE SID
= '*';
```

- For Oracle Database 19.15 and earlier, run this ALTER SYSTEM statement:

```
ALTER SYSTEM SET ENCRYPT_NEW_TABLESPACES = ALWAYS SCOPE = BOTH SID
= '*';
```

- e. In the primary and standby databases, define the `WALLET_ROOT` static initialization parameter:

```
ALTER SYSTEM SET WALLET_ROOT = '/u01/opt/oracle/product' SCOPE = SPFILE;
```

- f. Restart the primary and standby databases so that the preceding ALTER SYSTEM SET `WALLET_ROOT` and ALTER SYSTEM SET `TABLESPACE_ENCRYPTION` statements take effect.

- g. After the database restarts, configure TDE to use Oracle Key Vault as the first keystore and the auto-open TDE wallet in `WALLET_ROOT/tde` as the secondary TDE wallet.

Execute the following statement in both the primary and standby databases:

```
ALTER SYSTEM SET TDE_CONFIGURATION = "KESTORE_CONFIGURATION=OKV|FILE" SCOPE =
BOTH;
```

11. In the primary database, create your first TDE master encryption keys in Oracle Key Vault.

Check the `alert.log` of the standby database. The managed recovery process (MRP) should not be stopped, since the standby database finds the correct master key in the shared virtual wallet in Oracle Key Vault.

- a. **Primary root container:** Set the first master encryption key.

For all ADMINISTER KEY MANAGEMENT statements that do not change the TDE configuration, the password will be replaced by EXTERNAL STORE. This enables separation of duties between the database administrators and the Oracle Key Vault administrators because the Oracle Key Vault administrators do not need to share the Oracle Key Vault password with the database administrators.

```
sqlplus sys as syskm
Enter password: password
```

```
ADMINISTER KEY MANAGEMENT SET KEY FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE
CONTAINER = CURRENT;
```

- b. **All primary PDBs:** Set the first, tagged, master key for each open PDB. The benefit of tagging the PDB keys is that they can later be easily identified to belong to a certain PDB.

```
SELECT ' ADMINISTER KEY MANAGEMENT SET KEY
USING TAG '''||UPPER(SYS_CONTEXT('USERENV','CON_NAME'))||' '||TO_CHAR
(SYS_EXTRACT_UTC (SYSTIMESTAMP),
'YYYY-MM-DD HH24:MI:SS"Z"')||''' FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE;'
AS "SET KEY COMMAND" FROM DUAL;
```

- c. Execute the generated output of this SELECT statement.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
USING TAG 'pdb_name date time'
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE;
```

12. Perform the following steps in the root container.

- a. Encrypt the USERS, SYSTEM, and SYSAUX tablespaces in CDB\$ROOT.

For example:

```
ALTER TABLESPACE USERS ENCRYPTION ONLINE [USING 'AES256'] ENCRYPT;
ALTER TABLESPACE SYSAUX ENCRYPTION ONLINE [USING 'AES256'] ENCRYPT;
ALTER TABLESPACE SYSTEM ENCRYPTION ONLINE ENCRYPT;
```

The SYSTEM tablespace can only be encrypted with the default algorithm (AES128 if not changed in step 10c).

- b. Observe the alert.log of the standby database to confirm that the USERS tablespace there is also encrypted.

13. Encrypt the PDB tablespaces:

- a. Encrypt the USERS, SYSTEM, SYSAUX and all application tablespaces in the PDBs.

Encrypting the TEMP and UNDO tablespaces is optional because data from encrypted tablespaces is tracked and automatically encrypted before being written into TEMP or UNDO.

```
ALTER TABLESPACE USERS ENCRYPTION ONLINE [USING 'algorithm'] ENCRYPT;
ALTER TABLESPACE SYSTEM ENCRYPTION ONLINE ENCRYPT;
ALTER TABLESPACE SYSAUX ENCRYPTION ONLINE [USING 'algorithm'] ENCRYPT;
```

The SYSTEM tablespace can only be encrypted with the database default algorithm, which is AES128 unless it has been changed in the previous step that you performed on the primary and standby databases. If you want to encrypt the SYSTEM tablespace with another algorithm, then you can rekey the SYSTEM tablespace, for example: For example:

```
ALTER TABLESPACE SYSTEM ENCRYPTION ONLINE USING 'any_supported_algorithm' REKEY;
```

Observe the alert.log of the standby database to confirm the encryption and rekey operations are applied there as well.

- b. Optionally, encrypt the UNDO and TEMP tablespaces.

For example, to encrypt an UNDO tablespace named UNDOTBS1:

```
ALTER TABLESPACE UNDOTBS1 ENCRYPTION ONLINE [USING 'algorithm'] ENCRYPT;
```

You cannot use the ALTER TABLESPACE statement to encrypt an existing TEMP tablespace. To encrypt a TEMP tablespace, you must create a new one and encrypt it. First, extract the DDL that was used to create the original TEMP tablespace.

```
SELECT DBMS_METADATA.GET_DDL ('TABLESPACE', 'TEMP') FROM DBA_TEMP_FILES;
```

Modify the output so that tempfile and tablespace name are different. For example:

```
CREATE TEMPORARY TABLESPACE "TEMP_ENC"
TEMPFILE '/u01/opt/oracle/oradata/${ORACLE_SID}/${PDB-NAME}/
temp01_enc.dbf'
SIZE 146800640 AUTOEXTEND ON NEXT 655360
MAXSIZE 32767M EXTENT MANAGEMENT LOCAL UNIFORM SIZE 1048576
ENCRYPTION [USING 'algorithm'] ENCRYPT;
```

You only need to add the ENCRYPTION [USING 'algorithm'] ENCRYPT clause if you need to apply an algorithm other than the default. Omitting the ENCRYPTION [USING 'algorithm'] ENCRYPT clause will automatically encrypt the TEMP tablespace with the default algorithm.

Next, make this new encrypted TEMP tablespace the default temp tablespace of this PDB, and then drop the old clear-text TEMP tablespace.

```
ALTER DATABASE DEFAULT TEMPORARY TABLESPACE TEMP_ENC;
DROP TABLESPACE TEMP INCLUDING CONTENTS AND DATAFILES;
```

14. Optionally, create a tablespace and table in the primary database PDB.

When you create the tablespace in the primary database without encryption keywords in that statement. It will be encrypted with AES128 by default unless the database default algorithm has been changed in an earlier step, when you changed the database default algorithm from AES128 to either AES192 or AES256.

```
CREATE TABLESPACE protected DATAFILE SIZE 50M;
```

```
CREATE TABLE SYSTEM.TEST TABLESPACE protected
AS SELECT * FROM DBA_OBJECTS;
```

15. Confirm that you can select from the table that is stored in an encrypted tablespace.

```
SELECT COUNT(*), OWNER FROM SYSTEM.TEST
GROUP BY OWNER
ORDER BY 1 DESC;
```

16. On the **primary** and **standby** databases, run the following statements in the root:

```
SELECT c.name AS PDB_NAME, t.name
AS TBS_NAME, e.ENCRYPTIONALG
```

```
AS ALG, e.STATUS from v$tablespace t, v$encrypted_tablespaces e, v$containers c
WHERE e.ts# = t.ts# AND e.con_id = t.con_id and e.con_id = c.con_id order by
e.con_id, t.name;
```

PDB_NAME	TBS_NAME	ALG	STATUS
-----	-----	-----	-----
CDB\$ROOT	SYSAUX	AES256	NORMAL
CDB\$ROOT	SYSTEM	AES256	NORMAL
CDB\$ROOT	USERS	AES256	NORMAL
FINPDB19C	PROTECTED01	AES256	NORMAL
FINPDB19C	SYSAUX	AES256	NORMAL
FINPDB19C	SYSTEM	AES256	NORMAL
FINPDB19C	USERS	AES256	NORMAL

Note that the TEMP tablespace is not listed in the output of the standby databases.

17. Optionally, validate the configuration.

- a. Perform an Oracle Data Guard switchover between the primary and standby databases.

See *Oracle Data Guard Concepts and Administration*.

Perform the following steps in the new primary database.

- b. Select from the encrypted table in your PDB.

Because there is an auto-open connection into Oracle Key Vault, the following query does not require that you enter the Oracle Key Vault password.

```
SELECT COUNT(*), OWNER FROM SYSTEM.TEST
GROUP BY OWNER
ORDER BY 1 DESC;
```

24 rows selected.

- c. Rekey the PDB.

```
SELECT ' ADMINISTER KEY MANAGEMENT SET KEY
USING TAG ''||UPPER(SYS_CONTEXT('USERENV', 'CON_NAME'))||' '||TO_CHAR
(SYS_EXTRACT_UTC (SYSTIMESTAMP),
'YYYY-MM-DD HH24:MI:SS"Z"')||''
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE;'
AS "RE-KEY COMMAND" FROM DUAL;
```

- d. Execute the generated output of this SELECT statement.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
USING TAG 'pdb_name date time'
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE;
```

- e. Perform another Oracle Data Guard switchover.

See *Oracle Data Guard Concepts and Administration*.

- f. Select from the encrypted table in your PDB.

Because there is an auto-open connection into Oracle Key Vault, the following query does not require that you enter the Oracle Key Vault password.

```
SELECT COUNT(*), OWNER FROM SYSTEM.TEST
GROUP BY OWNER
ORDER BY 1 DESC;
```

24 rows selected.

g. Rekey the PDB.

```
SELECT ' ADMINISTER KEY MANAGEMENT SET KEY
USING TAG '''||UPPER(SYS_CONTEXT('USERENV', 'CON_NAME'))||' '''||TO_CHAR
(SYS_EXTRACT_UTC (SYSTIMESTAMP),
'YYYY-MM-DD HH24:MI:SS"Z"'')||'''
FORCE KEYSTORE
IDENTIFIED BY EXTERNAL STORE;'
AS "RE-KEY COMMAND" FROM DUAL;
```

h. Execute the generated output of this SELECT statement.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
USING TAG 'pdb_name date time'
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE;
```

10.2.4 Configuring Wallet-Based Transparent Data Encryption in Oracle Data Guard

You can configure wallet-based Transparent Data Encryption (TDE) in an Oracle Data Guard environment.

The following scenario shows how to configure software keystore-based TDE in a single-instance, multitenant Oracle Data Guard environment with one physical standby database. The version for the primary and standby databases must be release 19.6 or later. To complete this procedure, you must perform each step in the sequence shown. After you complete this configuration, you can migrate the primary and standby databases from the software keystore to Oracle Key Vault. Oracle recommends that you monitor the alert logs of both primary and standby databases.

1. In the primary and standby databases, create the directories that are needed for a software keystore-based TDE configuration.

For example:

```
# mkdir -pv /etc/ORACLE/KEYSTORES/finance/tde_seps
# cd /etc
# chown -Rv oracle:oinstall ./ORACLE
# chmod -Rv 700 ./ORACLE
```

2. In the CDB root of the primary and standby databases, as a user who has the ALTER SYSTEM privilege, run the following statements in SQL*Plus to set the appropriate parameters.

```
ALTER SYSTEM SET TABLESPACE_ENCRYPTION = AUTO_ENABLE SCOPE = SPFILE; --
Oracle Database 19.16 or later
ALTER SYSTEM SET ENCRYPT_NEW_TABLESPACE = ALWAYS SCOPE = BOTH; --Before
release 19.16
ALTER SYSTEM SET WALLET_ROOT = '/etc/ORACLE/KEYSTORES/$ORACLE_SID' SCOPE =
SPFILE;
```

3. Restart the primary and standby databases.
4. Set the following parameters in the primary and standby databases:

```
ALTER SYSTEM SET "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM" = 'AES256'
SCOPE = BOTH;
```

```
ALTER SYSTEM SET TDE_CONFIGURATION = "KEystore_CONFIGURATION=FILE" SCOPE =
BOTH;
```

5. Configure tablespace encryption.

- a. On the primary database, as a user with the SYSKM privilege, create a password-protected and a (local) auto-open software keystore.

When you create this keystore, Oracle Database automatically creates the `WALLET_ROOT/tde` directory.

```
ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY
tde_software_keystore_password;
ADMINISTER KEY MANAGEMENT CREATE LOCAL AUTO_LOGIN KEYSTORE FROM
KEYSTORE IDENTIFIED BY tde_software_keystore_password;
```

- b. Enable separation of duties by hiding the TDE wallet password in another (local) auto-open wallet to replace the keystore password with `EXTERNAL STORE` for those `ADMINISTER KEY MANAGEMENT` commands that do not change the TDE configuration in the primary and all standby databases:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'tde_software_keystore_password'
FOR CLIENT 'TDE_WALLET'
TO LOCAL AUTO_LOGIN KEYSTORE '/etc/ORACLE/KEYSTORES/finance/tde_seps';
```

6. Set the first key in the CDB\$ROOT container of the primary database.

```
ADMINISTER KEY MANAGEMENT SET KEY FORCE KEYSTORE IDENTIFIED BY EXTERNAL
STORE WITH BACKUP CONTAINER = CURRENT;
```

7. Log in to each open primary PDB and set the first, tagged key by executing the output of this command:

```
SELECT 'ADMINISTER KEY MANAGEMENT SET KEY USING TAG
'''||UPPER(SYS_CONTEXT('USERENV','CON_NAME'))||' '||TO_CHAR
(SYS_EXTRACT_UTC (SYSTIMESTAMP), 'YYYY-MM-DD HH24:MI:SS"Z"')||'''
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE
WITH BACKUP;' AS "SET KEY COMMAND";
```

8. Copy the software keystores from primary to standby databases.

The local auto-open software keystore of the primary cannot be opened on the standby, so it is excluded from the copy process. For example:

```
$ rsync -rvpt --exclude '*.sso' /etc/ORACLE/KEYSTORES/finance/tde/
standby_host:/etc/ORACLE/KEYSTORES/finance/tde/
```

9. On the CDB root of the standby, as a user who has the SYSKM administrative privilege, create a local auto-login software keystore:

```
ADMINISTER KEY MANAGEMENT CREATE LOCAL AUTO_LOGIN KEYSTORE
FROM KEYSTORE IDENTIFIED BY tde_keystore_password;
```

10. As a user with the ALTER TABLESPACE privilege, encrypt the SYSTEM, SYSAUX, and USERS tablespaces in the root container database:

```
ALTER TABLESPACE USERS ENCRYPTION ONLINE ENCRYPT;  
ALTER TABLESPACE SYSTEM ENCRYPTION ONLINE ENCRYPT;  
ALTER TABLESPACE SYSAUX ENCRYPTION ONLINE ENCRYPT;
```

11. As a user with the ALTER TABLESPACE privilege, encrypt the SYSTEM, SYSAUX, USERS, and all sensitive application tablespaces in the PDB.

```
ALTER TABLESPACE USERS ENCRYPTION ONLINE ENCRYPT;  
ALTER TABLESPACE SYSTEM ENCRYPTION ONLINE ENCRYPT;  
ALTER TABLESPACE SYSAUX ENCRYPTION ONLINE ENCRYPT;
```

12. Optionally, encrypt the UNDO and TEMP tablespaces in the primary PDBs.

- a. Perform the following ALTER TABLESPACE statement:

```
ALTER TABLESPACE UNDOTBS1 ENCRYPTION ONLINE ENCRYPT;
```

- b. Create a new encrypted TEMP tablespace with the same parameters that were applied to the original TEMP tablespace.

You cannot use the ALTER TABLESPACE statement to encrypt an existing TEMP tablespace, so you must create a new one.

First, extract the DDL that was used to create the original TEMP tablespace.

```
SELECT DBMS_METADATA.GET_DDL ('TABLESPACE', 'TEMP') FROM DBA_TEMP_FILES;
```

Second, modify the output so that the tempfile and tablespace names are different. For example:

```
CREATE TEMPORARY TABLESPACE "TEMP_ENC"  
TEMPFILE '/u01/opt/oracle/oradata/${ORACLE_SID}/${PDB-NAME}/  
temp01_enc.dbf'  
SIZE 146800640 AUTOEXTEND ON NEXT 655360  
MAXSIZE 32767M EXTENT MANAGEMENT LOCAL UNIFORM SIZE 1048576
```

- c. Make the new encrypted TEMP tablespace the default TEMP tablespace:

```
ALTER DATABASE DEFAULT TEMPORARY TABLESPACE TEMP_ENC;
```

- d. Drop the old clear-text TEMP tablespace:

```
DROP TABLESPACE TEMP INCLUDING CONTENTS AND DATAFILES;
```

- e. Rename the new TEMP tablespace.

```
ALTER TABLESPACE TEMP_ENC RENAME TO TEMP;
```


13. Optionally, create a small sample tablespace (which will be encrypted with AES256 even if no ENCRYPTION syntax is given), and create a copy of the DBA_OBJECTS table in this encrypted sample tablespace.

```
CREATE TABLESPACE PROTECTED DATAFILE SIZE 50M;
CREATE TABLE SYSTEM.TEST TABLESPACE PROTECTED AS SELECT * FROM DBA_OBJECTS;
```

14. Optionally, select from the encrypted copy of the DBA_OBJECTS table.

For example:

```
SELECT OWNER, COUNT(*) FROM SYSTEM.test GROUP BY OWNER ORDER BY 2 DESC;

24 rows selected.
```

15. Connect to the CDB root of the primary and standby databases and confirm.

```
SELECT C.NAME AS PDB_NAME, T.NAME AS TBS_NAME, E.ENCRYPTIONALG AS ALG,
E.STATUS
FROM V$TABLESPACE T, V$ENCRYPTED_TABLESPACES E, V$CONTAINERS C
WHERE E.TS# = T.TS# AND E.CON_ID = T.CON_ID AND E.CON_ID = C.CON_ID ORDER
BY E.CON_ID, T.NAME;
```

Output similar to the following should appear:

PDB_NAME	TBS_NAME	ALG	STATUS
-----	-----	-----	-----
CDB\$ROOT	SYSAUX	AES256	NORMAL
CDB\$ROOT	SYSTEM	AES256	NORMAL
CDB\$ROOT	USERS	AES256	NORMAL
FINPDB19C	PROTECTED01	AES256	NORMAL
FINPDB19C	SYSAUX	AES256	NORMAL
FINPDB19C	SYSTEM	AES256	NORMAL
FINPDB19C	USERS	AES256	NORMAL

The standby does not have a TEMP tablespace, so it will not be listed here.

16. Connect to the dgmgrrl utility on the standby as a user who has the SYSDBG administrative privilege.
17. Perform a switchover operation.

```
DGMGRL> switchover to '<standby_db_name>' wait 5;
```

18. To confirm the auto-open software keystore functionality, select from encrypted data (for example, the copy of the DBA_OBJECTS table in the PDB) of the new primary database after the role switch.

For example:

```
SELECT OWNER, COUNT(*) FROM SYSTEM.test GROUP BY OWNER ORDER BY 2 DESC;

24 rows selected.
```

19. Oracle recommends that you create and then use the key. This way, the managed recovery process on the standby database is not interrupted due to unknown TDE master encryption keys. In the new primary PDB, run the following statement:

```
ADMINISTER KEY MANAGEMENT CREATE KEY FORCE KEYSTORE IDENTIFIED BY EXTERNAL
STORE WITH BACKUP;
```

20. Copy the software keystore to the standby, excluding the local auto-open keystores, and automatically deleting the obsolete local auto-open keystore on the standby side.

For example:

```
$ rsync -rvpt --exclude '*.sso' --delete-excluded /etc/ORACLE/KEYSTORES/
finance/tde/ standby-host:/etc/ORACLE/KEYSTORES/finance/tde/
```

21. On the standby, re-create a local auto-open software keystore from the updated password-protected keystore so that the new key is immediately available to the standby database when needed.

```
ADMINISTER KEY MANAGEMENT CREATE LOCAL AUTO_LOGIN KEYSTORE
FROM KEYSTORE IDENTIFIED BY tde_keystore_password;
```

22. Execute the output of the following command:

```
SELECT ' ADMINISTER KEY MANAGEMENT
USE KEY '''||KEY_ID||'''
USING TAG '''||SYS_CONTEXT('USERENV', 'CON_NAME')||' '||
TO_CHAR (SYS_EXTRACT_UTC (SYSTIMESTAMP), 'YYYY-MM-DD HH24:MI:SS"Z"')||'''
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE
WITH BACKUP;' AS "USE KEY COMMAND"
FROM V$ENCRYPTION_KEYS
WHERE TAG IS NULL;
```

The USE KEY clause updates the associations in the software keystore, so the updated password-protected wallet is copied to the standby, again excluding the local auto-open software keystore and deleting the obsolete local auto-open keystore on the receiving standby side, for example, with the following command:

```
$ rsync -rvpt --exclude '*.sso' --delete-excluded /etc/ORACLE/KEYSTORES/
finance/tde/ standby-host:/etc/ORACLE/KEYSTORES/finance/tde/
```

23. On the CDB root of the primary, recreate the local auto-login software keystore.

```
ADMINISTER KEY MANAGEMENT CREATE LOCAL AUTO_LOGIN KEYSTORE FROM KEYSTORE
IDENTIFIED BY tde_keystore_password;
```

24. Using the dgmgrl utility, perform a switchover operation to the standby.

For example:

```
DGMGRL> switchover to '<standby_db_name>' wait 5;
```

25. Optionally, to confirm the auto-open functionality, select from an encrypted table (for example, the copy of the DBA_OBJECTS table).

```
SELECT OWNER, COUNT(*) FROM SYSTEM.test GROUP BY OWNER ORDER BY 2 DESC;
```

24 rows selected.

10.2.5 Migrating a TDE Wallet in an Oracle Data Guard Environment to Oracle Key Vault

After you have configured TDE wallet-based Transparent Data Encryption (TDE) in an Oracle Data Guard environment, you can migrate primary and standby databases to Oracle Key Vault, without downtime.

The following scenario shows how to configure one endpoint for each of the primary and standby databases, and then migrate the primary and standby databases from a TDE wallet to Oracle Key Vault. This scenario uses a single-instance, multitenant Oracle Data Guard environment with one physical standby database. The version for the primary and standby databases must be release 19.6 or later. To complete this procedure, you must perform each step in the sequence shown. See [Oracle Key Vault RESTful Services Administrator's Guide](#) for how an Oracle Key Vault administrator can create these files, which are later used by database administrators to automatically onboard their databases into Oracle Key Vault.

1. Copy the primary deployment script to the primary database host.

For example:

```
$ rsync -v 192.168.1.29:/directory_on_shared_server/OKVdeploy-DG-  
primary.tgz .
```

2. Copy the secondary deployment script to all standby database hosts.

For example:

```
$ rsync -v 192.168.1.29:/directory_on_shared_server/OKVdeploy-DG-  
standby.tgz .
```

3. On the primary and standby database hosts, extract the archive.

```
$ tar -xzf OKVdeploy-DG*
```

4. On the primary database, run the primary-run-me.sh script.

This script creates the deployment script (okv-ep.sh), which contains unique names for the wallet and endpoint that it creates in Oracle Key Vault for the primary database.

```
$ ./primary-run-me.sh
```

```
% Total    % Received % Xferd  Average Speed   Time    Time       Time
Current                                  Dload  Upload   Total   Spent    Left
Speed
100 3750k  100 3750k    0     0  13.5M      0 --:--:-- --:--:-- --:--:--
13.5M
Archive:  okvrestclipackage.zip
  inflating: ./lib/okvrestcli.jar
```

```
#!/bin/bash
mkdir -pv /etc/ORACLE/KEYSTORES/finance/okv
okv manage-access wallet create --wallet FINANCE --unique FALSE
okv admin endpoint create --endpoint FINANCE_on_SanDiego --description
"SanDiego.us.oracle.com, 192.168.56.193"
  --subgroup "USE CREATOR SUBGROUP" --unique FALSE --strict-ip-check TRUE
okv manage-access wallet set-default --wallet FINANCE --endpoint
FINANCE_on_SanDiego
expect << _EOF
  set timeout 120
  spawn okv admin endpoint provision --endpoint FINANCE_on_SanDiego --
location /etc/ORACLE/KEYSTORES/finance/okv --auto-login FALSE
  expect "Enter Oracle Key Vault endpoint password: "
  send "change-on-install\r"
  expect eof
_EOF
```

5. On the standby databases, run the secondary-run-me.sh script.

This script creates the deployment script (okv-ep.sh), which contains unique names for the endpoint that it creates in Oracle Key Vault for the standby databases. Note that the standby endpoint uses the primary wallet as its default wallet, so that primary and standby databases have access to same key material.

```
$ ./secondary-run-me.sh
```

```
% Total    % Received % Xferd  Average Speed   Time    Time       Time
Current
                                Dload  Upload  Total  Spent  Left
Speed
100 3750k  100 3750k    0     0  16.0M      0 --:--:-- --:--:-- --:--:--
16.1M
Archive:  okvrestclipackage.zip
  inflating: ./lib/okvrestcli.jar
#!/bin/bash
mkdir -pv /etc/ORACLE/KEYSTORES/finance/okv
okv admin endpoint create --endpoint FINANCE_on_Phoenix
  --description "Phoenix.us.oracle.com, 192.168.56.194"
  --subgroup "USE CREATOR SUBGROUP"
  --unique FALSE
  --strict-ip-check TRUE
okv manage-access wallet set-default --wallet FINANCE --endpoint
FINANCE_on_Phoenix
expect << _EOF
  set timeout 120
  spawn okv admin endpoint provision --endpoint FINANCE_on_Phoenix --
location /etc/ORACLE/KEYSTORES/finance/okv --auto-login FALSE
  expect "Enter Oracle Key Vault endpoint password: "
  send "change-on-install\r"
  expect eof
_EOF
```

6. On the primary database host, run the okv-ep.sh script.

This script creates a wallet and endpoint for the primary database in Oracle Key Vault. It makes the wallet the default wallet of that endpoint. The following okv admin endpoint

provision command downloads and installs the Oracle Key Vault endpoint software into the existing directory, `WALLET_ROOT/okv`.

```
$ ./deploy-OKV.sh

{
  "result" : "Success",
  "value" : {
    "status" : "PENDING",
    "locatorID" : "87E59AAD-0AAA-4AE8-ADCE-01D283ECE9C4"
  }
}
{
  "result" : "Success",
  "value" : {
    "status" : "PENDING",
    "locatorID" : "D13CC460-7BFB-451D-9998-DA387FC45783"
  }
}
{
  "result" : "Success"
}
spawn okv admin endpoint provision --endpoint finance_on_SanDiego --
location /etc/ORACLE/KEYSTORES/finance/okv --auto-login FALSE
Enter Oracle Key Vault endpoint password:
{
  "result" : "Success"
}
```

7. On the standby database hosts, run the `okv-ep.sh` script.

This script creates an endpoint for the standby database in Oracle Key Vault. It makes the primary wallet the default wallet of that endpoint. The following `okv admin endpoint provision` command downloads and installs the Oracle Key Vault endpoint software into the existing directory `WALLET_ROOT/okv`.

```
$ ./deploy-OKV.sh

{
  "result" : "Success",
  "value" : {
    "status" : "PENDING",
    "locatorID" : "4208FBB5-5FD4-47EE-B3DB-FA8277BAFB84"
  }
}
{
  "result" : "Success"
}
spawn okv admin endpoint provision --endpoint finance_on_Phoenix --
location /etc/ORACLE/KEYSTORES/finance/okv --auto-login FALSE
Enter Oracle Key Vault endpoint password:
{
  "result" : "Success"
}
```

8. On the primary and standby database hosts, run the `root.sh` script in the `WALLET_ROOT/okv/bin`.

This script deploys the PKCS#11 library into the correct destination directory.

```
$ sudo /etc/ORACLE/KEYSTORES/finance/okv/bin/root.sh
```

9. On the primary and standby databases, change the default password to a strong password that the database administrator should not know.

This password is the same for all databases that used the deployment script.

```
$ /etc/ORACLE/KEYSTORES/finance/okv/bin/okvutil changepwd -l /etc/ORACLE/KEYSTORES/finance/okv/ssl/ -t wallet
```

```
Enter wallet password: current_endpoint_password
Enter new wallet password: new_endpoint_password
Confirm new wallet password: new_endpoint_password
Wallet password changed successfully
```

10. Optionally, update the endpoint parameters.

This step and the next two steps are optional. By default, the persistent cache is a password-protected wallet that is protected with the endpoint password from the preceding step. With the change in steps [10](#), [11](#), and [12](#), the persistent cache will be protected with a random password, which implies that the persistent cache will expire after database shutdown, and needs to be rebuilt after each database restart. Additionally, the persistent cache expiration time is changed to 4 hours, but the persistent cache refresh window is extended to 21 days.

```
$ okv admin endpoint update --generate-json-input | jq '.service.options
|= ({endpoint} | .endpointConfiguration
|= (.expirePkcs11PersistentCacheOnDatabaseShutdown = "TRUE"
| .pkcs11PersistentCacheRefreshWindow = "P21D"
| .pkcs11PersistentCacheTimeout = "PT4H"))'
> ./update-endpoint.json; more ./update-endpoint.json
```

```
{
  "service": {
    "category": "admin",
    "resource": "endpoint",
    "action": "update",
    "options": {
      "endpoint": "#VALUE",
      "endpointConfiguration": {
        "expirePkcs11PersistentCacheOnDatabaseShutdown": "TRUE",
        "pkcs11PersistentCacheRefreshWindow": "P21D",
        "pkcs11PersistentCacheTimeout": "PT4H"
      }
    }
  }
}
```

11. Optionally, on the primary, check the status of the endpoints on both the primary and standby databases.

For example, for a primary with an endpoint called `finance_on_SanDiego` and a standby with an endpoint called `finance_on_Phoenix`:

```
$ okv admin endpoint check-status --endpoint finance_on_SanDiego
$ okv admin endpoint check-status --endpoint finance_on_Phoenix
```

12. After both endpoints are active, apply and re-use the `update-endpoint.json` file to update all endpoints of the primary and standby databases.:

```
$ okv admin endpoint update --from-json ./update-endpoint.json --endpoint
finance_on_SanDiego
$ okv admin endpoint update --from-json ./update-endpoint.json --endpoint
finance_on_Phoenix
```

13. On the primary database host, upload the current and retired master encryption keys from the TDE wallet into the virtual wallet in Oracle Key Vault.

For example:

```
$ /etc/ORACLE/KEYSTORES/finance/okv/bin/okvutil upload -l /etc/ORACLE/
KEYSTORES/finance/tde/ -t wallet -g FINANCE -v 3
```

14. In the root of the primary, using SQL*Plus, add the Oracle Key Vault password from step 9 into the TDE wallet to enable an auto-open Oracle Key Vault connection.

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'Oracle_Key_Vault_password' FOR
CLIENT 'OKV_PASSWORD'
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE WITH BACKUP;
```

15. Copy the updated TDE wallet from the primary database host to the standby database hosts, excluding the local auto-login TDE wallet and deleting the obsolete local auto-login TDE wallet on the standby side.

For example:

```
$ rsync -rvpt --exclude '*.sso' --delete-excluded /etc/ORACLE/KEYSTORES/
finance/tde/ standby-host:/etc/ORACLE/KEYSTORES/finance/tde/
```

16. On the CDB root of the standby, using SQL*Plus, recreate the local auto-login TDE wallet, and then close it.

```
ADMINISTER KEY MANAGEMENT CREATE LOCAL AUTO_LOGIN KEYSTORE
FROM KEYSTORE IDENTIFIED BY TDE_wallet_password;
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE CONTAINER = ALL;
```

At this stage, the primary and standby databases are ready to be migrated into Oracle Key Vault.

17. In the primary and standby databases, change the `TDE_CONFIGURATION` dynamic parameter to `OKV|FILE`.

```
ALTER SYSTEM SET TDE_CONFIGURATION = "KEYSTORE_CONFIGURATION=OKV|FILE"
SCOPE = BOTH;
```

18. Run the `ADMINISTER KEY MANAGEMENT...MIGRATE` command.

This command decrypts the data encryption keys with the most recent key from the TDE wallet, and re-encrypts them with a new TDE master encryption key that is created in Oracle Key Vault. If this is a container database, then each open PDB will go through these steps. This entails no downtime; the operation takes approximately 10 to 15 seconds for each PDB.

```
ADMINISTER KEY MANAGEMENT SET KEY
IDENTIFIED BY "Oracle_Key_Vault_password"
FORCE KEYSTORE MIGRATE USING "TDE_wallet_password";
```

19. Add the Oracle Key Vault password, and remove the old TDE wallet password, from the local auto-login wallet that enables to replace the TDE wallet password on the SQL*plus command line with EXTERNAL STORE.

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'Oracle_Key_Vault_password'
FOR CLIENT "OKV_PASSWORD"
TO LOCAL AUTO_LOGIN KEYSTORE '/etc/ORACLE/KEYSTORES/finance/tde_seps';

ADMINISTER KEY MANAGEMENT DELETE SECRET FOR CLIENT 'TDE_WALLET'
FROM LOCAL AUTO_LOGIN KEYSTORE '/etc/ORACLE/KEYSTORES/finance/tde_seps';
```

20. Now that primary and standby databases have been migrated to Oracle Key Vault, delete the old wallets (only because you have uploaded the keys into Oracle Key Vault in step [13](#)).

```
$ rm -v /etc/ORACLE/KEYSTORES/finance/tde/*
```

21. On the primary and standby databases, create a local auto-login TDE wallet in `WALLET_ROOT/tde` that only contains the Oracle Key Vault password to enable an auto-login Oracle Key Vault configuration.

For example:

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'Oracle_Key_Vault_password'
FOR CLIENT 'OKV_PASSWORD' TO LOCAL AUTO_LOGIN KEYSTORE '/etc/ORACLE/
KEYSTORES/finance/tde';
```

22. Create a tag for each key that is associated with each PDB.

During migration, Oracle Key Vault created a new key for each PDB, but without a tag. The tag that you create makes it easier to find which key belongs to which PDB. You should give each key a tag that matches the PDB name, and a time stamp.

```
SELECT ' ADMINISTER KEY MANAGEMENT SET TAG ''' || SYS_CONTEXT('USERENV',
'CON_NAME') || ' ' || TO_CHAR (SYS_EXTRACT_UTC (ACTIVATION_TIME),
'YYYY-MM-DD HH24:MI:SS"Z"') || ' ' FOR ' ' || KEY_ID || ' '
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE;' AS "SET TAG COMMAND"
FROM V$ENCRYPTION_KEYS
WHERE CREATOR_PDBNAME = SYS_CONTEXT('USERENV', 'CON_NAME')
ORDER BY CREATION_TIME DESC FETCH FIRST 1 ROWS ONLY;
```

10.2.6 Isolating an Encrypted PDB in an Oracle Data Guard Environment

To isolate PDB's in an Oracle Data Guard environment, you must manually perform the configuration.

This is because the `ADMINISTER KEY MANAGEMENT` command that is run on the primary does not affect the standby. First, you isolate the PDB in the primary database. The `ADMINISTER KEY MANAGEMENT ISOLATE KEYSTORE` command performs the necessary isolation tasks, such as changing the PDB's `TDE_CONFIGURATION` parameter to `FILE` and moving the key from the united mode wallet to the newly created isolated mode wallet. Next, you must perform these same tasks manually on the standby to complete the isolation process.

1. If you have not done so you, log in to the CDB root of the primary database, and then isolate the PDB.

```
ADMINISTER KEY MANAGEMENT ISOLATE KEYSTORE
IDENTIFIED BY "new_isolated_pdb_keystore_password"
FROM ROOT KEYSTORE
IDENTIFIED BY "root_keystore_password"
EXTERNAL STORE WITH BACKUP;
```

2. Log in to the server where the standby database is configured.
3. Ensure that there is a directory for the PDB (for example, `$WALLET_ROOT/pdb_guid/tde/`).
4. Connect to the PDB as a user who has the `ALTER SYSTEM` privilege.
5. Set the `KEYSTORE_CONFIGURATION` parameter for this PDB.

```
ALTER SYSTEM SET TDE_CONFIGURATION="KEYSTORE_CONFIGURATION=FILE"
SCOPE=BOTH;
```

6. At the command line, from the CDB root, copy the wallet (`.p12` and `.sso`) from the primary database to the standby; place this wallet in the `$WALLET_ROOT/pdb_guid/tde/` directory.
7. Close and then re-open the CDB root's wallet.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE
FORCE KEYSTORE IDENTIFIED BY root_wallet_password;
```

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
FORCE KEYSTORE IDENTIFIED BY root_wallet_password;
```

8. Connect to the PDB.
9. Open the wallet in the PDB.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
FORCE KEYSTORE IDENTIFIED BY pdb_wallet_password;
```

10.3 How Transparent Data Encryption Works with Oracle Real Application Clusters

Oracle Real Application Clusters (Oracle RAC) nodes can share both a TDE wallets and an external keystore.

- [About Using Transparent Data Encryption with Oracle Real Application Clusters](#)
Oracle requires a shared TDE wallet for Oracle Real Application Clusters (Oracle RAC), or a shared common virtual wallet in Oracle Key Vault among cluster instances.
- [Configuring TDE in Oracle Real Application Clusters for Oracle Key Vault](#)
You can migrate encrypted Oracle Real Application Cluster (Oracle RAC) database instances to centralized key management provided by Oracle Key Vault. The migration process has been automated and is integrated with the provisioning workflow in Oracle Exadata Database Service on Cloud@Customer and Oracle Autonomous Database on Dedicated Exadata Infrastructure.

10.3.1 About Using Transparent Data Encryption with Oracle Real Application Clusters

Oracle requires a shared TDE wallet for Oracle Real Application Clusters (Oracle RAC), or a shared common virtual wallet in Oracle Key Vault among cluster instances.

A TDE configuration with Oracle Key Vault uses a network connection from each instance of the database to the external key manager. In Oracle Key Vault, you must create one endpoint for each instance of the Oracle RAC-enabled database, and one virtual wallet for each Oracle RAC-enabled database. Then, make that virtual wallet the default wallet of all endpoints that belong to that database. In an Oracle RAC-enabled Data Guard configuration, all instances (primary and all standby databases) share that one virtual wallet. With this configuration, set key and re-key operations are completely transparent because all participating instances are automatically synchronized. This eliminates the need to manually copy the TDE wallet to each of the other nodes in the cluster.

Oracle does not support the use of individual TDE wallets for each Oracle RAC node. Instead, use shared wallets for TDE in the Oracle RAC environment. This enables all of the instances to access the same shared TDE wallet. If your site uses Oracle Automatic Storage Management Cluster File System (Oracle ACFS), then this is the preferred location for a shared wallet. Directly sharing the wallet in Oracle Automatic Storage Management (Oracle ASM) (for example, `+DATA/$ORACLE_UNQNAME/WALLETS`) is an alternative if Oracle ACFS is not available.

Keystore operations (such as opening or closing the keystore, or rekeying the TDE master encryption key) can be issued on any one Oracle RAC instance. Internally, the Oracle database takes care of synchronizing the keystore context on each Oracle RAC node, so that the effect of the keystore operation is visible to all of the other Oracle RAC instances in the cluster. Similarly, when a TDE master encryption key rekey operation takes place, the new key becomes available to each of the Oracle RAC instances. You can perform other keystore operations, such as exporting TDE master encryption keys, rotating the keystore password, merging keystores, or backing up keystores, from a single instance only.

When using a shared file system, ensure that the `WALLET_ROOT` static system parameter for all of the Oracle RAC instances point to the same shared TDE wallet location, as follows:

```
ALTER SYSTEM SET WALLET_ROOT = '+DATA/$ORACLE_UNQNAME' SCOPE = SPFILE SID = '*';

ALTER SYSTEM SET TDE_CONFIGURATION = "KESTORE_CONFIGURATION=FILE" SCOPE = BOTH SID =
'*';
```

Note

- If you have the `ENCRYPTION_WALLET_LOCATION` parameter set, then be aware this parameter is deprecated. Oracle recommends that you use the `WALLET_ROOT` static initialization parameter and `TDE_CONFIGURATION` dynamic initialization parameter instead.
- Storing TDE master encryption keys in individual wallets per Oracle Real Application Clusters (Oracle RAC) instance is not supported. As an alternative, use Oracle Key Vault for centralized key management across your on-premises or Cloud-based database deployments, or Oracle Automatic Storage Management (Oracle ASM), or Oracle ASM Cluster File System (Oracle ACFS) to provide local shared wallets.

10.3.2 Configuring TDE in Oracle Real Application Clusters for Oracle Key Vault

You can migrate encrypted Oracle Real Application Cluster (Oracle RAC) database instances to centralized key management provided by Oracle Key Vault. The migration process has been automated and is integrated with the provisioning workflow in Oracle Exadata Database Service on Cloud@Customer and Oracle Autonomous Database on Dedicated Exadata Infrastructure.

The following scenario assumes that you have a multitenant two-node Oracle RAC configuration. In this procedure, you must complete the following steps in the order shown. After you have completed this procedure, the Oracle RAC environment will exclusively use Oracle Key Vault for key management for Transparent Data Encryption. This procedure assumes that you have installed the January 2020 (19.6) or later upgrade for Oracle Database release 19c.

Before you begin, monitor the alert logs of your running Oracle RAC database. The Java version that is included in the default Oracle Database release 19c installation can be used to install the Oracle Key Vault client with the RESTful services.

1. Download the Oracle Key Vault deployment script that the Oracle Key Vault administrators prepared to enable database administrators to automatically register their Oracle databases with Oracle Key Vault.

Oracle Key Vault RESTful Services Administrator's Guide has an example of how to create a script to automatically enroll Oracle databases as endpoints. The deployment scripts reside on a shared file system from which database administrators can download. There are two different versions of these deployment scripts. One script is for only the first node (which is called lead node in this procedure) and the other script is for all other nodes (which are called secondary nodes in this procedure). You can use these scripts for an Oracle RAC or an Oracle Data Guard environment.

Another component that the Oracle Key Vault administrators prepare and add to the deployment script is a configuration file that contains all details for the deployment scripts to connect to Oracle Key Vault.

2. Copy the two deployment scripts (`primary.zip` and `secondary.zip`) that an Oracle Key Vault administrator created for database administrators to download from a shared location.

- a. Copy the `primary.zip` file to the lead node.

```
$ scp user@ip_address:/path/to/file/primary.zip .
```

- b. Copy the `secondary.zip` file to each secondary node.

```
$ scp user@ip_address:/path/to/file/secondary.zip .
```

3. Extract the zip files.

- a. **On the lead node:** Extract the `primary.zip` file.

```
$ unzip primary.zip
```

- b. **On the secondary nodes:** Extract the `secondary.zip` file.

```
$ unzip secondary.zip
```

4. Create the following directories on all nodes:

For example:

```
$ mkdir -pv /u01/opt/oracle/product/okv
$ mkdir -pv /u01/opt/oracle/product/tde
$ mkdir -pv /u01/opt/oracle/product/tde_seps
```

In this specification:

- The `/u01/opt/oracle/product` directory will be defined as `WALLET_ROOT` in a later step.
 - `/u01/opt/oracle/product/okv` is the installation directory for the Oracle Key Vault client software. Depending on how the `TDE_CONFIGURATION` parameter is set, the Oracle Database will look for the Oracle Key Vault client software in `wallet_root/okv`.
 - `/u01/opt/oracle/product/tde` will store an auto-login wallet, which only contains the future Oracle Key Vault password, enabling an auto-login Oracle Key Vault configuration. Depending on how `TDE_CONFIGURATION` is set, the Oracle Database will look for the TDE wallet or an auto-open wallet for Oracle Key Vault, in `wallet_root/tde`.
 - `/u01/opt/oracle/product/tde_seps` will store an auto-login wallet, which only contains the future Oracle Key Vault password. This will hide the Oracle Key Vault password from the `SQL*Plus` command line and potentially from the database administrator to enforce separation of duties between Oracle database administrators and Oracle Key Vault administrators.
5. Execute the `primary-run-me.sh` and `secondary-run-me.sh` scripts, which contain the commands for the RESTful API to execute in Oracle Key Vault.

The Oracle Key Vault RESTful services will execute these commands in order to register this database in Oracle Key Vault with unique wallet and endpoint names.

- a. **On the lead node:** This script creates a shared wallet (for the lead and all secondary nodes) and an endpoint in Oracle Key Vault, associates this endpoint for the lead node with the shared wallet, and downloads and installs the Oracle Key Vault client into an **existing** installation directory. With the `WALLET_ROOT` configuration, this directory is `wallet_root/okv`.

```
$ more primary-run-me.sh
```

```
#!/bin/bash
export EP_NAME=${ORACLE_SID^^}_on_$(hostname -s)
export WALLET_NAME=${ORACLE_UNQNAME^^}${HOSTNAME//[!0-9]/}
curl -Ok https://192.168.1.181:5695/okvrestclpackage.zip --tlsv1.2
```

```

unzip -Voj okvrestclipackage.zip lib/okvrestcli.jar -d ./lib
cat > /home/oracle/deploy-OKV.sh << EOF
#!/bin/bash
okv manage-access wallet create --wallet ${WALLET_NAME} --unique FALSE
okv admin endpoint create --endpoint ${EP_NAME} --description "$(hostname -f) $(hostname -i)" --subgroup "USE CREATOR SUBGROUP" --unique FALSE
okv admin endpoint update --endpoint ${EP_NAME} --strict-ip-check TRUE
okv manage-access wallet set-default --wallet ${WALLET_NAME} --endpoint ${EP_NAME}
expect << _EOF
    set timeout 120
    spawn okv admin endpoint provision --endpoint ${EP_NAME} --location /etc/ORACLE/KEYSTORES/${ORACLE_UNQNAME^^}/okv --auto-login FALSE
    expect "Enter Oracle Key Vault endpoint password: "
    send "change-on-install\r"
    expect eof
_EOF
EOF

```

- b. **Secondary nodes:** This script only creates an endpoint for the secondary nodes, associates the endpoint of the secondary nodes with the shared wallet, and downloads and installs the Oracle Key Vault client into the **existing** installation directory on each secondary node.

```
$ more run-me.sh
```

```

#!/bin/bash
export EP_NAME=${ORACLE_SID^^}_on_$(hostname -s)
export WALLET_NAME=${ORACLE_UNQNAME^^}${HOSTNAME//[!0-9]/}
curl -Ok --tlsv1.2 https://<OKV-IP-addr>:5695/okvrestclipackage.zip
unzip -Voj okvrestclipackage.zip lib/okvrestcli.jar -d ./lib
cat > /home/oracle/deploy-OKV.sh << EOF
#!/bin/bash
okv admin endpoint create --endpoint ${EP_NAME} --description "$(hostname -f) $(hostname -i)" --subgroup "USE CREATOR SUBGROUP" --unique FALSE
okv admin endpoint update --endpoint ${EP_NAME} --strict-ip-check TRUE
okv manage-access wallet set-default --wallet ${WALLET_NAME} --endpoint ${EP_NAME}
expect << _EOF
    set timeout 120
    spawn okv admin endpoint provision --endpoint ${EP_NAME} --location /etc/ORACLE/KEYSTORES/${ORACLE_UNQNAME^^}/okv --auto-login FALSE
    expect "Enter Oracle Key Vault endpoint password: "
    send "change-on-install\r"
    expect eof
_EOF
EOF

```

6. After successful installation of the Oracle Key Vault client, execute the `root.sh` script to install the PKCS library on all nodes.

```
# Oracle_Key_Vault_installation_directory/bin/root.sh
```

The following output should appear:

```

Creating directory: /opt/oracle/extapi/64/hsm/oracle/1.0.0/
Copying PKCS library to /opt/oracle/extapi/64/hsm/oracle/1.0.0/
Setting PKCS library file permissions

```

7. Execute the Oracle Key Vault `okvutil changepwd` command on all nodes to change the password for the Oracle Key Vault client that you installed.

Because all database administrators downloaded the same deployment script, all databases have the same password into Oracle Key Vault. This step enables each database to have a unique password.

```
$ /u01/opt/oracle/product/okv/bin/okvutil changepwd -t wallet -l /u01/opt/oracle/product/okv/ssl/
```

```
Enter wallet password: default_password
Enter new wallet password: Oracle_Key_Vault_password
Confirm new wallet password: Oracle_Key_Vault_password
Wallet password changed successfully
```

8. On all nodes, add the Oracle Key Vault password into a local auto-login wallet to hide the newly changed password from database administrators.

```
sqlplus c##sec_admin as syskm
Enter password: password
```

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'Oracle_Key_Vault_password'
FOR CLIENT 'OKV_PASSWORD'
TO LOCAL AUTO_LOGIN KEYSTORE '/u01/opt/oracle/product/tde_seps';
```

9. In the root container, execute the ALTER SYSTEM statement to set the static WALLET_ROOT parameter to configure the encryption wallet location for all instances:

- a. Set the WALLET_ROOT parameter.

```
ALTER SYSTEM SET WALLET_ROOT='/u01/opt/oracle/product' SCOPE = SPFILE;
```

- b. Depending on the version of Oracle Database 19c that you have, set either the TABLESPACE_ENCRYPTION or ENCRYPT_NEW_TABLESPACES parameter.

- For Oracle Database 19.16 and later, run the following ALTER SYSTEM statement:

```
ALTER SYSTEM SET TABLESPACE_ENCRYPTION = 'value' SCOPE = SPFILE SID
= '*';
```

- For Oracle Database 19.15 and earlier, run this ALTER SYSTEM statement:

```
ALTER SYSTEM SET ENCRYPT_NEW_TABLESPACES = ALWAYS SCOPE = BOTH SID
= '*';
```

10. Restart the database.

11. In the root container, use the ALTER SYSTEM statement to set the dynamic TDE_CONFIGURATION parameter.

For example:

```
ALTER SYSTEM SET TDE_CONFIGURATION = "KEYSTORE_CONFIGURATION=OKV"
SCOPE = BOTH SID = '*';
```

12. Optionally, define the database default encryption algorithm after applying Oracle patch 30398099.

By default, Oracle Database applies the AES128 algorithm to encryption clauses that do not specify an encryption algorithm. Patch 30398099 allows you to choose from the AES128, AES192, and AES256 encryption algorithms. Patch 30398099 has been integrated into the April 2021 RU for Oracle Database release 19c (19.11), so this functionality is available without extra patches. Note that the following parameter is preceded by an underscore.

```
ALTER SYSTEM SET "_TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM" = 'encryption_algorithm'
SCOPE = BOTH SID = '*';
```


13. In the root container, open the keystore, which opens the connection to Oracle Key Vault for the root container and all open PDBs.

Note that the Oracle Key Vault password has been replaced in all subsequent ADMINISTER KEY MANAGEMENT commands with EXTERNAL STORE, because the database automatically retrieves the Oracle Key Vault password from the local auto-login wallet that you created in Step 8.

```
ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN
IDENTIFIED BY EXTERNAL STORE
CONTAINER = ALL;
```

14. In the root container, set the master encryption key.

```
ADMINISTER KEY MANAGEMENT SET KEY
IDENTIFIED BY EXTERNAL STORE
CONTAINER = CURRENT;
```

15. Create and activate a tagged master encryption key in all PDBs in this container.

The benefit of adding tagged master encryption keys to PDBs is that it enables you to easily identify keys that belong to a certain PDB.

- a. Connect to each PDB and execute the following SELECT statement to create an ADMINISTER KEY MANAGEMENT command that contains the PDB name and time stamp as a tag for the PDB's master encryption key.

```
CONNECT sec_admin@pdb_name AS SYSKM
Enter password: password
```

```
SELECT ' ADMINISTER KEY MANAGEMENT SET KEY
USING TAG '''||SYS_CONTEXT('USERENV', 'CON_NAME')||' '||
TO_CHAR (SYSDATE, 'YYYY-MM-DD HH24:MI:SS')||'''
IDENTIFIED BY EXTERNAL STORE;' "SET KEY COMMAND" FROM DUAL;
```

- b. Execute the generated output of this SELECT statement.

For example:

```
ADMINISTER KEY MANAGEMENT SET KEY
USING TAG 'pdb_name date time'
IDENTIFIED BY EXTERNAL STORE;
```

16. On all nodes, add the Oracle Key Vault password into an auto-login wallet to enable auto-login connection into Oracle Key Vault.

This step is mandatory in Oracle RAC. Having an auto-login connection into Oracle Key Vault is especially important when Oracle RAC nodes are automatically restarted (for example, while applying quarterly release upgrades using the opatchauto patch tool).

```
ADMINISTER KEY MANAGEMENT ADD SECRET 'Oracle_Key_Vault_password'
FOR CLIENT 'OKV_PASSWORD'
TO LOCAL AUTO_LOGIN KEYSTORE '/u01/opt/oracle/product/tde';
```

If you are using a release earlier than Oracle Database release 19.13, then use for client 'HSM_PASSWORD', not FOR CLIENT 'OKV_PASSWORD'.

17. In the root container, execute the ALTER SYSTEM statement to change the TDE_CONFIGURATION parameter.

For example:

```
ALTER SYSTEM SET TDE_CONFIGURATION = "KEYSTORE_CONFIGURATION=OKV|FILE"
SCOPE = BOTH SID = '*';
```

18. For versions of Oracle Database earlier than release 19.16, change the default behavior of the database to always encrypt new tablespaces with the AES128 algorithm (or the algorithm that you specified in Step [12](#)), even if the CREATE TABLESPACE command does not contain the encryption clauses.

Execute this statement once on any node:

```
ALTER SYSTEM SET ENCRYPT_NEW_TABLESPACES = ALWAYS SCOPE = BOTH SID = '*';
```

19. From the root, encrypt sensitive credential data with AES256 for database links in the SYS.LINK\$ and SYS.SCHEDULER\$_CREDENTIAL system tables.

This command requires the SYSKM administrative privilege:

```
sqlplus c##sec_admin as syskm
Enter password: password
```

```
ALTER DATABASE DICTIONARY ENCRYPT CREDENTIALS;
```

20. Log in to the PDB and create a tablespace.

For example, to create a tablespace named protected:

```
CREATE TABLESPACE protected DATAFILE SIZE 50M;
```

21. Confirm that the tablespace is encrypted even though the encryption clauses were omitted.

```
SELECT C.NAME AS pdb_name, T.NAME AS tablespace_name, E.ENCRYPTIONALG AS ALG
FROM V$TABLESPACE T, V$ENCRYPTED_TABLESPACES E, V$CONTAINERS C
WHERE E.TS# = T.TS# AND E.CON_ID = T.CON_ID AND E.CON_ID = C.CON_ID
ORDER BY E.CON_ID, T.NAME;
```

22. Create a table in the encrypted tablespace that you just created.

For example:

```
CREATE TABLE SYSTEM.test TABLESPACE protected
AS SELECT * FROM DBA_OBJECTS;
```

23. Select from this table to confirm that you can read encrypted data:

```
SELECT COUNT(*), OWNER FROM SYSTEM.test
GROUP BY OWNER
ORDER BY 1 DESC;
```

24. In the PDBs, encrypt the existing tablespaces.

Optionally, encrypt the SYSTEM, SYSAUX, and USERS tablespaces. If you omit the encryption algorithm, then the default algorithm (AES128, or the algorithm that you specified in Step [12](#)) is applied.

```
ALTER TABLESPACE tablespace_name ENCRYPTION ONLINE ENCRYPT;
```

25. Optionally, validate the configuration.

- a. Confirm that the auto-login for Oracle Key Vault is working.

You can test this by restarting the database, logging into the PDB, and then selecting from the encrypted table. To restart the database:

```
$ srvctl stop database -db database_name -o immediate
$ srvctl start database -db database_name
```

After logging in to the PDB, select from the SYSTEM.test table.

```
SELECT COUNT(*), OWNER FROM SYSTEM.test
GROUP BY OWNER
ORDER BY 1 DESC;
```

- b. Confirm that the master encryption key re-key operations in all open PDBs are successful.

First, as a user who has the SYSKM administrative privilege, execute the following SELECT statement to create an ADMINISTER KEY MANAGEMENT command that contains the PDB name and time stamp.

```
SELECT ' ADMINISTER KEY MANAGEMENT SET KEY
USING TAG ''||SYS_CONTEXT('USERENV','CON_NAME')||' '||
TO_CHAR(SYSDATE,'YYYY-MM-DD HH24:MI:SS')||''
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE;' "RE-KEY COMMAND" FROM DUAL;
```

Next, execute the generated output of this SELECT statement.

```
ADMINISTER KEY MANAGEMENT SET KEY
USING TAG 'pdb_name date time'
FORCE KEYSTORE IDENTIFIED BY EXTERNAL STORE;
```

- c. From the root container, re-key previously encrypted sensitive credential data in the SYS.LINK\$ and SYS.SCHEDULER\$_CREDENTIAL system tables.

This command requires the SYSKM administrative privilege:

```
ALTER DATABASE DICTIONARY REKEY CREDENTIALS;
```

- d. Drop the protected tablespace and its table, test.

```
DROP TABLESPACE protected
INCLUDING CONTENTS AND DATAFILES;
```

Related Topics

- [Supported Encryption and Integrity Algorithms](#)
The supported Advanced Encryption Standard cipher keys, including tablespace and database encryption keys, can be either 128, 192, or 256 bits long. Tablespace and database encryption use the 128-bit length cipher key.
- [Oracle Exadata Database Service on Cloud@Customer: Manage Encryption Keys](#)
- [Oracle Autonomous Database on Dedicated Exadata Infrastructure: Manage Encryption Keys](#)
- [MOS Note: Migration of File based TDE to Oracle Key Vault for Oracle Exadata Database Service on Cloud@Customer Using Automation via REST](#)

10.4 How Transparent Data Encryption Works with SecureFiles

SecureFiles, which stores LOBS, has three features: compression, deduplication, and encryption.

- [About Transparent Data Encryption and SecureFiles](#)
SecureFiles encryption uses TDE to provide the encryption facility for LOBs.
- [Example: Creating a SecureFiles LOB with a Specific Encryption Algorithm](#)
The CREATE TABLE statement can create a SecureFiles LOB with encryption specified.
- [Example: Creating a SecureFiles LOB with a Column Password Specified](#)
The CREATE TABLE statement can create a SecureFiles LOB with a column password.

10.4.1 About Transparent Data Encryption and SecureFiles

SecureFiles encryption uses TDE to provide the encryption facility for LOBs.

When you create or alter tables, you can specify the SecureFiles encryption or LOB columns that must use the SecureFiles storage. You can enable the encryption for a LOB column by either using the current Transparent Data Encryption (TDE) syntax or by using the ENCRYPT clause as part of the LOB parameters for the LOB column. The DECRYPT option in the current syntax or the LOB parameters turn off encryption.

10.4.2 Example: Creating a SecureFiles LOB with a Specific Encryption Algorithm

The CREATE TABLE statement can create a SecureFiles LOB with encryption specified.

[Example 10-1](#) shows how to create a SecureFiles LOB in a CREATE TABLE statement.

Example 10-1 Creating a SecureFiles LOB with a Specific Encryption Algorithm

```
CREATE TABLE table1 ( a BLOB ENCRYPT USING 'AES256')
  LOB(a) STORE AS SECUREFILE (
    CACHE
  );
```

10.4.3 Example: Creating a SecureFiles LOB with a Column Password Specified

The CREATE TABLE statement can create a SecureFiles LOB with a column password.

[Example 10-2](#) shows an example of creating a SecureFiles LOB that uses password protections for the encrypted column.

All of the LOBS in the LOB column are encrypted with the same encryption specification.

Example 10-2 Creating a SecureFiles LOB with a Column Password Specified

```
CREATE TABLE table1 (a VARCHAR2(20), b BLOB)
  LOB(b) STORE AS SECUREFILE (
    CACHE
    ENCRYPT USING 'AES192' IDENTIFIED BY password
  );
```

10.5 How Transparent Data Encryption Works with Oracle Call Interface

Transparent Data Encryption does not have any effect on the operation of Oracle Call Interface (OCI).

For most practical purposes, TDE is transparent to OCI except for the row shipping feature. You cannot use the OCI row shipping feature with TDE because the key to make the row usable is not available at the receipt-point.

10.6 How Transparent Data Encryption Works with Editions

Transparent Data Encryption does not have any effect on the Editions feature of Oracle Database.

For most practical purposes, TDE is transparent to Editions. Tables are always noneditioned objects. TDE Column Encryption encrypts columns of the table. Editions are not affected by TDE tablespace encryption.

- [How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT](#)
Many of the clauses from the ALTER SYSTEM statement correspond to the ADMINISTER KEY MANAGEMENT statement.

10.6.1 How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Many of the clauses from the ALTER SYSTEM statement correspond to the ADMINISTER KEY MANAGEMENT statement.

[Table 10-2](#) compares the Transparent Data Encryption usage of the ALTER SYSTEM statement and the orapki utility from previous releases with the ADMINISTER KEY MANAGEMENT statement.

Table 10-2 How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Behavior	ALTER SYSTEM or orapki	ADMINISTER KEY MANAGEMENT
Creating a keystore	For TDE wallets: <pre>ALTER SYSTEM SET ENCRYPTION KEY ["certificate_ID"] IDENTIFIED BY keystore_password;</pre> For external keystores, the keystore is available after you configure the external keystore.	For TDE wallets: <pre>ADMINISTER KEY MANAGEMENT CREATE KEYSTORE IDENTIFIED BY TDE_wallet_password</pre> For external keystores, the keystore is available after you configure the external keystore.
Creating an auto-login keystore	<pre>orapki wallet create -wallet wallet_location -auto_login [-pwd password]</pre>	For TDE wallets: <pre>ADMINISTER KEY MANAGEMENT CREATE [LOCAL] AUTO_LOGIN KEYSTORE FROM KEYSTORE IDENTIFIED BY TDE_wallet_password;</pre> This type of keystore applies to TDE wallets only.
Opening a keystore	<pre>ALTER SYSTEM SET [ENCRYPTION] WALLET OPEN IDENTIFIED BY keystore_password;</pre>	<pre>ADMINISTER KEY MANAGEMENT SET KEYSTORE OPEN [FORCE KEYSTORE] IDENTIFIED BY keystore_password EXTERNAL STORE [CONTAINER = ALL CURRENT];</pre>
Closing a keystore	<pre>ALTER SYSTEM SET [ENCRYPTION] WALLET CLOSE IDENTIFIED BY keystore_password;</pre>	For both software and external keystores: <pre>ADMINISTER KEY MANAGEMENT SET KEYSTORE CLOSE [IDENTIFIED BY keystore_password] [CONTAINER = ALL CURRENT];</pre>

Table 10-2 (Cont.) How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Behavior	ALTER SYSTEM or orapki	ADMINISTER KEY MANAGEMENT
Migrating from an external keystore to a TDE wallet	ALTER SYSTEM SET [ENCRYPTION] KEY IDENTIFIED BY <i>wallet_password</i> REVERSE MIGRATE USING "external_key_manager_password" WITH BACKUP;	ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY IDENTIFIED BY <i>keystore_password</i> REVERSE MIGRATE USING "external_key_manager_password" WITH BACKUP [USING 'backup_identifier'];
Migrating from a TDE wallet to Oracle Key Vault or Oracle Cloud Interface (OCI) Key Management Service (KMS)	ALTER SYSTEM SET [ENCRYPTION] KEY IDENTIFIED BY "Oracle_Key_Vault_password" MIGRATE USING <i>wallet_password</i> ;	ADMINISTER KEY MANAGEMENT SET [ENCRYPTION] KEY IDENTIFIED BY "Oracle_Key_Vault_password" MIGRATE USING <i>wallet_password</i> ;
Changing a keystore password	orapki wallet change_pwd -wallet <i>wallet_location</i> [-oldpwd <i>password</i>] [-newpwd <i>password</i>]	For password-based TDE wallets: ADMINISTER KEY MANAGEMENT ALTER KEYSTORE PASSWORD IDENTIFIED BY <i>TDE_wallet_old_password</i> SET <i>TDE_wallet_new_password</i> [WITH BACKUP [USING 'backup_identifier']]; For external keystores, you close the keystore, change it in the external key manager interface, and then reopen the keystore.
Backing up a password-based TDE wallet	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT BACKUP KEYSTORE [USING 'backup_identifier'] IDENTIFIED BY <i>TDE_wallet_password</i> [TO 'TDE_wallet_location'];
Merging two TDE wallets into a third new keystore	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT MERGE KEYSTORE 'TDE_wallet1_location' [IDENTIFIED BY <i>TDE_wallet1_password</i>] AND KEYSTORE 'TDE_wallet2_location' [IDENTIFIED BY <i>TDE_wallet2_password</i>] INTO NEW KEYSTORE 'TDE_wallet3_location' IDENTIFIED BY <i>TDE_wallet3_password</i> ;
Merging one TDE wallet into another existing keystore	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT MERGE KEYSTORE 'TDE_wallet1_location' [IDENTIFIED BY <i>TDE_wallet1_password</i>] INTO EXISTING KEYSTORE 'TDE_wallet2_location' IDENTIFIED BY <i>TDE_wallet2_password</i> [WITH BACKUP [USING 'backup_identifier']];

Table 10-2 (Cont.) How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Behavior	ALTER SYSTEM or orapki	ADMINISTER KEY MANAGEMENT
Setting or rekeying the master encryption key	ALTER SYSTEM SET [ENCRYPTION] KEY IDENTIFIED BY <i>keystore_password</i> ; Note: The ALTER SYSTEM SET ENCRYPTION KEY statement does not update the V\$ENCRYPTION_KEYS dynamic view after you rekey the encryption key.	ADMINISTER KEY MANAGEMENT SET ENCRYPTION KEY [USING TAG 'tag'] IDENTIFIED BY <i>keystore_password</i> WITH BACKUP [USING 'backup_identifier'] [CONTAINER = ALL CURRENT]; After you rekey the encryption key, the V\$ENCRYPTION_KEYS dynamic view is updated.
Creating a master encryption key for later user	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT CREATE KEY [USING TAG 'tag'] IDENTIFIED BY <i>keystore_password</i> [WITH BACKUP [USING 'backup_identifier']] [CONTAINER = (ALL CURRENT)];
Activating a master encryption key	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT USE KEY 'key_identifier' [USING TAG 'tag'] IDENTIFIED BY <i>keystore_password</i> [WITH BACKUP [USING 'backup_identifier']];
Creating custom tags for master encryption keys	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT SET TAG 'tag' FOR 'master_key_identifier' IDENTIFIED BY <i>keystore_password</i> [WITH BACKUP [USING 'backup_identifier']];
Exporting a master encryption key	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT EXPORT [ENCRYPTION] KEYS WITH SECRET "export_secret" TO 'file_path' IDENTIFIED BY <i>keystore_password</i> [WITH IDENTIFIER IN 'key_id1', 'key_id2', 'key_idn' (SQL_query)]
Importing a master encryption key	No ALTER SYSTEM or orapki equivalent for this functionality	ADMINISTER KEY MANAGEMENT IMPORT [ENCRYPTION] KEYS WITH SECRET "import_secret" FROM 'file_name' IDENTIFIED BY <i>keystore_password</i> [WITH BACKUP [USING 'backup_identifier']];

Table 10-2 (Cont.) How ALTER SYSTEM and orapki Map to ADMINISTER KEY MANAGEMENT

Behavior	ALTER SYSTEM or orapki	ADMINISTER KEY MANAGEMENT
Storing Oracle Database secrets in a keystore	No ALTER SYSTEM or orapki equivalent for this functionality	For TDE wallets: <pre>ADMINISTER KEY MANAGEMENT ADD SECRET UPDATE SECRET DELETE SECRET "secret" FOR CLIENT 'client_identifier' [USING TAG 'tag'] IDENTIFIED BY keystore_password [WITH BACKUP [USING 'backup_identifier'];</pre> For external keystores: <pre>ADMINISTER KEY MANAGEMENT ADD SECRET UPDATE SECRET DELETE SECRET "secret" FOR CLIENT 'client_identifier' [USING TAG 'tag'] IDENTIFIED BY "external_key_manager_password";</pre>

Wallet Conversion Before Switching to Next-Generation provider FIPS 140-3 Mode

If you plan to switch the database to the next-generation provider in FIPS 140-3 mode, verify that the keystore file (ewallet.p12) is encrypted with AES-256. Wallets created before Oracle Database 19.22 may use 3DES for the PKCS#12 container encryption, which is not compatible with FIPS 140-3.

Perform this conversion and verify wallet accessibility before enabling FIPS 140-3 mode. If the wallet cannot be opened after enabling FIPS 140-3, the TDE master encryption key is inaccessible and encrypted data cannot be read.

To check the wallet encryption algorithm:

```
openssl pkcs12 -info -in ./ewallet.p12 -nodes
```

Use the following orapki command to convert the wallet encryption from 3DES to AES-256:

```
orapki wallet convert -wallet wallet_location -compat_v12
```

When operating in FIPS mode, append the -fips140_mode flag:

```
orapki wallet convert -wallet wallet_location -compat_v12 -fips140_mode
```

This command converts the PKCS#12 container encryption to AES-256 while preserving all keys and certificates stored in the wallet. Perform this conversion while the database is still using the legacy provider or the next-generation provider in non-FIPS mode.

The equivalent ADMINISTER KEY MANAGEMENT operation is not available for wallet container encryption conversion. You must use the orapki utility for this operation.

10.7 Configuring Transparent Data Encryption to Work in a Multidatabase Environment

Each Oracle database on the same server (such as databases sharing the same Oracle binary but using different data files) must access its own TDE keystore.

Keystores are not designed to be shared among databases. By design, there must be one keystore per database. You cannot use the same keystore for more than one database.

- To configure the use of keystores in a multidatabase environment, use one of the following options:
 - **Option 1:** Specify the keystore location by individually setting the `WALLET_ROOT` static initialization parameter and the `TDE_CONFIGURATION` dynamic initialization parameter (its `KEYSTORE_CONFIGURATION` attribute set to `FILE`) for each CDB (or standalone database). You must set the `KEYSTORE_CONFIGURATION` attribute to `FILE` in order for the `WALLET_ROOT` parameter to work.

For example:

```
WALLET_ROOT = $ORACLE_BASE/admin/db_unique_name  
TDE_CONFIGURATION="KEYSTORE_CONFIGURATION=FILE"
```

- **Option 2:** If `WALLET_ROOT` and `TDE_CONFIGURATION` are not set, and if the databases share the same Oracle home, then ensure that the `WALLET_LOCATION` and `ENCRYPTION_WALLET_LOCATION` parameters in `sqlnet.ora` are **not** set. By default, `sqlnet.ora` is located in the `$ORACLE_BASE/network/admin` directory (if `$ORACLE_BASE` is set) or in the `$ORACLE_HOME/network/admin` directory.

This enables Oracle Database to use the keystore that is located in either the `$ORACLE_BASE/admin/db_unique_name/wallet` (assuming `$ORACLE_BASE` is set) or the `$ORACLE_HOME/admin/db_unique_name/wallet` directory.

- **Option 3:** If options 1 and 2 are not feasible, then use separate `sqlnet.ora` files, one for each database. Ensure that you correctly set the `TNS_ADMIN` environment variable to point to the correct database configuration. However, be aware that the `ENCRYPTION_WALLET_LOCATION` parameter in `sqlnet.ora` is deprecated, starting with release 19c, in favor of the `WALLET_ROOT` and `TDE_CONFIGURATION` initialization parameters.

Caution

Using a keystore from another database can cause partial or complete data loss.

Related Topics

- [Transparent Data Encryption Keystore Search Order](#)
The search order for the TDE keystore depends on how you have set either the instance initialization parameters, the `sqlnet.ora` parameters, or the environment variables.
- *SQL*Plus User's Guide and Reference*

10.8 RMAN Encryption Algorithm Configuration

RMAN backup encryption on Oracle Database 19c supports AES128, AES192, and AES256 in CFB mode. The default algorithm is AES128. Oracle recommends AES256.

The following algorithms are available for Oracle Database 19c:

Table 10-3 RMAN Backup Encryption Algorithms

Algorithm	Mode	Default	Recommended
AES128	CFB	Yes	No
AES192	CFB	No	No
AES256	CFB	No	Yes

To configure AES256 as the RMAN encryption algorithm:

```
CONFIGURE ENCRYPTION ALGORITHM 'AES256';
```

The RMAN encryption algorithms are the same regardless of whether the database uses the legacy provider or the next-generation provider.

Frequently Asked Questions About Transparent Data Encryption

Users frequently have questions about transparency and performance issues with Transparent Data Encryption.

- [Transparency Questions About Transparent Data Encryption](#)
Transparent Data encryption handles transparency in data in a variety of ways.
- [Performance Questions About Transparent Data Encryption](#)
There are several performance issues to consider when using Transparent Data Encryption.

11.1 Transparency Questions About Transparent Data Encryption

Transparent Data encryption handles transparency in data in a variety of ways.

Security auditors occasionally ask detailed questions about the encryption used by Transparent Data Encryption (TDE). They request information about TDE keys, algorithms, lengths, and keystores and then directly compare to requirements of regulations such as PCI-DSS. This topic contains important details about TDE encryption and key management. This information is current as of Oracle Database 12c (12.1.0.2). It is intended to help TDE customers respond to auditor questions quickly and accurately.

1. Is Transparent Data Encryption compatible with my application software?

Transparent Data Encryption is compatible with applications by default because it does not alter the inbound SQL statements or the outbound SQL query results. Oracle executes internal testing and validation of certain Oracle and third-party application software to capture helpful deployment tips or scripts, and to evaluate performance profiles.

Be aware of the difference between Transparent Data Encryption and the DBMS_CRYPT0 PL/SQL package. This package is intended for different customer use cases. It is an API and toolkit solution and as such, it is non-transparent.

2. Is Transparent Data Encryption compatible with other Oracle Database tools and technologies that I am using?

One of the chief benefits of Transparent Data Encryption is its integration with frequently used Oracle Database tools and technologies such as high-availability clusters, storage compression, backup compression, data movement, database backup and restore, and database replication. Specific Oracle technologies that are integrated directly with Transparent Data Encryption include Oracle Real Application Clusters (Oracle RAC), Oracle Recovery Manager (RMAN), Oracle Data Guard, Advanced Compression, Oracle Data Pump, and Oracle GoldenGate, among others. Transparent Data Encryption also has special points of integration with Oracle Exadata that fully use unique features of Oracle-engineered systems.

Transparent Data Encryption also works easily with security features of the Oracle Database. With Transparent Data Encryption, privilege grants, roles, Oracle Database Vault realms, Virtual Private Database policies, and Oracle Label Security labels remain in effect. You can use these and other security features in tandem with Transparent Data Encryption encryption.

3. Are there any known Transparent Data Encryption limitations or incompatibilities?

- **TDE column encryption:** TDE column encryption encrypts and decrypts data transparently when data passes through the SQL layer. Some features of Oracle will bypass the SQL layer, and hence cannot benefit from TDE column encryption. The following are known database features that TDE column encryption does not support, and their relevant software version numbers:
 - Materialized View Logs (not supported prior to Oracle Database 11g release 2)
 - Synchronous and asynchronous change data capture for data warehousing (CDC)
 - Transportable Tablespaces
 - LOBs

Note that Secure Files were introduced in Oracle Database 11g release 1, so it is not supported with TDE column encryption prior to that release

- **TDE tablespace encryption:** TDE tablespace encryption encrypts all content that is stored in the tablespace at the block level in storage, and it generally does not conflict with other database features. TDE tablespace encryption does not have any of the limitations that TDE column encryption has. However, you can use full transportable tablespaces (TTS) with Oracle Data Pump compression and encryption when going from a TDE-encrypted source to a TDE-encrypted destination. You must have an Oracle Database release 12c or later database instance available so that you can use its key export or keystore (wallet) merge capabilities to get the correct master encryption key to the destination database host without having to overwrite the original Oracle wallet file. This process is subject to the standard TTS limitations, and you must remember to check for compatible endianness.

4. What types of keys and algorithms does TDE use?

TDE uses a two-tier key architecture:

- **Data Encryption Keys (DEKs):** Used to encrypt individual columns or tablespaces. Supported algorithms: AES128, AES192, AES256, 3DES168 (deprecated), ARIA (128, 192, 256), GOST256, and SEED128. The default for column encryption is AES192 (CBC mode). The default for tablespace encryption is AES128 (CFB mode). Oracle recommends AES256 for both.
- **Master Encryption Keys (MEKs):** Used to encrypt the DEKs. Stored in the TDE keystore (wallet or external key manager). The default algorithm is AES256 in CBC mode. Additional algorithms (ARIA256, SEED128, GOST256) are available depending on the cryptographic provider.

When using the next-generation provider in FIPS 140-3 mode, 3DES168 is desupported for DEKs. When using the next-generation provider in any mode, ARIA and SEED are not supported for MEKs. In FIPS mode (both FIPS 140-2 and FIPS 140-3), only AES128, AES192, and AES256 are available for TDE operations. Oracle recommends AES256 for both DEKs and MEKs, as it provides the broadest provider compatibility and all modes of symmetric encryption using AES256 are considered quantum resistant.

5. Does the choice of cryptographic provider affect my existing TDE-encrypted data?

Existing encrypted data remains accessible as long as the encryption algorithm used is supported by the active cryptographic provider and FIPS mode. If you switch to the next-generation provider in FIPS 140-3 mode, any data encrypted with 3DES168 may become inaccessible. Re-encrypt such data with an AES algorithm before switching providers.

Similarly, if your TDE master encryption key uses ARIA256 or SEED128, you must rekey to AES256 before switching to the next-generation provider. Rekeying the master key does not require re-encrypting the data; only the DEK wrappers are re-encrypted with the new MEK.

6. How do I check and upgrade my TDE wallet encryption?

To check what encryption algorithm your wallet uses:

```
openssl pkcs12 -info -in ./ewallet.p12 -nodes
```

Wallets created before Oracle Database 19.22 may use 3DES encryption. To convert to AES-256:

```
orapki wallet convert -wallet wallet_location -compat_v12
```

The converted wallet is compatible with both the legacy provider and the next-generation provider.

7. How are Oracle keystores containing master encryption keys protected?

There are three different types of keystore to consider when you use an Oracle wallet as the keystore for master encryption keys: password-based, auto-login, and local auto-login. All of these keystore externalize master encryption keys, so they are separate from TDE-encrypted data. Oracle recommends that you place wallet files in local or network directories that are protected by tight file permissions and other security measures.

The password-based wallet is an encrypted key storage file (`ewallet.p12`) that follows the PKCS #12 standard. It is encrypted by a password-derived key according to the PKCS #5 standard. A human user must enter a command containing the password for the database to open the wallet, decrypt its contents, and gain access to keys. The password-based wallet is the default keystore for TDE master keys. In the past, it was encrypted using the 3DES168 encryption algorithm and CBC operating mode. The `orapki` command `convert wallet` enables you to convert password-based wallets to AES256 and CBC operating mode. *Oracle Database Security Guide* provides details about using `orapki` to convert wallets.

Auto-login wallets (`cwallet.sso`) optionally are derived from standard password-based wallets for special cases where automatic startup of the database is required with no human interaction to enter a wallet password. When using auto-login wallet, the master password-based wallet must be preserved because it is needed to rekey the master encryption key. In addition to the best practice of storing auto-login wallet in a local or network directory that is protected by tight file permissions, the file contents are scrambled by the database using a proprietary method for added security. A slight variation on the auto-login wallet called local auto-login wallet has similar behavior. One notable difference with local auto-login wallet is that its contents are scrambled using additional factors taken from the host machine where the file was created. This renders the local auto-login wallet unusable on other host machines. Details of the host factors and scrambling technique are proprietary.

8. Do I need to convert my wallet before switching to the next-generation provider?

If your TDE keystore file (`ewallet.p12`) uses 3DES encryption for the PKCS#12 container and you plan to use the next-generation provider in FIPS 140-3 mode, you must convert the wallet encryption to AES-256 before enabling FIPS 140-3.

Wallets created before Oracle Database 19.22 are likely encrypted with 3DES. Wallets created from 19.22 onward use AES-256 by default.

To check your wallet encryption:

```
openssl pkcs12 -info -in ./ewallet.p12 -nodes
```

To convert the wallet:

```
orapki wallet convert -wallet wallet_location -compat_v12
```

This converts the container encryption from 3DES to AES-256 without affecting the keys or certificates inside the wallet. The converted wallet is compatible with both the legacy provider and the next-generation provider.

If you are using the next-generation provider in non-FIPS mode, wallet conversion is recommended but not strictly required.

The wallet passphrase can be up to 128 bytes.

9. What is Oracle Key Vault and how does it manage TDE master keys?

Oracle Key Vault centrally manages TDE master keys, Oracle wallets, Java keystores, and more. It helps you to take control of proliferating keys and key storage files. It includes optimizations specifically for TDE and other components of the Oracle stack. For more information about using Oracle Key Vault with TDE, see the product pages on Oracle Technology Network and *Oracle Key Vault Administrator's Guide*.

11.2 Performance Questions About Transparent Data Encryption

There are several performance issues to consider when using Transparent Data Encryption.

1. What is the typical performance overhead from Transparent Data Encryption?

There are many different variables involved in the creation of an accurate Transparent Data Encryption performance test. The results can vary depending on the test environment, test case or workload, measurement metrics or methods, and so on. Oracle cannot guarantee a specific performance overhead percentage that can apply in all possible scenarios. In practice, the performance tests by many Transparent Data Encryption customers are often in the low single digits as a percentage, but that is not universally the case.

If possible, use Oracle Real Application Testing (Oracle RAT) to capture a real production workload and then replay it against Transparent Data Encryption to get a true indication of the performance overhead that you can expect within your environment.

2. How can I tune for optimal Transparent Data Encryption performance?

- **TDE column encryption:**
 - Limit the crypto processing by only encrypting the subset of columns that are strictly required to be protected. In addition, turn off the optional integrity checking feature.
 - After you apply column encryption, rebuild the column indexes.
- **TDE tablespace encryption:** TDE tablespace encryption improves performance by caching unencrypted data in memory in the SGA buffer cache. This feature reduces the number of crypto operations that must be performed when users run SELECT queries, which draw from the SGA instead of drawing from disk. (Drawing from disk forces the database to perform decrypt operations.) Ensure that the size of the SGA buffer cache is large enough to take full advantage of this performance optimization.

Another major performance boost comes from using hardware and software that supports CPU-based cryptographic acceleration available in Intel AES-NI and Oracle SPARC T4/T5. To take advantage of this feature, you must be running a recent version of the database, have a recent version of the operating system installed, and be using hardware that includes crypto acceleration circuitry within its CPUs/cores.

Database compression further speeds up Transparent Data Encryption performance because the crypto processing occurs on data that already is compressed, resulting in less total data to encrypt and decrypt.

- **In general:**

- Ensure that you have applied the latest patches, which you can download from My Oracle Support at <https://support.oracle.com/portal/>
- When you specify an encryption algorithm, remember that AES is slightly faster than 3DES. Use AES128 where possible. Be aware that the performance benefit is small.
- Use Exadata (described in *Oracle Database Testing Guide*), which includes additional performance benefits.

3. Are there specific issues that may slow down TDE performance, and if so, how do I avoid them?

TDE tablespace performance is slower if the database cannot use CPU-based hardware acceleration on the host machine due to factors such as older hardware, an older database version, or an older operating system.

Note the following with regard to specific database workloads:

- **Encrypting the whole data set at once (for example, while doing “Bulk Data Load” into an Oracle data warehouse):** Lower crypto performance has been observed during bulk load of new data into the database or data warehouse. New data cannot be cached in SGA, so TDE tablespace encryption performance optimizations are bypassed. Hence, Transparent Data Encryption has no bonus performance benefits in this type of operation.

Follow these guidelines:

- Ensure that the database is running on servers with CPU-based cryptographic acceleration. This accelerates not only decrypt operations, but also encrypt operations as well (for loading new data). Take the crypto processing out of band by pre-encrypting the data set and then using Transportable Tablespaces (TTS) to load into the database. Try to parallelize this procedure where possible. This requires the database instance to copy the required TDE key to the keystore on the destination database. The procedure may not be feasible when there is a fixed time window for encryption and loading, and these must be done serially.
- Consider using TDE column encryption. Encrypt only the handful of sensitive regulated columns instead of encrypting an entire tablespace.

- **Decrypting an entire data set at once (for example, while performing a full table scan by reading directly from disk, with no reading from SGA):**

Lower crypto performance is observed when running full table scan queries where data is read directly from storage. Certain performance optimizations of TDE tablespace encryption are bypassed (no caching). Hence, Transparent Data Encryption has no bonus performance benefits in this type of operation.

Follow these guidelines:

- Ensure that the database is running on servers with CPU-based cryptographic acceleration.
- Retest the full table scan queries with a larger SGA size to measure performance when data is read from cache. Try setting the Oracle event number 10949 to disable direct path read.

- Partition the database so that less data is scanned by full table scan operations. Production databases often use partitioning for this kind of scenario (that is, to limit the total amount of data scanned).
- Consider using TDE column encryption. Encrypt only the handful of sensitive regulated columns instead of encrypting an entire tablespace.

Related Topics

- [Performance Overhead of Transparent Data Encryption](#)
Transparent Data Encryption tablespace encryption has small associated performance overhead.
- *Oracle Database Testing Guide*
- *Oracle Database Testing Guide*

Using sqlnet.ora to Configure Transparent Data Encryption Keystores

Configuring TDE with `sqlnet.ora` parameters has been deprecated, but is still possible depending on your database configuration.

- [About the Keystore Location in the sqlnet.ora File](#)
Configuring TDE with the deprecated `sqlnet.ora` parameters is possible, for example when databases that were upgraded to databases that support the new static and dynamic TDE initialization parameters.
- [Configuring the sqlnet.ora File for a Software Keystore Location](#)
The `sqlnet.ora` file can be used to configure the keystore location for a regular file system, for multiple database access, and for use with Oracle Automatic Storage Management (ASM).
- [Example: Configuring a Software Keystore for a Regular File System](#)
You can configure a software keystore for a regular file system.
- [Example: Configuring a TDE Wallet When Multiple Databases Share the Same Host](#)
You can configure multiple databases to share the same host by setting the `WALLET_ROOT` parameter.
- [Example: Configuring a Software Keystore for an Oracle Automatic Storage Management Disk Group](#)
In an Oracle Real Applications Clusters (Oracle RAC) environment, the `WALLET_ROOT` parameter points to a shared directory in Oracle Automatic Storage Management (ASM) that is accessible from all Oracle RAC instances of that database.

12.1 About the Keystore Location in the sqlnet.ora File

Configuring TDE with the deprecated `sqlnet.ora` parameters is possible, for example when databases that were upgraded to databases that support the new static and dynamic TDE initialization parameters.

After you upgrade, Oracle recommends that you reconfigure your databases to use the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters to configure the keystore location. These new parameters are required for PDBs that are in isolated mode.

However, if you do want to configure the `sqlnet.ora` file instead, be aware that the instance initialization file-based approach using the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters is recommended. It avoids the possibility of inconsistent configuration that can arise when using `sqlnet.ora` because each connection to the database re-reads the contents of the `sqlnet.ora` parameter file. With the instance initialization file based approach, because the `WALLET_ROOT` parameter is read only once when the database instance starts, there is no risk of inconsistency. In a multitenant environment, the use of the `WALLET_ROOT` initialization parameter is preferred over the use of `sqlnet.ora` because it enables each tenant PDB to perform independent management operations.

If you have not set the `WALLET_ROOT` and `TDE_CONFIGURATION` initialization parameters, then Oracle Database checks the `sqlnet.ora` file for the directory location of the keystore. This applies to whether the keystore is a software keystore, an external keystore, or an Oracle Key

Vault keystore. You must edit the `sqlnet.ora` file to define a directory location for the keystore that you plan to create. Ensure that this directory exists beforehand. Preferably, this directory should be empty.

Note the following behavior when you must edit the `sqlnet.ora` file in Oracle Real Application Clusters (Oracle RAC) or a multitenant environment:

- **In an Oracle RAC environment:** If you are using the `srvctl` utility and if you want to include environment variables in the `sqlnet.ora` configuration file, then you must set these environment variables in both the operating system and the `srvctl` environment. Oracle recommends that you place the keystore on a shared file system, such as Oracle Automatic Storage Management (ASM) or ACFS.
- **In a multitenant environment:** Only united mode is supported if `sqlnet.ora` is used for TDE configuration in the multitenant environment. Isolated mode is only supported if the `WALLET_ROOT` and `TDE_CONFIGURATION` parameters are used for TDE configuration.

In the `sqlnet.ora` file, you must set the `ENCRYPTION_WALLET_LOCATION` parameter to specify the keystore location. However, be aware that `ENCRYPTION_WALLET_LOCATION` is deprecated, starting with Oracle Database release 19c in favor of using the `WALLET_ROOT` and `TDE_CONFIGURATION` initialization parameters.

By default, the `sqlnet.ora` file is located in the `$ORACLE_HOME/network/admin` location, or in the location set by the `TNS_ADMIN` environment variable. Ensure that you have properly set the `TNS_ADMIN` environment variable to point to the correct `sqlnet.ora` file. When the keystore location is not set in the `sqlnet.ora` file, then the `V$ENCRYPTION_WALLET` view displays the default location. You can check the location and status of the keystore in the `V$ENCRYPTION_WALLET` view.

Related Topics

- [Transparent Data Encryption Keystore Search Order](#)
The search order for the TDE keystore depends on how you have set either the instance initialization parameters, the `sqlnet.ora` parameters, or the environment variables.
- *SQL*Plus User's Guide and Reference*
- *Oracle Database Net Services Reference*

12.2 Configuring the sqlnet.ora File for a Software Keystore Location

The `sqlnet.ora` file can be used to configure the keystore location for a regular file system, for multiple database access, and for use with Oracle Automatic Storage Management (ASM).

- To create a software keystore on a regular file system, use the following format when you edit the `sqlnet.ora` file:

```
ENCRYPTION_WALLET_LOCATION=
(SOURCE=
(METHOD=FILE)
(METHOD_DATA=
(DIRECTORY=path_to_keystore)))
```

If the `path_to_keystore` will contain an environment variable, then set this variable in the environment where the database instance is started and before you start the database. If you are using the `srvctl` utility to start the database, then set the environment variable in the `srvctl` environment as well, using the following command:

```
srvctl setenv database -db database_name -env  
"environment_variable_name=environment_variable_value"
```

- To create a software keystore on a shared file system for Oracle Real Application Clusters (Oracle RAC), use the following format when you edit the `sqlnet.ora` file:

```
ENCRYPTION_WALLET_LOCATION =  
  (SOURCE = (METHOD = FILE)  
    (METHOD_DATA =  
      (DIRECTORY = +DATA/$ORACLE_UNQNAME)))
```

If the `DIRECTORY` setting will include an environment variable, then set this variable in the environment where the database instance is started and before you start the database. If you are using the `srvctl` utility to start the database, then set the environment variable in the `srvctl` environment as well, using a command similar to the following, assuming the name of the Oracle RAC instance is `FINRAC`:

```
srvctl setenv database -db FINRAC -t "ORACLE_UNQNAME=FINRAC,  
ORACLE_BASE=/u01/opt/oracle"
```

Note

Starting with Oracle Database release 19c, the `ENCRYPTION_WALLET_LOCATION` parameter is deprecated. Instead, use the `WALLET_ROOT` static initialization parameter and the `TDE_CONFIGURATION` dynamic initialization parameter to configure the wallet location. `WALLET_ROOT` and `TDE_CONFIGURATION` can be used for a regular file system, multiple database access, and ASM.

12.3 Example: Configuring a Software Keystore for a Regular File System

You can configure a software keystore for a regular file system.

The following example shows how to configure a software keystore location in the `sqlnet.ora` file for a regular file system in which the database name is `orcl`.

```
ENCRYPTION_WALLET_LOCATION=  
  (SOURCE=  
    (METHOD=FILE)  
    (METHOD_DATA=  
      (DIRECTORY=/etc/ORACLE/WALLETS/orcl)))
```

Note

Starting with Oracle Database release 19c, the `ENCRYPTION_WALLET_LOCATION` parameter is deprecated. Instead, use the `WALLET_ROOT` static initialization parameter and the `TDE_CONFIGURATION` dynamic initialization parameter to configure the wallet location.

Related Topics

- [Step 1: Configure the TDE Wallet Location and Type](#)
You must configure the TDE wallet location and type by setting `WALLET_ROOT` in `init.ora` and `TDE_CONFIGURATION` in the database instance.

12.4 Example: Configuring a TDE Wallet When Multiple Databases Share the Same Host

You can configure multiple databases to share the same host by setting the `WALLET_ROOT` parameter.

Because the `WALLET_ROOT` parameter is internal to an Oracle database, you only need to set the parameter to enable multiple databases to share the same host.

For example:

```
ALTER SYSTEM SET WALLET_ROOT = '/etc/ORACLE/KEYSTORES/${ORACLE_SID}' SCOPE = SPFILE;
```

To implement clean separation of individual TDE wallets for each database, add the `ORACLE_SID` (or `ORACLE_UNQNAME` in Oracle Real Application Clusters) into the `WALLET_ROOT` parameter setting.

12.5 Example: Configuring a Software Keystore for an Oracle Automatic Storage Management Disk Group

In an Oracle Real Applications Clusters (Oracle RAC) environment, the `WALLET_ROOT` parameter points to a shared directory in Oracle Automatic Storage Management (ASM) that is accessible from all Oracle RAC instances of that database.

The following example shows you how to set `WALLET_ROOT` and `TDE_CONFIGURATION` for a software keystore in ASM:

```
ALTER SYSTEM SET WALLET_ROOT = '+DATA/unique_name_of_database' SCOPE = SPFILE SID = '*';
```

For example, if you set this value to `+DATA/FINRAC`, then the `/tde` directory is automatically generated when you create a wallet.

Related Topics

- [Configuring Keystores for Automatic Storage Management](#)
You can store a TDE wallet on an Automatic Storage Management (ASM) disk group.

Part II

Using Oracle Data Redaction

Part II describes how to use Oracle Data Redaction.

- [Introduction to Oracle Data Redaction](#)
Oracle Data Redaction is the ability to redact sensitive data in real time.
- [Oracle Data Redaction Features and Capabilities](#)
Oracle Data Redaction provides a variety of ways to redact different types of data.
- [Configuring Oracle Data Redaction Policies](#)
An Oracle Data Redaction policy defines how to redact data in a column based on the table column type and the type of redaction you want to use.
- [Managing Oracle Data Redaction Policies in Oracle Enterprise Manager](#)
Oracle Enterprise Manager Cloud Control (Cloud Control) can manage Oracle Data Redaction policies and formats.
- [Using Oracle Data Redaction with Oracle Database Features](#)
Oracle Data Redaction can be used with other Oracle features, but some Oracle features may have restrictions with regard to Oracle Data Redaction.
- [Security Considerations for Oracle Data Redaction](#)
Oracle provides guidelines for using Oracle Data Redaction.

Introduction to Oracle Data Redaction

Oracle Data Redaction is the ability to redact sensitive data in real time.

- [What Is Oracle Data Redaction?](#)
Redaction is the process of selectively removing or obscuring sensitive or confidential information from documents or databases. Oracle Data Redaction enables you to redact data that is returned from queries issued by applications.
- [When to Use Oracle Data Redaction](#)
Unlike data masking, which masks sensitive data by permanently replacing it with fictitious but realistic data, data redaction masks data as it is received from the database, leaving the data in the database unchanged.
- [Benefits of Using Oracle Data Redaction](#)
Oracle Data Redaction provides several benefits when you use it to protect your data.
- [Target Use Cases for Oracle Data Redaction](#)
Oracle Data Redaction addresses common use case scenarios.

13.1 What Is Oracle Data Redaction?

Redaction is the process of selectively removing or obscuring sensitive or confidential information from documents or databases. Oracle Data Redaction enables you to redact data that is returned from queries issued by applications.

You can redact column data by using one of the following methods:

- **Full redaction.** You redact all of the contents of the column data. The redacted value returned to the querying application user depends on the data type of the column. For example, columns of the NUMBER data type are redacted with a zero (0), and character data types are redacted with a single space.
- **Partial redaction.** You redact a portion of the column data. For example, you can redact a Social Security number with asterisks (*), except for the last 4 digits.
- **Regular expressions.** You can use regular expressions to look for patterns of data to redact. For example, you can use regular expressions to redact email addresses, which can have varying character lengths. It is designed for use with character data only.
- **Random redaction.** The redacted data presented to the querying application user appears as randomly generated values each time it is displayed, depending on the data type of the column.
- **Nullify redaction.** The Nullify redaction type redacts all the data in a column and replaces it with null values.
- **No redaction.** The No redaction type option enables you to test the internal operation of your redaction policies, with no effect on the results of queries against tables with policies defined on them. You can use this option to test the redaction policy definitions before applying them to a production environment.

Oracle Database applies the redaction at runtime when users access the data (that is, at query-execution time). This solution works well in a production system. During the time that the

data is being redacted, all of the data processing is performed normally, and the back-end referential integrity constraints are preserved.

Data redaction can help you comply with industry regulations such as Payment Card Industry Data Security Standard (PCI DSS) by, for example, helping to restrict access to card holder data by business need to know.

Related Topics

- *Oracle Database Security Guide*

13.2 When to Use Oracle Data Redaction

Unlike data masking, which masks sensitive data by permanently replacing it with fictitious but realistic data, data redaction masks data as it is received from the database, leaving the data in the database unchanged.

Data Redaction enables you to easily disguise the data using several different redaction styles.

Oracle Data Redaction is ideal for situations in which you must redact specific characters out of the result set of queries of Personally Identifiable Information (PII) returned to certain application users. For example, you may want to present a U.S. Social Security number that ends with the numbers 4320 as ***-**-4320.

Oracle Data Redaction is particularly suited for call center applications and other applications that are read-only. Take care when using Oracle Data Redaction with applications that perform updates back to the database, because redacted data can be written back to this database.

13.3 Benefits of Using Oracle Data Redaction

Oracle Data Redaction provides several benefits when you use it to protect your data.

These benefits are as follows:

- Applies the redaction at runtime, with no impact on underlying data or storage requirements
- Runs with little or no performance impact
- Requires no application changes
- Enables you to specify and manage Oracle Redaction with scripts or through the Oracle Enterprise Manager user interface
- Is available for both on-premises and cloud databases

13.4 Target Use Cases for Oracle Data Redaction

Oracle Data Redaction addresses common use case scenarios.

- [Oracle Data Redaction for Sensitive Data in Read-Only Static Pages](#)
Oracle Data Redaction enables you to redact sensitive data in application screens that have read-only static pages, such as dashboards and reports.
- [Oracle Data Redaction for Preventing Data Exposure by Management Tools](#)
Oracle Data Redaction prevents sensitive data from being exposed by data management tools, such as tools for loading and viewing data.

- [Oracle Data Redaction to Prevent Disclosure of Data from Offline Analytics](#)
Oracle Data Redaction prevents the disclosure of sensitive data to users who perform offline analytics on production data.
- [Oracle Data Redaction Use with Database Applications](#)
Oracle Data Redaction protects sensitive data that is displayed in database applications.
- [Oracle Data Redaction with Ad Hoc Database Queries Considerations](#)
You may encounter situations where it is convenient to redact sensitive data for ad hoc queries that are performed by database users.

13.4.1 Oracle Data Redaction for Sensitive Data in Read-Only Static Pages

Oracle Data Redaction enables you to redact sensitive data in application screens that have read-only static pages, such as dashboards and reports.

You can use Oracle Data Redaction to dynamically redact data for operational interfaces such as dashboards or reports. You can define a redaction policy on sensitive columns so that the data is redacted before it is passed to the application. Because the pages are for static display only, and data is not posted back to the database, redacted data would not be written back to the database and potentially corrupt records.

13.4.2 Oracle Data Redaction for Preventing Data Exposure by Management Tools

Oracle Data Redaction prevents sensitive data from being exposed by data management tools, such as tools for loading and viewing data.

Many applications include tools that enable users to load and manage their data. An example could be an SaaS application that allows subscribers to bulk load and manage customer information. You can define a redaction policy on the sensitive data so that sensitive data is redacted when it is displayed to these users when they perform these administrative activities.

13.4.3 Oracle Data Redaction to Prevent Disclosure of Data from Offline Analytics

Oracle Data Redaction prevents the disclosure of sensitive data to users who perform offline analytics on production data.

Oracle Data Redaction can be used to prevent the exposure of sensitive information to users who perform analytics on data that is contained in a data warehouse. You can define a redaction policy on sensitive data so that it is redacted as it is retrieved from the database and displayed to the user of the analytics software.

13.4.4 Oracle Data Redaction Use with Database Applications

Oracle Data Redaction protects sensitive data that is displayed in database applications.

Data Redaction is transparent to application users because it preserves the original data type and (optionally) the formatting. It is highly transparent to the database because the data remains the same in buffers, caches, and storage—only being changed at the last minute just before SQL query results are returned to the caller. The redaction is enforced consistently across all of the applications that use the same underlying database. You can specify which application users should see only redacted data by checking application user information that is passed into the database through the SYS_CONTEXT function; you can redact data based on

attributes of the current database or application user; and you can implement multiple logical conditions within a given redaction policy. In addition, Data Redaction is implemented in a way that minimizes performance overhead. These characteristics make Oracle Data Redaction particularly well suited for usage by a range of applications, analytics tools, reporting tools, and monitoring tools that share common production databases. Although its primary target is redaction of production data for applications, Oracle Data Redaction also can be used with custom data administration tools (where they are not issuing ad hoc queries).

Related Topics

- [Oracle Data Redaction and Data Masking and Subsetting Pack](#)
Oracle Enterprise Manager Data Masking and Subsetting Pack can be used to create a development or test copy of a production database.

13.4.5 Oracle Data Redaction with Ad Hoc Database Queries Considerations

You may encounter situations where it is convenient to redact sensitive data for ad hoc queries that are performed by database users.

For example, in the course of supporting a production application, a user may need to run ad hoc database queries to troubleshoot and fix an urgent problem with the application. Even though Oracle Data Redaction is not designed to prevent data exposure to database users who run ad hoc queries directly against the database, it can provide an additional layer to reduce the chances of accidental data exposure. Because such users may have rights to change data, alter the database schema, and circumvent the SQL query interface entirely, it is possible for a malicious user to bypass Data Redaction policies in certain circumstances.

Be aware that Data Redaction does not place any restriction on the WHERE clause of ad hoc SQL, so the WHERE clause can be used in an iterative fashion to infer the actual data even when there is a Data Redaction policy on the queried column and only the redacted value is displayed.

Remember that the Oracle Database security tools are designed to be used together to improve overall security. By deploying one or more of these tools as a complement to Oracle Data Redaction, you can securely increase your overall security posture.

Related Topics

- [Oracle Data Redaction General Security Guidelines](#)
It is important to understand general security guidelines for using Oracle Data Redaction.

Oracle Data Redaction Features and Capabilities

Oracle Data Redaction provides a variety of ways to redact different types of data.

- [Full Data Redaction to Redact All Data](#)
Full data redaction redacts the entire contents of the specified table or view column.
- [Partial Data Redaction to Redact Sections of Data](#)
In partial data redaction, you redact portions of the displayed output.
- [Regular Expressions to Redact Patterns of Data](#)
Regular expressions redact specific data within a column data value, based on a pattern search.
- [Redaction Using Null Values](#)
You can create an Oracle Data Redaction policy that redacts column data by replacing it with null values.
- [Random Data Redaction to Generate Random Values](#)
In random data redaction, the entire value is redacted by replacing it with a random value.
- [Comparison of Full, Partial, and Random Redaction Based on Data Types](#)
The full, partial, and random data redaction styles affect the Oracle built-in, ANSI, user-defined, and Oracle supplied types in different ways.
- [No Redaction for Testing Purposes](#)
You can create a Data Redaction policy that does not perform redaction.
- [Central Management of Named Data Redaction Policy Expressions](#)
You can create a library of named policy expressions that can be used in the columns of multiple tables and views.

14.1 Full Data Redaction to Redact All Data

Full data redaction redacts the entire contents of the specified table or view column.

By default the output is displayed as follows:

- **Character data types:** The output text is a single space.
- **Number data types:** The output text is a zero (0).
- **Date-time data types:** The output text is set to the first day of January, 2001, which appears as 01-JAN-01.

Full redaction is the default and is used whenever a Data Redaction policy specifies the column but omits the `function_type` parameter setting. When you run the `DBMS_REDACT.ADD_POLICY` procedure, to set the `function_type` parameter setting for full redaction, you enter the following setting:

```
function_type    => DBMS_REDACT.FULL
```

You can use the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure to change the full redaction output to different values.

Related Topics

- [Syntax for Creating a Full Redaction Policy](#)
The `DBMS_REDACT.ADD_POLICY` procedure enables you to create a full redaction policy.

14.2 Partial Data Redaction to Redact Sections of Data

In partial data redaction, you redact portions of the displayed output.

You can set the position within the [actual data](#) at which to begin the redaction, the number of characters to redact starting from that position, and the redaction character to use. This type of redaction is useful for situations where you want it to be obvious to the person viewing the data that it was redacted in some way. Typically, you use this type of redaction for credit cards or ID numbers.

Be aware that partial data redaction requires that your data width remain fixed. If you want to redact columns containing string values of variable length, then you must use regular expressions.

To specify partial redaction, you must set the `DBMS_REDACT.ADD_POLICY` procedure `function_type` parameter to `DBMS_REDACT.PARTIAL` and use the `function_parameters` parameter to define the partial redaction behavior.

The displayed output for partial data redaction can be as follows:

- **Character data types:** When partially redacted, a Social Security number (represented as a hyphenated string within a character data type) with value 987-65-4320 could be redacted so that it is displayed as shown in the following examples. The code on the right specifies how to redact the character data: it specifies the expected input format of the actual data, the format to use for the display of the redacted output, the start position at which to begin the redaction, the character to use for the redaction, and how many characters to redact. The first example uses a predefined format (in previous releases called a shortcut) for character data type Social Security numbers, and the second example replaces the first five numbers with an asterisk (*) while preserving the hyphens (-) in between the numbers.

```
XXX-XX-4320    function_parameters => DBMS_REDACT.REDACT_US_SSN_F5,
```

```
***-**-4320    function_parameters => 'VVVFVVVVVV,VV-VV-VVV,*,1,5',
```

- **Number data types:** The partially redacted NUMBER data type Social Security number 987654328 could appear as follows. Both redact the first five digits. The first example uses a predefined format that is designed for Social Security numbers in the NUMBER data type, and the second replaces the first five numbers with the number 9, starting from the first digit.

```
XXXXX4328     function_parameters => DBMS_REDACT.REDACT_NUM_US_SSN_F5,
```

```
999994328     function_parameters => '9,1,5',
```

- **Date-time data types:** Partially redacted datetime values can appear simply as different dates. For example, the date 29-AUG-11 10.20.50.000000 AM could appear as follows. In the first example, the day of the month is redacted to 02 (using the setting d02) and in the second example, the month is redacted to DEC (using m12). The uppercase values show the actual month (M), year (Y), hour (H), minute (M), and second (S).

```
02-AUG-11 10.20.50.000000 AM function_parameters => 'Md02YHMS',
29-DEC-11 10.20.50.000000 AM function_parameters => 'm12DYHMS',
```

Related Topics

- [Regular Expressions to Redact Patterns of Data](#)
Regular expressions redact specific data within a column data value, based on a pattern search.
- [Syntax for Creating a Partial Redaction Policy](#)
The DBMS_REDACT.ADD_POLICY statement enables you to create policies that redact specific parts of the data returned to the application.

14.3 Regular Expressions to Redact Patterns of Data

Regular expressions redact specific data within a column data value, based on a pattern search.

For example, you can redact the user name of email addresses, so that only the domain shows (for example, replacing hpreston in the email address hpreston@example.com with [redacted] so that it appears as [redacted]@example.com). To perform the redaction, set the DBMS_REDACT.ADD_POLICY procedure function_type parameter to either DBMS_REDACT.REGEXP or DBMS_REDACT.REGEXP_WIDTH, and then use the following parameters to build the regular expression:

- A string search pattern (that is, the values to search for), such as:

```
regexp_pattern      => '(.+)\.(.+)\.[A-Za-z]{2,4}'
```

This setting looks for a pattern of the following form:

```
one_or_more_characters@one_or_more_characters.2-4_characters_in_range_A-Z_or_a-z
```

- A replacement string, which replaces the value matched by the regexp_pattern setting. The replacement string can include back references to sub-expressions of the main regular expression pattern. The following example replaces the data before the @ symbol (from the regexp_pattern setting) with the text [redacted]. The \2 setting refers to the second match group, which is (.+)\.[A-Za-z]{2,4} from the regexp_pattern setting.

```
regexp_replace_string => '[redacted]@\2'
```

- The starting position for the string search string, such as the first character of the data, such as:

```
regexp_position      => DBMS_REDACT.RE_BEGINNING
```

- The kind of search and replace operation to perform, such as the first occurrence, every fifth occurrence, or all of the occurrences, such as:

```
regexp_occurrence    => DBMS_REDACT.RE_ALL
```

- The default matching behavior for the search and replace operation, such as whether the search is case-sensitive (i sets it to be not case-sensitive):

```
regexp_match_parameter => 'i'
```

In addition to the default parameters, you can use a set of predefined formats that enable you to use commonly used regular expressions for telephone numbers, email addresses, and credit card numbers.

Related Topics

- [Syntax for Creating a Regular Expression-Based Redaction Policy](#)
The `regexp_*` parameters of the `DBMS_REDACT.ADD_POLICY` procedure can create a regular expression-based redaction policy.

14.4 Redaction Using Null Values

You can create an Oracle Data Redaction policy that redacts column data by replacing it with null values.

This feature enables you to use the `DBMS_REDACT.NULLIFY` function to hide all of the sensitive data in a table or view column and replace it with null values. You can set this function by using the `function_type` parameter of the `DBMS_REDACT.ADD_POLICY` or `DBMS_REDACT.ALTER_POLICY` procedure.

For example:

```
function_type      => DBMS_REDACT.NULLIFY
```

Related Topics

- [Creating a DBMS_REDACT.NULLIFY Redaction Policy](#)
You can create Oracle Data Redaction policies that return null values for the displayed value of the table or view column.

14.5 Random Data Redaction to Generate Random Values

In random data redaction, the entire value is redacted by replacing it with a random value.

The redacted values displayed in the result set of the query change randomly each time application users run the query.

This type of redaction is useful in cases where you do not want it to be obvious that the data was redacted. It works especially well for number and datetime data types, where it is difficult to distinguish between random and real data.

The displayed output for random values changes based on the data type of the redacted column, as follows:

- **Character data types:** The random output is a mixture of characters (for example, HTU[G{\pjkEWcK). It behaves differently for the `CHAR` and `VARCHAR2` data types, as follows:
 - **CHAR data type:** The redacted output is always in the same character set as the character set of the column. The byte length of the redacted output is always the same as the column definition length (that is, the column length that was provided at the time of table creation). For example, if the column is `CHAR(20)`, then a string of 20 random characters is provided in the redacted output of the user's query.
 - **VARCHAR2 data type:** For random redaction of a `VARCHAR` data type, the redacted output is always in the same character set as the character set of the column. The length of the redacted output is limited based on the length of the [actual data](#) in the column. No characters in excess of the length of the actual data are displayed. For example, if the column is `VARCHAR2(20)` and the row being redacted contains actual data with a length of 12, then a string of 12 random characters (not 20) is provided in the redacted output of the user's query for that row.
- **Number data types:** Each actual number value is redacted by replacing it with a random, non-negative number modulo the absolute value of the actual data. This redaction results

in random numbers that do not exceed the precision of the actual data. For example, the number 987654321 can be redacted by replacing it with any of the numbers 12345678, 13579, 0, or 987654320, but not by replacing it with any of the numbers 987654321, 99987654321, or -1. The number -123 could be redacted by replacing it with the numbers 122, 0, or 83, but not by replacing it with any of the numbers 123, 1123, or -2.

The only exception to the above is when the actual value is an integer between -1 and 9. In this case, the actual data is redacted by replacing it with a random, non-negative integer modulo ten (10).

- **Date-time data types:** When values of the date data type are redacted using random Data Redaction, Oracle Database displays them with random dates that are always different from those of the actual data.

The setting for using random redaction is as follows:

```
function_type    => DBMS_REDACT.RANDOM
```

Related Topics

- [Syntax for Creating a Random Redaction Policy](#)
A random redaction policy presents the redacted data to the querying application user as randomly generated values, based on the column data type.

14.6 Comparison of Full, Partial, and Random Redaction Based on Data Types

The full, partial, and random data redaction styles affect the Oracle built-in, ANSI, user-defined, and Oracle supplied types in different ways.

- [Oracle Built-in Data Types Redaction Capabilities](#)
Oracle Data Redaction handles the Oracle built-in data types depending on the type of Data Redaction policies that are used.
- [ANSI Data Types Redaction Capabilities](#)
Oracle Data Redaction converts ANSI data types in specific ways, depending on the type of redaction that the Data Redaction policy has.
- [Built-in and ANSI Data Types Full Redaction Capabilities](#)
For full redaction, the default redacted value depends on whether the data type is Oracle built-in or ANSI.
- [User-Defined Data Types or Oracle Supplied Types Redaction Capabilities](#)
Several data types or types are not supported by Oracle Data Redaction.

14.6.1 Oracle Built-in Data Types Redaction Capabilities

Oracle Data Redaction handles the Oracle built-in data types depending on the type of Data Redaction policies that are used.

[Table 14-1](#) describes the Oracle Data Redaction support for Oracle built-in data types.

Table 14-1 Redaction Support for Oracle Built-in Data Types

Column Data Type	Full	Partial	Regexp	Random
Character ¹	Yes	Yes	Yes	Yes

Table 14-1 (Cont.) Redaction Support for Oracle Built-in Data Types

Column Data Type	Full	Partial	Regexp	Random
Number ²	Yes	Yes	No	Yes
Raw ³	No	No	No	No
Date-time ⁴	Yes	Yes	No	Yes
Interval ⁵	No	No	No	No
BFILE	No	No	No	No
BLOB	Yes	No	No	No
CLOB	Yes	No	Yes	No
NCLOB	Yes	No	Yes	No
ROWID	No	No	No	No
UROWID	No	No	No	No

¹ Includes CHAR, VARCHAR2 (including long VARCHAR2, for example, VARCHAR2(20000)), NCHAR, NVARCHAR2

² Includes NUMBER, FLOAT, BINARY_FLOAT, BINARY_DOUBLE

³ Includes LONG RAW, RAW

⁴ Includes DATE, TIMESTAMP, TIMESTAMP WITH TIME ZONE, TIMESTAMP WITH LOCAL TIME ZONE

⁵ Includes INTERVAL YEAR TO MONTH, INTERVAL DAY TO SECOND

14.6.2 ANSI Data Types Redaction Capabilities

Oracle Data Redaction converts ANSI data types in specific ways, depending on the type of redaction that the Data Redaction policy has.

[Table 14-2](#) compares how the full, partial, and random redaction styles work for ANSI data types, with regard to how they are converted and their support status.

Table 14-2 Redaction Support for the ANSI Data Types

Data Type	How Converted	Full Redaction	Partial Redaction	Regexp	NULL Redaction	Random Redaction
CHARACTER(n), CHAR(n)	Converted to CHAR(n)	Yes	Yes	Yes	Yes	Yes
CHARACTER VARYING(n), CHAR VARYING(n)	Converted to VARCHAR2(n)	Yes	Yes	Yes	Yes	Yes
NATIONAL CHARACTER(n), NATIONAL CHAR(n), NCHAR(n)	Converted to NCHAR(n)	Yes	Yes	Yes	Yes	Yes
NATIONAL CHARACTER VARYING(n), NATIONAL CHAR VARYING(n), NCHAR VARYING(n)	Converted to NVARCHAR 2(n)	Yes	Yes	Yes	Yes	Yes

Table 14-2 (Cont.) Redaction Support for the ANSI Data Types

Data Type	How Converted	Full Redaction	Partial Redaction	Regexp	NULL Redaction	Random Redaction
NUMERIC(<i>p, s</i>) DECIMAL(<i>p, s</i>)	Converted to NUMBER(<i>p, s</i>)	Yes	Yes	Yes	Yes	Yes
INTEGER, INT, SMALLINT	Converted to NUMBER(38)	Yes	Yes	Yes	Yes	Yes
FLOAT, DOUBLE PRECISION	Converted to FLOAT(126)	Yes	Yes	Yes	Yes	Yes
REAL	Converted to FLOAT(63)	Yes	Yes	Yes	Yes	Yes
GRAPHIC, LONG VARCHAR, VARCHAR2, TIME	No conversion	No	No	No	No	No

14.6.3 Built-in and ANSI Data Types Full Redaction Capabilities

For full redaction, the default redacted value depends on whether the data type is Oracle built-in or ANSI.

[ANSI Data Types Redaction Capabilities](#) shows the default settings for both Oracle built-in and ANSI data type columns that use full redaction.

Table 14-3 Default Settings and Categories for Columns That Use Full Redaction

Data Type	Default Redacted Value	Data Type Category
CHARACTER	Single space (" ")	Oracle built-in
CHARACTER(<i>n</i>), CHAR(<i>n</i>)	Single space (" ")	ANSI
CHARACTER VARYING(<i>n</i>), CHAR VARYING(<i>n</i>)	Single space (" ")	ANSI
NATIONAL CHARACTER(<i>n</i>), NATIONAL CHAR(<i>n</i>), NCHAR(<i>n</i>)	Single space (" ")	ANSI
NATIONAL CHARACTER VARYING(<i>n</i>), NATIONAL CHAR VARYING(<i>n</i>), NCHAR VARYING(<i>n</i>)	Single space (" ")	ANSI
NUMBER	Zero (0)	Oracle built-in
NUMERIC(<i>p, s</i>) DECIMAL(<i>p, s</i>)	Zero (0)	Oracle built-in

Table 14-3 (Cont.) Default Settings and Categories for Columns That Use Full Redaction

Data Type	Default Redacted Value	Data Type Category
INTEGER, INT, SMALLINT	Zero (0)	ANSI
FLOAT, DOUBLE PRECISION	Zero (0)	ANSI
REAL	Zero (0)	ANSI
DATE-TIME	01-01-01 or 01-01-01 01:00:00	Oracle built-in
BLOB	Oracle's raw representation of [redacted] 1	Oracle built-in
CLOB	[redacted]	Oracle built-in
NCLOB	[redacted]	Oracle built-in

¹ If you have changed the character set, then you may need to invoke the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure to set the value to the raw representation in the new character set, as follows:

```
DECLARE
  new_red_blob BLOB;
BEGIN
  DBMS_LOB.CREATETEMPORARY(new_red_blob, TRUE);
  DBMS_LOB.WRITE(new_red_blob, 10, 1, UTL_RAW.CAST_TO_RAW('[redacted]'));
  dbms_redact.update_full_redaction_values(
    blob_val      => new_red_blob);
  DBMS_LOB.FREETEMPORARY(new_red_blob);
END;
/
```

After you run this procedure, restart the database.

See also [Altering the Default Full Data Redaction Value](#) for more information about using the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure.

14.6.4 User-Defined Data Types or Oracle Supplied Types Redaction Capabilities

Several data types or types are not supported by Oracle Data Redaction.

[Table 14-4](#) compares how the full, partial, regular expression, and random redaction styles work for user-defined and Oracle-supplied types.

Table 14-4 Redaction Support for the User-Defined Data Types or Oracle-Supplied Types

Data Type or Type	Full Redaction	Partial Redaction	Regexp	NULL Redaction	Random Redaction
User-defined data types	No	No	No	No	No
Oracle supplied types: Any types, XML types, Oracle Spatial types	No	No	No	No	No

14.7 No Redaction for Testing Purposes

You can create a Data Redaction policy that does not perform redaction.

This is useful for cases in which you have a redacted base table, yet you want a specific application user to have a view that always shows the [actual data](#). You can create a new view of the redacted table and then define a Data Redaction policy for this view. The policy still exists on the base table, but no redaction is performed when the application queries using the view as long as the `DBMS_REDACT.NONE` function_type setting was used to create a policy on the view.

14.8 Central Management of Named Data Redaction Policy Expressions

You can create a library of named policy expressions that can be used in the columns of multiple tables and views.

By having named policy expressions, you can centrally manage all of the policy expressions within a database.

When you modify the policy expression, the change is reflected in all table columns that use the expression. The policy expression takes precedence over the expression setting in the Data Redaction policy. To create the policy expression, you must use the `DBMS_REDACT.CREATE_POLICY_EXPRESSION` procedure, and to apply the policy expression to a column, you use `DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL`. This feature provides flexibility to redact different columns in a table or view, based on different runtime conditions.

For example, consider a use case that involves a customer care application. A customer calls the customer care center to request a return on a recent purchase. A level 1 support representative of the call center must first verify the order ID, customer name, and customer address before initiating the return. During the process, there is no need for the level 1 support representative to view the customer's credit card number. So, the credit card column is redacted when the support representative queries the customer details in the call center application. When the return is initiated, a sales representative from the return department may need to view the credit card number to process the return. However, there is no need for the sales representative to view the expiration date of the credit card. So, when the sales representative queries the customer details in the same application, the credit card number is visible but the expiration date is redacted.

In this use case, different columns in the customer details table must be redacted in different ways, based on who the logged in user is. Oracle Data Redaction simplifies the implementation of this use case by using named Data Redaction policy expressions. This type of policy expression enables you to define and associate different policy expressions on different columns in the same table or view. Moreover, you can centrally manage named policy

expressions within a database. Any updates that you make to a named policy expression are immediately propagated to all of the associated table or view columns.

Related Topics

- [Creating and Managing Multiple Named Policy Expressions](#)

A named, centrally managed Oracle Data Redaction policy expression can be used in multiple redaction policies and applied to multiple tables or views.

Configuring Oracle Data Redaction Policies

An Oracle Data Redaction policy defines how to redact data in a column based on the table column type and the type of redaction you want to use.

- [About Oracle Data Redaction Policies](#)
An Oracle Data Redaction policy defines the conditions in which redaction must occur for a table or view.
- [Who Can Create Oracle Data Redaction Policies?](#)
Because data redaction involves the protection of highly sensitive data, only trusted users should create Oracle Data Redaction policies.
- [Planning an Oracle Data Redaction Policy](#)
Before you create a Oracle Data Redaction policy, you should plan the data redaction policy that best suits your site's needs.
- [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#)
To create a Data Redaction policy, you must use the DBMS_REDACT.ADD_POLICY procedure.
- [Using Expressions to Define Conditions for Data Redaction Policies](#)
The expression parameter in the DBMS_REDACT.ADD_POLICY procedure sets the conditions to which the policy applies.
- [Creating and Managing Multiple Named Policy Expressions](#)
A named, centrally managed Oracle Data Redaction policy expression can be used in multiple redaction policies and applied to multiple tables or views.
- [Creating a Full Redaction Policy and Altering the Full Redaction Value](#)
You can create a full redaction policy to redact all contents in a data column, and optionally, you can alter the default full redaction value.
- [Creating a DBMS_REDACT.NULLIFY Redaction Policy](#)
You can create Oracle Data Redaction policies that return null values for the displayed value of the table or view column.
- [Creating a Partial Redaction Policy](#)
In partial data redaction, you can redact portions of data, and for different kinds of data types.
- [Creating a Regular Expression-Based Redaction Policy](#)
A regular expression-based redaction policy enables you to redact data based on a search-and-replace model.
- [Creating a Random Redaction Policy](#)
A random redaction policy presents redacted data as randomly generated values, such as Ukjs132[[]]s.
- [Creating a Policy That Uses No Redaction](#)
You can create policies that use no redaction at all, for when you want to test the policy in a development environment.
- [Exemption of Users from Oracle Data Redaction Policies](#)
You can exempt users from having Oracle Data Redaction policies applied to the data they access.

- [Altering an Oracle Data Redaction Policy](#)
The `DBMS_REDACT.ALTER_POLICY` procedure enables you to modify Oracle Data Redaction policies.
- [Redacting Multiple Columns](#)
You can redact more than one column in a Data Redaction policy.
- [Disabling and Enabling an Oracle Data Redaction Policy](#)
You can disable and then reenable Oracle Data Redactions policies as necessary.
- [Dropping an Oracle Data Redaction Policy](#)
The `DBMS_REDACT.DROP_POLICY` procedure drops Oracle Data Redaction policies.
- [Tutorial: SQL Expressions to Build Reports with Redacted Values](#)
SQL expressions can be used to build reports based on columns that have Oracle Data Redaction policies defined on them.
- [Using Trace Files to Troubleshoot Oracle Data Redaction Policies](#)
Trace files for Oracle Data Redaction can be generated for either the system level or the session level.
- [Oracle Data Redaction Policy Data Dictionary Views](#)
Oracle Database provides data dictionary views that list information about Data Redaction policies.

15.1 About Oracle Data Redaction Policies

An Oracle Data Redaction policy defines the conditions in which redaction must occur for a table or view.

A Data Redaction policy has the following characteristics:

- The Data Redaction policy defines the following: What kind of redaction to perform, how the redaction should occur, and when the redaction takes place. Oracle Database performs the redaction at execution time, just before the data is returned to the application.
- A Data Redaction policy can fully redact values, partially redact values, or randomly redact values. In addition, you can define a Data Redaction policy to not redact any data at all, for when you want to test your policies in a test environment.
- A Data Redaction policy can be defined with a policy expression which allows for different application users to be presented with either redacted data or [actual data](#), based on whether the policy expression returns TRUE or FALSE. Redaction takes place when the boolean result of evaluating the policy expression is TRUE. For security reasons, the functions and operators that can be used in the policy expression are limited to `SYS_CONTEXT` and a few others. User-created functions are not allowed. Policy expressions can make use of the `SYS_SESSION_ROLES` namespace with the `SYS_CONTEXT` function to check for enabled roles.
- Different Data Redaction policy expressions can be created and then applied individually for different columns within the same table or view.

[Table 15-1](#) lists the procedures in the `DBMS_REDACT` package.

Table 15-1 DBMS_REDACT Procedures

Procedure	Description
<code>DBMS_REDACT.ADD_POLICY</code>	Adds a Data Redaction policy to a table or view
<code>DBMS_REDACT.ALTER_POLICY</code>	Modifies a Data Redaction policy

Table 15-1 (Cont.) DBMS_REDACT Procedures

Procedure	Description
DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL	Applies a Data Redaction policy expression to a table or view column
DBMS_REDACT.CREATE_POLICY_EXPRESSION	Creates a Data Redaction policy expression
DBMS_REDACT.DISABLE_POLICY	Disables a Data Redaction policy
DBMS_REDACT.DROP_POLICY	Drops a Data Redaction policy
DBMS_REDACT.DROP_POLICY_EXPRESSION	Drops a Data Redaction policy expression
DBMS_REDACT.ENABLE_POLICY	Enables a Data Redaction policy
DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES	Globally updates the full redaction value for a given data type. You must restart the database instance before the updated values can be used.
DBMS_REDACT.UPDATE_POLICY_EXPRESSION	Updates a Data Redaction policy expression

See Also

- *Oracle Database PL/SQL Packages and Types Reference* for detailed information about the DBMS_REDACT PL/SQL package
- [Managing Oracle Data Redaction Policies in Oracle Enterprise Manager](#) for information about using Oracle Enterprise Manager Cloud Control to create and manage Oracle Data Redaction policies and formats

15.2 Who Can Create Oracle Data Redaction Policies?

Because data redaction involves the protection of highly sensitive data, only trusted users should create Oracle Data Redaction policies.

To create or manage Oracle Data Redaction policies, you must have the EXECUTE privilege on the DBMS_REDACT PL/SQL package.

In addition to the EXECUTE privilege on the DBMS_REDACT package, you must have the following system privileges:

- For redaction policies on objects in your own schema, you must have the CREATE TABLE or CREATE ANY TABLE system privilege.
- For redaction policies on objects in another user's schema, you must have the CREATE ANY TABLE system privilege.

These privilege requirements apply when you use the DBMS_REDACT package to create, alter, enable, disable, drop, or otherwise manage Oracle Data Redaction policies and policy expressions.

You do not need any privileges to access the underlying tables or views that will be protected by the policy.

To find the system privileges that a user has been granted, you can query the `DBA_SYS_PRIVS` data dictionary view.

Related Topics

- *Oracle Database PL/SQL Packages and Types Reference*

15.3 Planning an Oracle Data Redaction Policy

Before you create a Oracle Data Redaction policy, you should plan the data redaction policy that best suits your site's needs.

1. Ensure that you have the privileges required to create and manage the Data Redaction policy. You must have the `EXECUTE` privilege on the `DBMS_REDACT` PL/SQL package. In addition, for policies on objects in your own schema, you must have the `CREATE TABLE` or `CREATE ANY TABLE` system privilege; for policies on objects in another user's schema, you must have the `CREATE ANY TABLE` system privilege.
2. Determine the data type of the table or view column that you want to redact.
3. Determine if the base object to which you want to add the Data Redaction policy has dependent objects. If it does have dependent objects, then these objects will become invalid when the Data Redaction policy is added to the base object, and these objects will be recompiled automatically when they are used.

Alternatively, you can proactively recompile them yourself by using an `ALTER . . . COMPILE` statement. Be aware that invalidating dependent objects by adding a Data Redaction policy on their base object, and causing them to need to be recompiled, can decrease performance in the overall system. Oracle recommends that you only add a Data Redaction policy to an object that has dependent objects during off-peak hours or during a scheduled downtime.

4. Ensure that this column is not used in an Oracle Virtual Private Database (VPD) row filtering condition. That is, it must not be part of the VPD predicate generated by the VPD policy function.
5. Decide on the type of redaction that you want to perform: full, random, partial, regular expressions, or none.
6. Decide which users to apply the Data Redaction policy to.
7. Based on this information, create the Data Redaction policy by using the `DBMS_REDACT.ADD_POLICY` procedure.
8. Configure the policy to have additional columns to be redacted.

After you create the Data Redaction policy, it is automatically enabled and ready to redact data.

Related Topics

- [Redacting Multiple Columns](#)
You can redact more than one column in a Data Redaction policy.

15.4 General Syntax of the `DBMS_REDACT.ADD_POLICY` Procedure

To create a Data Redaction policy, you must use the `DBMS_REDACT.ADD_POLICY` procedure.

The complete syntax for the `DBMS_REDACT.ADD_POLICY` procedure is as follows:

```

DBMS_REDACT.ADD_POLICY (
  object_schema      IN VARCHAR2 := NULL,
  object_name        IN VARCHAR2 := NULL,
  policy_name        IN VARCHAR2,
  policy_description  IN VARCHAR2 := NULL,
  column_name        IN VARCHAR2 := NULL,
  column_description  IN VARCHAR2 := NULL,
  function_type      IN BINARY_INTEGER := DBMS_REDACT.FULL,
  function_parameters IN VARCHAR2 := NULL,
  expression         IN VARCHAR2,
  enable             IN BOOLEAN := TRUE,
  regexp_pattern     IN VARCHAR2 := NULL,
  regexp_replace_string IN VARCHAR2 := NULL,
  regexp_position    IN BINARY_INTEGER := 1,
  regexp_occurrence  IN BINARY_INTEGER := 0,
  regexp_match_parameter IN VARCHAR2 := NULL);

```

In this specification:

- **object_schema:** Specifies the schema of the object on which the Data Redaction policy will be applied. If you omit this setting (or enter NULL), then Oracle Database uses the current user's name. Be aware that the meaning of "current user" here can change, depending on where you invoke the DBMS_REDACT.ADD_POLICY procedure.

For example, suppose user `mpike` grants user `fbrown` the EXECUTE privilege on a definer's rights PL/SQL package called `mpike.protect_data` in `mpike`'s schema. From within this package, `mpike` has coded a procedure called `protect_cust_data`, which invokes the DBMS_REDACT.ADD_POLICY procedure. User `mpike` has set the `object_schema` parameter to NULL.

When `fbrown` invokes the `protect_cust_data` procedure in the `mpike.protect_data` package, Oracle Database attempts to define the Data Redaction policy around the object `cust_data` in the `mpike` schema, not the `cust_data` object in the schema that belongs to `fbrown`.

- **object_name:** Specifies the name of the table or view to which the Data Redaction policy applies.
- **policy_name:** Specifies the name of the policy to be created. Ensure that this name is unique in the database instance. You can find a list of existing Data Redaction policies by querying the POLICY_NAME column of the REDACTION_POLICIES data dictionary view.
- **policy_description:** Specifies a brief description of the purpose of the policy.
- **column_name:** Specifies the column whose data you want to redact. Note the following:
 - **You can apply the Data Redaction policy to multiple columns.** If you want to apply the Data Redaction policy to multiple columns, then after you use DBMS_REDACT.ADD_POLICY to create the policy, run the DBMS_REDACT.ALTER_POLICY procedure as many times as necessary to add each of the remaining required columns to the policy. See [Altering an Oracle Data Redaction Policy](#).
 - **Only one policy can be defined on a table or view.** You can, however, create a new view on the table, and by defining a second redaction policy on this new view, you can choose to redact the columns in a different way when a query is issued against this new view. When deciding how to redact a given column, Oracle Database uses the policy of the earliest view in a view chain.
 - **If you do not specify a column (for example, by entering NULL), then no columns are redacted by the policy.** This enables you to create your policies so that they are in place, and then later on, you can add the column specification when you are ready.

- **Do not use a column that is currently used in an Oracle Virtual Private Database (VPD) row filtering condition.** In other words, the column should not be part of the VPD predicate generated by the VPD policy function. (See [Oracle Data Redaction and Oracle Virtual Private Database](#) for more information about using Data Redaction with VPD.)
- **You cannot define a Data Redaction policy on a virtual column.** In addition, you cannot define a Data Redaction policy on a column that is involved in the SQL expression of any virtual column. If you are using the DBMS_STATS PL/SQL package to collect database statistics, then be aware that DBMS_STATS will add an invisible virtual column to the table internally when you create column groups. As a result, Oracle Database returns an ORA-28073: The column *column_name* has an unsupported data type or attribute error if you add a Data Redaction policy on a column that is also the column that is used to create column groups.

A user with access to the DBA_STAT_EXTENSIONS data dictionary view can look in the EXTENSION column of DBA_STAT_EXTENSIONS using the following query to find the virtual column expression (that is internally created by DBMS_STATS), to see if it contains any column name that is being provided to a DBMS_REDACT.ADD_POLICY call to create a Data Redaction policy.

```
SELECT EXTENSION_NAME, EXTENSION, CREATOR
FROM DBA_STAT_EXTENSIONS
WHERE TABLE_NAME='table_name_that_you_attempted_to_redact';
```

- **column_description:** Specifies a brief description of the column that you are redacting.
- **function_type:** Specifies a function that sets the type of redaction. See the following sections for more information:
 - [Syntax for Creating a Full Redaction Policy](#)
 - [Syntax for Creating a Partial Redaction Policy](#)
 - [Syntax for Creating a Regular Expression-Based Redaction Policy](#)
 - [Syntax for Creating a Random Redaction Policy](#)
 - [Syntax for Creating a Policy with No Redaction](#)

If you omit the function_type parameter, then the default redaction function_type setting is DBMS_REDACT.FULL.

- **function_parameters:** Specifies how the column redaction should appear for partial redaction. See [Syntax for Creating a Partial Redaction Policy](#).
- **expression:** Specifies a Boolean SQL expression to determine how the policy is applied. Redaction takes place only if this policy expression evaluates to TRUE. See [Using Expressions to Define Conditions for Data Redaction Policies](#).

By default, a Data Redaction policy expression applies to all the columns that belong to the Data Redaction policy defined on that table or view. Alternatively, you can choose to create and associate a policy expression for individual columns to override the existing expression. These column level expressions are called as named policy expressions. See [Creating and Managing Multiple Named Policy Expressions](#).

- **enable:** When set to TRUE, enables the policy upon creation. When set to FALSE, it creates the policy as a disabled policy. The default is TRUE. After you create the policy, you can disable or enable it. See the following sections:
 - [Disabling an Oracle Data Redaction Policy](#)
 - [Enabling an Oracle Data Redaction Policy](#)

- `regexp_pattern`, `regexp_replace_string`, `regexp_position`, `regexp_position`, `regexp_occurrence`, `regexp_match_parameter`: Enable you to use regular expressions to redact data, either fully or partially. If the `regexp_pattern` does not match anything in the [actual data](#), then full redaction will take place, so be careful when specifying the `regexp_pattern`. Ensure that all of the values in the column conform to the semantics of the regular expression you are using. See [Syntax for Creating a Regular Expression-Based Redaction Policy](#) for more information.

15.5 Using Expressions to Define Conditions for Data Redaction Policies

The expression parameter in the `DBMS_REDACT.ADD_POLICY` procedure sets the conditions to which the policy applies.

- [About Using Expressions in Data Redaction Policies](#)
The `DBMS_REDACT.ADD_POLICY` and `DBMS_REDACT.ALTER_POLICY` expression parameter defines a Boolean expression that must evaluate to TRUE to enable a redaction.
- [Supported Functions for Data Redaction Expressions](#)
You can create expressions that use functions to return specific types of data, such as `SYS_CONTEXT` namespaces.
- [Applying the Redaction Policy Based on User Environment](#)
You can apply a Data Redaction policy based on the user's environment, such as the session user name or a client identifier.
- [Applying the Redaction Policy Based on Database Roles](#)
You can apply a Data Redaction policy based on a database role, such as the DBA role.
- [Applying the Redaction Policy Based on Oracle Label Security Label Dominance](#)
You can set a condition on which to apply a Data Redaction policy based on the dominance of Oracle Label Security labels.
- [Applying the Redaction Policy Based on Application Express Session States](#)
You can apply a Data Redaction policy based on an Oracle Application Express (APEX) session state.
- [Applying the Redaction Policy to All Users](#)
You can apply the policy irrespective of the context to any user, with no filtering.

15.5.1 About Using Expressions in Data Redaction Policies

The `DBMS_REDACT.ADD_POLICY` and `DBMS_REDACT.ALTER_POLICY` expression parameter defines a Boolean expression that must evaluate to TRUE to enable a redaction.

The expression that is defined in the expression parameter is the default expression for the Oracle Data Redaction policy. If you apply a named policy expression for the columns that will be redacted by the Data Redaction policy, then the named policy expression takes precedence over the expression defined in the Data Redaction policy.

You can create expressions that make use of other Oracle Database features. For example, you can create expressions that are based on a user's environment (using the `SYS_CONTEXT` and `XS_SYS_CONTEXT` functions), character string functions, the Oracle Label Security label dominance functions, or Oracle Application Express functions.

Follow these guidelines when you write the expression:

- Use only the following operators: AND, OR, IN, NOT IN, =, !=, <>, <, >, >=, <=

- Because the expression must evaluate to TRUE for redaction, be careful when making comparisons with NULL. Remember that in SQL the value NULL is undefined, so comparisons with NULL tend to return FALSE.
- Do not use user-created functions in the expression parameter; this is not permitted.
- Remember that for user SYS and users who have the EXEMPT REDACTION POLICY privilege, all of the Data Redaction policies are bypassed, so the results of their queries are not redacted. See the following topics for more information about users who are exempted from Data Redaction policies:
 - [Exemption of Users from Oracle Data Redaction Policies](#)
 - [Oracle Data Pump Security Model for Oracle Data Redaction](#)

15.5.2 Supported Functions for Data Redaction Expressions

You can create expressions that use functions to return specific types of data, such as SYS_CONTEXT namespaces.

- [Expressions Using Namespace Functions](#)
You can use the SYS_CONTEXT and XS_SYS_CONTEXT namespace functions in Data Redaction expressions.
- [Expressions Using the SUBSTR Function](#)
You can use the SUBSTR function, which returns portion (such as characters 1–3) of the character string specified, in Data Redaction expressions. The first parameter must be a constant string or a call to the SYS_CONTEXT function or the XS_SYS_CONTEXT function.
- [Expressions Using Length of Character String Functions](#)
You can use the following functions, which return the length of character strings, in Data Redaction expressions. Oracle Database also checks that the arguments to each of these operators is either a constant string or a call to the SYS_CONTEXT or XS_SYS_CONTEXT function.
- [Expressions Using Oracle Application Express Functions](#)
You can use Oracle Application Express functions in Data Redaction expressions.
- [Expressions Using Oracle Label Security Functions](#)
You can use Oracle Label Security functions with Data Redaction expressions.

15.5.2.1 Expressions Using Namespace Functions

You can use the SYS_CONTEXT and XS_SYS_CONTEXT namespace functions in Data Redaction expressions.

Table 15-2 Expressions Using Namespace Functions

Namespace Function	Description
SYS_CONTEXT	Returns the value associated with a namespace. The following namespace functions are valid: • USERENV (default namespace), which includes values such as SESSION_USER and CLIENT_IDENTIFIER. • SYS_SESSION_ROLES, which contains attributes for each role • XS\$SESSION, which contains attributes for the user session. • User-defined namespaces, but these must exist in the DBA_CONTEXT catalog view before the policy expression is created.

Table 15-2 (Cont.) Expressions Using Namespace Functions

Namespace Function	Description
XS_SYS_CONTEXT	Similar to SYS_CONTEXT but designed for an Oracle Real Application Security environment. XS_SYS_CONTEXT supports the same namespaces that SYS_CONTEXT supports.

See Also

- *Oracle Database SQL Language Reference* for more information about SYS_CONTEXT
- *Oracle Database Real Application Security Administrator's and Developer's Guide* for more information about XS_SYS_CONTEXT

15.5.2.2 Expressions Using the SUBSTR Function

You can use the SUBSTR function, which returns portion (such as characters 1–3) of the character string specified, in Data Redaction expressions. The first parameter must be a constant string or a call to the SYS_CONTEXT function or the XS_SYS_CONTEXT function.

Table 15-3 Expressions Using SUBSTR String Functions

SUBSTR String Function	Description
SUBSTR	Returns a portion of the input char value, beginning at <i>character position</i> , <i>substring_length</i> characters long. SUBSTR calculates length using characters as defined by the input character set.
SUBSTRB	Returns the specified portion of the input value in bytes
SUBSTRC	Returns the specified portion of the input value in Unicode complete characters
SUBSTR2	Returns the specified portion of the input value in UCS2 code points
SUBSTR4	Returns the specified portion of the input value in UCS4 code points

See Also

Oracle Database SQL Language Reference for more information about the SUBSTR functions

15.5.2.3 Expressions Using Length of Character String Functions

You can use the following functions, which return the length of character strings, in Data Redaction expressions. Oracle Database also checks that the arguments to each of these operators is either a constant string or a call to the SYS_CONTEXT or XS_SYS_CONTEXT function.

Table 15-4 Expressions Using Character String Functions

Character String Function	Description
LENGTH	Returns the length of the input char value. LENGTH calculates length using characters as defined by the input character set.
LENGTHB	Returns the length of the input value in bytes
LENGTHC	Returns the length of the input value in Unicode complete characters
LENGTH2	Returns the length of the input value in UCS2 code points
LENGTH4	Returns the length of the input value in UCS4 code points

See Also

Oracle Database SQL Language Reference for more information about the LENGTH functions

15.5.2.4 Expressions Using Oracle Application Express Functions

You can use Oracle Application Express functions in Data Redaction expressions.

Table 15-5 Oracle Application Express Functions

Oracle Application Express Function	Description
V	Returns the session state for an item. It is a wrapper for the APEX_UTIL.GET_SESSION_STATE function
NV	Returns the numeric value for a numeric item. It is a wrapper for the APEX_UTIL.GET_NUMERIC_SESSION_STATE function

Related Topics

- *Oracle Application Express API Reference*

15.5.2.5 Expressions Using Oracle Label Security Functions

You can use Oracle Label Security functions with Data Redaction expressions.

For the functions in the bold font, Oracle Data Redaction checks that their parameters are either constants or calls to only one of the SA_UTL.NUMERIC_LABEL, CHAR_TO_LABEL, and SA_SESSION.LABEL functions, and that the arguments to those functions are constant.

Table 15-6 Oracle Label Security Functions

Oracle Label Security Function	Description
<code>LBACSYS.OLS_LABEL_DOMINATES</code>	Checks if the session label of an Oracle Label Security policy dominates or is equal to another OLS label
DOMINATES	Checks if one OLS label is dominant to a second OLS label. Deprecated in Oracle Database 12c release 1 (12.1); use the <code>OLS_DOMINATES</code> or <code>OLS_DOM</code> function instead.
OLS_DOMINATES	Checks if one OLS label is dominant to a second OLS label
OLS_DOM	Checks if one OLS label is dominant to a second OLS label
DOM	Checks if one OLS label is dominant to a second OLS label
OLS_STRICTLY_DOMINATES	Checks if one OLS label is dominant to a second OLS label and is not equal to it
STRICTLY_DOMINATES	Checks if one OLS label is dominant to a second OLS label and is not equal to it
S_DOM	Checks if one OLS label is dominant to a second OLS label and is not equal to it. Deprecated in Oracle Database 12c release 1 (12.1); use the <code>OLS_DOMINATES</code> or <code>OLS_DOM</code> function instead.
SA_UTL.DOMINATES	Checks if one OLS label dominates a second OLS label or if the session label for a given OLS policy dominates an OLS label
<code>SA_UTL.CHECK_READ</code>	Checks if a user can read a policy-protected row
<code>SA_UTL.NUMERIC_LABEL</code>	Returns the current session OLS label
<code>CHAR_TO_LABEL</code>	Converts a character string to an OLS label tag
<code>SA_SESSION.LABEL</code>	Returns the label that is associated with the specified OLS policy

Related Topics

- *Oracle Label Security Administrator's Guide*

15.5.3 Applying the Redaction Policy Based on User Environment

You can apply a Data Redaction policy based on the user's environment, such as the session user name or a client identifier.

- Use the `USERENV` namespace of the `SYS_CONTEXT` function in the `DBMS_REDACT.ADD_POLICY` expression parameter to apply the policy based on a user's environment.

For example, to apply the policy only to the session user name `psmith`:

```
expression => 'SYS_CONTEXT(''USERENV'', 'SESSION_USER') = 'PSMITH'''
```

① See Also

Oracle Database SQL Language Reference for information about more namespaces that you can use with the SYS_CONTEXT function

15.5.4 Applying the Redaction Policy Based on Database Roles

You can apply a Data Redaction policy based on a database role, such as the DBA role.

- Use the SYS_SESSION_ROLES namespace in the SYS_CONTEXT function to apply the policy based on a user role.

This namespace contains attributes for each role. The value of the attribute is TRUE if the specified role is enabled for the querying application user; the value is FALSE if the role is not enabled.

For example, suppose you wanted only supervisors to be allowed to see the actual data. The following example shows how to use the DBMS_REDACT.ADD_POLICY expression parameter to set the policy to show the actual data to any application user who has the supervisor role enabled, but redact the data for all of the other application users.

```
expression => 'SYS_CONTEXT(''SYS_SESSION_ROLES'', 'SUPERVISOR') = ''FALSE'''
```

You can specify multiple roles with the expression parameter. The following example checks for the presence of the supervisor, clerk, and temp_worker roles.

```
expression => 'SYS_CONTEXT(''SYS_SESSION_ROLES'', 'SUPERVISOR') = ''TRUE''
OR SYS_CONTEXT(''SYS_SESSION_ROLES'', 'CLERK') = ''TRUE''
OR SYS_CONTEXT(''SYS_SESSION_ROLES'', 'TEMP_WORKER') = ''TRUE'''
```

Related Topics

- *Oracle Database SQL Language Reference*

15.5.5 Applying the Redaction Policy Based on Oracle Label Security Label Dominance

You can set a condition on which to apply a Data Redaction policy based on the dominance of Oracle Label Security labels.

- Use the public standalone function OLS_LABEL_DOMINATES to check the dominance of a session label. This function returns 1 (TRUE) if the session label of the specified policy_name value dominates or is equal to the label that is specified by the label parameter; otherwise, it returns 0 (FALSE).

For example, to apply a Data Redaction policy only in cases where the session label for the policy hr_ols_pol does not dominate nor is equal to label hs:

```
expression => 'OLS_LABEL_DOMINATES (''hr_ols_pol'', 'hs') = 0'
```

15.5.6 Applying the Redaction Policy Based on Application Express Session States

You can apply a Data Redaction policy based on an Oracle Application Express (APEX) session state.

- Use either of the following public Application Express APIs in the `DBMS_REDACT.ADD_POLICY` expression parameter to apply the policy on an Oracle Application Express session state:
 - `V`, which is a synonym for the `APEX_UTIL.GET_SESSION_STATE` function
 - `NV`, which is a synonym for the `APEX_UTIL.GET_NUMERIC_SESSION_STATE` function

For example, to set the `DBMS_REDACT.ADD_POLICY` expression parameter if you wanted redaction to take place when the application item called `G_JOB` has the value `CLERK`:

```
expression => 'V(''APP_USER'') != ''mavis@example.com'' or V(''APP_USER'') is null'
```

You can, for example, use these functions to redact data based on a job or a privilege role that is stored in a session state in an APEX application.

If you want redaction to take place when the querying user is *not* within the context of an APEX application (when the query is issued from outside the APEX framework, for example directly through SQL*Plus), then use an `IS NULL` clause as follows. This policy expression causes actual data to be shown to user `mavis` only when her query comes from within an APEX application. Otherwise, the query result is redacted.

① See Also

Oracle Application Express API Reference

15.5.7 Applying the Redaction Policy to All Users

You can apply the policy irrespective of the context to any user, with no filtering.

However, be aware that user `SYS` and users who have the `EXEMPT REDACTION POLICY` privilege are always except from Oracle Data Redaction policies.

- To apply the policy to users who are not `SYS` or have been granted the `EXEMPT REDACTION POLICY` privilege, write the `DBMS_REDACT.ADD_POLICY` expression parameter to evaluate to `TRUE`.

For example:

```
expression => '1=1'
```

① See Also

[Exemption of Users from Oracle Data Redaction Policies](#)

15.6 Creating and Managing Multiple Named Policy Expressions

A named, centrally managed Oracle Data Redaction policy expression can be used in multiple redaction policies and applied to multiple tables or views.

- [About Data Redaction Policy Expressions to Define Conditions](#)
A named Oracle Data Redaction policy expression is designed to work as an alternative to the policy expression that is used in existing Data Redaction policies.
- [Creating and Applying a Named Data Redaction Policy Expression](#)
The `DBMS_REDACT.CREATE_POLICY_EXPRESSION` and `DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL` enable you to create and apply a named Data Redaction policy expression.
- [Updating a Named Data Redaction Policy Expression](#)
You can use the `DBMS_REDACT.UPDATE_POLICY_EXPRESSION` procedure to update a Data Redaction policy expression. The update takes place immediately and is reflected in all columns that use the policy expression.
- [Dropping a Named Data Redaction Expression Policy](#)
You can use the `DBMS_REDACT.DROP_POLICY_EXPRESSION` procedure to drop a Data Redaction expression policy.
- [Tutorial: Creating and Sharing a Named Data Redaction Policy Expression](#)
This tutorial shows how to create an Oracle Data Redaction policy expression, apply it to multiple tables, and centrally manage the policy expression.

15.6.1 About Data Redaction Policy Expressions to Define Conditions

A named Oracle Data Redaction policy expression is designed to work as an alternative to the policy expression that is used in existing Data Redaction policies.

A named policy expression enables you to redact data based on runtime conditions. This type of policy can only affect whether or not redaction takes place on columns of the table or view on which the redaction policy is defined. By default, a Data Redaction policy expression applies to all the columns that belong to the Data Redaction policy defined on that table or view. Alternatively, you can choose to create and associate a policy expression for individual columns of a table or view. These column level expressions are called as named policy expressions; in other words, a policy expression with a name. A named policy expressions has the following properties:

You can use Data Redaction policy expressions in the following ways.:

- A single Data Redaction policy expression can be shared by more than one Data Redaction policy by applying it to columns that are a part of separate Data Redaction policies.
- Each named policy expression can be associated with multiple columns of the same or different tables or views.
- Each named policy expression can be associated with columns within the same or different Data Redaction policies.
- The named policy expression overrides the default policy expression of the associated columns. The default policy expression still applies to redaction columns that have no named policy expressions applied to them.
- Any updates made to a named policy expression apply to all of the column associations of the expression.

- You cannot associate multiple named policy expressions for the same column.
- In a multitenant environment, you cannot associate named policy expressions with columns in a different pluggable database (PDB).

The column to which you apply a named policy expression must already be redacted by a Data Redaction policy. After the named policy expression is applied, the result of its evaluation takes precedence over that of the default policy expression when deciding whether or not to redact the column. When you modify a named policy expression, the changes are applied to all the tables and views that use it. In a multitenant environment, as with Data Redaction policies, a named policy expression is valid only in the PDB in which it was created, and can only be applied to columns of objects within the PDB in which it was created.

[Table 15-7](#) describes the DBMS_REDACT PL/SQL procedures that you can use to create and manage named policy expressions. To find information about policy expressions, query the REDACTION_EXPRESSIONS data dictionary view.

Table 15-7 DBMS_REDACT Policy Expression Procedures

Procedure	Description
DBMS_REDACT.CREATE_POLICY_EXPRESSION	Creates a Data Redaction policy expression
DBMS_REDACT.UPDATE_POLICY_EXPRESSION	Updates a Data Redaction policy expression
DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL	Applies a Data Redaction policy expression to a table or a view column
DBMS_REDACT.DROP_POLICY_EXPRESSION	Drops a Data Redaction policy expression

Related Topics

- [Managing Named Data Redaction Policy Expressions Using Enterprise Manager](#)
You can manage Oracle Data Redaction policy expressions in Enterprise Manager Cloud Control.

15.6.2 Creating and Applying a Named Data Redaction Policy Expression

The DBMS_REDACT.CREATE_POLICY_EXPRESSION and DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL enable you to create and apply a named Data Redaction policy expression.

1. Connect to the PDB as a user who has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
2. Ensure that the COMPATIBLE initialization parameter is set to 12.2.0.0 or later.
To find the current COMPATIBLE setting, use the SHOW PARAMETER command.
3. To create the policy expression, run the DBMS_REDACT.CREATE_POLICY_EXPRESSION procedure.

For example:

```
BEGIN
  DBMS_REDACT.CREATE_POLICY_EXPRESSION (
    policy_expression_name      => 'redact_pol',
    expression                  => '1=1',
    policy_expression_description => 'Determines whether the column will be redacted');
END;
/
```

4. Run the DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL procedure to apply the policy expression to a table or view column.

For example, assume that you have already created a Data Redaction policy on the SALARY column of the HR.EMPLOYEES table, as follows:

```
BEGIN
  DBMS_REDACT.ADD_POLICY (
    object_schema      => 'hr',
    object_name        => 'employees',
    policy_name        => 'overall_policy',
    expression         => '1=0');
END;
/
BEGIN
  DBMS_REDACT.ALTER_POLICY (
    object_schema      => 'hr',
    object_name        => 'employees' ,
    policy_name        => 'overall_policy',
    function_type      => DBMS_REDACT.FULL,
    action             => DBMS_REDACT.ADD_COLUMN,
    column_name        => 'SALARY ');
END;
/
```

Then you can apply the policy expression to the SALARY table as follows:

```
BEGIN
  DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL (
    object_schema      => 'hr',
    object_name        => 'employees',
    column_name        => 'salary',
    policy_expression_name => 'redact_pol');
END;
/
```

In this specification:

- **object_schema:** Specifies the schema of the object on which the policy expression will be used. If you omit this setting (or enter NULL), then Oracle Database uses the name of the current schema.
- **object_name:** Specifies the name of the table or view to be used for the policy expression.
- **column_name:** Specifies the column to which you want to apply the policy expression.
- **policy_expression_name:** Specifies the name of the policy expression.

After you create an Oracle Data Redaction policy expression, you can apply it to a column of a table or view which is part of an existing Data Redaction policy.

15.6.3 Updating a Named Data Redaction Policy Expression

You can use the DBMS_REDACT.UPDATE_POLICY_EXPRESSION procedure to update a Data Redaction policy expression. The update takes place immediately and is reflected in all columns that use the policy expression.

You can query the REDACTION_EXPRESSIONS data dictionary view to find existing Data Redaction policy expressions.

1. Connect to the PDB as a user who has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
2. Ensure that the COMPATIBLE initialization parameter is set to 12.2.0.0 or later.

To find the current COMPATIBLE setting, use the SHOW PARAMETER command.

3. Run the DBMS_REDACT.UPDATE_POLICY_EXPRESSION procedure to perform the update.

For example:

```
BEGIN
  DBMS_REDACT.UPDATE_POLICY_EXPRESSION(
    policy_expression_name => 'redact_pol',
    expression             => '1=0');
END;
/
```

15.6.4 Dropping a Named Data Redaction Expression Policy

You can use the DBMS_REDACT.DROP_POLICY_EXPRESSION procedure to drop a Data Redaction expression policy.

You can query the REDACTION_EXPRESSIONS data dictionary view to find existing Data Redaction policy expressions.

1. Connect to the PDB as a user who has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
2. Ensure that the COMPATIBLE initialization parameter is set to 12.2.0.0 or later.

To find the current COMPATIBLE setting, use the SHOW PARAMETER command.

3. Remove the named policy expression's association with any table or view column.

You cannot drop a policy expression if it is associated with an existing table or view column. To remove a given column's association with a named policy expression (to revert to redacting that column based on the evaluation result of the default policy expression), you must set the policy_expression_name parameter of the DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL procedure to NULL.

For example:

```
BEGIN
  DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL(
    object_schema    => 'hr',
    object_name      => 'employees',
    column_name      => 'salary',
    policy_expression_name => null);
END;
/
```

4. Run DBMS_REDACT.DROP_POLICY_EXPRESSION to drop the policy expression.

For example:

```
BEGIN
  DBMS_REDACT.DROP_POLICY_EXPRESSION(
    policy_expression_name => 'redact_pol');
END;
/
```

15.6.5 Tutorial: Creating and Sharing a Named Data Redaction Policy Expression

This tutorial shows how to create an Oracle Data Redaction policy expression, apply it to multiple tables, and centrally manage the policy expression.

- [Step 1: Create Users for This Tutorial](#)
You must create two users for this tutorial: dr_admin, who will create the Oracle Data Redaction policies, and hr_clerk, who will test them.
- [Step 2: Create an Oracle Data Redaction Policy](#)
User dr_admin is ready to create an Oracle Data Redaction policy to protect the HR.EMPLOYEES and HR.JOBS tables.
- [Step 3: Test the Oracle Data Redaction Policy](#)
User hr_clerk is ready to query the tables that have redacted data.
- [Step 4: Create and Apply a Policy Expression to the Redacted Table Columns](#)
Next, user dr_admin is ready to create a Data Redaction policy expression and apply it to two of the three redacted table columns.
- [Step 5: Test the Data Redaction Policy Expression](#)
User hr_clerk is now ready to test the hr_redact_pol policy expression.
- [Step 6: Modify the Data Redaction Policy Expression](#)
User dr_admin decides to modify the Data Redaction policy expression so that user HR will have access to the redacted data, not user hr_clerk.
- [Step 7: Test the Modified Policy Expression](#)
Users HR and hr_clerk are ready to test the modified Data Redaction policy expression.
- [Step 8: Remove the Components of This Tutorial](#)
If you do not need the components of this tutorial, then you can remove them.

15.6.5.1 Step 1: Create Users for This Tutorial

You must create two users for this tutorial: dr_admin, who will create the Oracle Data Redaction policies, and hr_clerk, who will test them.

Before you begin this tutorial, ensure that the COMPATIBLE initialization parameter is set to 12.2.0.0 or later. You can check this setting by using the SHOW PARAMETER command.

1. Connect to a PDB as user SYS with the SYSDBA administrative privilege.
2. Create the dr_admin and hr_clerk user accounts.

```
GRANT CREATE SESSION TO dr_admin IDENTIFIED BY password;  
GRANT CREATE SESSION TO hr_clerk IDENTIFIED BY password;
```
3. Grant the EXECUTE privilege to the dr_admin user.

```
GRANT EXECUTE ON DBMS_REDACT TO dr_admin;
```
4. Connect as user HR.
5. Grant hr_clerk the SELECT privilege on the EMPLOYEES and JOBS tables.

```
GRANT SELECT on EMPLOYEES to hr_clerk;  
GRANT SELECT on JOBS to hr_clerk;
```

15.6.5.2 Step 2: Create an Oracle Data Redaction Policy

User dr_admin is ready to create an Oracle Data Redaction policy to protect the HR.EMPLOYEES and HR.JOBS tables.

1. Connect to the PDB as user dr_admin.
2. Create the hr_emp_redact_comp_pol policy, which will perform full redaction of the HR.EMPLOYEES.SALARY column.

```

BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'hr',
    object_name   => 'employees',
    column_name   => 'salary',
    policy_name   => 'hr_emp_redact_comp_pol',
    function_type => DBMS_REDACT.FULL,
    expression    => '1=1');
END;
/

```

3. Alter the `hr_redact_comp_pol` policy to also redact the `COMMISSION_PCT` column of the `HR.EMPLOYEES` table.

```

BEGIN
  DBMS_REDACT.ALTER_POLICY(
    object_schema => 'hr',
    object_name   => 'employees',
    policy_name   => 'hr_emp_redact_comp_pol',
    action        => DBMS_REDACT.ADD_COLUMN,
    column_name   => 'commission_pct',
    function_type => DBMS_REDACT.FULL);
END;
/

```

4. Create the `hr_jobs_redact_comp_pol` policy for the `max_salary` column of the `HR.JOBS` table.

```

BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'hr',
    object_name   => 'jobs',
    column_name   => 'max_salary',
    policy_name   => 'hr_jobs_redact_comp_pol',
    function_type => DBMS_REDACT.FULL,
    expression    => '1=1');
END;
/

```

At this stage, the data in the `HR.EMPLOYEES.SALARY`, `HR.EMPLOYEES.COMMISSION_PCT`, and `HR.JOBS.MAX_SALARY` columns are redacted.

15.6.5.3 Step 3: Test the Oracle Data Redaction Policy

User `hr_clerk` is ready to query the tables that have redacted data.

1. Connect as user `hr_clerk`.
2. Query the `HR.EMPLOYEES` table.

```
SELECT SALARY, COMMISSION_PCT FROM HR.EMPLOYEES WHERE SALARY > 15000;
```

The output should be as follows:

```

      SALARY COMMISSION_PCT
-----
          0
          0
          0

```

3. Query the `HR.JOBS` table.

```
SELECT MAX_SALARY FROM HR.JOBS WHERE MAX_SALARY > 15000;
```

The output should be as follows:

```
MAX_SALARY
-----
          0
          0
          0
          0
          0
```

15.6.5.4 Step 4: Create and Apply a Policy Expression to the Redacted Table Columns

Next, user `dr_admin` is ready to create a Data Redaction policy expression and apply it to two of the three redacted table columns.

This policy expression will enable user `hr_clerk` to view the redacted data.

1. Connect as user `dr_admin`.
2. Create the policy expression.

```
BEGIN
  DBMS_REDACT.CREATE_POLICY_EXPRESSION(
    policy_expression_name => 'hr_redact_pol',
    expression              => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') !=
    ''HR_CLERK''');
END;
/
```

This expression returns `FALSE` for the `hr_clerk` user, which enables the `hr_clerk` user to view actual data in the `HR.EMPLOYEES` and `HR.JOBS` tables that are subject to the Data Redaction policies.

3. Apply the `hr_redact_pol` policy expression to the `HR.EMPLOYEES.SALARY` column.

```
BEGIN
  DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL(
    object_schema    => 'hr',
    object_name      => 'employees',
    column_name      => 'salary',
    policy_expression_name => 'hr_redact_pol');
END;
/
```

4. Apply the `hr_redact_pol` policy expression to the `HR.JOBS.MAX_SALARY` column.

```
BEGIN
  DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL(
    object_schema    => 'hr',
    object_name      => 'jobs',
    column_name      => 'max_salary',
    policy_expression_name => 'hr_redact_pol');
END;
/
```

User `hr_clerk` can view data in the `HR.EMPLOYEES.SALARY` and `HR.JOBS.MAX_SALARY`, but the data in the `HR.EMPLOYEES.COMMISSION_PCT` column will still be redacted for this user.

15.6.5.5 Step 5: Test the Data Redaction Policy Expression

User `hr_clerk` is now ready to test the `hr_redact_pol` policy expression.

1. Connect as user hr_clerk.

2. Query the HR.EMPLOYEES table.

```
SELECT SALARY, COMMISSION_PCT FROM HR.EMPLOYEES WHERE SALARY > 15000;
```

The output should be as follows:

```

SALARY COMMISSION_PCT
-----
24000
17000
17000
```

User hr_clerk now can view the SALARY column data, but still has not access to the COMMISSION_PCT column data.

3. Query the HR.JOBS table.

```
SELECT MAX_SALARY FROM HR.JOBS WHERE MAX_SALARY > 15000;
```

The output should be as follows:

```

MAX_SALARY
-----
40000
30000
16000
16000
20080
```

User hr_clerk now can view the MAX_SALARY column data.

15.6.5.6 Step 6: Modify the Data Redaction Policy Expression

User dr_admin decides to modify the Data Redaction policy expression so that user HR will have access to the redacted data, not user hr_clerk.

1. Connect as user dr_admin.
2. Modify the hr_redact_pol policy as follows:

```

BEGIN
  DBMS_REDACT.UPDATE_POLICY_EXPRESSION(
    policy_expression_name => 'hr_redact_pol',
    expression              => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') != ''HR''');
END;
/
```

15.6.5.7 Step 7: Test the Modified Policy Expression

Users HR and hr_clerk are ready to test the modified Data Redaction policy expression.

1. Connect as user HR.
2. Query the HR.EMPLOYEES table.

```
SELECT SALARY, COMMISSION_PCT FROM HR.EMPLOYEES WHERE SALARY > 15000;
```

The output should be as follows:

```

SALARY COMMISSION_PCT
-----
24000
```



```
17000
17000
```

User HR now has access to the redacted data. A query by HR on the HR.JOBS.MAX_SALARY column will produce similar results.

```
SELECT MAX_SALARY FROM HR.JOBS WHERE MAX_SALARY > 15000;
```

```
MAX_SALARY
-----
40000
30000
16000
16000
20080
```

3. Connect as user hr_clerk.
4. Query the HR.EMPLOYEES and HR.JOBS tables and then observe the results.

```
SELECT SALARY, COMMISSION_PCT FROM HR.EMPLOYEES WHERE SALARY > 15000;
```

```
SALARY COMMISSION_PCT
-----
0
0
0
```

```
SELECT MAX_SALARY FROM HR.JOBS WHERE MAX_SALARY > 15000;
```

```
MAX_SALARY
-----
0
0
0
0
0
```

15.6.5.8 Step 8: Remove the Components of This Tutorial

If you do not need the components of this tutorial, then you can remove them.

1. Connect as user dr_admin.
2. Modify the policy expression so that it is no longer associated with the table columns that are associated with the expression.

To do so, you must set the policy_expression_name parameter to NULL.

```
BEGIN
DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL(
  object_schema => 'hr',
  object_name   => 'employees',
  column_name   => 'salary',
  policy_expression_name => null);
END;
/

BEGIN
DBMS_REDACT.APPLY_POLICY_EXPR_TO_COL(
  object_schema => 'hr',
  object_name   => 'jobs',
  column_name   => 'max_salary',
  policy_expression_name => null);
```

```
END;  
/
```

3. Drop the policy expressions.

```
BEGIN  
  DBMS_REDACT.DROP_POLICY_EXPRESSION(  
    policy_expression_name => 'hr_redact_pol');  
END;  
/
```

4. Drop the hr_emp_redact_comp_pol and hr_jobs_redact_comp_pol Data Redaction policies.

```
BEGIN  
  DBMS_REDACT.DROP_POLICY (  
    object_schema => 'hr',  
    object_name   => 'employees',  
    policy_name   => 'hr_emp_redact_comp_pol');  
END;  
/
```

```
BEGIN  
  DBMS_REDACT.DROP_POLICY (  
    object_schema => 'hr',  
    object_name   => 'jobs',  
    policy_name   => 'hr_jobs_redact_comp_pol');  
END;  
/
```

5. Connect as the SYSTEM user or a user who has privileges to drop user accounts.
6. Drop the dr_admin and hr_clerk user accounts.

```
DROP USER dr_admin;  
DROP USER hr_clerk;
```

15.7 Creating a Full Redaction Policy and Altering the Full Redaction Value

You can create a full redaction policy to redact all contents in a data column, and optionally, you can alter the default full redaction value.

- [Creating a Full Redaction Policy](#)
A full data redaction policy redacts all the contents of a data column.
- [Altering the Default Full Data Redaction Value](#)
The DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES procedure alters the default full data redaction value.

15.7.1 Creating a Full Redaction Policy

A full data redaction policy redacts all the contents of a data column.

- [About Creating Full Data Redaction Policies](#)
To set a redaction policy to redact all data in the column, you must set the function_type parameter to DBMS_REDACT.FULL.
- [Syntax for Creating a Full Redaction Policy](#)
The DBMS_REDACT.ADD_POLICY procedure enables you to create a full redaction policy.

- [Example: Full Redaction Policy](#)
You can use the `DBMS_REDACT.ADD_POLICY` PL/SQL procedure to create a full redaction policy.
- [Example: Fully Redacted Character Values](#)
You can use the `DBMS_REDACT.ADD_POLICY` PL/SQL procedure to create a policy that fully redacts character values.

15.7.1.1 About Creating Full Data Redaction Policies

To set a redaction policy to redact all data in the column, you must set the `function_type` parameter to `DBMS_REDACT.FULL`.

By default, `NUMBER` data type columns are replaced with zero (0) and character data type columns are replaced with a single space (). You can modify this default by using the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure.

Related Topics

- [Altering the Default Full Data Redaction Value](#)
The `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure alters the default full data redaction value.

15.7.1.2 Syntax for Creating a Full Redaction Policy

The `DBMS_REDACT.ADD_POLICY` procedure enables you to create a full redaction policy.

The `DBMS_REDACT.ADD_POLICY` fields for creating a full data redaction policy are as follows:

```
DBMS_REDACT.ADD_POLICY (
  object_schema      IN VARCHAR2 := NULL,
  object_name        IN VARCHAR2,
  column_name        IN VARCHAR2 := NULL,
  policy_name        IN VARCHAR2,
  function_type       IN BINARY_INTEGER := NULL,
  expression         IN VARCHAR2,
  enable             IN BOOLEAN := TRUE);
```

In this specification:

- `object_schema`, `object_name`, `column_name`, `policy_name`, `expression`, `enable`: See [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#).
- `function_type`: Specifies the function used to set the type of redaction. Enter `DBMS_REDACT.FULL`.

If you omit the `function_type` parameter, then the default redaction `function_type` setting is `DBMS_REDACT.FULL`.

Remember that the data type of the column determines which `function_type` settings that you are permitted to use. See [Comparison of Full, Partial, and Random Redaction Based on Data Types](#).

15.7.1.3 Example: Full Redaction Policy

You can use the `DBMS_REDACT.ADD_POLICY` PL/SQL procedure to create a full redaction policy.

[Example 15-1](#) shows how to use full redaction for all the values in the `HR.EMPLOYEES` table `COMMISSION_PCT` column. The `expression` parameter applies the policy to any user querying the

table, except for users who have been granted the EXEMPT REDACTION POLICY system privilege.

Example 15-1 Full Data Redaction Policy

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'hr',
    object_name   => 'employees',
    column_name   => 'commission_pct',
    policy_name   => 'redact_com_pct',
    function_type => DBMS_REDACT.FULL,
    expression    => '1=1');
END;
/
```

Query and redacted result:

```
SELECT COMMISSION_PCT FROM HR.EMPLOYEES;
```

```
COMMISSION_PCT
-----
0
0
0
```

Related Topics

- [Exemption of Users from Oracle Data Redaction Policies](#)
You can exempt users from having Oracle Data Redaction policies applied to the data they access.

15.7.1.4 Example: Fully Redacted Character Values

You can use the DBMS_REDACT.ADD_POLICY PL/SQL procedure to create a policy that fully redacts character values.

[Example 15-2](#) shows how to redact fully the user IDs of the user_id column in the mavis.cust_info table. The user_id column is of the VARCHAR2 data type. The output is a blank string. The expression setting enables users who have the MGR role to view the user IDs.

Example 15-2 Fully Redacted Character Values

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'mavis',
    object_name   => 'cust_info',
    column_name   => 'user_id',
    policy_name   => 'redact_cust_user_ids',
    function_type => DBMS_REDACT.FULL,
    expression    => 'SYS_CONTEXT(''SYS_SESSION_ROLES'', ''MGR'') = ''FALSE''');
END;
/
```

Query and redacted result:

```
SELECT user_id FROM mavis.cust_info;
```

```
USER_ID
-----
0
```

0
0

15.7.2 Altering the Default Full Data Redaction Value

The `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure alters the default full data redaction value.

- [About Altering the Default Full Data Redaction Value](#)
You can alter the default displayed values for full Data Redaction policies.
- [Syntax for the `DBMS\_REDACT.UPDATE\_FULL\_REDACTION\_VALUES` Procedure](#)
The `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure accommodates the standard supported Oracle Database data types.
- [Modifying the Default Full Data Redaction Value](#)
To modify the default full data redaction value, use the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure.

15.7.2.1 About Altering the Default Full Data Redaction Value

You can alter the default displayed values for full Data Redaction policies.

By default, 0 is the redacted value when Oracle Database performs full redaction (`DBMS_REDACT.FULL`) on a column of the `NUMBER` data type. If you want to change it to another value (for example, 7), then you can run the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure to modify this value. The modification applies to all of the Data Redaction policies in the current database instance. After you modify a value, you must restart the database for it to take effect. You can find the current values by querying the `REDACTION_VALUES_FOR_TYPE_FULL` data dictionary view.

Be aware that this change affects all Data Redaction policies in the database that use full data redaction. Before you alter the default full data redaction value, examine the affect that this change would have on existing full Data Redaction policies.

15.7.2.2 Syntax for the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` Procedure

The `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure accommodates the standard supported Oracle Database data types.

The syntax is as follows:

```
DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES (
  number_val      IN NUMBER          NULL,
  binfloat_val    IN BINARY_FLOAT    NULL,
  bindouble_val   IN BINARY_DOUBLE   NULL,
  char_val        IN CHAR             NULL,
  varchar_val     IN VARCHAR2         NULL,
  nchar_val       IN NCHAR            NULL,
  nvarchar_val    IN NVARCHAR2        NULL,
  date_val        IN DATE             NULL,
  ts_val          IN TIMESTAMP         NULL,
  tswtz_val       IN TIMESTAMP WITH TIME ZONE NULL,
  blob_val        IN BLOB             NULL,
  clob_val        IN CLOB             NULL,
  nclob_val       IN NCLOB            NULL);
```

In this specification:

- `number_val` modifies the default value for columns of the NUMBER data type.
- `binfloat_val` modifies the default value for columns of the BINARY_FLOAT data type.
- `bindouble_val` modifies the default value for columns of the BINARY_DOUBLE data type.
- `char_val` modifies the default value for columns of the CHAR data type.
- `varchar_val` modifies the default value for columns of the VARCHAR2 data type.
- `nchar_val` modifies the default value for columns of the NCHAR data type.
- `nvarchar_val` modifies the default value for columns of the NVARCHAR2 data type.
- `date_val` modifies the default value for columns of the DATE data type.
- `ts_val` modifies the default value for columns of the TIMESTAMP data type.
- `tswtz_val` modifies the default value for columns of the TIMESTAMP WITH TIME ZONE data type.
- `blob_val` modifies the default value for columns of the BLOB data type.
- `clob_val` modifies the default value for columns of the CLOB data type.
- `nclob` modifies the default value for columns of the NCLOB data type.

15.7.2.3 Modifying the Default Full Data Redaction Value

To modify the default full data redaction value, use the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure.

1. Connect to the PDB as a user who has the EXECUTE privilege on the DBMS_REDACT PL/SQL package and who has administrative privileges, such as users who have been granted the DBA role.

2. Check the value that you want to change.

For example, to check the current value for columns that use the NUMBER data type:

```
SELECT NUMBER_VALUE FROM REDACTION_VALUES_FOR_TYPE_FULL;
```

```
NUMBER_VALUE
-----
0
```

3. Run the `DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES` procedure to modify the value.

For example:

```
EXEC DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES (number_val => 7);
```

4. Close and then re-open the PDB.

```
ALTER PLUGGABLE DATABASE pdb_name CLOSE IMMEDIATE;
ALTER PLUGGABLE DATABASE pdb_name OPEN;
```

15.8 Creating a DBMS_REDACT.NULLIFY Redaction Policy

You can create Oracle Data Redaction policies that return null values for the displayed value of the table or view column.

- [About Creating a Policy That Returns Null Values](#)
The `DBMS_REDACT.NULLIFY function_type` parameter redacts all the data in a column and replace it with null values.

- [Syntax for Creating a Policy That Returns Null Values](#)
The DBMS_REDACT.ADD_POLICY procedure can create a redaction policy that performs a full redaction and displays null values for the redacted columns.
- [Example: Redaction Policy That Returns Null Values](#)
The DBMS_REDACT.ADD_POLICY procedure will return null values for the COMMISSION_PCT column of the HR.EMPLOYEES table.

15.8.1 About Creating a Policy That Returns Null Values

The DBMS_REDACT.NULLIFY function_type parameter redacts all the data in a column and replace it with null values.

You can use this function type on all supported column types that the DBMS_REDACT.FULL function type supports. It also supports the CLOB and NCLOB data types. To use the DBMS_REDACT.NULLIFY function, you must first ensure that the COMPATIBLE parameter is set to 12.2.0.0 or later.

15.8.2 Syntax for Creating a Policy That Returns Null Values

The DBMS_REDACT.ADD_POLICY procedure can create a redaction policy that performs a full redaction and displays null values for the redacted columns.

The syntax for using DBMS_REDACT.ADD_POLICY to return null values is as follows:

```
DBMS_REDACT.ADD_POLICY (
  object_schema      IN VARCHAR2 := NULL,
  object_name        IN VARCHAR2,
  column_name        IN VARCHAR2 := NULL,
  policy_name        IN VARCHAR2,
  function_type       IN BINARY_INTEGER := NULL,
  expression         IN VARCHAR2,
  enable             IN BOOLEAN := TRUE);
```

In this specification:

- object_schema, object_name, column_name, policy_name, expression, enable: See [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#).
- function_type: Specifies the function used to set the type of redaction. Enter DBMS_REDACT.NULLIFY.

If you omit the function_type parameter, then the default setting is DBMS_REDACT.FULL.

Remember that the data type of the column determines which function_type settings that you are permitted to use. See [Comparison of Full, Partial, and Random Redaction Based on Data Types](#).

15.8.3 Example: Redaction Policy That Returns Null Values

The DBMS_REDACT.ADD_POLICY procedure will return null values for the COMMISSION_PCT column of the HR.EMPLOYEES table.

The expression parameter applies the policy to any user who queries the table, except for users who have been granted the EXEMPT REDACTION POLICY system privilege.

[Example 15-3](#) shows how to create the Oracle Data Redaction policy.

Example 15-3 Redaction Policy That Returns Null Values

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'hr',
    object_name   => 'employees',
    column_name   => 'commission_pct',
    policy_name   => 'nullify_com_pct',
    function_type => DBMS_REDACT.NULLIFY,
    expression    => '1=1');
END;
/
```

Query and redacted result:

```
SELECT COMMISSION_PCT FROM HR.EMPLOYEES;
```

```
COMMISSION_PCT
-----
```

Related Topics

- [Exemption of Users from Oracle Data Redaction Policies](#)
You can exempt users from having Oracle Data Redaction policies applied to the data they access.

15.9 Creating a Partial Redaction Policy

In partial data redaction, you can redact portions of data, and for different kinds of data types.

- [About Creating Partial Redaction Policies](#)
In partial data redaction, only a portion of the data, such as the first five digits of an identification number, are redacted.
- [Syntax for Creating a Partial Redaction Policy](#)
The DBMS_REDACT.ADD_POLICY statement enables you to create policies that redact specific parts of the data returned to the application.
- [Creating Partial Redaction Policies Using Fixed Character Formats](#)
The DBMS_REDACT.ADD_POLICY function_parameters parameter enables you to use fixed character formats.
- [Creating Partial Redaction Policies Using Character Data Types](#)
The DBMS_REDACT.ADD_POLICY function_parameters parameter enables you to redact character data types.
- [Creating Partial Redaction Policies Using Number Data Types](#)
The DBMS_REDACT.ADD_POLICY function_parameters parameter can redact number data types.
- [Creating Partial Redaction Policies Using Date-Time Data Types](#)
The DBMS_REDACT.ADD_POLICY function_parameters parameter can redact date-time data types.

15.9.1 About Creating Partial Redaction Policies

In partial data redaction, only a portion of the data, such as the first five digits of an identification number, are redacted.

For example, you can redact most of a credit card number with asterisks (*), except for the last 4 digits. You can create policies for columns that use character, number, or date-time data types. For policies that redact character data types, you can use fixed character redaction formats. If you have the Enterprise Manager for Oracle Database 12.1.0.7 plug-in deployed on your system, then you can also create and save custom redaction formats.

Note

In previous releases, the term shortcut was used for the term format.

15.9.2 Syntax for Creating a Partial Redaction Policy

The `DBMS_REDACT.ADD_POLICY` statement enables you to create policies that redact specific parts of the data returned to the application.

The `DBMS_REDACT.ADD_POLICY` fields for creating a partial redaction policy are as follows:

```
DBMS_REDACT.ADD_POLICY (
  object_schema      IN VARCHAR2 := NULL,
  object_name        IN VARCHAR2,
  column_name        IN VARCHAR2 := NULL,
  policy_name        IN VARCHAR2,
  function_type       IN BINARY_INTEGER := NULL,
  function_parameters IN VARCHAR2 := NULL,
  expression         IN VARCHAR2,
  enable             IN BOOLEAN := TRUE);
```

In this specification:

- `object_schema`, `object_name`, `column_name`, `policy_name`, `expression`, `enable`: See [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#)
- `function_type`: Specifies the function used to set the type of redaction. Enter `DBMS_REDACT.PARTIAL`.
- `function_parameters`: The parameters that you set here depend on the data type of the column specified for the `column_name` parameter. See the following sections for details:
 - [Creating Partial Redaction Policies Using Fixed Character Formats](#)
 - [Creating Partial Redaction Policies Using Character Data Types](#)
 - [Creating Partial Redaction Policies Using Number Data Types](#)
 - [Creating Partial Redaction Policies Using Date-Time Data Types](#)

15.9.3 Creating Partial Redaction Policies Using Fixed Character Formats

The `DBMS_REDACT.ADD_POLICY` `function_parameters` parameter enables you to use fixed character formats.

- [Settings for Fixed Character Formats](#)
Oracle Data Redaction provides special predefined formats to configure policies that use fixed characters.
- [Example: Partial Redaction Policy Using a Fixed Character Format](#)
You can use the `DBMS_REDACT.ADD_POLICY` PL/SQL procedure to create a partial redaction policy that uses a fixed character format.

15.9.3.1 Settings for Fixed Character Formats

Oracle Data Redaction provides special predefined formats to configure policies that use fixed characters.

[Table 15-8](#) describes `DBMS_REDACT.ADD_POLICY` function_parameters parameter formats that you can use for commonly redacted identity numbers (such as Social Security numbers or Canadian Social Insurance Numbers), postal codes, and credit cards that use either the `VARCHAR2` or `NUMBER` data types for their columns.

Table 15-8 Partial Fixed Character Redaction Formats

Format	Description
<code>DBMS_REDACT.REDACT_US_SSN_F5</code>	Redacts the first 5 numbers of Social Security numbers when the column is a <code>VARCHAR2</code> data type. For example, the number 987-65-4320 becomes XXX-XX-4320.
<code>DBMS_REDACT.REDACT_US_SSN_L4</code>	Redacts the last 4 numbers of Social Security numbers when the column is a <code>VARCHAR2</code> data type. For example, the number 987-65-4320 becomes 987-65-XXXX.
<code>DBMS_REDACT.REDACT_US_SSN_ENTIRE</code>	Redacts the entire Social Security number when the column is a <code>VARCHAR2</code> data type. For example, the number 987-65-4320 becomes XXX-XX-XXXX.
<code>DBMS_REDACT.REDACT_NUM_US_SSN_F5</code>	Redacts the first 5 numbers of Social Security numbers when the column is a <code>NUMBER</code> data type. For example, the number 987654320 becomes XXXXX4320.
<code>DBMS_REDACT.REDACT_NUM_US_SSN_L4</code>	Redacts the last 4 numbers of Social Security numbers when the column is a <code>NUMBER</code> data type. For example, the number 987654320 becomes 98765XXXX.
<code>DBMS_REDACT.REDACT_NUM_US_SSN_ENTIRE</code>	Redacts the entire Social Security number when the column is a <code>NUMBER</code> data type. For example, the number 987654320 becomes XXXXXXXX.
<code>DBMS_REDACT.REDACT_SIN_NUMBER</code>	Redacts the Canadian Social Insurance number by replacing the first 6 digits by 9 (number). For example, 123456789 is redacted to 999999789.
<code>DBMS_REDACT.REDACT_SIN_UNFORMATTED</code>	Redacts the Canadian Social Insurance number by replacing the first 6 digits by X (string). For example, 123456789 is redacted to XXXXXX789.
<code>DBMS_REDACT.REDACT_SIN_FORMATTED</code>	Redacts the Canadian Social Insurance Number by replacing the first 6 digits by X (string). For example, 123-456-789 is redacted to XXX-XXX-789.
<code>DBMS_REDACT.REDACT_UK_NIN_FORMATTED</code>	Redacts the UK National Insurance number by replacing the first 6 digits by X (string) but leaving the alphabetic characters as is. For example, ET 27 02 23 D is redacted to ET XX XX XX D.
<code>DBMS_REDACT.REDACT_UK_NIN_UNFORMATTED</code>	Redacts the UK National Insurance number by replacing the first 6 digits by X (string) and leaving the alphabetic characters as is. For example, ET270223D is redacted to ETXXXXXXD.

Table 15-8 (Cont.) Partial Fixed Character Redaction Formats

Format	Description
DBMS_REDACT.REDACT_CCN_FORMATTED	Redacts the credit card number (other than American Express) by replacing everything but the last 4 digits by *. For example, the credit card number 5105-1051-0510-5100 is redacted to ****_****_****_5100.
DBMS_REDACT.REDACT_CCN_NUMBER	Redacts the credit card number (other than American Express) by replacing everything but the last 4 digits by 0. For example, the credit card number 5105105105105100 is redacted to *****5100.
DBMS_REDACT.REDACT_CCN16_F12	Redacts a 16-digit credit card number (other than American Express), leaving the last 4 digits displayed. For example, 5105 1051 0510 5100 becomes ****_****_****-5100.
DBMS_REDACT.REDACT_AMEX_CCN_FORMATTED	Redacts the American Express credit card number by replacing the digits with * except the last 5 digits. For example, the credit card number 3782 822463 10005 is redacted to ****_***** 10005.
DBMS_REDACT.REDACT_AMEX_CCN_NUMBER	Redacts the American Express Credit Card Number by replacing the digits with 0 except the last 5 digits. For example, the credit card number 3782 822463 10005 is redacted to 0000 000000 10005.
DBMS_REDACT.REDACT_ZIP_CODE	Redacts a 5-digit postal code when the column is a VARCHAR2 data type. For example, 95476 becomes XXXXX.
DBMS_REDACT.REDACT_NUM_ZIP_CODE	Redacts a 5-digit postal code when the column is a NUMBER data type. For example, 95476 becomes XXXXX.
DBMS_REDACT.REDACT_DATE_EPOCH	Redacts all dates to 01-JAN-70.
DBMS_REDACT.REDACT_NA_PHONE_FORMATTED	Redacts the North American phone number by leaving the area code, but replacing everything else with X. For example, 650-555-0100 is redacted to 650-XXX-XXXX.
DBMS_REDACT.REDACT_NA_PHONE_NUMBER	Redacts the North American phone number by leaving the area code, but replacing everything else with 0. For example, 6505550100 gets redacted to 650000000.
DBMS_REDACT.REDACT_NA_PHONE_UNFORMATTE D	Redacts the North American phone number by leaving the area code, but replacing everything else with X. For example, 6505550100 is redacted to 650XXXXXX.
DBMS_REDACT.REDACT_DATE_MILLENNIUM	Redacts dates that are in the DD-MON-YY format to 01-JAN-00 (January 1, 2000).

① See Also

[General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#) for information about other DBMS_REDACT.ADD_POLICY parameters

15.9.3.2 Example: Partial Redaction Policy Using a Fixed Character Format

You can use the DBMS_REDACT.ADD_POLICY PL/SQL procedure to create a partial redaction policy that uses a fixed character format.

[Example 15-4](#) shows how Social Security numbers in a VARCHAR2 data type column and can be redacted using the REDACT_US_SSN_F5 format.

Example 15-4 Partially Redacted Character Values

```
BEGIN
DBMS_REDACT.ADD_POLICY(
  object_schema    => 'mavis',
  object_name      => 'cust_info',
  column_name      => 'ssn',
  policy_name      => 'redact_cust_ssns3',
  function_type     => DBMS_REDACT.PARTIAL,
  function_parameters => DBMS_REDACT.REDACT_US_SSN_F5,
  expression       => '1=1',
  policy_description => 'Partially redacts 1st 5 digits in SS numbers',
  column_description => 'ssn contains Social Security numbers');
END;
/
```

Query and redacted result:

```
SELECT ssn FROM mavis.cust_info;
```

```
SSN
-----
XXX-XX-4320
XXX-XX-4323
XXX-XX-4325
XXX-XX-4329
```

15.9.4 Creating Partial Redaction Policies Using Character Data Types

The DBMS_REDACT.ADD_POLICY function_parameters parameter enables you to redact character data types.

- [Settings for Character Data Types](#)
Oracle Data Redaction provides special settings to configure policies that use character data types.
- [Example: Partial Redaction Policy Using a Character Data Type](#)
The DBMS_REDACT.ADD_POLICY PL/SQL procedure can create a partial redaction policy that uses a character data type.

15.9.4.1 Settings for Character Data Types

Oracle Data Redaction provides special settings to configure policies that use character data types.

When you set the `DBMS_REDACT.ADD_POLICY` `function_parameters` parameter to define partial redaction of character data types, enter values for the following settings in the order shown. Separate each value with a comma

① Note

Be aware that you must use a fixed width character set for the partial redaction. In other words, each character redacted must be replaced by another of equal byte length. If you want to use a variable-length character set (for example, UTF-8), then you must use a regular expression-based redaction. See [Syntax for Creating a Regular Expression-Based Redaction Policy](#) for more information.

The settings are as follows:

1. **Input format:** Defines how the data is currently formatted. Enter V for each character that potentially can be redacted, such as all of the digits in a credit card number. Enter F for each character that you want to format using a formatting character, such as hyphens or blank spaces in the credit card number. Ensure that each character has a corresponding V or F value. (The input format values are not case-sensitive.)
2. **Output format:** Defines how the displayed data should be formatted. Enter V for each character to be potentially redacted. Replace each F character in the input format with the character that you want to use for the displayed output, such as a hyphen. (The output format values are not case-sensitive.)
3. **Mask character:** Specifies the character to be used for the redaction. Enter a single character to use for the redaction, such as an asterisk (*).
4. **Starting digit position:** Specifies the starting V digit position for the redaction.
5. **Ending digit position:** Specifies the ending V digit position for the redaction. Do not include the F positions when you decide on the ending position value.

For example, the following setting redacts the first 12 V digits of the credit card number 5105 1051 0510 5100, and replaces the F positions (which are blank spaces) with hyphens to format it in a style normally used for credit card numbers, resulting in ****-****-****-4320.

```
function_parameters => 'VVVFVVVFVVVFVVV,VVV-VVV-VVV-VVV,*,1,12',
```

① See Also

[General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#) for information about other `DBMS_REDACT.ADD_POLICY` parameters

15.9.4.2 Example: Partial Redaction Policy Using a Character Data Type

The `DBMS_REDACT.ADD_POLICY` PL/SQL procedure can create a partial redaction policy that uses a character data type.

[Example 15-5](#) shows how to redact Social Security numbers that are in a `VARCHAR2` data type column and to preserve the character hyphens in the Social Security number.

Example 15-5 Partially Redacted Character Values

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'mavis',
    object_name   => 'cust_info',
    column_name   => 'ssn',
    policy_name   => 'redact_cust_ssns2',
    function_type  => DBMS_REDACT.PARTIAL,
    function_parameters => 'VVVFVVVVVV,VVV-VV-VVVV,*,1,5',
    expression    => '1=1',
    policy_description => 'Partially redacts Social Security numbers',
    column_description => 'ssn contains character Social Security numbers');
END;
/
```

Query and redacted result:

```
SELECT ssn FROM mavis.cust_info;
```

```
SSN
-----
***_**-4320
***_**-4323
***_**-4325
***_**-4329
```

15.9.5 Creating Partial Redaction Policies Using Number Data Types

The `DBMS_REDACT.ADD_POLICY` `function_parameters` parameter can redact number data types.

- [Settings for Number Data Types](#)
When you set values for the number data type, you must specify a mask character, a starting digit position, and ending digit position.
- [Example: Partial Redaction Policy Using a Number Data Type](#)
The `DBMS_REDACT.ADD_POLICY` procedure can create a partial redaction policy that uses a number data type.

15.9.5.1 Settings for Number Data Types

When you set values for the number data type, you must specify a mask character, a starting digit position, and ending digit position.

For partial redaction of number data types, you can enter values for the following settings for the `function_parameters` parameter of the `DBMS_REDACT.ADD_POLICY` procedure, in the order shown.

1. **Mask character:** Specifies the character to display. Enter a number from 0 to 9.

2. **Starting digit position:** Specifies the starting digit position for the redaction, such as 1 for the first digit.
3. **Ending digit position:** Specifies the ending digit position for the redaction.

For example, the following setting redacts the first five digits of the Social Security number 987654321, resulting in 999994321.

```
function_parameters => '9,1,5',
```

① See Also

[General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#) for information about other DBMS_REDACT.ADD_POLICY parameters

15.9.5.2 Example: Partial Redaction Policy Using a Number Data Type

The DBMS_REDACT.ADD_POLICY procedure can create a partial redaction policy that uses a number data type.

[Example 15-6](#) shows how to partially redact a set of Social Security numbers in the mavis.cust_info table, for any application user who logs in. (Hence, the expression parameter evaluates to TRUE.)

This type of redaction is useful when the application is expecting a formatted number and not a string. In this scenario, the Social Security numbers are in a column of the data type NUMBER. In other words, the ssn column contains numbers only, not other characters such as hyphens or blank spaces.

Example 15-6 Partially Redacted Data Redaction Numeric Values

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema      => 'mavis',
    object_name        => 'cust_info',
    column_name        => 'ssn',
    policy_name        => 'redact_cust_ssns1',
    function_type       => DBMS_REDACT.PARTIAL,
    function_parameters => '7,1,5',
    expression         => '1=1',
    policy_description  => 'Partially redacts Social Security numbers',
    column_description  => 'ssn contains numeric Social Security numbers');
END;
/
```

Query and redacted result:

```
SELECT ssn FROM mavis.cust_info;
```

```
SSN
-----
777774320
777774323
777774325
777774329
```


15.9.6 Creating Partial Redaction Policies Using Date-Time Data Types

The `DBMS_REDACT.ADD_POLICY` function_parameters parameter can redact date-time data types.

- [Settings for Date-Time Data Types](#)
Oracle Data Redaction provides special settings for configuring date-time data types.
- [Example: Partial Redaction Policy Using Date-Time Data Type](#)
The `DBMS_REDACT.ADD_POLICY` procedure can create a partial redaction policy that uses the date-time data type.

15.9.6.1 Settings for Date-Time Data Types

Oracle Data Redaction provides special settings for configuring date-time data types.

For partial redaction of date-time data types, enter values for the following `DBMS_REDACT.ADD_POLICY` function_parameters parameter settings.

Enter these values in the order shown:

1. m: Redacts the month. To redact with a month name, append 1–12 to lowercase m. For example, m5 displays as MAY. To omit redaction, enter an uppercase M.
2. d: Redacts the day of the month. To redact with a day of the month, append 1–31 to a lowercase d. For example, d7 displays as 07. If you enter a higher number than the days of the month (for example, 31 for the month of February), then the last day of the month is displayed (for example, 28). To omit redaction, enter an uppercase D.
3. y: Redacts the year. To redact with a year, append 1–9999 to a lowercase y. For example, y1984 displays as 84. To omit redaction, enter an uppercase Y.
4. h: Redacts the hour. To redact with an hour, append 0–23 to a lowercase h. For example, h20 displays as 20. To omit redaction, enter an uppercase H.
5. m: Redacts the minute. To redact with a minute, append 0–59 to a lowercase m. For example, m30 displays as 30. To omit redaction, enter an uppercase M.
6. s: Redacts the second. To redact with a second, append 0–59 to a lowercase s. For example, s45 displays as 45. To omit redaction, enter an uppercase S.

① See Also

[General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#) for information about other `DBMS_REDACT.ADD_POLICY` parameters

15.9.6.2 Example: Partial Redaction Policy Using Date-Time Data Type

The `DBMS_REDACT.ADD_POLICY` procedure can create a partial redaction policy that uses the date-time data type.

[Example 15-7](#) shows how to partially redact a date. This example redacts the birth year of customers; replacing it with 13, but retaining the remaining values.

Example 15-7 Partially Redacted Data Redaction Using Date-Time Values

```

BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema    => 'mavis',
    object_name      => 'cust_info',
    column_name      => 'birth_date',
    policy_name      => 'redact_cust_bdate',
    function_type     => DBMS_REDACT.PARTIAL,
    function_parameters => 'mdy2013HMS',
    expression       => '1=1',
    policy_description => 'Replaces birth year with 2013',
    column_description => 'birth_date contains customer's birthdate');
END;
/

```

Query and redacted result:

```
SELECT birth_date FROM mavis.cust_info;
```

```

BIRTH_DATE
07-DEC-13 09.45.40.000000 AM
12-OCT-13 04.23.29.000000 AM

```

15.10 Creating a Regular Expression-Based Redaction Policy

A regular expression-based redaction policy enables you to redact data based on a search-and-replace model.

- [About Creating Regular Expression-Based Redaction Policies](#)
Regular expression-based redaction enables you to search for patterns of data to redact.
- [Syntax for Creating a Regular Expression-Based Redaction Policy](#)
The `regexp_*` parameters of the `DBMS_REDACT.ADD_POLICY` procedure can create a regular expression-based redaction policy.
- [Regular Expression-Based Redaction Policies Using Formats](#)
The `DBMS_REDACT.ADD_POLICY` procedure `regexp_pattern` and `regexp_replace_string` parameters both support formats.
- [Custom Regular Expression Redaction Policies](#)
You can customize regular expressions in Data Redaction policies.

15.10.1 About Creating Regular Expression-Based Redaction Policies

Regular expression-based redaction enables you to search for patterns of data to redact.

For example, you can use regular expressions to redact email addresses, which can have varying character lengths. It is designed for use with character data only. You can use formats for the search and replace operation, or you can create custom pattern formats.

You cannot use regular expressions to redact a subset of the values in a column. The `REGEXP_PATTERN` (regular expression pattern) must match *all* of the values in order for the `REGEXP_REPLACE_STRING` setting to take effect, and the `REGEXP_REPLACE_STRING` must change the value.

For rows where the `REGEXP_PATTERN` fails to match, Data Redaction performs `DBMS_REDACT.FULL` redaction. This mitigates the risk of a mistake in the `REGEXP_PATTERN` which causes the regular expression to fail to match all of the values in the column, from showing the actual data for those rows which it failed to match.

In addition, if no change to the value occurs as a result of the `REGEXP_REPLACE_STRING` setting during regular expression replacement operation, Data Redaction performs `DBMS_REDACT.FULL` redaction.

15.10.2 Syntax for Creating a Regular Expression-Based Redaction Policy

The `regexp_*` parameters of the `DBMS_REDACT.ADD_POLICY` procedure can create a regular expression-based redaction policy.

The `DBMS_REDACT.ADD_POLICY` fields for creating a regular expression-based data redaction policy are as follows:

```
DBMS_REDACT.ADD_POLICY (
  object_schema      IN VARCHAR2 := NULL,
  object_name        IN VARCHAR2,
  column_name        IN VARCHAR2 := NULL,
  policy_name        IN VARCHAR2,
  function_type       IN BINARY_INTEGER := NULL,
  expression         IN VARCHAR2,
  enable             IN BOOLEAN := TRUE,
  regexp_pattern      IN VARCHAR2 := NULL,
  regexp_replace_string IN VARCHAR2 := NULL,
  regexp_position     IN BINARY_INTEGER := 1,
  regexp_occurrence   IN BINARY_INTEGER := 0,
  regexp_match_parameter IN VARCHAR2 := NULL);
```

In this specification:

- `object_schema`, `object_name`, `column_name`, `policy_name`, `expression`, `enable`: See [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#).
- `function_type`: Specifies the type of redaction. For regular expression based redaction, use either `DBMS_REDACT.REGEXP` or `DBMS_REDACT.REGEXP_WIDTH`.

If you use the `DBMS_REDACT.REGEXP` redaction type, then no truncation occurs. This applies even if the redacted value is wider than the column width, and if the Oracle Call Interface width attribute (`OCI_ATTR_CHAR_SIZE`) of the column is *not* preserved. (It becomes 4000, just as it does when the `REGEXP_REPLACE` SQL operator is used on a column.)

Using the `DBMS_REDACT.REGEXP_WIDTH` redaction type truncates any redacted value that exceeds the width of the column, and ensures that the OCI width attribute of the column (`OCI_ATTR_CHAR_SIZE`) remains unchanged.

Note the following:

- Use the `DBMS_REDACT.REGEXP_WIDTH` function type if your applications depend on the value of the `OCI_ATTR_CHAR_SIZE` attribute. For example, applications that are built using the Oracle OLE DB Provider interface are sensitive to the value of the `OCI_ATTR_CHAR_SIZE` attribute. If you use `DBMS_REDACT.REGEXP` as the redaction type, then the `OCI_ATTR_CHAR_SIZE` always becomes 4000. This setting makes it unsuitable as the redaction type of policies on tables that are used by Oracle OLE DB based applications. See *Oracle Call Interface Programmer's Guide* for more information about Oracle Call Interface parameter attributes.
- When you set the `function_type` parameter to `DBMS_REDACT.REGEXP` or `DBMS_REDACT.REGEXP_WIDTH`, omit the `function_parameters` parameter from the `DBMS_REDACT.ADD_POLICY` procedure.
- Specify the regular expression parameters in much the same way that you specify the `pattern`, `replace`, `position`, `occurrence`, and `match_parameter` arguments to the

REGEXP_REPLACE SQL function. See *Oracle Database SQL Language Reference* for information about the REGEXP_REPLACE SQL function.

- `regexp_pattern`: Describes the search pattern for data that must be matched. If it finds a match, then Oracle Database replaces the data as specified by the `regexp_replace_string` setting. See the following sections for more information:
 - [Regular Expression-Based Redaction Policies Using Formats](#)
 - [Custom Regular Expression Redaction Policies](#)
- `regexp_replace_string`: Specifies how you want to replace the data to be redacted. See the following sections for more information:
 - [Regular Expression-Based Redaction Policies Using Formats](#)
 - [Custom Regular Expression Redaction Policies](#)
- `regexp_position`: Specifies the starting position for the string search. The value that you enter must be a positive integer indicating the character of the `column_name` data where Oracle Database should begin the search. The default is 1 or the `DBMS_REDACT.RE_BEGINNING` format, meaning that Oracle Database begins the search at the first character of the `column_name` data.
- `regexp_occurrence`: Specifies how to perform the search and replace operation. The value that you enter must be a nonnegative integer indicating the occurrence of the replace operation:
 - If you specify 0 or the `DBMS_REDACT.RE_ALL` format, then Oracle Database replaces all the occurrences of the match.
 - If you specify the `DBMS_REDACT.RE_FIRST` format, then Oracle Database replaces the first occurrence of the match.
 - If you specify a positive integer *n*, then Oracle Database replaces the *n*th occurrence of the match.

If the occurrence is greater than 1, then the database searches for the second occurrence beginning with the first character following the first occurrence of pattern, and so forth.

- `regexp_match_parameter`: Specifies a text literal that lets you change the default matching behavior of the function. The behavior of this parameter is the same for this function as for the REGEXP_REPLACE SQL function. See *Oracle Database SQL Language Reference* for detailed information.

To filter the search so that it is not case sensitive, specify the `RE_CASE_INSENSITIVE` format.

15.10.3 Regular Expression-Based Redaction Policies Using Formats

The `DBMS_REDACT.ADD_POLICY` procedure `regexp_pattern` and `regexp_replace_string` parameters both support formats.

- [Regular Expression Formats](#)
The regular expression formats represent commonly used expressions, such as the replacement of digits within a credit card number.
- [Example: Regular Expression Redaction Policy Using Formats](#)
The `DBMS_REDACT.ADD_POLICY` procedure can create a regular expression redaction policy that uses formats.

15.10.3.1 Regular Expression Formats

The regular expression formats represent commonly used expressions, such as the replacement of digits within a credit card number.

[Table 15-9](#) describes the formats that you can use with the `regexp_pattern` parameter in the `DBMS_REDACT.ADD_POLICY` procedure.

Table 15-9 Formats for the `regexp_pattern` Parameter

Format	Description
<code>DBMS_REDACT.RE_PATTERN_ANY_DIGIT</code>	Searches for any digit. Replaces the identified pattern with the characters specified by the <code>regexp_replace_string</code> parameter. The <code>DBMS_REDACT.RE_PATTERN_ANY_DIGIT</code> is commonly used with the following values of the <code>regexp_replace_string</code> parameter: <pre>regexp_replace_string => DBMS_REDACT.RE_REDACT_WITH_SINGLE_X,</pre> This setting replaces any matched digit with the X character. The following setting replaces any matched digit with the 1 character. <pre>regexp_replace_string => DBMS_REDACT.RE_REDACT_WITH_SINGLE_1,</pre>
<code>DBMS_REDACT.RE_PATTERN_CC_L6_T4</code>	Searches for the middle digits of any credit card (other than American Express) that has 6 leading digits and 4 trailing digits. Replaces the identified pattern with the characters specified by the <code>regexp_replace_string</code> parameter. The appropriate <code>regexp_replace_string</code> setting to use with this format is <code>DBMS_REDACT.RE_REDACT_CC_MIDDLE_DIGITS</code>, which finds any credit card that could have 6 leading and 4 trailing digits left as actual data. It then redacts the middle digits.
<code>DBMS_REDACT.RE_PATTERN_CCN</code>	Matches credit card numbers other than American Express credit card numbers. The appropriate <code>regexp_replace_string</code> setting to use with this format is <code>DBMS_REDACT.RE_REDACT_CCN</code>. The end result is a redaction of all the digits except the last 4.
<code>DBMS_REDACT.RE_PATTERN_AMEX_CCN</code>	Matches American Express credit card numbers. The appropriate <code>regexp_replace_string</code> setting to use with this format is <code>DBMS_REDACT.RE_REDACT_AMEX_CCN</code>. The end result is a redaction of all the digits except the last 5.

Table 15-9 (Cont.) Formats for the regexp_pattern Parameter

Format	Description
DBMS_REDACT.RE_PATTERN_US_PHONE	Searches for any U.S. telephone number. Replaces the identified pattern with the characters specified by the <code>regexp_replace_string</code> parameter The appropriate <code>regexp_replace_string</code> setting to use with this format is <code>DBMS_REDACT.RE_REDACT_US_PHONE_L7</code>, which finds United States phone numbers and then redacts the last 7 digits.
DBMS_REDACT.RE_PATTERN_EMAIL_ADDRESS	Searches for any email address. Replaces the identified pattern with the characters specified by the <code>regexp_replace_string</code> parameter The appropriate <code>regexp_replace_string</code> settings that you can use with this format are as follows: <code>RE_REDACT_EMAIL_NAME</code>, which finds any email address and redacts the email user name <code>RE_REDACT_EMAIL_DOMAIN</code>, which finds any email address and redacts the email domain <code>RE_REDACT_EMAIL_ENTIRE</code>, which finds any email address and redacts the entire email address
DBMS_REDACT.RE_PATTERN_IP_ADDRESS	Searches for an IP address. Replaces the identified pattern with the characters specified by the <code>regexp_replace_string</code> parameter. The appropriate <code>regexp_replace_string</code> setting to use with this format is <code>DBMS_REDACT.RE_REDACT_IP_L3</code>, which replaces the last section of the dotted decimal string representation of an IP address with the characters 999 to indicate that it was redacted.

[Table 15-10](#) describes formats that you can use with the `regexp_replace_string` parameter in the `DBMS_REDACT.ADD_POLICY` procedure.

Table 15-10 Formats for the regexp_replace_string Parameter

Format	Description
DBMS_REDACT.RE_REDACT_WITH_SINGLE_X	Replaces the data with a single X character for each of the actual data characters. For example, the credit card number 5105 1051 0510 5100 could be replaced with XXXX XXXX XXXX XXXX.
DBMS_REDACT.RE_REDACT_WITH_SINGLE_1	Replaces the data with a single 1 digit for each of the actual data digits. For example, the credit card number 5105 1051 0510 5100 could be replaced with 1111 1111 1111 1111.

Table 15-10 (Cont.) Formats for the regexp_replace_string Parameter

Format	Description
DBMS_REDACT.RE_REDACT_CC_MIDDLE_DIGITS	Redacts the middle digits in credit card numbers, as specified by setting the regexp_pattern parameter with the RE_PATTERN_CC_L6_T4 format. The redaction replaces each redacted character with an X. For example, the credit card number 5105 1051 0510 5100 could be replaced with 5105 10XX XXXX 5100.
DBMS_REDACT.RE_REDACT_CCN	Redacts the first 12 digits of a credit card number other than an American Express card number. For example, 4012888888881881 is redacted to *****1881.
DBMS_REDACT.RE_REDACT_AMEX_CCN	Redacts the first 10 digits of an American Express number. For example, 378282246310005 is redacted to *****10005.
DBMS_REDACT.RE_REDACT_PHONE_L7	Redacts the last 7 digits of U.S. telephone numbers, as specified by setting the regexp_pattern parameter with the RE_PATTERN_US_PHONE format. The redaction replaces each redacted character with an X. This setting only applies to hyphenated phone numbers, not phone numbers with spaces. For example, the telephone number 415-555-0100 could be replaced with 415-XXX-XXXX.
DBMS_REDACT.RE_REDACT_EMAIL_NAME	Redacts the email name as specified by setting the regexp_pattern parameter with the RE_PATTERN_EMAIL_ADDRESS format. The redaction replaces the email user name with four x characters. For example, the email address psmith@example.com could be replaced with xxxx@example.com.
DBMS_REDACT.RE_REDACT_EMAIL_DOMAIN	Redacts the email domain name as specified by setting the regexp_pattern parameter with the RE_PATTERN_EMAIL_ADDRESS format. The redaction replaces the domain with five x characters. For example, the email address psmith@example.com could be replaced with psmith@xxxxx.com.
DBMS_REDACT.RE_REDACT_IP_L3	Redacts the last three digits of the IP address as specified by setting the regexp_pattern parameter with the RE_PATTERN_IP_ADDRESS format. For example, the IP address 192.0.2.254 could be replaced with 192.0.2.999, which is an invalid IP address.

① See Also

[General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#) for information about other DBMS_REDACT.ADD_POLICY parameters

15.10.3.2 Example: Regular Expression Redaction Policy Using Formats

The `DBMS_REDACT.ADD_POLICY` procedure can create a regular expression redaction policy that uses formats.

[Example 15-8](#) shows how to use regular expression formats to redact credit card numbers.

Example 15-8 Regular Expression Data Redaction Character Values

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema    => 'mavis',
    object_name      => 'cust_info',
    column_name      => 'cc_num',
    policy_name      => 'redact_cust_cc_nums',
    function_type     => DBMS_REDACT.REGEXP,
    function_parameters => NULL,
    expression       => '1=1',
    regexp_pattern    => DBMS_REDACT.RE_PATTERN_CC_L6_T4,
    regexp_replace_string => DBMS_REDACT.RE_REDACT_CC_MIDDLE_DIGITS,
    regexp_position   => DBMS_REDACT.RE_BEGINNING,
    regexp_occurrence => DBMS_REDACT.RE_FIRST,
    regexp_match_parameter => DBMS_REDACT.RE_CASE_INSENSITIVE,
    policy_description => 'Regular expressions to redact credit card numbers',
    column_description => 'cc_num contains customer credit card numbers');
END;
/
```

Query and redacted result:

```
SELECT cc_num FROM mavis.cust_info;
```

```
CC_NUM
-----
401288XXXXXX1881
411111XXXXXX1111
555555XXXXXX1111
511111XXXXXX1118
```

15.10.4 Custom Regular Expression Redaction Policies

You can customize regular expressions in Data Redaction policies.

- [Settings for Custom Regular Expressions](#)
Oracle Data Redaction provides special settings to configure policies that use regular expressions.
- [Example: Custom Regular Expression Redaction Policy](#)
The `DBMS_REDACT.ADD_POLICY` procedure `regexp*` parameters can create a custom regular expression redaction policy.

15.10.4.1 Settings for Custom Regular Expressions

Oracle Data Redaction provides special settings to configure policies that use regular expressions.

To create custom regular expression redaction policies, you use the following parameters in the `DBMS_REDACT.ADD_POLICY` procedure:

- `regexp_pattern`: This pattern is usually a text literal and can be of any of the data types `CHAR`, `VARCHAR2`, `NCHAR`, or `NVARCHAR2`. The pattern can contain up to 512 bytes. For further information about writing the regular expression for the `regexp_pattern` parameter, see the description of the `pattern` argument of the `REGEXP_REPLACE` SQL function in *Oracle Database SQL Language Reference*, because the support that Data Redaction provides for regular expression matching is similar to that of the `REGEXP_REPLACE` SQL function.
- `regexp_replace_string`: This data can be of any of the data types `CHAR`, `VARCHAR2`, `NCHAR`, or `NVARCHAR2`. The `regexp_replace_string` can contain up to 500 back references to subexpressions in the form `\n`, where `n` is a number from 1 to 9. If you want to include a backslash (`\`) in the `regexp_replace_string` setting, then you must precede it with the escape character, which is also a backslash. For example, to literally replace the matched pattern with `\2` (rather than replace it with the second matched subexpression of the matched pattern), you enter `\\2` in the `regexp_replace_string` setting. For more information, see *Oracle Database SQL Language Reference*.

① See Also

[General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#) for information about other `DBMS_REDACT.ADD_POLICY` parameters

15.10.4.2 Example: Custom Regular Expression Redaction Policy

The `DBMS_REDACT.ADD_POLICY` procedure `regexp*` parameters can create a custom regular expression redaction policy.

[Example 15-9](#) shows how to use regular expressions to redact the `emp_id` column data. In this example, taken together, the `regexp_pattern` and `regexp_replace_string` parameters do the following: first, find the pattern of 9 digits. For reference, break them into three groups that contain the first 3, the next 2, and then the last 4 digits. Then, replace all 9 digits with `XXXXX` concatenated with the third group (the last 4 digits) as found in the original pattern.

Query and redacted result:

```
SELECT emp_id FROM mavis.cust_info;

EMP_ID
-----
XXXXX1234
XXXXX5678
```

Example 15-9 Partially Redacted Data Redaction Using Regular Expressions

```
BEGIN
DBMS_REDACT.ADD_POLICY(
  object_schema    => 'mavis',
  object_name      => 'cust_info',
  column_name      => 'emp_id',
  policy_name      => 'redact_cust_ids',
  function_type     => DBMS_REDACT.REGEXP,
  expression       => '1=1',
  regexp_pattern   => '(\d\d\d)(\d\d)(\d\d\d\d)',
  regexp_replace_string => 'XXXXX\3',
  regexp_position  => 1,
  regexp_occurrence => 0,
  regexp_match_parameter => 'i',
```

```

    policy_description    => 'Redacts customer IDs using regular expression',
    column_description    => 'emp_id contains employee ID numbers');
END;
/

```

15.11 Creating a Random Redaction Policy

A random redaction policy presents redacted data as randomly generated values, such as UkJsl32[[]]]s.

- [Syntax for Creating a Random Redaction Policy](#)
A random redaction policy presents the redacted data to the querying application user as randomly generated values, based on the column data type.
- [Example: Random Redaction Policy](#)
You can use the DBMS_REDACT.ADD_POLICY PL/SQL procedure create a random redaction policy.

15.11.1 Syntax for Creating a Random Redaction Policy

A random redaction policy presents the redacted data to the querying application user as randomly generated values, based on the column data type.

Be aware that LOB columns are not supported.

The DBMS_REDACT.ADD_POLICY fields for creating a random redaction policy are as follows:

```

DBMS_REDACT.ADD_POLICY (
    object_schema          IN VARCHAR2 := NULL,
    object_name            IN VARCHAR2,
    column_name            IN VARCHAR2 := NULL,
    policy_name            IN VARCHAR2,
    function_type          IN BINARY_INTEGER := NULL,
    expression             IN VARCHAR2,
    enable                 IN BOOLEAN := TRUE);

```

In this specification:

- object_schema, object_name, column_name, policy_name, expression, enable: See [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#).
- function_type: Specifies the function used to set the type of redaction. Enter DBMS_REDACT.RANDOM.

If you omit the function_type parameter, then the default redaction function_type setting is DBMS_REDACT.FULL.

Remember that the data type of the column determines which function_type settings that you are permitted to use. See [Comparison of Full, Partial, and Random Redaction Based on Data Types](#).

15.11.2 Example: Random Redaction Policy

You can use the DBMS_REDACT.ADD_POLICY PL/SQL procedure create a random redaction policy.

[Example 15-10](#) shows how to generate random values. Each time you run the SELECT statement, the output will be different.

Example 15-10 Randomly Redacted Data Redaction Values

```

BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'mavis',
    object_name   => 'cust_info',
    column_name   => 'login_username',
    policy_name   => 'redact_cust_rand_username',
    function_type => DBMS_REDACT.RANDOM,
    expression    => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') = ''APP_USER''');
END;
/

```

Query and redacted result:

```
SELECT login_username FROM mavis.cust_info;
```

```

LOGIN_USERNAME
-----
N[CG{\pTVcK

```

15.12 Creating a Policy That Uses No Redaction

You can create policies that use no redaction at all, for when you want to test the policy in a development environment.

- [Syntax for Creating a Policy with No Redaction](#)
The None redaction type option can be used to test the internal operation of redaction policies.
- [Example: Performing No Redaction](#)
The DBMS_REDACT.ADD_POLICY procedure can create a policy that performs no redaction.

15.12.1 Syntax for Creating a Policy with No Redaction

The None redaction type option can be used to test the internal operation of redaction policies.

The None redaction type has no effect on the query results against tables that have policies defined on them. You can use this option to test the redaction policy definitions before applying them to a production environment. Be aware that LOB columns are not supported.

The DBMS_REDACT.ADD_POLICY fields for creating a policy with no redaction are as follows:

```

DBMS_REDACT.ADD_POLICY (
  object_schema      IN VARCHAR2 := NULL,
  object_name        IN VARCHAR2,
  column_name        IN VARCHAR2 := NULL,
  policy_name        IN VARCHAR2,
  function_type       IN BINARY_INTEGER := NULL,
  expression         IN VARCHAR2,
  enable             IN BOOLEAN := TRUE);

```

In this specification:

- object_schema, object_name, column_name, policy_name, expression, enable: See [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#).
- function_type: Specifies the functions used to set the type of data redaction. Enter DBMS_REDACT.NONE.

If you omit the `function_type` parameter, then the default redaction `function_type` setting is `DBMS_REDACT.FULL`.

15.12.2 Example: Performing No Redaction

The `DBMS_REDACT.ADD_POLICY` procedure can create a policy that performs no redaction.

[Example 15-11](#) shows how to create a Data Redaction policy that does not redact any of the displayed values.

Example 15-11 No Redacted Data Redaction Values

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'mavis',
    object_name   => 'cust_info',
    column_name   => 'user_name',
    policy_name   => 'redact_cust_no_vals',
    function_type => DBMS_REDACT.NONE,
    expression    => '1=1');
END;
/
```

Query and redacted result:

```
SELECT user_name FROM mavis.cust_info;
```

```
USER_NAME
-----
IDA NEAU
```

15.13 Exemption of Users from Oracle Data Redaction Policies

You can exempt users from having Oracle Data Redaction policies applied to the data they access.

To do so, you should grant the users the `EXEMPT REDACTION POLICY` system privilege. Grant this privilege to trusted users only.

In addition to users who were granted this privilege, user `SYS` is also exempt from all Data Redaction policies. The person who creates the Data Redaction policy is by default not exempt from it, unless this person is user `SYS` or has the `EXEMPT REDACTION POLICY` system privilege.

Note the following:

- Users who have the `INSERT` privilege on a table can insert values into a redacted column, regardless of whether a Data Redaction policy exists on the table. Data Redaction only affects SQL `SELECT` statements (that is, queries) issued by a user, and has no effect on any other SQL issued by a user, including `INSERT`, `UPDATE`, or `DELETE` statements. (See the next bullet for exceptions to this rule.)
- Users cannot perform a `CREATE TABLE AS SELECT` where any of the columns being selected (source columns) is protected by a Data Redaction policy (and similarly, any DML operation where the source is a redacted column), unless the user was granted the `EXEMPT REDACTION POLICY` system privilege.
- The `EXEMPT REDACTION POLICY` system privilege is included in the `DBA` role, but this privilege must be granted explicitly to users because it is not included in the `WITH ADMIN OPTION` for `DBA` role grants. Users who were granted the `DBA` role are exempt from redaction

policies because the DBA role contains the EXP_FULL_DATABASE role, which is granted the EXEMPT REDACTION POLICY system privilege.

Related Topics

- [Restriction of Administrative Access to Oracle Data Redaction Policies](#)
You can restrict the list of users who can create, view and edit Data Redaction policies.
- [Oracle Data Pump Security Model for Oracle Data Redaction](#)
The DATAPUMP_EXP_FULL_DATABASE role includes the powerful EXEMPT REDACTION POLICY system privilege.

15.14 Altering an Oracle Data Redaction Policy

The DBMS_REDACT.ALTER_POLICY procedure enables you to modify Oracle Data Redaction policies.

- [About Altering Oracle Data Redaction Policies](#)
The DBMS_REDACT.ALTER_POLICY procedure alters a Data Redaction policy.
- [Syntax for the DBMS_REDACT.ALTER_POLICY Procedure](#)
The DBMS_REDACT.ALTER_POLICY procedure syntax can be used to alter all types of the Data Redaction policies.
- [Parameters Required for DBMS_REDACT.ALTER_POLICY Actions](#)
The DBMS_REDACT.ALTER_POLICY procedure provides parameters that can perform various actions, such as adding or modifying a column.
- [Tutorial: Altering an Oracle Data Redaction Policy](#)
You can redact multiple columns in a table or view, with each column having its own redaction setting.

15.14.1 About Altering Oracle Data Redaction Policies

The DBMS_REDACT.ALTER_POLICY procedure alters a Data Redaction policy.

If the policy is already enabled, then you do not need to disable it first, and after you alter the policy, it remains enabled.

You can find the names of existing Data Redaction policies by querying the POLICY_NAME column of the REDACTION_POLICIES data dictionary view, and information about the columns, functions, and parameters specified in a policy by querying the REDACTION_COLUMNS view. To find the current value for policies that use full data redaction, you can query the REDACTION_VALUES_FOR_TYPE_FULL data dictionary view.

The action parameter specifies the type of modification that you want to perform. At a minimum, you must include the object_name and policy_name parameters when you run this procedure.

Related Topics

- [REDACTION_COLUMNS Data Dictionary View Behavior When a View Is Invalid](#)
When an Oracle Data Redaction policy exists on a column of a view, and the view becomes invalid, the Data Redaction policy remains visible in the REDACTION_COLUMNS data dictionary view.

15.14.2 Syntax for the DBMS_REDACT.ALTER_POLICY Procedure

The DBMS_REDACT.ALTER_POLICY procedure syntax can be used to alter all types of the Data Redaction policies.

The syntax for the DBMS_REDACT.ALTER_POLICY procedure is as follows:

```
DBMS_REDACT.ALTER_POLICY (  
  object_schema      IN VARCHAR2 := NULL,  
  object_name        IN VARCHAR2 := NULL,  
  policy_name        IN VARCHAR2,  
  action             IN BINARY_INTEGER := DBMS_REDACT.ADD_COLUMN,  
  column_name        IN VARCHAR2 := NULL,  
  function_type       IN BINARY_INTEGER := DBMS_REDACT.FULL,  
  function_parameters IN VARCHAR2 := NULL,  
  expression         IN VARCHAR2 := NULL,  
  regexp_pattern     IN VARCHAR2 := NULL,  
  regexp_replace_string IN VARCHAR2 := NULL,  
  regexp_position    IN BINARY_INTEGER := NULL,  
  regexp_occurrence  IN BINARY_INTEGER := NULL,  
  regexp_match_parameter IN VARCHAR2 := NULL,  
  policy_description IN VARCHAR2 := NULL,  
  column_description IN VARCHAR2 := NULL);
```

In this specification:

- **action:** Enter one of the following values to define the kind of action to use:
 - DBMS_REDACT.MODIFY_COLUMN if you plan to change the column_name value.
 - DBMS_REDACT.ADD_COLUMN if you plan to add a new column (in addition to columns that are already protected by the policy) for redaction. This setting is the default for the action parameter.
 - DBMS_REDACT.DROP_COLUMN if you want to remove redaction from a column.
 - DBMS_REDACT.MODIFY_EXPRESSION if you plan to change the expression value. Each policy can have only one policy expression. In other words, when you modify the policy expression, you are replacing the existing policy expression with a new policy expression.
 - DBMS_REDACT.SET_POLICY_DESCRIPTION if you want to change the description of the policy.
 - DBMS_REDACT.SET_COLUMN_DESCRIPTION if you want to change the description of the column.

① See Also

- [Parameters Required for DBMS_REDACT.ALTER_POLICY Actions](#)
- [General Syntax of the DBMS_REDACT.ADD_POLICY Procedure](#) for information about the remaining parameters

15.14.3 Parameters Required for DBMS_REDACT.ALTER_POLICY Actions

The DBMS_REDACT.ALTER_POLICY procedure provides parameters that can perform various actions, such as adding or modifying a column.

[Table 15-11](#) shows the combinations of these parameters.

Table 15-11 Parameters Required for Various DBMS_REDACT.ALTER_POLICY Actions

Desired Alteration	Parameters to Set
Add or modify a column	• action (DBMS_REDACT.MODIFY_COLUMN) • column_name • function_type • function_parameters (if necessary) • regexp* (if necessary)
Change the policy expression	• action (DBMS_REDACT.MODIFY_EXPRESSION) • expression
Change the description of the policy	• action (DBMS_REDACT.SET_POLICY_DESCRIPTION) • policy_description
Change the description of the column	• action (DBMS_REDACT.SET_COLUMN_DESCRIPTION) • column_description
Drop a column	• action (DBMS_REDACT.DROP_COLUMN) • column_name

15.14.4 Tutorial: Altering an Oracle Data Redaction Policy

You can redact multiple columns in a table or view, with each column having its own redaction setting.

The exercise in this section shows how to modify a Data Redaction policy so that multiple columns are redacted. It also shows how to change the expression setting for the policy. To accomplish this, you must run the DBMS_REDACT.ALTER_POLICY procedure in stages.

1. Connect to the PDB as a user who has privileges to create users and grant them privileges.

2. Create the following users:

```
GRANT CREATE SESSION TO dr_admin IDENTIFIED BY password;
GRANT CREATE SESSION TO sales_rep IDENTIFIED BY password;
GRANT CREATE SESSION TO support_rep IDENTIFIED BY password;
```

3. Grant EXECUTE on the DBMS_REDACT PL/SQL package to user dr_admin:

```
GRANT EXECUTE ON DBMS_REDACT TO dr_admin;
```

4. Connect as user OE.

5. Create and populate a table that contains customer credit card information.

```
CREATE TABLE cust_order_info(
  first_name varchar2(20),
  last_name varchar2(20),
  address varchar2(30),
  city varchar2(30),
  state varchar2(3),
  zip varchar2(5),
```

```
cc_num varchar(19),
cc_exp varchar2(7));
```

```
INSERT INTO cust_order_info VALUES ('Jane','Dough','39 Mockingbird Lane', 'San
Francisco', 'CA', 94114, '5105 1051 0510 5100', '10/2018');
INSERT INTO cust_order_info VALUES ('Mary','Hightower','2319 Maple Street',
'Sonoma', 'CA', 95476, '5111 1111 1111 1118', '03/2019');
INSERT INTO cust_order_info VALUES ('Herbert','Donahue','292 Winsome Way', 'San
Francisco', 'CA', 94117, '5454 5454 5454 5454', '08/2018');
```

6. Grant the SELECT privilege on the cust_order_info table to the sales_rep and support_rep users.

```
GRANT SELECT ON cust_order_info TO sales_rep, support_rep;
```

7. Connect as user dr_admin.
8. Create and enable policy to redact the credit card number.

```
BEGIN DBMS_REDACT.ADD_POLICY(
    object_schema => 'oe',
    object_name   => 'cust_order_info',
    column_name   => 'cc_num',
    policy_name   => 'redact_cust_cc_info',
    function_type => DBMS_REDACT.REGEXP,
    function_parameters => NULL,
    expression    => '1=1',
    regexp_pattern => DBMS_REDACT.RE_PATTERN_CCN,
    regexp_replace_string => DBMS_REDACT.RE_REDACT_CCN,
    regexp_position => NULL,
    regexp_occurrence => NULL,
    regexp_match_parameter => NULL,
    policy_description => 'Partially redacts credit card info',
    column_description => 'cc_num_number lists credit card numbers');
END;
/
```

9. Modify the policy to include redaction of the expiration date.

```
BEGIN DBMS_REDACT.ALTER_POLICY(
    object_schema => 'oe',
    object_name   => 'cust_order_info',
    policy_name   => 'redact_cust_cc_info',
    action        => DBMS_REDACT.ADD_COLUMN,
    column_name   => 'cc_exp',
    function_type  => DBMS_REDACT.RANDOM);
END;
/
```

10. Modify the policy again, to use a condition so that the sales_rep user views the redacted values and the support_rep user views the [actual data](#).

```
BEGIN
    DBMS_REDACT.ALTER_POLICY(
        object_schema => 'oe',
        object_name   => 'cust_order_info',
        policy_name   => 'redact_cust_cc_info',
        action        => DBMS_REDACT.MODIFY_EXPRESSION,
        expression    => 'SYS_CONTEXT(''USERENV'', 'SESSION_USER') = ''SALES_REP''');
END;
/
```

11. To test the policy, have the two users query the cust_order_info table.

First, connect as support_rep and query the table.


```
SELECT cc_num, cc_exp FROM OE.cust_order_info;
```

CC_NUM	CC_EXP
5105 1051 0510 5100	10/2018
5111 1111 1111 1118	03/2019
5454 5454 5454 5454	08/2018

User support_rep can view the actual data. Next, connect as sales_rep and query the table.

```
SELECT cc_num, cc_exp FROM OE.cust_order_info;
```

CC_NUM	CC_EXP
*****5100	1ST=033
*****1119	OZA.W4C
*****5454	B(9+;01

The actual data is redacted using for user sales_rep.

- As user dr_admin, alter the cust_order_info to include a condition so that only support_rep can see the redacted data but sales_rep cannot.

```
BEGIN
  DBMS_REDACT.ALTER_POLICY(
    object_schema => 'oe',
    object_name   => 'cust_order_info',
    policy_name   => 'redact_cust_cc_info',
    action        => DBMS_REDACT.MODIFY_EXPRESSION,
    expression    => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') =
''SUPPORT_REP''');
END;
/
```

- Have the users test the policy again.

First, support_rep tests the policy:

```
SELECT cc_num, cc_exp FROM OE.cust_order_info;
```

CC_NUM	CC_EXP
*****5100	1^XMF~`
*****1119	qz+9=#S
*****5454	*KCaUkm

User support_rep can no longer view the actual data; it is now redacted.

Next, connect as sales_rep and query the table.

```
SELECT cc_num, cc_exp FROM OE.cust_order_info;
```

CC_NUM	CC_EXP
5105 1051 0510 5100	10/2018
5111 1111 1111 1118	03/2019
5454 5454 5454 5454	08/2018

User sales_rep now can view the actual data.

- If you do not need the components of this tutorial, then you can remove them. Connect as dr_admin and then drop the policy.

```
BEGIN
  DBMS_REDACT.DROP_POLICY (
    object_schema => 'oe',
    object_name   => 'cust_order_info',
    policy_name   => 'redact_cust_cc_info');
END;
/
```

Connect as a security administrator and drop the users.

```
DROP USER dr_admin;
DROP USER sales_rep;
DROP USER support_rep;
```

Connect as user OE and drop the cust_order_info table.

```
DROP TABLE cust_order_info;
```

15.15 Redacting Multiple Columns

You can redact more than one column in a Data Redaction policy.

- [Adding Columns to a Data Redaction Policy for a Single Table or View](#)
You can redact columns of different data types, using different redaction types, for one table or view.
- [Example: Redacting Multiple Columns](#)
The DBMS_REDACT.ALTER_POLICY procedure can redact multiple columns.

15.15.1 Adding Columns to a Data Redaction Policy for a Single Table or View

You can redact columns of different data types, using different redaction types, for one table or view.

1. Create the policy for the first column that you want to redact.
2. Use the DBMS_REDACT.ALTER_POLICY procedure to add the next column to the policy.

As necessary, set the action, column_name, function_type, and function_parameters (or the parameters that begin with regexp_) parameters to define the redaction for the new column, but do not change the object_schema, object_name, policy_name, or expression parameters. Each redacted column continues to have the same redaction parameters that were used to create it.

15.15.2 Example: Redacting Multiple Columns

The DBMS_REDACT.ALTER_POLICY procedure can redact multiple columns.

[Example 15-12](#) shows how to add a column to an existing Data Redaction policy. In this example, the action parameter specifies that a new column must be added, using DBMS_REDACT.ADD_COLUMN. The name of the new column, card_num, is set by the column_name parameter.

Example 15-12 Adding a Column to a Data Redaction Policy

```
BEGIN
  DBMS_REDACT.ALTER_POLICY(
```

```

object_schema => 'mavis',
object_name   => 'cust_info',
policy_name   => 'redact_cust_user_ids',
action        => DBMS_REDACT.ADD_COLUMN,
column_name   => 'card_num',
function_type  => DBMS_REDACT.FULL,
function_parameters => '',
expression    => 'SYS_CONTEXT(''SYS_SESSION_ROLES'', 'ADM') = ''TRUE''';
END;
/

```

15.16 Disabling and Enabling an Oracle Data Redaction Policy

You can disable and then reenable Oracle Data Redactions policies as necessary.

- [Disabling an Oracle Data Redaction Policy](#)
The `DBMS_REDACT.DISABLE_POLICY` procedure disables Oracle Data Redaction policies.
- [Enabling an Oracle Data Redaction Policy](#)
The `DBMS_REDACT.ENABLE_POLICY` procedure enables Oracle Data Redaction policies.

15.16.1 Disabling an Oracle Data Redaction Policy

The `DBMS_REDACT.DISABLE_POLICY` procedure disables Oracle Data Redaction policies.

You can find the names of existing Data Redaction policies and whether they are enabled by querying the `POLICY_NAME` and `ENABLE` columns of the `REDACTION_POLICIES` view. However, as long as the policy still exists, you cannot create another policy for that table or view, even if the original policy is disabled. In other words, if you want to create a different policy on the same table column, then you must drop the first policy before you can create and use the new policy.

- To disable a Data Redaction policy, run the `DBMS_REDACT.DISABLE_POLICY` procedure, using the following syntax:

```

DBMS_REDACT.DISABLE_POLICY (
  object_schema  IN VARCHAR2 DEFAULT NULL,
  object_name    IN VARCHAR2,
  policy_name    IN VARCHAR2);

```

In this specification:

- `object_schema`: Specifies the schema of the object on which the Data Redaction policy will be applied. If you omit this setting (or enter `NULL`), then Oracle Database uses the name of the current schema.
- `object_name`: Specifies the name of the table or view to be used for the Data Redaction policy.
- `policy_name`: Specifies the name of the policy to be disabled.

[Example 15-13](#) shows how to disable a Data Redaction policy.

Example 15-13 Disabling a Data Redaction Policy

```

BEGIN
  DBMS_REDACT.DISABLE_POLICY (
    object_schema => 'mavis',
    object_name   => 'cust_info',
    policy_name   => 'redact_cust_user_ids');
END;
/

```

15.16.2 Enabling an Oracle Data Redaction Policy

The `DBMS_REDACT.ENABLE_POLICY` procedure enables Oracle Data Redaction policies.

Immediately after you create a new policy, you do not need to enable it; the creation process handles that for you. To find the names of existing Data Redaction policies and whether they are enabled, you can query the `POLICY_NAME` and `ENABLE` columns of the `REDACTION_POLICIES` view. After you run the procedure to enable the policy, the enablement takes effect immediately.

- To enable a Data Redaction policy, run the `DBMS_REDACT.ENABLE_POLICY` procedure, using the following syntax.

```
DBMS_REDACT.ENABLE_POLICY (  
    object_schema      IN VARCHAR2 DEFAULT NULL,  
    object_name        IN VARCHAR2,  
    policy_name        IN VARCHAR2);
```

In this specification:

- `object_schema`: Specifies the schema of the object on which the Data Redaction policy will be applied. If you omit this setting (or enter `NULL`), then Oracle Database uses the name of the current schema.
- `object_name`: Specifies the name of the table or view to be used for the Data Redaction policy.
- `policy_name`: Specifies the name of the policy to be enabled.

[Example 15-14](#) shows how to enable a Data Redaction policy.

Example 15-14 Enabling a Data Redaction Policy

```
BEGIN  
    DBMS_REDACT.ENABLE_POLICY (  
        object_schema => 'mavis',  
        object_name   => 'cust_info',  
        policy_name   => 'redact_cust_user_ids');  
END;  
/
```

15.17 Dropping an Oracle Data Redaction Policy

The `DBMS_REDACT.DROP_POLICY` procedure drops Oracle Data Redaction policies.

You can drop an Oracle Data Redaction policy whether it is enabled or disabled. You can find the names of existing Data Redaction policies, by querying the `POLICY_NAME` column of the `REDACTION_POLICIES` view. When you drop a table or view that is associated with an Oracle Data Redaction policy, the policy is automatically dropped. As a best practice, drop the policy first, and then drop the table or view afterward. See [Dropped Oracle Data Redaction Policies When the Recycle Bin Is Enabled](#) for more information.

- To drop a Data Redaction policy, run the `DBMS_REDACT.DROP_POLICY` procedure.

Use the following syntax:

```
DBMS_REDACT.DROP_POLICY (  
    object_schema      IN VARCHAR2 DEFAULT NULL,  
    object_name        IN VARCHAR2,  
    policy_name        IN VARCHAR2);
```

In this specification:

- `object_schema`: Specifies the schema of the object to which the Data Redaction policy applies. If you omit this setting (or enter NULL), then Oracle Database uses the name of the current schema.
- `object_name`: Specifies the name of the table or view to be used for the Data Redaction policy.
- `policy_name`: Specifies the name of the policy to be dropped.

After you run the `DBMS_REDACT.DROP_POLICY` procedure, the drop takes effect immediately.

[Example 15-15](#) shows how to drop a Data Redaction policy.

Example 15-15 Dropping a Data Redaction Policy

```
BEGIN
  DBMS_REDACT.DROP_POLICY (
    object_schema => 'mavis',
    object_name   => 'cust_info',
    policy_name   => 'redact_cust_user_ids');
END;
/
```

15.18 Tutorial: SQL Expressions to Build Reports with Redacted Values

SQL expressions can be used to build reports based on columns that have Oracle Data Redaction policies defined on them.

The values used in the SQL expression will be redacted. This redaction occurs in such a way that the redaction takes place before the SQL expression is evaluated: the result value that is displayed in the report is the end result of the evaluated SQL expression over the redacted values, rather than the redacted result of the SQL expression as a whole.

1. Create the following Data Redaction policy for the `HR.EMPLOYEES` table.

This policy will replace the first 4 digits of the value from the `SALARY` column with the number 9 and the first digit of the value from the `COMMISSION_PCT` column with a 9.

```
BEGIN
  DBMS_REDACT.ADD_POLICY(
    object_schema => 'HR',
    object_name   => 'EMPLOYEES',
    column_name   => 'SALARY',
    column_description => 'emp_sal_comm shows employee salary and commission',
    policy_name   => 'redact_emp_sal_comm',
    policy_description => 'Partially redacts the emp_sal_comm column',
    function_type => DBMS_REDACT.PARTIAL,
    function_parameters => '9,1,4',
    expression    => '1=1');
END;
/
BEGIN
  DBMS_REDACT.ALTER_POLICY(
    object_schema => 'HR',
    object_name   => 'EMPLOYEES',
    policy_name   => 'redact_emp_sal_comm',
    action        => DBMS_REDACT.ADD_COLUMN,
    column_name   => 'COMMISSION_PCT',
```

```

function_type      => DBMS_REDACT.PARTIAL,
function_parameters => '9,1,1',
expression         => '1=1');
END;
/

```

2. Log in to the HR schema and then run the following report.

This report will use the SQL expression (SALARY + COMMISSION_PCT) to combine the employees' salaries and commissions.

```

SELECT (SALARY + COMMISSION_PCT) total_emp_compensation
FROM HR.EMPLOYEES
WHERE DEPARTMENT_ID = 80;

```

```

TOTAL_EMP_COMPENSATION
-----
                9999.9
                9999.95
                99990.95
...

```

3. Use SQL expressions for the report, including concatenation.

For example:

```

SELECT 'Employee ID ' || EMPLOYEE_ID ||
       ' has a salary of ' || SALARY ||
       ' and a commission of ' || COMMISSION_PCT || '.' detailed_emp_compensation
FROM HR.EMPLOYEES
WHERE DEPARTMENT_ID = 80
ORDER BY EMPLOYEE_ID;

```

```

DETAILED_EMP_COMPENSATION
-----
Employee ID 150 has a salary of 99990 and a commission of .9.
Employee ID 151 has a salary of 9999 and a commission of .95.
Employee ID 152 has a salary of 9999 and a commission of .95.
...

```

4. Connect the user who created the redact_emp_sal_comm Data Redaction policy and then run the following statement to drop the policy.

```

BEGIN
  DBMS_REDACT.DROP_POLICY (
    object_schema => 'HR',
    object_name   => 'EMPLOYEES',
    policy_name   => 'redact_emp_sal_comm');
END;
/

```

15.19 Using Trace Files to Troubleshoot Oracle Data Redaction Policies

Trace files for Oracle Data Redaction can be generated for either the system level or the session level.

You must set the tracing to high. There is no low tracing level for Oracle Data Redaction.

1. Connect to the PDB as a user who has the ALTER SYSTEM or ALTER SESSION privilege.
2. Enable tracing.

- At the system level, for all sessions:

```
ALTER SYSTEM SET events 'TRACE[RADM] disk=high';
```

- At the session level, for the current session:

```
ALTER SESSION SET events 'TRACE[RADM] disk=high';
```

To find problems with queries involving redacted columns, include `/* KZDMPRQB_DUMP */` in the query. For example:

```
SELECT /* KZDMPRQB_DUMP */ redacted_column FROM
table_with_redaction_policy;
```

Search for KCCF2MASK and KCCF2MASKDEP in the dump. The subsequent part of the dump files gives Oracle Redaction-specific insights.

3. To disable tracing:

- At the system level, for all sessions:

```
ALTER SYSTEM SET EVENTS 'TRACE[RADM] OFF';
```

- At the session level, for the current session:

```
ALTER SESSION SET EVENTS 'TRACE[RADM] OFF';
```

15.20 Oracle Data Redaction Policy Data Dictionary Views

Oracle Database provides data dictionary views that list information about Data Redaction policies.

Before you can query these views, you must be granted the `SELECT_CATALOG_ROLE` role.

[Table 15-12](#) lists the Data Redaction data dictionary views.

Table 15-12 Data Redaction Views

View	Description
REDACTION_COLUMNS	Describes all of the redacted columns in the database, providing the the owner of the table or view within which the column resides, the object name, the column name, the type of redaction function, the parameters to the redaction function (if any), and a description of the redaction policy. If a policy expression has been created, displays the default object-wide policy expression's SQL expression.
REDACTION_EXPRESSIONS	Displays the names of existing policy expressions and their SQL expressions
REDACTION_POLICIES	Describes all of the data redaction policies in the database. It includes information about the object owner, object name, policy name, policy expression, whether the policy is enabled, and a description of the Data Redaction policy.
REDACTION_VALUES_FOR_TYPE_FULL	Shows the current redaction values for Data Redaction policies that use full redaction

Managing Oracle Data Redaction Policies in Oracle Enterprise Manager

Oracle Enterprise Manager Cloud Control (Cloud Control) can manage Oracle Data Redaction policies and formats.

- [About Using Oracle Data Redaction in Oracle Enterprise Manager](#)
Oracle Enterprise Manager Cloud Control provides an unified user interface for creating and managing Oracle Data Redaction policies.
- [Oracle Data Redaction Workflow](#)
First, you should create sensitive column types and formats if necessary, and then create the Oracle Data Redaction policy afterward.
- [Management of Sensitive Column Types in Enterprise Manager](#)
A sensitive column type categorizes table column sensitive information into a sensitive information type, such as credit card numbers.
- [Managing Oracle Data Redaction Formats Using Enterprise Manager](#)
Oracle Data Redaction provides redaction formats to be used directly within a redaction policy to redact data.
- [Managing Oracle Data Redaction Policies Using Enterprise Manager](#)
You can create, edit, view, and delete Oracle Data Redaction policies in Enterprise Manager Cloud Control.
- [Managing Named Data Redaction Policy Expressions Using Enterprise Manager](#)
You can manage Oracle Data Redaction policy expressions in Enterprise Manager Cloud Control.

16.1 About Using Oracle Data Redaction in Oracle Enterprise Manager

Oracle Enterprise Manager Cloud Control provides an unified user interface for creating and managing Oracle Data Redaction policies.

You can do the following:

- Create and manage custom Oracle Data Redaction formats, which were previously known as Data Redaction shortcuts. (This functionality is not available from the command line.)
- Create and manage sensitive column types directly from the Oracle Data Redaction pages. While you create a Data Redaction policy, Cloud Control uses sensitive column types to obtain the Oracle Data Redaction formats that are relevant to the column that you are redacting.

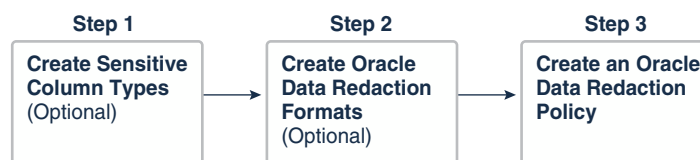
Note

Ensure that you have the latest plug-in for Oracle Enterprise Manager. If you have the Oracle Database plug-in release 13.1.1.0.0, then you can create named Data Redaction policy expressions in Oracle Enterprise Manager.

16.2 Oracle Data Redaction Workflow

First, you should create sensitive column types and formats if necessary, and then create the Oracle Data Redaction policy afterward.

The following figure illustrates this process:



1. (Optional) If you want to map the database columns (that contain the data that you want to redact) to new sensitive column types, then create the required sensitive column types as described in [Management of Sensitive Column Types in Enterprise Manager](#).
2. (Optional) If you want to redact the data (present in a particular database column) using a custom redaction format, then create the required redaction format as described in [Creating a Custom Oracle Data Redaction Format Using Enterprise Manager](#).
3. Create an Oracle Data Redaction policy for the required database, as described in [Creating an Oracle Data Redaction Policy Using Enterprise Manager](#).

Note

When you create an Oracle Data Redaction policy, it is enabled by default. For information on how to disable an enabled redaction policy, see [Enabling or Disabling an Oracle Data Redaction Policy in Enterprise Manager](#).

16.3 Management of Sensitive Column Types in Enterprise Manager

A sensitive column type categorizes table column sensitive information into a sensitive information type, such as credit card numbers.

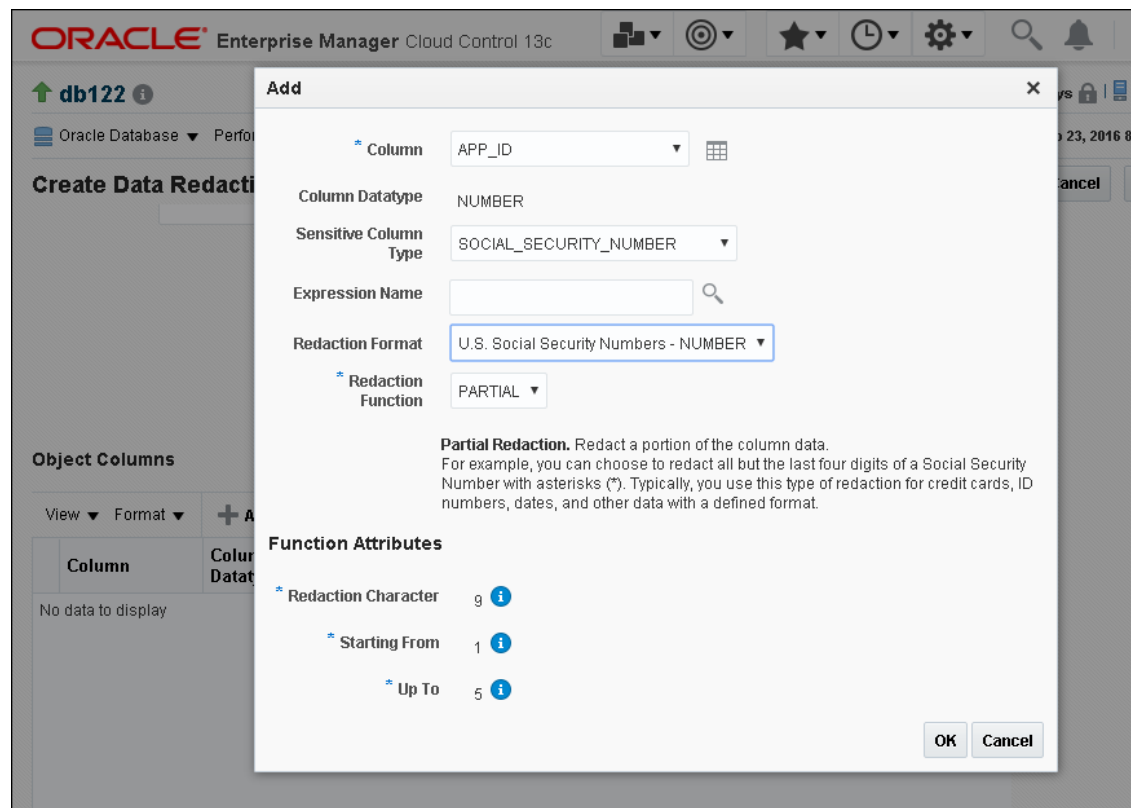
Sensitive column types use a combination of the column name, column comments, and the data pattern defined using a regular expression to tag a column to a particular sensitive information type.

While you create Oracle Data Redaction policies, redaction formats are filtered on the basis of the chosen sensitive column type, thus saving time and effort. For example, if the database table column that you want to redact contains U.S. Social Security numbers, and you select the

SOCIAL_SECURITY_NUMBER sensitive column type for the column while adding it to the Oracle Data Redaction policy, the default redaction formats that you can use to redact the column data are filtered, and only the relevant redaction formats are displayed.

[Figure 16-1](#) illustrates the filtering of Oracle Data Redaction formats based on sensitive column types.

Figure 16-1 Oracle Data Redaction Formats Filtered on the Basis of Sensitive Column Types



Note

This functionality is available only if you have the Enterprise Manager for Oracle Database plug-in 12.1.0.7 or later deployed in your system.

As part of the Application Data Modeling feature, Oracle provides a number of default sensitive column types that a database column can be mapped to.

[Figure 16-2](#) displays some of the default sensitive column types. To access this page, click **Manage Sensitive Column Types** on the Data Redaction Formats page.

Figure 16-2 Default Sensitive Column Types

Application Data Modeling 		
Application Data Modeling > Sensitive Column Types		
Sensitive Column Types		
View ▼	 Create...  Create Like...  Edit...  Delete... 	
Name	Description	Author
 CREDIT_CARD_NUMBER	Identifies credit card number columns. Samples: 5199-1234-1234-123...	Oracle
 EMAIL_ID	Identifies email address columns. Samples: jsmith@comgmt.com, Ja...	Oracle
 IP_ADDRESS	Identifies IP address columns. Samples: 7.7.7.1, 78.78.78.12, 789.789...	Oracle
 ISBN_10	Identifies 10 digit International Standard Book Number columns. Samp...	Oracle
 ISBN_13	Identifies 13 digit International Standard Book Number columns. Samp...	Oracle
 NATIONAL_INSURANCE_NUMBER	Identifies National Insurance number (UK) columns. Samples: ZR 50 1...	Oracle
 PHONE_NUMBER	Identifies phone number columns. Samples: 555-1212, (123)555-121...	Oracle
 SOCIAL_INSURANCE_NUMBER	Identifies Social Insurance Number (Canada) columns. Samples: 884-...	Oracle
 SOCIAL_SECURITY_NUMBER	Identifies Social Security number columns. Samples: 123-45-6789, 12...	Oracle
 UNDEFINED	Sensitive column type not defined.	Oracle
 UNIVERSAL_PRODUCT_CODE	Identifies Universal Product Code columns. Samples: 1-23456-78901-...	Oracle

If none of the default sensitive column types are suitable for the database column that contains the data that you want to redact, you can create a new sensitive column type, or create a sensitive column type that is based on an existing sensitive column type.

16.4 Managing Oracle Data Redaction Formats Using Enterprise Manager

Oracle Data Redaction provides redaction formats to be used directly within a redaction policy to redact data.

- [About Managing Oracle Data Redaction Formats Using Enterprise Manager](#)
The Oracle Data Redaction formats are used for commonly redacted data, such as ID numbers, credit cards, or phone numbers.
- [Creating a Custom Oracle Data Redaction Format Using Enterprise Manager](#)
You can create and save custom Oracle Data Redaction formats using Enterprise Manager Cloud Control.
- [Editing a Custom Oracle Data Redaction Format Using Enterprise Manager](#)
You can edit custom Oracle Data Redaction formats using Enterprise Manager Cloud Control, but not in SQL*Plus.
- [Viewing Oracle Data Redaction Formats Using Enterprise Manager](#)
Enterprise Manager Cloud Control displays the details of the Oracle-supplied and custom Oracle Data Redaction formats.
- [Deleting a Custom Oracle Data Redaction Format Using Enterprise Manager](#)
You can delete custom Oracle Data Redaction formats using Enterprise Manager Cloud Control.

16.4.1 About Managing Oracle Data Redaction Formats Using Enterprise Manager

The Oracle Data Redaction formats are used for commonly redacted data, such as ID numbers, credit cards, or phone numbers.

You can use several default Oracle Data Redaction formats (previously known as Oracle Data Redaction templates). As an example of the Oracle Data Redaction formats, a set of Social Security number formats enable you to quickly designate ways to redact Social Security numbers, such as redacting the first five numbers of the Social Security number.

[Figure 16-3](#) displays the default Oracle Data Redaction formats.

Figure 16-3 Default Oracle Data Redaction Formats

Format Name	Sensitive Column Type	Function Type	Description
American Express Credit Card Numbers - Formatted	CREDIT_CARD_NUMBER	PARTIAL	Redact the American Express Credit Card Number by replacing every 12th digit with the digit 0.
American Express Credit Card Numbers - NUMBER	CREDIT_CARD_NUMBER	PARTIAL	Redact the American Express Credit Card Number by replacing every 12th digit with the digit 0.
American Express Credit Card Numbers - Partially Redacted	CREDIT_CARD_NUMBER	REGEX	Redact the American Express Credit Card Number by replacing every 12th digit with the digit 0.
American Express Credit Card Numbers - Random	CREDIT_CARD_NUMBER	RANDOM	Redact the American Express Credit Card Number by replacing every 12th digit with the digit 0.
Canadian Social Insurance Numbers - Formatted	SOCIAL_INSURANCE_NUMBER	PARTIAL	Redact the Canadian Social Insurance Number by replacing every 9th digit with the digit 0.
Canadian Social Insurance Numbers - NUMBER	SOCIAL_INSURANCE_NUMBER	PARTIAL	Redact the Canadian Social Insurance Number by replacing every 9th digit with the digit 0.
Canadian Social Insurance Numbers - Random	SOCIAL_INSURANCE_NUMBER	RANDOM	Redact the Canadian Social Insurance Number by replacing every 9th digit with the digit 0.
Canadian Social Insurance Numbers - VARCHAR	SOCIAL_INSURANCE_NUMBER	PARTIAL	Redact the Canadian Social Insurance Number by replacing every 9th digit with the digit 0.
Credit Card Numbers - Formatted	CREDIT_CARD_NUMBER	PARTIAL	Redact the Credit Card Number by replacing every 12th digit with the digit 0.
Credit Card Numbers - NUMBER	CREDIT_CARD_NUMBER	PARTIAL	Redact the Credit Card Number by replacing every 12th digit with the digit 0.
Credit Card Numbers - Partially Redacted	CREDIT_CARD_NUMBER	REGEX	Redact the Credit Card Number by replacing every 12th digit with the digit 0.

Each default Oracle Data Redaction format consists of a specific redaction function that determines the redacted output when the redaction format is used in an Oracle Data Redaction policy. For example, the Credit Card Numbers - NUMBER default redaction format replaces the first twelve digits of the column data with the digit 0, when it is used in an Oracle Data Redaction policy. That is, if the column data is 5555555555554444, the redacted output will be 0000000000004444.

If you have deployed the Enterprise Manager for Oracle Database plug-in 12.1.0.7 or higher on your system, then you can also create and save custom redaction formats, which you can then use in your redaction policies.

16.4.2 Creating a Custom Oracle Data Redaction Format Using Enterprise Manager

You can create and save custom Oracle Data Redaction formats using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.

The URL is as follows:

`https://host:port/em`

2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.

Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.

6. Select the **Formats** tab.
7. Do one of the following:
 - To create a new redaction format, click **Create**.
 - To create a redaction format that is based on a default format, select the format and then click **Create Like**.

If you select **Create**, then the following dialog box appears:

Create

* Format Name

* Description

Sensitive Column Type

UNDEFINED

* Redaction Function

REGEX

Regular Expression Based Redaction.

Specifies a regular expression that represents the column data that will be redacted.

Function Attributes

* Pattern

Specifies the regular expression pattern to be searched. Example: 'd\d\d\d\d\d678' for number like '012345678'

* Replace String

Example: Use 'XXXXXXXX3' (replace string) to redact '012345678' (actual value) which matches 'd\d\d\d (d\d\d) (d\d\d)' (regexp pattern) to 'XXXXXXXX678' (redacted value). Note that the '3' in the replace string preserves the actual data in the third set of parentheses in the pattern.

* Position

Specifies the starting position of the string search. The default is 1, meaning it begins the search from the first character of column data.

* Occurrence

Specifies how to perform the search and replace operation. Zero means it replaces all occurrences. Positive integer 'n' would replace nth occurrence of the string.

Match Parameter

Ignore case

Specifies the matching parameters for the REGEX redaction function.

OK

Cancel

- Provide a name and a description for the redaction format that you want to create.

If you want to map the redaction format to a particular sensitive column type (such that the created redaction format can be used to redact the data of a column that is associated with the sensitive column type), then select a value for **Sensitive Column Type**.

Select the function that the format should use to redact the column data. For **Redaction Function**, select as follows:

- FULL** if the format should redact the entire column data.
- PARTIAL** if the format should redact only a part of the column data. Ensure that you provide the function attributes, as well as the data type that you want to use the redaction format for.
- REGEX** if the format should redact data based on a regular expression. Ensure that you provide the function attributes.
- RANDOM** if the format should redact data in a random manner, using randomly generated values

- **NONE** if the format will be used to only test the definition of a redaction policy, and not redact any column data
9. Click **OK** to create and save the custom redaction format.
This format now can be used to create a redaction policy.

Related Topics

- [Oracle Data Redaction Features and Capabilities](#)
Oracle Data Redaction provides a variety of ways to redact different types of data.
- [Creating an Oracle Data Redaction Policy Using Enterprise Manager](#)
You can create an Oracle Data Redaction policy using Enterprise Manager Cloud Control.

16.4.3 Editing a Custom Oracle Data Redaction Format Using Enterprise Manager

You can edit custom Oracle Data Redaction formats using Enterprise Manager Cloud Control, but not in SQL*Plus.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
 2. From the **Targets** menu, select **Databases**.
 3. Select **Search List**, then click the name of a database target.
 4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
 5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
 6. Select the **Formats** tab.
 7. Select the custom redaction format that you want to edit, then click **Edit**.
- A dialog box similar to the following appears:

Edit

* **Format Name** American Express Credit Cards - FULL

* **Description** Redact the American Ex

Sensitive Column Type CREDIT_CARD_NUMBER ▼

* **Redaction Function** FULL ▼

Full Redaction. Redact all the contents of the column data. The redacted value returned to the querying user depends on the data type of the column. For example, columns of the NUMBER data type are redacted with a zero (0) and character data types are redacted with a blank space. These default values can be changed if necessary.

OK Cancel

8. (Optional) Choose to edit the format description, sensitive column type, redaction function, and the redaction function attributes.
9. Click **OK** to save the edited format.

16.4.4 Viewing Oracle Data Redaction Formats Using Enterprise Manager

Enterprise Manager Cloud Control displays the details of the Oracle-supplied and custom Oracle Data Redaction formats.

1. Log into Oracle Enterprise Manager Cloud Control as either user **SYSTEM** or **SYSMAN**.

The URL is as follows:

`https://host:port/em`

2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.

Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.

6. Select the **Formats** tab.

The Data Redaction Formats page appears, similar to the following page.

Data Redaction

Oracle Data Redaction provides an easy way to quickly redact sensitive information that is displayed in applications without altering the underlying database blocks on disk or in cache. Data is redacted in real-time according to flexible multi-factor policies. Data Redaction is licensed as part of Oracle Advanced Security.

Policies
Expressions
Formats

View
Format

Create
 Create Like
 Edit
 View
 Delete
 Refresh
 Manage Sensitive Column Types

Format Name	Sensitive Column Type	Function Type	Description
American Express Credit Card Numbers - Formatted	CREDIT_CARD_NUMBER	PARTIAL	Redact the American Express Credit Card Number
American Express Credit Card Numbers - NUMBER	CREDIT_CARD_NUMBER	PARTIAL	Redact the American Express Credit Card Number
American Express Credit Card Numbers - Partially Redacted	CREDIT_CARD_NUMBER	REGEX	Redact the American Express Credit Card Number
American Express Credit Card Numbers - Random	CREDIT_CARD_NUMBER	RANDOM	Redact the American Express Credit Card Number
Canadian Social Insurance Numbers - Formatted	SOCIAL_INSURANCE_NUMBER	PARTIAL	Redact the Canadian Social Insurance Number b
Canadian Social Insurance Numbers - NUMBER	SOCIAL_INSURANCE_NUMBER	PARTIAL	Redact the Canadian Social Insurance number b
Canadian Social Insurance Numbers - Random	SOCIAL_INSURANCE_NUMBER	RANDOM	Redact the Canadian Social Insurance Number b
Canadian Social Insurance Numbers - VARCHAR	SOCIAL_INSURANCE_NUMBER	PARTIAL	Redact the Canadian Social Insurance number b
Credit Card Numbers - Formatted	CREDIT_CARD_NUMBER	PARTIAL	Redact the Credit Card Number by replacing ever
Credit Card Numbers - NUMBER	CREDIT_CARD_NUMBER	PARTIAL	Redact the Credit Card Number by replacing ever
Credit Card Numbers - Partially Redacted	CREDIT_CARD_NUMBER	REGEX	Redact the Credit Card Number by replacing ever

7. Select the required redaction format, then click **View**.

16.4.5 Deleting a Custom Oracle Data Redaction Format Using Enterprise Manager

You can delete custom Oracle Data Redaction formats using Enterprise Manager Cloud Control.

You can only delete custom Oracle Data Redaction formats, and not the redaction formats that are provided by Oracle.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.

The URL is as follows:

`https://host:port/em`

2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.

5. Log in to the database, if you are prompted to do so.

Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.

6. Select the **Formats** tab.
7. Select the custom redaction format that you want to delete, and then click **Delete**.
8. In the Confirmation dialog box, click **Yes** or **No**.

16.5 Managing Oracle Data Redaction Policies Using Enterprise Manager

You can create, edit, view, and delete Oracle Data Redaction policies in Enterprise Manager Cloud Control.

- [About Managing Oracle Data Redaction Policies Using Enterprise Manager](#)
Use the Data Redaction page in Cloud Control to manage Oracle Data Redaction policies.
- [Creating an Oracle Data Redaction Policy Using Enterprise Manager](#)
You can create an Oracle Data Redaction policy using Enterprise Manager Cloud Control.
- [Editing an Oracle Data Redaction Policy Using Enterprise Manager](#)
You can edit an Oracle Data Redaction policy using Enterprise Manager Cloud Control.
- [Viewing Oracle Data Redaction Policy Details Using Enterprise Manager](#)
You can find Oracle Data Redaction policy details such as whether the policy is enabled by using Enterprise Manager Cloud Control.
- [Enabling or Disabling an Oracle Data Redaction Policy in Enterprise Manager](#)
An Oracle Data Redaction policy is executed at run time only if it is enabled. When you create an Oracle Data Redaction policy, it is enabled by default.
- [Deleting an Oracle Data Redaction Policy Using Enterprise Manager](#)
You can delete an Oracle Data Redaction policy using Enterprise Manager Cloud Control.

16.5.1 About Managing Oracle Data Redaction Policies Using Enterprise Manager

Use the Data Redaction page in Cloud Control to manage Oracle Data Redaction policies.

To redact the data present in a particular database table or view column, you must create an Oracle Data Redaction policy. Data is redacted using a redaction format that is specified by the Oracle Data Redaction policy. To redact data, you can use any of the Oracle-supplied redaction formats, or create and use a custom redaction format. If the table or view column that contains the data that you want to redact is mapped to a sensitive column type, Oracle uses the mapping to recommend suitable redaction formats for the data. Thus, Oracle Data Redaction policies encapsulate database schemas, database table and view columns, sensitive column types, and Oracle Data Redaction formats.

[Figure 16-4](#) shows the Data Redaction page, which enables you to create and manage Oracle Data Redaction policies in Cloud Control.

Figure 16-4 Oracle Data Redaction Policies Page

Data Redaction

Oracle Data Redaction provides an easy way to quickly redact sensitive information that is displayed in applications without altering the underlying database blocks on disk or in cache. Data is redacted in real-time according to flexible multi-factor policies. Data Redaction is licensed as part of Oracle Advanced Security.

Policies Expressions Formats

▲ Search Data Redaction Policies

Schema %

Table/View %

Policy Name %

Go

Policies

View ▼ Format ▼ Create Edit View Enable Disable Delete Freeze Wrap

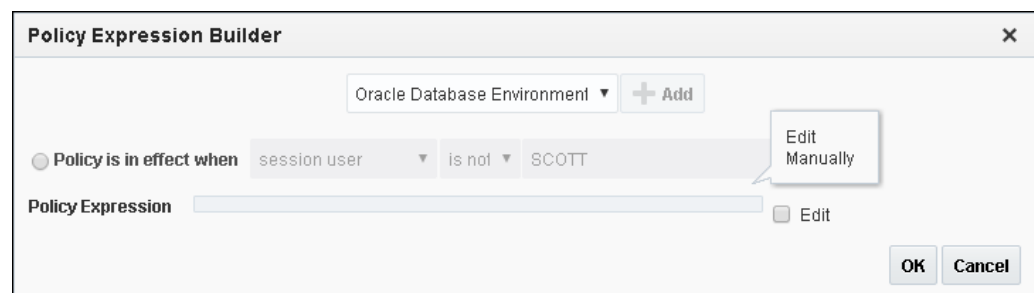
Schema	Table/View	Policy Name	Enabled	Redacted Columns
HR	EMPLOYEES	test	✓	1
HR	JOBS	dr_redact	✓	0

16.5.2 Creating an Oracle Data Redaction Policy Using Enterprise Manager

You can create an Oracle Data Redaction policy using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target for which you want to create an Oracle Data Redaction policy.

4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
6. In the Policies section of the Policies tab, select **Create**.
If this is the first time that you are creating a Data Redaction policy, then the Data Redaction: Set up for enabling column sensitive type discovery dialog box appears. This feature enables the use of column sensitive type discovery for Data Redaction policies.
To accomplish this, Enterprise Manager creates the GET_COL_DATA_SENSITIVE_TYPES procedure in the DBSNMP schema. To perform a sensitive type discovery for a selected column while creating an Oracle Data Redaction policy, a user must have the EXECUTE privilege on the DBSNMP.GET_COL_DATA_SENSITIVE_TYPES procedure. If the database is protected by Oracle Database Vault, then ensure that any users who must create Data Redaction policies are participants to realms that protect the DBSNMP schema.
7. If the Data Redaction: Set up for enabling column sensitive type discovery dialog box appears and if the current login user does not have the correct requirements, select a credential of a user who has the EXECUTE privilege on DBSNMP.GET_COL_DATA_SENSITIVE_TYPES. Then click OK.
8. On the Create Data Redaction Policy page, enter the following information:
 - **Schema:** Enter (or search for) the name of the schema that contains the data you want to redact.
 - **Table/View:** Enter (or search for) the table or field that contains the column you want to redact.
 - **Policy Name:** Enter a for the policy, such as emp_wages_po1.
 - **Default Expression:** Enter the default expression. The default setting is 1=1, which means that the policy always will be enforced. If you are not familiar with the components of a policy expression, then click the pencil icon beside the **Policy Expression** field to use Policy Expression Builder. Select **Policy is in effect when**, select the required conditions, then click **Add**. Click **Edit** if you want to edit the policy expression manually. After building the required policy expression, click **OK**. The Policy Expression Builder appears as follows:



9. In the Object Columns section, click **Add** to add a table or view column to the redaction policy.

A dialog box similar to the following appears:

The redaction policy is applied only on the table or view columns that are added to it.

10. From the **Column** menu, select the table or view column to which you want to apply the redaction policy.

To the right of the **Column** menu is an icon that you can click to view the contents of the selected column.

If the column contains sensitive data and has been mapped to a sensitive column type, then from the **Sensitive Column Type** menu, select the sensitive column type that it has been mapped to. If the search pattern in the **Sensitive Column Type** menu matches, then the sensitive column type is selected by default. For example, for a column listing credit card numbers, if there is a match, then the menu will list **Undefined** and **CREDIT_CARD_TYPE**. If there is no sensitive column type created, then the default **Sensitive Column Type** menu listing is only **Undefined**.

11. From the **Redaction Format** menu, select the redaction format that you want to use.

The drop-down list is populated with the Oracle Database-supplied redaction formats, as well as the custom redaction formats that you have created and saved.

If you do not want to use a pre-defined redaction format (that is, an Oracle-Database supplied redaction format, or a custom redaction format that you have created), and instead want to specify the redaction details while creating the redaction policy, select **CUSTOM** for **Redaction Format**.

The Add dialog box adjusts to accommodate the type of redaction format and function that you select. For example, if you select the **CUSTOM** redaction format and the **REGEX** redaction function, then the Function Attributes region appears in the dialog box.

12. From the **Redaction Function** menu, select the function that you want to use to redact the column data.

Select **FULL** if you want to redact the entire column data, **PARTIAL** if you want to redact only a part of the column data, **REGEX** if you want to redact the column data based on a regular expression, **RANDOM** if you want to redact the column data in a random manner, using randomly generated values, or **NONE** if you only want to test the definition of the redaction policy, and not redact any column data. Note that all the redaction functions may not be applicable for a particular redaction format. The drop-down list displays only the redaction functions that are applicable for the selected redaction format.

If you selected **CUSTOM** for **Redaction Format** in the previous step, and **PARTIAL** or **REGEX** for **Redaction Function**, ensure that you specify the function attributes.

13. Click **OK**.
14. Repeat these steps starting with Step 8 for all the columns that you want to add to the redaction policy.
15. On the Create Data Redaction Policy page, click **OK** to create the data redaction policy.
When you create an Oracle Data Redaction policy, it is enabled by default.

Related Topics

- [Creating a Custom Oracle Data Redaction Format Using Enterprise Manager](#)
You can create and save custom Oracle Data Redaction formats using Enterprise Manager Cloud Control.
- [Oracle Data Redaction Features and Capabilities](#)
Oracle Data Redaction provides a variety of ways to redact different types of data.
- [Enabling or Disabling an Oracle Data Redaction Policy in Enterprise Manager](#)
An Oracle Data Redaction policy is executed at run time only if it is enabled. When you create an Oracle Data Redaction policy, it is enabled by default.

16.5.3 Editing an Oracle Data Redaction Policy Using Enterprise Manager

You can edit an Oracle Data Redaction policy using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then search for and click the name of the database target for which the Oracle Data Redaction policy that you want to edit was created.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.

- In the Policies section of the **Policies** tab, select the redaction policy that you want to edit, then click **Edit**.

Data Redaction

Oracle Data Redaction provides an easy way to quickly redact sensitive information that is displayed in applications without altering the underlying database blocks on disk or in cache. Data is redacted in real-time according to flexible multi-factor policies. Data Redaction is licensed as part of Oracle Advanced Security.

Policies Expressions Formats

▲ Search Data Redaction Policies

Schema %

Table/View %

Policy Name %

Go

Policies

View ▼ Format ▼ Create Edit View Enable Disable Delete Freeze Wrap

Schema	Table/View	Policy Name	Enabled	Redacted Columns
HR	EMPLOYEES	emp_comm_pol	✓	0

- On the Edit Data Redaction Policy page, choose to edit the policy expression, add new columns to the redaction policy, modify the redaction details of a column that is a part of the policy, or delete a column from the redaction policy.

You can do the following:

- To add a new column to the redaction policy, in the Object Columns section, click **Add**, select the table or view column that you want to add, then specify the redaction details.
 - To modify the redaction details of a column that is a part of the policy, select the column, click **Modify**, then edit the redaction details.
 - To delete a column from the redaction policy, select the column, then click **Delete**.
- On the Edit Data Redaction Policy page, after editing the required fields, click **OK** to save and enable the edited redaction policy.

16.5.4 Viewing Oracle Data Redaction Policy Details Using Enterprise Manager

You can find Oracle Data Redaction policy details such as whether the policy is enabled by using Enterprise Manager Cloud Control.

You can disable an enabled redaction policy, or enable a disabled redaction policy using Enterprise Manager Cloud Control.

- Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.

The URL is as follows:

`https://host:port/em`

- From the **Targets** menu, select **Databases**.
- Select **Search List**, then search for and click the name of the database target for which the Oracle Data Redaction policy that you want to view was created.
- On the home page of the database target, from the **Security** menu, select **Data Redaction**.

5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
6. In the Policies section of the Policies tab, do one of the following:
 - Select the name of the policy in the table.
 - Select the required redaction policy, then click **View**.
7. To exit, click **OK**.

16.5.5 Enabling or Disabling an Oracle Data Redaction Policy in Enterprise Manager

An Oracle Data Redaction policy is executed at run time only if it is enabled. When you create an Oracle Data Redaction policy, it is enabled by default.

You can disable an enabled redaction policy, or enable a disabled redaction policy using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then search for and click the name of the database target for which the Oracle Data Redaction policy that you want to enable or disable was created.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
6. In the Policies section of the Policies tab, select the redaction policy that you want to enable or disable, and then click **Enable** or **Disable**.

Policies				
View ▼ Format ▼		Create	Edit	View
		Enable	Disable	Delete
		Freeze	Wrap	
Schema	Table/View	Policy Name	Enabled	Redacted Columns
HR	EMPLOYEES	emp_comm_pol	✓	2

7. In the Confirmation dialog box, click **Yes** or **No**.

16.5.6 Deleting an Oracle Data Redaction Policy Using Enterprise Manager

You can delete an Oracle Data Redaction policy using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.

3. Select **Search List**, then search for and click the name of the database target for which the Oracle Data Redaction policy that you want to delete was created.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
6. In the Policies section of the Policies tab, select the redaction policy that you want to delete, and then click **Delete**.
7. In the Confirmation dialog box, click **Yes** or **No**.

16.6 Managing Named Data Redaction Policy Expressions Using Enterprise Manager

You can manage Oracle Data Redaction policy expressions in Enterprise Manager Cloud Control.

- [About Named Data Redaction Policy Expressions in Enterprise Manager](#)
You can create and apply named Oracle Data Redaction policy expression to multiple columns in tables and views in Oracle Enterprise Manager Cloud Control.
- [Creating a Named Data Redaction Policy Expression in Enterprise Manager](#)
You can create and apply a named Oracle Data Redaction policy expression using Enterprise Manager Cloud Control.
- [Editing a Named Data Redaction Policy Expression in Enterprise Manager](#)
You can edit a named Oracle Data Redaction policy expression using Enterprise Manager Cloud Control.
- [Viewing Named Data Redaction Policy Expressions in Enterprise Manager](#)
You can view named Oracle Data Redaction policy expressions using Enterprise Manager Cloud Control.
- [Deleting a Named Data Redaction Policy Expression in Enterprise Manager](#)
You can delete named Oracle Data Redaction policy expressions using Enterprise Manager Cloud Control.

16.6.1 About Named Data Redaction Policy Expressions in Enterprise Manager

You can create and apply named Oracle Data Redaction policy expression to multiple columns in tables and views in Oracle Enterprise Manager Cloud Control.

When you modify the policy expression, the change is reflected in all redacted columns in the database instance that use the policy expression. Cloud Control enables you to create, edit, view, apply to columns, and delete policy expressions. Before you can create and use named Data Redaction policy expressions, ensure that the COMPATIBLE initialization parameter is set to 12.2.0.0 or later.

Related Topics

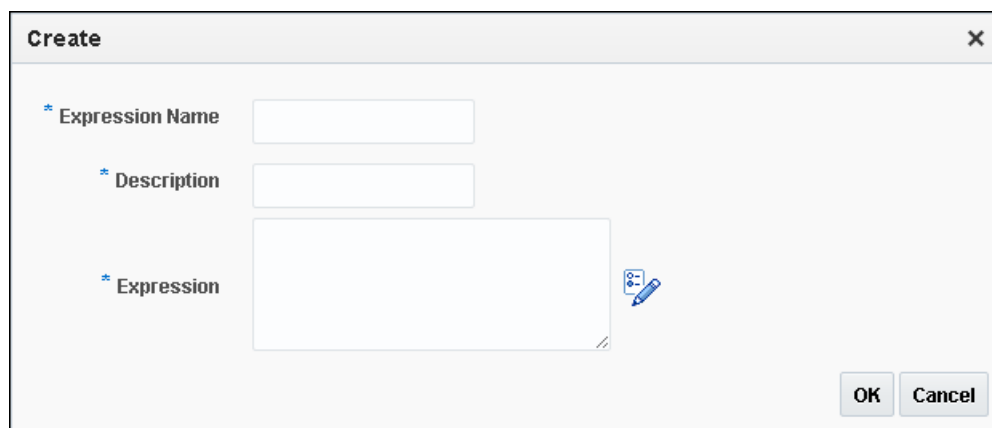
- [Creating and Managing Multiple Named Policy Expressions](#)
A named, centrally managed Oracle Data Redaction policy expression can be used in multiple redaction policies and applied to multiple tables or views.

16.6.2 Creating a Named Data Redaction Policy Expression in Enterprise Manager

You can create and apply a named Oracle Data Redaction policy expression using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user **SYSTEM** or **SYSMAN**.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target for which you want to create an Oracle Data Redaction policy.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the **EXECUTE** privilege on the **DBMS_REDACT** PL/SQL package.
6. On the Oracle Data Redaction page, select the **Expressions** tab.
7. Click **Create**.

The Create dialog box appears.



8. In the Create dialog box, enter the following information:
 - **Expression Name:** Enter a name for the policy expression. Existing policy expressions are listed on the Data Redaction page.
 - **Description:** Enter a brief description of the policy.
 - **Expression:** Enter the expression. For more complex expressions, such as applying or exempting the policy from specific users, click the Policy Expression Builder icon at the right of the **Expression** field. Click **OK** in the Policy Expression Builder to create the expression.
9. Click **OK** in the Create dialog box.

After you create the policy expression, it is listed in the Data Redaction page and ready to be associated with a Data Redaction policy.

10. In the Data Redaction page, select the **Policies** tab.
11. Under Policies, select the row for the policy that redacts the column to which you want to apply the policy expression, and then click **Edit**.
12. Under Object Columns, select the column that you want and then click the **Modify** button.
13. In the Modify dialog box, select the expression from the **Expression Name** list.
14. Click **OK**, and then click **OK** again in the Edit Data Redaction Policy dialog box.

16.6.3 Editing a Named Data Redaction Policy Expression in Enterprise Manager

You can edit a named Oracle Data Redaction policy expression using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target for which you want to create an Oracle Data Redaction policy.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
6. On the Oracle Data Redaction page, select the **Expressions** tab.
7. Select the policy expression that you want to edit and then click **Edit**.

Data Redaction

Oracle Data Redaction provides an easy way to quickly redact sensitive information that is displayed in applications without altering the underlying database blocks on disk or in cache. Data is redacted in real-time according to flexible multi-factor policies. Data Redaction is licensed as part of Oracle Advanced Security.

Policies **Expressions** Formats

View Format Create Create Like Edit View Delete Refresh

Expression Name	Description	Expression
HR_REDACT_POL	Exempts specific users from polic...	SYS_CONTEXT('USERENV', 'SESSION_USE...
LOGGED_IN_SCOTT_USER	Logged in as scott user	SYS_CONTEXT('USERENV', 'SESSION_USE...

Rows Selected 1

8. In the Edit dialog box, modify the **Description** and **Expression** fields as necessary. For more complex expressions, click the Policy Expression Builder icon, and then click **OK** after you have recreated the expression.
9. Click **OK** in the Edit dialog box.

16.6.4 Viewing Named Data Redaction Policy Expressions in Enterprise Manager

You can view named Oracle Data Redaction policy expressions using Enterprise Manager Cloud Control.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target for which you want to create an Oracle Data Redaction policy.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
6. On the Oracle Data Redaction page, select the **Expressions** tab.
7. Select the policy expression that you want to view and then click **View**.
The View dialog box appears, showing the definition of the policy expression.
8. Click **OK** to exit the View dialog box.

16.6.5 Deleting a Named Data Redaction Policy Expression in Enterprise Manager

You can delete named Oracle Data Redaction policy expressions using Enterprise Manager Cloud Control.

The deletion process first dissociates the policy expression from all columns to which it is applied.

1. Log into Oracle Enterprise Manager Cloud Control as either user SYSTEM or SYSMAN.
The URL is as follows:
`https://host:port/em`
2. From the **Targets** menu, select **Databases**.
3. Select **Search List**, then click the name of a database target for which you want to create an Oracle Data Redaction policy.
4. On the home page of the database target, from the **Security** menu, select **Data Redaction**.
5. Log in to the database, if you are prompted to do so.
Ensure that you log in to the database as a user that has the EXECUTE privilege on the DBMS_REDACT PL/SQL package.
6. On the Oracle Data Redaction page, select the **Expressions** tab.

7. Select the policy expression that you want to delete, and then click **Delete**.
The Delete Expressions confirmation dialog box appears.
8. Click **OK**.

Using Oracle Data Redaction with Oracle Database Features

Oracle Data Redaction can be used with other Oracle features, but some Oracle features may have restrictions with regard to Oracle Data Redaction.

- [Oracle Data Redaction General Usage Guidelines](#)
It is important to understand usage guidelines for using Oracle Data Redaction.
- [Oracle Data Redaction and DML and DDL Operations](#)
Oracle Data Redaction affects DML and DDL operations, especially for users who issue ad-hoc SQL against tables with redacted columns.
- [Oracle Data Redaction and Nested Functions, Inline Views, and the WHERE Clause](#)
You can use Oracle Data Redaction with nested functions, inline views, and the WHERE clause in SELECT statements.
- [Oracle Data Redaction and Queries on Columns Protected by Data Redaction Policies](#)
Queries that include the DISTINCT or ORDER BY clause on columns that are protected by Oracle Data Redaction policies may return 0 rows.
- [Oracle Data Redaction and Database Links](#)
Do not create Oracle Data Redaction policies on database views that reference database links.
- [Oracle Data Redaction and Aggregate Functions](#)
Aggregate functions can affect performance overhead on Oracle Data Redaction policies.
- [Oracle Data Redaction and Object Types](#)
You can use object types to model real-world entities such as customer accounts.
- [Oracle Data Redaction and XML Generation](#)
You cannot use XML generation functions on columns that have Oracle Data Redaction policies defined on them.
- [Oracle Data Redaction and Editions](#)
You cannot redact editioned views.
- [Oracle Data Redaction and Oracle Data Warehouse Query Rewrite Operation](#)
Oracle Data Warehouse request queries are subject to any Oracle Data Redaction policies that are present against the relations that are specified in the query.
- [Oracle Data Redaction in a Multitenant Environment](#)
In a multitenant environment, Oracle Data Redaction policies apply only to the objects within the current pluggable database (PDB).
- [Oracle Data Redaction and Oracle Virtual Private Database](#)
Oracle Data Redaction does not affect Oracle Virtual Private Database policies because the VPD inline view, which contains the VPD predicate, acts on actual values.
- [Oracle Data Redaction and Oracle Database Real Application Security](#)
Oracle Data Redaction differs from Oracle Database Real Application Security because of how security is implemented for applications.
- [Oracle Data Redaction and Oracle Database Vault](#)
You can use Oracle Data Redaction in an Oracle Database Vault environment.

- [Oracle Data Redaction and Oracle Data Pump](#)
Oracle Data Pump export operations can affect objects that have Oracle Data Redaction policies.
- [Oracle Data Redaction and Data Masking and Subsetting Pack](#)
Oracle Enterprise Manager Data Masking and Subsetting Pack can be used to create a development or test copy of a production database.
- [Oracle Data Redaction and JSON](#)
JavaScript Object Notation (JSON) can be used to create is json constraints on table columns.

17.1 Oracle Data Redaction General Usage Guidelines

It is important to understand usage guidelines for using Oracle Data Redaction.

- Do not include any redacted columns in a SQL expression that is used in a GROUP BY clause in a SQL statement. Oracle does not support this, and raises an ORA-00979: not a GROUP BY expression error. This happens because internally the expression in the SELECT list must be modified by Data Redaction, but this causes it to no longer be found when it comes time to process the GROUP BY clause (which is currently not updated by Data Redaction) leading to this unintended error message.
- Do not include any redacted columns in a SQL expression that is used in both the DISTINCT clause and ORDER BY clause in a SQL statement. Oracle does not support this, and raises an error: ORA-01791: not a SELECTed expression. This happens because internally the expression in the SELECT list must be modified by Data Redaction, but this causes it to no longer be found when it comes time to process the GROUP BY clause, leading to this unintended error message.
- An ORA-28094: SQL construct not supported by data redaction error is raised if a query involves a set operation - UNION, INTERSECT or MINUS of redacted columns and each branch of the set operation does not have the same redaction policy. This error is not raised if the user who is executing a query has the EXEMPT REDACTION POLICY system privilege. This error is only raised for users who are not exempt from the redaction policy.

To avoid the ORA-28094 error, ensure that the query has the following properties:

- When a column in the UNION has a redaction policy, the corresponding column in each branch of the UNION must use a redaction policy with the same values for all of its properties:
 - * Function type
 - * Function parameters or REGEXP parameters
 - * Policy expression
 - * Enable flagIt can be a different redaction policy, but all these properties must be the same.
- In an inline view, a UNION cannot have a subquery.
- In a CREATE VIEW definition or an inline view, there cannot be any SQL expressions (such as CONCAT, SUM, TRIM, MIN, and MAX) that involve a redacted column.
- An ORA-28093: operation on column %s is not supported by data redaction error is raised if a user issues a query that involves XML syntax and if the user does not have the EXEMPT REDACTION POLICY system privilege. The ORA-28093 is raised only for users who are not exempt from the redaction policy, and they cannot use XML syntax that involves redacted columns, whereas, user who are exempt from the redaction policy are allowed to.

- Oracle does not support the following:
 - Adding a Data Redaction policy on a virtual column
 - Adding a Data Redaction policy on the base column of a virtual column
 - Adding a virtual column whose base column has a Data Redaction policy defined on it
 - Creating a functional index on a column with Data Redaction policy

17.2 Oracle Data Redaction and DML and DDL Operations

Oracle Data Redaction affects DML and DDL operations, especially for users who issue ad-hoc SQL against tables with redacted columns.

Note the following:

- Oracle Data Redaction treats the `RETURNING INTO` clause of a DML statement as a query, even though the result is not displayed. The result that is sent to the buffer is what would have been displayed had the `RETURNING INTO` clause been run as an ordinary SQL query, rather than as part of a DML statement. If your application performs further processing on the buffer that contains the `RETURNING INTO` value, then consider changing the application because it may not expect to find a redacted value in the buffer.
- If a redacted column appears as the source in a DML or DDL operation, then Oracle Data Redaction considers this as an attempt to circumvent the policy and prevents it with an `ORA-28081: Insufficient privileges - the command references a redacted object` error unless you have the `EXEMPT REDACTION POLICY` system privilege. Internally, Oracle Data Pump issues these kinds of operations, so you may also need to grant the `EXEMPT REDACTION POLICY` system privilege to a user if they need to perform schema-level exports of tables that have redacted columns.
- Internally, Oracle Data Mining issues DML and DDL operations, so you may need to grant the `EXEMPT REDACTION POLICY` system privilege to a user if the user must create data mining models on tables that have redacted columns.

17.3 Oracle Data Redaction and Nested Functions, Inline Views, and the WHERE Clause

You can use Oracle Data Redaction with nested functions, inline views, and the `WHERE` clause in `SELECT` statements.

Oracle Data Redaction policies work as follows:

- **Nested functions are redacted innermost.** For example, in `SELECT SUM(AVG(TO_NUMBER((X))) FROM HR.EMPLOYEES WHERE ...`, the `TO_NUMBER` function is redacted first, followed by `AVG`, and then last the `SUM` function.
- **Inline views are redacted outermost.** For example, in `SELECT XYZ ... AS SELECT A... AS SELECT B... AS SELECT C...`, `SELECT XYZ` is redacted first, followed by `AS SELECT A`, then `AS SELECT B`, and so on. `AS SELECT C` is redacted last.
- **The `WHERE` clause is never redacted.** Data Redaction redacts only data in the column `SELECT` list.

17.4 Oracle Data Redaction and Queries on Columns Protected by Data Redaction Policies

Queries that include the `DISTINCT` or `ORDER BY` clause on columns that are protected by Oracle Data Redaction policies may return 0 rows.

This happens because redaction preserves the semantics of the query. For example, a query such as `SELECT * table_name WHERE sensitive_column LIKE 'value';` would likely result in 0 rows returned, because the redacted value would not match the `'value'` value entered in the `WHERE` clause. To work around this issue, rewrite the query to include an inline view so that the semantic layer can find the column. For example, instead of the following query:

```
SELECT DISTINCT sensitive_column
FROM table_name
ORDER BY sensitive_column;
```

Write the query as follows:

```
SELECT sensitive_column FROM
(SELECT DISTINCT sensitive_column
FROM table_name
ORDER BY sensitive_column);
```

17.5 Oracle Data Redaction and Database Links

Do not create Oracle Data Redaction policies on database views that reference database links.

You can find information about existing database links by querying the `DBA_DB_LINKS` data dictionary view.

① See Also

Oracle Database Administrator's Guide for detailed information about database links

17.6 Oracle Data Redaction and Aggregate Functions

Aggregate functions can affect performance overhead on Oracle Data Redaction policies.

Because Oracle Data Redaction dynamically modifies the value of each row in a column, certain SQL queries that use aggregate functions cannot take full advantage of database optimizations that presume the row values to be static.

In the case of SQL queries that call aggregate functions, it may be possible to notice performance overhead due to redaction.

17.7 Oracle Data Redaction and Object Types

You can use object types to model real-world entities such as customer accounts.

An object type is a user-defined type. You cannot redact object types. This is because Database Redaction cannot handle all of the possible ways that object types can be

configured, because they are user defined. You can find the type that an object uses by querying the OBJECT_NAME and OBJECT_TYPE columns of the ALL_OBJECTS data dictionary view.

17.8 Oracle Data Redaction and XML Generation

You cannot use XML generation functions on columns that have Oracle Data Redaction policies defined on them.

Oracle XML DB Developer's Guide describes the kinds of SQL functions to which this restriction applies. This restriction applies irrespective of whether the Oracle Data Redaction policy has been enabled or disabled, or is active for the querying user.

17.9 Oracle Data Redaction and Editions

You cannot redact editioned views.

In addition to not being able to redact editioned views, you cannot use a redacted column in the definition of any editioned view. You can find information about editions by querying the DBA_EDITIONS data dictionary view.

17.10 Oracle Data Redaction and Oracle Data Warehouse Query Rewrite Operation

Oracle Data Warehouse request queries are subject to any Oracle Data Redaction policies that are present against the relations that are specified in the query.

Query rewrite may rewrite the query to use a materialized view instead of accessing the detail relations, but only if it can guarantee to deliver exactly the same results as if the rewrite had not occurred. Specifically, the query rewrite must retain and respect any Oracle Data Redaction policies against the relations that are specified in the request query. However, any Oracle Data Redaction policies against the materialized view itself do not have any effect when the materialized view is accessed during query rewrite. This is because the data is already protected by the Oracle Data Redaction policies against the relations in the request query.

17.11 Oracle Data Redaction in a Multitenant Environment

In a multitenant environment, Oracle Data Redaction policies apply only to the objects within the current pluggable database (PDB).

You cannot create a Data Redaction policy for a multitenant container database (CDB). This is because the objects for which you create Data Redaction policies typically reside in a PDB. If you have the SYSDBA privilege, then you can list all the PDBs in a CDB by running the SHOW PDBS command.

As with the CDB root, you cannot create Data Redaction policies in an application root.

17.12 Oracle Data Redaction and Oracle Virtual Private Database

Oracle Data Redaction does not affect Oracle Virtual Private Database policies because the VPD inline view, which contains the VPD predicate, acts on actual values.

Oracle Data Redaction differs from Oracle Virtual Private Database in the following ways:

- Oracle Data Redaction provides more redacting features than Oracle Virtual Private Database, which only supports NULL redacting. Many applications cannot use NULL redacting, so Data Redaction is a good solution for these applications.
- Oracle Virtual Private Database policies can be static, dynamic, and context sensitive, whereas Data Redaction policies only allow static and context-sensitive policy expressions.
- Data Redaction permits only one policy to be defined on a table or view, whereas you can define multiple Virtual Private Database policies on an object.
- Data Redaction is when application users try to access an object that is protected by a Data Redaction policy using a synonym, but (unlike Oracle Virtual Private Database) Data Redaction does not support the creation of policies directly on the synonyms themselves.

17.13 Oracle Data Redaction and Oracle Database Real Application Security

Oracle Data Redaction differs from Oracle Database Real Application Security because of how security is implemented for applications.

Oracle Data Redaction differs from Oracle Database Real Application Security in that Real Application Security provides a comprehensive authorization framework for application security.

Column security within Real Application Security is based on application privileges that are defined by applications using the Real Application Security framework.

See Also

Oracle Database Real Application Security Administrator's and Developer's Guide for information about how you can protect table columns with custom application privileges

17.14 Oracle Data Redaction and Oracle Database Vault

You can use Oracle Data Redaction in an Oracle Database Vault environment.

For example, if there is an Oracle Database Vault realm around an object, a user who does not belong to the authorized list of realm owners or participants cannot see the object data, regardless of whether the user was granted the `EXEMPT REDACTION POLICY` privilege. If the user attempts a DML or DDL statement on the data, error messages result.

17.15 Oracle Data Redaction and Oracle Data Pump

Oracle Data Pump export operations can affect objects that have Oracle Data Redaction policies.

- [Oracle Data Pump Security Model for Oracle Data Redaction](#)
The `DATAPUMP_EXP_FULL_DATABASE` role includes the powerful `EXEMPT REDACTION POLICY` system privilege.

- [Export of Objects That Have Oracle Data Redaction Policies Defined](#)
You can export objects that have already had Oracle Data Redaction policies defined on them.
- [Export of Data Using the EXPDP Utility access_method Parameter](#)
Oracle Data Pump can export data from a schema that contains an object that has a Data Redaction policy.
- [Import of Data into Objects Protected by Oracle Data Redaction](#)
During import operations into Oracle Data Redaction-protected objects, be aware of issues that may occur.

17.15.1 Oracle Data Pump Security Model for Oracle Data Redaction

The DATAPUMP_EXP_FULL_DATABASE role includes the powerful EXEMPT REDACTION POLICY system privilege.

Remember that by default the DBA role is granted the DATAPUMP_EXP_FULL_DATABASE role as well as DATAPUMP_IMP_FULL_DATABASE.

This enables users who were granted these roles to be exempt from Data Redaction policies. This means that, when you export objects with Data Redaction policies defined on them, the [actual data](#) in the protected tables is copied to the Data Pump target system without being redacted. Users with these roles, including users who were granted the DBA role, are able to see the actual data in the target system.

However, by default, all of the Data Redaction policies associated with any tables and views in the Data Pump source system are also included in the export and import operation (along with the objects themselves) and applied to the objects in the target system, so the data is still redacted when users query the objects in the target system.

Related Topics

- [Exemption of Users from Oracle Data Redaction Policies](#)
You can exempt users from having Oracle Data Redaction policies applied to the data they access.

17.15.2 Export of Objects That Have Oracle Data Redaction Policies Defined

You can export objects that have already had Oracle Data Redaction policies defined on them.

- [Finding Type Names Used by Oracle Data Pump](#)
You must find the type names Oracle Data Pump uses before exporting objects that have Oracle Data Redaction policies defined on these objects.
- [Exporting Only the Data Dictionary Metadata Related to Data Redaction Policies](#)
You can export only the data dictionary metadata that is related to data redaction policies and full redaction settings.
- [Importing Objects Using the INCLUDE Parameter in IMPDP](#)
You can import objects using Oracle Database Pump.

17.15.2.1 Finding Type Names Used by Oracle Data Pump

You must find the type names Oracle Data Pump uses before exporting objects that have Oracle Data Redaction policies defined on these objects.

After you find these types, you should use these types as parameters for the INCLUDE directive to the IMPDP utility, to selectively export only metadata of these specific types to the dump file.

- To find type names, query the DATABASE_EXPORT_OBJECTS view.

For example:

```
SELECT OBJECT_PATH
FROM DATABASE_EXPORT_OBJECTS
WHERE OBJECT_PATH LIKE 'RADM_%';
```

Output similar to the following appears:

```
OBJECT_PATH
-----
RADM_FPTM
RADM_POLICY
```

17.15.2.2 Exporting Only the Data Dictionary Metadata Related to Data Redaction Policies

You can export only the data dictionary metadata that is related to data redaction policies and full redaction settings.

This kind of Data Pump export could, for example, be used if you must use the same set of Data Redaction policies and settings across development, test, and production databases. Because the flag `content=metadata_only` is specified, the dump file does not contain any actual data.

- To export only the data dictionary metadata related to data redaction policies and full redaction settings, enter an EXPDP utility command similar to the following:

```
expdp system/password \
full=y \
COMPRESSION=NONE \
content=metadata_only \
INCLUDE=RADM_FPTM,RADM_POLICY\
directory=my_directory \
job_name=my_job_name \
dumpfile=my_data_redaction_policy_metadata.dmp
```

See Also

- *Oracle Database Utilities* for detailed information about the INCLUDE parameter of the EXPDP utility
- *Oracle Database Utilities* for detailed information about metadata filters

17.15.2.3 Importing Objects Using the INCLUDE Parameter in IMPDP

You can import objects using Oracle Database Pump.

- To import the objects, include these names in the INCLUDE parameter in the IMPDP utility command, based on the output from querying the OBJECT_PATH column in the DATABASE_EXPORT_OBJECTS view.

17.15.3 Export of Data Using the EXPDP Utility access_method Parameter

Oracle Data Pump can export data from a schema that contains an object that has a Data Redaction policy.

If you are using Oracle Data Pump to perform full database export operations using the Data Pump default settings (direct_path), and if you receive error messages that you do not understand, then use this section to repeat the operation in such a way as to better understand the error.

If you try to use the Oracle Data Pump Export (EXPDP) utility with the access_method parameter set to direct_path to export data from a schema that contains an object that has a Data Redaction policy defined on it, then the following error message may appear and the export operation fails:

```
ORA-31696: unable to export/import TABLE_DATA:"schema.table" using client specified  
DIRECT_PATH method
```

This problem only occurs when you perform a schema-level export as a user who was not granted the EXP_FULL_DATABASE role. It does not occur during a full database export, which requires the EXP_FULL_DATABASE role. The EXP_FULL_DATABASE role includes the EXEMPT REDACTION POLICY system privilege, which bypasses Data Redaction policies.

To find the underlying problem, try the EXPDP invocation again, but do not set the access_method parameter to direct_path. Instead, use either automatic or external_table. The underlying problem could be a permissions problem, for example:

```
ORA-28081: Insufficient privileges - the command references a redacted object.
```

① See Also

Oracle Database Utilities for more information about using Data Pump Export.

17.15.4 Import of Data into Objects Protected by Oracle Data Redaction

During import operations into Oracle Data Redaction-protected objects, be aware of issues that may occur.

Inadvertent Drop of Oracle Data Redaction Policies

During an import operation, be careful that you do not inadvertently drop data redaction policies that protect imported data.

Consider a scenario in which the source tables that were exported using the Oracle Data Pump Export (EXPDP) utility do not have Oracle Data Redaction policies. However, the

destination tables to which the data is to be imported by using Oracle Data Pump Import (IMPDP) have Oracle Data Redaction policies.

During the Data Pump import operation, the status of the Data Redaction policies on the objects being imported depends on the CONTENT option of IMPDP command.

- If you use the CONTENT=ALL or CONTENT=METADATA_ONLY option in the IMPDP command, then the Data Redaction policies on the destination tables are dropped. You must recreate the Data Redaction policies.
- If you use CONTENT=DATA_ONLY in the IMPDP command, then the Data Redaction policies on the destination tables are not dropped.

Data Redaction Policy Not Being Imported

During an import operation, you could get an ORA-28069: A Data Redaction Policy Already Exists On This Object error.

Consider a scenario in which the source tables that were exported using the Oracle Data Pump Export (EXPDP) utility have Oracle Data Redaction policies. The destination tables to which the data is being imported by using Oracle Data Pump (IMPDP) also have Oracle Data Redaction policies.

During the Data Pump import operation, you could get the ORA-28069 error. To avoid this error, you must import only the data using the CONTENT=DATA_ONLY parameter. Alternatively, you can drop the redaction policies on the target objects and then import all the data.

Related Topics

- *Oracle Database Utilities*

17.16 Oracle Data Redaction and Data Masking and Subsetting Pack

Oracle Enterprise Manager Data Masking and Subsetting Pack can be used to create a development or test copy of a production database.

To accomplish this, you can mask this data in bulk, and then put the resulting masked data in the development or test copy.

You can still apply Data Redaction policies to the non-production database, in order to redact columns that contain data that was already masked by Oracle Enterprise Manager Data Masking and Subsetting Pack.

Remember that Oracle Enterprise Manager Data Masking and Subsetting Pack is used to mask data sets when you want to move the data to development and test environments. Data Redaction is mainly designed for redacting at runtime for production applications in a consistent fashion across multiple applications, without having to make application code changes.

17.17 Oracle Data Redaction and JSON

JavaScript Object Notation (JSON) can be used to create is json constraints on table columns.

However, you cannot create an Oracle Data Redaction policy on a table column that has the is json constraint. If you attempt to do so, an ORA-28073 - The column column_name has an unsupported datatype error is raised. As a workaround solution, Oracle recommends that you

create a relational view that uses the JSON_TABLE row source operator on top of the JSON object, and then apply the Data Redaction policy to this view.

See *Oracle Database SQL Language Reference* for more information about JSON_TABLE.

Security Considerations for Oracle Data Redaction

Oracle provides guidelines for using Oracle Data Redaction.

- [Oracle Data Redaction General Security Guidelines](#)
It is important to understand general security guidelines for using Oracle Data Redaction.
- [Restriction of Administrative Access to Oracle Data Redaction Policies](#)
You can restrict the list of users who can create, view and edit Data Redaction policies.
- [How Oracle Data Redaction Affects the SYS, SYSTEM, and Default Schemas](#)
Both users SYS and SYSTEM automatically have the EXEMPT REDACTION POLICY system privilege.
- [Policy Expressions That Use SYS_CONTEXT Attributes](#)
Be careful when writing a policy expression that depends on a SYS_CONTEXT attribute that is populated by an application.
- [Oracle Data Redaction Policies on Materialized Views](#)
You can create Oracle Data Redaction policies on materialized views and on their base tables.
- [REDACTION_COLUMNS Data Dictionary View Behavior When a View Is Invalid](#)
When an Oracle Data Redaction policy exists on a column of a view, and the view becomes invalid, the Data Redaction policy remains visible in the REDACTION_COLUMNS data dictionary view.
- [Dropped Oracle Data Redaction Policies When the Recycle Bin Is Enabled](#)
You should check if the recycle bin is enabled before you drop Oracle Data Redaction policies.

18.1 Oracle Data Redaction General Security Guidelines

It is important to understand general security guidelines for using Oracle Data Redaction.

- Oracle Data Redaction is not intended to protect against attacks by regular and privileged database users who run ad hoc queries directly against the database. If the user can issue arbitrary SQL or PL/SQL statements, then they will be able to access the actual value.
- Oracle Data Redaction is not intended to protect against users who run ad hoc SQL queries that attempt to determine the actual values by inference. Inference refers to query that is designed to find data by repeatedly trying queries. For example, to find the users who earn the highest salaries, an intruder could use the following query:

```
SELECT FIRST_NAME, LAST_NAME, SALARY
FROM HR.EMPLOYEES
WHERE SALARY > 16000
ORDER BY SALARY DESC;
```

FIRST_NAME	LAST_NAME	SALARY
-----	-----	-----
Steven	King	24000

Neena	Kochhar	17000
Lex	De Haan	17000

- Oracle Data Redaction relies on the database and application context values. For applications, it is the responsibility of the application to properly initialize the context value.
- Oracle Data Redaction is not enforced for users who are logged in using the SYSDBA administrative privilege.
- Certain DDL statements that attempt to copy the actual data out from under the control of a data redaction policy (that is, CREATE TABLE AS SELECT, INSERT AS SELECT) are blocked by default, but you can disable this behavior by granting the user the EXEMPT REDACTION POLICY system privilege.
- Oracle Data Redaction does not affect day-to-day database operations, such as backup and recovery, Oracle Data Pump exports and imports, Oracle Data Guard operations, and replication.

18.2 Restriction of Administrative Access to Oracle Data Redaction Policies

You can restrict the list of users who can create, view and edit Data Redaction policies.

To accomplish this, you can limit who has the EXECUTE privilege on the DBMS_REDACT package and by limiting who has the SELECT privilege on the REDACTION_POLICIES and REDACTION_COLUMNS views.

You also can restrict who is exempted from redaction by limiting the EXEMPT REDACTION POLICY privilege. If you use Oracle Database Vault to restrict privileged user access, then you can use realms to restrict granting of EXEMPT REDACTION POLICY.

Related Topics

- [Oracle Data Redaction and Oracle Database Vault](#)
You can use Oracle Data Redaction in an Oracle Database Vault environment.
- [Exemption of Users from Oracle Data Redaction Policies](#)
You can exempt users from having Oracle Data Redaction policies applied to the data they access.
- Introduction to Oracle Database Vault

18.3 How Oracle Data Redaction Affects the SYS, SYSTEM, and Default Schemas

Both users SYS and SYSTEM automatically have the EXEMPT REDACTION POLICY system privilege.

SYSTEM has the EXP_FULL_DATABASE role, which includes the EXEMPT REDACTION POLICY system privilege.

This means that the SYS and SYSTEM users can always bypass any existing Oracle Data Redaction policies, and will always be able to view data from tables (or views) that have Data Redaction policies defined on them.

Follow these guidelines:

- Do not create Data Redaction policies on the default Oracle Database schemas, including the SYS and SYSTEM schemas.
- Be aware that granting the EXEMPT REDACTION POLICY system privilege to additional roles may enable users to bypass Oracle Data Redaction, because the grantee role may have been granted to additional roles.
- Do not revoke the EXEMPT REDACTION POLICY system privilege from the roles that it was granted to by default.

18.4 Policy Expressions That Use SYS_CONTEXT Attributes

Be careful when writing a policy expression that depends on a SYS_CONTEXT attribute that is populated by an application.

The application might not always populate that attribute.

If the user somehow connects directly (rather than through the application), then the SYS_CONTEXT attribute would not have been populated. If you do not handle this NULL scenario in your policy expression, you could unintentionally reveal [actual data](#) to the querying user.

For example, suppose you wanted to create a policy expression that intends to redact the query results for everyone except users who have the client identifier value of SUPERVISOR. The following expression unintentionally enables querying users who have NULL as the value for their CLIENT_IDENTIFIER to see the real data:

```
SYS_CONTEXT('USERENV', 'CLIENT_IDENTIFIER') IS NOT 'SUPERVISOR'
```

A more rigorous policy expression redacts the result of the query if the client identifier is not set, that is, it has a NULL value.

```
SYS_CONTEXT('USERENV', 'CLIENT_IDENTIFIER') IS NOT 'SUPERVISOR' OR IS NULL
```

Remember that in SQL, comparisons with NULL are undefined, and are thus FALSE, but redaction only takes place when the policy expression evaluates to TRUE.

18.5 Oracle Data Redaction Policies on Materialized Views

You can create Oracle Data Redaction policies on materialized views and on their base tables.

However, ensure that the creator of the materialized view, or the user who performs the refresh of the materialized view, is not blocked by any Data Redaction policies. In other words, the user performing the materialized view creation or refresh operations should be exempt from the Data Redaction policy. As a best practice, when you create a new materialized view, treat it as a copy of the actual table, and then create a separate Data Redaction policy to protect it.

18.6 REDACTION_COLUMNS Data Dictionary View Behavior When a View Is Invalid

When an Oracle Data Redaction policy exists on a column of a view, and the view becomes invalid, the Data Redaction policy remains visible in the REDACTION_COLUMNS data dictionary view.

For example, a view can become invalid if one of its columns refers to a column that was dropped from a table upon which the view depends.

The column continues to be visible in the REDACTION_COLUMNS data dictionary view because the Data Redaction policy is not automatically dropped when the view becomes invalid.

Instead, the decision on whether to drop the Data Redaction policy is taken when the view is subsequently altered.

This approach was chosen in preference to automatically dropping the Data Redaction policy when the view becomes invalid because it is less error-prone and presents less risk of accidentally displaying actual data from the underlying table.

By deferring the decision to when the view is being altered, it allows the view to be recompiled after the column is restored to the table. After the column is restored to the table and the view is recompiled, then the view becomes valid and still has its Data Redaction policy in place.

On the other hand, if the invalid view definition was subsequently replaced with a valid view definition which no longer contains the column that the Data Redaction policy was previously defined on, it is at that point that the Data Redaction policy is automatically dropped. The REDACTION_COLUMNS data dictionary view is then updated to no longer show the column (since it is no longer part of the new view's definition).

18.7 Dropped Oracle Data Redaction Policies When the Recycle Bin Is Enabled

You should check if the recycle bin is enabled before you drop Oracle Data Redaction policies.

If you drop a table or view that has an Oracle Data Redaction policy defined on it when the recycle bin feature is enabled, and if you query the REDACTION_COLUMNS or REDACTION_POLICIES data dictionary views before you purge the recycle bin, then you will see object names such as BIN\$. . . (for example, BIN\$1Xu5PSW5VaPgQxGS5AoAEA==\$0).

This is normal behavior. These policies are removed when you purge the recycle bin.

To find if the recycle bin is enabled, you can run the SHOW PARAMETER RECYCLEBIN command in SQL*Plus.

① See Also

Oracle Database Administrator's Guide for information about purging objects from the recycle bin

Glossary

actual data

In Oracle Data Redaction, the data in a protected table or view. An example of actual data could be the number 123456789, and the [redacted data](#) version of this number could be 999996789.

auto-login TDE wallet

A [software keystore](#) that is protected by a system-generated password and does not need to be explicitly opened by a security administrator. Auto-login TDE wallets are automatically opened when accessed and can be used on any computer that runs an Oracle database. For example, consider an Oracle RAC environment that has four nodes, and each node is on a different computer. This environment uses an auto-login keystore. When a REKEY operation is performed on node 1, the auto-login and password-based keystores must be copied to the computers that host nodes 2, 3, and 4. In this configuration, the auto-login keystores will be opened on all four nodes when required.

cipher suite

A set of authentication, encryption, and data integrity algorithms used to exchange messages between network nodes using Transport Layer Security (TLS). During a TLS handshake, for example, the two nodes negotiate to see which cipher suite they will use when transmitting messages back and forth.

ciphertext

Message text that has been encrypted.

See also [encrypted text](#).

data redaction

The ability to mask data with different values in real time, that is, at the moment a user tries to access the data. You can mask all of the data, a partial subset of the data, or display random values in place of the data. It does not change the actual data in the database.

decryption

The process of converting an encrypted message (the [ciphertext](#)), back to its original message ([plaintext](#)).

encrypted text

Text that has been encrypted, using an encryption algorithm and an encryption key; the output stream of an encryption process. The text is not readable or decipherable, without decrypting it first. Also called [ciphertext](#).

encryption

The process of converting an original message ([plaintext](#)) to an encrypted message ([ciphertext](#)).

external keystore

A container that stores a Transparent Data Encryption key for a hardware security module. (The previous term for external keystore was hardware keystore.) An external keystore can be Oracle Key Vault or Oracle Cloud Infrastructure (OCI) Key Management Service (KMS). External keystores provide centralized key management for TDE-enabled databases.

hardware security module

A physical device that provides secure storage for encryption keys. Oracle does not support third-party hardware security modules to provide key management for Transparent Data Encryption-enabled databases. (See My Oracle Support note [2310066.1](#) for more information.) Oracle Key Vault can provide third-party hardware security modules to provide root-of-trust to Oracle Key Vault. See *Oracle Key Vault Root of Trust HSM Configuration Guide*.

inference

A query that is designed to find data by repeatedly trying queries. For example, to find the users who earn the highest salaries, an intruder could use the following query:

```
SELECT FIRST_NAME, LAST_NAME, SALARY FROM HR.EMPLOYEES WHERE SALARY > 16000 ORDER BY SALARY DESC;
```

FIRST_NAME	LAST_NAME	SALARY
-----	-----	-----
Steven	King	24000
Neena	Kochhar	17000
Lex	De Haan	17000

key pair

A [public key](#) and its associated [private key](#). See [public and private key pair](#).

keystore

A general term for any container that stores encryption keys, such as Transparent Data Encryption keys and other encrypted data. It can refer to a [TDE wallet](#), which is specific to

Oracle Database, or to an [external keystore](#), which is specific to Oracle Key Vault or Oracle Cloud Interface (OCI) Key Management Service (KMS).

local auto-login TDE wallet

A [TDE wallet](#) that is local and restricted to the computer on which it was created.

See also [auto-login TDE wallet](#).

mask

The ability to redact data from a user or an application.

Oracle-managed tablespace

An Oracle-supplied tablespace that contains information necessary for the correct functioning (confidentiality, integrity, and availability) of the database system. This information includes the system's data dictionary, the system's temporary sort area, the system's undo segment, and the system's auxiliary data. This information is only expected to be updated internally by the Oracle database server itself, and does not normally be updated directly by users.

password-based TDE wallet

A [TDE wallet](#) that must be opened with a password before it can be accessed.

plaintext

Message text that has not been encrypted.

private key

In public-key cryptography, this key is the private key that is known only to its owner. It is primarily used for encrypting message digests used with digital signatures.

See [public and private key pair](#).

public key

One of two keys that are used in public key cryptography, the other key being the [private key](#). In typical public key cryptography usage, the public key is used to encrypt data or verify digital signatures. The the private key is used to decrypt data or generate digital signatures. The public key, unlike the private key, can be made available to anyone whereas the private key must remain secret.

See [public and private key pair](#).

public key encryption

The process where the sender of a message encrypts the encryption key of the recipient. Upon delivery, the message is decrypted by the recipient using its private key.

public and private key pair

A set of two related numbers used for [encryption](#) and [decryption](#), where one is called the [private key](#) and the other is called the [public key](#). Public keys are typically made widely available, while private keys are held by their respective owners. Data encrypted with either a public key or a private key from a [key pair](#) can be decrypted with its associated key from the key pair.

public key infrastructure (PKI)

Information security technology utilizing the principles of public key cryptography. Public key cryptography involves encrypting and decrypting information using a shared public and private key pair. Provides for secure, private communications within a public network.

redacted data

Masked data that is displayed to the querying user. For example, if the [actual data](#) is 3714-4963-5398-4321, then the redacted data could appear, depending on the Data Redaction policy, as XXXX-XXXX-XXXX-4321.

salt

In cryptography, a way to strengthen the security of encrypted data. Salt is a random string that is added to the data before it is encrypted, making it more difficult for attackers to steal the data by matching patterns of ciphertext to known ciphertext samples. Salt is often also added to passwords, before the passwords are hashed, to avoid dictionary attacks, a method that attackers use to determine sensitive passwords. The addition of salt to a password before hashing makes it more difficult for intruders to match the hash values (sometimes called verifiers) with their dictionary list of common password hash values, because they do not know the salt beforehand.

software keystore

See [TDE wallet](#),

tablespace encryption key

An encryption key for the encryption of a tablespace. The TDE tablespace encryption key encrypts the tablespace encryption key, which in turn encrypts and decrypts data in the tablespace.

TDE master encryption key

A key that is stored within a [TDE wallet](#) or a [external keystore](#). For table encryption, this key encrypts the TDE table key, and for tablespace encryption, it encrypts the tablespace encryption key.

See also [keystore](#).

TDE table key

An encryption key that is associated with a table whose columns are marked for encryption. The TDE master encryption key encrypts this table encryption key.

TDE wallet

A container that stores a Transparent Data Encryption a TDE master encryption key for use as an [auto-login TDE wallet](#), a [local auto-login TDE wallet](#), or a [password-based TDE wallet](#) to store and manage security credentials for an individual entity. TDE wallets are specific to Oracle Database only. A [Wallet Resource Locator \(WRL\)](#) provides all of the necessary information to locate the TDE wallet. TDE wallets were previously referred to as [software keystores](#).

wallet obfuscation

The ability to store and access an Oracle [TDE wallet](#) without querying the user for a password before access (supports single sign-on (SSO)).

Wallet Resource Locator (WRL)

A tool that provides all of the necessary information to locate a [TDE wallet](#). It is a path to an operating system directory that contains a TDE wallet.

Index

Symbols

, when to use, [5](#)

A

ad hoc tools

Oracle Data Redaction, [4](#)

ADMINISTER KEY MANAGEMENT

isolated mode operations, [2](#)

isolated mode operations not allowed, [6](#)

united mode operations allowed in, [2](#)

united mode operations not allowed, [6](#)

administrative access to policies, restricting, [2](#)

aggregate functions

affect on Data Redaction policy optimization,
[4](#)

ALTER SYSTEM statement

how compares with ADMINISTER KEY
MANAGEMENT statement, [7](#), [38](#)

APEX_UTIL.GET_NUMERIC_SESSION_STATE
function

Oracle Data Redaction policies (NV public
function), [13](#)

APEX_UTIL.GET_SESSION_STATE function

Oracle Data Redaction policies (V public
function), [13](#)

applications

database applications and Oracle Data
Redaction, [3](#)

modifying to use Transparent Data
Encryption, [7](#)

auto login keystores

and Transparent Data Encryption (TDE), [40](#)

Automatic Storage Management (ASM)

moving TDE wallets from, [13](#)

non-OMF-compliant system pointing to ASM
location, [22](#)

standalone database pointing to ASM
location, [20](#)

TDE wallet location configuration, [20](#)

TDE wallet pointing to ASM location, [21](#)

C

CDBs, [1](#)

cloning PDBs with encrypted data, [39](#)

cloning PDBs with encrypted data in isolated
mode, [44](#)

cloning PDBs with encrypted data, about, [39](#)

Data Redaction masking policies, [5](#)

moving PDB from one CDB to another in
united mode, [35](#)

moving PDB from one PDB to another, [41](#)

PDBs with encrypted data, [35](#)

preserving keystore passwords in PDB move
operations, [41](#)

preserving keystore passwords in PDB move
operations in united mode, [35](#)

remotely cloning PDBs with encrypted data in
isolated mode, [45](#), [46](#)

remotely cloning PDBs with encrypted data in
united mode, [40](#), [42](#)

change data capture, synchronous, [18](#)

cloning and upgrading encrypted PDBs

TDE Academy videos on, [42](#)

closing external keystores, [23](#)

closing TDE wallets, [23](#)

column encryption

about, [5](#)

changing algorithm, [26](#)

changing encryption key, [26](#)

creating encrypted table column with default
algorithm, [19](#)

creating encrypted table column with non-
default algorithm, [20](#)

creating index on encrypted column, [25](#)

data loads from external file, [11](#)

data types to encrypt, [17](#)

existing tables

about, [24](#)

adding encrypted column to, [24](#)

disabling encryption, [24](#)

encrypting unencrypted column, [24](#)

external tables, [22](#)

incompatibilities, [1](#)

limitations, [1](#)

performance, optimum, [4](#)

salt, [25](#)

- column encryption (*continued*)
 - security considerations, [4](#)
 - skipping integrity check, [21](#)
- column sensitive type discovery
 - enabling when creating a Data Redaction policy, [11](#)
- compliance
 - Transparent Data Encryption, [2](#)
- compression of Transparent Data Encryption data, [3](#)
- configuring software keystores
 - creating local auto-login keystore, [6](#)

D

- data at rest, [1](#)
- data deduplication of Transparent Data Encryption data, [3](#)
- data redaction
 - See Oracle Data Redaction
- Data Redaction supported functions, [7](#)
- data storage
 - Transparent Data Encryption, [6](#)
- database close operations
 - keystores, [11](#)
- database links
 - with Oracle Data Redaction policies, [4](#)
- database roles
 - Data Redaction policies, [12](#)
- databases
 - about encrypting, [27](#)
 - encrypting existing, [47](#)
 - encrypting offline, [48](#)
 - encrypting online, [49](#)
- DDL statements
 - Oracle Data Redaction policies, [3](#)
- decryption
 - tablespaces, offline, [37](#), [40](#)
 - tablespaces, online, [41](#)
- DISTINCT clause, Data Redaction policies, [2](#)
- DML statements
 - Oracle Data Redaction policies, [3](#)

E

- editing custom formats, [8](#)
- editing policies, [14](#)
- Editions
 - Transparent Data Encryption, [37](#)
- encrypted columns
 - data loads from external files, [11](#)
- encrypting data
 - in isolated mode, [14](#), [17](#)
 - in united mode, [20](#)
- encryption, [5](#)
 - algorithm, setting default, [35](#)

- encryption (*continued*)
 - cloning PDBs with encrypted data, [39](#)
 - cloning PDBs with encrypted data in isolated mode, [44](#)
 - databases offline, [48](#)
 - databases online, [49](#)
 - encrypting future tablespaces, [36](#)
 - about, [35](#)
 - existing databases, [47](#)
 - procedure, [36](#)
 - remotely cloning PDBs with encrypted data in isolated mode, [45](#), [46](#)
 - remotely cloning PDBs with encrypted data in united mode, [40](#), [42](#)
 - supported encryption algorithms, [41](#)
 - tablespaces, offline, [37](#)
 - tablespaces, online, [41](#)
 - See also Transparent Data Encryption (TDE)
- encryption algorithms, supported, [41](#)
- encryption keys
 - setting in isolated mode, [13](#)
 - setting in united mode, [12](#)
- ENCRYPTION_WALLET_LOCATION parameter
 - convert to WALLET_ROOT and TDE_CONFIGURATION, [49](#)
- Errors:
 - ORA-46694, [37](#)
- EXEMPT REDACTION POLICY privilege
 - using with Database Vault, [2](#)
- expressions, [7](#)
 - LENGTH functions, character string, [10](#)
 - namespace functions, [8](#)
 - Oracle Application Express, [10](#)
 - Oracle Label Security functions, [10](#)
 - SUBSTR function, [9](#)
- external credential store, external keystores, [5](#)
- external credential store, external keystores, sqlnet.ora, [5](#)
- external credential store, password-based TDE wallets, [5](#)
- external credential store, password-based TDE wallets, sqlnet.ora, [5](#)
- external files
 - loading data to tables with encrypted columns, [11](#)
- external keystores, [11](#)
 - about, [7](#)
 - backing up, [8](#)
 - changing password in isolated mode, [20](#)
 - changing password in united mode, [22](#)
 - closing, [23](#)
 - closing in isolated mode, [24](#)
 - closing in united mode, [25](#)
 - heartbeat batch size, [16](#)
 - opening in isolated mode, [15](#)
 - plugging PDBs, [38](#), [43](#)

external keystores (*continued*)
 unplugging PDBs, [38](#), [43](#)
 using external keystore, [5](#)
 using external keystore, sqlnet.ora, [5](#)
 external store for passwords
 open and close operations in CDB, [43](#), [47](#)
 external tables, encrypting columns in
 ORACLE_DATPUMP, [11](#)
 ORACLE_LOADER, [11](#)
 EXTERNAL_STORE clause, [5](#)

G

GROUP BY clause, Data Redaction policies, [2](#)
 guidelines
 materialized views and Data Redaction, [3](#)
 recycle bin and Data Redaction, [4](#)
 SYS_CONTEXT values and Data Redaction, [3](#)
 guidelines, general usage
 redacted columns and DISTINCT clause, [2](#)
 redacted columns and GROUP BY clause, [2](#)
 redacted columns and ORDER BY clause, [2](#)
 guidelines, security
 ad hoc query attacks and Data Redaction, [1](#)
 application context value handling by Data Redaction policies, [1](#)
 day-to-day operations and Data Redaction, [1](#)
 DDL statements and Data Redaction policies, [1](#)
 exhaustive SQL queries and inference and Data Redaction, [1](#)

H

HEARTBEAT_BATCH_SIZE initialization
 parameter, [16](#)

I

import/export utilities, original, [18](#)
 index range scans, [3](#)
 indexes
 creating on encrypted column, [25](#)
 inference, used to find data by repeatedly using a query, [1](#)
 inline views
 Data Redaction policies order of redaction, [3](#)
 Data Redaction redaction, [3](#)
 intruders
 ad hoc query attacks, [1](#)
 isolated mode, [2](#), [6](#)
 about, [1](#)
 ADMINISTER KEY MANAGEMENT
 operations allowed in, [2](#)

isolated mode (*continued*)
 ADMINISTER KEY MANAGEMENT
 operations not allowed in, [6](#)
 backing up TDE wallets, [21](#)
 changing PDB keystore from CDB root, [8](#)
 configuring external keystores, about, [14](#)
 configuring for Oracle Key Vault, [15](#)
 configuring keystore location and keystore type, [7](#)
 configuring TDE wallets, about, [11](#)
 creating TDE master encryption key for later use, [26](#)
 creating TDE wallet, [11](#)
 encrypting data, [14](#), [17](#)
 encryption key, setting, [13](#)
 exporting or importing master encryption keys, [49](#)
 exporting, importing TDE master encryption keys, [48](#)
 external keystores, closing, [24](#)
 external keystores, opening, [15](#)
 lost control file, [9](#)
 master encryption keys
 moving key from PDB to CDB root, [37](#)
 master encryption keys, migrating, [17](#)
 migrating from external keystore to password TDE wallet, [36](#)
 migrating from password TDE wallet to external keystore, [35](#)
 moving encryption key into new TDE wallet, [29](#)
 moving PDB from one CDB to another, [41](#)
 Oracle RAC, [10](#)
 password change for external keystores, [20](#)
 password change for TDE wallets, [19](#)
 plugging PDB with master encryption keys stored in external keystore, [43](#)
 plugging PDBs with encrypted data into CDB, [42](#)
 secrets stored in external keystores, [34](#)
 secrets stored in TDE wallets, [33](#)
 setting new encryption key, [16](#)
 TDE wallets, closing, [24](#)
 TDE wallets, opening, [12](#)
 uniting PDB keystore, [37](#)
 unplugging PDBs, [43](#)

J

JSON
 Oracle Data Redaction, [10](#)

K

keystore location
 setting for isolated mode, [7](#)

keystore location (*continued*)
 setting for united mode, [7](#)

keystore type
 setting for isolated mode, [7](#)
 setting for united mode, [7](#)

keystores
 about, [6](#)
 architecture, [5](#)
 auto login, [40](#)
 auto-login, open and close operations in
 CDBs, [43, 47](#)
 backing up isolated mode password-protected
 TDE wallets
 procedure, [21](#)
 backing up password-protected TDE wallets
 backup identifier rules, [6](#)
 procedure, [7](#)
 backing up united mode password-protected
 TDE wallets
 procedure, [23](#)
 changing Oracle Key Vault password, [4](#)
 changing passwords for protected-protected
 software keystores, [3](#)
 closing external keystores, [23](#)
 closing in CDBs, [43, 47](#)
 closing TDE wallets, [23](#)
 creating when PDB is closed, [38](#)
 database close operations, [11](#)
 deleting, [25](#)
 deleting unused, [32](#)
 deleting unused, about, [47](#)
 deleting unused, procedure, [48](#)
 external, [11](#)
 external keystore
 configuration process, [11](#)
 external, changing password in isolated
 mode, [20](#)
 external, changing password in united mode,
 [22](#)
 hardware, opening in isolated mode, [15](#)
 merging
 one into another existing keystore in
 isolated mode, [22](#)
 one into another existing TDE wallet, [10](#)
 migrating
 creating master encryption key for
 external keystore-based
 encryption, [15](#)
 external keystore to TDE wallet, [16](#)
 keystore order after migration, [18](#)
 migration using Oracle Key Vault, [19](#)
 moving out of ASM, [13](#)
 multiple databases sharing same host, setting
 for, [4](#)
 non-OMF-compliant system pointing to ASM
 location, [22](#)

keystores (*continued*)
 opening in CDBs, [43, 47](#)
 Oracle Database secrets
 about, [51](#)
 storing in external keystore, [54](#)
 storing in software keystore, [51](#)
 password access, [2](#)
 password preservation in PDB move
 operations, [41](#)
 password preservation in PDB move
 operations in united mode, [35](#)
 reverting keystore creation operation, [39](#)
 search order for, [2](#)
 software, changing password in isolated
 mode, [19](#)
 software, changing password in united mode,
 [21](#)
 standalone database pointing to ASM
 location, [20](#)
 TDE master encryption key merge differing
 from import or export, [46](#)
 TDE wallets, creating in united mode, [10](#)
 using auto-login external keystore, [56](#)

L

LENGTH functions, character string
 expressions, [10](#)

M

masking
 See Oracle Data Redaction

materialized views
 Data Redaction guideline, [3](#)
 Transparent Data Encryption tablespace
 encryption, [6](#)

migration
 migrating from external keystore to password
 TDE wallet, [36](#)
 migrating from password TDE wallet to
 external keystore, [35](#)

moving encryption key into new keystore
 about, [47](#)
 procedure, [48](#)

multitenant container databases
 See CDBs

N

namespace functions
 expressions, [8](#)

nested functions
 Data Redaction policies order of redaction, [3](#)

NV public function
 (APEX_UTIL.GET_NUMERIC_SESSION

_STATE function), Data Redaction policies, [13](#)

O

OLS_LABEL_DOMINATES public function

Data Redaction policies, [12](#)

opening connection to Oracle Key Vault, [12](#), [14](#)

opening TDE wallets, [8](#)

operations allowed in, [2](#), [2](#)

operations not allowed in, [6](#), [6](#)

ORA-00979 error

not a GROUP BY expression error, [2](#)

ORA-28081

Insufficient privileges - the command references a redacted object error, [3](#)

ORA-28365 error

wallet is not open, [29](#)

ORA-46680 error, [36](#)

ORA-46694 error, [37](#)

ORA-65040 error, [38](#)

Oracle Application Express

filtering using by session state in Data Redaction policies, [13](#)

Oracle Application Expression

expressions, [10](#)

Oracle Call Interface

Transparent Data Encryption, [37](#)

Oracle Data Guard

isolated keystore on PDB, [28](#)

TDE and Oracle Key Vault, [10](#)

TDE master encryption keys, removing from standby database, [33](#)

Transparent Data Encryption, [6](#)

wallet-based TDE configuration, [17](#), [22](#)

Oracle Data Pump

encrypted columns, [2](#)

encrypted data, [2](#)

encrypted data with database links, [4](#)

encrypted data with dump sets, [3](#)

exported data from Data Redaction policies, [9](#)

exporting Oracle Data Redaction objects, [8](#)

imported data from Data Redaction policies, [9](#)

Oracle Data Redaction security policy, [7](#)

Oracle Data Redaction, [1](#), [4](#)

about, [1](#)

ad hoc tools, [4](#)

aggregate functions, [4](#)

benefits, [2](#)

CDBs, [5](#)

columns with XML-generated data, [5](#)

creating custom format, [5](#)

database applications, [3](#)

DBMS_REDACT.ALTER_POLICY procedure

about, [49](#)

example, [51](#)

Oracle Data Redaction (*continued*)

DBMS_REDACT.ALTER_POLICY procedure (*continued*)

parameters required for various actions, [51](#)

syntax, [50](#)

DBMS_REDACT.DISABLE_POLICY

about, [55](#)

example, [55](#)

syntax, [55](#)

DBMS_REDACT.DROP_POLICY

about, [56](#)

examples, [56](#)

syntax, [56](#)

DBMS_REDACT.ENABLE_POLICY

about, [56](#)

example, [56](#)

syntax, [56](#)

DBMS_REDACT.UPDATE_FULL_REDACTION_VALUES

procedure

about, [26](#)

syntax, [26](#)

using, [27](#)

deleting policies, [16](#)

editing custom format, [8](#)

editions, [5](#)

Enterprise Manager Cloud Control, [5](#), [8](#), [11](#)

Enterprise Manager Cloud Control workflow, [2](#)

Enterprise Manager Cloud Control, about, [1](#)

exporting data using Data Pump Export, [9](#)

exporting objects using Data Pump, [8](#)

full data redaction

about, [1](#)

creating policy for, [24](#)

examples, [24](#)

modifying default value, [26](#)

syntax, [24](#)

functions used in expressions, [8](#)

how differs from Oracle Database Real Application

Security masking, [6](#)

how differs from Oracle Virtual Private Database

masking, [5](#)

importing data using Data Pump Import, [9](#)

inline views order of redaction, [3](#)

JSON, [10](#)

managing policies, [11](#)

named policy expressions

about, [9](#)

nested functions order of redaction, [3](#)

no data redaction

about, [9](#), [47](#)

creating policies for, [47](#)

example, [48](#)

syntax, [47](#)

Oracle Data Pump security policy, [7](#)

Oracle Data Warehouse query rewrites, [5](#)

Oracle Enterprise Manager Data Masking and Subsetting Pack, [10](#)

Oracle Data Redaction (*continued*)

partial data redaction

- about, [2](#)
- character types, policies for, [34](#)
- data-time data types, [37](#)
- example using character data type, [35](#)
- example using data-time data type, [37](#)
- example using fixed character format, [33](#)
- example using number data type, [36](#)
- formats, fixed character, [31](#)
- number data types, [35](#)
- syntax, [30](#)

policy expressions

- about, [14](#)
- creating, [15](#)
- dropping, [17](#)
- tutorial, [17](#)
- updating, [16](#)

prevention of data exposure by management tools, [3](#)queries on columns protected by Data Redaction policies, [4](#)

random data redaction

- about, [46](#)
- creating policies for, [46](#)
- example, [46](#)

randomized data redaction

- about, [4](#)

redaction of data in read-only static pages, [3](#)redaction of data used in offline analytics, [3](#)

regular expression data redaction

- creating policies for, [38](#)
- custom, creating policies for, [44](#)
- example, [44](#)
- example of custom, [45](#)
- formats, [41](#)
- formats, creating policies for, [40](#)
- settings for, [44](#)
- syntax, [39](#)

regular expression redaction

- about, [3](#)

returning null values

- about, [28](#)
- example, [28](#)
- syntax, [28](#)

SYS schema objects, [2](#)SYSTEM schema objects, [2](#)use cases, [2](#)when to use, [2](#)WHERE clause redaction, [3](#)

Oracle Data Redaction formats

- creating in Cloud Control, [5](#)
- deleting in Cloud Control, [10](#)
- editing in Cloud Control, [8](#)
- Enterprise Management Cloud Control, managing in, [5](#)

Oracle Data Redaction formats (*continued*)Enterprise Manager Cloud Control, sensitive column types, [2](#)Enterprise Manager Cloud Control, viewing in, [9](#)

Oracle Data Redaction import operations

inadvertently dropping policies, [9](#)ORA-28069, [9](#)

Oracle Data Redaction partial redaction

creating policies for, [29](#)Oracle Data Redaction policies, [11](#)about, [2](#)altering, [49](#)building reports, [57](#)

creating

examples, [25](#)general syntax, [4](#)creating in Cloud Control, [11](#)deleting in Cloud Control, [16](#)disabling, [55](#)disabling in Cloud Control, [16](#)dropping, [56](#)editing in Cloud Control, [14](#)enabling, [56](#)Enterprise Manager Cloud Control, viewing in, [15](#)exempting users from, [48](#)

expressions

by Application Express session state, [13](#)by database role, [12](#)by OLS label dominance, [12](#)by user environment, [11](#)

filtering users

about, [7](#)no filtering, [13](#)finding information about, [59](#)Oracle Enterprise Manager Cloud Control, [16](#)redacting multiple columns in one policy, [54](#)trace files, [58](#)

Oracle Data Redaction policy expressions

Cloud Control, about, [17](#)creating in Cloud Control, [18](#)deleting in Cloud Control, [20](#)editing in Cloud Control, [19](#)viewing in Cloud Control, [20](#)Oracle Data Redaction, database links, [4](#)

Oracle Data RedactionEnterprise Manager Cloud Control

deleting custom format, [10](#)

Oracle Database Real Application Security

Data Redaction, [6](#)

Oracle Database Vault

using with Data Redaction, [2](#)Oracle Enterprise Manager Cloud Control, [14](#)creating custom formats, [5](#)creating policy expressions, [18](#)

Oracle Enterprise Manager Cloud Control (*continued*)

- deleting policy expressions, [20](#)
- disabling policies, [16](#)
- editing policy expressions, [19](#)
- Oracle Data Redaction, [5](#), [8](#), [16–20](#)
- Oracle Data Redaction formats, viewing in, [9](#)
- Oracle Data Redaction, creating policies, [11](#)
- Oracle Data Redaction, viewing details of a policy, [15](#)
- policy expressions, about, [17](#)
- viewing policy expressions, [20](#)

Oracle Enterprise Manager Data Masking and Subsetting Pack

- Oracle Data Redaction impact, [10](#)

Oracle GoldenGate

- storing secrets in Oracle keystores, [58](#)

Oracle Key Vault

- migration of keystores, [19](#)

Oracle Key Vault connection

- opening in united mode, [15](#)

Oracle Label Security

- functions using Data Redaction expressions, [10](#)

Oracle Real Application Clusters

- Oracle Key Vault and TDE in multitenant configuration, [30](#)
- Transparent Data Encryption, [29](#)

Oracle Recovery Manager

- Transparent Data Encryption, [24](#)

Oracle Securefiles

- Transparent Data Encryption, [36](#)

Oracle Virtual Private Database (VPD)

- Data Redaction, [5](#)

Oracle-managed tablespaces, [27](#)

orapki utility

- how compares with ADMINISTER KEY MANAGEMENT statement, [7](#), [38](#)

ORDER BY clause, Data Redaction policies, [2](#)

original import/export utilities, [18](#)

P

passwords

- access to for ADMINISTER KEY MANAGEMENT operations, [2](#)
- preserving in PDB move operations, [41](#)
- preserving in PDB move operations in united mode, [35](#)

PDBs, [1](#)

- Data Redaction policies, [5](#)
- finding TDE keystore status for all PDBs, [44](#)
- master encryption keys
 - exporting, [48](#)
 - importing, [48](#)
- unplugging with encrypted data, [36](#)

performance

- Transparent Data Encryption, [5](#)

pluggable databases

- See PDBs

policy expressions, Oracle Data Redaction, [14](#)

R

recycle bin

- Data Redaction policies and, [4](#)

REDACTION_COLUMNS data dictionary view

- invalid views, [3](#)

rekeying

- master encryption key, [40](#)
- TDE master encryption key in isolated mode, [28](#)
- TDE master encryption key in united mode, [30](#)

reports

- based Data Redaction policies, [57](#)

returning null values

- about, [4](#)

S

salt

- removing, [25](#)

salt (TDE)

- adding, [25](#)

secrets

- storing Oracle Database secrets in keystore
 - about, [51](#)
 - storing in external keystore, [54](#)
 - storing in software keystore, [51](#)

SecureFiles

- Transparent Data Encryption, [36](#)

sensitive credential data, [37](#)

software keystores

- closing in united mode, [24](#)
- creating in united mode, [10](#)

SUBSTR function

- expressions, [9](#)

synchronous change data capture, [18](#)

SYS user

- Data Redaction policies, [2](#)

SYS_CONTEXT function

- Data Redaction policies, [3](#)
- SYS_SESSION_ROLES namespace used in Data Redaction, [12](#)

SYS_SESSION_ROLES SYS_CONTEXT namespace

- Data Redaction, [12](#)

SYSTEM user

- Data Redaction policies, [2](#)

T

- tablespace encryption
 - about, [3](#)
 - architecture, [3](#)
 - creating encrypted tablespaces, [34](#)
 - examples, [34](#)
 - incompatibilities, [1](#)
 - opening keystore, [32](#)
 - performance overhead, [5](#)
 - performance, optimum, [4](#)
 - procedure, [31](#)
 - restrictions, [30](#)
 - security considerations for plaintext
 - fragments, [5](#)
 - setting tablespace key, [33](#)
 - storage overhead, [6](#)
- tablespace master encryption key
 - setting, [33](#)
- TABLESPACE_ENCRYPTION initialization
 - parameter, [8](#)
- TABLESPACE_ENCRYPTION_DEFAULT_ALGORITHM dynamic parameter, [35](#)
- tablespaces
 - about encrypting, [27](#)
 - comparison between offline and online
 - conversions, [27](#)
 - Oracle managed, closed TDE keystore impact
 - on encrypted, [29](#)
 - rekeying encryption algorithm, [41](#)
 - tablespaces
 - encrypting, [41](#)
- tablespaces, offline decryption
 - procedure, [40](#)
- tablespaces, offline encryption
 - about, [37](#)
 - procedure, [38](#)
- tablespaces, online encryption
 - about, [41](#)
 - decrypting, [45](#)
 - finishing interrupted job, [46](#)
 - procedure, [42](#)
 - rekeying, [44](#)
- TDE
 - See Transparent Data Encryption (TDE)
- TDE Academy, [42](#)
- TDE column encryption
 - restrictions, [18](#)
- TDE columns
 - migrating from release 11g, [1](#)
- TDE master encryption key, [3](#)
 - creating for later use in isolated mode, [26](#)
 - creating for later use in united mode, [28](#)
- TDE master encryption keys
 - activating
 - about, [33](#)
- TDE master encryption keys (*continued*)
 - activating (*continued*)
 - example, [34](#)
 - procedure, [33](#)
 - activating in isolated mode, [27](#)
 - activating in united mode, [29](#)
 - architecture, [5](#)
 - attributes, [35](#)
 - creating for later use
 - about, [31](#)
 - examples, [32](#)
 - procedure, [31](#)
 - custom attribute tags
 - about, [36](#)
 - creating, [37](#)
 - creating in isolated mode, [30](#)
 - creating in united mode, [31](#)
 - disabling not allowed, [38](#)
 - ENCRYPTION_WALLET_LOCATION to
 - WALLET_ROOT and
 - TDE_CONFIGURATION
 - creating, [49](#)
 - exporting, [43](#)
 - exporting in PDBs, [48](#)
 - finding currently used encryption key in united
 - mode, [31](#)
 - finding currently used TDE master encryption
 - key, [36](#)
 - importing, [45](#)
 - importing in PDBs, [48](#)
 - keystore merge differing from import or
 - export, [46](#)
 - outside the database
 - about, [28](#)
 - outside the database
 - create, [28](#)
 - creating in isolated mode, [25](#)
 - creating in united mode, [26](#)
 - rekeying, [40](#), [30](#), [28](#)
 - removing automatically from standby
 - database, [33](#)
 - resetting in keystore, [39](#)
 - setting in keystore, [38](#)
- TDE tablespaces
 - migrating from release 11g, [1](#)
- TDE wallet location
 - setting, [4](#)
- TDE wallet type
 - setting, [4](#)
- TDE wallets
 - about, [7](#)
 - ASM-based, [20](#)
 - backing up password-protected TDE wallets
 - about, [6](#)
 - changing password in isolated mode, [19](#)
 - changing password in united mode, [21](#)

TDE wallets (*continued*)

- closing in isolated mode, [24](#)
- configuration process, [2](#)
- deleting unused in isolated mode, [29](#)
- merging
 - about, [9](#)
 - auto-login into password-protected, [11](#)
 - reversing merge operation, [11](#)
 - two into a third new TDE wallet, [10](#)
 - two into a third new TDE wallet in isolated mode, [23](#)

migrating

- password key into external keystore, [15](#)
- moving TDE wallet to a new location, [12](#)
- opening in isolated mode, [12](#)
- opening in united mode, [10](#)
- opening, about, [8](#)
- password-based using external keystore, [5](#)
- password-based using external keystore, sqlnet.ora, [5](#)

- pointing to ASM location, [21](#)

- software, opening in isolated mode, [12](#)

- software, opening in united mode, [10](#)

Transparent Data Encryption (TDE), [1](#), [5](#)

- about, [1](#)

- about configuration, [1](#)

- benefits, [2](#)

column encryption

- about, [5](#), [16](#)

- adding encrypting column to existing table, [24](#)

- changing algorithm, [26](#)

- changing encryption key, [26](#)

- creating encrypted column in external table, [22](#)

- creating index on encrypted column, [25](#)

- creating tables with default encryption algorithm, [19](#)

- creating tables with non-default encryption algorithm, [20](#)

- data types supported, [17](#)

- disabling encryption in existing column, [24](#)

- encrypting columns in existing tables, [24](#)

- encrypting existing column, [24](#)

- encryption and integrity algorithms, [9](#)
- restrictions, [18](#)

- salt in encrypted columns, [25](#)

- columns with identity columns, [18](#)

- compatibility with application software, [1](#)

- compatibility with Oracle Database tools, [1](#)

- compression of encrypted data, [3](#)

configuring external keystores

- about, [11](#)

- reconfiguring TDE wallet, [15](#), [19](#)

- setting master encryption key, [14](#), [16](#)

Transparent Data Encryption (TDE) (*continued*)

- configuring external keystores (*continued*)

- sqlnet.ora configuration, [11](#)

- configuring external keystores in isolated mode

- reconfiguring TDE wallet, [17](#)

- configuring Oracle Key Vault for united mode, [14](#)

- configuring software keystores

- creating auto-login keystore, [6](#)

- creating password-protected keystore, [6](#)

- configuring TDe wallets

- about, [3](#)

- data deduplication of encrypted data, [3](#)

- editions, [37](#)

- encryption and integrity algorithms, [9](#)

- finding information about, [50](#)

- frequently asked questions, [1](#)

- incompatibilities, [1](#)

keystore management

- changing Oracle Key Vault keystore password, [4](#)

- changing protected-protected software keystore password, [3](#)

- closing external keystores, [23](#)

- closing TDE wallet, [23](#)

- merging keystores, one into an existing in isolated mode, [22](#)

- merging TDE wallets, about, [9](#)

- merging TDE wallets, one into an existing, [10](#)

- migrating password key and external keystore, reverse migration, [16](#)

- TDE master encryption key attributes, [35](#)

- keystore search order, [2](#)

keystores

- about, [6](#)

- benefits, [6](#)

- types, [7](#)

master encryption key

- rekeying, [40](#)

- rekeying in united mode, [30](#)

master encryption key attributes

- creating custom tags, [37](#)

master encryption keys

- setting in keystore procedure, [38](#)

- setting in keystore, about, [38](#)

- modifying applications for use with, [7](#)

- multidatabase environments, [42](#)

- multitenant environment, [11](#)

- non-OMF-compliant system pointing to ASM location, [22](#)

- opening the connection to Oracle Key Vault, [12](#), [14](#)

- Oracle Call Interface, [37](#)

Transparent Data Encryption (TDE) (*continued*)

- Oracle Data Guard, isolated keystore on PDB, [28](#)
- Oracle Data Pump
 - export and import operations on dump sets, [3](#)
 - export and import operations on encrypted columns, [2](#)
 - export operations on database links, [4](#)
- Oracle Data Pump export and import operations
 - about, [2](#)
- Oracle Real Application Clusters
 - about, [29](#)
 - Oracle Key Vault in multitenant configuration, [30](#)
- Oracle Recovery Manager, [24](#)
 - TDE wallets, [24](#)
- PDBs
 - finding keystore status for all PDBs, [44](#)
- performance
 - database workloads, [4](#)
 - decrypting entire data set, [4](#)
 - optimum, [4](#)
 - worst case scenario, [4](#)
- performance overheads
 - about, [5](#)
 - typical, [4](#)
- privileges required, [2](#)
- SecureFiles, [36](#)
- security considerations
 - column encryption, [4](#)
 - general advice, [4](#)
 - plaintext fragments, [5](#)
- standalone database pointing to ASM
 - location, [20](#)
- storage overhead, [6](#)
- storing Oracle GoldenGate secrets, [58](#)
- tablespace encryption
 - about, [3, 26](#)
 - creating, [33](#)
 - encryption and integrity algorithms, [9](#)
 - examples, [34](#)
 - opening keystore, [32](#)
 - restrictions, [30](#)
 - setting master encryption key, [33](#)
- tablespace encryption, setting with COMPATIBLE parameter, [31](#)
- TDE Academy, [42](#)
- TDE master encryption key
 - rekeying in isolated mode, [28](#)
- TDE master encryption key attributes
 - about, [36](#)
 - creating custom tags in isolated mode, [30](#)
 - creating custom tags in united mode, [31](#)

Transparent Data Encryption (TDE) (*continued*)

- TDE master encryption keys
 - exporting and importing, [42](#)
- TDE Master Encryption Keys
 - resetting in keystore, [39](#)
- TDE wallet management
 - ASM-based TDE wallet, [20](#)
 - backing up password-protected TDE wallets, [6](#)
 - merging TDE wallets, auto-login into password-protected, [11](#)
 - merging TDE wallets, reversing merge operation, [11](#)
 - merging TDE wallets, two into a third new TDE wallet, [10](#)
 - merging TDE wallets, two into a third new TDE wallet in isolated mode, [23](#)
 - migrating password key and external keystore, [15](#)
 - migrating password key and external keystore, master encryption key creation, [15](#)
- TDE wallet pointing to ASM location, [21](#)
- views, [50](#)
- wallet-based, in Oracle Data Guard, [17, 22](#)
- Transparent Data Encryption (TDE) keystores
 - deleting, [25](#)
 - features affected if deleted, [26](#)
- Transparent Data Encryption (TDE) TDE wallets
 - moving TDE wallet to a new location, [12](#)
- Transparent Data Encryption (TDE), Oracle Data Guard
 - about, [6](#)
 - about controlling tablespace encryption, [6](#)
 - configuring, [10](#)
 - controlling tablespace encryption, [8](#)
- Transparent Data Encryption (TDE) integrity
 - column encryption
 - creating tables without integrity checks (NOMAC), [21](#)
 - improving performance, [21](#)
 - NOMAC parameter (TDE), [21](#)
- transportable tablespaces, [18](#)
- tutorials
 - named Data Redaction policy expressions, [17](#)

U

- united mode, [2, 6](#)
 - about, [1](#)
 - about managing cloned PDBs with encrypted data, [39](#)
- ADMINISTER KEY MANAGEMENT
 - operations allowed in, [2](#)
- ADMINISTER KEY MANAGEMENT
 - operations not allowed in, [6](#)

united mode (*continued*)

- backing up TDE wallets, [23](#)
- cloning PDB with encrypted data, [39](#)
- configuring CDBs for PDBs for Oracle Key Vault, about, [13](#)
- configuring software keystores, about, [9](#)
- configuring, procedure, [7](#)
- creating TDE master encryption key for later use, [28](#)
- encrypting data, [20](#)
- encryption key, setting, [12](#)
- external keystores, closing, [25](#)
- finding keystore status for all PDBs, [44](#)
- heartbeat batch size for external keystores, [16](#)
- isolating PDB keystore, [33](#)
- keystore open and close operations, [43](#)
- master encryption keys
 - moving key from CDB root to PDB, [33](#)
- moving TDE master encryption key into new keystore, [32](#)
- Oracle Key Vault connection, opening, [15](#)
- password change for external keystores, [22](#)
- password change for TDE wallets, [21](#)
- remotely cloning PDB with encrypted data, [40](#), [45](#)
- remotely cloning PDBs with encrypted data, [42](#), [46](#)

united mode (*continued*)

- setting TDE master encryption key, [18](#)
- software keystores, closing, [24](#)
- TDE wallets, creating in, [10](#)
- TDE wallets, opening, [10](#)
- utilities, import/export, [18](#)

V

V public function

- (APEX_UTIL.GET_SESSION_STATE function), Data Redaction policies, [13](#)

V\$ENCRYPTION_WALLET view

- keystore order after migration, [18](#)

views

- Data Redaction, [59](#)

W

WALLET_ROOT parameter

- convert from
 - ENCRYPTION_WALLET_LOCATION, [49](#)

X

- XML generation, [5](#)